

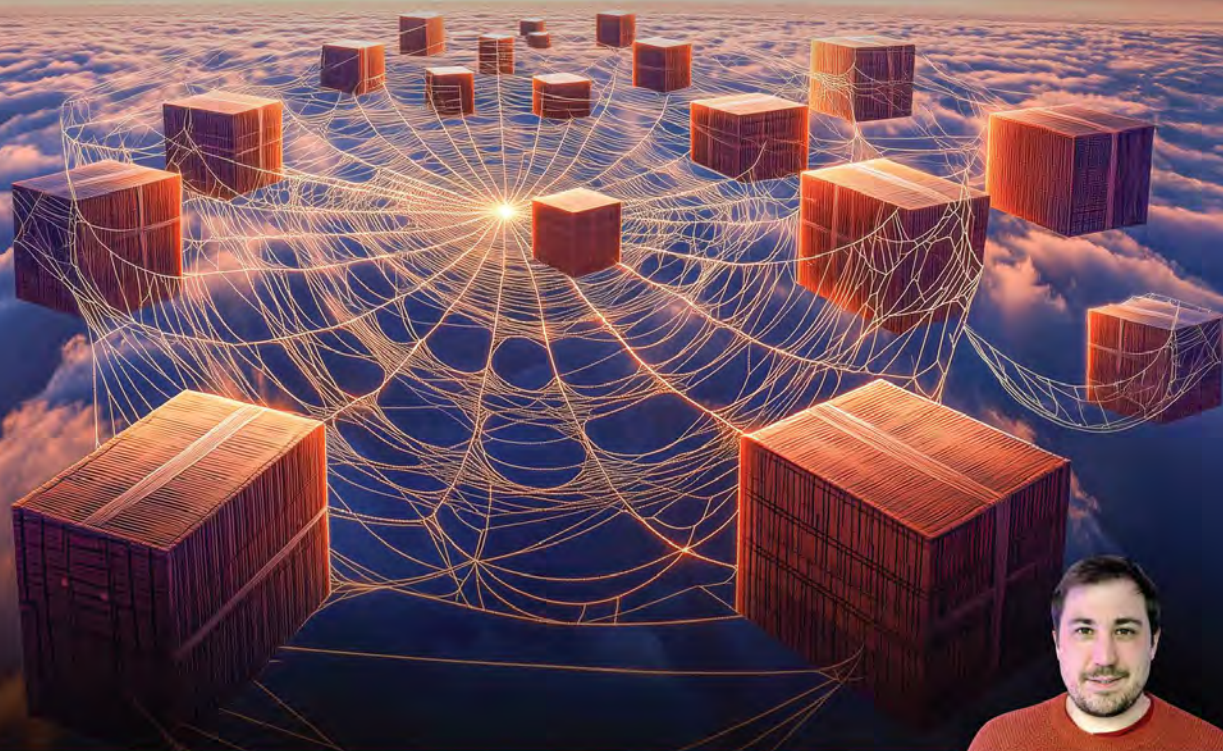
EXPERT INSIGHT

THIRD EDITION

Rust

Web Programming

A hands-on guide to Rust for modern web development,
with microservices and nanoservices



Maxwell Flitton

<packt>

Rust Web Programming

Third Edition

A hands-on guide to Rust for modern web development, with
microservices and nanoservices

Maxwell Flitton



Rust Web Programming

Third Edition

Copyright © 2026 Packt Publishing

All rights reserved. No part of this book may be reproduced, stored in a retrieval system, or transmitted in any form or by any means, without the prior written permission of the publisher, except in the case of brief quotations embedded in critical articles or reviews.

Every effort has been made in the preparation of this book to ensure the accuracy of the information presented. However, the information contained in this book is sold without warranty, either express or implied. Neither the author, nor Packt Publishing or its dealers and distributors, will be held liable for any damages caused or alleged to have been caused directly or indirectly by this book.

Packt Publishing has endeavored to provide trademark information about all of the companies and products mentioned in this book by the appropriate use of capitals. However, Packt Publishing cannot guarantee the accuracy of this information.

Portfolio Director: Ashwin Nair

Relationship Lead: Bhavya Rao

Project Manager: Ruvika Rao

Content Engineer: Runcil Rebello

Technical Editor: Rohit Singh

Copy Editor: Safis Editing

Indexer: Rekha Nair

Proofreader: Runcil Rebello

Production Designer: Ganesh Bhadwalkar

Growth Lead: Priyadarshini Sharma

First published: February 2021

Second edition: January 2023

Third edition: January 2026

Production reference: 1230126

Published by Packt Publishing Ltd.

Grosvenor House

11 St Paul's Square

Birmingham

B3 1RB, UK.

ISBN 978-1-83588-776-9

www.packtpub.com

Contributors

About the author

Maxwell Flitton is the CTO at Kioko, where he applies the engineering techniques present in this book to build production systems with 100% unit test coverage. Kioko is essentially *Cursor for CAD*, helping teams move faster from ideas to manufacturable geometry and working with Formula 1 teams on designing components for their cars.

Maxwell previously worked in open-source financial loss modeling at Oasis LMF. After that, he joined SurrealDB, where he helped build the database's machine learning engine, before moving on to Kioko. He was also an early software engineer at Monolith AI (since acquired by CoreWeave). Maxwell is an Honorary Researcher at King's College London, applying Rust to computer vision for surgical robotics.

In 2011, Maxwell earned a BSc in Nursing from the University of Lincoln (UK). While working 12-hour shifts in A&E departments, he completed a Physics degree with the Open University, followed by a Postgraduate Diploma in Physics and Engineering in Medicine at UCL (London).

About the reviewers

Joshua Mo is currently a DevRel Engineer who is working at the intersection of Rust and AI with Playgrounds Analytics Inc. He is part of the core maintainer team for Rig, an agentic AI framework written in Rust. He also regularly gives talks on Rust, as well as other subjects. His primary area of work involves WebAssembly (via wasm-bindgen) and he is currently spearheading a project to make Rig usable from JavaScript via WebAssembly. He is also a coach at Codebar, which is a charity dedicated to facilitating the growth of diverse tech communities by running free programming workshops for minority groups within tech.

Jack Walters is currently a software engineer working at Faculty in the Frontier team. Before that, he has worked at Roke and Nemisindo as a software engineer. He has completed his MSc in Computer Science from Queen Mary University of London in 2021. In his free time, Jack is a musician and has composed, produced, and arranged a jazz and neo-soul EP.

Table of Contents

Preface	xvii
Free benefits with your book	xxii
Chapter 1: A Quick Introduction to Rust	1
Technical requirements	2
What is Rust?	2
Why is Rust revolutionary? • 3	
Reviewing data types and variables in Rust	5
Using strings in Rust • 7	
Using integers and floats • 9	
8-byte integers • 10	
16-byte integers • 10	
Introducing floats • 12	
Storing data in arrays • 12	
Storing data in vectors • 14	
Mapping data with enums • 15	
Mapping data with HashMaps • 16	
Handling results and errors • 18	
Controlling variable ownership	21
Copying variables • 22	
Moving variables • 23	
Immutable borrowing of variables • 25	
Mutable borrowing of variables • 27	
Scopes • 29	
Running through lifetimes • 30	
Building Structs	33
Summary	38

Questions	39
Answers	39
Further reading	40
Chapter 2: Useful Rust Patterns for Web Programming	41
Technical requirements	42
Verifying with traits	42
Metaprogramming with macros	46
Mapping messages with macros	49
Defining our contracts and contract handler • 50	
Defining functions to handle contracts • 50	
Building a register contract macro • 51	
Using our macro in a program • 52	
Configuring our functions with traits	54
Defining the trait signature • 54	
Implementing our trait for the database handle • 55	
Calling our macro with our trait • 56	
Checking struct state with the compiler	58
Defining the state of a struct • 58	
Changing the state of a struct • 59	
Summary	60
Questions	61
Answers	61
Chapter 3: Designing Your Web Application in Rust	63
Technical requirements	64
Managing a software project with Cargo	64
Basic Rust compilation • 64	
Building with Cargo • 65	
Shipping crates with Cargo • 67	
Documenting with Cargo • 68	
Interacting with Cargo • 72	
Structuring code through nanoservices	77
Building to-do structs • 77	
Configuring our file structure • 78	
Defining the task status • 79	
Building our to-do structs • 80	

Managing structs with an API • 82	
Storing tasks with our data access layer • 86	
<i>Defining our module layout • 86</i>	
<i>Building our JSON file handle • 87</i>	
<i>Building storage API functions • 89</i>	
Creating a task using our DAL	91
Summary	94
Questions	94
Answers	95
Chapter 4: Async Rust	97
Technical requirements	98
Understanding threads and processes	98
Understanding async and await with Rust	101
Implementing our own async task queue	105
Building a task-spawning function • 106	
Building an async sleep future • 107	
Running our async sleep future • 109	
Exploring high-level concepts of tokio	110
Implementing an HTTP server in Hyper • 113	
<i>Setting up the Hyper server environment • 113</i>	
<i>Building the Hyper request handle function • 114</i>	
<i>Building the main function for running the Hyper server • 114</i>	
<i>Implementing HTTP2 for our Hyper server • 116</i>	
Summary	117
Questions	117
Answers	117
Chapter 5: Handling HTTP Requests	119
Technical requirements	119
Launching a basic web server	120
Defining our file layout • 120	
Writing our server code • 121	
Running our server • 123	
Connecting the core to the server	124
Defining our core file layout • 124	
Coding our networking layer • 126	

Refactoring our to-do items	127
Serving our to-do items	130
Defining the JSON body • 130	
Defining the get all API • 131	
Connecting get_all to our server • 132	
Handling errors in API endpoints	135
Defining our glue module • 136	
Building our custom error • 137	
Mapping our custom error • 140	
Integrating our glue module into our dal • 141	
Integrating our glue module into our core layer • 144	
Integrating our glue module into our networking layer • 146	
Summary	147
Questions	147
Answers	147
 Chapter 6: Processing HTTP Requests	 149
Technical requirements	149
Passing parameters via the URL	150
Passing data via POST body	153
Defining the core create function • 153	
Wrapping our core in the networking layer • 154	
<i>Configuring our create network route • 157</i>	
<i>Testing our endpoint with cURL • 158</i>	
Deleting resources using the DELETE method	159
Defining our delete JSON file method • 160	
Adding delete to our networking layer • 161	
Defining the delete route • 162	
Refactoring our JSON file code • 162	
Automating running our server	164
Updating resources using a PATCH method	167
Extracting data from HTTP request headers	170
Defining our header token modules • 170	
Implementing an Actix header extraction trait • 171	
Extracting headers from requests in the Actix networking layer • 173	
Summary	174
Questions	175
Answers	175

Chapter 7: Displaying Content in the Browser	177
Technical requirements	178
Building out a development setup	178
Connecting our development environment and application • 182	
Running our frontend server • 183	
Serving the frontend from Rust	186
Defining our frontend embedding dependencies • 186	
Embedding the HTML file • 188	
Embedding the React application • 189	
Serving the embedded frontend • 189	
Connecting backend API endpoints to the frontend	194
Defining our interfaces • 199	
Defining our API utils • 201	
Defining our GET API call • 203	
Integrating our get API call in our main app • 204	
Building out the rest of the API calls • 206	
Creating React components	208
Building a create item form • 209	
Injecting our form component into our app • 211	
Testing our create item form • 212	
Rendering multiple React components	213
Inserting styles with CSS	218
Serving our CSS • 218	
Defining page borders with CSS for different screen sizes • 220	
Styling React components with CSS • 222	
Summary	226
Questions	226
Answers	227
Chapter 8: Injecting Rust in the Frontend with WebAssembly	229
Technical requirements	229
What is WASM?	230
Setting up our WASM build	231
Defining our WASM module • 231	
Loading WASM in the frontend	236
Inspecting a WASM interface • 237	
Integrating WASM loading into our React component • 239	

Passing our WASM output to subcomponents • 240	
Loading WASM on the local machine	243
Building a WASM kernel • 244	
Building a WASM library • 246	
Building a WASM client • 249	
Summary	254
Questions	254
Answers	254
 Chapter 9: Data Persistence with PostgreSQL	 257
Technical requirements	257
Building our PostgreSQL database	258
Why we should use a proper database • 258	
Why use Docker? • 258	
How to use Docker • 259	
Running a database in Docker • 261	
Exploring routing and ports in Docker • 262	
Running Docker in the background with Bash scripts • 266	
Adding SQLX to the data access layer	268
Defining our database transactions	272
Connecting our transactions to the core	279
Connecting our transactions to the server	281
Creating our database migrations	285
Refactoring our frontend	287
Summary	290
Questions	290
Answers	291
Appendix	291
 Chapter 10: Managing User Sessions	 297
Technical requirements	297
Building our auth server	298
Building our data access layer • 299	
Building our auth core module • 300	
Networking layer • 301	
Defining our user data model	303
Storing passwords	305

Verifying passwords	308
Creating users	309
Defining our create user database transactions • 309	
Defining our core create API endpoint • 310	
Defining our networking create API endpoint • 311	
Refactoring our JWT	312
Restructuring our JWT for a unique ID • 313	
Creating a get key function for encoding • 315	
Creating an encode function to encode user credentials • 316	
Creating a decode function to extract user credentials • 316	
Building our login API	317
Getting users via email in the data access layer • 317	
Creating our core login API • 319	
Mounting our core login function to our server • 320	
Interacting with our auth server	323
Adding authentication to our frontend	326
Building a login API call • 326	
Adding tokens to our API calls • 328	
Building a login form component • 328	
Connecting the login form to the app • 331	
Summary	332
Questions	333
Answers	333
Appendix	333
 Chapter 11: Communicating Between Servers	 337
Technical requirements	337
Getting users from auth with the unique ID	337
Adding get by unique ID to dal • 338	
Adding get by unique ID to core • 339	
Adding get by unique ID to networking • 339	
Making auth accessible to other servers	340
Tethering users to to-do items	344
Linking our to-do items to users in the database • 345	
Adding user IDs to data access transactions • 348	
Adding user IDs to core functions • 354	
Adding user IDs to networking functions • 355	

Testing our server-to-server communication with Bash	358
Summary	361
Questions	361
Answers	361

Chapter 12: Caching Auth Sessions 363

Technical requirements	363
What is caching?	364
Setting up Redis	365
Building our Redis module	367
Defining the user session • 369	
Building the login process • 374	
Building the logout process • 375	
Building the update process • 376	
Building our Redis client	378
Building the login/logout client • 379	
Building the update client • 381	
Connecting our cache to our servers	382
Building our cache kernel • 383	
Calling the kernel from our to-do server • 386	
Calling the kernel from our auth server • 390	
Summary	394
Questions	394
Answers	395

Chapter 13: Observability Through Logging 397

Technical requirements	397
What are RESTful services?	397
Building frontend code on command • 398	
What is logging?	399
Logging via the terminal • 401	
Defining a logger • 401	
Creating a logging middleware • 403	
Integrating our logger into our servers • 406	
Logging via a database • 408	
What is an actor? • 408	
Building our logging actor • 409	

<i>Update logging functions • 412</i>	
<i>Configuring our logging database • 414</i>	
Summary	417
Questions	417
Answers	417
Chapter 14: Unit Testing	421
Technical requirements	421
The importance of unit testing	421
Writing our first unit test	424
Testing our core	426
Unit testing our networking layer	430
Defining our test module • 431	
Mocking our database handle • 432	
Mocking our user cache handle • 434	
Defining our mock service • 436	
Writing tests • 436	
Unit testing our data access layer	442
Defining a teardown SQL command • 442	
Creating a DAL unit test • 443	
Refactoring our testing script • 446	
Summary	448
Questions	449
Answers	449
Chapter 15: End-to-End Testing	451
Technical requirements	451
The importance of end-to-end testing	451
Building our to-do HTTP kernel	453
Building our create item HTTP kernel • 456	
Building our delete item HTTP kernel • 459	
Building our atomic end-to-end tests	461
Writing the end-to-end test • 463	
Running our atomic end-to-end test • 466	
Testing workflows with end-to-end tests	468
Configuring existing code to accommodate the workflow tests • 469	
Building a kernel API call for creating a user • 470	

Building a kernel API call for logging in • 472	
Creating a workflow test workspace • 474	
Creating a delete workflow test • 475	
Updating our test runner bash script • 478	
Summary	480
Questions	480
Answers	480
 Chapter 16: Deploying Our Application on AWS	 483
Technical requirements	484
Setting up our build environment	484
Setting up an AWS SSH key for an Amazon EC2 instance • 484	
Setting up our AWS account • 487	
Setting up our Terraform build • 492	
Building a server build pipeline	497
Packaging our system with Docker	500
Building the base Docker image for all servers • 500	
Defining the build config for our monolith • 505	
Defining the build config for our microservices • 508	
Distributing our system with Docker	512
Deploying our system with Docker	515
Writing a Terraform script for our server and database • 516	
Writing a Docker Compose system for our server • 518	
Writing a deployment script for our system • 520	
Summary	522
Questions	523
Answers	523
 Chapter 17: Configuring HTTPS with NGINX on AWS	 525
Technical requirements	525
What is HTTPS?	526
Implementing HTTPS locally with Docker Compose	528
Installing SSL • 531	
Attaching a URL to our deployed application on AWS	534
Attaching an Elastic IP to our server • 536	
Registering a domain name • 540	
Enforcing HTTPS on our application on AWS	545

Getting certificates for our URL • 546	
Prepping our EC2 instances • 549	
Creating a load balancer for our traffic • 550	
Creating security groups to lock down and secure traffic • 554	
Updating our deployment script • 559	
Attaching our URL to the load balancer • 560	
Summary	564
Questions	564
Answers	564
Further reading	565
 Chapter 18: Practicalities of Using Microservices and Nanoservices	 567
Technical requirements	567
When to use nanoservices, microservices, and monoliths	568
The negative effects of microservices • 568	
<i>Container orchestration • 568</i>	
<i>Flying blind • 570</i>	
<i>Network calls are expensive • 572</i>	
The positive effects of microservices • 573	
<i>Rolling out releases • 573</i>	
<i>Isolation of code • 574</i>	
<i>Flexibility to experiment • 575</i>	
Managing private repositories with dockpack	576
Integrating Rust nanoservices with other languages using C interfaces	579
Creating our to-do Rust library • 580	
Creating our C interface • 581	
Interfacing with our Rust nanoservice with Python • 584	
Summary	585
 Chapter 19: Low-Level Networking	 587
Technical requirements	587
Building a TCP server	587
Building a kernel for key-value messages • 588	
Building a TCP server • 589	
Building a TCP client • 592	
Implementing HTTP on top of TCP	595
Defining a basic server • 595	

Defining an HTTP client from scratch • 596	
Implementing HTTPS on top of TCP	598
Exploring WebSockets	601
Summary	604
Questions	604
Answers	605
Chapter 20: Distributed Computing	607
Technical requirements	607
Introduction to queuing	608
Defining the layout of the queuing system	611
Client • 611	
Compute unit and kernel • 612	
Server and worker • 612	
Other configurations • 612	
Building our kernel	614
Building our compute unit	614
Building our worker	617
Building our server	623
Building our client	625
Running our queuing system	627
Summary	630
Questions	630
Answers	630
Chapter 21: Unlock Your Exclusive Benefits	631
Other Books You May Enjoy	637
Index	641

Preface

Do you want to build modern web applications that are fast, safe, and scalable but without sacrificing developer productivity? Rust is no longer limited to systems programming. It has rapidly emerged as a powerful and reliable choice for web development, combining memory safety, performance, and concurrency in a single language. This book shows you why Rust is becoming a serious contender for building web applications and how you can start using it today, even if you have no prior experience with Rust. Designed with a gentler learning curve, this new edition helps you understand Rust concepts in the context of web development, making it easier to transition from other programming languages.

You'll begin by exploring the Rust ecosystem for the web and getting hands-on with popular and emerging frameworks such as Actix, Axum, Rocket, and Hyper. You'll expand into frontend integration, injecting Rust into the browser using WebAssembly and configuring secure HTTPS connections with NGINX.

As you progress, you'll dive deeper into asynchronous programming, exploring TCP networking, framing, and the design of async systems. These concepts will help you understand how Rust handles concurrency and performance at scale, preparing you to build responsive and resilient web services.

Throughout the book, you'll apply what you learn by building a complete to-do application with authentication using a microservice-based architecture. This application will compile into a single Rust binary, embedding a frontend JavaScript application directly within it. You'll also implement end-to-end atomic testing and create a deployment pipeline to ensure your application is production-ready.

By the end of this book, you'll have a deep understanding of why Rust matters for web development and the confidence to build robust, scalable, and high-performance web applications from scratch using Rust.

Who this book is for

This book on web programming with Rust is for web developers who have programmed in traditional languages, such as Python, Ruby, and JavaScript, and are looking to develop high-performance web applications with Rust. Although no prior experience with Rust is necessary, a solid understanding of web development principles and basic knowledge of HTML, CSS, and JavaScript is required if you want to get the most out of this book.

What this book covers

Chapter 1, A Quick Introduction to Rust, focuses on what's different about Rust and provides the basics of the Rust programming language.

Chapter 2, Useful Rust Patterns for Web Programming, covers the syntax and borrowing rules of the strict type system of Rust.

Chapter 3, Designing Your Web Application in Rust, covers building and managing applications in Rust.

Chapter 4, Async Rust, focuses on the basics of async Rust and how async runtimes work under the hood.

Chapter 5, Handling HTTP Requests, covers building a basic Rust server that handles HTTP requests using the Actix Web framework.

Chapter 6, Processing HTTP Requests, focuses on extracting and handling data from an incoming HTTP request.

Chapter 7, Displaying Content in the Browser, covers displaying data from the server and sending requests to the server from the browser with HTML, CSS, and JavaScript with React.

Chapter 8, Injecting Rust in the Frontend with WebAssembly, focuses on concepts around serving WebAssembly modules to the frontend to be loaded and used in the browser.

Chapter 9, Data Persistence with PostgreSQL, covers managing and structuring data in PostgreSQL and interacting with the database with the Rust web server.

Chapter 10, Managing User Sessions, covers authentication and managing of user sessions when making requests to the web server.

Chapter 11, Communicating Between Servers, focuses on getting servers talking to each other either directly in memory or via HTTP requests.

Chapter 12, Caching Auth Sessions, covers building custom caching functionality directly inside Redis to control the state of the user session.

Chapter 13, Observability Through Logging, discusses the need for a logging setup and how this helps with debugging and monitoring the health of the system.

Chapter 14, Unit Testing, helps you to understand the importance of unit testing and enables you to write unit tests.

Chapter 15, End-to-End Testing, focuses on understanding the importance of end-to-end testing and building end-to-end tests.

Chapter 16, Deploying Our Application on AWS, covers building automated build and deployment pipelines to deploy on AWS using Docker and automating the infrastructure building with Terraform.

Chapter 17, Configuring HTTPS with NGINX on AWS, covers configuring HTTPS and routing to servers via load balancing on AWS with NGINX and routing traffic to different applications depending on endpoints in the URL.

Chapter 18, Practicalities of Using Microservices and Nanoservices, explores the positive and negative effects of microservices in relation to collaborating with other developers and managing a complex system as well as how nanoservices can help improve these trade-offs.

Chapter 19, Low-Level Networking, elaborates on how protocols are built to help you understand networking on a deeper level.

Chapter 20, Distributed Computing, covers implementing a queuing system where the tasks can be distributed across multiple computers to be processed using Redis.

To get the most out of this book

You will need to know some basic concepts around HTML and CSS. You will also need to have some basic understanding of JavaScript. However, the HTML, CSS, and JavaScript is only needed for displaying data in the browser. If you are just reading the book to build backend API servers in Rust, then knowledge of HTML, CSS, and JavaScript is not needed.

Some basic understanding of programming concepts such as functions and loops will also be needed as these will not be covered in the book.

Software/hardware covered in the book	Operating system requirements
Rust	Windows, macOS, or Linux (any)
Node (for JavaScript)	Windows, macOS, or Linux (any)
Python 3	Windows, macOS, or Linux (any)
Docker	Windows, macOS, or Linux (any)
docker-compose	Windows, macOS, or Linux (any)
Postman	Windows, macOS, or Linux (any)
Terraform	Windows, macOS, or Linux (any)

To take full advantage of the deployment on AWS chapters, you will need an AWS account, which might cost if you are not eligible for the Free Tier. However, the builds are automated with Terraform, so spinning up builds and tearing them down will be quick and easy, so you will not need to keep the infrastructure running while working through the book, keeping the costs to a minimum.

By the of this book, you will have a solid foundation of building and deploying Rust servers. However, it must be noted that Rust is a powerful language. As this book focuses on web programming and deployment, there is scope for improvement in Rust programming after the book. Further reading on Rust is advised after the book to enable you to solve more complex problems.

Download the example code files

This book includes a complete downloadable code bundle containing all the example projects and files used throughout the chapters. We recommend downloading the bundle so you can follow along smoothly and experiment with the examples.

Use the bundle as a practical starting point. Modify it, extend it, and apply what you learn by creating your own variations as you progress through the chapters.

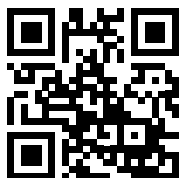
Get the code bundle

If you bought the book directly from Packt:

1. Go to **packtpub.com**
2. Click your **profile picture** and select **Your Orders**
3. Find this book and click **Download Code**

If you bought this book from Amazon or any other channel partner:

1. Go to `packtpub.com/unlock` or scan the following QR code:



2. Search for this book
3. Sign up or log in to your free Packt account
4. Upload your proof of purchase and download the code bundle locally

Usage note: You're free to use and modify this code for personal learning and non-commercial projects.

Download the color images

Your purchase includes a color, DRM-free PDF copy of this book, ideal for viewing color images, screenshots, and diagrams. Refer to *Free benefits with your book* at the end of the *Preface* to unlock your PDF copy.

Conventions used

There are a number of text conventions used throughout this book.

CodeInText: Indicates code words in text, database table names, folder names, filenames, file extensions, pathnames, dummy URLs, user input, and X/Twitter handles. For example: “The `User#handle_github_event` method deals only with domain objects, so it doesn't escape to the upper layers.”

A block of code is set as follows:

```
struct DbCacheHandle <T: CanCreate, X: CanDelete> {  
    create_handle: T,  
    delete_handle: X  
}
```

When we wish to draw your attention to a particular part of a code block, the relevant lines or items are set in bold:

```
struct DbCacheHandle <T: CanCreate, X: CanDelete> {  
    create_handle: T,  
    delete_handle: X  
}
```

Any command-line input or output is written as follows:

```
|      capitalize!(32);  
|      ----- in this macro invocation  
|  
= help: the trait `std::iter::FromIterator<char>` is not implemented for  
`{integer}`
```

Bold: Indicates a new term, an important word, or words that you see on the screen. For instance, words in menus or dialog boxes appear in the text like this. For example: “We can inspect the reason why by clicking on the **Not Secure** statement.”



Warnings or important notes appear like this.



Tips and tricks appear like this.

Get in touch

Feedback from our readers is always welcome.

General feedback: If you have questions about any aspect of this book or have any general feedback, please email us at customercare@packt.com and mention the book’s title in the subject of your message.

Errata: Although we have taken every care to ensure the accuracy of our content, mistakes do happen. If you have found a mistake in this book, we would be grateful if you reported this to us. Please visit <http://www.packt.com/submit-errata>, click **Submit Errata**, and fill in the form.

Piracy: If you come across any illegal copies of our works in any form on the internet, we would be grateful if you would provide us with the location address or website name. Please contact us at copyright@packt.com with a link to the material.

If you are interested in becoming an author: If there is a topic that you have expertise in and you are interested in either writing or contributing to a book, please visit <http://authors.packt.com/>.

Free benefits with your book

This book comes with free benefits to support your learning. Activate them now for instant access (see the “*How to Unlock*” section for instructions).

Here’s a quick overview of what you can instantly unlock with your purchase:



Complete Code Bundle

Download the full source code for this book’s examples and projects.



DRM-Free PDF Version

Download DRM-free PDF and ePub copies of this book.



7-Day Packt Library Access

Get 7-day unlimited access to 8,000+ books and videos. No credit card required.

Available for first-time Packt+ trial users only.



Next-Gen Reader Access

Read this book on Packt Reader with progress sync, dark mode and note-taking.

How to Unlock

Scan the QR code (or go to packtpub.com/unlock). Search for this book by name, confirm the edition, and then follow the steps on the page.



UNLOCK NOW

Note: Keep your invoice handy. Purchases made directly from Packt don't require one.

Share your thoughts

Once you've read *Rust Web Programming, Third Edition*, we'd love to hear your thoughts! Please [click here](#) to go straight to the Amazon review page for this book and share your feedback.

Your review is important to us and the tech community and will help us make sure we're delivering excellent quality content.

1

A Quick Introduction to Rust

Rust is growing in popularity, but it has a reputation for having a steep learning curve. However, if taught correctly, this learning curve can be reduced. By covering the basic rules around Rust, as well as learning how to manipulate a range of data types and variables, we will be able to write simple programs in the same fashion as dynamically typed languages, using a similar number of lines of code.

Understanding the basics is the key to effective web programming in Rust. I have maintained entire Kubernetes clusters where all the servers are written in Rust. Because I utilized traits, error handling, and generics effectively, it made reusing code in Rust extremely effective. In my experience, it takes fewer lines of code to build out Rust servers than it does Python servers if the basics of Rust are well utilized.

The goal of this chapter is to cover the main differences between Rust and generic dynamic languages and to provide you with a quick understanding of how to utilize Rust.

In this chapter, we will cover the following topics:

- What is Rust?
- Reviewing data types and variables in Rust
- Controlling variable ownership
- Building structs

Once we have covered the main concepts in this chapter, you will be able to code basic programs in Rust that will run. You will also be able to debug your programs and understand the error messages that are thrown by the Rust compiler. As a result, you will have the foundations to be productive in Rust. You will also be able to move on to structuring Rust code over multiple files.

In this chapter and the following chapter, we will cover enough Rust to be productive in web programming. However, we will not be able to do the entire Rust programming language justice in two chapters. If you want to dive deeper into the Rust programming language after reading this chapter, then you can explore the further reading list at the end of the chapter.

Your purchase includes a free PDF copy + code bundle

Your purchase includes a DRM-free PDF copy of this book, the code bundle, and additional exclusive extras. See the *Free benefits with your book* section in the *Preface* to unlock them instantly and maximize your learning.

Technical requirements

For this chapter, we only need access to the internet, as we will use the online Rust playground to implement the code. The code examples provided can be run in the online Rust playground at <https://play.rust-lang.org/>.

You can download the example project and code for this book by following the instructions in the **Download the example code files** section in the *Preface* of this book.

This chapter's code files are included in the downloadable code bundle.

What is Rust?

Rust is a cutting-edge systems programming language that has been making waves since Mozilla Research introduced it in 2010. With a focus on safety, concurrency, and performance, Rust is a formidable alternative to traditional languages like C and C++. Its most notable feature, the ownership system, enforces rigorous memory safety rules at compile time. This approach effectively eradicates common pitfalls like null pointer dereferencing and buffer overflows, all without needing a garbage collector.

Designed for high performance, Rust provides granular control over hardware and memory, making it perfect for developing operating systems, game engines, and other performance-critical applications. Its syntax is both modern and expressive, offering features typically seen in higher-level languages, such as pattern matching and algebraic data types, while retaining the efficiency required for system-level programming. Consequently, Rust has rapidly attracted a strong and active community, bolstered by excellent documentation and a burgeoning ecosystem of libraries and tools.

Rust's adoption has been nothing short of remarkable, particularly in tech sectors where safety and performance are crucial. Major players like Microsoft, Amazon, and Dropbox have integrated Rust into their technology stacks to capitalize on its reliability and efficiency. Rust's consistent ranking since 2016 as the most loved programming language in Stack Overflow's annual developer surveys is a testament to its appeal. Developers laud Rust for its unique blend of performance and safety, alongside its supportive and vibrant community.

Moreover, Rust's ecosystem, highlighted by its package manager, Cargo, offers a seamless development experience. Cargo simplifies dependency management, project building, and testing, streamlining workflows that can be cumbersome in other systems programming languages. This robust combination of features positions Rust as a unique and powerful tool for developers aiming to create reliable, high-performance software.

Why is Rust revolutionary?

With programming, there is usually a trade-off between speed and resources and development speed and safety. Low-level languages such as C/C++ can give a developer fine-grained control over a computer, with fast code execution and minimal resource consumption. However, this is not free. Languages such as C/C++ need manual memory management, which can introduce bugs and security vulnerabilities. A simple example of this is a buffer overflow attack.



A buffer overflow attack occurs when the programmer does not allocate enough memory. For instance, if the buffer only has a size of 15 bytes, and 20 bytes are sent, then the excess 5 bytes might be written past the boundary. An attacker can exploit this by passing in more bytes than the buffer can handle. This can potentially overwrite areas that hold executable code with their own code.

There are other ways to exploit a program that does not have correctly managed memory. On top of increased vulnerabilities, it takes more code and time to solve a problem in a low-level language. As a result of this, C++ web frameworks do not take up a large share of web development. Instead, it usually makes sense to go for high-level languages such as Python, Ruby, and JavaScript. Using such languages will generally result in a developer solving problems safely and quickly.

However, it must be noted that this memory safety comes at a cost. These high-level languages generally keep track of all the variables defined and their references to a memory address. When there are no more variables pointing to a memory address, the data in that memory address gets deleted. This process is called garbage collection and consumes extra resources and time, as a program must be stopped to clean up the variables.

With Rust, memory safety is ensured without the costly garbage collection process. Rust ensures memory safety through a set of ownership rules, checked at compile time with a borrow checker. Because of this, Rust enables rapid, safe problem-solving with truly performant code, thus breaking the speed/safety trade-off.



Memory safety is the property of programs having memory pointers that always point to valid memory.

With more data processing, traffic, and complex tasks lifted into the web stack, Rust, with its growing number of web frameworks and libraries, has now become a viable choice for web development. This has led to some truly amazing results in the web space for Rust. In 2020, Shimul Chowdhury ran a series of tests against servers with the same specs but different languages and frameworks. The results can be seen in the following figure (note that the Rust frameworks comprise Actix Web and Rocket):

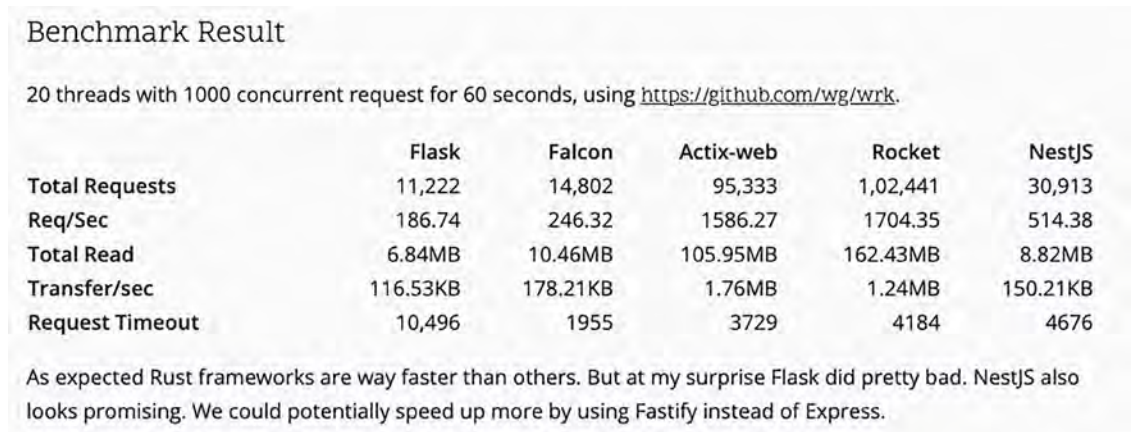


Figure 1.1 – Results of different frameworks and languages by Shimul Chowdhury (found at <https://www.shimul.dev/en/blog/2020/benchmarking-flask-falcon-actix-web-rocket-nestjs/>)

In the preceding figure, we can see that there are some variations in the languages and frameworks. These Rust servers are in a completely different league when it comes to total requests handled and data transferred. Other languages, such as Golang, have come onto the scene, but the lack of garbage collection in Rust has managed to outshine Golang. This was demonstrated in Jesse Howarth's 2020 blog post *Why Discord is switching from Go to Rust* (<https://discord.com/blog/why-discord-is-switching-from-go-to-rust>). In this post, it was clear that Golang servers were producing latency spikes, whereas the Rust servers were not.

The garbage collection that Golang was implementing to keep the memory safe resulted in two-minute spikes. But it does not stop there. In 2022, AWS produced a report called “*Sustainability with Rust*,” wherein they created a ratio of energy consumption, resulting in the following table:

Language	Energy Rating
C	1.00
Rust	1.03
JavaScript	4.45
PHP	29.30
Ruby	69.91
Python	75.88

Table 1.1 – Energy Rating of Languages from an AWS report (found at <https://aws.amazon.com/blogs/opensource/sustainability-with-rust/>)

Rust was second only to C in energy consumption and startup time. AWS is not the only fan of Rust. In 2024, the Whitehouse recommended Rust over C and C++ for future projects in the “*Back to the Building Blocks: A Pat Toward Secure and Measurable Software*” report. Finally, the compatibility of Rust with other systems has created an inflection point where we can integrate it. For instance, PostgreSQL and Redis now support modules that can be written in Rust and uploaded. Cloudflare has written a battle-tested load balancer in Rust that serves more than 40 million internet requests per second.

I personally think that Rust is the future of backend web programming. Like C pushed forward hardware, where all developers agreed on standards and rowed in the same direction, you will hopefully appreciate after finishing this book that Rust could push forward a backend standard, where the developer has a memory-safe language that can interact directly with other components, such as load balancers and databases, in a way that garbage-collected languages just cannot.

Why Rust? cheatsheet



- **Balance:** Rust balances speed, resource efficiency, development speed, and safety without compromising any aspect.
- **Low-level control:** Like C/C++, Rust provides fine-grained control over the computer, leading to fast code execution and minimal resource consumption.
- **Rust’s memory safety:** Languages like Python, Ruby, and JavaScript offer memory safety via garbage collection, but at the cost of additional resources and potential performance overhead. Rust ensures memory safety through compile-time ownership rules and a borrow checker, eliminating the need for garbage collection.
- **Web development:** Rust’s growing number of web frameworks and libraries make it a strong candidate for web development, offering both safety and performance.
- **Integration:** Rust’s compatibility with systems like PostgreSQL and Redis, as well as its use in high-performance components like Cloudflare’s load balancer, demonstrate its versatile integration capabilities.

Now that we understand why we want to code in Rust, we can move on to reviewing data types in the next section.

Reviewing data types and variables in Rust

If you have coded in another language before, you will have used variables and handled different data types. However, Rust does have some quirks that can put off developers. This is especially true if the developer has come from a dynamic language, as these quirks mainly revolve around memory management and reference to variables. These can be intimidating initially, but when you get to understand them, you will learn to appreciate them.

Some people might hear about these quirks and wonder why they should bother with the language at all. This is understandable, but these quirks are why Rust is such a paradigm-shifting language. Working with borrow checking and wrestling with concepts such as lifetimes and references gives us the high-level memory safety of a dynamic language, such as Python. However, we can also get memory-safe, low-level resources, such as those delivered by C and C++.

This means that we do not have to worry about dangling pointers, buffer overflows, null pointers, segmentation faults, data races, and other issues when coding in Rust. Issues such as null pointers and data races can be hard to debug. The borrow-checking rules enforced are a good trade-off, as we must learn Rust's quirks to get the speed and control of non-memory-safe languages, but we do not get the headaches these non-memory-safe languages introduce.

Before we do any web development, we need to run our first program. We can do this in the Rust playground at <https://play.rust-lang.org/>.

If you have never visited the Rust playground before, you will see the following layout once you are there:

```
fn main() {  
    println!("hello world");  
}
```

The preceding code will look like the following screenshot in the online Rust playground, after we have pressed the RUN button on the top left-hand side of the screen:

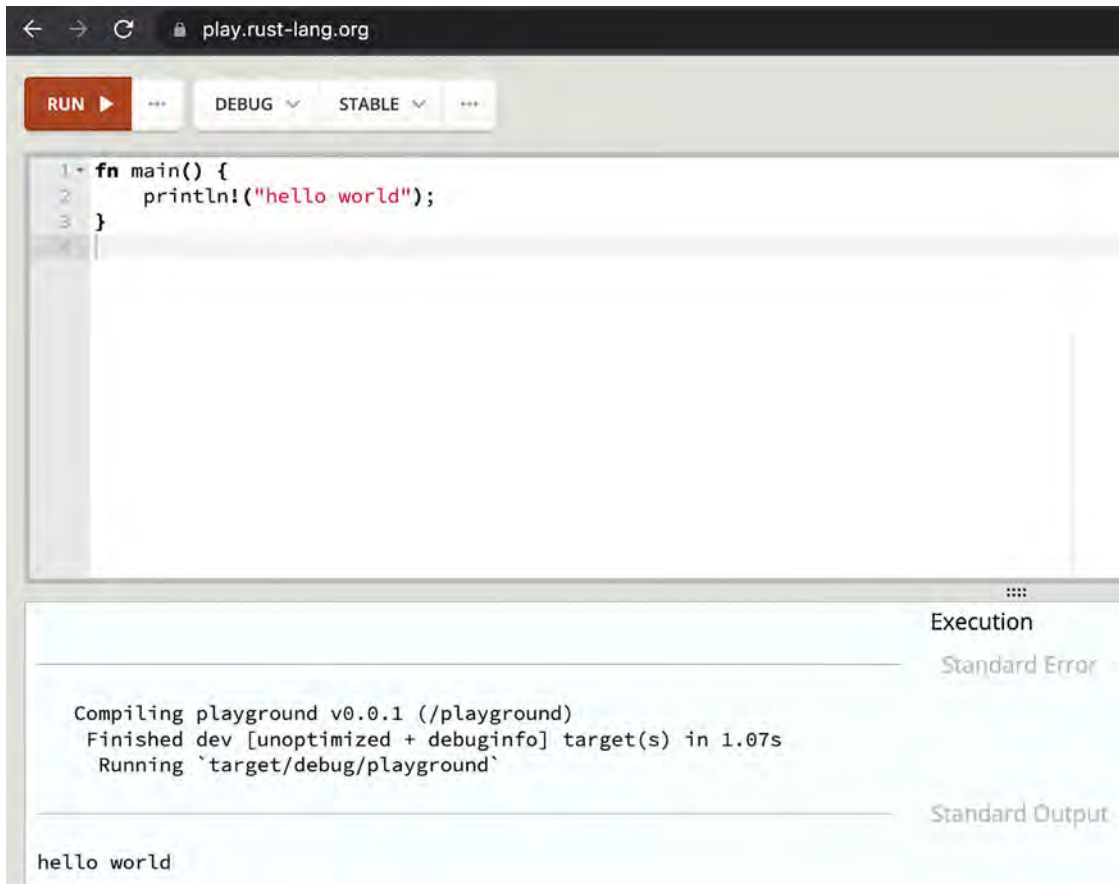


Figure 1.2 – View of the online Rust playground

In our `hello world` code, what we have is a `main` function, which is our entry point. This function fires when we run our program. All programs have entry points. If you have not heard of the concept before, due to coming from a dynamic language, the entry point is the script file that you point your interpreter at. For Python, a closer analogy would be the `main` block that runs if the file is directly run by the interpreter, denoted as follows:

```
if __name__ == "__main__":  
    print("Hello, World!")
```

If you were to code in Python, you would probably see this used in a Flask application.

Right now, we have not done anything new. This is a standard *Hello, World!* example with a little change in syntax; however, even with this example, the string that we print is not all that it seems. For instance, let us write our own function that accepts a string and prints it out with the following code:

```
fn print(message: str) {  
    println!("{}", message);  
}  
fn main() {  
    let message = "hello world";  
    print(message);  
}
```

This code should work in other interpreted languages, such as Python or JavaScript. We pass it into our function and print it. However, if we do print it, we get the following printout:

```
10 |     print(message);  
    |         ^^^^^^ doesn't have a size known at compile-time  
    |  
    = help: the trait `Sized` is not implemented for `str`  
    = note: all function arguments must have a statically known size
```

This is not very straightforward, but it brings us to the first area we must understand if we are to code in Rust, which is strings. Don't worry; strings are the quirkiest variables that you need to get your head around to write functional Rust code.

Using strings in Rust

Before we explore the error in the previous section, let us rectify it so that we know what to work toward. We can get the `print` function to work without any errors with the following code:

```
fn print(message: String) {  
    println!("{}", message);  
}  
fn main() {  
    let message = String::from("hello world");  
    print(message);  
}
```

What we did was create a `String` from "hello world" and pass it into the `print` function. This time, the compiler did not throw an error because we always know the size of a `String`, so we can keep the right amount of memory free for it. This may sound counterintuitive because strings are usually of different lengths; it would not be a very flexible programming language if we were only allowed to use the same length of letters for every string in our code. We can make all strings have the same size of memory by passing round pointers, implemented as a vector of bytes, which in Rust is denoted as `Vec<u8>`. This holds a reference to the string content (`str`, also known as a string slice) in the heap memory, as seen in the following figure:

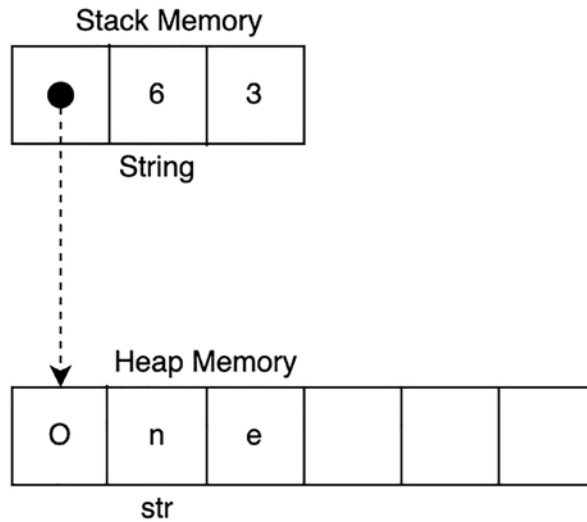


Figure 1.3 – A string’s relationship to `str` “one”

We can see in Figure 1.3 that a string is a vector of three numbers. One is the actual memory address of the `str` it references. The second number is the size of the memory allocated, and the third is the length of the string content. Therefore, we can access string literals in our code without having to pass variables of various sizes around our code. We know that `String` has a set size and, therefore, can allocate this size in the `print` function parameter. Note that `String` is on the stack memory while our string literal is on the heap memory.

Considering that we know that `String` has a set size while our string literal varies, we can deduce that the stack memory is used for predictable memory sizes and is allocated ahead of time when the program runs. The stack memory allocation order is decided on compilation and optimized by the compiler. Our heap memory is dynamic, and therefore, memory is allocated when it is needed.

Now that we know the basics of strings, we can use the different ways in which they are created, as seen in the following code:

```
let string_one = "hello world".to_owned();
let string_two = "hello world".to_string();
let string_three = string_two.clone();
```


Note that creating `string_three` is costly, as we must copy the underlying data in the heap, and heap operations are expensive. This is not a unique quirk of Rust. In our example, we are just experiencing what happens under the hood. For instance, if we alter strings in Python, we will have different outcomes:

```
# slower method
data = ["one", "two", "three", "four"]
string = ""
for i in data:
    string += i
# faster method
"".join(data)
```

Looping through and adding the strings is slower because Python must allocate new memory and copy an entire string to that new memory address. The `join` method is faster because Python can allocate the memory of all the data of the list and then copy over the strings in the array, meaning the string must only be copied once. This shows us that although high-level languages like Python may not force you to think about the memory allocation of strings, you will still end up paying the price if you don't acknowledge it.

We can pass a string literal into the `print` function by borrowing it, as seen in the following Rust code:

```
fn print(message: &str) {
    println!("{}", message);
}

fn main() {
    let message: &str = "hello world";
    print(message);
}
```

The `&str` is a type annotation of the type of variable that is being assigned to `message`. We do not need the type annotation; the compiler will work out the type automatically. However, for our exercise, it does illustrate what is happening. The borrow is denoted by `&`. We will cover borrowing later in the chapter. For now, however, we can deduce that the borrow is only a fixed-size reference to a variable-sized string slice. If the borrow was not a fixed size, we would not be able to pass it into the `print` function because we would not know the size.

At this point, we can comfortably use strings in Rust productively. The next concept that we must understand before we start writing Rust programs is integers and floats.

Using integers and floats

In most high-level web programming languages, we just assign a float or integer to a variable name and move on with the program. However, from what we have been exposed to in the previous section on strings, we now understand that we must worry about memory size when using strings in Rust. This is no different with integers and floats. We know that integers and floats have a range of sizes. Therefore, we must tell Rust what we pass around our code.

Rust supports signed integers, which are denoted by `i`, and unsigned integers, which are denoted by `u`. Both unsigned and signed integers consist of 8, 16, 32, 64, or 128 bits. We will cover what unsigned and signed integers are in this section.

8-byte integers

Exploring the math behind numbers being represented in binary is not relevant for this book; however, we do need to understand the range of numbers that can be represented with several bits, as this will help us understand what the different types of floats and integers in Rust denote.

Because binary is either a 0 or a 1, we can calculate the range of unsigned integers that can be represented by the bits, by raising 2 to the power of the number of bits we have. For example, if we have an integer that is represented by 8 bits, 2 to the power of 8 equates to 256. We must remember that 0 is also represented. Considering this, an integer of 8 bits has a range of 0 to 255. We can test this calculation with the following code:

```
let number: u8 = 256;
```

This is one higher than the range that we calculated. As a result, we should not be surprised to see the following overflow error:

```
the literal `256` does not fit into the type
`u8` whose range is `0..=255`
```

So we can deduce that if we lower the unsigned integer to 255, it will pass. However, let's say we change the unsigned integer into a signed integer with the following code:

```
let number: i8 = 255;
```

We will see that we get a helpful error message, as follows:

```
the literal `255` does not fit into the type
`i8` whose range is `-128..=127`
```

With this helpful error message, we can see that a signed integer considers negative numbers, so the absolute value that a signed integer can take is roughly half.

16-byte integers

We can increase the range by assigning the number as a 16-bit signed integer with the following code:

```
let number: i16 = 255;
```

This would work. However, let us add our 16-bit integer with our 8-bit integer, using the following code:

```
let number = 255i16;
let number_two = 5i8;
let result = number + number_two;
```

The previous code might look a little different to you. All we have done in the preceding code is define the data type with a suffix instead. So `number` has a value of 255 and a type of `i16`, and `number_two` has a value of 5 and a type of `i8`. If we run the previous code, we get the following error:

```
11 |     let result = number + number_two;
    |               ^ no implementation for `i16 + i8`
    |
    = help: the trait `Add<i8>` is not implemented for `i16`
```

We will cover traits later in this chapter. For now, all we must understand is that we cannot add the two different integers. If they were both the same type, then we could.

We can change the integer type through casting, using `as`, as seen in the following line of code:

```
let result = number + number_two as i16;
```

This means that `number_two` is now a 16-bit integer, and `result` will be 260. However, we must be careful with casting because if we were to do it the wrong way, we could end up with a silent bug, which is unusual for Rust.



Embrace the errors when learning Rust

Rust's strict compilation process can seem daunting, but it's a key feature that prevents runtime failures common in other languages. Rust refuses to compile code with errors, ensuring safety and reliability. By encountering and understanding these errors early, you can learn Rust more effectively.

Trying out code variants without error warnings can lead to confusion and frustration later. Teaching Rust by showing errors upfront and explaining them helps us to grasp the language's principles, making our learning process smoother and more engaging.

If we cast `number` as `i8` instead of casting `number_two` as `i16`, then `result` would equate to 4, which does not make sense because 255 + 5 equals 260. This is because `i8` is smaller than `i16`. So, if we cast an `i16` integer as an `i8` integer, we are essentially chopping off some of the data, by just taking the lower bits of the number and disregarding the upper bits. Therefore, `number` ends up being a -1 if we cast it to an `i8` integer. To be safer, we can use the `i8::from` function, as seen in the following code:

```
let result = i8::from(number) + number_two;
```

Running this will give us the following error:

```
let result = i8::from(number) + number_two;
|               ^^^^^^^ the trait `From<i16>` is not
                        implemented for `i8`
```

Again, we will go over traits later on in the chapter, but we can see in the preceding code that because the trait `From<i16>` is not implemented for an `i8` integer, we cannot cast an `i8` integer into an `i16` integer. With this understanding, we are free to work with integers safely and productively.



Type system to the rescue

Type errors for multiplying two different numerical types prevent silent bugs. If the type system did not flag the differences, then we could have bugs that get thrown when the program is running or, even worse, silent bugs where we don't know that there is a bug.

Previously, I worked on a financial loss calculation engine. If the number was not loaded to the correct precision of the spec, then the calculations at the end of the financial loss model would be wildly different. The type system here ensures that we do not multiply a number that is a different precision to the spec defined in the code.

Introducing floats

One last point about integer sizes in Rust is that they are not continuous. The supported sizes are shown in the following table:

Bits	Calculation	Size
8	2^8	256
16	2^{16}	65536
32	2^{32}	4294967296
64	2^{64}	1.8446744e+19
128	2^{128}	3.4028237e+38

Table 1.2 – Size of integer types

When it comes to floats, Rust accommodates `f32` and `f64` floating point numbers. Both these floating-point types support negative and positive values. Declaring a floating-point variable requires the same syntax as integers, as seen in the following code:

```
let float: f32 = 2.6;
```

With this, we can comfortably work with integers and floats in our Rust code. However, we know as developers that just declaring floats and integers is not very useful. We want to be able to contain and loop through them. In the next section, we will do just that with vectors and arrays.

Storing data in arrays

In Rust, we can store our floats, integers, and strings in arrays and vectors. First, we will focus on arrays.

Arrays are stored in stack memory. Knowing this, and remembering what we learned about strings, we can deduce that arrays are of a fixed size. This is because, as we remember, if a variable is stored on the stack, then the memory is allocated on compilation and loaded into the stack when the program starts. We can define an array of integers, loop through it, print each integer, and then access an integer by index with the following code:

```
fn main() {  
    let int_array: [i32; 3] = [1, 2, 3];  
    for i in int_array {  
        println!("{}", i);  
    }  
    println!("{}", int_array[1]);  
}
```

With the previous code, we define the type and size by wrapping them in square brackets. For instance, if we were going to create an array of floats with a length of 4, we would use `int_array: [f32; 4] = [1.1, 2.2, 3.3, 4.4]`. Running the preceding code will give you the following printout:

```
1  
2  
3  
2
```

In the preceding printout, we can see that the loop works and we can access the second integer with square brackets. Although the memory size of the array is fixed, we can still change it. This is where mutability comes in.

When we define a variable as mutable, it means that we can mutate it. In other words, we can alter the value of the variable after it has been defined if it is mutable. If you tried to update any of the variables in the code that we have written in this chapter, you will have realized that you can't. This is because all variables in Rust are immutable by default. However, we can make any variable in Rust mutable by putting a `mut` tag in front of the variable name.

Going back to the fixed array, we cannot change the size of it, meaning we cannot append/push new integers to it due to it being stored in stack memory. However, if we define a mutable array, we can update parts of it with other integers that are the same memory size. An example of this is the following code:

```
fn main() {  
    let mut mutable_array: [i32; 3] = [1, 2, 0];  
    mutable_array[2] = 3;  
    println!("{:?}", mutable_array);  
    println!("{}", mutable_array.len());  
}
```

In the preceding code, we can see that the last integer in our array is updated to 3. We then print out the full array and then the length. You may have also noted that the first print statement of the preceding code now employs `{:?}`. This calls the `Debug` trait. If `Debug` is implemented for the thing that we are trying to print, then the full representation of the thing we print is displayed in the console. You can also see that we print out the result of the length of the array. Running this code will give the following printout:

```
[1, 2, 3]  
3
```

With the preceding printout, we can confirm that the array is now updated. We can also access slices with our arrays. To demonstrate this, we can create an array of 100 zeros. We can then take a slice of this and print it out with the following code:

```
fn main() {
    let slice_array: [i32; 100] = [0; 100];
    println!("length: {}", slice_array.len());
    println!("slice: {:?}", &slice_array[5 .. 8]);
}
```

Running the preceding code will result in the following printout:

```
length: 100
slice: [0, 0, 0]
```

We are now able to be productive with arrays. Arrays can be useful for caching. For instance, if we know the amount that we need to store, then we can use arrays effectively. However, we have only managed to store one type of data in the array. If we tried to store strings and integers in the same array, we would have a problem. How would we define the type? This problem applies to all collections, such as vectors and **HashMaps**. There are multiple ways to do this, but the most straightforward is using enums.

Storing data in vectors

What we have covered with arrays can be applied to vectors. The only difference is that we do not have to define the length and can increase the size of the vector if needed. This is because vectors put their data onto the heap, meaning that vectors are pointers, pointing to a collection of items in heap memory that are all next to each other, making vectors fast for looping through and reading data. If the allocated memory on the heap runs out, the vector can allocate a bigger section of memory and copy the existing data over to the new memory address. To demonstrate this flexibility, we will create a vector of strings and then add a string to the end, with the following code:

```
let mut string_vector: Vec<&str> = vec!["one", "two", "three"];
println!("{:?}", string_vector);
string_vector.push("four");
println!("{:?}", string_vector);
```

In the preceding code, we can see that we use the `vec!` macro to create the vector of strings. You may have noticed with macros such as `vec!` and `println!` that we can vary the number of inputs. We will cover macros later in the chapter. Running the preceding code will result in the following printout:

```
["one", "two", "three"]
["one", "two", "three", "four"]
```

We can also create an empty vector with the new function from the `Vec` struct, with `let _empty_vector: Vec<&str> = Vec::new();`.

You may be wondering when to use vectors and when to use arrays. Vectors are more flexible. You may be tempted to opt for arrays for performance gains. At face value, this seems logical, as arrays are stored in the stack. Accessing the stack is going to be quicker because the memory sizes can be computed at compile time, making the allocation and deallocation simpler compared to the heap. However, because it is on the stack, it cannot outlive the scope that it is allocated. Moving a vector around would require moving a pointer around. However, moving an array requires copying the whole array. Therefore, copying fixed-size arrays is more expensive than moving a vector. If you have a small amount of data that you only need in a small scope and you know the size of the data, then reaching for an array does make sense. However, if you're going to be moving the data around, even if you know the size of the data, using vectors is a better choice.

We must also take note of our initial introduction of vectors. If the heap memory allocated for a vector runs out, the data is copied over to a new address. This copying over is not free. If you know the size of the data you need, you can prevent this copying with a `Vec::with_capacity(100)` constructor, initially setting aside the memory you need. 100 is just an example of a size of items to be put in a vector; you have the freedom to input whatever value you need. Don't worry if you exceed this later; the memory reallocation will still work.

Now that we can be productive with basic collections, we can move on to a more advanced collection, a `HashMap`.

Mapping data with enums

Enums are, well, enums. In dynamic languages such as Python, you may not have had to use them, due to being able to pass any type anywhere you want. However, they are still available. Enum is short for enumerated type and basically defines a type with possible variants. In our case, we want our array to store strings and integers in the same collection. We can do this by initially defining our enum with the following code:

```
enum SomeValue {  
    StringValue(String),  
    IntValue(i32)  
}
```

In the preceding code, we can see that we defined an enum with the name of `SomeValue`. We then denoted that `StringValue` holds the value of a string and that `IntValue` holds the value of an integer. We can then define an array with a length of 4, consisting of two strings and two integers, with the following code:

```
let multi_array: [SomeValue; 4] = [  
    SomeValue::StringValue(String::from("one")),  
    SomeValue::IntValue(2),  
    SomeValue::StringValue(String::from("three")),  
    SomeValue::IntValue(4)  
];
```

In the preceding code, we can see that we wrap our strings and integers in our enum. Now, looping through and exporting it is going to be another task. For instance, there are things that we can do to an integer that we cannot do to a string, and vice versa. Considering this, we are going to have to use a `match` statement when looping through the array, as seen in the following code:

```
for i in multi_array {  
    match i {  
        SomeValue::StringValue(data) => {  
            println!("The string is: {}", data);  
        },  
        SomeValue::IntValue(data) => {  
            println!("The int is: {}", data);  
        }  
    }  
}
```

In the preceding code, we can see that if `i` is `SomeValue::StringValue`, we then assign the data wrapped in `SomeValue::StringValue` to the variable name `data`. We then pass `data` into the inner scope to be printed. We use the same approach with our integer. Even though we are merely printing to demonstrate the concept, we can do anything in these inner scopes to the `data` variable that the type allows us to. Running the preceding code gives the following printout:

```
The string is: one  
The int is: 2  
The string is: three  
The int is: 4
```

Using enums to wrap data and `match` statements to handle them can be applied to `HashMaps` and `vectors`.

Mapping data with HashMaps

In some other languages, `HashMaps` are referred to as dictionaries. They have a key and a value. We can insert and get values using the key. Now that we have learned about handling collections, we can get a little more adventurous in this section.

We can create a simple profile of a game character. In this character profile, we are going to have a name, age, and a list of items that they have. This means that we need an enum that houses a string, an integer, and a vector that also houses strings. We will want to print out the complete `HashMap` to see if our code is correct in one glance. To do this, we are going to implement the `Debug` trait for our enum, as seen in the following code:

```
#[derive(Debug)]  
enum CharacterValue {  
    Name(String),
```



```
    Age(i32),
    Items(Vec<String>)
}
```

In the preceding code, we can see that we have annotated our enum with the `derive` attribute. An attribute is metadata that can be applied to the `CharacterValue` enum in this case. The `derive` attribute tells the compiler to provide a basic implementation of a trait. So, in the preceding code, we are telling the compiler to apply the basic implementation of `Debug` to the `CharacterValue` enum. With this, we can then create a new `HashMap` that has keys pointing to the values we defined in the preceding code with the following:

```
use std::collections::HashMap;
fn main() {
    let mut profile: HashMap<&str, CharacterValue> = HashMap::new();
}
```

We stated that it is mutable because we are going to insert values with the following code:

```
profile.insert("name", CharacterValue::Name("Maxwell".to_string()));
profile.insert("age", CharacterValue::Age(34));
profile.insert("items", CharacterValue::Items(vec![
    "laptop".to_string(),
    "book".to_string(),
    "coat".to_string()
]));

println!("{:?}", profile);
```

We can see that we have inserted all the data that we need. Running this would give us the following printout:

```
{"items": Items(["laptop", "book", "coat"]), "age": Age(34),
 "name": Name("Maxwell")}
```

In the preceding output, we can see that our data is correct. Inserting it is one thing; however, we now must get it out again. We can do this with a `get` associated function.



Associated functions are functions attached to a type, such as an enum, struct, or datatype. They can reference the entity that they are associated with, or even consume them. In some cases, a reference to the type they are associated with isn't needed at all. Later on in the book, we will associate functions with database handle structs where the functions have no reference to the struct.

The `get` associated function returns an `Option` type. The `Option` type returns either `Some` or `None`. So if we were to get `name` from our `HashMap`, we would need to do two matches, as seen in the following code:

```
match profile.get("name") {
    Some(value_data) => {
        match value_data {
            CharacterValue::Name(name) => {
                println!("the name is: {}", name);
            },
            _ => panic!("name should be a string")
        }
    },
    None => {
        println!("name is not present");
    }
}
```

In the preceding code, we can check to see if there is a name in the keys. If there is not, then we just print out that it was not present. If the name key is present, we then move on to our second check, which prints out the name if it is `CharacterValue::Name`. However, there is something wrong if the name key does not house `CharacterValue::Name`. So, we add only one more check to match, which is `_`. This is a catch, meaning anything else. We are not interested in anything other than `CharacterValue::Name`. Therefore, the `_` catch maps to a `panic!` macro, which essentially throws an error.

We could make this shorter. If we know that the name key is going to be in the `HashMap`, we can employ the `unwrap` function with the following code:

```
match profile.get("name").unwrap() {
    CharacterValue::Name(name) => {
        println!("the name is: {}", name);
    },
    _ => panic!("name should be a string")
}
```

The `unwrap` function directly exposes the result. However, if the result is `None`, then it will directly result in an error terminating the program, which would look like the following printout:

```
thread 'main' panicked at 'called `Option::unwrap()` on a `None` value'
```

Using the `unwrap` function is risky, and we should try and avoid it as much as possible. We can avoid using the `unwrap` function by handling our results and errors, which we cover in the next section.

Handling results and errors

In the previous section, we learned that directly unwrapping `Option`, resulting in `None`, panics a thread. There is another outcome that can also throw an error if unsuccessfully unwrapped, and this is `Result`.

The `Result` type can return either `Ok` or `Err`. To demonstrate this, we can create a basic function that returns a `Result` type, based on a simple `Boolean` that we pass into it, with the following code:

```
fn error_check(check: bool) -> Result<i8, &'static str> {  
    if check {  
        Err("this is an error")  
    }  
    else {  
        Ok(1)  
    }  
}
```

In the preceding code, we can see that we return `Result<i8, &'static str>`. This means that we return an integer if `Result` is `Ok`, or we return an integer if `Result` is `Err`. The `&'static str` variable is basically our error string. We can tell that it's a reference because of `&`. The `'static` part means that the reference is valid for the entire lifetime of the running program. If this does not make sense now, don't worry; we will cover lifetimes later in the chapter.

Now that we have created our error-checking function, we can test to see what these outcomes look like with the following code:

```
fn main() {  
    println!("{:?}", error_check(false));  
    println!("{:?}", error_check(false).is_err());  
    println!("{:?}", error_check(true));  
    println!("{:?}", error_check(true).is_err());  
}
```

Running the preceding code gives us the following printout:

```
Ok(1)  
false  
Err("this is an error")  
true
```

In the preceding output, we can see that it returned exactly what we wanted. Also note that we can run the `is_err()` function on `Result` variable, resulting in `false` if we return `Ok` or `true` if we return `Err`. We can also directly unwrap but add extra tracing to the stack trace with the following `expect` function:

```
let result: i8 = error_check(true).expect("this has been caught");
```

The preceding function will result in the following printout:

```
thread 'main' panicked at 'this has been caught: "this is an error"'
```

Through the preceding example, we can see that we get the message from the `expect` function first, and then the error message returned in `Result`. With this understanding, we can throw, handle, and add extra tracing to errors.

We can now throw errors, but how can we handle them? An obvious choice would be a `match` statement, but we can also use the `?` operator. For instance, let us return the same return type for another function that calls our `error_check` function, as seen in the following code:

```
fn error_check_two(check: bool) -> Result<i8, &'static str> {
    let outcome: i8 = error_check(check)?;
    Ok(outcome)
}
```

Here, we can see that the `?` operator is used when calling `error_check(check)?`. What happens here is that if the `error_check` function returns an error, that error is just directly returned as an error for the `error_check_two` function. If the `error_check` function returns an `Ok` result, then this is automatically unwrapped and assigned to the variable `outcome`. This will save you a lot of code. Throughout the book, we will map errors so that we can exploit the `?` operator, instead of writing matches everywhere. We will even configure our errors to automatically construct an HTTP response. It's a complete myth that we need to write a lot of Rust code for web programming.

Rust data types and variables cheatsheet

- **Strings:** The `String` type is heap-allocated and its size is known, whereas `&str` is a fixed-size reference to a string slice. Use `String::from` or `.to_string()` to create a `String`. Use `&str` to borrow a string slice without ownership transfer.
- **Integers and floats:** Rust requires explicit size declarations for integers (e.g., `i8`, `u16`) and floats (e.g., `f32` and `f64`). Different integer sizes have specific ranges (e.g., `u8` ranges from 0 to 255, and `i8` ranges from -128 to 127). Type errors are caught at compile time to prevent runtime bugs.
- **Mutability:** Variables are immutable by default. Use `mut` to make a variable mutable, allowing its value to be changed.
- **Arrays:** Fixed-size, stack-allocated collections. Here's an example: `let int_array: [i32; 3] = [1, 2, 3];`
- **Vectors:** Dynamic-size, heap-allocated collections. Here's an example: `let mut vec = Vec::new(); vec.push(1);`
- **Enums:** Used to define a type with possible variants. Here's an example: `enum SomeValue { StringValue(String), IntValue(i32) }`
- **Result type:** Used for functions that can return an error. Here's an example: `fn error_check(check: bool) -> Result<i8, &'static str> { ... }`
- **Error handling:** Use `match`, `unwrap`, or the `?` operator to handle results and propagate errors.

However, we are getting more exposed to lifetimes and borrow references as we move forward. Now is the time to address this by understanding variable ownership.

Controlling variable ownership

As we remember from the beginning of the chapter, Rust does not have a garbage collector. However, it has memory safety. It achieves this by having strict rules around variable ownership. These rules are enforced when Rust is being compiled. If you are accustomed to a dynamic language, then this can initially lead to frustration. This is known as *fighting the borrow checker*. Sadly, this unjustly gives Rust its *steep learning curve* reputation, as when you are fighting the borrow checker without knowing what is going on, it can seem like an impossible task to write even the most basic programs. However, if we take the time to learn the rules before we try and code anything too complex, the knowledge of the rules and the helpfulness of the compiler will make writing code in Rust fun and rewarding. Again, I take the time to remind you that Rust has been the most favorite language nine years in a row. This is not because it's impossible to get anything done in it. The people who vote for Rust in these surveys understand the rules around ownership. Rust's compiling, checking, and enforcing of these rules protect against the following errors:

- **Use-after-frees:** This occurs when memory is accessed once it has been freed, which can cause crashes. It can also allow hackers to execute code via this memory address.
- **Dangling pointers:** This occurs when a reference points to a memory address that no longer houses the data that the pointer was referencing. Essentially, this pointer now points to null or random data.
- **Double frees:** This occurs when allocated memory is freed and then freed again. This can cause a program to crash and increases the risk of sensitive data being revealed. It also enables a hacker to execute arbitrary code.
- **Segmentation faults:** This occurs when the program tries to access the memory it's not allowed to access.
- **Buffer overrun:** An example of this error is reading off the end of an array. This can cause a program to crash.

To protect against these errors and thus achieve memory safety, Rust enforces the following rules:

- Values are owned by the variables assigned to them.
- As soon as the variable moves out of the scope of where it was defined, it is then deallocated from the memory.
- Values can be referenced and altered if we adhere to the rules for copying, moving, immutable borrowing, and mutable borrowing.

Knowing the rules is one thing, but to practically work with the rules in Rust code, we need to understand copying, moving, and borrowing in more detail.

Copying variables

Copying occurs when a value is copied. Once it has been copied, the new variable owns the value, while the existing variable also continues to own its value.

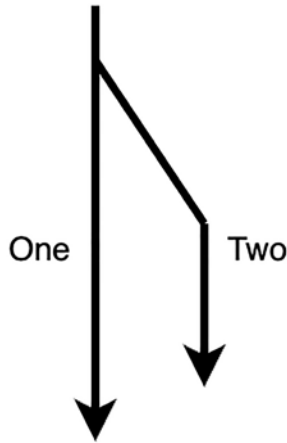


Figure 1.4 – Variable copy path

In *Figure 1.4*, we can see that the path of **One** is still solid, which denotes that it has not been interrupted and can be handled as if the copy did not happen. Path **Two** is merely a copy, and there is also no difference in the way in which it can be utilized, as if it were self-defined.

Note that if the variable has a copy trait, then the variable will automatically be copied without us having to write any extra code, as seen in the following code:

```
let one: i8 = 10;
let two: i8 = one + 5;
println!("{}", one);
println!("{}", two);
```

Running the preceding code will give us the following printout:

```
10
15
```

In the preceding example, we appreciate that the very fact that variables `one` and `two` can be printed indicates that `one` has been copied for `two` to utilize. To test this, we can test our example with strings using the following code:

```
let one = "one".to_string();
let two = one;
println!("{}", one);
println!("{}", two);
```

Running this code will result in the following error:

```
move occurs because `one` has type `String`, which does not implement the
`Copy` trait
```

Because strings do not implement the Copy trait, the code does not work, as one was moved to two.



It is only natural to wonder why strings do not have the Copy trait. This is because the string is a pointer to a string literal. If we were to copy strings, we would have multiple unconstrained pointers to the same string literal data, which would be dangerous.

However, the code will run if we get rid of `println!("{}", one);`. This brings us to the next concept that we must understand, moving.

Moving variables

Moving refers to when a value is moved from one variable to another. However, unlike copying, the original variable no longer owns the value.

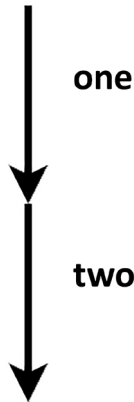


Figure 1.5 – Variable move path

From what we can see in *Figure 1.5*, one can no longer be accessed once it's moved to two. To really establish what is going on here and how strings are affected, we can set up some code designed to fail, as follows:

```
let one: String = String::from("one");
let two: String = one + " two";
println!("{}", two);
println!("{}", one);
```

Running the preceding code gives the following error:

```
let one: String = String::from("one");
    --- move occurs because `one` has type
    `String`, which does not implement the
    `Copy` trait
let two: String = one + " two";
    ----- `one` moved due to usage in operator
println!("{}", two);
println!("{}", one);
    ^^^ value borrowed here after move
```

As we can see, the compiler has been helpful here. It shows us where the string was moved to and where the value of that string is borrowed. So we can make the code run instantly by merely removing the line `println!("{}", one);`. However, we want to be able to use that print function at the bottom of the preceding code block. We should not have to constrain the functionality of the code due to the rules implemented by Rust. We can solve this by using the `to_owned` function with the following code:

```
let two: String = one.to_owned() + " two";
```

The `to_owned` function is available because strings implement the `ToOwned` trait. We will cover traits later in the chapter, so carry on reading even if you do not know what this means yet. We could also have used `clone` on the string. We must note that `to_owned` is a generalized implementation of `clone`. However, it does not really matter which approach we use.

Because of this, we can explore the move concept using strings. If we force our string outside of the scope with a function, we can see how this affects our move. This can be done with the following code:

```
fn print(value: String) {
    println!("{}", value);
}
fn main() {
    let one = "one".to_string();
    print(one);
    println!("{}", one);
}
```

If we run the preceding code, we will get an error, stating that the `print` function moved the `one` value. As a result, the `println!("{}", one);` line borrows `one` after it is moved into the `print` function. The key part of this message is the word *borrow*. To understand what is going on, we need to explore the concept of immutable borrowing.

Immutable borrowing of variables

An immutable borrow occurs when a variable can be referenced by another variable without having to clone or copy it. This essentially solves our problem. If the borrowed variable falls out of scope, then it is not deallocated from the memory and the original reference to the value can still be used.

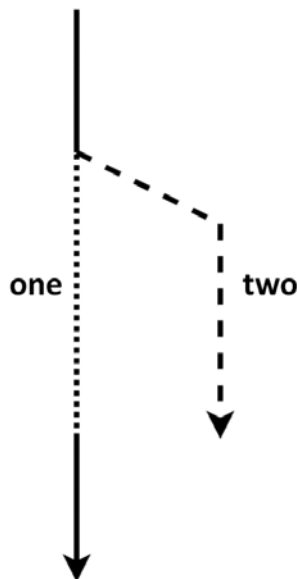


Figure 1.6 – Immutable borrow path

We can see in *Figure 1.6* that *two* borrows the value from *one*. It must be noted that when *one* is borrowed from, *one* is locked and cannot be accessed until the borrow is finished.

To perform a borrow operation, we apply a prefix with `&`. This can be demonstrated with the following code:

```
fn print(value: &String) {  
    println!("{}", value);  
}  
  
fn main() {  
    let one = "one".to_string();  
    print(&one);  
    println!("{}", one);  
}
```

In the preceding code, we can see that our immutable borrow enables us to pass a string into the `print` function and still print it afterward. This can be confirmed with the following `println!`:

```
one  
one
```

From what we see in our code, the immutable borrow that we performed can be demonstrated in *Figure 1.7*.

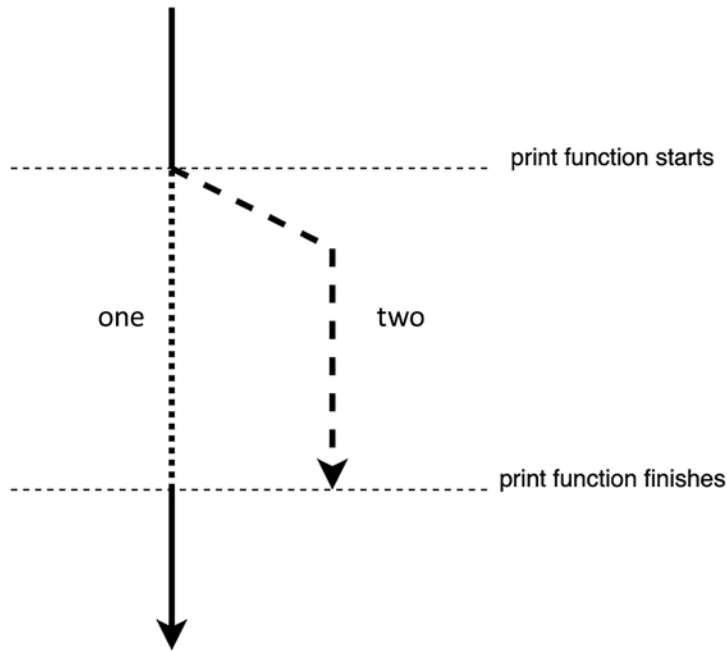


Figure 1.7 – Immutable borrow in relation to the print function

In the preceding figure, we can see that `one` is not available when the `print` function is running. We can demonstrate this with the following code:

```
fn print_two(value: &String, value_two: String) {
    println!("{}", value);
    println!("{}", value_two);
}

fn main() {
    let one = "one".to_string();
    print_two(&one, one);
    println!("{}", one);
}
```

If we run the preceding code, we will get the following error:

```
Print_two(&one, one);
----- ^^^ move out of `one` occurs here
|       |
|       borrow of `one` occurs here
borrow later used by call
```

We can see that we cannot utilize one, even though it is utilized in the print function after &one. This is because the lifetime of &one is throughout the entire lifetime of the print function. Thus, we can conclude that *Figure 1.7* is correct. However, we can run one more experiment. We can change value_one to a borrow to see what happens with the following code:

```
fn print_two(value: &String, value_two: &String) {
    println!("{}", value);
    println!("{}", value_two);
}

fn main() {
    let one = "one".to_string();
    print_two(&one, &one);
    println!("{}", one);
}
```

In the preceding code, we can see that we do two immutable borrows of one, and the code then runs. This highlights an important fact: we can make as many immutable borrows as we like. This is safe because both borrows cannot mutate the value; therefore, we are confident that we know what the borrow points to. However, what happens if the borrow is mutable? To understand, we must explore mutable borrows.

Mutable borrowing of variables

A mutable borrow is essentially the same as an immutable borrow, except that the borrow is mutable and that we cannot have more than one mutable borrow at the same time. To understand this, let's say that we have a mutable reference, and this mutable reference updates the vector to the point where the memory gets reallocated. Therefore, the immutable reference will point to memory that is no longer allocated to the vector. This can lead to undefined behavior.

With mutable borrows, we can change the borrowed value. To demonstrate this, we can create a print statement that will alter the borrowed value before printing it. We then print it in the main function to establish that the value has been changed with the following code:

```
fn print(value: &mut i8) {
    *value += 1;
    println!("In function the value is: {}", value);
}

fn main() {
    let mut one: i8 = 5;
    print(&mut one);
    println!("In main the value is: {}", one);
}
```

Running the preceding code will give us the following printout:

```
In function the value is: 6
In main the value is: 6
```

The preceding output proves that one is 6 even after the lifetime of the mutable reference in the print function has expired. We can see that in the print function, we update the value of one using a `*` operator. This is called a dereference operator. This dereference operator exposes the underlying value so that it can operate. This all seems straightforward, but is it exactly like our immutable references? If we remember, we could have multiple immutable references. We can put this to the test with the following code:

```
fn print_two(value: &mut i8, value_two: &mut i8) {
    *value += 1;
    println!("In function the value is: {}", value);
    *value_two += 1;
}
fn main() {
    let mut one: i8 = 5;
    print_two(&mut one, &mut one);
    println!("In main the value is: {}", one);
}
```

In the preceding code, we can see that we make two mutable references and pass them through, just like in the previous section but with immutable references. However, running it gives us the following error:

```
error[E0499]: cannot borrow `one` as mutable more than once at a time
```

Through this example, we can confirm that we cannot have more than one mutable reference at a time. This prevents data races and has given Rust the *fearless concurrency* tag.

Controlling variable ownership cheatsheet



- **Error prevention:** Rust prevents dangling pointers, double frees, segmentation faults, and buffer overruns.
- **Copying:** Types with the Copy trait can be duplicated without explicit code.
- **Moving:** Values are transferred and the original variable loses ownership.
- **Immutable borrowing:**
 - Allows you to reference a variable without transferring ownership
 - Multiple immutable borrows are permitted, since they do not mutate the value
- **Mutable borrowing:** Only one mutable borrow at a time is allowed, enabling safe value alterations.
- **Dereference operator:** Used to access the value referenced by a pointer.

With what we have covered here, we can now be productive when the compiler is combined with the borrow checker. However, we have only touched on the concepts of scope and lifetimes. The use of them has been intuitive, but like the rules around borrowing, we need to dive into scopes and then lifetimes in more detail.

Scopes

To understand scopes, let us go back to how we declare variables. You will have noticed that when we declare a new variable, we use `let`. When we do, that variable is the only one that owns the resource. Therefore, if the value is moved or reassigned, then the initial variable no longer owns the value. When a variable is moved, it is essentially moved into another scope. Variables declared in an outer scope can be referenced in an inner scope, but a variable declared in an inner scope cannot be accessed in the inner scope once the inner scope has expired. We can break down some code into scopes, as shown in the following diagram:

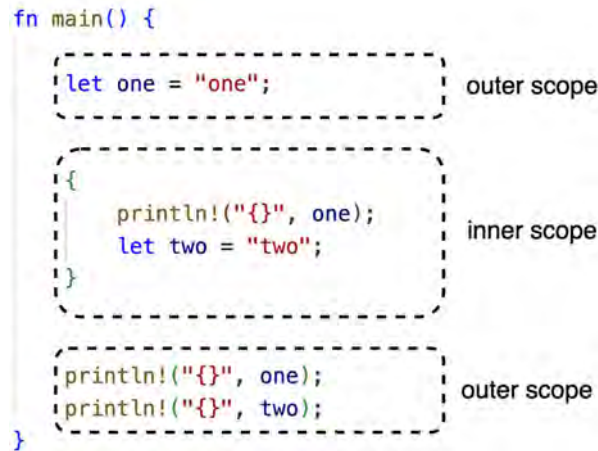


Figure 1.8 – Basic Rust code broken into scopes

Figure 1.8 shows us that we can create an inner scope by merely using curly brackets. Applying what we just learned about scopes to Figure 1.8, can you work out whether it would compile? If it would crash, how would it?

If you guessed that it would result in a compiler error, then you are correct. Running the code would result in the following error:

```
println!("{}", two);
    ^^^ not found in this scope
```

Because `one` is defined in the inner scope, we will not be able to reference it in the outer scope. We can solve this problem by declaring the variable in the outer scope but assigning the value in the inner scope, with the following code:

```
fn main() {
    let one = &"one";
    let two: &str;
```

```

{
    println!("{}", one);
    two = &"two";
}
println!("{}", one);
println!("{}", two);
}

```

In the preceding code, we can see that we do not use `let` when assigning the value because we have already declared the variable in the outer scope. Running the preceding code gives us the following printout:

```

one
one
two

```

We also must remember that if we move a variable into a function, then the variable gets destroyed once the scope of the function finishes. We cannot access the variable after the execution of the function, even though we declared the variable before the execution of the function. This is because once the variable has been moved into the function, it is no longer in the original scope. It has been moved. And because the variable has been moved to that scope, it is then bound to its lifetime. This brings us to our next section, lifetimes.

Running through lifetimes

Understanding lifetimes will wrap up our exploration of borrowing rules and scopes. We can explore the effect of lifetimes with the following code:

```

fn main() {
    let one: &i8;
    {
        let two: i8 = 2;
        one = &two;
    } // -----> two lifetime stops here
    println!("r: {}", one);
}

```

With the preceding code, we declare `one` before the inner scope starts. However, we assign it to have a reference of `two`. This `two` only has the lifetime of the inner scope, so the lifetime dies before we try and print it out. This is established with the following error:

```

one = &two;    }    println!("r: {}", one);
    ^^^^      -          --- borrow later used here
    |         |
    |         | `two` dropped here while still borrowed
    |         | borrowed value does not live long enough

```

two is dropped when the lifetime of two has finished. With this, we can state that the lifetimes of one and two are not equal.

While it is great that this is flagged when compiling, Rust does not stop here. This concept also applies to functions. Let's say that we build a function that references two integers, compares them, and returns the highest integer reference. The function is an isolated piece of code. In this function, we can denote the lifetimes of the two integers. This is done by using the ' prefix, which is a lifetime notation. The names of the notations can be anything you come up with, but it is conventional to use a, b, c, and so on. We can explore this by creating a simple function that takes in two integers and returns the highest one, with the following code:

```
fn get_highest<'a>(first_number: &'a i8, second_number: &'a
    i8) -> &'a i8 {
    if first_number > second_number {
        first_number
    } else {
        second_number
    }
}

fn main() {
    let one: i8 = 1;
    let outcome: &i8;
    {
        let two: i8 = 2;
        let outcome: &i8 = get_highest(&one, &two);
    }
    println!("{}", outcome);
}
```

As we can see, the first and second lifetimes have the same notation of a. They both must be present for the duration of the function. Note that the function returns an i8 integer with the lifetime of a. If we were to try and use lifetime notation on function parameters without a borrow, we would get some very confusing errors. In short, it is not possible to use lifetime notation without a borrow. This is because if we do not use a borrow, the value passed into the function is moved into the function. Therefore, its lifetime is the lifetime of the function. This seems straightforward; however, when we run it, we get the following error:

```
println!("{}", outcome);}
^^^^^^ use of possibly-uninitialized `outcome`
```

The error occurs because all the lifetimes of the parameters passed into the function and the returned integer are all the same. Therefore, the compiler does not know what could be returned. As a result, two could be returned. If two is returned, then the result of the function will not live long enough to be printed. However, if one is returned, then it will. Therefore, there is a possibility of not having a value to print after the inner scope is executed. In a dynamic language, we would be able to run code that runs the risk of referencing variables that have not been initialized yet. However, with Rust, we can see that if there is a possibility of an error like this, it will not compile.

In the short term, it might seem like Rust takes longer to code, but as the project progresses, this strictness will save a lot of time by preventing silent bugs. In conclusion of our error, there is no way of solving our problem with the exact function and main layout that we have. We would either move our printing of the outcome into the inner scope or clone the integers and pass them into the function.

We can create one more function to explore functions with different lifetime parameters. This time, we will create a `filter` function. If the first number is lower than the second number, we will then return `0`; otherwise, we will return the first number. This can be achieved with the following code:

```
fn filter<'a, 'b>(first_number: &'a i8, second_number: &'b
    i8) -> &'a i8 {
    if first_number < second_number {
        &0
    } else {
        first_number
    }
}

fn main() {
    let one: i8 = 1;
    let outcome: &i8;
    {
        let two: i8 = 2;
        outcome = filter(&one, &two);
    }
    println!("{}", outcome);
}
```

The preceding code works because we know the lifetimes are different. The first parameter has the same lifetime as the returned integer. If we were to implement `filter(&two, &one)` instead, we would get an error stating that the outcome does not live long enough to be printed.



Variable scope and lifetimes cheatsheet

- **Variable ownership:** Variables declared with `let` own their values; reassignment moves ownership.
- **Scope rules:** Outer scope variables are accessible in inner scopes, but not vice versa.
- **Function scope:** Variables moved into a function are destroyed once the function scope ends.
- **Lifetime mismatch:** Variables must outlive the references assigned to them.
- **Function lifetimes:** Lifetime annotations ensure that function parameters and return values live long enough.

We have now covered all that we need to know for now to write productive code in Rust without the borrow checker getting in our way. We now need to move on to creating bigger building blocks for our programs, allowing us to focus on tackling the complex problems we want to solve with code. We will start this with a versatile building block of programs, **structs**.

Building Structs

In modern high-level dynamic languages, objects have been the bedrock for building big applications and solving complex problems, and for good reason. Objects enable us to encapsulate data, functionality, and behavior. In Rust, we do not have objects. However, we do have structs that can hold data in fields. We can then manage the functionality of these structs and group them together with traits. This is a powerful approach, and it gives us the benefits of objects without the high coupling, as highlighted in the following figure:

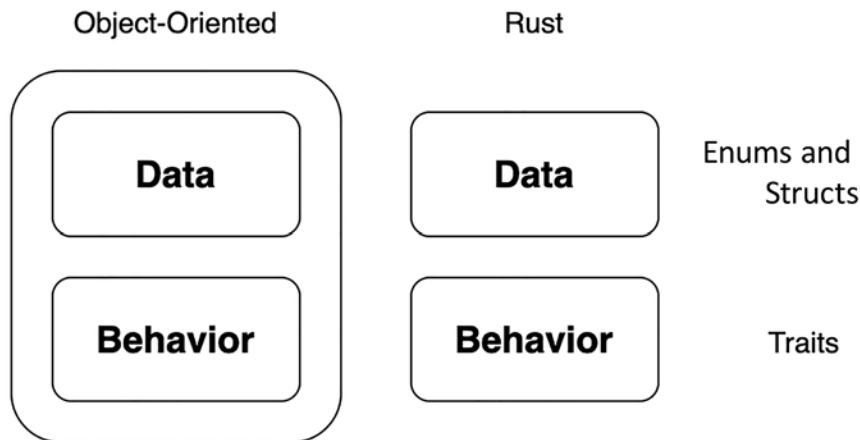


Figure 1.9 – Difference between Rust structs and objects

We will start with something basic by creating a Human struct with the following code:

```
#[derive(Debug)]
struct Human<'a> {
    name: &'a str,
    age: i8,
    current_thought: &'a str
}
```

In the preceding code, we can see that our string literal fields have the same lifetime as the struct itself. We have also applied the Debug trait to the Human struct, so we can print it out and see everything. We can then create the Human struct and print it out, using the following code:

```
fn main() {
    let developer = Human{
        name: "Maxwell Flitton",
        age: 34,
        current_thought: "nothing"
    };
    println!("{:?}", developer);
    println!("{}", developer.name);
}
```

Running the preceding code will give us the following printout:

```
Human { name: "Maxwell Flitton", age: 34, current_thought: "nothing" }
Maxwell Flitton
```

We can see that our fields are what we expect. However, we can change our string slice fields to strings to get rid of lifetime parameters. We may also want to add another field where we can reference another Human struct under a friend field. However, we may also have no friends. We can account for this by creating an enum that is either a friend or not and assigning it to a friend field, as seen in the following code:

```
#[derive(Debug)]
enum Friend {
    HUMAN(Human),
    NIL
}

#[derive(Debug)]
struct Human {
    name: String,
    age: i8,
    current_thought: String,
    friend: Friend
}
```

We can then define the `Human` struct initially with no friends, just to see if it works with the following code:

```
let developer = Human{
    name: "Maxwell Flitton".to_string(),
    age: 32,
    current_thought: "nothing".to_string(),
    friend: Friend::NIL
};
```

However, when we run the compiler, it does not work. I would like to think this is because the compiler cannot believe that I have no friends. But alas, it has to do with the compiler not knowing how much memory to allocate for this declaration. This is shown through the following error code:

```
enum Friend {    HUMAN(Human),    NIL}#[derive(Debug)]
^^^^^^^^^^^^          ----- recursive without indirection
|
recursive type has infinite size
```

Because of the enum, theoretically, the memory needed to store this variable could be infinite. One `Human` struct could reference another `Human` struct as a `friend` field, which could in turn reference another `Human` struct, resulting in a potentially infinite number of `Human` structs being linked together through the `friend` field. We can solve this problem with pointers. Instead of storing all the data of a `Human` struct in the `friend` field, we store a memory address that we know has a maximum value because it's a standard integer. This memory address points to where another `Human` struct is stored in the memory. As a result, the program knows exactly how much memory to allocate when it crosses a `Human` struct, irrespective of whether the `Human` struct has a `friend` field or not. This can be achieved by using a `Box` struct, which is essentially a smart pointer for our enum, with the following code:

```
#[derive(Debug)]
enum Friend {
    HUMAN(Box<Human>),
    NIL
}
```

So now, our enum states whether the friend exists or not, and if so, it has a memory address if we need to extract information about this friend. We can achieve this with the following code:

```
fn main() {
    let another_developer = Human{
        name: "Caroline Morton".to_string(),
        age: 30,
        current_thought: "I need to code!!".to_string(),
        friend: Friend::NIL
    };
}
```

```

let developer = Human{
    name: "Maxwell Flitton".to_string(),
    age: 34,
    current_thought: "nothing".to_string(),
    friend: Friend::HUMAN(Box::new(another_developer))
};
match &developer.friend {
    Friend::HUMAN(data) => {
        println!("{}", data.name);
    },
    Friend::NIL => {}
}
}

```

In the preceding code, we can see that we have created one Human struct, and then another Human struct with a reference to the first Human struct as a friend field. We then access the second Human struct's friend through the friend field. Remember, we must handle both possibilities, as the friend field could be a nil value.

While it is exciting that friends can be made, if we take a step back, we can see that there is a lot of code written for each human we create. This is not helpful if we must create a lot of humans in a program. We can reduce this by implementing some functionality for our struct. We will essentially create a constructor for the struct with extra functions, so we can add optional values if we want. We will also make the thought field optional. So a basic struct with a constructor populating only the most essential fields can be achieved with the following code:

```

#[derive(Debug)]
struct Human {
    name: String,
    age: i8,
    current_thought: Option<String>,
    friend: Friend
}
impl Human {
    fn new(name: &str, age: i8) -> Human {
        return Human{
            name: name.to_string(),
            age: age,
            current_thought: None,
            friend: Friend::NIL
        }
    }
}
}

```


We can see that structs combined with enums and functions that have been implemented with these structs can be powerful building blocks. We can define fields and functionality with only a small amount of code if we have defined our structs well. However, writing the same functionality for multiple structs can be time-consuming and result in a lot of repeated code. If you have worked with objects before, you may have utilized inheritance for that.

Rust goes one better. It has traits, which we will explore in the next chapter.

Building structs cheatsheet



- **No objects:** Rust uses structs instead of objects to encapsulate data and functionality.
- **Struct definition:** Structs hold data in fields (e.g., `struct Human { name: String, age: i8 }`).
- **String literals in structs:** String literals in structs require lifetime parameters.
- **Box for recursive types:** Use `Box` to handle recursive types, preventing infinite memory allocation.
- **Printing structs:** Implement the `Debug` trait to print struct details.
- **Constructors in structs:** Implement a new function in the `impl` block for struct initialization.
- **Function calling:** Use `::` for static methods and `.` for instance methods. Add methods to the `impl` block to chain function calls (e.g., `with_thought`).
- **Traits for reusability:** Use traits to avoid repeated code and enhance functionality across multiple structs.

Summary

With Rust, we have seen that there are some traps when coming from a dynamic programming language background. However, with a little bit of knowledge of referencing and basic memory management, we can avoid common pitfalls and write safe, performant code quickly that can handle errors.

In this chapter, we also covered the concepts of borrowing and referencing in Rust. While adhering to borrowing rules requires more effort than coding in a garbage-collected language, we have a deeper understanding of how variables are placed in memory. This deeper understanding is safer as we know exactly what data we point to. For example, if we were using a language like Python and created an instance of an object when we passed it into two dictionaries (Python's version of a hash map), then if we updated one instance, the other value in the hash map would also be updated because it is a shared reference to the same memory address. The developer, however, may never know. In Rust, the borrow-checking rules make what is going on explicit. Scopes also make it explicit when a variable is dropped from memory.

We now know enough Rust to get started. In the next chapter, we will cover useful patterns for web programming and metaprogramming.

Questions

1. What is the difference between a `str` and a `String`?
2. Why can't string slices be passed into a function (string slice meaning `str` as opposed to `&str`)?
3. How do we access the data belonging to a key in a `HashMap`?
4. When a function results in an error, can we handle other processes, or will the error crash the program instantly?
5. Why does Rust only allow one mutable borrow at a point in time?
6. When would we need to define two different lifetimes in a function?
7. How can structs link to the same struct via one of their fields?
8. How can we add extra functionality to a struct where the functionality can also be implemented by other structs?

Answers

1. A `String` is a fixed-size reference stored in the stack that points to string-type data on the heap. A `str` is an immutable sequence of bytes stored somewhere in memory. Because the size of the `str` is unknown, it can only be handled by a pointer, `&str`.
2. Since we do not know the size of the string slice at compile time, we cannot allocate the correct amount of memory for it. Strings, on the other hand, have a fixed-size reference stored on the stack that points to the string slice on the heap. Because we know this fixed size of the string reference, we can allocate the correct amount of memory and pass it through to a function.
3. We use the `HashMap`'s `get` function. However, we must remember that the `get` function merely returns an `Option` struct. If we are confident that there is something there or we want the program to crash if nothing is found, we can directly `unwrap` it. However, if we don't want that, we can use a `match` statement and handle the `Some` and `None` output as we wish.
4. No, results must be unwrapped before exposing the error. A simple `match` statement can handle unwrapping the result and managing the error as we see fit.
5. Rust only allows one mutable borrow to prevent memory unsafety. In Goregaokar's blog (see the *Further reading* section), the example of an enum is used to illustrate this. If an enum supports two different data types (`String` and `i64`), a mutable reference of the string variant of the enum is made, and then another reference is made, the mutable reference can change the data, and then the second reference would still reference the string variant of the enum. The second reference would then try to dereference the string variant of the enum, potentially causing a segmentation fault. Elaboration on this example and others is provided in the *Further reading* section.
6. We would need to define two different lifetimes when the result of a function relies on one of the lifetimes and the result of the function is needed outside of the scope of where it is called.
7. If a struct references itself in one of its fields, the size could be infinite, as it could continue to reference itself continuously. To prevent this, we can wrap the reference to the struct in the field in a `Box` struct.

8. We can define a trait that has multiple functions and this trait with these functions can be applied to multiple different structs.

Further reading

1. *Hands-On Functional Programming in Rust* (2018) Andrew Johnson: *Chapter 1, Generics and Structs*
2. *Mastering Rust* (2019) Rahul Sharma and Vesa Kaihlavirta: *Chapter 1, A Tour of the Language*
3. *The Problem With Single-threaded Shared Mutability* (2015) by Manish Goregaokar: <https://manishearth.github.io/blog/2015/05/17/the-problem-with-shared-mutability/>
4. Rust Project Developers, 2024. *The Rust Programming Language*. [online] Available at: <https://doc.rust-lang.org/book/> [Accessed 21 June 2024]
5. Latimer, N., 2020. *Programming Rust*. [online] Available at: <https://doc.rust-lang.org/stable/rust-by-example/> [Accessed 21 June 2024].

Get this book's PDF copy, code bundle, and more

Scan the QR code (or go to packtpub.com/unlock). Search for this book by name, confirm the edition, and then follow the steps on the page.



UNLOCK NOW

Note: Have your invoice handy. Purchases made directly from the Packt website don't require an invoice.

2

Useful Rust Patterns for Web Programming

Knowing the syntax and borrowing rules of Rust can get us building programs. However, unlike dynamic programming languages, Rust has a strict type system. If you do not know how to get creative with traits, you can end up creating a lot of excessive code to solve problems. In this chapter, we will cover how to enforce certain parameter checks with traits, increasing the flexibility of the parameters of functions that accept structs. We will also explore metaprogramming with macros to reduce the amount of repetitive code we have to write. These macros will also enable us to simplify and effectively communicate with other developers what our code does. We will also utilize the compiler to check the state of structs as they evolve, improving the safety of the program.

In this chapter, we will cover the following topics:

- Verifying with traits
- Metaprogramming with macros
- Mapping messages with macros
- Configuring our functions with traits
- Checking struct state with the compiler

Once we have covered the main concepts in this chapter, you will be able to achieve more flexibility with your code, without having to write loads of excessive code.



Macros and traits empower Rust to be more concise with web programming. For instance, if we implement a `Serialize` trait for a struct, we can slot this struct into database schemas, or directly use the same struct to extract a JSON body from a request.

With traits and macros, we can write highly customizable web API endpoints with minimal code. We can also build our own traits for the compiler to verify restrictions that we want to impose.

Technical requirements

You can download the example project and code for this book by following the instructions in the [Download the example code files](#) section in the *Preface* of this book.

This chapter's code files are included in the downloadable code bundle.

Verifying with traits

Enums can empower structs so that they can handle multiple types. This can also be translated for any type of function or data structure. However, this can lead to a lot of repetition. Take, for instance, a `User` struct. Users have a core set of values, such as a username and password; however, they could also have extra functionality based on roles. With users, we must check roles before firing certain processes based on the **traits** that the user has implemented.

Through the creation of a simple program that defines users and their roles, we can demonstrate how to wrap up structs with traits:

1. We define our users with the following code:

```
struct AdminUser {
    username: String,
    password: String
}
struct User {
    username: String,
    password: String
}
```

We can see in the preceding code that the `User` and `AdminUser` structs have the same fields. For this exercise, we just need two different structs to demonstrate the effect that traits have on them. Now that our structs are defined, we can move on to our next step, which is creating the traits.

2. The traits that we have for this example are `create`, `edit`, and `delete`. We will be implementing these traits in our structs, using them to assign permissions to our users. We can create these three traits with the following code:

```
trait CanEdit {
    fn edit(&self) {
        println!("admin is editing");
    }
}
trait CanCreate {
    fn create(&self) {
        println!("admin is creating");
    }
}
trait CanDelete {
```

```
fn delete(&self) {
    println!("admin is deleting");
}
}
```

We can see that the functions for the traits only take in `self`. We also reference `admin` in the trait functions. This is because the `admin` can have access to all these traits. If other roles implement these traits, they can overwrite the functions that they need to. We will see this in action later on.

We cannot make any references to the fields in the functions to `self` as we do not know what structs will be implemented. However, as mentioned earlier, we can override functions when we implement the trait in the struct if needed.

Getters and setters

We could use traits to allow getting and setting of fields in our struct. This is because the fields in our structs are private, meaning that they cannot be directly accessed from outside of the struct. Setter and getter traits would take the following form:



```
trait GetUsername {
    fn get_username(&self) -> &str;
}

trait SetUsername {
    fn set_username(&mut self, username: String);
}
```

The setter has a mutable reference to `self`, so the struct is not consumed by the setter function. The mutable reference enables us to alter the state of the struct, which is what setters do.

If we are to return `self`, we will need to wrap it in a `Box` struct, as the compiler will not know the size of the struct being returned. We also must note that the signature of the function (input parameters and return values) must be the same as the original declaration of the trait if we overwrite the function for a struct.

Now that we have defined the traits, we can move on to the next step of implementing the traits to define roles for our user.

3. With our roles, we can make our `admin` have every permission and our `user` only the edit permission. This can be achieved with the following code:

```
impl CanDelete for AdminUser {}
impl CanCreate for AdminUser {}
impl CanEdit for AdminUser {}

impl CanEdit for User {
```

```

    fn edit(&self) {
        println!("A standard user {} is editing",
                self.username);
    }
}

```

From our previous step, we can remember that all the functions already worked for the admin by printing out that the admin is doing the action. Therefore, we do not have to do anything for the implementation of the traits for the admin. We can also see that we can implement multiple traits for a single struct. This adds a lot of flexibility. In our user implementation of the `CanEdit` trait, we have overwritten the `edit` function so that we can have the correct statement printed out.

Now that we have implemented the traits, our user structs have permission in the code to enter scopes that require those traits. We can now build the functions for using these traits.

4. We could utilize the functions in the traits by directly running them in the `main` function on the structs that have implemented them. However, if we do this, we will not see their true power in this exercise. We may also want this standard functionality throughout our program in the future when we span multiple files.
5. The following code shows how we create functions that utilize the traits:

```

fn create<T: CanCreate>(user: &T) -> () {
    user.create();
}

fn edit<T: CanEdit>(user: &T) -> () {
    user.edit();
}

fn delete<T: CanDelete>(user: &T) -> () {
    user.delete();
}

```

The preceding notation is like the lifetime annotation. We use angle brackets before the input definitions to define the trait we want to accept at `T`. We then state that we will accept a borrowed struct that has implemented the trait as `&T`. This means that any struct that implements that specific trait can pass through the function; we are depending on abstractions rather than concrete implementations. Because we know what the trait can do, we can then use the functions associated with the trait defined in the parameter. However, because we do not know what struct is going to be passed through, we cannot utilize specific fields unless we also require specific getter traits that we create ourselves. Remember, we can also overwrite a trait function to utilize struct fields when we implement the trait for the struct, however. This might seem rigid, but the process enforces good, isolated, decoupled coding that is safe. For instance, let us say we remove a function from a trait or remove a trait from a struct. The compiler would refuse to compile until all the effects of this change were complete. Thus, we can see that, especially for big systems, Rust is safe and can save time by reducing the risk of silent bugs.

Now that we have defined the functions, we can use them in the main function.

6. We can test to see if all the traits work with the following code:

```
fn main() {  
    let admin = AdminUser{  
        username: "admin".to_string(),  
        password: "password".to_string()  
    };  
    let user = User{  
        username: "user".to_string(),  
        password: "password".to_string()  
    };  
    create(&admin);  
    edit(&admin);  
    edit(&user);  
    delete(&admin);  
}
```

We can see that the functions that accept traits are used just like any other function.

Running the entire program will give us the following printout:

```
admin is creating  
admin is editing  
A standard user is editing  
admin is deleting
```

In our output, we can see that the overriding of the edit function for the User struct works. We can also specify multiple trait bounds. For instance, we could have the following function definition:

```
fn cache<T: CanCreate + CanDelete>(user: &T) -> () {  
    . . .  
}
```

Here, we are saying that the user must have the permission to create and delete entries.

This leads me to my opinion that traits are more powerful than object inheritance. To qualify this, let us think about building a game. In one function, we deal damage from one player to another. In a traditional class-based programming language, we could build a class that is a player character. This class has the methods take and deal damage. We then build out different types of characters, such as Orc, Elf, Human, etc., that all inherit this class. This all seems reasonable, and we start coding away. However, what about buildings? Buildings could theoretically take damage, but they are not player characters. Also, buildings by themselves cannot really cause damage. We are now stuck rewriting our structure or writing new functions that accommodate buildings and having from if else if conditional logic on what function to call.

However, if we have a function where one parameter must have the `DealDamage` trait implemented, and the other parameter must have the `TakeDamage` trait implemented, our participants of this function can come and go with little friction. In my experience, developers who complain about Rust being a rigid language are not utilizing traits. I have found Rust to be more flexible than many object-oriented languages because of traits.

We have now learned enough about traits to be productive with web development. From here, traits get even more powerful, and we will be using them for some key parts of our web programming. For instance, several Rust web frameworks have traits that execute before the request is processed by the view/API endpoint. Implementing structs with these traits automatically loads the view function with the result of the `trait` function. This can be database connections, extraction of tokens from headers, or anything else we wish to work with.

Throughout the book, we will utilize traits to enable dependency injection into our API endpoints. This means we can mock things such as database connections or swap them out with just a few lines of code. With the strict compiler checks, we can make aggressive changes to our data access layers, deployment methods, and networking protocols, and sleep well at night. With Rust, I've been able to pivot approaches to my web apps at a quicker and safer pace than any dynamic programming language I have worked with.

While traits can define a common interface so that different structs can interact with our code, this can lead to repetitive code. This is where metaprogramming with macros comes in to reduce repetitive code.

Metaprogramming with macros

Metaprogramming can generally be described as a way in which a program can manipulate itself based on certain instructions.

Considering the strong typing Rust has, one of the simplest ways in which we can meta-program is by using generics. A classic example of demonstrating generics is through coordinates, as follows:

```
struct Coordinate <T> {  
    x: T,  
    y: T  
}  
  
fn main() {  
    let one = Coordinate{x: 50, y: 50};  
    let two = Coordinate{x: 500, y: 500};  
    let three = Coordinate{x: 5.6, y: 5.6};  
}
```

In the preceding snippet, the `Coordinate` struct is defined with a single generic parameter, `T`. This `T` is a placeholder that can represent any type. When we create an instance of `Coordinate`, Rust's compiler replaces `T` with specific types. In our example:

- `one` uses `T` as `i32`
- `three` uses `T` as `f64`

This flexibility allows the `Coordinate` struct to handle different types of numbers without needing implementations for each type.

We can increase the flexibility of our `Coordinate` struct by adding more generic parameters with the following code:

```
struct Coordinate <T, X> {  
    x: T,  
    y: X  
}  
  
fn main() {  
    let one = Coordinate{x: 50, y: 500};  
    let two = Coordinate{x: 5.6, y: 500};  
    let three = Coordinate{x: 5.6, y: 50};  
}
```

What is happening in the preceding code with generics is that the compiler is looking for all instances where the struct is used, creating structs with the types used when the compilation is running.

Enhancing generics with traits

Generics do not just stop at allowing different data types into a struct or function. We can also set trait requirements for generics as seen ahead:



```
struct DbCacheHandle <T: CanCreate, X: CanDelete> {  
    create_handle: T,  
    delete_handle: X  
}
```

Here we can see that we can house delete and create handles in a struct. This would enable us to implement database caching handles for a range of different caching approaches and database backends. It is flexible and safe!

Now that we have covered generics, we can move on to the main mechanism of metaprogramming, macros.

Macros enable us to write code that writes code at compilation time.



To compare this approach with other languages, Lisp, C, and C++ also have macros, while in Python, metaprogramming is achieved with the decorator and meta classes. C# has source generators.

In our Rust code, we have already been using macros in our print functions. The `!` notation at the end of the function denotes that this is a macro that's being called.

Defining our own macros is a blend of defining a function with reference to outside code with metaprogramming. This means that we write reusable code that generates code. To demonstrate this, we will define a macro that capitalizes a string with the following code:

```
macro_rules! capitalize {
    ($a: expr) => {
        let mut v: Vec<char> = $a.chars().collect();
        v[0] = v[0].to_uppercase().nth(0).unwrap();
        $a = v.into_iter().collect();
    }
}

fn main() {
    let mut x = String::from("test");
    capitalize!(x);
    println!("{}", x);
}
```

Instead of using the term `fn`, we use the `macro_rules!` definition. We then say that `$a` is the expression passed into the macro. We get the expression, convert it into a vector of chars, then make the first char uppercase, and then convert it back to a string. Note that we don't return anything in the `capitalize` macro, and when we call the macro, we don't assign a variable to it. However, when we print the `x` variable at the end, we can see that it is capitalized. This does not behave like an ordinary function. We also must note that we didn't define a type. Instead, we just said it was an expression; the macro still does checks via traits. Passing an integer into the macro creates the following error:

```
| capitalize!(32);
| ----- in this macro invocation
|
= help: the trait `std::iter::FromIterator<char>` is not implemented for
`{integer}`
```

These checks happen due to the macro instructing the compiler to write the Rust code. Once the compiler has written the Rust code from the macro instructions, the generated code goes through the same checks that the rest of the Rust code goes through. We refer to the generated code of the macro as the expansion of the macro.

To see how our macros expand, we can install the `expand` tool with the following command:

```
cargo install cargo-expand
```

We can then perform the `cargo expand` command on our project. We will see that the following `main` function:


```
fn main() {  
    let mut x = String::from("test");  
    capitalize!(x);  
    println!("{}", x);  
}
```

gets expanded to the following code:

```
fn main() {  
    let mut x = String::from("test");  
    let mut v: Vec<char> = x.chars().collect();  
    v[0] = v[0].to_uppercase().nth(0).unwrap();  
    x = v.into_iter().collect();  
    {  
        ::std::io::_print(format_args!("{}", x));  
    };  
}
```

Here, we can see that our `capitalize` macro has been replaced with the code that we wrote in the macro, and that our `println!` macro has also been expanded.

Lifetimes, blocks, literals, paths, metaprogramming, and more can also be passed instead of an expression. In web development, a lot of the macros are already defined in third-party packages. Because of this, we do not need to write macros ourselves to get a web app up and running. Instead, we will mainly be using derive macros out of the box. However, writing our own macros can be powerful in web programming, for example, when it comes to networking, where I have used my own macros to match messages sent over a socket to the correct function. We will explore this in the next section.

Mapping messages with macros

Throughout the book, we will be using a web framework to match HTTP requests to the right function to be handled. However, there are times when you want to accept a message over a TCP connection and match the handling function based on the message received. (It does not have to be via TCP; I have found this type of macro to be useful for module interfaces or receiving messages via a channel.) In this section, we will implement a simple macro that will reduce the amount of code we need to write when mapping a struct to a function.

We map our messages using macros by carrying out the following steps:

1. Defining our contracts and contract handler
2. Defining functions to handle contracts
3. Building a register contract macro
4. Using our macro in a program

We will start with step 1.

Defining our contracts and contract handler

To map messages, we initially must define the data contracts that we will be mapping to our functions. We do this with the following code:

```
#[derive(Debug)]
pub struct ContractOne {
    input_data: String,
    output_data: Option<Result<String, String>>
}
#[derive(Debug)]
pub struct ContractTwo {
    input_data: String,
    output_data: Option<Result<String, String>>
}
```

With these data contracts, we have an input field and an output field, which is populated when the handling function has finished handling the data contract. Sometimes, if there is a possibility that there might be an error, we can have our `output_data` field as an `Option<Result<T>>`. This means that we can send the contract back over the network, and the client can unpack the `output_data` field, which will have been populated with an error if an error has occurred, and have a choice on how to handle the error.

We now need to send one of these contracts over a channel, into a function, or over a network. However, we want the option of sending any contract we want. We can do this by wrapping the contracts in an enum like the following code:

```
#[derive(Debug)]
pub enum ContractHandler {
    ContractOne(ContractOne),
    ContractTwo(ContractTwo),
}
```

Our handler can now be sent over a network and can be one of two contracts. At this stage, we could manually write the logic to handle our contracts with a simple `match` statement, but this manual approach does not scale. Here, we move on to our next step of defining our contract functions before we can build a macro to handle these contracts at scale.

Defining functions to handle contracts

Now that we have our contracts wrapped in an enum that can be sent, we must focus on the handling of these contracts with functions. We have the following functions to handle our contracts:

```
fn handle_contract_one(mut contract: ContractOne) -> ContractOne {
    println!("{}", contract.input_data);
    contract.output_data = Some(Ok("Output Data".to_string()));
}
```

```

    contract
  }
  fn handle_contract_two(mut contract: ContractTwo) -> ContractTwo {
    println!("{}", contract.input_data);
    contract.output_data = Some(Ok("Output Data".to_string()));
    contract
  }

```

These functions are not exciting, but they do simulate a process where the contract is accepted, updated, and then returned.

Building a register contract macro

With our contracts and our handle functions in place, we can now focus on our macro that maps the contract to the handle function.

Our macro has the following signature:

```

#[macro_export]
macro_rules! register_contract_routes {
  (
    $handler_enum:ident,
    $fn_name:ident,
    $( $contract:ident => $handler_fn:path ),* ) => {
      . . .
    };
}

```

The signature is a little daunting, so we will focus on the hardest expression. Once we understand that, everything else will fall into place. To understand the line `$( $contract:ident => $handler_fn:path, *)`, we must break it down:

- `$( and ), *`: These delimiters indicate a repetition pattern. The `$(` starts the repetition, and `), *` means none or multiple expressions are separated by commas. This allows the macro to accept multiple `contract => handler_fn` pairs.
- `$contract:ident`: `$contract` is a metavariable. In macros, metavariables are placeholders that will be matched and substituted with actual code or identifiers when the macro is expanded. `:ident` specifies that `$contract` should match an identifier. An identifier in Rust is a name used for variables, functions, structs, etc.
- `=>`: This is a literal token that must appear exactly as is in the macro input. It separates the contract from the handler function path. Here, we are introducing a new syntax that Rust can handle. Because Rust has a strict compiler, we can fearlessly create our own syntax on the fly.
- `$handler_fn:path`: `$handler_fn` is another metavariable. `:path` specifies that `$handler_fn` should match a path. In Rust, a path can be a simple identifier (such as a function name) or a more complex qualified path (such as `module::submodule::function`).

Putting it all together, this line defines a macro rule that matches zero or more pairs of `contract => handler_fn`, allowing us to easily map contracts to their respective handler functions within the generated function.

For the other two inputs, `$handler_enum:indent` is the enum that houses the different data contracts, and `$fn_name:indent` is the name of the function we want generated for handling all the mapping, as we may want multiple different mappers. We do not want name clashes.

Inside our macro, we define our function, and loop through all our data contract and function mappings with the following code. Do not worry if you do not visualize the output; the generated code of this macro is provided after we call the macro:

```
pub fn $fn_name(received_msg: $handler_enum) -> $handler_enum {
    match received_msg {
        $(
            $handler_enum::$contract(inner) => {
                let executed_contract = $handler_fn(
                    inner
                );
                return $handler_enum::$contract(
                    executed_contract
                )
            }
        )*
    }
}
```

The `$(...)*` is the loop. We can see that we unwrap the data contract in the match statement, pass the unwrapped contract into the mapped function, and then wrap the response of that mapped function into our enum again and return it. This means that we do not have to write the match arm for every variant of the enum.

Using our macro in a program

We can now call our macro with our handler enum, contracts, and functions with the following code:

```
register_contract_routes!(
    ContractHandler,
    handle_contract,
    ContractOne => handle_contract_one,
    ContractTwo => handle_contract_two
);
```

Here, we can see that it is much clearer as to what is going on. It is also scalable and repeatable. Our code is also maintainable. If we want to update the way in which the handle function is called, we only must do that update once in the macro as opposed to repeating ourselves around the codebase.

The output of our macro looks like this:

```
pub fn handle_contract(received_msg: ContractHandler)
  -> ContractHandler {
  match received_msg {
    ContractHandler::ContractOne(inner) => {
      let executed_contract = handle_contract_one(
        inner
      );
      return ContractHandler::ContractOne(
        executed_contract
      );
    }
    ContractHandler::ContractTwo(inner) => {
      let executed_contract = handle_contract_two(
        inner
      );
      return ContractHandler::ContractTwo(
        executed_contract
      );
    }
  }
}
```

Here, we can see that we match the incoming message, and then pass it into the mapped function, getting the result contract with the result and returning it.

Finally, we can refine a contract, wrap it in the handle enum, and call our mapping function with the following code:

```
fn main() {
  let contract_one = ContractOne {
    input_data: "Contract One".to_string(),
    output_data: None
  };
  let outcome = handle_contract(
    ContractHandler::ContractOne(contract_one)
  );
  println!("{:?}", outcome);
}
```

Running this program will give us the following printout:

```
Contract One
ContractOne(ContractOne {
    input_data: "Contract One",
    output_data: Some(Ok("Output Data"))
})
```

And there we have it; we have effectively mapped our data contracts to functions using a macro! But let us not stop here. We can combine macros and traits to enable the flexible configuration of our functions that we are mapping to.

Configuring our functions with traits

In a Rust web program, we generally have a series of layers and APIs. These layers usually consist of a frontend, backend, and data access layer. With these layers, we can have multiple different frameworks and engines. Throughout the book, we will be building out our web application so we can swap these frameworks and engines out.

These frameworks should have minimal footprint on your code, with clear separation between the interfaces. With web frameworks, you still must run a server using the web framework. We can swap out different engines, frameworks, and crates by using traits with no reference to `self`.

To demonstrate how this works, we are going to add another endpoint and data contract that gets a user by name from the database. Our database could be any database. We do not want our code to commit to having a particular data storage engine to run; this would not be flexible. To make an interaction with a database flexible, we implement a trait that has a signature for the data transaction. We can then implement that trait for a range of different databases. We will explore this concept by implementing a trait for getting users. We start by defining the trait signature.

Defining the trait signature

We can start by defining our user struct with the following code:

```
#[derive(Debug)]
pub struct User {
    name: String,
    age: u32
}
```

We then define a trait that lays out the signature of getting users with the following code:

```
trait GetUsers {
    fn get_users() -> Vec<User>;
    fn get_user_by_name(name: &str) -> Option<User> {
        let users = Self::get_users();
        for user in users {
```

```
        if user.name == name {
            return Some(user);
        }
    }
    None
}
```

This is not an optimal implementation, as in normal database queries, we would perform the filter in the database and return the filtered data. However, for our example, this is easier to implement. We must note that the reference to `Self` in the `get_user_by_name` function is capitalized, meaning that we are referring to the struct implementing the trait, as opposed to an instance of that struct. Therefore, we do not need to create an instance of our struct to call the `get_user_by_name` function. We can now use our trait for a database handle.

Implementing our trait for the database handle

We can then implement our trait for a database engine as in the following example code:

```
pub struct PostgresDB;

impl GetUsers for PostgresDB {
    fn get_users() -> Vec<User> {
        vec![
            User {
                name: "John".to_string(),
                age: 30
            },
            User {
                name: "Jane".to_string(),
                age: 25
            },
        ]
    }
}
```

In a working application, we would be making a connection to a database and passing in a query. However, databases take a while to set up, and this chapter is about useful patterns. Don't worry, throughout the book, we will build out a working data access layer that initially uses files, and then migrates over to a fully working Postgres database running in Docker.

Now that our database handle works, we must define a contract that carries the input and output data needed for the database operation. This data contract takes the following definition:

```
#[derive(Debug)]
pub struct GetUserContract {
    pub name: String,
    pub user: Option<User>
}
```

We now have everything to define our function handle with the following code:

```
fn handle_get_user_by_name<T: GetUsers>(contract: GetUserContract)
-> GetUserContract {
    let user = T::get_user_by_name(&contract.name);
    GetUserContract {
        name: contract.name,
        user: user
    }
}
```

Here, we can see that the generic type `T` has the `GetUsers` trait bound. We do not make any reference to `T` in the parameters of the function; therefore, we do not need to pass in any instances of anything that has implemented the `GetUserContract` trait. We can now put it all together by calling our macro utilizing our trait.

Calling our macro with our trait

We then add our contract to our handler with the following code:

```
#[derive(Debug)]
pub enum ContractHandler {
    ContractOne(ContractOne),
    ContractTwo(ContractTwo),
    GetUserContract(GetUserContract)
}
```

And our macro call now looks like the following:

```
register_contract_routes!(
    ContractHandler,
    handle_contract,
    ContractOne => handle_contract_one,
    ContractTwo => handle_contract_two,
    GetUserContract => handle_get_user_by_name::<PostgresDB>
);
```


Here, we can see that we have slotted our Postgres handler into the mapping. We can test to see whether this works with the following code:

```
fn main() {
    . . .
    let get_user_contract = GetUserContract {
        name: "John".to_string(),
        user: None
    };
    let outcome = handle_contract(
        ContractHandler::GetUserContract(
            get_user_contract
        )
    );
    println!("{:?}", outcome);
}
```

Running our code again will give us the following printout:

```
Contract One
ContractOne(ContractOne {
  input_data: "Contract One",
  output_data: Some(Ok("Output Data"))
})
GetUserContract(
  GetUserContract {
    name: "John", user: Some(User {
      name: "John", age: 30
    })
  }
)
```

This gives us a lot of power. We can slot any struct into that handle function if the struct has implemented the `GetUsers` trait. This struct could be an HTTP request to another server, a file handle, an in-memory store, or another database. We will exploit this approach again throughout the book, as over multiple chapters, we will build out a data access layer that can support multiple different storage engines.



No matter what you are building or what language you are building it in, clear boundaries between your code and external dependencies such as databases, HTTP calls, and other I/O operations are a must. There is no benefit in embedding I/O operations into your code. For instance, at the time of writing this book, I am working for the Rust database, SurrealDB. It currently supports its own key-value store, RocksDB, memory, WASM, and TiKV. It would make our lives a lot harder if we did not have clear interfaces between these storage engines.

Clean interfaces are nice to work with and powerful. However, with the type checking of Rust, someone (not me) could argue that excessive interfaces could result in a lot of different structs. This is where the type state pattern comes in, where we get to check the struct state with the compiler.

Checking struct state with the compiler

There are times when the state of a struct and certain processes are just not appropriate anymore. A nice, clear example would be a database transaction. While a database transaction is in progress, certain processes are appropriate, such as adding another operation to the transaction. You may also want to commit your transaction or roll it back. However, once we have committed our transaction, we cannot roll it back or add another operation to the transaction; we need another transaction for this. Considering this, I would want my compiler to check and ensure that I am not passing in a transaction into certain functions once the transaction is no longer in progress. The compiler will not only give me instant feedback, but it will also be safer, as it could be that only certain edge cases could cause a bug, increasing the chance of the bug slipping through tests. We can start by defining a state with our struct.

Defining the state of a struct

To get this compiler checking, I need a state in my struct. I then need that state to change once certain processes have triggered. We can capture the different states of the transaction with the following code:

```
use std::marker::PhantomData;

struct InProgress;
struct Committed;
struct RolledBack;

struct Transaction<State> {
    id: u32,
    state: PhantomData<State>,
}
```

Here, we can see that we are creating a type of transaction based on one of the structs, which is the state of the transaction. `PhantomData` is a placeholder that does not actually hold any data, but the compiler can take note of the phantom data when compiling. We can rely on the compiler to check the logical flow of the code and how the state of `PhantomData` changes. We can then enact restrictions on the struct based on the state of the `PhantomData` field. We get to see this approach in action when we implement some functions for the `Transaction` struct with the following code:

```
impl Transaction<InProgress> {
    fn new(id: u32) -> Self {
        Transaction {
            id,
            state: PhantomData,
        }
    }
}
```

```

    }
    fn commit(self) -> Transaction<Committed> {
        . . .
    }
    fn rollback(self) -> Transaction<RolledBack> {
        . . .
    }
}

```

Now that we have defined the outline of our state, we can change the state of the struct.

Changing the state of a struct

For our commit function, we have the following code:

```

fn commit(self) -> Transaction<Committed> {
    println!("Transaction {} committed.", self.id);
    Transaction {
        id: self.id,
        state: PhantomData,
    }
}

```

Here, we can see that `Transaction<InProgress>` is consumed, and `Transaction<Committed>` is returned.

For our rollback function, we have the following definition:

```

fn rollback(self) -> Transaction<RolledBack> {
    println!("Transaction {} rolled back.", self.id);
    Transaction {
        id: self.id,
        state: PhantomData,
    }
}

```

With the rollback function, we also consume `Transaction<InProgress>` but return a `Transaction<RolledBack>` transaction. So, now we are ready to have a function that only accepts a transaction that is in progress with the following code:

```

fn process_in_progress_transaction(tx: &Transaction<InProgress>) {
    println!(
        "Processing transaction {} which is in progress.",
        tx.id
    );
}

```

And here we have it: if a commit or rollback function was called on our transaction, we will not be able to pass the transaction into the function. We can test this with the following code:

```
fn main() {
    let tx = Transaction::<InProgress>::new(1);
    let tx = tx.commit();
    process_in_progress_transaction(&tx);
}
```

And if we try and run it, we get the following printout:

```
process_in_progress_transaction(&tx);
----- ^^^
expected `&Transaction<InProgress>`,
found `&Transaction<Committed>`
```

That is not only quick feedback; it is also straight to the point of where the problem is. We are told that we have already committed our transaction before trying to pass it into the function. The only thing that has changed is the `PhantomData` field, which is a zero-sized struct.

This pattern is known as the type state pattern, where the state of a struct acts as a type. The type state pattern enables us to get the compiler to check that the struct is a certain state, giving us very quick feedback on potential bugs. We could use this type state pattern for the following:

- **User sessions:** When the user has logged out, the state changes. Or the state could be a user role, so only certain parts of the codebase are available to user structs that have a certain role.
- **Resource allocation:** Here, we could ensure that resources are used effectively. We could have states such as `Available`, `InUse`, or `Released`.
- **Cache management:** States such as `Empty`, `Fetching`, or `Available` could dictate what areas of the code are available for a struct handling a cache or access to a cache.
- **Multi-step processes:** States such as `Step1`, `Step2`, `Step3`, and so on could force progression of a struct, ensuring that it could never go back a step or skip a step.

We could keep going on, but with this list, we get the picture of how the type state pattern could be implemented and what problems it could solve.

Summary

In this chapter, we covered some useful Rust-specific patterns that will take our code to the next level. We can enable multiple permissions for a struct using multiple traits. We then scaled our ability to write Rust code with macros, where we mapped data contracts with functions to handle them. Finally, we revisited traits to configure functions and implemented the type state pattern to lock down our struct.

These approaches can help you solve problems in Rust in an elegant, secure, and scalable way. You can use these approaches to solve problems you generally come across outside of web programming, too. Throughout the book, we will be revisiting these approaches to solve problems in web programming. Hopefully, with these approaches, you can see that Rust is a safe, yet flexible and powerful language.

Whenever I am feeling a little too sure of myself, I remind myself that some super smart people came up with the Rust programming language and traits. This always reminds me of how much smarter than I people can be. I hope you are also in awe of this language.

In the next chapter, we will be exploring how powerful the build and package manager tools are in Rust when we design the basics of our web application.

Questions

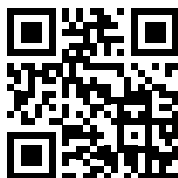
1. How can we enable multiple permissions for a struct?
2. How can we ensure that two permissions are enabled for a struct that can be passed into a function?
3. Let us say I wanted a struct that had an ID field and nothing more. I wanted that ID field to be a string in some cases and an `i32` in others. How could I achieve this?
4. I want to have a function that calls an I/O function in it, but I want the implementation of the I/O call to be flexible, enabling other developers to slot in different implementations. How can I do this?
5. Let us say I have a struct that passes through a process of steps, and I want to lock down my struct so it cannot be passed into functions outside of the current step. How can I do this?

Answers

1. We can declare a trait per permission. We can then implement multiple traits in a struct.
2. We can ensure that two traits are implemented for the struct being passed into the function with `<T: TraitA + TraitB>`.
3. We would exploit generics, where the generic parameter would map to the ID field; we could then create `StructA<i32>` and `StructA<String>`.
4. We can define a trait where the functions for the trait do not have a reference to `self`. This trait can then be passed to the function as a trait parameter with `function_a<T: SomeTrait>()`, so we can define the function struct with `function_a<SomeStruct>()`.
5. We can implement the type state pattern.

Learn more on Discord

To join the Discord community for this book – where you can share feedback, ask questions to the author, and learn about new releases – follow this QR code: <https://packt.link/EaKXL>



3

Designing Your Web Application in Rust

We previously explored useful patterns in Rust, enabling us to exploit compiler checks and meta programming to make our code even more elegant. However, just improving our Rust skillset in isolated environments is not enough to effectively build fully working programs. As any experienced engineer will tell you, structuring code across multiple files and directories is an important aspect of building software.

In this chapter, we will build a basic command-line to-do program. We manage the dependencies needed to build our command-line program with Rust's **Cargo**. Our program will be structured in a scalable way where we build and manage our own modules, which will be imported into other areas of the program and utilized. We will learn these concepts by building a to-do application spanning multiple files that creates, edits, and deletes to-do tasks. This application will save our multiple to-do application files locally, and we will be able to interact with our application using a command-line interface.

In this chapter, we will cover the following topics:

- Managing a software project with Cargo
- Structuring code through nanoservices
- Creating a task using our data access layer

By the end of this chapter, you will be able to build applications in Rust that can be packaged and used. You will also be able to use third-party packages in your code. As a result, you will be able to build any command-line application that does not require a server or a graphical user interface if you understand the problem you are trying to solve and can break it down into logical chunks.

In this chapter, we also start to write code that is going to be built on in later chapters, as this book guides you through building a microservices system. If you are already familiar with using Cargo to build Rust projects, you may want to skip ahead to the Interacting with Cargo section, as this is the start of our web system.

We cover unit tests in *Chapter 14*; however, from the Structuring code through nanoservices section, it is advised that you check out the code repo for this chapter, as the code is annotated with extra comments and there are also documented unit tests to show you the behavior we expect.

Technical requirements

As we move toward building web apps in Rust, we are going to have to start relying on third-party packages to do some of the heavy lifting for us. Rust manages dependencies through a package manager called Cargo. To use Cargo, install Rust on your computer from the following URL: <https://www.rust-lang.org/tools/install>.

Installing Rust provides not only the programming language Rust but also the dependency manager Cargo. You can download the example project and code for this book by following the instructions in the **Download the example code files** section in the *Preface* of this book.

This chapter's code files are included in the downloadable code bundle.

Managing a software project with Cargo

We use Cargo to compile our Rust code and manage dependencies for the compilation process. However, before we explore the Cargo tool, it's worth understanding some basics around Rust compilation.

When compiling Rust code, the Rust compiler goes through similar steps to C and C++. Rust files are compiled into object files using optimization techniques. These object files are then linked with the required libraries, and a binary output file is produced. Lucky for us, these processes are hidden, unlike C and C++, making it easier.

Before we explore the package manager that is Cargo, let's look at how to perform basic Rust compilation in more detail.

Basic Rust compilation

We should start by building a basic single-file application. To do this, we initially must create a file called `hello_world.rs` in a local directory to house the Rust code that we are going to compile.



The `.rs` extension denotes that the file is a Rust file. To be honest, it does not matter what the extension is. If there is viable Rust code written in that file, the compiler will compile and run it without any issues. However, having different extensions might confuse other developers and code editors and cause problems when importing code from other Rust files. So, it is best to use `.rs` when naming your Rust files.

Inside our `hello_world.rs` file, we can have the following code:

```
fn main() {  
    println!("hello world");  
}
```


This is no different from our first code block in the previous chapter. Now that we have defined our entry point in our `hello_world.rs` file, we can compile the file with the following command:

```
rustc hello_world.rs
```

Once the compilation has finished, there will be a binary file in the same directory that can be run. If we compile it on Windows, we can run the binary with the following command:

```
.\hello_world.exe
```

If we compile it on Linux or macOS, we can run it with the following command:

```
./hello_world
```

Because we only built a simple `hello world` example, `hello world` will just be printed out. While this can be useful when building a simple application in one file, it is not recommended for managing programs spanning multiple files. It is not even recommended when relying on third-party modules. This is where Cargo comes in.

Cargo manages everything, including the running, testing, documentation, building/compiling, and third-party module dependencies, out of the box with a few simple commands. We will cover these commands throughout this chapter. From what we have seen when running our `hello world` example, we must compile the code before we can run it, so we can now move on to the next section, where we build a basic application using Cargo.

Building with Cargo

Building with Cargo is straightforward. All we must do is navigate to a directory where we want to build our project and run the following command:

```
cargo new web_app
```

The preceding command builds a basic Cargo Rust project. If we explore this application, we'll see the following structure:

```
└─ web_app
   └─ Cargo.toml
      └─ src
         └─ main.rs
```

We can see that there is only one Rust file, and this is the `main.rs` file that is housed in the `src` directory. If you open the `main.rs` file, you will see that this is the same as the file that we made in the previous section. It is an entry point with the default code printing out `hello world` to the console. The dependencies and metadata for our project are defined in the `Cargo.toml` file.



The `Cargo.toml` file also houses the `version`, `name`, and `edition` fields under the `package` section. Every few years, Rust releases a new *edition*, which may include new syntax and features but maintains backward compatibility. The editions allow you to opt into new features gradually. `name` and `version` are details around the package that you are building.

It is possible for your code to become “stale” if you don’t keep your `Cargo.toml` file up to date. This means you might miss out on performance improvements, new features, or important fixes. Rust has three release channels: `stable`, `beta`, and `nightly`. New features and fixes are first introduced in the `nightly` channel, tested in `beta`, and then released in the `stable` channel every six weeks. You can switch to `nightly`, but it is not recommended as the syntax might change. I have only used `nightly` when there is a feature that is really needed that is not stable yet.

To keep your code up to date, you can utilize the following cargo commands:

- `cargo update`: This command updates your dependencies to the latest versions within the specified constraints.
- `rustup`: Rustup is a toolchain installer that makes it easy to switch between Rust versions and channels, ensuring you can always use the latest `stable`, `beta`, or `nightly` versions.
- `cargo audit`: This tool checks your dependencies for security vulnerabilities and other issues, helping you stay on top of potential problems.

If we want to run our program, we do not need to navigate to the `main.rs` file and run `rustc`. Instead, we can use Cargo and run it with the following command:

```
cargo run
```

When you do this, you will see the project compile and run with the following printout:

```
Compiling web_app v0.1.0 (/Users/maxwellflitton/Documents/
github/books/Rust-Web_Programming-three/chapter03/web_app)
Finished dev [unoptimized + debuginfo] target(s) in 0.15s
Running `target/debug/web_app`
hello world
```

Your printout will be slightly different because the base directory will be different. At the bottom, you will see `hello world`, which is what we expect. We can also see that the printout states that the compilation is unoptimized and that it is running in `target/debug/web_app`. We can navigate directly to the `target/debug/web_app` binary and run it just like we did in the previous section, as this is where the binary is stored. The `target` directory is where the files for compiling, running, and documenting our program reside. If we attach our code to a GitHub repository, we must make sure that the `target` directory is ignored by GitHub by putting it in the `.gitignore` file. This should be automatically added by the cargo tool, but it is always a good habit to check.

Right now, we are running the unoptimized version of our Rust program. This means that it is slower to run but quicker to compile. This makes sense as when we are developing, we will be compiling multiple times. However, if we want to run the optimized version, we can use the following command:

```
cargo run --release
```

The preceding command gives us the following printout:

```
Finished release [optimized] target(s) in 2.63s
Running `target/release/web_app`
hello world
```

In the preceding output, we can see that our optimized binary is in the `target/release/web_app` path.

Now that we have gotten our basic builds done, we can start to use Cargo to utilize third-party crates.

Shipping crates with Cargo

Third-party libraries in Rust are referred to as crates. Adding them and managing them with Cargo is straightforward. In this section, we will explore this process by utilizing the `rand` crate, available at <https://docs.rs/rand/latest/rand/index.html>.



The documentation for this crate is clear and well structured with links to structs, traits, and modules. This is not a reflection of the `rand` crate itself but is standard documentation for Rust, which we will cover in the next section.

To use this crate in our project, we open the `Cargo.toml` file and add the `rand` crate under the `[dependencies]` section, as follows:

```
[dependencies]
rand = "0.8.5"
```

Now that we've defined our dependency, we can use the `rand` crate to build a random number generator.

We will not be using a random number generator for our to-do application; however, generating random numbers is a nice and easy introduction to using third-party crates using the `ThreadRng` struct. The `ThreadRng` struct is a random number generator that generates an `f64` value between 0 and 1, which is elaborated on in the `rand` crate documentation at <https://docs.rs/rand/latest/rand/rngs/struct.ThreadRng.html>:

```
// src/main.rs
use rand::prelude::*;

fn generate_float(generator: &mut ThreadRng) -> f64 {
    let placeholder: f64 = generator.gen();
    return placeholder * 10.0
}
```

```
fn main() {
    let mut rng: ThreadRng = rand::thread_rng();
    let random_number = generate_float(&mut rng);
    println!("{}", random_number);
}
```

In the preceding code, we have defined a function called `generate_float`, which uses the crate to generate and return a float between 0 and 10. Once we've done this, we print the number. The implementation of the `rand` crate is handled by the `rand` documentation. Our `use` statement imports the `rand` crate. When using the `rand` crate for generating a float, the documentation tells us to import `(*)` from the `rand::prelude` module, which simplifies the importing of common items.



You can view the documentation for `Rand` at <https://docs.rs/rand/latest/rand/prelude/index.html>.

With a few clicks on the introduction page of the `rand` documentation, we can dig into the declarations of the structs and functions used in the demonstration.

Now that our code is built, we can run our program with the `cargo run` command. While Cargo is compiling, it pulls code from the `rand` crate and compiles that into the binary. We can also note that there is now a `cargo.lock` file. As we know that `cargo.toml` is for us to describe our own dependencies, `cargo.lock` is generated by Cargo, and we should not edit it ourselves as it contains exact information about our dependencies.



It is generally a good idea not to commit `cargo.lock` files to your Git if you are building a library. However, if you are building an application, committing the `cargo.lock` file can help builds be reproducible. I personally avoid committing the `cargo.lock` files, but I have worked on big professional projects that do.

This seamless functionality combined with the easy-to-use documentation shows how Rust improves the development process through significant gains via the development ecosystem, as well as the quality of the language. However, all these gains from the documentation are not purely dependent on the third-party libraries; we can also autogenerate our own documentation.

Documenting with Cargo

Speed and safety are not the only benefits of picking a language such as Rust to develop in. The software engineering community continues to learn and grow. Simple things such as good documentation can make or break a project. To demonstrate this, we can define Markdown language within the Rust file with the following code:

```
// src/main.rs
/// This function generates a float number using a number
/// generator passed into the function.
///
```

```

/// # Arguments
/// * generator (&mut ThreadRng): the random number
/// generator to generate the random number
///
/// # Returns
/// (f64): random number between 0 -> 10
fn generate_float(generator: &mut ThreadRng) -> f64 {
    let placeholder: f64 = generator.gen();
    return placeholder * 10.0
}

```

In the preceding code, we've denoted the Markdown with the `///` markers. This does two things: it tells other developers who look at the code what the function does and renders Markdown in our auto-generation.



You can also define code tests in the documentation that are run when the testing suite is run. This is unique to Rust and ensures that the code in the documentation works. In *Chapter 12*, we will cover how to write and run these tests.

Before we run the `document` command, we can define and document a basic user struct and a basic user trait to also show how these are documented:

```

/// This trait defines the struct to be a user.
trait IsUser {
    /// This function proclaims that the struct is a user.
    ///
    /// # Arguments
    /// None
    ///
    /// # Returns
    /// (bool) true if user, false if not
    fn is_user() -> bool {
        return true
    }
}

/// This struct defines a user
///
/// # Attributes
/// * name (String): the name of the user
/// * age (i8): the age of the user
struct User {

```

```
    name: String,  
    age: i8  
}
```

Now that we have documented a range of different structures, we can run the auto-documentation process with the following command:

```
cargo doc --open
```

We can see that the documentation is rendered in the same way as the rand crate:

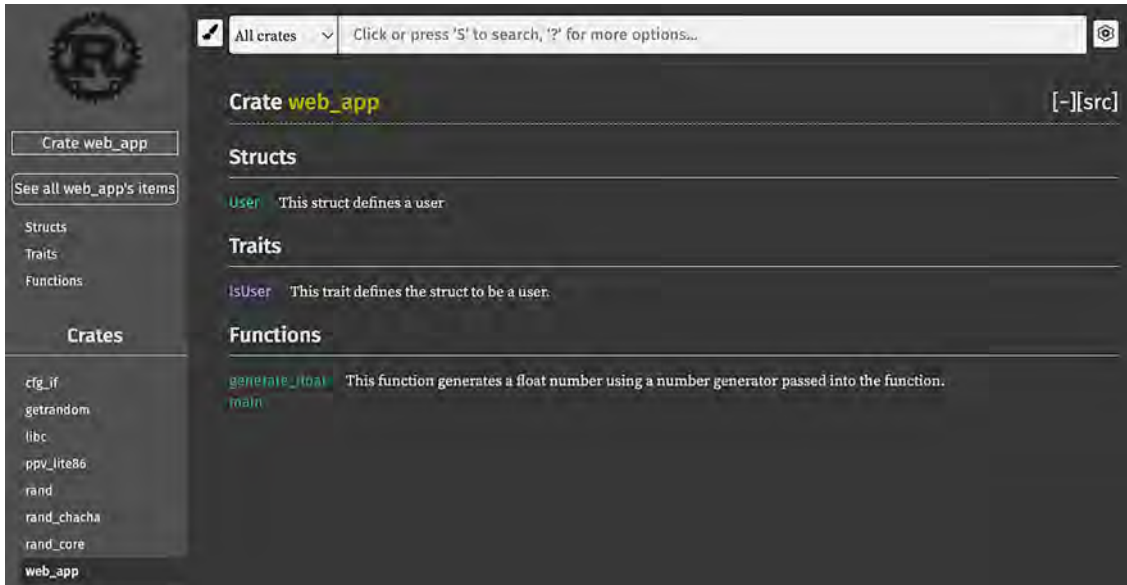


Figure 3.1 – Documentation view of the web app

In the preceding screenshot, we can see that **web_app** is a crate. We can also see that the documentation of the **rand** crate is involved (if we look at the bottom left of the screenshot, we can see the **rand** crate documentation just above our **web_app** crate documentation). If we click on the **User** struct, we can see the declaration of the struct, the Markdown that we wrote for the attributes, and the trait implications, as shown in the following figure:

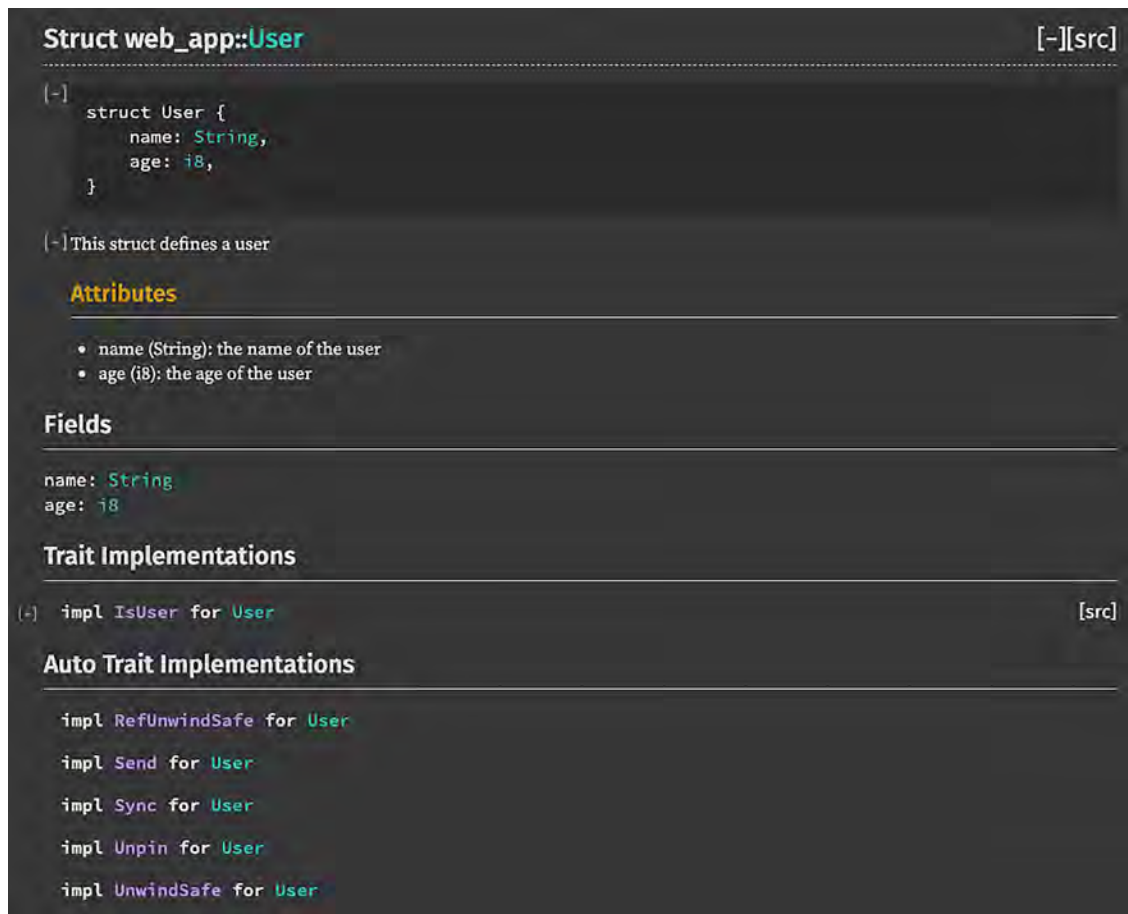


Figure 3.2 – Documentation on struct

It must be noted that in future sections of the book, we will not include Markdown in the code snippets to maintain readability. However, Markdown-documented code is provided in the book's GitHub repo.

Now that we have a well-documented, running Cargo project, we need to be able to pass parameters into it to enable different configurations to run depending on the context.

Cargo cheatsheet



Cargo projects define the metadata and dependencies in the `Cargo.toml` file, and the entry point is in the `src/main.rs` file.

The `cargo run` command compiles and runs the project. The `--release` flag compiles the binary in an optimized release mode, which is faster to run but slower to compile.

Dependencies are defined in the `[dependencies]` section of the `Cargo.toml` file.

`///` at the start of the line is documentation for an expression such as a struct, enum, or function. `//` is a single-line comment, and `//!` is documentation for the top of the file.

The `cargo doc --open` command renders the code documentation in the browser.

Interacting with Cargo

Now that we have our program running and using third-party modules, we can start to interact with our Rust programs through command-line inputs. This is where we start to build out our web application for most of the book. We will keep everything isolated, giving us maximum flexibility. Initially, our project will take the following file structure:

```
├─ Cargo.toml
├─ to_do
│   └─ core
│       ├── Cargo.toml
│       └─ src
│           └─ main.rs
```

Here, we have defined our `to_do` service. Inside the `to_do` service, we have a `core` module. The `core` module is where we run our core logic that handles the creation of to-do items. Later, we will build out the data access module and networking modules for the `to_do` service.

Before we define our root Cargo file, we must have a basic skeleton Cargo file with the following contents, to define the data around our Cargo workspace so other workspaces can reference this workspace:

```
# File: ./to_do/core/Cargo.toml
[package]
name = "to-do-core"
version = "0.1.0"
edition = "2021"

[dependencies]
```

Note that our system is called `to-do-core` as opposed to `core`. This is because `core` is a part of Rust that you can reference for uses such as manual memory management. If we name one of our workspaces `core`, we run the risk of clashes.

Because we have multiple Rust crates in our project, it will help to define where the Rust projects are in the repository, in workspaces. We can define our workspaces with the following code:

```
# File: ./Cargo.toml
[workspace]
resolver = "2"
members = [
    "to_do/core"
]
```

With these workspaces, when we run our cargo commands, these commands will be run on all our workspaces. We can also see that we have introduced a resolver.



A resolver in Rust's Cargo is a configuration setting that determines which version of Cargo's dependency resolution algorithm to use. It dictates how Cargo handles the dependencies, optional dependencies, and feature sets of a project. By specifying the resolver, such as `resolver="2"`, in the `Cargo.toml` file, developers can take advantage of the improved dependency resolution algorithm introduced in Cargo 1.51. This ensures more accurate and flexible handling of dependencies, leading to consistent and predictable builds across different environments. Defining the resolver helps maintain stability, compatibility, and clarity in how dependencies are managed throughout the project's lifecycle.

Now, let us get some basic interaction with our system where we can pass in commands to our program using the command line. To enable our program to have some flexibility depending on the context, we need to be able to pass parameters into our program and keep track of the parameters in which the program is running. We can do this using the `std` (standard library) identifier with the following code in our file:

```
///! File: to_do/core/src/main.rs
use std::env;

fn main() {
    let args: Vec<String> = env::args().collect();
    println!("{:?}", args);
}
```

In the preceding code, we can see that we collect the arguments passed into the program into a vector and then print out the arguments in debug mode. Let us run the following command to run our core with the arguments `["one", "two", "three"]`:

```
cargo run -p to-do-core one two three
```

Running the preceding command gives the following printout:

```
["target/debug/to-do-core", "one", "two", "three"]
```

Here, we can see that our `args` vector has the arguments that we passed in. This is not surprising as many other languages also accept arguments passed into the program via the command line. We must note as well that the path to the binary is also included.



I am using the project named `core`, hence the `target/debug/to-do-core` path.

We can also see from the command-line arguments that we are running in debug mode. Let us try to run a release version of our program with the following command:

```
cargo run -p to-do-core --release one two three
```

We would receive the following printout:

```
["target/release/to-do-core", "one", "two", "three"]
```

From the preceding output, we can see that `--release` is not in our vector. However, this does give us some extra functionality to play with.

So far, we have passed in some basic commands; however, this is not helpful or scalable. There would also be a lot of boilerplate code written for us to implement help guides for users. To scale our command-line interface, we can lean on the `clap` crate to handle arguments passed into the program, with the following dependency:

```
# File: to_do/core/Cargo.toml
[dependencies]
clap = { version = "4.5.16", features = ["derive"] }
```



At the time of writing, the `clap` crate requires `rustc 1.74` at a minimum. If you need to update your Rust version, run the `rustup update` command.

To flesh out our understanding of command-line interfaces, we can develop a toy application that merely takes in a few commands and prints them out. To do this, we must import what we need from the `clap` crate in the `main.rs` file with the following code:

```
//! File: to_do/core/src/main.rs
use clap::Parser;
```

We now get to define the parameters that need to be passed into our program with the following code:

```
//! File: to_do/core/src/main.rs

/// Simple program for booking
```

```
#[derive(Parser, Debug)]
#[command(version, about, long_about = None)]
struct Args {
    /// first name of user
    #[arg(short, long)]
    first_name: String,

    /// last name of user
    #[arg(short, long)]
    last_name: String,

    /// age of the user
    #[arg(short, long, default_value_t = 1)]
    age: u8,
}
```

Here, we need to keep the comments in the preceding code, as these comments will be shown in the help menu in the terminal. Here, we can see that the first name and last name are strings, and we also require an age, but we have a default of 1. To consume and print out these arguments, we only need the following code:

```
fn main() {
    let args = Args::parse();

    println!("{:?}", args.first_name);
    println!("{:?}", args.last_name);
    println!("{:?}", args.age);
}
```

Now that we have a working example of how to pass command-line arguments, we can interact with our application to see how it displays by running the following command:

```
cargo run -p to-do-core -- --help
```

The middle `--` before `--help` tells Cargo to pass all the arguments after `--` into `clap` as opposed to cargo.

The preceding command will give us the following printout:

```
Simple program for booking

Usage: core [OPTIONS] --first-name <FIRST_NAME> --last-name <LAST_NAME>

Options:
  -f, --first-name <FIRST_NAME>  first name of user
  -l, --last-name <LAST_NAME>    last name of user
```

```
-a, --age <AGE>      age of the user [default: 1]
-h, --help            Print help
-V, --version         Print version
```

In the preceding output, we can see how to directly interact with our compiled binary file. We also have a nice help menu.

To interact with Cargo, we need to run the following command to run our to-do-core while passing in arguments via clap:

```
cargo run -p to-do-core -- --first-name maxwell --last-name flitton --age 34
```

The preceding command will give the following printout:

```
"maxwell"
"flitton"
34
```

We can see that the parsing works as we have two strings and an integer. The reason why crates such as clap are useful is that they are essentially self-documenting. Developers can look at the code and know what arguments are being accepted and view the metadata around them. Users can get help on the inputs by merely passing in the help parameter. This approach reduces the risk of the documentation becoming outdated as it is embedded in the code that executes it. If you accept command-line arguments, it is advised that you use a crate such as clap for this purpose.



Passing comments into the program cheatsheet

Cargo.toml in the root of the project can handle multiple Cargo projects defined in the [workspace] section. The -p flag chooses the specific Cargo project for the command, such as cargo run -p project-one.

Arguments passed into the Rust program can be collected using env::args().

If using a cargo run command, the arguments passed in must be separated from the cargo command with --. An example is cargo run -- arg-one arg-two.

Clap is a useful third-party crate that can create an interface and help menu for passing commands into a Rust program.

Now that we have explored structuring our command-line interface so it can scale, we can investigate structuring our code over multiple files to scale it in the next section.

Structuring code through nanoservices

We can now begin our journey of building a web application. In the rest of this chapter, we will not touch a web framework or build an HTTP listener. This will happen in the next chapter. However, we will construct a to-do module that will interact with a JSON file. It is going to be structured in such a way that it can be inserted into any web application that we build with minimal effort. This to-do module will enable us to create, update, and delete to-do items. We will then interact with this via the command line. The process here is to explore how to build well-structured code that will scale and be flexible. To gain an understanding of this, we will break down the building of this module into the following chunks:

1. Building to-do structs
2. Managing structs with an API
3. Storing tasks with our data access layer using a JSON file (we will replace this with a proper database in later chapters)

Before we start tackling these steps, we are going to investigate the overall structure of our application. Our `to_do` directory is going to house several Cargo projects. These Cargo projects will build on each other to create a to-do service, which can be a server that runs by itself as a microservice in a cluster. However, because our service comprises isolated Cargo projects, we will also be able to compile our `to_do` service directly into another service as a module. I personally have used this approach when building medical simulation software for the German government. I called the approach **nanoservices**, due to being able to compile my entire cluster into one binary. But any of those nanoservices could evolve into a microservice if I needed it to. This also simplified and sped up the local builds when developing locally.

To see how nanoservices are built, we can start by building out our to-do structs.

Building to-do structs

Right now, we only have two structs for to-do items: ones that are waiting to be done and others that are already done. However, we might want to introduce other categories. For instance, we could add a backlog category, or an *on-hold* task for tasks that have been started but for one reason or another are blocked.

To avoid mistakes and repetitive code, we can build a Base struct and have that be utilized by other structs. The Base struct houses common fields and functions. An alteration of the Base struct will propagate to all other to-do structs. We will also need to define the type of to-do item. We could hard-code in strings for pending and done; however, this is not scalable and is also error-prone. To avoid this, we will use an enum to classify and define the presentation of the type of to-do item. To build our to-do structs, we must carry out the following steps:

1. Configure our file structure.
2. Define the task status.
3. Build our to-do structs.

We start with configuring our file structure.

Configuring our file structure

Before we write any code, we need to define the file structure for our to-do structs. Our entire application now needs to have the following outline:

```
├─ Cargo.toml
└─ to_do
    └─ core
        ├── Cargo.toml
        └─ src
            ├── api
            │   └─ mod.rs
            ├── enums.rs
            ├── main.rs
            └─ structs
                ├── base.rs
                ├── done.rs
                ├── mod.rs
                └─ pending.rs
```

Here, we can see that we have defined two modules, `api` and `structs`, in the core of the `to_do` service. You might notice that there is a `mod.rs` in the `api` and `structs` directories. `mod.rs` enables us to declare files in the module. For instance, we can declare the files in the `structs` module with the following code:

```
///! File: to_do/core/src/structs/mod.rs
mod base;
pub mod done;
pub mod pending;
```

This means that `done.rs` and `pending.rs` are publicly accessible inside and outside of the `structs` module. However, the `base.rs` file is only accessible inside the `structs` module because there is no `pub` keyword. This is because we are not expecting the struct in the `base.rs` file to be exposed outside of the `structs` module.

We now must declare the `api`, `structs`, and `enums` modules in our `main.rs` file to include our modules in the build with the following code:

```
///! File: to_do/core/src/main.rs
mod enums;
mod structs;
mod api;
```

If we compile our code, we will not have any problems. This means that our modules are being compiled into the Rust binary. Now that we have everything compiling, we need to decide what to work on first.

Defining the task status

The enums are isolated with no dependencies. In fact, a task status enum can supply all the to-do structs. Therefore, we will start with our enum in the `to_do/core/src/enums.rs` file. Our enum is defining the status of the task with the following code:

```
//! File: to_do/core/src/enums.rs
#[derive(Debug, PartialEq)]
pub enum TaskStatus {
    DONE,
    PENDING
}
```

Here, we can see that we implement the `PartialEq` trait. This is so we can perform the `==` on two different `TaskStatus` enum instances if we want to compare them. This will work in the code when it comes to defining the status of the task. However, if we want to write to a file or database, we are going to have to build a method to enable our enum to be represented in a string format. We can do this by implementing the `Display` trait for `TaskStatus`. First, we must import the `format` module to implement the `Display` trait with the following code:

```
//! File: to_do/core/src/enums.rs
use std::fmt;
```

We can then implement the `Display` trait for the `TaskStatus` struct with the following code:

```
//! File: to_do/core/src/enums.rs

impl fmt::Display for TaskStatus {
    fn fmt(&self, f: &mut fmt::Formatter) -> fmt::Result {
        match &self {
            &Self::DONE => {write!(f, "DONE")},
            &Self::PENDING => {write!(f, "PENDING")}
        }
    }
}
```

The implementation of the `Display` trait means that we can use the `to_string` function for our `TaskStatus` struct. If we were to use the `TaskStatus` in the following code:

```
println!("{}", TaskStatus::DONE);
println!("{}", TaskStatus::PENDING);
let outcome = TaskStatus::DONE.to_string();
println!("{}", outcome);
```

We would get the following printout:

```
DONE
PENDING
DONE
```

Here, we can see that when we pass `TaskStatus` into `println!`, the `Display` trait is automatically utilized.

Now that we have our `TaskStatus` enum defined, we can move on to building out the `Base` struct, which will be used to construct the other to-do structs that we build.

Building our to-do structs

We can define the `Base` struct in the `/to_do/core/src/structs/base.rs` file with the following code:

```
//! File: to_do/core/src/structs/base.rs
use super::super::enums::TaskStatus;

pub struct Base {
    pub title: String,
    pub status: TaskStatus
}
```

From the import at the top of the file, we can access the `TaskStatus` enum using `super::super`. We know that the `TaskStatus` enum is in a higher directory. From this, we can deduce that `super` gives us access to what is declared in the `/to_do/core/src/structs/mod.rs` file of the current directory. So, using `super::super` in a file in the `/to_do/core/src/` directory gives us access to what is defined in the `to_do/core/src/main.rs` file. Our `Base` struct is the raw data that can be saved and loaded from persistent storage.

Now that we have our `Base` struct, we can build our `Pending` and `Done` structs. These `Pending` and `Done` structs act as interfaces for the raw data in the `Base` struct. Because they are interfaces, we can lock down certain behavior and define the status for each one. This is achieved by composition with the following code:

```
//! File: core/src/structs/pending.rs
use super::base::Base;
use super::super::enums::TaskStatus;

pub struct Pending {
    pub super_struct: Base
}

impl Pending {
    pub fn new(input_title: &str) -> Self {
        let base = Base{
```



```
        title: input_title.to_string(),
        status: TaskStatus::PENDING
    };
    return Pending{super_struct: base}
}
}
```

Through the preceding code, we can see that our `super_struct` field houses our `Base` struct. We utilize our enum and define the status to be pending. This means that we only must pass the title into the constructor, and we have a struct with a title and a status of pending. Considering this, coding our `Done` struct should be straightforward in our `/to_do/structs/done.rs` file with the following code:

```
///! File: core/src/structs/done.rs
use super::base::Base;
use super::super::enums::TaskStatus;

pub struct Done {
    pub super_struct: Base
}

impl Done {
    pub fn new(input_title: &str) -> Self {
        let base = Base {
            title: input_title.to_string(),
            status: TaskStatus::DONE
        };
        return Done{super_struct: base}
    }
}
```

We can see that there is not much difference from the `Pending` struct definition apart from the `TaskStatus` enum having a `DONE` status. At this point, it might seem excessive to write two separate structs. Right now, we are in the discovery phase. If the functionality of our structs increases in the future, our structs are decoupled, meaning we can update the struct functionality without any pain. However, if the complexity does not explode, we could investigate refactoring the structs into one struct. However, we must also note that this approach is not the only legitimate way. Some developers like to start with just one page and branch out when the complexity increases. I personally do not like this approach as I have seen code get highly coupled before the decision is made to break it out, making the refactor harder. No matter what approach you take, if you keep track of the complexity, keep the code decoupled, and can refactor when needed, you are all good.

We have now made a basic module and exposed it to the `to_do/core/src/main.rs` file. For now, we can write some basic code that will use our module to create a task that is pending and another that is completed. This can be done with the following code:

```
#!/ File: to_do/core/src/main.rs
mod enums;
mod structs;
mod api;

use structs::done::Done;
use structs::pending::Pending;

fn main() {
    let done = Done::new("shopping");
    println!("{}", done.super_struct.title);
    println!("{}", done.super_struct.status);

    let pending = Pending::new("laundry");
    println!("{}", pending.super_struct.title);
    println!("{}", pending.super_struct.status);
}
```

In the preceding code, we imported our structs and created a pending and done struct. Running our code will give us the following printout:

```
shopping
DONE
laundry
PENDING
```

This stops the `main.rs` file from being overloaded with excessive code. If we were to stack more types of items that can be created, such as the on-hold or backlog items, the code in `main.rs` would balloon. This is where APIs come in, which we will explore in the next step.

Managing structs with an API

Right now, we have a core module in our `to_do` service, and we are now going to build out our `api` module. The `api` module essentially is an interface from outside with our core module. A separate `api` module might seem excessive; however, having an `api` module gives us the ability to have multiple different types of external interactions, as seen in *Figure 3.3*.

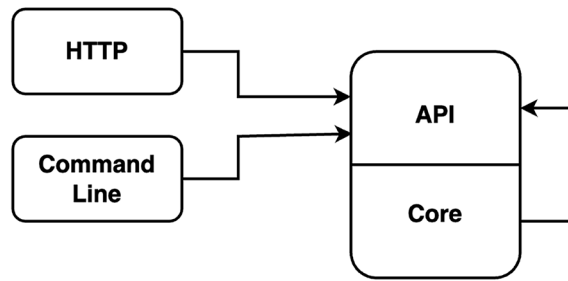


Figure 3.3 – The flexibility of an API layer

The `api` module gives other developers a clean interface to interact with. This clean interface enables processes such as an HTTP request or input from a command line to interact with our core if we can get our process to adhere to the API interface. We also get a simple auto-documentation benefit. If we must revisit our code, we can just look at the `api` module to see what interfaces are available as opposed to rooting in the `core` module. Separating the modules also helps with the separation of concerns. We focus on the core logic in the `core` module, whereas the `api` module handles the versioning, deprecating, responses, and messages around the interface. Both modules can quickly balloon in complexity when refactoring and new features get added.

For now, our `api` module takes the following form:

```

├─ Cargo.toml
├─ to_do
│   └─ core
│       ├── Cargo.toml
│       └─ src
│           ├── api
│           │   ├── basic_actions
│           │   │   ├── create.rs
│           │   │   └─ mod.rs
│           │   └─ mod.rs

```

Our `api` module is already defined in our `main.rs` file. So, all we need is the following declarations to have our `create` in the `api` module plugged in:

```

///! File: to_do/core/src/api/mod.rs
pub mod basic_actions;
///! File: to_do/core/src/api/basic_actions/mod.rs
pub mod create;

```

We are now ready to create our create API function in the `to_do/core/src/api/basic_actions/create.rs` file. First, we need the following imports:

```
///! File: to_do/core/src/api/basic_actions/create.rs
use std::fmt;
use crate::structs::{
    done::Done,
    pending::Pending,
};
use crate::enums::TaskStatus;
```

We can see that we are going to accept a `TaskStatus` and we are going to return either a `Done` or `Pending` struct. However, these are two different types of structs. To account for this, we are going to wrap these two structs in an enum with the following code:

```
///! File: to_do/core/src/api/basic_actions/create.rs
pub enum ItemTypes {
    Pending(Pending),
    Done(Done),
}
```

For our example right now, we are just going to print out our structs. It seems a little excessive to have to perform a match statement to just print out these two structs. Therefore, we are going to implement the `Display` trait for the `ItemTypes` enum. You saw an implementation of the `Display` trait in the previous chapter. Now is a good time to try and implement the `Display` trait yourself. If you have attempted this, your code should have a form like the following:

```
///! File: to_do/core/src/api/basic_actions/create.rs
impl fmt::Display for ItemTypes {
    fn fmt(&self, f: &mut fmt::Formatter) -> fmt::Result {
        match self {
            ItemTypes::Pending(pending) => write!(
                f, "Pending: {}",
                pending.super_struct.title
            ),
            ItemTypes::Done(done) => write!(
                f, "Done: {}",
                done.super_struct.title
            ),
        }
    }
}
```

We can now build our create function, which will accept a status and a title, returning an `ItemTypes` enum with the following code:

```
//! File: to_do/core/src/api/basic_actions/create.rs

pub fn create(title: &str, status: TaskStatus) -> ItemTypes {
    match status {
        TaskStatus::PENDING => {
            ItemTypes::Pending(Pending::new(&title))
        },
        TaskStatus::DONE => {
            ItemTypes::Done(Done::new(&title))
        },
    }
}
```

Here we can see that if we increase the types of structs that we support, we can just stack them in the create function. We can also see the power of decoupling here. For instance, we could make up a rule that only users of a certain credential can create done tasks. We could enforce that rule by performing a check on the user in the create function.

We now have everything ready to interact with our create API in our `main.rs` file, as seen with the following code:

```
//! File: to_do/core/src/main.rs

mod enums;
mod structs;
mod api;
use api::basic_actions::create::create;

fn main() {
    let to_do_item = create(
        "washing",
        enums::TaskStatus::PENDING
    );
    println!("{}", to_do_item);
}
```

Here, we can see that the code needed to interact with our create API is greatly simplified. Running the code gives us the following printout:

```
Pending: washing
```

Now that we have our API working, we need to build out a basic data storage system to store our to-do tasks.

Storing tasks with our data access layer

Our data access layer will handle all the interactions between logic and persistent storage. In this section, we will cover features allowing us to selectively compile the type of storage we want. We will store our tasks in a JSON file, but we will add support for databases in *Chapter 8*. Right now, we just want to get some storage system working so we can continue to develop our application with minimal dependencies.

JSON file considerations



Choosing to store our tasks in a JSON file is far from optimal. Every time we perform an operation on a file, we must make calls to the operating system, checking to see whether the file exists, whether the user has the correct permissions, and whether the file is locked. Async reads and writes are not widely supported; we cannot index, meaning we cannot select rows to load. This means that we must load the entire data into memory before we can perform an operation. JSON files are rarely a substitute for a proper database.

We can implement our storage layer by carrying out the following steps:

1. Defining our module layout
2. Building our JSON file handle
3. Building storage API functions

Let us start with the layout.

Defining our module layout

Our data access layer module takes the following form:

```
└─ to_do
   └─ core
       └─ Cargo.toml
           └─ src
               . . .
   └─ dal
       └─ Cargo.toml
           └─ src
               └─ json_file.rs
                   └─ lib.rs
```

Here, we can see that we have a `lib.rs` file in place of `main.rs`. This means that the Cargo project is a library and our entry point into that library is the `lib.rs` file. We will compile our data access layer (`dal`) library as a dependency into our core module to be used when we need it.

Now that we have the `dal` workspace, we need to declare the workspace in the root `Cargo.toml` with the following code:

```
# File: ./Cargo.toml
[workspace]
resolver = "2"
members = [
    "to_do/core",
    "to_do/dal"
]
```

Before we write any dal code, we need to define `Cargo.toml` for the dal library, which takes the following form:

```
# File: to_do/dal/Cargo.toml
[package]
name = "to-do-dal"
version = "0.1.0"
edition = "2021"

[features]
json-file = ["serde_json", "serde"]

[dependencies]
serde_json = { version="1.0.114", optional = true }
serde = { version="1.0.197", optional = true }
```

Here, we have added a features section with the feature `json-file`. The `serde` dependencies are utilized for the serialization of data. We can also see that both of our `serde` dependencies are optional. This means that the `serde` dependencies will only be compiled if directed to and are not compiled into the library by default. Our `json-file` feature declares that it needs both of our `serde` dependencies if the library is compiled with the `json-file` feature. This gives us fine-grained control over what is compiled. For instance, in *Chapter 9*, we will add Postgres support to our `to-do-dal` library. Based on the feature, our core module will be able to either just compile the code and dependencies needed for JSON file storage, just for Postgres, or both! With all the strict type checking, we can be brave with just compiling multiple different libraries with different features. If everything does not match up, then Rust will refuse to compile, so we don't have to stay up at night worrying whether the interface for one feature is the same as other features.

Now that our module layout is defined, we can move on to building our JSON file handle.

Building our JSON file handle

We now need to declare our JSON file code in our `lib.rs` file with the following code:

```
//! File: to_do/dal/src/lib.rs
#[cfg(feature = "json-file")]
pub mod json_file;
```

Here, what we are doing is compiling the declaration of our `json_file` module if the `json-file` feature is activated.

We can declare our `to-do-dal` library in our core `Cargo.toml` file with the following code:

```
# File: to_do/core/Cargo.toml
[dependencies]
clap = { version = "4.5.16", features = ["derive"] }
to-do-dal = { path = "../dal" }

[features]
json-file-storage = ["to-do-dal/json-file"]
default = ["json-file-storage"]
```

Here, we can see that we pass the feature declaration on to our dependencies. This gives our core the ability to choose the type of data storage when compiling without having to change any code. Now, when we build or run our core module, we add the features flag, resulting in the following command:

```
cargo run --features json-file-storage
```

If we run a build, we can see that our entire system compiles with no problems, meaning that we can now access our JSON file storage functions within our core module! However, note that we defined our JSON file feature as the default feature because we need a storage engine to compile. Therefore, we do not need to pass the features flag if we just want to use the JSON file storage. But knowing about features is important, as we will be using features for different storage engines. Now that everything is plugged in, we can build our functions that interact with our JSON file for storage.

Because we do not want to commit much effort to a placeholder method of reading and writing to a JSON file, we will have all our basic functions in the `json_file.rs` file of the `dal` library. In this file, we initially need the following traits and structs:

```
#!/ File: to_do/dal/src/json_file.rs
use serde::{de::DeserializeOwned, Serialize};
use std::collections::HashMap;
use std::env;
use std::fs::{OpenOptions, File};
use std::io::{Read, Write};
```

We will cover these uses as and when we define our functions. For our JSON data access, we will just build a total of six functions. This makes the flow easy to handle and read, and we do not plan on any complexity exploding; therefore, there will not be any need for abstractions.

Before we define any functions that interact with a file, we need a function that gets the file handle.



A file handle is an integer value that is used by the operating system to uniquely identify a file that was opened at the request of a user.

This function can then be called by all other functions performing an operation on the file. The file handle function can be defined by the following code:

```
// File: to_do/dal/src/json_file.rs
fn get_handle(path: Option<&str>) -> Result<File, String> {
    let path = match path {
        Some(p) => p,
        None => {
            &env::var("JSON_STORE_PATH").unwrap_or(
                "./tasks.json".to_string()
            )
        }
    };
    let file = OpenOptions::new()
        .read(true)
        .write(true)
        .create(true)
        .open(&path)
        .map_err(
            |e| format!("Error opening file: {}", e)
        )?;
    Ok(file)
}
```

Here, we can see that we accept a path to the file. However, if the path is not supplied, we look for an environment variable called `JSON_STORE_PATH`. If this variable is not defined, we then default to the path being `./tasks.json`. Now that we have our file handle, we can move on to building our storage API functions.

Building storage API functions

We can now move on to the most basic function, which is merely reading the file and returning all the results. This function is defined using the following code:

```
// File: to_do/dal/src/json_file.rs

pub fn get_all<T: DeserializeOwned>()
-> Result<HashMap<String, T>, String> {
    let mut file = get_handle(None)?;
    let mut contents = String::new();
    file.read_to_string(&mut contents).map_err(
        |e| format!("Error reading file: {}", e)
    )?;
    let tasks: HashMap<String, T> = serde_json::from_str(
```

```

        &contents).map_err(|e| format!("Error parsing JSON: {}", e)
    )?;
    Ok(tasks)
}

```

Here, we are starting to utilize the power of generics and traits. If the struct has implemented the `DeserializeOwned` trait, a hashmap of those structs mapped by keys as strings can be loaded and returned from the file. Note that we map errors to strings and that we get to use the `?` operator because the error type matches. We will investigate custom error types in *Chapter 4* as we will be able to construct custom HTTP responses based on the error.

We also want to save all the tasks that we have to the file. This is where we pass in the hashmap into the function and write it to the file with the following code to save all our to-do items:

```

// File: to_do/dal/src/json_file.rs

pub fn save_all<T: Serialize>(tasks: &HashMap<String, T>)
    -> Result<(), String> {
    let mut file = get_handle(None)?;
    let json = serde_json::to_string_pretty(tasks).map_err(
        |e| format!("Error serializing JSON: {}", e)
    )?;
    file.write_all(json.as_bytes()).map_err(
        |e| format!("Error writing file: {}", e)
    )?;
    Ok(())
}

```

We can see that the save function has the same use of generics as the get function. We now have everything we need. However, our application will be doing a lot of operations on a single task. We can define these operations ahead, so we do not have to repeat them anywhere else in the application. These functions are slightly repetitive with some variance. It would be a good idea to try and complete these functions yourself. If you do, then hopefully they look like the following:

```

// File: to_do/dal/src/json_file.rs

pub fn get_one<T: DeserializeOwned + Clone>(id: &str)
    -> Result<T, String> {
    let tasks = get_all::<T>()?;
    match tasks.get(id) {
        Some(t) => Ok(t.clone()),
        None => Err(format!("Task with id {} not found", id))
    }
}

```

```
pub fn save_one<T>(id: &str, task: &T)
-> Result<(), String>
where
    T: Serialize + DeserializeOwned + Clone,
{
    let mut tasks = get_all::<T>().unwrap_or_else(
        |_| HashMap::new()
    );
    tasks.insert(id.to_string(), task.clone());
    save_all(&tasks)
}

pub fn delete_one<T>(id: &str) -> Result<(), String>
where
    T: Serialize + DeserializeOwned + Clone,
{
    let mut tasks = get_all::<T>().unwrap_or(
        HashMap::new()
    );
    tasks.remove(id);
    save_all(&tasks)
}
```

Our data access layer is now fully defined. We can move on to utilizing the data access layer in the core.

Creating a task using our DAL

We now shift our focus back to creating a task. When creating this task, we need to store the newly created task using the data access layer. Storing the task in JSON will involve some serialization; therefore, we need to ensure that we have the `serde` crate installed, and our `Cargo.toml` file should have the following dependencies:

```
# File: to_do/core/Cargo.toml
[dependencies]
clap = { version = "4.5.16", features = ["derive"] }
serde = { version = "1.0.197", features = ["derive"] }
to-do-dal = { path = "../dal" }
```

We can now start writing code. We need to enable our `TaskStatus` to deserialize and serialize so we can write the task status of the to-do item to the JSON file. We also want our status to be constructed from a string.

Before we write any additional code to the `TaskStatus` enum, we need to use the following code:

```
///! File: to_do/core/src/enums.rs
use serde::{Serialize, Deserialize};
```

We can then apply the `serde` traits with the following code:

```
// File: to_do/core/src/enums.rs

#[derive(Serialize, Deserialize, Debug, Clone, PartialEq)]
pub enum TaskStatus {
    DONE,
    PENDING
}
```

Finally, we can then define a `from_string` function for our `TaskStatus` enum with the following code:

```
// File: to_do/core/src/enums.rs

impl TaskStatus {
    pub fn from_string(status: &String)
        -> Result<TaskStatus, String> {
        match status.to_uppercase().as_str() {
            "DONE" => Ok(TaskStatus::DONE),
            "PENDING" => Ok(TaskStatus::PENDING),
            _ => Err(format!("Invalid status: {}", status))
        }
    }
}
```

Now, we only need to slot in our data access function in our create API. In our create API file, we use the following function:

```
// File: to_do/core/src/api/basic_actions/create.rs
#[cfg(feature = "json-file-storage")]
use to_do_dal::json_file::save_one;
```

We define the feature because if we add another `save_one` function under a different storage engine, we will want our core to just work with the new feature. With this data access function, our create API function now has the following form:

```
// File: to_do/core/src/api/basic_actions/create.rs

pub fn create(title: &str, status: TaskStatus)
    -> Result<ItemTypes, String> {
    let _ = save_one(&title.to_string(), &status)?;
    match &status {
        TaskStatus::PENDING => {
            Ok(ItemTypes::Pending(Pending::new(&title)))
        },
    }
}
```

```

        TaskStatus::DONE => {
            Ok(ItemTypes::Done(Done::new(&title)))
        },
    }
}

```

We can see how flexible our storage implementation is. When we change out our storage later, it will not be a headache. This means that we have low technical debt right now. Even though our implementation has taken longer than just bashing out all our logic into one file, future refactors will be easier and quicker.

We can now rewrite our `main.rs` file to use everything that we have done so far. First things first, our use statements are defined by the following code:

```

///! File: to_do/core/src/main.rs
mod enums;
mod structs;
mod api;
use api::basic_actions::create::create;
use crate::enums::TaskStatus;
use clap::Parser;

```

With these use statements defined, our `main` function is roughly the same size. This is an achievement considering that we have added a storage layer and command-line interface, as seen in the following code:

```

// File: to_do/core/src/main.rs
#[derive(Parser, Debug)]
#[command(version, about, long_about = None)]
struct Args {
    #[arg(short, long)]
    title: String,
    #[arg(short, long)]
    status: String,
}

fn main() -> Result<(), String> {
    let args = Args::parse();
    let status_enum = TaskStatus::from_string(
        &args.status
    )?;
    let to_do_item = create(
        &args.title,
        status_enum
    )
}

```

```

    }?;
    println!("{}", to_do_item);
    Ok(())
}

```

We can see that our main function returns the same result signature. The `TaskStatus::from_string` and `create` functions can use the `?` operator. Now we can test whether our system works with the following terminal commands:

```

cargo run --features json-file-storage -- \
--title coding \
--status pending
cargo run --features json-file-storage -- \
--title washing \
--status done

```

If we inspect our JSON file after running those commands, it should take the following form:

```

{
  "washing": "DONE",
  "coding": "PENDING"
}

```

And here we have it, a basic terminal command that stores tasks in a JSON file!

Summary

What we have essentially done in this chapter is build a program that accepts some command-line inputs, interacts with a file, and edits it depending on the command and data from that file. The data is simple: a title and a status. We could have done this all in the `main` function with multiple `match` statements and `if`, `else if`, and `else` blocks. However, this is not scalable. Instead, we built structs that inherited other structs, which then implemented traits.

We also defined the structure of our to-do service with workspaces. Here, we enabled our service to have multiple layers so we can swap approaches to storage easily. We also explored how to structure a basic service with a main and data access layer.

In *Chapter 5*, we will build out our service for handling HTTP requests using a web framework. However, before we handle HTTP requests, we will explore `async` in detail, as we use `async rust` to handle incoming HTTP requests. *Chapter 4* is an isolated chapter, meaning that you do not need to read it if you just want to get to grips with web programming. In this case, you can skip to *Chapter 5*.

Questions

1. What does the `--release` argument in Cargo do when added to a build and run?
2. How can you enable an enum or struct to be passed into the `println!` macro?
3. In `nanoservices`, how does a basic service help keep technical debt down?

4. How does returning the same error type in functions reduce the amount of code we write?
5. How do we point to another Cargo project so we can compile that Cargo project into another Cargo project?
6. If we have a Cargo project, how can we choose to include an optional dependency to be compiled into our project?

Answers

1. In a build, the `--release` argument compiles the program in an optimized way as opposed to a debug compilation. In a run, the `--release` argument points to an optimized binary as opposed to the debug binary. An optimized binary takes longer to compile but will run at a faster pace.
2. If you implement the `fmt::Display` trait for the struct or enum where you have to write a `fmt` function that calls the `write!` macro. This `fmt` function will be called when we pass the struct or enum that has implemented the `fmt::Display` trait.
3. A basic service in nanoservices keeps technical debt down by having different layers, such as a core and data access layer. These layers keep concepts isolated so they can be slotted in and out. For instance, with features, we can easily swap out the storage type when needed because the storage logic is defined in the data access layer.
4. If we have two functions that return the signatures `Result<(), String>` and `Result<i32, String>`, we can use the `?` operator to avoid match statements because both of them have the same error type.
5. Under `[dependencies]` in the `Cargo.toml` file, we can point to the path of the Cargo project that we want to add using syntax such as the following: `dal = {path = "../dal"}`.
6. We can opt in and out of dependencies with features. We can set optional to true for the dependency, and then declare that optional dependency for a feature.

Get this book's PDF copy, code bundle, and more

Scan the QR code (or go to packtpub.com/unlock). Search for this book by name, confirm the edition, and then follow the steps on the page.



UNLOCK NOW

Note: Have your invoice handy. Purchases made directly from the Packt website don't require an invoice.

4

Async Rust

We are so close to building Rust code that handles HTTP requests. However, before we do that, it would be beneficial to really understand async programming in Rust, as the average web framework utilizes async code. However, it must be noted that you do not need to understand async code fully to code web servers in Rust. I have met plenty of web programmers who have built adequate servers who do not know how async code works under the hood.



Feel free to skip this chapter if you are stretched for time or do not want to understand async Rust code at a deeper level. This chapter is isolated, so if you skip it, it will not affect the microservices system that we are building throughout the book.

If you are unsure, I would highly advise that you complete this chapter. Understanding how async code works will give you a stronger ability to debug issues and avoid pitfalls. It will also enable you to utilize async Rust fully, diversifying the solutions you can offer in web programming.

In this chapter, we will cover the following topics:

- Understanding threads and processes
- Understanding async and await in Rust
- Implementation of our own async task queue
- Exploring high-level concepts of tokio

By the end of this chapter, you will understand what async programming is and how async tasks are processed in Rust by building your own async task queue. You will also be able to implement your own futures by implementing your own async sleep function. Finally, you will be able to apply the skills to a web context by building an HTTP1 and HTTP2 server on top of tokio using the Hyper crate.



If you do not understand what asynchronous programming is at this point, do not worry; we will explain it throughout the chapter.

Technical requirements

This chapter relies on cURL, which can be installed by visiting the following link: <https://help.ubidots.com/en/articles/2165289-learn-how-to-install-run-curl-on-windows-macosx-linux>.

You can download the example project and code for this book by following the instructions in the **Download the example code files** section in the *Preface* of this book.

This chapter's code files are included in the downloadable code bundle.

Understanding threads and processes

Up until this chapter, we have been writing code in a sequential manner. This is good enough for standard scripts; however, in web development, asynchronous programming is important, as there are multiple requests to servers, and API calls introduce idle time.

In some other languages, such as Python, we can build web servers without touching any asynchronous concepts. While asynchronous concepts are utilized in these web frameworks, the implementation is defined under the hood. However, for most Rust web frameworks, async Rust is directly implemented for handling endpoints.

When it comes to utilizing asynchronous code, there are two main concepts we must understand:

- **Processes:** A process is a program that is being executed. It has its own memory stack, registers for variables, and code.
- **Threads:** A thread is a lightweight process that is managed independently by a scheduler. The scheduler could either be part of the program, meaning that the program is creating, running, and closing threads (these are known as green threads), or the scheduler could be an operating system scheduler. This is where the program makes requests to the operating system to manage the threads. Threads share data with other threads and the main program. Threads are spawned by processes.

This is demonstrated in the following classic diagram:

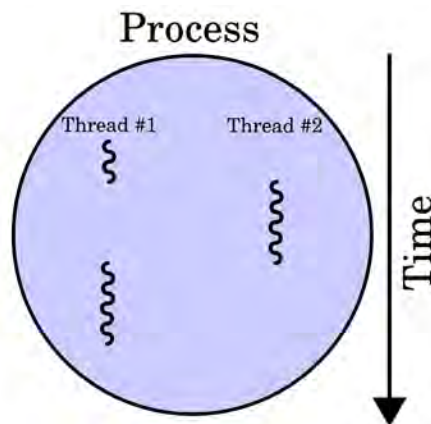


Figure 4.1 – Relationship between threads and processes [source: Cburnett (2007) (<https://commons>).

wikimedia.org/wiki/File:Multithreaded_process.svg), CC BY-SA 3.0 (<https://creativecommons.org/licenses/by-sa/3.0/deed.en>)]

Now that we understand what threads are and what relation they have to our code on a high-level basis, we can play with a toy example to understand how to utilize threads in our code and see the effects of these threads firsthand. A classic example is to build a basic function that merely sleeps, blocking time. This can simulate a time-expensive function such as a network request. We can run it sequentially with the following code:

```
use std::{thread, time};

fn do_something(number: i8) -> i8 {
    println!("number {} is running", number);
    let two_seconds = time::Duration::new(2, 0);
    thread::sleep(two_seconds);
    return 2
}

fn main() {
    let now = time::Instant::now();
    let one: i8 = do_something(1);
    let two: i8 = do_something(2);
    let three: i8 = do_something(3);
    println!("time elapsed {:?}", now.elapsed());
    println!("result {}", one + two + three);
}
```

Running the preceding code will give us the following printout:

```
number 1 is running
number 2 is running
number 3 is running
time elapsed 6.0109845s
result 6
```

In the preceding output, we can see that our time-expensive functions run in the order that we expect them to. It also takes just over six seconds to run the entire program, which makes sense since we are running three expensive functions that sleep for two seconds each. Our expensive function also returns 2. When we add the results of all three expensive functions together, we get a result of 6.

We can speed up our program to roughly two seconds for the entire program by spinning up three threads at the same time and waiting for them to complete before moving on. Waiting for the threads to complete before moving on is called *joining*. So, before we start spinning off threads, we must import the join handler with the following code:

```
use std::thread::JoinHandle;
```

We can now spin up threads in our main function with the following code:

```
let now = time::Instant::now();

let thread_one: JoinHandle<i8> = thread::spawn(
    || do_something(1));
let thread_two: JoinHandle<i8> = thread::spawn(
    || do_something(2));
let thread_three: JoinHandle<i8> = thread::spawn(
    || do_something(3));

let result_one = thread_one.join();
let result_two = thread_two.join();
let result_three = thread_three.join();

println!("time elapsed {:?}", now.elapsed());

println!("result {}", result_one.unwrap() +
        result_two.unwrap() +
        result_three.unwrap());
```

Running the preceding code gives us the following printout:

```
number 1 is running
number 3 is running
number 2 is running
time elapsed 2.002991041s
result 6
```

As we can see, the whole process took just over two seconds to run. This is because all three threads are running concurrently. We can also notice that thread 3 is fired before thread 2. Do not worry if you get a sequence of 1, 2, 3. Threads finish in an indeterminate order. The threads are scheduled in a deterministic order of 1, 2, 3; however, there are thousands of events happening under the hood that require the CPU to do something. As a result, the exact time slice that each thread gets is never the same. These tiny changes add up. Because of this, we cannot guarantee that the threads will finish in a determinate order.

Looking back at how we spin off threads, we can see that we pass a closure into our thread. If we try and just pass the `do_something` function through the thread, we get an error complaining that the compiler expected an `FnOnce<>` closure and found `i8` instead. This is because a standard closure implements the `FnOnce<>` public trait, whereas our `do_something` function simply returns `i8`. When `FnOnce<>` is implemented, the closure can be called only once. This means that when we create a thread, we can ensure that the closure can only be called once, and then when it returns, the thread ends. As our `do_something` function is the final line of the closure, `i8` is returned.



The `FnOnce<()>` trait only gets enforced if the context requires it. This means that if we were to call the closure outside of the thread context, we could call it multiple times. Remember that we can implement multiple traits; therefore, an expression can implement the `FnOnce`, `FnMut`, and `Fn` traits.

Notice that we also directly unwrap our results. From what we know, we can deduce that the `join` function on the `JoinHandle` struct returns a `Result`, which we also know can be an `Err` or `Ok`. We know it is going to be `Ok` unwrapping the result directly because we are merely sleeping and then returning an integer. We also printed out the results, which were indeed integers. However, the returned error type is not what you would expect. The full `Result` type we get is `Result<i8, Box<dyn Any + Send>>`.

We already know what `Box` is; however, the `dyn Any + Send` part seems new. `dyn` stands for **dynamic dispatch**. Dynamic dispatch means that the exact method implementation is not known until runtime. However, the implementation must implement the trait that the `dyn` keyword is pointing to. This gives us more flexibility. There is a slight performance cost with dynamic dispatch as there has to be a lookup on a function table called a **vtable** for the function being called.

`Any + Send` refers to two traits that must be implemented. The `Any` trait is for dynamic typing, meaning that the data type can be anything. The `Send` trait means that it is safe to be moved from one thread to another. `Send` also means that it is safe to copy from one thread to another. Now that we understand this, we could handle the results of threads by merely matching the `Result` outcome, and then down-casting the error into a string to get the error. There is nothing stopping you from logging failures of threads or spinning up new threads based on the outcomes of previous threads. Thus, we can see how powerful the `Result` struct is.



There is more we can do with threads, such as give them names or pass data between them with channels. However, the focus of this book is web programming, not async Rust. For further reading on this topic, I have written a book on async Rust: *Async Rust: Unleashing the Power of Fearless Concurrency* (2024).

We now understand how to spin up threads in Rust, what they return, and how to handle them. With this information, we can move on to the next section, where we will understand the `async` and `await` syntax that is going to be used in our web server.

Understanding `async` and `await` with Rust

The `async` and `await` syntax manages the same concepts covered in the previous section; however, there are some nuances. Instead of simply spawning off threads, we create **futures** and then manipulate them as and when needed.

The term `async` denotes the expression of an async task, known as a future, that is going to be run on the async runtime. The `await` expression can be called inside a future, on another future. `await` halts the progress of the async task until the future being awaited on is polled to completion.



Polling to completion is going to be confusing to you if you do not understand how async runtimes handle their async tasks. Do not worry: we cover the handling of async tasks and polling futures in the next section. For now, we will just focus on futures and using `await` syntax.

In computer science, a future is an unprocessed computation. This is where the result is not yet available, but when we call or wait, the future will be populated with the result of the computation. Another way of describing this is that a future is a way of expressing a value that is not yet ready. As a result, a future is not exactly a thread. In fact, threads can use futures to maximize their potential. For instance, let us say that we have several network connections. We could have an individual thread for each network connection. This is better than sequentially processing all connections, as a slow network connection would prevent other, faster connections from being processed down the line until it itself is processed, resulting in a slower processing time overall. However, spinning up threads for every network connection is not free as we must allocate memory for the thread. Instead, we can have a future for each network connection. These network connections can be processed by a thread from a thread pool when the future is ready, rather than allocating a thread for the duration of a network connection. We can therefore see why futures are used in web programming as there are a lot of concurrent connections.



Futures can also be referred to as promises, delays, or deferred.

To explore futures, we will create a new Cargo project and utilize the futures created in the Cargo.toml file:

```
[dependencies]
tokio = { version = "1.36.0", features = ["full"] }
```

`tokio` is an async runtime that has a thread pool. When we await on a future when the `tokio` runtime is running, our future is essentially processed by one of the threads in the `tokio` runtime.

With the `tokio` crate installed, we can import what we need in our `main.rs` using the following code:

```
use std::{thread, time};
```

We can define futures by using `async` syntax. We can now define our `do_something` function with the following code:

```
async fn do_something(number: i8) -> i8 {
    println!("number {} is running", number);
    let two_seconds = time::Duration::new(2, 0);
    thread::sleep(two_seconds);
    return 2
}
```

Our `do_something` function essentially does what the code says it does, which is print out what number it is, sleep for two seconds, and then return an integer. However, if we were to directly call it, we would not get `i8`. Instead, calling our `do_something` function directly will give us `Future<Output = i8>`.

We can run our future and time it in the `main` function with the following code:

```
#[tokio::main(worker_threads = 1)]
async fn main() {
    let now = time::Instant::now();
    let future_one = do_something(1);
    let outcome = future_one.await;
    println!("time elapsed {:?}", now.elapsed());
    println!("Here is the outcome: {}", outcome);
}
```

We can see that the `[tokio::main]` macro enables us to turn our `main` function into an `async` function. Running the preceding code will give us the following printout:

```
number 1 is running
time elapsed 2.00018789s
Here is the outcome: 2
```

This is what is expected. However, note that if we enter an extra `sleep` function before `future_one`, we utilize `await` with the following code:

```
#[tokio::main(worker_threads = 1)]
async fn main() {
    let now = time::Instant::now();
    let future_one = do_something(1);
    let two_seconds = time::Duration::new(2, 0);
    thread::sleep(two_seconds);
    let outcome = future_one.await;
    println!("time elapsed {:?}", now.elapsed());
    println!("Here is the outcome: {}", outcome);
}
```

We will get the following printout:

```
number 1 is running
time elapsed 4.000269667s
Here is the outcome: 2
```

We can see that our future does not execute until we apply the `await` syntax.

We can send our async task to the executor straight away and then wait on it later.



In general software engineering terms, an executor is a tool that manages and executes tasks.

We can test spawning the task and waiting for the result later with the following code:

```
#[tokio::main(worker_threads = 1)]
async fn main() {
    let now = time::Instant::now();
    let future_one = tokio::spawn(do_something(1));
    let two_seconds = time::Duration::new(2, 0);
    thread::sleep(two_seconds);
    let outcome = future_one.await.unwrap();
    println!("time elapsed {:?}", now.elapsed());
    println!("Here is the outcome: {}", outcome);
}
```

This gives us the following printout:

```
number 1 is running
time elapsed 2.005152292s
Here is the outcome: 2
```

Here, we can see that our time elapsed has halved! This is because `tokio::spawn` sends the task to the tokio worker thread to be executed while the main thread processes the `sleep` function in the `main` function. However, we could increase the number of tasks spawning by one with the following code:

```
let future_one = tokio::spawn(do_something(1));
let future_two = tokio::spawn(do_something(2));
let two_seconds = time::Duration::new(2, 0);
thread::sleep(two_seconds);
let _ = future_one.await.unwrap();
let _ = future_two.await.unwrap();
```


Our time elapsed increases to roughly four seconds. This is because we only have one worker thread running for the async runtime, and the `sleep` function is blocking. We could increase the number of threads to two to get the time elapsed back down to roughly two seconds. However, this is not scalable as we could have thousands of async tasks. It would not make sense to spin up thousands of threads. Instead, we can make our `sleep` function async by using the tokio `sleep` function for our `do_something` function, giving this function the following form:

```
async fn do_something(number: i8) -> i8 {
    println!("number {} is running", number);
    let two_seconds = time::Duration::new(2, 0);
    tokio::time::sleep(two_seconds).await;
    return 2
}
```

The `await` syntax of the `sleep` function enables the thread to switch to other async tasks to progress them while the sleep duration passes. If we run our main function again, we can see that the duration time is back down to two seconds.

We can now start to see the potential benefits of async Rust. We can shoot off multiple async tasks to be processed by the async runtime thread pool and continue with our program, waiting on the async tasks later when we need the results from them. While we are getting on with other things, those async tasks can be processed. However, do we really understand what is going on under the hood? To solidify our knowledge of what is going on, let's implement our own async task queue.

Implementing our own async task queue

For this section, you will need to start a new workspace, as we will be building our own async queue and not relying on tokio. To get an appreciation for what is happening when we see async code in our web server, we are going to implement our own queue that schedules async tasks. Before we write any code, we need the following dependencies:

```
[dependencies]
async-task = "4.7.0"
futures-lite = "2.2.0"
flume = "0.11.0"
```

When we go through the steps to implement our own async task queue, we will demonstrate how and when we will use such dependencies. Before we get the chance, however, we also need to use the following structs and traits:

```
use std::{future::Future, panic::catch_unwind, thread};
use std::time::{Duration, Instant};
use std::pin::Pin;
use std::task::{Context, Poll};

use async_task::{Runnable, Task};
```

```
use futures_lite::future;
use std::sync::LazyLock;
```

We are now ready to move on to our first step: creating an async task queue.

Building a task-spawning function

Our async queue needs to be ergonomic. This means that we must have a function that a developer can use to just spawn the task and not think about anything else, just like the `tokio::spawn` function we used in the previous section. Our `spawn_task` function takes the following form:

```
fn spawn_task<F, T>(future: F) -> Task<T>
where
    F: Future<Output = T> + Send + 'static,
    T: Send + 'static,
{
    . . .
}
```

We can see that we pass in a future and return a task. Here, the future needs to implement the `Future` trait for the same outcome type of the task that is returned from the function. We can also see that the future needs to implement the `Send` trait, meaning that the future can be sent between threads. This makes sense as we will have another thread processing async tasks.

We must also note that the future needs to have a `'static` lifetime, meaning it must have a lifetime of the entire program. This is because a developer has the right to send a future to be spawned and never have to wait for that future to be finished. We could also have a future that just never completes. Therefore, we could have async tasks being processed for the entire lifetime of the program. We can also see that the task needs to be sent between threads and that it should also have a `'static` lifetime for the same reasons as the future.

Inside our `spawn_task` function, we need to define our async queue, which is done using the following code:

```
static QUEUE: LazyLock<flume::Sender<Runnable>> = LazyLock::new(|| {
    . . .
});
```

Our queue is a flume channel with runnables in it. A channel is essentially a queue with a sender and receiver that can traverse threads, enabling us to send variables to other threads. Every time we spawn a task, we get a runnable handle that the runnable uses to poll the task to see whether it has completed.

Our `QUEUE` variable is the sender (also known as a transmitter) that we use to send futures to our executor thread to be scheduled and run. We can also see that there is a `Lazy::new`. This means that the code inside the lazy block only gets evaluated once and the outcome and state of that lazy block are stored throughout the lifetime of the program. So, no matter how many times we call the `spawn_task` function, we only construct one queue, once. All other `spawn_task` function calls after the initial call are referencing the same queue defined in the `spawn_task` function's initial call.

Inside our lazy block, we define our channel, spawn a thread, and process incoming futures from the channel in that thread, returning the sender so that our code outside the lazy block can send messages to our executor thread. This transport mechanism is achieved by the following code:

```
static QUEUE: Lazy<flume::Sender<Runnable>> = Lazy::new(|| {
    let (tx, rx) = flume::unbounded::<Runnable>();
    thread::spawn(move || {
        while let Ok(runnable) = rx.recv() {
            let _ = catch_unwind(|| runnable.run());
        }
    });
    tx
});
```

The `runnable.run()` function is polling the future. We will cover polling in the next subsection when we build our own sleeping future. `catch_unwind` basically catches any panics (a panic refers to throwing an error). We do not need the `catch_unwind` function to get our system running; however, it would be a brittle system if a developer builds an async function that errored, and this erroring broke our entire async runtime.

Finally, we need to schedule our runnable when we send it to our queue to be polled, returning a task handle so the developer spawning the task can wait for the result with the following code:

```
let schedule = |runnable| QUEUE.send(runnable).unwrap();
let (runnable, task) = async_task::spawn(future, schedule);
runnable.schedule();
return task
```

Our `spawn_task` function is now complete. Let us move on to building our own sleep future.

Building an async sleep future

In the previous step, we mentioned polling as a concept. In Rust, a future is essentially a coroutine (a component that allows the execution to be suspended and resumed) that keeps its own state and can be polled. When the future is polled, the polling function can return either a ready or pending outcome. If the outcome is ready, when the future is completed, the return value is given to the context polling the future. If the future is pending, then the future is put into a cycle to be polled again. The exact implementations of the cycle to be polled can differ between async runtimes, but essentially, the async executor will come back to the future to poll it again until the future is ready.

Advanced runtimes such as `tokio` will interact with the operating system and wait for a signal, depending on the type of future. For instance, if we are making a network call, there is no use in continually polling the future, waiting for data to be received from the network card outside of the program. Instead, the async runtime will receive an alert from the network card that the response is in and will reanimate the future and poll it to completion. Achieving this ourselves is beyond the scope of this book, but it does shed light on how efficient these mature async systems are.

For our sleep future, we do not want to hold up the executor; otherwise, the executor will not be able to process other futures while our sleeping future is blocking the executor thread. We can achieve this using timestamps instead of a sleep function. This approach means that we create a timestamp of when the sleep future was created, and we also give a duration of the sleep. These two values mean that we can check the current time of when the future is polled and calculate the time elapsed. We can see this in action when we create our poll function.

Before we do that, we must define our async sleep struct with the following code:

```
struct AsyncSleep {
    start_time: Instant,
    duration: Duration,
}

impl AsyncSleep {
    fn new(duration: Duration) -> Self {
        Self {
            start_time: Instant::now(),
            duration,
        }
    }
}
```

Now that our struct is defined, we can implement the Future trait for our struct with the following code:

```
impl Future for AsyncSleep {
    type Output = ();

    fn poll(self: Pin<&mut Self>, cx: &mut Context<'_>)
        -> Poll<Self::Output> {
        . . .
    }
}
```

Pin stops the AsyncSleep struct from moving in memory, so we do not hit the wrong memory address when polling the future. Pinning does not dictate whether the data is stored on the heap or stack; it merely tells the compiler that the memory is not to be moved. Pinning is needed because futures get moved into other processes to get polled to completion. However, if we want to poll our future again and the future has moved, we run the risk of accessing a memory address that no longer has the future, resulting in undefined behavior. Pinning ensures that the future stays at the same memory address.

The poll function itself is called until the future is completed. The total number of polls can vary depending on whether you have an external signal to reanimate the future, the number of other futures in the queue being polled (the more futures in the system, the less frequently we can poll them), and the complexity of the future being polled.

The context is where we get the waker so that we can wake up the future so that it can be polled again. Unfortunately, we must remember that this is a web programming book, and we are covering async to understand web programming at a deeper level. Entire books have been written on async Rust; if we keep going deeper into async, it would consume the whole book, so we will move on to the logic inside our poll function that takes the following form:

```
let elapsed_time = self.start_time.elapsed();
if elapsed_time >= self.duration {
    Poll::Ready(())
} else {
    cx.waker().wake_by_ref();
    Poll::Pending
}
```

Here, we can see that we calculate the elapsed time. If the time has elapsed, we then return a ready. If not, we return a pending. Calculating the elapsed time means that our sleeping future will temporarily block the executor to see whether the time has elapsed. If the time has not elapsed, then the executor moves on to the next future to poll and will come back later to check whether the time has elapsed again by polling it again. This means that we can process thousands of sleep functions on our single thread. With this in mind, we can now move on to running our async code in our main thread.

Running our async sleep future

When it comes to calling our async sleep future, we can stack our future in another future, which is an async function with the following code:

```
async fn sleeping(label: u8) {
    println!("sleeping {}", label);
    AsyncSleep::new(Duration::from_secs(3)).await;
    println!("progressing sleep {}", label);
    AsyncSleep::new(Duration::from_secs(2)).await;
    println!("done sleeping {}", label);
}
```

Here, we can see that our async function merely sleeps twice and prints out simple progress statements as the async function progresses.

We can test our async code in our main function with the following code. Note that we have removed the async definition for our main function and removed tokio, as we are using our own async task queue:

```
fn main() {
    let handle_one = spawn_task(sleeping(1));
    let handle_two = spawn_task(sleeping(2));
    let handle_three = spawn_task(sleeping(3));

    println!("before the sleep");
```

```
std::thread::sleep(Duration::from_secs(5));  
println!("before the block");  
  
future::block_on(handle_one);  
future::block_on(handle_two);  
future::block_on(handle_three);  
}
```

Here, we can see that we spawn three async tasks that are all going to sleep for five seconds each. We then block the main thread. This is to test that our async sleeps are truly async. If they are not, then we will not get through all our sleep functions before the main sleep is finished.

We have now implemented everything we need to run our program. In running it, we get the following printout:

```
before the sleep  
sleeping 1  
sleeping 2  
sleeping 3  
progressing sleep 1  
progressing sleep 2  
progressing sleep 3  
done sleeping 1  
done sleeping 2  
done sleeping 3  
before the block
```

Here, we can see that all our sleep functions execute before the sleep in the main thread has finished. This means that our system is truly async! However, if this was a bit of a headache for you, do not worry. In Rust web programming, you will not be asked to manually implement async code for your server. Most of the async functionality has been built for you, but having a grasp of what is going on under the hood is beneficial.

Now that we have seen what async is, we need to understand it in the wider context of web programming. We will start doing this by exploring high-level concepts of tokio.

Exploring high-level concepts of tokio

There is some contention about this, but tokio is the main async runtime that web frameworks run on. Unlike our single-threaded async queue, tokio has multiple threads with task stealing, as seen in *Figure 4.2*:

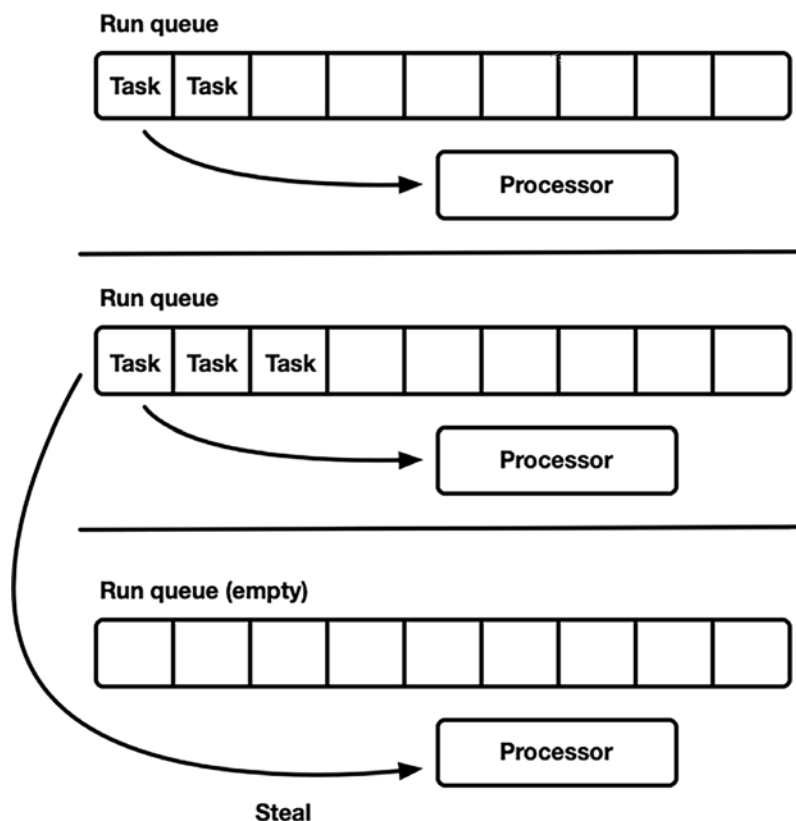


Figure 4.2 – Speeding up the tokio runtime [source: tokio documentation (2019) (<https://tokio.rs/blog/2019-10-scheduler>)]

Here, we can see that the tokio runtime has multiple worker threads that all have their own queues. On top of multiple workers, tokio also implements “task stealing.” This is where the worker will steal a task from another queue if that worker does not have its own tasks to process. In the first section of this chapter, we used tokio to display some functionality of async runtimes. However, we restricted the number of worker threads to one to avoid the task stealing from masking how blocking works. We can see that our simple implementation of an async task queue does not really match up to the full features that a runtime such as tokio offers. Unless you have some specific needs for your runtime, it is advised that you use an established runtime such as tokio for your application.

A concept to seriously consider regarding the choice of runtime is the interaction of other crates. For instance, an HTTP request is a good use of async, as there is no CPU usage when waiting for a response from a server. Because the CPU is idle, it makes sense to switch context and process something else when waiting for a response from the server. This is why most of the networking libraries and crates support async.

However, when someone builds out a library that supports async, the library must support that specific runtime. This may be seen as controversial by some, but when a library supports async, chances are that they have implemented async interfaces for the tokio runtime.

To get an appreciation for how tokio and other async runtimes are used in web programming, we can look at the standard example of a TCP server supported by tokio using the following code.

To avoid bloating the chapter, we are not going to try and run the example code. We're just looking at it to understand the flow of handling:

```
#[tokio::main]
async fn main() -> Result<(), Box<dyn Error>> {
    let addr = env::args()
        .nth(1)
        .unwrap_or_else(|| "127.0.0.1:8080".to_string());

    let listener = TcpListener::bind(&addr).await?;
    println!("Listening on: {}", addr);

    loop {
        // Asynchronously wait for an inbound socket.
        let (mut socket, _) = listener.accept().await?;

        tokio::spawn(async move {
            // process the TCP request
        });
    }
}
```

Here, we can see that the TCP listener is defined in the main thread. We also run a continuous loop in the main thread listening for new incoming TCP requests. When we get a new TCP request, we spawn a new async task to process that request, and then go back to listening for more incoming TCP requests. This means that our main thread is not held up processing requests. Instead, our main thread can spend all the time listening for requests.

If you want the code to run, you must build your own async request handle function that handles the incoming connection. In this handle function, you would have to read from the TCP stream and write to that same stream.



You may have noticed that there is a standard library TCP listener. The standard library TCP listener is blocking, meaning that the listener will hold up the thread until the listener receives an incoming message. A tokio TCP listener, on the other hand, will wait for a notification from the operating system that there is a message in the port that is being listened to. While the listener is waiting, the tokio runtime can process other async tasks.

Seeing as HTTP is built on top of TCP (although HTTP3 is built on top of UDP), we can assume that most HTTP web frameworks will be passing their requests off to the async runtime to be processed. This highlights a potentially big problem. If our async tasks handling a request have a long blocking component, this could severely slow down our ability to process new incoming requests, so it's best to avoid CPU-intensive tasks for async programming. You will see this advice a lot online. The one-liner of not performing CPU-heavy tasks in async is not true for all use cases, however. Essentially, an async runtime gives you an easy way to access a thread pool and for those worker threads to cycle through async tasks, polling them and coming back to them if there are multiple awaits in the async task. If the context makes sense, you can use your async runtime how you like. However, in terms of using an async runtime to handle incoming network requests, we should avoid CPU-heavy async tasks; otherwise, we will block new incoming requests from being processed. This does not mean that we can never have CPU-intensive code for our API endpoints, though.

In tokio, we can handle blocking tasks with the `tokio::spawn_blocking` function. This essentially passes the blocking async task to another thread pool, so our handling of new incoming requests is not blocked. To get a feel for how tokio handles incoming HTTP requests, we can move on to implementing an HTTP server with Hyper.

Implementing an HTTP server in Hyper

The Hyper crate is essentially a layer above tokio. While we will not be using the Hyper crate throughout the book, as the low-level nature of the Hyper crate would require us to write a lot of boilerplate code, exploring a simple Hyper server will help us to appreciate the basic handling of an HTTP request. To build our HTTP server, we will need to carry out the following steps:

1. Setting up the Hyper server environment.
2. Building the Hyper request handle function.
3. Building the main function for running the Hyper server.
4. Implementing HTTP2 for our Hyper server.

We will start by setting up our environment.

Setting up the Hyper server environment

We begin this step by adding the following dependencies:

```
[dependencies]
hyper = { version = "1.2.0", features = ["full"] }
tokio = { version = "1.36.0", features = ["full"] }
http-body-util = "0.1.1"
hyper-util = { version = "0.1.3", features = ["full"] }
```

With these dependencies, we are going to use the following structs and traits in our `main.rs` file with the following code:

```
use std::convert::Infallible;
use std::net::SocketAddr;
```

```

use http_body_util::Full;
use hyper::body::Bytes;
use hyper::server::conn::http1;
use hyper::service::service_fn;
use hyper::{Request, Response};
use hyper_util::rt::TokioIo;
use tokio::net::TcpListener;

```

Now we can start writing server code, starting with the handle.

Building the Hyper request handle function

We will see how the preceding code is used as we build the server. Before we write any code in the main function, we can build our function that handles all the incoming requests with the following code:

```

async fn handle(body: Request<hyper::body::Incoming>)
-> Result<Response<Full<Bytes>>, Infallible> {
    println!("Received a request");
    println!("Headers: {:?}", body.headers());
    println!("Body: {:?}", body.into_body());
    Ok(Response::new(Full::new(Bytes::from("Hello, World!"))))
}

```

Here, we can see that we just print out the header of the request and the body. After printing out the request data, we merely return a simple hello world message. Our handle function would be a good place to perform routing to other async functions, but for our example, we are just going to return the simple message, no matter what endpoint or method you use. We can see that the error type is `Infallible`. This basically means that it cannot happen. `Infallible` is essentially a placeholder to map results. This makes sense in our case as you can see that we have no unwraps to access the data we are printing. Throughout the book, we will build our own custom error handling, but for this simple example, `Infallible` is useful to us. We can now use this handle function in the main function for our server.

Building the main function for running the Hyper server

For our main function, we run our server with the following code:

```

#[tokio::main]
async fn main()
-> Result<(), Box<dyn std::error::Error + Send + Sync>> {
    let addr = SocketAddr::from(([127, 0, 0, 1], 3000));
    let listener = TcpListener::bind(addr).await?;
    loop {
        let (stream, _) = listener.accept().await?;
        let io = TokioIo::new(stream);
    }
}

```

```
        tokio::task::spawn(async move {  
            . . .  
        });  
    }  
}
```

As we have seen before, we are binding a TCP listener to a socket and then running an infinite loop listening to incoming requests, and spawning async tasks to handle these requests. The difference here is the following line:

```
let io = TokioIo::new(stream);
```

The `TokioIo` struct is from the `Hyper` utils crate. This `TokioIo` struct is essentially a wrapper around the implementation of the tokio IO traits.

Now that we have adapted the tokio stream of bytes from the request, we can work with the `Hyper` crate. Inside our `tokio::task::spawn` block, we handle the stream of bytes with the following code:

```
if let Err(err) = http1::Builder::new()  
    .serve_connection(io, service_fn(handle))  
    .await  
{  
    println!("Error serving connection: {:?}", err);  
}
```

Here, we serve this HTTP1 connection to our `handle` function. If we run our server using the `cargo run` command, we can make an HTTP GET request by putting the server URL in the browser to get the result shown in *Figure 4.3*.



Figure 4.3 – Browser view of our Hyper server HTTP request

If we look at the terminal that is running our server, the data in the request header and body will be printed out (providing the printout in the book would needlessly bloat the chapter). We can see that our server is running as we expected, but what about HTTP2?

HTTP2 is a binary protocol. With HTTP2, you can also do multiplexing and encryption, which is the HTTPS that you see on most (nearly all) modern websites. We will cover HTTPS in *Chapter 17*. Right now, we are going to bypass the encryption and just implement HTTP2 for our Hyper server.

Implementing HTTP2 for our Hyper server

We start by using the following:

```
use hyper::server::conn::http2;
use hyper::rt::Executor;
use std::future::Future;
```

We now need to implement our async runtime adapter for Hyper so that our Hyper server can send the requests to the async runtime, using the following code:

```
#[derive(Clone)]
struct TokioExecutor;

impl<F> Executor<F> for TokioExecutor
where
    F: Future + Send + 'static,
    F::Output: Send + 'static,
{
    fn execute(&self, future: F) {
        tokio::spawn(future);
    }
}
```

We can put whatever we want in the execute function if it is spawning a task on an async runtime. As we can see, the traits needed for the future being passed into the execute function are the same requirements for the spawn_task function we defined when building our own async task queue.

We can now swap our HTTP1 builder out for our HTTP2 builder with the following code:

```
if let Err(err) = http2::Builder::new(TokioExecutor)
```

However, if we run our server again and try to refresh our browser page, we will get a response that the page is not working, as depicted in *Figure 4.4*:



This page isn't working

127.0.0.1 sent an invalid response.

ERR_INVALID_HTTP_RESPONSE

Figure 4.4 – Browser view of our Hyper server HTTP2 request

This is because, by default, most browsers implement HTTPS connections when performing an HTTP2 request. Seeing as we are not implementing HTTPS right now, we must test our connection using cURL to bypass this step with the following command:

```
curl --http2-prior-knowledge http://127.0.0.1:3000/
```

The cURL terminal command then returns the hello world message. With this example, we can now see how async runtimes interact with web servers.

Summary

We have come to the end of our async exploration. As mentioned earlier in the chapter, we could keep going to the point of writing an entire book on async Rust; however, we have learned enough async Rust to navigate web programming in Rust efficiently.

We now understand what an async future is and how it passes through the async runtime. Furthermore, we also know how web servers utilize async runtimes to handle incoming network requests. Now that you have a deeper understanding of async, we are going to be handling HTTP requests in the next chapter with async functions and a web framework.

Questions

1. What is the difference between multiple threads and multiple async tasks?
2. How can I create an async sleep future that is non-blocking?
3. How does a web service typically use an async runtime to handle incoming network requests?
4. Why is it a bad idea to have CPU-intensive async functions for API endpoints?
5. How can we handle CPU-intensive async functions?
6. What is the overall high-level flow of an async task in an async task queue?

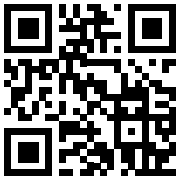
Answers

1. A thread has its own memory and processes CPU tasks. Multiple threads can process multiple CPU computations at the same time. Async tasks have their own state and can be polled to see whether they are completed or not. Async tasks are usually for non-blocking tasks, such as waiting for a response from a network. Because these async tasks are non-blocking, a single thread can handle multiple async tasks, looping through and polling tasks to see whether they are finished or not.
2. We can create a non-blocking sleep future by creating a struct that has a field with the time that the struct was created, and another field for the duration of the sleep. We then implement the Future trait for the struct where the poll function gets the current time, calculates the time elapsed from the created field, and returns a ready if the duration has passed, or a pending if the duration has not passed.

3. A typical web service creates a TCP listener on a port in the main thread. The service then starts an infinite loop in the main thread listening for incoming network requests. Once an incoming network request is accepted, the bytes from the network request are then passed into an async task, and that async task is passed to the async runtime to handle so that the main thread can go back to listening for incoming requests.
4. An async function is an async task that gets processed on the async runtime. If the async task is too CPU-intensive, it can block the worker threads of the async runtime, which will block the processing and progression of new incoming network requests.
5. We can handle CPU-intensive async functions by using the `tokio::spawn_blocking` function. This essentially passes the blocking async task to another thread pool, so our handling of new incoming requests is not blocked.
6. When an async task is spawned, it is set to a runnable and put on a queue. One or more worker threads will take tasks from that queue and poll them. If the task is finished, then it is ready and no longer polled, and the result of that async task can be returned to the context that has the handle of that task. However, if the async task is not complete, it will return a pending and will be polled again in time by a worker thread.

Learn more on Discord

To join the Discord community for this book – where you can share feedback, ask questions to the author, and learn about new releases – follow this QR code: <https://packt.link/EaKXL>



5

Handling HTTP Requests

After the detour of understanding `async` on a deeper level, we are now going back to the project that we were working on in *Chapter 3*. So far, we have structured our to-do module in a flexible, scalable, and reusable manner. However, this can only get us so far in terms of web programming. We want our to-do module to reach people quickly without the user having to install Rust on their own computers. We can do this with a web framework.

In this chapter, we will build on the core logic of our to-do items and connect this core logic to a server. By the end of this chapter, you will be able to build a server that has a data access layer, core layer, and networking layer. You will also get some exposure in handling errors across all Cargo workspaces and refactoring some of our code as our requirements for our server get more defined over time.

We will cover the following topics in this chapter:

- Launching a basic web server
- Connecting the core to the server
- Refactoring our to-do items
- Serving our to-do items
- Handling errors in API endpoints

We will start this journey by building a simple server that accepts incoming HTTP requests.

Technical requirements

You can download the example project and code for this book by following the instructions in the **Download the example code files** section in the *Preface* of this book.

This chapter's code files, as well as the server code we created in *Chapter 3*, are included in the downloadable code bundle.

Launching a basic web server

In this section, we are merely going to get an Actix server running using the Actix Web framework before connecting our server to our core code.



Why Actix Web?

This is now the third edition of the book. Several people have emailed me asking about a range of different web frameworks. The truth is, the choice of framework is close to trivial when it comes to implementing the API of that web framework. If you structure your code well, you should be able to switch these frameworks with minimal effort. Throughout this book, we will structure our code so we can use multiple different web frameworks at the same time. Actix is chosen for the book because we must choose one framework to demonstrate web programming concepts, and Actix is the most mature and stable web framework at the time of writing this book.

The GitHub repository of this book will also have the implementations of Axum, Rocket, and Hyper, but they will not be covered directly in the book to avoid bloat. However, the concepts covered will be the same, with the same file structure. This book will highlight differences in the other frameworks when they come up, but you will have to visit the GitHub repository for the full code examples.

Before we write any code, we must define our file layout.

Defining our file layout

For our server, we are going to create a networking layer for our to-do nanoservice and put a Cargo project for our Actix server in that networking layer directory. With our server module, the file layout for our nanoservice should take the following form:

```
├─ Cargo.toml
├─ to_do
│   ├── core
│   │   └─ . . .
│   ├── dal
│   │   └─ . . .
│   └─ networking
│       └─ actix_server
│           ├── Cargo.toml
│           └─ src
│               └─ main.rs
```

With this new layout, the `Cargo.toml` file at the root of our project should have the following workspaces defined:


```
# File: ./Cargo.toml
[workspace]
resolver = "2"
members = [
    "to_do/core",
    "to_do/dal",
    "to_do/networking/actix_server"
]
```

Declaring all the workspaces in the root `Cargo.toml` file enables us to run a cargo command in the root of our system, and have this cargo command apply to all the workspaces at the same time. The workspaces also help IDEs map the code when performing syntax highlighting and checking.

We are nearly finished with the boilerplate code. Before we write any server code, we need to define the server's dependencies in the server `Cargo.toml` file with the following:

```
# File: ./to_do/networking/actix_server/Cargo.toml

[package]
name = "to-do-actix-server"
version = "0.1.0"
edition = "2021"

[dependencies]
tokio = { version = "1.36.0", features = ["full"] }
actix-web = "4.9.0"
```

We can now write our server `main.rs` file.

Writing our server code

We begin with the following structs and traits:

```
//! File: ./to_do/networking/actix_server/src/main.rs
use actix_web::{web, App, HttpServer, Responder, HttpRequest};
```

We will see how these structs and traits are used as and when we use them.

Recall from the previous chapter, servers typically handle incoming requests by passing these requests as async tasks into the async runtime to be handled. It should not be a surprise that our server API endpoint is an async task defined by the following code:

```
// File: ./to_do/networking/actix_server/src/main.rs
async fn greet(req: HttpRequest) -> impl Responder {
    let name = req.match_info().get("name").unwrap_or("World");
    format!("Hello {}!", name)
}
```

With the preceding code, we can see that our API endpoint receives the HTTP request and returns anything that has implemented the `Responder` trait. We then extract the name from the endpoint of the URL or world string if the name is not in the URL endpoint. Our view then returns a string.

To see what we can automatically return as a response, we can check the `Responder` trait in the Actix Web docs and scroll down to the *Implementations on Foreign Types* section, as seen in Figure 5.1:

Implementations on Foreign Types

```
[+] impl Responder for &'static str
[+] impl Responder for &'static [u8]
[+] impl Responder for &String
[+] impl Responder for Cow<'_, str>
[+] impl Responder for String
[+] impl Responder for Vec<u8>
[+] impl Responder for ResponseBuilder
[+] impl Responder for ByteString
[+] impl<R, E> Responder for Result<R, E>
    where
        R: Responder,
        E: Into<Error>,
[+] impl<R: Responder> Responder for (R, StatusCode)
[+] impl<R: Responder> Responder for Option<R>
```

Figure 5.1 – Implementations of Foreign types [Source: Actix Web (2024), https://docs.rs/actix-web/latest/actix_web/trait.Responder.html#foreign-impls]

Figure 5.1 shows that we can essentially return strings and bytes as a response to the HTTP request.

Now that we have our API endpoint, we can build our server with the following code:

```
// File: ./to_do/networking/actix_server/src/main.rs
#[tokio::main]
async fn main() -> std::io::Result<()> {
    HttpServer::new(|| {
        App::new()
            .route("/", web::get().to(greet))
            .route("/{name}", web::get().to(greet))
            .route("/say/hello", web::get().to(||
```

```
        async { "Hello Again!" })})
    })
    .workers(4)
    .bind("127.0.0.1:8080")?
    .run()
    .await
}
```

In the preceding code, we can see that we are running `HttpServer` after constructing it using the `HttpServer::new` function. Knowing what we know now, we can see that we have passed in a closure that returns the `App` struct. We can see that we have used several different ways to define a view. We defined a view by building a function or an async closure. We assign the `greet` function that we have created to our application server as the route view with the `.route("/", web::get().to(greet))` command. We also see that we can pass in the name from the URL to our `greet` function with the `.route("/{name}", web::get().to(greet))` command. Finally, we pass in a closure into the final route. All of this enables us to route incoming requests to the right function to be processed, depending on the URL and HTTP method. Our server is now ready to run.

Running our server

With our configuration, let's say we run the following command:

```
cargo run -p to-do-actix-server
```

The `-p` parameter tells Cargo which Cargo workspace to run. Running our server workspace gives us the following printout:

```
Finished dev [unoptimized + debuginfo] target(s) in 0.21s
Running `target/debug/to-do-actix-server`
```

We can see in the preceding output that, right now, there is no logging, but we are running a basic web server. This is expected, and we will configure logging later.

Now that our server is running, we should expect the following URL inputs and outputs on the browser:

```
http://127.0.0.1:8080/
Hello World!
```

```
http://127.0.0.1:8080/Maxwell
Hello Maxwell!
```

```
http://127.0.0.1:8080/say/hello
Hello Again!
```

And there we have it!

If you have cloned the Github repository, we can run the Axum version of this server with the following command:

```
cargo run -p to-do-axum-server
```



You can run the Rocket server with the following command:

```
cargo run -p to-do-rocket-server
```

We can see with the Rocket example that logging is already configured. Rocket is basically a “batteries included” framework where things like logging are already set up.

You can run the Hyper server with the following command:

```
cargo run -p to-do-hyper-server
```

Our server is running, but it is also isolated. In the next section, we are going to connect our server to the core module.

Connecting the core to the server

When it comes to connecting our networking layer to our core layer, we map our api modules on both the core and networking layers in the same way to avoid confusion.

Defining our core file layout

To map both layers, we must add the following new files for the core:

```
├─ to_do
│   └─ core
│       └─ src
│           ├── api
│           │   ├── basic_actions
│           │   │   ├── create.rs
│           │   │   ├── delete.rs
│           │   │   ├── get.rs
│           │   │   ├── mod.rs
│           │   │   └─ update.rs
│           │   └─ mod.rs
│           ├── lib.rs
│           └─ . . .
```

We can see that we are adding a file for each basic action that we expect to perform on a to-do task. Also, note that we added a `src/lib.rs` file. This turns our core module into a lib so it can be mounted into the networking applications. The networking layer will interact with the core module through the `src/lib.rs` file.



If we wanted to, we could now delete the `src/main.rs` file as our core module now has a `lib` interface. However, we will be using this for a quick test later in the chapter, so hold onto the file until that test is done.

The module file in the basic actions module now takes the following form:

```
#!/ File: to_do/core/src/api/basic_actions/mod.rs
pub mod create;
pub mod get;
pub mod delete;
pub mod update;
```

We are keeping the files public as we want the `src/lib.rs` file to expose them to another workspace that uses our core module, with the following code:

```
#!/ File: to_do/core/src/lib.rs
pub mod api;
pub mod structs;
pub mod enums;
```

Our core module is now ready to be used in other Cargo workspaces.

For our server to accommodate our core module, we need to add the following files to our networking layer that will interact with our core layer. This networking layer essentially gets an HTTP request, processes that request, and feeds the extracted data from the HTTP request into the core functions:

```
└─ to_do
  └─ networking
    └─ actix_server
      └─ ...
        └─ src
          ├── api
          │   ├── basic_actions
          │   │   ├── create.rs
          │   │   ├── delete.rs
          │   │   ├── get.rs
          │   │   ├── mod.rs
          │   │   └─ update.rs
          │   └─ mod.rs
          └─ main.rs
```

Here, we are keeping our `main.rs` file because this Cargo workspace is not going to be used elsewhere yet. However, we are going to be using the `main.rs` file to run the server. With our files defined, we can now move on to coding our networking layer

Coding our networking layer

It should not be a surprise that the module file in the basic actions takes the following form:

```
///! File: to_do/networking/actix_server/src/api/basic_actions/mod.rs
pub mod create;
pub mod get;
pub mod delete;
pub mod update;
```

In the api module file, we expose the basic actions with the following code:

```
///! File: to_do/networking/actix_server/src/api/mod.rs
pub mod basic_actions;
```

We declare the api module in our main.rs file with the following code:

```
///! File: to_do/networking/actix_server/src/main.rs
...
mod api;
...
```

All our code is now stitched up and ready to talk to each other. The only thing left to do is declare our core module in the Cargo.toml file with the following code:

```
#! File: to_do/networking/actix_server/Cargo.toml
...
[dependencies]
tokio = { version = "1.36.0", features = ["full"] }
actix-web = "4.9.0"
to-do-core = { path = "../core" }
```

We can see that our to-do-core dependency points to where the Cargo.toml file of the to-do-core module is, relative to the networking Cargo.toml file. Building with this relative path will result in an error if there is no Cargo.toml file with the package named to-do-core in that path. If the Cargo.toml file is situated in that relative path, the Rust code of the to-do-core module will be compiled into our networking layer and referenced just like any other dependency. We can now run a cargo build command. If the build passes, we know that everything is going to play nicely with each other. This is one of the many reasons why I love Rust. The compiler makes it hard for beginners to do basic things; however, when you start growing the complexity of your system, when the system compiles, you know that you are not going to get basic bugs in production.



Wait, isn't this over-engineered?

It is reasonable to have some alarm bells going off at what we have just done. Our nanoservice now consists of three Cargo workspaces, and all we have so far is a read and write to a basic JSON file, an isolated create function that talks to the JSON file, and an isolated server that just returns a couple of strings. We have produced a lot of files and directories just for this functionality.

While there is some utility to the term “over-engineered,” it is vague and is at risk of being abused. In my experience, a lot of people do not look at the bigger picture when accusing an approach of being over-engineered. As we stick with our approach, you will get to experience the flexibility that we have when running our application locally or on a server. The ease of swapping out layers so our nanoservice will be able to run as a microservice in its own Docker container, or just compile into another Cargo workspace, will prevent over-engineering in the future, as you will not have to run multiple Docker containers to develop against the entire system. And as systems get bigger with multiple developers and teams, trust me, you will thank your past self that you took this approach.

Now that everything is compiling, we can move on to serving our to-do items, but we might need to refactor them first.

Refactoring our to-do items

If we want to serve our to-do items, we need to get our server to talk to our core, which will then get all the to-do items from our JSON file. Right now, just to represent either a pending or a done item, we need to navigate between three structs. Our `TaskStatus` enum handles the logic and serialization of the status of these structs.

Now is a good time to look at the structs in our core module and have a quick think on how you could represent them in a single vector to be displayed in the browser.

If you did try and work out how to convert the to-do items into a single vector, you may have considered wrapping the items in an enum and then implementing a serialization trait to display these to-do items. This is like how we handle the writing of the to-do item to a file. However, this is a good time to listen to those alarm bells. We are having to do a lot of work to handle a variance in a status title. This complexity does not give us any advantages, so we need to act now to prevent this over-engineered approach from getting more embedded into our system, as it will be harder to rip out the longer we leave it.

David Farley (author of the book *Modern Software Engineering*) says, “the outcome of a surgical procedure is not better or safer because the surgeon uses a blunter knife.” Likewise, we do not have a safe system because we do not touch or change chunks of our code. In fact, it’s the opposite. A sign of a safe, well-designed system that is handled by skilled engineers is the ability to confidently change chunks of the system as we find out more, and still manage to keep the system stable. Right now, let us retreat to a safer, simpler position so the handling of our to-do items is easier to manage. We can do this by completely ripping out the `to_do/core/src/structs/` directory and replacing it with a `to_do/core/src/structs.rs` file. Inside this file, we can define a to-do item with the following code:

```

//! File: to_do/core/src/structs.rs
use std::fmt;
use serde::{Serialize, Deserialize};
use crate::enums::TaskStatus;

#[derive(Serialize, Deserialize, Debug, Clone)]
pub struct TodoItem {
    pub title: String,
    pub status: TaskStatus
}

```

We can directly implement the serialization traits because our `TaskStatus` enum has already implemented them. We also want our `TodoItem` struct to print out in the same way our enum wrapping our two different previous structs did so; we can do this by implanting the `Display` trait with the following code:

```

// File: to_do/core/src/structs.rs
impl fmt::Display for TodoItem {
    fn fmt(&self, f: &mut fmt::Formatter) -> fmt::Result {
        match self.status {
            TaskStatus::PENDING => write!(
                f, "Pending: {}",
                self.title
            ),
            TaskStatus::DONE => write!(
                f, "Done: {}",
                self.title
            ),
        }
    }
}

```

We now only need to refactor our `create api` function with the following code:

```

//! File: to_do/core/src/api/basic_actions/create.rs
use crate::enums::TaskStatus;

```



```

use crate::structs::ToDoItem;

#[cfg(feature = "json-file-storage")]
use to_do_dal::json_file::save_one;

pub fn create(title: &str, status: TaskStatus)
    -> Result<ToDoItem, String> {
    let item = ToDoItem {
        title: title.to_string(),
        status
    };
    let _ = save_one(&title.to_string(), &item)?;
    Ok(item)
}

```

We can see that the create function is simpler than our previous implementation.

If you kept the `src/main.rs` file, we can clear our `tasks.json` file so that we have the same file contents after running the following commands test our program:

```

cargo run -- --title washing --status pending
cargo run -- --title coding --status done

```

We can then check our `tasks.json` JSON file, which should have the following content:

```

{
  "coding": {
    "title": "coding",
    "status": "DONE"
  },
  "washing": {
    "title": "washing",
    "status": "PENDING"
  }
}

```

Here, we see that we still can access the task with the title as the key, but the corresponding values contain the title and status of the task. We did not change any of the code in the data access layer. Instead, we are seeing how our to-do items are serialized. We are now ready to return all our to-do items to the browser.



Make sure that you keep the `task.json` file in the root of the entire project, next to the `Cargo.toml` file that defines the workspaces. We will use this JSON file for testing our get API endpoint.

Serving our to-do items

We are now at the stage of serving all our items to the browser. However, before we touch any of the server code, we need to add another struct from our core module. We can start by defining the JSON body.

Defining the JSON body

We need a container that houses two lists of items for the pending and done items, which takes the following form:

```
// File: to_do/core/src/structs.rs
use std::collections::HashMap;
...
#[derive(Serialize, Deserialize, Debug, Clone)]
pub struct AllToDoItems {
    pub pending: Vec<ToDoItem>,
    pub done: Vec<ToDoItem>
}
```

This struct enables us to structure the entire state of the to-do items and also serialize and deserialize the state, letting us save/load to JSON files, and also send data to the browser.

You might recall that we load the data from the JSON file in a HashMap form, meaning that we are going to need a `from_hashmap` function for our `AllToDoItems` struct. We can add this with the following code:

```
// File: to_do/core/src/structs.rs
impl AllToDoItems {
    pub fn from_hashmap(all_items: HashMap<String, ToDoItem>)
        -> AllToDoItems {
        let mut pending = Vec::new();
        let mut done = Vec::new();

        for (_, item) in all_items {
            match item.status {
                TaskStatus::PENDING => pending.push(item),
                TaskStatus::DONE => done.push(item)
            }
        }
        AllToDoItems {
            pending,
            done
        }
    }
}
```

We can now utilize our new container struct by building our `get_all` API function.

Defining the get all API

Our `get_all` function is defined with the following code:

```
// File: to_do/core/src/api/basic_actions/get.rs
use to_do_dal::json_file::get_all as get_all_handle;
use crate::structs::{
    TodoItem,
    AllToDoItems
};
pub async fn get_all() -> Result<AllToDoItems, String> {
    Ok(AllToDoItems::from_hashmap(
        get_all_handle::<TodoItem>()?
    ))
}
```

Wow, there is not much code there. However, we can see what type of data is being loaded as the value of the hashmap with the `get_all_handle::<TodoItem>()`. We exploit the `?` operator to reduce the need for a match statement, and then directly feed the data into the `from_hashmap` function.

Here, our core API function is fusing the logic of the core module and the data access layer. The data from our core API function is then passed to the Actix API function, which can then return the data to the browser, as seen in *Figure 5.2*:

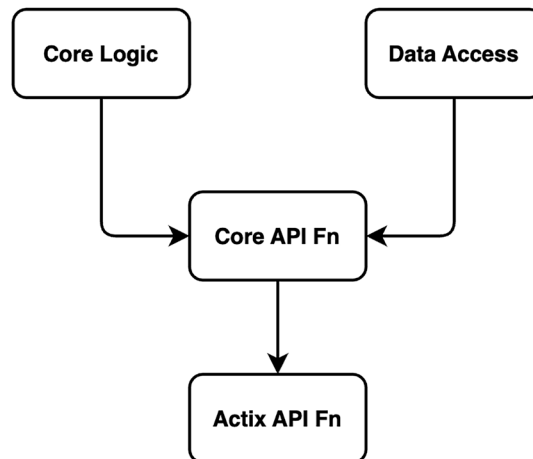


Figure 5.2 – Pathway for getting all to-do items

We can now build our Actix API function with the following code:

```
///! File: to_do/networking/actix_server
///! /src/api/basic_actions/get.rs
use to_do_core::api::basic_actions::get::get_all as get_all_core;
use actix_web::HttpResponse;

pub async fn get_all() -> HttpResponse {
    let all_items = match get_all_core().await {
        Ok(items) => items,
        Err(e) => return HttpResponse::InternalServerError().json(e)
    };
    HttpResponse::Ok().json(all_items)
}
```

We can see that we are renaming the core function that we import with a `_core`. This is just to make it easier on the eye. When another developer looks at the code, they will know exactly where the calls to the core are happening.

The match statement here does bloat the code a little bit, but we will handle this in the next section. We can see that if there is any error, we return it with an internal server error response code.

We now must connect our `get_all` async function to our server.

Connecting `get_all` to our server

We should keep our API module endpoint definitions isolated, as we want to be able to version control the different API modules. We can define the `get_all` URI with the following code:

```
///! File: to_do/networking/actix_server
///! /src/api/basic_actions/mod.rs
...
use actix_web::web::{ServiceConfig, get, scope};

pub fn basic_actions_factory(app: &mut ServiceConfig) {
    app.service(
        scope("/api/v1")
        .route("get/all", get().to(get::get_all))
    );
}
```

In the preceding code, our function is called a factory. A factory is a standard design pattern where objects are constructed or things are configured. In our factory pattern, we are assembling API endpoints and attaching them to our `ServiceConfig` struct, which is going to be mounted to the server. Factories are often used to encapsulate object creation or setup logic.

For our factory function, we can see that we pass in a mutable reference of a `ServiceConfig` struct. This struct allows bigger applications to split up configuration into different files, and it enables us to define things like views to the server in different fields. We then apply a service to the `ServiceConfig` struct.

We should define all backend API endpoints with `"/api/"` to differentiate backend endpoints from the frontend endpoints. We also have `"/v1/"` to facilitate versioning.



When there are breaking changes to the API, we should keep the previous version, but put the new breaking change API in the new version. We then alert users about the update in the version and give a date as to when the lower version will no longer be supported or removed.

Now that we have defined our endpoint factory for our basic actions, we need one last factory that collects all the other factories and calls them with the following code:

```
///! File: to_do/networking/actix_server/src/api/mod.rs
pub mod basic_actions;
use actix_web::web::ServiceConfig;

pub fn views_factory(app: &mut ServiceConfig) {
    basic_actions::basic_actions_factory(app);
}
```

With this, we only need to configure our server with our views factory. We do this with the following code:

```
///! File: to_do/networking/actix_server/src/main.rs
#[tokio::main]
async fn main() -> std::io::Result<()> {
    HttpServer::new(|| {
        App::new().configure(api::views_factory)
    })
    .workers(4)
    .bind("127.0.0.1:8080")?
    .run()
    .await
}
```

We see that we have defined the number of workers here. Workers are processes that have their own state and a copy of the application. They run by themselves with no concurrency issues, and the incoming requests are distributed between these workers. By default, the number of workers is the number of CPUs available.

If we run our server and hit our endpoint in the browser, we get the result seen in *Figure 5.3*:



Figure 5.3 – Return data from the get all API endpoint

And there we have it; our server is now returning the to-do items in a sorted fashion. However, remember that we were relying on a match statement in our web API function? We can reduce the need for this by defining our own error handling so we can exploit the `?` operator in our API function.

You can see in the GitHub repository how we handle different API endpoints for the following web frameworks:

- **Axum:** Our views factory is the same as for Actix Web; however, we pass in a Router as opposed to a ServiceConfig, and we return the Router struct from the factory with the new API endpoints added.
- **Rocket:** The views factory returns `routes![]` with the routes inside the square brackets. This macro is then mounted to the server using the `.mount("/api/v1", basic_actions_factory());` function. A macro above the API function itself defines the method and the last part of the URI.
- **Hyper:** Because Hyper is more low-level, we have a general handle function that accepts the incoming HTTP request. We then extract the URI and method, and match the method and URI using a match statement, passing the HTTP request to the mapped function.

We have nearly wrapped up this chapter. We just need to handle errors for the web server API function.

Handling errors in API endpoints

We are at the final hurdle of this chapter, where we want to get rid of the match statement in our API functions for our server. The match statement is a result of us handling an error. As our server grows, we would have to repeat the clunky handling of errors again and again, and there is a risk that we could mishandle the error. To prevent having to manually handle the errors for every API endpoint, we can bake the HTTP responses into our error handling. To do this, we can create an error type that can construct an HTTP response, so we can exploit the `?` operator in our web API function, and the server will simply respond with an HTTP response. However, this means that all our Cargo workspaces need to share the same custom error types. This is where a glue module will come in. A glue module is a crate that all other workspaces install. With this glue module, all Cargo workspaces can seamlessly pass the same error type between each other.

Glue modules can help us pass the same data types between different crates so they can communicate more effectively. We see this in *Figure 5.4*, where we can see that the different layers can pass errors between each other because every layer is referencing the error struct from the glue module.

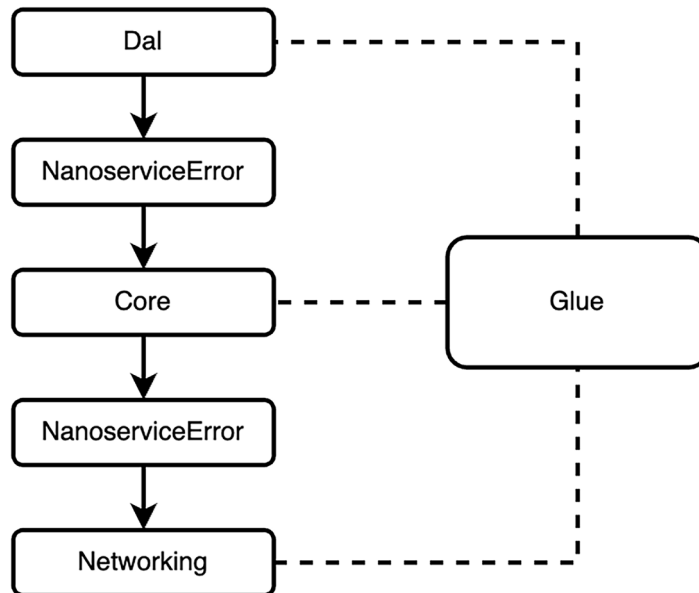


Figure 5.4 – Glue module enabling layers to pass errors between each other

It does not stop there. Throughout the book, we will build another server, which is the authentication server, to log in and manage authentication sessions for users. The authentication server will have a dal, core, and networking layer. Throughout the book, we will cover how to deploy servers separately. Because we have the layers, we will also cover how to compile the auth server into the to-do server binary. This enables the to-do server to directly call the auth core API functions because the auth server will also reference structs such as errors from the same glue module.



This is why I decided to call it a glue module, because it enables any of our systems to compile into each other and pass structs such as errors or auth tokens between each other. When rewriting the Rust Kubernetes cluster with Caroline Morton for the third time, we found that having a glue module helped with upgrading tens of servers, enabling them to talk to each other and compile together. Because certain packages were used in the glue module, we only had to update the package in the glue module, and all servers got the update.

Now that we understand the power that we get from a glue module, we can code our glue module.

Defining our glue module

Before we create our glue module, we must appreciate what is happening to our whole system. Our nanoservice is just sitting there. If we put the glue module next to our nanoservice, then it can get confusing as to what is what. To reduce the risk of confusion, we need to shift our nanoservice into its own directory called nanoservices. The nanoservices directory is where we will put other nanoservices, such as the authentication service.

Considering that the glue module is to be used everywhere, we can put the glue module in the root, giving us the following Cargo.toml file in our root:

```
[workspace]
resolver = "2"
members = [
    "glue",
    "nanoservices/to_do/core",
    "nanoservices/to_do/dal",
    "nanoservices/to_do/networking/actix_server"
]
```

Our glue module should have the file following layout:

```
└─ glue
   └─ Cargo.toml
   └─ src
      └─ errors.rs
      └─ lib.rs
```

We can now define the Cargo.toml file of the glue module with the following code:

```
# File: glue/Cargo.toml
[package]
name = "glue"
version = "0.1.0"
edition = "2021"
```



```
[dependencies]
actix-web = { version = "4.9.0", optional = true }
serde = { version = "1.0.197", features = ["derive"] }
thiserror = "1.0.58"

[features]
actix = ["actix-web"]
```

Here, we can see that the `actix-web` dependency is optional and only used if the `actix` feature is enabled. This is because we only want to compile Actix Web if we are directly mapping the error in the Actix server. There is no need for the core or data access layers to rely on the `actix-web` dependency. In the future, we can add features of other web frameworks as and when we need them.

In the `lib.rs` file of the `glue` module, we make the errors publicly available with the following code:

```
///! File: glue/src/lib.rs
pub mod errors;
```

We can now move on to our error definitions in our `src/errors.rs` file.

Building our custom error

First, we are going to need the following structs and traits:

```
///! File: glue/src/errors.rs
use serde::{Deserialize, Serialize};
use thiserror::Error;
use std::fmt;

#[cfg(feature = "actix")]
use actix_web::{
    HttpResponse,
    error::ResponseError,
    http::StatusCode
};
```

Note that we are only using the `actix-web` dependency if the `actix` feature is enabled.

We can now define the different statuses that the error can respond with using the following code:

```
// File: glue/src/errors.rs
#[derive(Error, Debug, Serialize, Deserialize, PartialEq)]
pub enum NanoServiceErrorStatus {
    #[error("Requested resource was not found")]
    NotFound,
    #[error("You are forbidden to access requested resource.")]
}
```

```

    Forbidden,
    #[error("Unknown Internal Error")]
    Unknown,
    #[error("Bad Request")]
    BadRequest,
    #[error("Conflict")]
    Conflict,
    #[error("Unauthorized")]
    Unauthorized
}

```

The `#[error("something")]` derive macro essentially implements the `Display` trait for our enum. If a field is decorated with `#[error("something")]`, then that field will display a “something” when the `Display` trait is utilized.

We can now define our nanoservice error with the following code:

```

// File: glue/src/errors.rs
#[derive(Serialize, Deserialize, Debug, Error)]
pub struct NanoServiceError {
    pub message: String,
    pub status: NanoServiceErrorStatus
}
impl NanoServiceError {
    pub fn new(message: String, status: NanoServiceErrorStatus)
        -> NanoServiceError {
        NanoServiceError {
            message,
            status
        }
    }
}
}

```

Next, we need to implement a response trait for our nanoservice error. We start by implementing the `Display` trait for our nanoservice error with the following code:

```

// File: glue/src/errors.rs
impl fmt::Display for NanoServiceError {
    fn fmt(&self, f: &mut fmt::Formatter<'_>) -> fmt::Result {
        write!(f, "{}", self.message)
    }
}
}

```

And with this, we can now implement the `ResponseError` trait for the nanoservice error. We do this with the following code:

```
// File: glue/src/errors.rs
#[cfg(feature = "actix")]
impl ResponseError for NanoServiceError {
    fn status_code(&self) -> StatusCode {
        . . .
    }
    fn error_response(&self) -> HttpResponse {
        . . .
    }
}
```

Implementing the `ResponseError` trait for our `NanoServiceError` struct enables the `NanoServiceError` to be converted into an HTTP response with a message and error code. This means that in our networking layer, if our core function returns an error, we can just send that error directly back to the client!

For our status code function, we need to merely map our status number with the Actix Web status code, as follows:

```
// File: glue/src/errors.rs
fn status_code(&self) -> StatusCode {
    match self.status {
        NanoServiceErrorStatus::NotFound =>
            StatusCode::NOT_FOUND,
        NanoServiceErrorStatus::Forbidden =>
            StatusCode::FORBIDDEN,
        NanoServiceErrorStatus::Unknown =>
            StatusCode::INTERNAL_SERVER_ERROR,
        NanoServiceErrorStatus::BadRequest =>
            StatusCode::BAD_REQUEST,
        NanoServiceErrorStatus::Conflict =>
            StatusCode::CONFLICT,
        NanoServiceErrorStatus::Unauthorized =>
            StatusCode::UNAUTHORIZED
    }
}
```

For our error response function, we need to construct an HTTP response from our status and message with the following code:

```
// File: glue/src/errors.rs
fn error_response(&self) -> HttpResponse {
```

```

let status_code = self.status_code();
HttpResponse::build(status_code).json(self.message.clone())
}

```

And our error can now construct an HTTP response.



For the other web frameworks, we can implement the following traits for our custom error:

- **Axum:** IntoResponse trait
- **Hyper:** A custom trait that just converts to a response
- **Rocket:** Responder trait

You can see these implementations in the GitHub repository for this chapter in the errors file of the glue module. Each implementation is guarded by a feature flag, meaning that you do not have to install dependencies of other frameworks, just the framework you choose.

Our custom error is now ready to be used. We now need to build some macros to help us map the errors in our system.

Mapping our custom error

We know that there are functions that return different types of errors that are out of our control. For instance, reading a file would rely on functions that we did not write. The file reading functions could return errors such as an I/O error. This is where the `map_err` function comes in. Basically, the `map_err` function converts one type of error into another type of error. For example, if we were to open a file, we could map the potential error of that file open operation to a nanoservice error with the following code:

```

let mut file = File::open(&path).map_err(|err|
    NanoServiceError::new(
        format!(
            "Failed to open file: {} - {}",
            path.as_ref().display(),
            err
        ),
        NanoServiceErrorStatus::Unknown,
    )
)?;

```

However, we do not want to have to write the `map_err` function every time we want to convert an error to a nanoservice error. We can prevent repetitive code with the following macro:

```

// File: glue/src/errors.rs
#[macro_export]
macro_rules! safe_eject {
    ($e:expr, $err_status:expr) => {
        $e.map_err(|x| NanoServiceError::new(

```

```

        x.to_string(),
        $err_status)
    )
};
($e:expr, $err_status:expr, $message_context:expr) => {
    $e.map_err(|x| NanoServiceError::new(
        format!("{: {}}", $message_context, x.to_string()),
        $err_status
    )
)
};
}

```

Here, we can see that there are two arms to the macro. These arms are defined by the `(... ) =>` pattern. If we just pass in the expression to be mapped and the error status, the first branch will be triggered, and we will map the error message to the nanoservice error. However, if we pass in the expression to be mapped, the status, and a message, the second arm will be triggered, and the macro will add extra context to the error message, letting us know where the error is happening. This will save us from repeating ourselves.

We can now see how our errors are implemented. There is going to be a fair bit of refactoring, but it is better to correct all this now than later, where the refactor will require more work. Also, we will get some experience with what refactoring code feels like in our Rust system. We can start with our data access module.

Integrating our glue module into our dal

In this section, we are going to be importing our glue module into the data access layer of the to-do service so we can implement our errors. In the data access `Cargo.toml` file, we add the following dependency:

```

# File: nanoservices/to_do/dal/Cargo.toml
[dependencies]
...
glue = { path = "../../glue" }

```

We then use our error structs and macros in our `json_file.rs` file with the following code:

```

// File: nanoservices/to_do/dal/src/json_file.rs
...
use glue::errors::{
    NanoServiceError,
    NanoServiceErrorStatus
};
use glue::safe_eject;

```

With this, we can now replace all our `String` error return types with our nanoservices error. For instance, our `handle` function takes the following form:

```
// File: nanoservices/to_do/dal/src/json_file.rs
fn get_handle(path: Option<&str>) -> Result<File, NanoServiceError>{
    let path = match path {
        Some(p) => p,
        None => {
            &env::var("JSON_STORE_PATH").unwrap_or(
                "./tasks.json".to_string()
            )
        }
    };
    safe_eject!(OpenOptions::new()
        .read(true)
        .write(true)
        .create(true)
        .open(&path),
        NanoServiceErrorStatus::Unknown,
        "Error writing tasks to JSON to file"
    )
}
```

Here, we can see that getting our file handle is more robust. Our macro converts the error of getting the file handle into a `NanoServiceError` error. We can also see that we add context to the error, so if this error is returned, we know it is to do with the JSON file. Our `get_all` function is now defined with the following code:

```
// File: nanoservices/to_do/dal/src/json_file.rs
pub fn get_all<T: DeserializeOwned>()
    -> Result<HashMap<String, T>, NanoServiceError> {
    let mut file = get_handle(None)?;
    let mut contents = String::new();
    safe_eject!(
        file.read_to_string(&mut contents),
        NanoServiceErrorStatus::Unknown,
        "Error reading JSON file to get all tasks"
    )?;
    let tasks: HashMap<String, T> = safe_eject!(
        serde_json::from_str(&contents),
        NanoServiceErrorStatus::Unknown,
        "Error parsing JSON file"
    )?;
}
```

```
    Ok(tasks)
}
```

It must be noted before you run anything that the core layer has not been updated to return a `NanoServiceError` yet. If you try and compile the entire system, you will get an error. Right now, you will not be able to compile the entire system until we have updated the error signatures of the core and networking layers. We can see how our `safe_eject!` macro lifts out the repetitive code, and we just focus on defining the expression to be evaluated, the error status, and the optional context message of where the error is happening, which will be presented alongside the error message. Our `safe_eject!` macro is mainly used for third-party results, as we aim for all our functions to return the nanoservice error.

For the `save_all` function, we have the following code:

```
// File: nanoservices/to_do/dal/src/json_file.rs
pub fn save_all<T: Serialize>(tasks: &HashMap<String, T>)
    -> Result<(), NanoServiceError> {
    let mut file = get_handle(None)?;
    let json = safe_eject!(
        serde_json::to_string_pretty(tasks),
        NanoServiceErrorStatus::Unknown,
        "Error serializing JSON before saving tasks"
    );
    safe_eject!(
        file.write_all(json.as_bytes()),
        NanoServiceErrorStatus::Unknown,
        "Error writing tasks to JSON to file"
    );
    Ok(())
}
```

Our `get_one` function takes the following form:

```
// File: nanoservices/to_do/dal/src/json_file.rs
pub fn get_one<T: DeserializeOwned + Clone>(id: &str)
    -> Result<T, NanoServiceError> {
    let tasks = get_all::<T>()?;
    match tasks.get(id) {
        Some(t) => Ok(t.clone()),
        None => Err(
            NanoServiceError::new(
                format!(
                    "Task with id {} not found",
                    id
                )
            )
        )
    }
}
```

```

        ),
        NanoServiceErrorStatus::Unknown
    )
}
}

```

We then build our `save_one` function with the following code:

```

// File: nanoservices/to_do/dal/src/json_file.rs
pub fn save_one<T>(id: &str, task: &T)
    -> Result<(), NanoServiceError>
where
    T: Serialize + DeserializeOwned + Clone,
{
    let mut tasks = get_all::<T>().unwrap_or_else(
        |_| HashMap::new()
    );
    tasks.insert(id.to_string(), task.clone());
    save_all(&tasks)
}

```

And finally, the `delete_one` function takes the following form:

```

// File: nanoservices/to_do/dal/src/json_file.rs
pub fn delete_one<T>(id: &str) -> Result<(), NanoServiceError>
where
    T: Serialize + DeserializeOwned + Clone,
{
    let mut tasks = get_all::<T>().unwrap_or(HashMap::new());
    tasks.remove(id);
    save_all(&tasks)
}

```

With this, our data access layer is ready to be used by our core layer.

Integrating our glue module into our core layer

For our core layer, we add the following dependency:

```

# File: nanoservices/to_do/core/Cargo.toml
[dependencies]
...
glue = { path = "../../glue" }

```


For our create API endpoint, we just need to update the signature of the create function to return the `NanoServiceError` with the following code:

```
///! nanoservices/to_do/core/src
///! /api/basic_actions/create.rs
...
use glue::errors::NanoServiceError;

pub fn create(title: &str, status: TaskStatus)
    -> Result<ToDoItem, NanoServiceError> {
    . . .
}
```

We also do the same with our get function, with the following code:

```
///! nanoservices/to_do/core/src
///! /api/basic_actions/get.rs
...
use glue::errors::NanoServiceError;

pub async fn get_all()
    -> Result<AllToDoItems, NanoServiceError> {
    . . .
}
```

Our endpoints are now supported; however, we also must get main working as well. This means our `TaskStatus` supports `NanoServiceError` with the following code:

```
///! nanoservices/to_do/core/src/enums.rs
. . .
use glue::errors::NanoServiceError;
. . .
impl TaskStatus {

    pub fn from_string(status: &String)
        -> Result<TaskStatus, NanoServiceError> {
        match status.to_uppercase().as_str() {
            "DONE" => Ok(TaskStatus::DONE),
            "PENDING" => Ok(TaskStatus::PENDING),
            _ => Err(
                NanoServiceError::new(
                    "Invalid status".to_string(),
                    NanoServiceErrorStatus::BadRequest
                )
            )
        }
    }
}
```

```

    )
  }
}
}

```

And finally, our main function has the following signature:

```

//! nanoservices/to_do/core/src/main.rs
...
use glue::errors::NanoServiceError;
...
fn main() -> Result<(), NanoServiceError> {
    ...
    Ok(())
}

```

Our core layer is now ready, so finally, we can configure our networking.

Integrating our glue module into our networking layer

Unlike the core and dal layers, we will have the Actix feature for our glue package with the following Cargo.toml file code:

```

# File: nanoservices/to_do/networking/actix_server/Cargo.toml
glue = { path = "../..../glue", features = ["actix"] }

```

With our glue package, our create function file for our API endpoint takes the following form:

```

//! File: nanoservices/to_do/networking/actix_server/
//! src/api/basic_actions/get.rs
use core::api::basic_actions::get::get_all as get_all_core;
use actix_web::HttpResponse;
use glue::errors::NanoServiceError;

pub async fn get_all() -> Result<HttpResponse, NanoServiceError> {
    Ok(HttpResponse::Ok().json(get_all_core().await?))
}

```

And that is it! I appreciate that there has been a fair amount of reworking with the juggling of multiple workspaces. However, the hard, tedious parts are now done. Adding new API endpoints from now on is going to be simple. Slotting in new nanoservices and layers will also be effortless. With this in mind, we are ready to move on to our next chapter of processing HTTP requests.

Summary

In this chapter, we built a functioning nanoservice that has a core layer, a data access layer, and a networking layer. While getting there took a bit of refactoring, our work resulted in a well-structured server where all the layers compile together to be served by the networking layer. To aid in error handling, we also built a glue module that was used by all layers, enabling us to propagate the same error type through all the layers in the nanoservice. At this stage, slotting in other web frameworks or data storage approaches is as simple as creating another Cargo workspace in the networking layer, or another feature in the data access layer.

Also in this chapter, we chained factories to enable our views to be constructed on the fly and added to the server. With this chained factory mechanism, we can slot entire view modules in and out of the configuration when the server is being built.

In the next chapter, we will work with processing requests and responses. We will learn how to pass parameters, bodies, headers, and forms to views and process them, returning JSON. We will be using these new methods with the to-do module we built in *Chapter 3* to enable our interaction with to-do items to be achieved through server views.

Questions

1. What parameter is passed into the `HttpServer::new` function, and what does the parameter return?
2. How can we work out what data types an API function can return automatically?
3. How can we enable data handling across all Cargo workspaces, including the HTTP response?
4. Why is it a good idea to have `/api/v1` in the endpoint of the URL?
5. How can we support multiple dependencies but be selective on what dependencies we compile, depending on our needs, such as networking dependencies?
6. For our networking API endpoints, we have view factories that are chained into the main API factory. What is the advantage of chaining factories?

Answers

1. A closure is passed into the function. It has to return the `App` struct so the `bind` and `run` functions can be acted on after the `HttpServer::new` function has fired.
2. We can look at the documentation for the `Responder` trait to see which data types have implemented the `Responder` trait.
3. We create a separate Cargo workspace where we define an error struct. All other workspaces can then compile this workspace as a dependency and return the error struct as the error type. For the HTTP response, we need to implement the `ResponseError` for our error.
4. The `api` part of the URL enables us to differentiate between frontend and backend HTTP requests. The `v1` part enables us to version control the API endpoint and enables us to support multiple versions.

5. We can use features. For example, we can define a dependency as optional and state that this dependency is used if we use a feature that we tether to that dependency. We can then optionally compile the `#[cfg(feature = "some-feature")]` code.
6. Chaining factories gives us flexibility on how individual modules are constructed and how they are orchestrated. The factory inside the module focuses on how the module is constructed, and the factory outside the module focuses on how the different modules are orchestrated. This isolation enables us to swap out modules, as all the logic is maintained in the module. Chaining factories also enables us to keep bloat to a minimum. For instance, our `main.rs` file only focuses on the running of the server, and is not overrun by mapping URL endpoints.

Get this book's PDF copy, code bundle, and more

Scan the QR code (or go to packtpub.com/unlock). Search for this book by name, confirm the edition, and then follow the steps on the page.



UNLOCK NOW

Note: Have your invoice handy. Purchases made directly from the Packt website don't require an invoice.

6

Processing HTTP Requests

In the previous chapter, we utilized the Actix web framework to serve basic views. However, this can only get us so far when it comes to extracting data from the request and passing data back to the user. In this chapter, we will build on the code from *Chapter 5*, to build server views that process to-do items. We will then explore **JSON serialization** for extracting data and returning it to make our views more user-friendly. We will also extract data from the header with middleware before it hits the view. We will explore the concepts around data serialization and extracting data from requests by building out the create, edit, and delete to-do item endpoints for our to-do application.

In this chapter, we will cover the following topics:

- Passing parameters via the URL
- Passing data via the POST body
- Deleting resources using the DELETE method
- Automating running our server
- Updating resources using the PATCH method
- Extracting data from HTTP request headers

Once you have finished this chapter, you will be able to build a basic Rust server in Actix, Axum, Hyper, or Rocket that can send and receive data in the URL, in the body using JSON, and in the header of the HTTP request. This is essentially a fully functioning API server in Rust without a proper database for data storage, authentication of users, or displaying of content in the browser. However, these concepts are covered in the next three chapters. You are on the home stretch to having a fully working Rust server up and running. Let's get started!

Technical requirements

You can download the example project and code for this book by following the instructions in the **Download the example code files** section in the *Preface* of this book.

This chapter's code files, featuring the implementation of the changes in the core layer from *Chapter 5*, are included in the downloadable code bundle.

You will also be making requests to your server using the cURL command-line tool, which enables you to make network requests. You can download cURL directly through your OS package manager.

Passing parameters via the URL

Passing parameters via the URL is the easiest way to pass data to the server. We can explore this concept by passing the name of a to-do item and returning that single to-do item to the client.

Before we touch the networking layer, we must build the core function that loads the data and filters for the to-do item by name, which can be done with the following code:

```

///! File: nanoservices/to_do/core/src/api/basic_actions/get.rs

. . .
use glue::errors::{
    NanoServiceError,
    NanoServiceErrorStatus
};
. . .
pub async fn get_by_name(name: &str)
-> Result<ToDoItem, NanoServiceError> {
    Ok(get_all_handle::<ToDoItem>()?
        .remove(name)
        .ok_or(
            NanoServiceError::new(
                format!("Item with name {} not found", name),
                NanoServiceErrorStatus::NotFound
            )
        )?)
}

```

Note that our filter at this stage is essential, as we are loading all the data, extracting the item that we want, and then returning it. We will optimize our filter in *Chapter 9* when we move over to a proper database. For now, our filter works for us, so we can continue to develop our features.

We can also see that we use the `remove` function to extract the item. This means that we do not have to clone the item that we are extracting, reducing the amount of memory required. We do not worry that the item has been removed from the `HashMap` because we are not going to write that `HashMap` to the file.

We now need to connect our core `get_by_name` function to our HTTP server. We will wrap the core `get_by_name` function in a function that handles the HTTP request, but we first need to expose the core functionality and networking utils by redefining our imports with the following code:

```

///! File: nanoservices/to_do/networking/actix_server
///! src/api/basic_actions/get.rs
use to_do_core::api::basic_actions::get::{

```

```

    get_all as get_all_core,
    get_by_name as get_by_name_core
};
use actix_web::{
    HttpResponse,
    HttpRequest
};
use glue::errors::{
    NanoServiceError,
    NanoServiceErrorStatus
};

```

We can see that we are using the status and the HTTP request. Here, our incoming HTTP request needs to be accepted, and the name parameter must be extracted from the URL.

We then need to handle this extraction. We can do this with the following code:

```

//! File: nanoservices/to_do/networking/actix_server
//! src/api/basic_actions/get.rs

pub async fn get_by_name(req: HttpRequest)
-> Result<HttpResponse, NanoServiceError> {
    let name = match req.match_info().get("name") {
        Some(name) => name,
        None => {
            return Err(
                NanoServiceError::new(
                    "Name not provided".to_string(),
                    NanoServiceErrorStatus::BadRequest
                )
            )
        }
    };
    Ok(HttpResponse::Ok().json(get_by_name_core(name).await?))
}

```

Here, we can see that if the name parameter is not found in the URL, we return a bad request. We then just return the data from the core `get_by_name` function, exploiting the `?` operator at the bottom of the function, as a not found HTTP response will be automatically returned to the client if the item is not found.

We have nearly finished our get API endpoint. All we must do now is connect our API endpoint to our factory. We can do this by adding the following line:

```
///! File: nanoservices/to_do/networking/actix_server
///! src/api/basic_actions/mod.rs
. . .
.route("get/{name}", get().to(get::get_by_name))
. . .
```

We need to have that to-do item in our tasks.json JSON file in the root of our system. If we do not have the JSON file in the root, the server will look for the JSON file to access the state of to-do items and not find the file, returning an error. With the JSON file, we can run our server with the following command:

```
cargo run -p to-do-actix-server
```

While our server is running, we can visit our individual get URL in the browser. We will get a response like the following:



Figure 6.1 – Our response from our individual GET endpoint

If we pass a to-do item name that we know is not in the JSON file, we will get a not found response, as seen in Figure 6.2.



Figure 6.2 – Our not found response from our individual GET endpoint

And here we have it, our get individual to-do item is now fully working.

Remember the flexibility we have with our network layer!

It is reasonable to wonder why we have two different layers to just get the simple filter that returns one item. Remember, our core functionality is separated from the networking layer; therefore, we can swap out the web framework easily if we need to. Also, more importantly, we can compile this nanoservice into another program or server by just targeting the core workspace. It is important to keep this flexibility in mind to justify the effort of maintaining these layers.



You can inspect the code on GitHub to see how the other frameworks extract the name from the URL:

- **Rocket:** The `<name>` is nested in the route macro above the function that serves the API endpoint.
- **Axum:** The `Path` struct is wrapped around the name variable and placed as a parameter in the function that serves the endpoint. This `Path` struct acts as an extractor that implements a trait that extracts data from the request. We will learn how to implement these traits for Actix. The approach of implementing extractor traits is pretty much the same for Rocket and Axum.
- **Hyper:** Here, the URI is split into a vector and matched. The function that is called with the request depends on the match arm from the URI.

Even though we can get our to-do items, we cannot yet do much with them. We must start altering the state of our to-do items. Our next step is refactoring our create to-do item via POST and JSON data.

Passing data via POST body

While passing parameters via the URL is simple, there are a few problems with the URL approach. URLs can be cached by the browser, and the user might end up selecting an autocompleted URL with data in the URL. This is good if we are using the URL to pass parameters to get data, such as visiting your profile on a social media application; however, we do not want to accidentally select a cached URL that alters data in the application. In this section, we are going to explore POST requests by creating a new to-do item and returning all the to-do items.

Passing data in the URL is also limited to simple data types. For instance, if we wanted to pass a list of `HashMap`s to the server, it would be hard to pass such a data struct through the URL without doing some other form of serialization. This is where POST requests and JSON bodies come in. We can pass JSON data via the HTTP request body.

Defining the core create function

We have already built our create function, but we need to refactor it so that we can integrate the create function into our web server. Our create function refactor results in the following code:

```
//! File: nanoservices/to_do/core/src/api/basic_actions/create.rs
use crate::structs::ToDoItem;
```

```
#[cfg(feature = "json-file-storage")]
use to_do_dal::json_file::save_one;
use glue::errors::NanoServiceError;

pub async fn create(item: ToDoItem)
-> Result<ToDoItem, NanoServiceError> {
    let _ = save_one(&item.title.to_string(), &item)?;
    Ok(item)
}
```

Remember that our `ToDoItem` struct has already implemented the serializing structs to be read and written to the JSON file. Here, we have turned our `create` function into an `async` function. Right now, we are not using any `async` functionality, but in the future, we will be using `async` to connect to databases and make HTTP requests inside our API endpoints. Considering this, it makes sense just to make API functions in the core module `async`. If you kept the `main.rs` file in the core workspace, you now must delete it, or convert the `main` function in the `main.rs` file into `async` and pass a constructed `ToDoItem` into the `create` function.

We can now wrap the core `create` function in our server.

Wrapping our core in the networking layer

First, we need to use the following:

```
///! File: nanoservices/to_do/networking/actix_server
///! /src/api/basic_actions/create.rs
use to_do_core::api::basic_actions::{
    create::create as create_core,
    get::get_all as get_all_core
};
use to_do_core::structs::ToDoItem;
use glue::errors::NanoServiceError;
use actix_web::{
    HttpResponse,
    web::Json
};
```

With these structs and traits, we can define our `create` API web endpoint with the following code:

```
///! File: nanoservices/to_do/networking/actix_server
///! /src/api/basic_actions/create.rs

pub async fn create(body: Json<ToDoItem>)
-> Result<HttpResponse, NanoServiceError> {
    let _ = create_core(body.into_inner()).await?;
```

```
Ok(HttpResponse::Ok().json(get_all_core().await?))
}
```

Wow, that is compact! But there is a lot going on here. The `Json` struct implements traits that extract data from the body of the HTTP request before the `create` function is called. If the HTTP request body can be serialized into a `ToDoItem` struct, then this is done, and we have a `ToDoItem` struct passed into the `create` function from the HTTP request body. If a `ToDoItem` struct cannot be constructed from the JSON body of the HTTP request, then a bad request response is returned to the client with the serialization error message.

The `Json<ToDoItem>` works because we have implemented the `Deserialize` trait for the `ToDoItem` struct. We then insert our item into the JSON file we are currently using as storage with the `create_core` function. We can also see that the `create_core` function is assigned `let _ =`. This is telling the compiler that we are not interested in the outcome of this function unless it returns an error; therefore, the compiler does not need to allocate memory for the outcome of the `create_core` function.

Finally, we get all the data from the JSON file with the `get_all_core` function and return the items in a JSON body. Again, this is not optimal as our JSON file is not a replacement for a proper database, but it will work for now for creating a new to-do item and returning all the to-do items.

At this stage, we can take a step back and appreciate the consistency that the three major alternative web frameworks have achieved. While we have not coded them directly in this book, they are available in the GitHub repo. For our Axum networking layer, our `create` API endpoint takes the following form:

```
///! File: nanoservices/to_do/networking/axum_server
///! /src/api/basic_actions/create.rs
...
pub async fn create(Json(body): Json<ToDoItem>)
-> Result<impl IntoResponse, NanoServiceError> {
    let _ = create_core(body).await?;
    let all_items = get_all_core().await?;
    Ok((StatusCode::OK, Json(all_items)))
}
```



To avoid bloating the chapter, the imports and routing are defined in the GitHub repository. Here, we can see that we are using Axum's JSON extractor. Because our functions, errors, and structs implement the `serde Serialize` and `Deserialize` traits, and our errors convert to HTTP responses, our integration for other frameworks is still minimal.

For our Rocket implementation, we have the following function:

```
///! File: nanoservices/to_do/networking/rocket_server
///! /src/api/basic_actions/create.rs
...
#[post("/create", data = "<body>")]
```

```
pub async fn create(body: Json<ToDoItem>)
  -> Result<Json<AllToDoItems>, NanoServiceError> {
  let _ = create_core(body.into_inner()).await?;
  Ok(Json(get_all_core().await?))
}
```

Again, we just use the JSON extractor that Rocket has provided. Finally, our Hyper server is a little more involved; we do not have the from request extractors, so we must build our own body extractor with the following code:

```
///! File: nanoservices/to_do/networking/hyper_server
///! /src/utls.rs
...
pub async fn extract_body<X: DeserializeOwned>
  (req: Request<Incoming>) -> Result<X, NanoServiceError> {
  let whole_body = safe_eject!(
    req.collect().await,
    NanoServiceErrorStatus::BadRequest
 )?.aggregate();
  let body: X = safe_eject!(
    serde_json::from_reader(whole_body.reader()),
    NanoServiceErrorStatus::BadRequest,
    "Failed to parse JSON body"
  )?;
  Ok(body)
}
```

We then use this body extraction function for our create API endpoint with the following code:

```
///! File: nanoservices/to_do/networking/hyper_server
///! /src/api/basic_actions/create.rs
...
pub async fn create(req: Request<Incoming>)
  -> Result<Response<Full<Bytes>>, NanoServiceError> {
  let todo_item = extract_body::<ToDoItem>(req).await?;
  create_core(todo_item).await?;
  let json_body = safe_eject!(
    serde_json::to_string(&get_all_core().await?),
    NanoServiceErrorStatus::Unknown
  )?;
  safe_eject!(
```



```

        Response::builder()
            .header(header::CONTENT_TYPE, "application/json")
            .body(Full::new(Bytes::from(json_body))),
        NanoServiceErrorStatus::Unknown
    )
}

```

This is because we manually match the URI and pass the entire HTTP request into the function. Therefore, we must do the conversions ourselves. However, our `safe_eject!` macro really helps consistent with minimal code.

Can we pass a JSON body in a GET instead of a POST request?



Yes, we can. Personally, I find these method definitions trivial, and I like to put all the data that I am passing to the server in a JSON body because it is easier to handle as the requirements and app evolve. I also find it easier to handle the serialization into a struct. However, this is just my opinion, and while most HTTP clients will allow you to put JSON bodies in a GET request, at the time of writing this book, Axios, a JavaScript HTTP client, will not allow JSON bodies in GET requests. There is also the added risk of intermediaries such as proxies and load balancers that might not expect a JSON body in a GET request and reject it. Depending on the configuration of the server, GET requests with JSON bodies could be classified as “request smuggling,” where an attacker tries to attach data to a request that is not supposed to be there.

I’ve worked on projects where I have put everything through POST methods, and while this means that the requests are not cached, for the initial phase, it has worked out fine. This means I can keep altering the bodies until I am happy with the structure. These requests can be refined later. You can read about the semantics of HTTP requests via the following link: <https://www.rfc-editor.org/rfc/rfc9110>.

After adding a couple of create files, it is worth running a `cargo build` command in the root of your system to make sure that your build is working before progressing.

We now need to plug our create API function into our web server.

Configuring our create network route

We start by importing the post function with the following code:

```

//! File: nanoservices/to_do/networking/actix_server
//! /src/api/basic_actions/mod.rs
...
use actix_web::web::{ServiceConfig, get, scope, post};
...

```

We then define our view in our factory with the following code:

```
///! File: nanoservices/to_do/networking/actix_server
///! /src/api/basic_actions/mod.rs
...
.route("create", post().to(create::create))
...
```

We can now run our server with our new create endpoint.

Testing our endpoint with cURL

While our server is running, we can test our create endpoint with the following cURL command in our terminal:

```
curl -X POST http://127.0.0.1:8080/api/v1/create \
-H "Content-Type: application/json" \
-d '{"title": "writing", "status": "PENDING"}'
```



If you are using Windows 11, you might have some problems with cURL leveraging PowerShell under the hood, meaning that Windows 11 is not directly using cURL. This “fake cURL” might not work. You can get around this by using HTTP files, which is explained at the following link: <https://learn.microsoft.com/en-us/aspnet/core/test/http-files?view=aspnetcore-8.0>.

For our cURL command, we can see that we include an -H flag. This is a header, telling the server that our request contains JSON. This can help the server with processing the request to decide if the server can process the request body as JSON. As mentioned before, some servers and load balancers can reject the request if the header does not match the contents under the suspicion of request smuggling.

After the cURL command has been run, we then get the following response:

```
{
  "pending":[
    {"title":"writing","status":"PENDING"},
    {"title":"washing","status":"PENDING"}
  ],
  "done":[
    {"title":"coding","status":"DONE"}
  ]
}
```

If we inspect our JSON file that we are using for storage, we should have the following contents:

```
{
  "coding": {
```

```
    "title": "coding",
    "status": "DONE"
  },
  "writing": {
    "title": "writing",
    "status": "PENDING"
  },
  "washing": {
    "title": "washing",
    "status": "PENDING"
  }
}
```

We can also test our serialization of the JSON body with the following command, to check that our API endpoint will not accept any status. If we were to accept any status, we would not be able to sort the to-do items into pending or done:

```
curl -X POST http://127.0.0.1:8080/api/v1/create \
-H "Content-Type: application/json" \
-d '{"title": "writing", "status": "test"}'
```

The preceding command has a status that should not be accepted. When we run this command, we should get the following response:

```
Json deserialize error: unknown variant `test`, expected `DONE`
or `PENDING` at line 1 column 37
```

With this response, we can see that we are not going to be passing data that is going to corrupt the data store, and we get a helpful response on why we are rejecting the incorrect data.



This guard will also work for Axum, Rocket, and Hyper because we have defined the serialization rules in our core module for the `ToDoItem` struct, meaning that we will only accept pending and done status tasks.

And there we have it. We can see that our POST create HTTP endpoint is now working. So, we can now get our items and add new ones. However, what if we create an item by accident and need to delete it? This is where DELETE methods come in.

Deleting resources using the DELETE method

DELETE methods are like GET methods. We could technically pass data in the JSON body of the HTTP request, but it is not standard practice like the GET method. For our DELETE post, there should be enough data in the URL that we can perform a delete. In our case, the title of the item is enough to delete the item from storage, for the sake of simplicity.

Defining our delete JSON file method

Before we define the core and server functions, we must ensure that our data access layer function returns the right message if we do not find the right item when deleting from our store. Some slight refactoring is needed where we return the right error if there is no item found in the store, using the following code:

```

///! File: nanoservices/to_do/dal/src/json_file.rs

pub fn delete_one<T>(id: &str) -> Result<T, NanoServiceError>
where
    T: Serialize + DeserializeOwned + Clone + std::fmt::Debug,
{
    let mut tasks = get_all:::<T>().unwrap_or(HashMap::new());
    match tasks.remove(id) {
        Some(deleted_item) => {
            save_all(&tasks)?;
            Ok(deleted_item)
        },
        None => Err(
            NanoServiceError::new(
                format!("Task with title {} not found", id),
                NanoServiceErrorStatus::NotFound
            )
        )
    }
}

```

We can now define our core delete function with the following code:

```

///! File: nanoservices/to_do/core/src/basic_actions/delete.rs

use to_do_dal::json_file::delete_one;
use crate::structs::ToDoItem;
use glue::errors::NanoServiceError;

pub async fn delete(id: &str) -> Result<ToDoItem, NanoServiceError> {
    delete_one:::<ToDoItem>(id)
}

```

We can then wrap the core delete function into our networking layer. The approach should run along the same lines as the get individual item API endpoint, as we are extracting the name parameter out of the URL. Now would be a good time to try and build the delete endpoint function for the server yourself.

Adding delete to our networking layer

If you attempted to build the delete endpoint function yourself, hopefully you will have used the following structs and traits:

```

///! File: nanoservices/to_do/networking/actix_server
///! /src/api/basic_actions/delete.rs
use to_do_core::api::basic_actions::{
    delete::delete as delete_core,
    get::get_all as get_all_core
};
use actix_web::{
    HttpResponse,
    HttpRequest
};
use glue::errors::{
    NanoServiceError,
    NanoServiceErrorStatus
};

```

We then define the delete endpoint with the following code:

```

///! File: nanoservices/to_do/networking/actix_server
///! /src/api/basic_actions/delete.rs
...
pub async fn delete_by_name(req: HttpRequest)
-> Result<HttpResponse, NanoServiceError> {
    match req.match_info().get("name") {
        Some(name) => {
            delete_core(name).await?;
        },
        None => {
            return Err(
                NanoServiceError::new(
                    "Name not provided".to_string(),
                    NanoServiceErrorStatus::BadRequest
                )
            )
        }
    }
};
Ok(HttpResponse::Ok().json(get_all_core().await?))
}

```

We return a 200 OK status code. However, technically, our semantics are not correct. The body should be a status of what is deleted. This is why the core delete function returned the item that was deleted. However, here we return all the to-do items instead. This is to simplify our frontend when we start coding it as we will not have to make another HTTP request once we have performed an action on our to-do items.

Defining the delete route

We now need to plug our `delete_by_name` API function into our web server. To do this, firstly we need to import the delete function with the following code:

```

///! File: nanoservices/to_do/networking/actix_server
///! /src/api/basic_actions/mod.rs

. . .
use actix_web::web::{ServiceConfig, get, scope, post, delete};
. . .

```

We then define our view in our factory with the following code:

```

///! File: nanoservices/to_do/networking/actix_server
///! /src/api/basic_actions/mod.rs

. . .
.route("delete/{name}", delete().to(delete::delete_by_name))

```

If we then run our server, we can make a delete request with the following command:

```
curl -X DELETE http://127.0.0.1:8080/api/v1/delete/washing
```

However, we get the following error when making the request:

```
"Error parsing JSON file: trailing characters at line 10 column 4"
```

Thanks to all the effort that we put into our error handling, we know that there is something wrong with the parsing of the JSON file. Our `safe_eject!` macro has a string that has the context added to the error. So, if we go to our `nanoservices/to_do/dal/src/json_file.rs` file and search for "Error parsing JSON file", we will see that the error is occurring when we try and serialize the data loaded from the JSON file.

Refactoring our JSON file code

If we inspect our JSON file, we should have something like the following:

```

{
  "coding": {
    "title": "coding",
    "status": "DONE"
  },
  "washing": {

```

```

    "title": "washing",
    "status": "PENDING"
  }
} "washing": {
  "title": "washing",
  "status": "PENDING"
}
}

```

In the JSON file contents, we can see that there is some malformed JSON, implying that the file is not being cleared or truncated before writing the JSON data to the file. To fix this, we need to revert the JSON file to the previous state. We then must create a writer handle that truncates the file, ensuring that the file is empty before we write to the file with the following code:

```

//! File: nanoservices/to_do/dal/src/json_file.rs
fn get_write_handle(path: Option<&str>)
-> Result<File, NanoServiceError> {
  let path = match path {
    Some(p) => p,
    None => {
      &env::var("JSON_STORE_PATH").unwrap_or(
        "./tasks.json".to_string()
      )
    }
  };
  let file = safe_eject!(OpenOptions::new()
    .write(true)
    .create(true)
    .truncate(true) // ensures file is empty
    .open(&path),
    NanoServiceErrorStatus::Unknown,
    "Error reading JSON file (write handle)"
  )?;
  Ok(file)
}

```

Because the handle ensures that the file is empty, we will not get any parsing errors when writing our updated state to our JSON file. We can then swap our file handle in the `save_all` function with the following code:

```

//! File: nanoservices/to_do/dal/src/json_file.rs

pub fn save_all<T: Serialize>(tasks: &HashMap<String, T>)

```

```

-> Result<(), NanoServiceError> {
    let mut file = get_write_handle(None)?;
    . . .
    Ok(())
}

```

Now that the data access layer has been refactored, we must recompile our code. We then rerun our delete cURL command, and we should get the following printout in the terminal:

```

{
  "pending":[],

  "done":[
    {"title":"coding","status":"DONE"}
  ]
}

```

This shows that we have successfully deleted our to-do item!



For the other web frameworks, we are using methods that we have seen before, so the examples in the GitHub repo should make sense.

In this section, we got to see how useful the context of the error was as our error bubbles up to the HTTP response, so our system is not panicking. However, we also do not get a stack trace because the server has not crashed.

We are now at the stage where there are a few moving pieces and different endpoints. At this stage, it is good to start automating some of these steps so we can quickly check that our system is working as it should. Later, I dedicate two entire chapters to testing, as testing is a fundamental skill for keeping up the momentum of software development. However, to prevent frontloading too many concepts at once, we can build a basic Bash script that populates the JSON file with a state that we want and then performs some HTTP requests so we can inspect the output.

Automating running our server

To create the automated Bash script that orchestrates an end-to-end test, we must create a scripts directory and create a Bash file called `basic_test.sh` within it. We then create some boilerplate code that navigates to the root of our system, as follows:

```

#!/usr/bin/env bash
# File: scripts/basic_test.sh
# navigate to directory
SCRIPTPATH="$( cd "$(dirname "$0")" ; pwd -P )"
cd $SCRIPTPATH

```

```
cd ..
```

The first line of the preceding code is called a shebang line, which tells the computer that we are to execute the script using our shell. We then navigate to the directory where this script is and then move out one directory to be in the root. This is done because Bash scripts run in whatever working directory calls them. Because we are looking for specific files and we want to run our server, we want to ensure that our testing script runs in the root of the system every time we run the testing script, no matter where we call it from.

If you downloaded the GitHub repository, the functionality of our server will also work with Hyper, Axum, and Rocket. Therefore, we want the option to build and run any of these servers with the following code:

```
# File: scripts/basic_test.sh
framework=${1:-actix}
case $framework in
  actix)
    echo "Building and running Actix server..."
    cargo build -p to-do-actix-server
    cargo run -p to-do-actix-server &
    PID=$!
    ;;
  axum)
    echo "Building and running Axum server..."
    cargo build -p to-do-axum-server
    cargo run -p to-do-axum-server &
    PID=$!
    ;;
  hyper)
    echo "Building and running Hyper server..."
    cargo build -p to-do-hyper-server
    cargo run -p to-do-hyper-server &
    PID=$!
    ;;
  rocket)
    echo "Building and running Rocket server..."
    cargo build -p to-do-rocket-server
    cargo run -p to-do-rocket-server &
    PID=$!
    ;;
  *)
    echo "Unknown framework: $framework"
```

```

    echo "Usage: $0 [actix|axum|hyper|rocket]"
    exit 1
;;
esac
sleep 1

```

If we do not pass in any arguments to the Bash script, our default server that will run is Actix. With the & character, we are running the server in the background. We build the server before we run it, because we do not want to be compiling our server in the background while we are making HTTP requests to it. We then keep track of the process ID by assigning it to the variable PID, and we sleep for one second to ensure that the server is ready for HTTP requests. We will use the PID to kill the server after our HTTP requests have been carried out because the tests have finished, and we do not want a server running indefinitely, binding a port after the tests. We then remove our tasks.json file and create a new one with the state that we want, with the following code:

```

# File: scripts/basic_test.sh
rm tasks.json
rm output.txt
cat <<EOF > tasks.json
{}
EOF

```

We are keeping the state empty, but we could put any JSON state we wanted in the curly brackets. This ensures that the state is consistent every time we run the test, so we can see whether a change in our code later changes the behavior of our server.

Now that our state is defined, and our server is running in the background, we can test our API endpoints with the following code:

```

# File: scripts/basic_test.sh
curl -X POST http://127.0.0.1:8080/api/v1/create \
-H "Content-Type: application/json" \
-d '{"title": "writing", "status": "PENDING"}' >> output.txt

echo "" >> output.txt

curl -X POST http://127.0.0.1:8080/api/v1/create \
-H "Content-Type: application/json" \
-d '{"title": "coding", "status": "PENDING"}' >> output.txt

echo "" >> output.txt

curl -X DELETE http://127.0.0.1:8080/api/v1/delete/coding >> output.txt
kill $PID

```

Here, create a couple of pending items, and then delete one of the items. We also append the state of the items to the `output.txt` file after every request so we can see how the state progresses after each HTTP request.

We now run our Bash script with the following command:

```
sh scripts/basic_test.sh
```

We should get the following contents in our `output.txt` file:

```
{
  "pending": [{"title": "writing", "status": "PENDING"}],
  "done": []
}
{
  "pending": [{"title": "coding", "status": "PENDING"},
    {"title": "writing", "status": "PENDING"}],
  "done": []
}
{
  "pending": [{"title": "writing", "status": "PENDING"}],
  "done": []
}
```

Here, we can see that our delete and create API endpoints affect the state as expected. And there we have it; we can run this script again and again with consistent outcomes. We will keep adding to this script, and over time, you will get a feel for how much time building a testing harness saves you.

Deleting the to-do item titled `writing` is all well and good, but we also need to be able to edit the status of the to-do item. In our next section, we will edit our to-do items with PATCH methods.

Updating resources using a PATCH method

PATCH methods are essentially the same as a POST method. We want to put the data in the JSON body. However, the PATCH method is typically used for updating parts of a resource that have already been created. If we were to update the entire resource, then we would use the PUT method.

This is the last endpoint that we create in this chapter. Here, we want to accept a to-do item in the JSON body, get the item, and update it. This is a good point to try and build the API endpoint yourself. As a hint, it is advisable to start with the core, calling the data access layer. Here, you can load the items, check that the item is there, and update the item, inserting the updated item back into the state and writing the state. Once this is done, you then move on to the networking layer, which calls the core update function.

To build our API endpoint, we begin by importing the following:

```
#!/ File: nanoservices/to_do/core/src/api/basic_actions/update.rs
use to_do_dal::json_file::{
    get_all as get_all_handle,
    save_all
};
use crate::structs::ToDoItem;
use glue::errors::{
    NanoServiceError,
    NanoServiceErrorStatus
};
```

With these imports, we define the update function with the following code:

```
///! File: nanoservices/to_do/core/src/api/basic_actions/update.rs
pub async fn update(item: ToDoItem)
    -> Result<(), NanoServiceError> {
    let mut all_items = get_all_handle:::<ToDoItem>()?;
    if !all_items.contains_key(&item.title) {
        return Err(NanoServiceError::new(
            format!("Item with name {} not found", item.title),
            NanoServiceErrorStatus::NotFound
        ));
    }
    all_items.insert(item.title.clone(), item);
    save_all(&all_items)?;
    Ok(())
}
```

Here, we can see that we get all the items, and check to see whether the item we are trying to update is in the state of our to-do items. If it's not, we return an error.

Note that if we were using a PUT method, it is customary to allow the creation of the resource if the resource is not there. However, for us, we are only doing a partial update, which is the status. In our frontend, our update will only happen if the user clicks on a button associated with the to-do item. The item must be there before we can update it. If it is, we then update the item with the new status and save it.

Everything is ready here to move on to defining the API function for the server, which takes the following form:

```
///! File: nanoservices/to_do/networking/actix_server
///! /src/api/basic_actions/update.rs
use to_do_core::api::basic_actions::{
    update::update as update_core,
    get::get_all as get_all_core
};
use to_do_core::structs::ToDoItem;
use glue::errors::NanoServiceError;
use actix_web::{
    HttpResponse,
    web::Json
};

pub async fn update(body: Json<ToDoItem>)
    -> Result<HttpResponse, NanoServiceError> {
```



```

    let _ = update_core(body.into_inner()).await?;
    Ok(HttpResponse::Ok().json(get_all_core().await?))
}

```

The preceding code should not be a surprise at this point. We are merely extracting the correct data from the HTTP request, calling a core function, and returning an HTTP response.

Finally, we import the PUT method with the following code:

```

///! File: nanoservices/to_do/networking/actix_server
///! /src/api/basic_actions/mod.rs
. . .
use actix_web::web::{ServiceConfig, get, scope, post, delete, patch};
. . .

```

We also define the API endpoint view in the API factory with the following code:

```

///! File: nanoservices/to_do/networking/actix_server
///! /src/api/basic_actions/mod.rs
. . .
.route("update", patch().to(update::update))
. . .

```

We can now update our testing Bash script with the update API call using the following code:

```

# File: scripts/basic_test.sh
. . .
echo "" >> output.txt

curl -X PATCH http://127.0.0.1:8080/api/v1/update \
-H "Content-Type: application/json" \
-d '{"title": "writing", "status": "DONE"}' \
>> output.txt
kill $PID

```

With this addition, running the test Bash script gives us the following content in the output.txt file:

```

{"pending":[{"title":"writing","status":"PENDING"}],"done":[]}
{
  "pending":[{"title":"coding","status":"PENDING"},
    {"title":"writing","status":"PENDING"}],"done":[]}
}
{"pending":[{"title":"writing","status":"PENDING"}],"done":[]}
{"pending":[],"done":[{"title":"writing","status":"DONE"}]}

```

The content in the `output.txt` file shows that our update API endpoint works. And this is it, we have all our endpoints to handle to-do items. But what about extracting data from headers? While we do not need to extract anything from HTTP request headers to handle our to-do items, we will need to extract data from headers for things such as authentication. Extracting data from headers comes under the concept of processing HTTP requests.

Extracting data from HTTP request headers

In this section, we will not implement full authentication, as this will be addressed in *Chapter 10*. Here, we are just going to extract a string from the header.

Before we write any more code, we must add the following dependency to our `glue` module as we will be using futures in our implementation of the extractor traits:

```
# File: glue/Cargo.toml
. . .
[dependencies]
. . .
futures = "0.3.30"
```

When it comes to headers, we can store credentials or metadata around the HTTP request. Before we get into inspecting the headers of our HTTP requests, we must define our token in our `glue` workspace because we are going to use the token for authentication in the future. Every service should have the right to enforce authentication without having to rewrite the token implementation.

Defining our header token modules

We want our `glue` module to support header extraction for all our different web frameworks. Therefore, our `token.rs` file has the following imports and structure:

```
///! File: glue/src/token.rs
pub struct HeaderToken {
    pub message: String,
}
#[cfg(feature = "actix")]
mod actix_impl {
    use super::HeaderToken;
    pub use actix_web::{
        dev::Payload,
        FromRequest as ActixFromRequest,
        HttpRequest,
    };
    use futures::future::{Ready, ok, err};
    use crate::errors::{NanoServiceError, NanoServiceErrorStatus};
    . . .
```

```

}
#[cfg(feature = "rocket")]
mod rocket_impl {
    . . .
}
#[cfg(feature = "axum")]
mod axum_impl {
    . . .
}
#[cfg(feature = "actix")]
pub use actix_impl::ActixFromRequest;

```

Here, we can see that we wrap the implementation of the `FromRequest` trait inside its own module to avoid clashes. We are not going to cover the implementations of the other frameworks; however, Rocket and Axum are pretty much the same as Actix. The working code is in the GitHub repository for reference.

Because we are handling the API view functions directly with Hyper, the extract body and token from header functions have been moved to a `hyper_utils` directory in the glue module. Finally, we can see in the preceding code that we export our `ActixFromRequest` trait at the bottom of the file if we need to reference it outside of the file. (In the GitHub code repository, we also export the other web frameworks, but if you are not implementing the other frameworks, then it is best to just export the web frameworks that you are implementing.)

As we can also see, the `HeaderToken` struct houses a string, which is going to be the token extracted from the header.

Implementing an Actix header extraction trait

Now we move on to building the middleware that extracts the token inside our Actix module, using the following code:

```

///! File: glue/src/token.rs
impl ActixFromRequest for HeaderToken {
    type Error = NanoServiceError;
    type Future = Ready<Result<HeaderToken, NanoServiceError>>;

    fn from_request(req: &HttpRequest, _: &mut Payload)
        -> Self::Future {
        . . .
    }
}

```

Here, we return a result that can be either our nanoservice error or a header token. The Ready struct wraps this result and implements the Future trait, where the poll function instantly returns a Ready state with the result. We can see this code in the futures package:

```
///! package: futures::future
impl<T> Future for Ready<T> {
    type Output = T;

    #[inline]
    fn poll(mut self: Pin<&mut Self>, _cx: &mut Context<'_>)
        -> Poll<T> {
        Poll::Ready(self.0.take().expect(
            "Ready polled after completion")
        )
    }
}
```

What this means is that our from_request function returns a future that is going to be processed by our tokio runtime. However, that future is going to instantly be resolved after one poll, which is why there is no waker in the poll function of the Ready struct.

Now that we understand what our from_request function does, let's focus on the code inside the from_request function. Initially, we must extract the token from the header and return an unauthorized error if there is no token, using the following code:

```
///! File: glue/src/token.rs
let raw_data = match req.headers().get("token") {
    Some(data) => data,
    None => {
        return err(NanoServiceError {
            status: NanoServiceErrorStatus::Unauthorized,
            message: "token not in header under key 'token'"
                .to_string()
        })
    }
};
```

Note that we return an error wrapped in err. Lowercase err and lowercase ok just wrap the error or result in a Ready struct.

Now that we have extracted the data from the header, we can then convert the data to a string and return it with the code:

```
///! File: glue/src/token.rs
let message = match raw_data.to_str() {
```

```

Ok(token) => token.to_string(),
Err(_) => {
    return err(NanoServiceError {
        status: NanoServiceErrorStatus::Unauthorized,
        message: "token not a valid string".to_string()
    })
}
};
return ok(HeaderToken{message})

```

Our token is now ready to extract data from the header in the middleware.

Extracting headers from requests in the Actix networking layer

Finally, to make the token code accessible to users of our token module, we can declare the token module in our glue crate with the following code:

```

///! File: glue/src/mod.rs
pub mod errors;
pub mod token;

```

We can now update our update API endpoint with the following code:

```

///! File: nanoservices/to_do/networking/actix_server/
///! src/api/basic_actions/update.rs
. . .
use glue::{
    errors::NanoServiceError,
    token::HeaderToken
};
. . .
pub async fn update(token: HeaderToken, body: Json<ToDoItem>)
    -> Result<HttpResponse, NanoServiceError> {
    println!("Token: {}", token.message);
    let _ = create_core(body.into_inner()).await?;
    Ok(HttpResponse::Ok().json(get_all_core().await?))
}

```

Here, we can see that putting the HeaderToken as a parameter in the create function will execute the FromRequest trait and pass the result into the function if the FromRequest trait passes. This is the same as our Json<ToDoItem>.



For Axum and Rocket, we do the same as we have done here. We put the `HeaderToken` struct as a parameter for API functions that we want to require a token. For our Hyper implementation, we run our extract header token in the router and then directly pass the token into the API endpoint function.

Right now, for our token extraction implementations, we can only extract a string; however, in *Chapter 10*, we will upgrade our token to hold the user ID and encrypt the data.

If we now run our basic test Bash script, we get the following contents:

```
# File: outcome.txt
{"pending":[{"title":"writing","status":"PENDING"}],"done":[]}
{"pending":[
  {"title":"coding","status":"PENDING"},
  {"title":"writing","status":"PENDING"}
],"done":[]}
{"pending":[{"title":"writing","status":"PENDING"}],"done":[]}
"token not found in request header"
```

Here, we can see that our create and delete HTTP calls are working. However, our update did not work, as we did not have a token in the header. This is what we want because we now require our update API endpoint to have a token in the header.

We can add a token to our header with the following code:

```
# File: scripts/basic_test.sh
. . .
curl -X PATCH http://127.0.0.1:8080/api/v1/update \
  -H "Content-Type: application/json" \
  -H "token: some token" \
  -d '{"title": "writing", "status": "DONE"}' >> output.txt
kill $PID
```

Now that we have added a token, if we run our script again, we will see that our update works and that our state is back to normal. We can now slot in requirements for tokens in the header for any of our API endpoints.

Summary

In this chapter, we have put all of what we learned in the previous chapters to good use. We fused the logic from the core, which loads and saves to-do items from a JSON file, and looked at the to-do item process logic by using the basic views from Actix-web. With this, we have seen how the isolated modules click together. We will keep reaping the benefits of this approach in the next few chapters as we rip out the JSON file and replace it with a database.

We also managed to utilize the `serde` crate to serialize complex data structures. This allows our users

to get the full state update returned to them when they make an edit. We can see that the power of Rust enables us to do some highly customizable things to our server, without us having to dig deep into the framework.

Rust has a strong future in web development. With a few lines of code, we can build our own middleware. Our JSON serialization structs were made possible with just one line of code, and the traits provided by Actix enabled us to define the parameter in the view function, thus enabling the view to automatically extract the data from the body and serialize it into the struct. This scalable, powerful, and standardized way of passing data is more concise than many high-level languages. We can now fully interact with and inspect every part of the HTTP request. What's more, the consistency of extracting JSON from bodies and tokens from headers is consistent across Rocket, Axum, and Actix. This is a testament to the trait-based approach for Rust in general.

Now that we are processing and returning well-structured data to the user, we can start displaying it in an interactive way for our user to point and click when editing, creating, and deleting to-do items.

In the next chapter, we will be serving HTML, CSS, and JavaScript from the Actix-web server. This will enable us to see and interact with to-do items via a graphical user interface, with the JavaScript making API calls to the endpoints we defined in this chapter.

You may feel the urge to keep up the momentum of building a better API by adding a database now. However, with the frontend, there are a fair number of moving parts, and it is easier to establish the frontend and run/handle Docker at the same time, as we will be using Docker to run isolated databases.

Questions

1. What is the difference between a GET and a POST request?
2. Why would we have middleware when we check credentials?
3. How do you enable a custom struct to be directly returned in a view?
4. How do you enact middleware for the server?
5. How do you enable a custom struct to serialize data into the view?

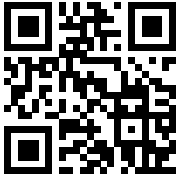
Answers

1. A GET request can be cached, and there are limits to the types and amount of data that can be sent. A POST request has a body, which enables more data to be transferred. Also, it cannot be cached.
2. We use middleware to open the header and check the credentials before sending the request to the desired view. This gives us the opportunity to prevent the body from being loaded by returning an auth error before loading the view, preventing the potentially malicious body.
3. For the struct to be directly returned, we will have to implement the Responder trait. During this implementation, we will have to define the `responded_to` function that accepts the HTTP request struct. The `responded_to` function will be fired when the struct is returned.

4. To enact middleware, we can implement the `FromRequest` trait for a struct, and then pass that struct as a parameter in the function of the API endpoint.
5. We decorate the struct with the `#[derive(Deserialize)]` macro. Once we have done this, we define the parameter type to be wrapped in a JSON struct: `parameter: web::Json<ToDoItem>`.

Learn more on Discord

To join the Discord community for this book – where you can share feedback, ask questions to the author, and learn about new releases – follow this QR code: <https://packt.link/EaKXL>



7

Displaying Content in the Browser

We are now at the stage where we can build a web application that can manage a range of HTTP requests with different methods and data. This is useful, especially if we are building a server for microservices. However, we might also want non-programmers to interact with our application to use it. To enable non-programmers to use our application, we must create a graphical user interface.

It must be noted that this chapter does not contain much Rust. This is because other languages exist to render a graphical user interface. We will mainly use HTML, **JavaScript (JS)**, and CSS. These tools are mature and widely used for frontend web development. Whilst I personally love Rust (otherwise I wouldn't be writing a book on it), we must use the right tool for the right job. At the time of writing this book, we can build a frontend application in Rust using the Yew framework. However, being able to fuse more mature tools into our Rust technical stack is a more valuable skill.

This chapter will cover the following topics:

- Building out a development setup
- Serving the frontend from Rust
- Connecting backend API endpoints to the frontend
- Creating React components
- Rendering multiple React components
- Inserting styles with CSS

In the previous edition of this book, we created a React server using the `npx` command, meaning that the React application was created using an out-of-the-box configuration. We then served the frontend separately. In this edition, we still set up a development server for quick iteration, but we'll embed all of the frontend application into the Rust binary and get the Rust server to directly serve the frontend itself, meaning that we only need one Rust binary for deployment, as opposed to running a Rust server and React server separately.

In this chapter, we will be building our frontend application with esbuild without using out-of-the-box preconfigured tools. This will help you understand how the frontend is built and served. This will also empower you to come up with creative solutions and solve problems when they come up. We will also be building the frontend React application in TypeScript, as JavaScript still dominates the frontend and has the richest ecosystem. WASM frontend applications written in Rust still have big payloads compared to JavaScript payloads.

By the end of this chapter, you will know how frontend assets are served, and you will be able to utilize this knowledge to get Rust code to serve JS frontend applications. You will also be able to build a React application with different components and insert CSS into that application so the users can interact with it. To get the frontend talking to the backend, you will learn how to make API calls in the frontend to the backend.

Technical requirements

You can download the example project and code for this book by following the instructions in the **Download the example code files** section in the *Preface* of this book.

This chapter's code files are included in the downloadable code bundle.

You will also need esbuild to compile the React application, so you will need to install esbuild. You can do this using the following link: <https://esbuild.github.io/getting-started/>

Building out a development setup

Before we can start writing code that is executed in the browser to display our to-do items, we must set up our development environment. For our environment to support a React application that is built using esbuild, we need the following file layout:

```
— ingress
— glue
— nanoservices
— scripts
— Cargo.toml
└─ frontend
    ├── esbuild.js
    ├── ts.config.json
    ├── package.json
    ├── serve.json
    ├── public
    │   └─ index.html
    └─ src
        └─ index.tsx
```

Here, we can see that we have created an `ingress` and `frontend` directory. The `ingress` directory will house all the frontend code and will also have the main Rust server that will serve the frontend

application and all backend endpoints. (The ingress directory is called ingress as the ingress of a system is where all the incoming traffic is handled.)

All our frontend code will be in the frontend directory, and all the assets that we will be serving after a frontend build will be in the frontend/public directory.



Why React?

At the time of writing this book, React is the dominant frontend framework. Multiple other frameworks have claimed that they are going to replace React, and while this is a possibility in the future, it has not happened yet. However, if you want to use another frontend framework, this chapter will still be useful. This chapter focuses on building and serving a React application. We will build the React application using esbuild, and we will serve the React app using two different methods, NGINX and the Rust server itself. If you want to use a different library, such as Angular or Vue, you can still build the frontend using esbuild, and you will still be able to serve the built frontend application using Rust or NGINX. The HTTP requests, CSS, and JSX will still be the same. However, you will have to read the specific API of the JavaScript framework that you want to use.

A good place for us to start is defining the parameters around the frontend build. This way, we will get to see where our entry point for the build is, the options for the build, and the output directory where the finished build is put.

To define the build, we must first import the following:

```
// File: frontend/esbuild.js
const esbuild = require('esbuild');
const cssModulesPlugin = require('esbuild-css-modules-plugin');
```

The esbuild is used to define the build, and the cssModulePlugin is a plugin that enables esbuild to locally scope CSS class names to the JavaScript/TypeScript file, importing and using that CSS. This removes clashes of CSS names from other JavaScript/TypeScript files. This will help us scale our code as the project grows and gets more complex.

With these imports, we then define the build with the following code:

```
// File: ingress/frontend/esbuild.js
esbuild.build({
  plugins: [cssModulesPlugin()],
  entryPoints: ['src/index.tsx'],
  bundle: true,
  outfile: 'public/bundle.js',
  format: 'esm',
  define: {
    'process.env.NODE_ENV': '"production"',
  },
},
```

```
minify: true,
sourcemap: true,
loader: {
  '.js': 'jsx',
  '.tsx': 'tsx',
  '.ts': 'ts',
  '.wasm': 'binary',
  '.css': 'css'
},
}).catch(() => process.exit(1));
```

Our build parameters have the following outcomes:

- **plugins:** Here, we add our CSS plugin, but we can also add other plugins if we want them.
- **entryPoints:** Defines the entry point for the build. So, if any files are linked to the `public/bundle.js` file via imports in the code, these files will be included in the build.
- **bundle:** Instructs esbuild to bundle all dependencies into a single output file. This makes it easier for us to deploy.
- **outfile:** Where the bundle will be put once the build has finished.
- **format:** Sets the module format of the output bundle to the ES module standard for JavaScript.
- **define:** Replaces instances of `process.env.NODE_ENV` with "production" in the code for optimizations.
- **minify:** Enables minification to cut out unnecessary symbols and whitespace, making the bundle file smaller.
- **sourcemap:** Maps the bundled code back to source files, which is handy for debugging.
- **loader:** Specifies the types of files that we load into the builder.

While this build config will enable us to build our React application with esbuild, we need some node modules and a node process to run esbuild. We can manually define our node dependencies and build processes in our `frontend/package.json` file with the following code (as opposed to using the `npm init` command):

```
// File: frontend/package.json
{
  "name": "es_react",
  "version": "1.0.0",
  "scripts": {
    "build": "node esbuild.js",
    "serve": "serve public"
  },
  "dependencies": {
    "react": "^18.2.0",
    "react-dom": "^18.2.0"
  }
}
```

```
    },  
    "devDependencies": {  
      "esbuild": "^0.24.0",  
      "esbuild-css-modules-plugin": "^3.1.0",  
      "serve": "^14.2.1"  
    }  
  }  
}
```

Here, we can see that our application dependencies are just React-based and these dependencies are defined in the "dependencies" section. We have also defined the dependencies that we want for the build and serving in the "devDependencies" section.

We can see that we are using esbuild for the build. The build is achieved by running node against the esbuild.js file, which is the build config. Once the build is done, there will be a bundle.js in the public directory where our whole application is compiled to.

We have also defined the esbuild and esbuild-css-modules-plugin that we are using for the application build, and the serve dependency enables us to serve the React app when developing our application. These are in the "devDependencies" section because we do not need to add these modules to our bundle to be served to the client.

Before we move on to writing any TypeScript code, we must define our TypeScript build in the ts.config.json file as follows:

```
{  
  "compilerOptions": {  
    "target": "es5",  
    "lib": ["dom", "dom.iterable", "esnext"],  
    "allowJs": true,  
    "skipLibCheck": true,  
    "esModuleInterop": true,  
    "allowSyntheticDefaultImports": true,  
    "strict": true,  
    "forceConsistentCasingInFileNames": true,  
    "module": "CommonJS",  
    "moduleResolution": "node",  
    "resolveJsonModule": true,  
    "isolatedModules": true,  
    "noEmit": true,  
    "jsx": "react-jsx"  
  },  
  "include": ["src"]  
}
```

To avoid bloating the chapter, we will not break down the TypeScript config. This book is a book about Rust web programming, not a book on TypeScript.

If we tried to build our system now, it would break, as we have not defined our app in our entry point. Our app entry point is JavaScript that gets an HTML element by ID, to insert our React app into that HTML element. If we were using TypeScript, the TypeScript would be converted into JavaScript when the bundle is created, as JavaScript is the code that runs in the browser.

For our entry point, we can build the logic around injecting our React application into the HTML file that is served by the server with the following code:

```
// File: frontend/src/index.tsx
import React from 'react';
import ReactDOM from "react-dom/client";

const App = () => {
  return (
    <div>
      <h1>Hello World</h1>
    </div>
  );
};

const root = ReactDOM.createRoot(document.getElementById('root'));
root.render(<App />);
```

For now, we are just injecting a `<h1>Hello World</h1>` into the element called `root` in the HTML file that is served. We will replace this placeholder later with our frontend application with to-do items.

Now we need to get our development environment to serve our application.

Connecting our development environment and application

To do this, we first must have an HTML file that has an HTML element called `root` and loads our JavaScript bundle file that houses our React application. This is essentially an entry point for serving our application, so we can call this HTML file `index.html`. We will put this HTML file in our public directory because this is where the assets being served are stored for our server to access.

Our `index.html` file takes the following form:

```
<!-- File: frontend/public/index.html -->
<!DOCTYPE html>
<html>
  <head>
    <title>My React App</title>
  </head>
  <body>
```

```
<div id="root"></div>
<script type="module" src="./bundle.js"></script>
</body>
</html>
```

We can configure the server to point to this `index.html` file with the server config file in the root directory of the following frontend:

```
// File: frontend/serve.json
{
  "rewrites": [
    { "source": "**", "destination": "./public/index.html" }
  ]
}
```

This means that we route all our requests to the server to the `index.html` file. Therefore, all requests are going to load the HTML file, which will download the bundled JavaScript file; this will then insert the React app into the root element of the `index.html` file. We can now run our server.

Running our frontend server

First, we must install the node modules for the building and serving of our application with the following command:

```
npm install
```

The `node_modules` directory will be created with all the packages needed for the building and serving of our application. We then build our application with the following command:

```
npm run build
```

After a stream of build logs, we should see the `bundle.js` file in the public directory. We can then serve our app with the following command:

```
npm run serve
```

This gives us the following printout:

```
> es_react@1.0.0 serve
> serve public

[
  Serving!
  - Local:    http://localhost:3000
  - Network:  http://192.168.1.83:3000
  Copied local address to clipboard!
]
```

If we visit the localhost in the browser, we are greeted with the content shown in *Figure 7.1*.



Figure 7.1 – Our Basic React app viewed for the first time

Our logs in our console will show a printout along the same lines as the following:

```
HTTP 29/10/2024 17:32:54 ::1 GET /
HTTP 29/10/2024 17:32:54 ::1 Returned 304 in 17 ms
HTTP 29/10/2024 17:32:54 ::1 GET /bundle.js
```

Here, we can see that our server gets the `index.html` file with the `HTTP 29/10/2024 17:32:54 ::1 GET /` log, as we routed the GET request to the `./public/index.html` file in our `serve.json` file earlier. The logs also show that we then get the `bundle.js` file. This makes sense as our `index.html` file loads the `bundle.js` file.

We can also inspect the network panel in our browser if we open developer tools in the browser. We do this by right-clicking on the web page in the browser and clicking **Inspect**. The **Network** tab in the developer tools will give us the log seen in *Figure 7.2*.

Name	Status	Type	Initiator	Size	Time	Waterfall
localhost	200	document	Other	561 B	3 ms	
bundle.js	200	script	(index):8	46.1 kB	11 ms	

Figure 7.2 – Network logs of serving our React app

We can see that the HTML document is served, and then the JavaScript script is served. And here we have it! Our development environment is ready to build and serve our React application.



As an alternative approach, we could have used the Create React App tool to create a React application and serve it. However, there is a reason why you're working through a book as opposed to speed-reading a tutorial online. Here, we are taking the time to go through how components in our stack work and interact with each other. This gives us more flexibility and the ability to build a system when there are multiple moving parts. For instance, we could reconfigure our `serve.json` file to have the following configuration:

```
{
  "rewrites": [
    {
      "source": "/test",
      "destination": "./public/test.html"
    },
  ],
}
```



```
{
  "source": "**",
  "destination": "./public/index.html"
}
]
```

As an alternative approach, we could have used the Create React App tool to create a React application and serve it. However, there is a reason why you're working through a book as opposed to speed-reading a tutorial online. Here, we are taking the time to go through how components in our stack work and interact with each other. This gives us more flexibility and the ability to build a system when there are multiple moving parts. For instance, we could reconfigure our `serve.json` file to have the following configuration:



```
{
  "rewrites": [
    {
      "source": "/test",
      "destination": "./public/test.html"
    },
    {
      "source": "**",
      "destination": "./public/index.html"
    }
  ]
}
```

With the preceding configuration, we can see that if the endpoint of the URL is `test`, we serve a different HTML file. This different HTML file could load a different JavaScript application using a different framework, or even load **WebAssembly (WASM)** assets. We can have multiple different bundles that are loaded by different HTML files. But we can even go further. Now that we know how the frontend is served, we do not have to use the `serve` node module; we can write our own servers that do this!

In the next section, we are going to apply what we have learned about bundling and serving frontend assets to embed those frontend assets into our Rust server binary in order for them to be served by our Rust server.

Serving the frontend from Rust

We now know that we can serve our React application if we can serve an HTML file that can point to the `bundle.js` file. Of course, we can do this using a Rust server. To achieve this, we can create an ingress workspace with the following directory layout:

```
├─ Cargo.toml
├─ README.md
├─ frontend
│   ├── esbuild.js
│   ├── package-lock.json
│   ├── package.json
│   ├── public
│   │   ├── bundle.js
│   │   ├── bundle.js.map
│   │   └─ index.html
│   ├── serve.json
│   ├── src
│   │   └─ index.tsx
│   └─ ts.config.json
└─ ingress
    ├── Cargo.toml
    └─ src
        └─ main.rs
```

Here, the server in the `ingress/src/main.rs` file is going to point to our `frontend/public` folder to serve the assets. Now that we have our outline, we can serve our frontend by carrying out the following steps:

1. Defining our frontend embedding dependencies
2. Embedding the HTML file
3. Embedding the React application
4. Serving the embedded frontend

We can start by defining our dependencies needed to embed the frontend and serve it.

Defining our frontend embedding dependencies

At this point, our `Cargo.toml` file in the root of our system now looks like the following:

```
# File: Cargo.toml
[workspace]
resolver = "2"
members = [
    "glue",
```

```

    "ingress",
    "nanoservices/to_do/core",
    "nanoservices/to_do/dal",
    "nanoservices/to_do/networking/actix_server"
]

```



By the end of the book, our ingress will accept all incoming requests and either serve frontend assets or backend endpoints. All our nanoservices will essentially be compiled into the ingress workspace. In this chapter, we will only focus on the serving of frontend assets. However, by *Chapter 16*, our entire system will be compiled into the ingress workspace.

Before we write any code for our ingress, our `ingress/Cargo.toml` file will need to have the following dependencies:

```

# File: ingress/Cargo.toml
[package]
name = "ingress"
version = "0.1.0"
edition = "2021"

[dependencies]
rust-embed = "8.3.0"
mime_guess = "2.0.4"
actix-web = "4.5.1"
tokio = {
    version = "1.35.0",
    features = ["macros", "rt-multi-thread"]
}

```

Here, we will be using the `rust-embed` crate to embed the frontend assets into our Rust binary on the build. We then use the `mime_guess` crate to work out what type of file we are serving when defining the file type, when returning it to the browser. This will help us serve CSS, images, WASM files, etc.



Other web frameworks for ingress

In the GitHub repository, the `alternative-ingresses` directory in the root of the system houses examples of ingresses written in Hyper, Axum, and Rocket. When there are changes in the approach, depending on the web framework, a high-level description of the differences will be covered in the book.

The last dependency we need to define is our HTML file in the root of the ingress workspace, which has the following content:

```
<!-- File: ingress/index.html -->
<!DOCTYPE html>
<html>
<head>
  <title>My To Do App</title>
</head>
<body>
  <div id="root"></div>
  <script type="module" src="./frontend/public/bundle.js">
  </script>
</body>
</html>
```

We now have everything in place to serve a frontend application. Finally, we can write our `main.rs` file in our ingress workspace to embed our HTML.

Embedding the HTML file

First, we must import the following structs and traits:

```
// File: ingress/src/main.rs
use actix_web::{
    web, App, HttpServer, Responder, HttpResponse, HttpRequest
};
use rust_embed::RustEmbed;
use std::path::Path;
```

Now we embed our entry point – that is, the `ingress/index.html` file. We also need to return the HTML in an HTTP response with the following code:

```
// File: ingress/src/main.rs
async fn index() -> HttpResponse {
    HttpResponse::Ok().content_type("text/html")
        .body(include_str!("../index.html"))
}
```

The `include_str!` macro embeds the contents of the file as a string, so that once the Rust code is compiled, we do not need to move the `index.html` file with the Rust binary; we only need the compiled Rust binary to serve the HTML. Our HTML file is now embedded in our Rust binary and ready to be served. We can now move on to embedding our React application.

Embedding the React application

If we want to embed our React application, we must embed the `ingress/frontend/public` directory with the following code:

```
// File: ingress/src/main.rs
#[derive(RustEmbed)]
#[folder = "../frontend/public"]
struct FrontendAssets;
```

With this, we can access files from the `frontend/public` directory via the `FrontendAssets` struct, as our frontend assets are now embedded into the Rust binary.

The `serve_frontend_asset` is going to be called by a catch-all function that is going to be connected to the server. Because we do not have any async calls in the `serve_frontend_asset` function, the `serve_frontend_asset` does not need to be async.

Now that we have embedded our frontend assets, we can then serve these assets.

Serving the embedded frontend

To serve a frontend asset, we get the path to the frontend asset and load the data, returning it with the following code:

```
// File: ingress/src/main.rs
fn serve_frontend_asset(path: String) -> HttpResponse {
    let file = match Path::new(&path).file_name() {
        Some(file) => file.to_str().unwrap(),
        None => return HttpResponse::BadRequest()
            .body("404 Not Found")
    };
    match FrontendAssets::get(file) {
        Some(content) => HttpResponse::Ok()
            .content_type(mime_guess::from_path(&file))
            .first_or_octet_stream().as_ref()
            .append_header(
                ("Cache-Control", "public, max-age=604800")
            )
            .body(content.data),
        None => HttpResponse::NotFound().body("404 Not Found")
    }
}
```

When it comes to getting a file from our `FrontendAssets` struct, we do not need a full path; instead, we only need the path from the `../frontend/public` path. Because of this, we trim the path of the request, as our HTML file loads the bundle with the `/frontend/public/bundle.js` path. So, when our embedded HTML file makes the request for the bundle, we will trim the path to `bundle.js` by just getting the `file_name` of the path.

We then see if the file has been embedded into the `FrontendAssets` struct. If the file is not present, we return a not found response. If there is a file, we use the `mime_guess` crate to insert the data type into the header of the response. We also insert a `Cache-Control` parameter into the header. The `max-age` is calculated in seconds, so our assets will be cached in the browser for a week, which is the standard timeframe for caching. Finally, we insert the data of the file in the body of the HTTP response, and there we have it, our embedded frontend assets can be extracted and served to the browser.



When editing the frontend application, we must remember that the application is cached in the browser. So once the edit is done, we restart the server, and then in the browser, right-click inside the pane and select **Inspect**. You can then right-click on the refresh button and select **Empty cache hard reload** to ensure that you load the updated frontend that has your edits; otherwise, you will not see the changes in your browser.

We could just wrap the `serve_frontend_asset` function in an `async` function that accepts a request and returns an HTTP response. However, we will want to serve frontend assets no matter what the URL request is and serve backend API endpoints. This means that we have a function that is executed when all other endpoints are not hit. This is essentially a catch-all.

Our function takes the following signature:

```
// File: ingress/src/main.rs
async fn catch_all(req: HttpRequest) -> impl Responder {
    . . .
    index().await
}
```

For the first check in our catch function, if the request has `/api/` in the URL, then we return a not found, because this request was clearly intended for a backend endpoint. If the request has hit the catch function, this means that the request has not matched with any of the backend endpoints. This check is defined with the following code:

```
// File: ingress/src/main.rs (function = catch_all)
if req.path().contains("/api/") {
    return HttpResponse::NotFound().finish()
}
```

For our next check, we can serve the frontend asset if there is a `/frontend/public` path in the request with the following code:

```
// File: ingress/src/main.rs (function = catch_all)
if req.path().contains("frontend/public") {
    return serve_frontend_asset(req.path().to_string())
}
```

Our next check is to inspect the file type with the following code:

```
// File: ingress/src/main.rs (function = catch_all)
let file_type = match mime_guess::from_path(&req.path())
    .first_raw() {
    Some(file_type) => file_type,
    None => "text/html"
};
```

If there is no file type in the request, it might just be hitting the route or our URL; therefore, we know that we want the `index.html`. However, the request might request another file type. For instance, when adding assets like images, it is common to reference the image path relative to the bundle, meaning that `/frontend/public` path is not in the asset path. Therefore, if the file type is not an HTML file, we can serve the frontend asset with the following code:

```
// File: ingress/src/main.rs (function = catch_all)
if !file_type.contains("text/html") {
    return serve_frontend_asset(req.path().to_string())
}
```

If the request passes all these checks, we can just serve the `index.html` file with `index.await` return, as shown in the initial signature of the `catch_all` function.

We are finally at the point of defining our server that has our `catch_all` function as the default service. We do this with the following code:

```
// File: ingress/src/main.rs
#[tokio::main]
async fn main() -> std::io::Result<()> {
    HttpServer::new(|| {
        App::new()
            .default_service(web::route().to(catch_all))
    })
    .bind("0.0.0.0:8001")?
    .run()
    .await
}
```

Now, when we perform a `cargo run` command, we can access our React application through our Rust server using the URL `0.0.0.0:8001/something/else`, as seen in *Figure 7.3*.

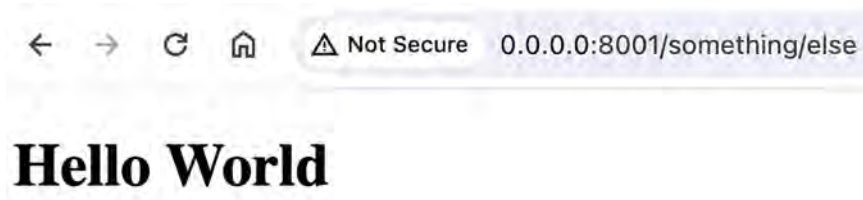


Figure 7.3 – Our Basic React app view from our Rust server

We can see that even though our URL has `/something/else/` at the end, our React application is served. We will let our React application handle the missing frontend endpoint.

If we make a backend call that our server does not support, we will get a not found response, as seen in *Figure 7.4*.



Figure 7.4 – A not found response to an API endpoint

Here, we can conclude that our Rust server is serving our frontend.

Other web frameworks serving embedded frontends

For the Axum, Hyper, and Rocket frameworks, the method of embedding the frontend into the Rust binary is the same as for the Actix example we covered. However, there are some slight changes in how we return the files for the frontend.

Axum

Here, our `serve_frontend_asset` function returns a `Response<Body>`, and we define the header and file just like we do for Actix Web. Our `catch_all` function has the following signature:

```
async fn catch_all(uri: axum::http::Uri) -> impl IntoResponse
```


Our `catch_all` function is mounted to our server using the `fallback` as seen in the following code:

```
let app = Router::new()
  .route("/", get(index))
  .fallback(catch_all);
```

Rocket

Due to Rocket having a higher-level API with more abstractions, we can serve our frontend by implementing our own custom responder with the following code:

```
struct CustomResponder<'r> {
    status: Status,
    content_type: ContentType,
    data: Cursor<Vec<u8>>,
    data_len: usize,
    _phantom: PhantomData<&'r ()>,
}

impl<'r> Responder<'r, 'static> for CustomResponder<'r> {
    fn respond_to(self, _: &'r Request<'_>)
        -> response::Result<'static> {
        Response::build()
            .status(self.status)
            .header(self.content_type)
            .sized_body(self.data_len, self.data)
            .ok()
    }
}
```

The `PhantomData` will enable the compiler to keep track of the lifetime. Our `serve_frontend_asset` and `catch_all` functions return the `CustomResponder`. We create the `Cursor` from the data file with the following code:

```
let content_type: ContentType = ContentType::from_extension(
    file.split('.').last().unwrap_or("")
).unwrap_or(ContentType::Any);
let raw_data = content.data.to_vec();
let data_len = raw_data.len();
let data = Cursor::new(raw_data);
```



Our `catch_all` function is mounted to our server using the `catcher!` macro with the following code:

```
let rocket = rocket::custom(&config);
let rocket = rocket.mount("/", routes![index,
    serve_frontend_asset]);
let rocket = rocket.register("/", catchers![catch_all]);
rocket.launch().await?;
```

Hyper

Because Hyper is lower level, we do not need to implement many extra steps. Our `serve_frontend_asset` function returns a `Response<Full<Bytes>>`. This response is constructed using the following code:



```
let bytes = Bytes::from(content.data.to_vec());
return Response::builder()
    .header("Content-Type", mime_type)
    .header("Cache-Control", "public, max-age=604800")
    .body(Full::new(bytes)).map_err(|e| NanoServiceError::new(
        e.to_string(),
        NanoServiceErrorStatus::Unknown
    ))
```

Because we are implementing our own `handle` function that manually matches the HTTP method and URL, after all the manual matches are exhausted, we can return the `catch_all` function with the following code:

```
let response: Result<Response<Full<Bytes>>, NanoServiceError> =
    match (
        req.method(), segments.as_slice()) {
        . . .
        _ => Ok(catch_all(path).await),
    };
```

For the next section, we will connect the API endpoints for our to-do items to the frontend.

Connecting backend API endpoints to the frontend

We are going to connect our frontend to our backend server via HTTP requests. This means that we must connect our to-do nanoservice to our ingress.

Before we can do this, we must add the following dependencies to our ingress `Cargo.toml` file with the following code:

```
# File: ingress/Cargo.toml
```

```
[dependencies]
. . .
actix-cors = "0.7.0"
to_do_server = {
    path = "../nanoservices/to_do/networking/actix_server",
    package = "to-do-actix-server"
}
```

We will use `actix-cors` to allow cross-origin requests, enabling our application to accept requests from different domains. Right now, this will not be an issue as our frontend will be making requests from the localhost on the same computer as the server. However, when we deploy our application, the React application will be running on a user's computer and their requests will come from their computer, not the same computer that our server is running on.

We have also been compiling our to-do nanoservice into our ingress binary. However, note that we assign an alias of our to-do-actix-server package to `to_do_server`. This is because we might have another nanoservice that has the networking layer called `actix_server` and we do not want clashes. We might also get errors in compiling our project at this point. It is likely that there is no `nanoservices/to_do/networking/actix_server/src/lib.rs` file. We need a `lib.rs` file to enable the to-do server to be a package. Inside the `lib.rs` file, we must make the API publicly accessible with the following code:

```
// File: nanoservices/to_do/networking/actix_server/src/lib.rs
pub mod api;
```

We can now redefine our server in the `main.rs` file of the ingress workspace. First, we use the following:

```
// File: ingress/src/main.rs
. . .
use to_do_server::api::views_factory as to_do_views_factory;
use actix_cors::Cors;
```

We can now attach the views factory and CORs to our server with the following code:

```
// File: ingress/src/main.rs

#[tokio::main]
async fn main() -> std::io::Result<()> {
    HttpServer::new(|| {
        let cors = Cors::default().allow_any_origin()
            .allow_any_method()
            .allow_any_header();

        App::new()
            .configure(to_do_views_factory)
            .wrap(cors)
            .default_service(web::route())
    })
    .bind("0.0.0.0:3000")
    .await
    .unwrap()
    .run()
    .await
    .unwrap()
}
```

```

        .to(catch_all))
    })
    .bind("0.0.0.0:8001")?
    .run()
    .await
}

```

This is it, our server is now ready to serve our to-do endpoints to anywhere in the world if we had our server running on a public server that others can make requests to.

Other web frameworks enabling CORs

For the Axum, Hyper, and Rocket frameworks there are slight differences in the way we define the CORs policy.

Axum

We need the following dependency for our CORs middleware:

```
tower-http = { version = "0.6.1", features = ["cors"] }
```

We then define the CORs and attach that middleware to the server with the following code:

```

let cors = CorsLayer::new()
    .allow_origin(Any)
    .allow_methods(Any)
    .allow_headers(Any);

let app = Router::new()
    .layer(cors)
    .route("/", get(index))
    .fallback(catch_all);

```

Rocket

For CORs middleware we must create a struct and implement the `Fairing` trait. The `Fairing` trait in Rocket allows us to hook into various stages of a request's lifecycle, such as when a request is received, when a response is being finalized, or when the application starts or stops. By implementing `Fairing`, we can modify responses to include the necessary CORS headers, ensuring that browsers allow cross-origin requests. Here's how we can do it with the following code:

```

pub struct CORS;

#[rocket::async_trait]
impl Fairing for CORS {
    fn info(&self) -> Info {

```





```

    Info {
        name: "Add CORS headers to responses",
        kind: Kind::Response
    }
}
async fn on_response<'r>(
    &self, _request: &'r Request<'_,
    response: &mut Response<'r>
) {
    response.set_header(Header::new(
        "Access-Control-Allow-Origin", "*"
    ));
    response.set_header(Header::new(
        "Access-Control-Allow-Methods",
        "POST, GET, PATCH, OPTIONS"
    ));
    response.set_header(
        Header::new(
            "Access-Control-Allow-Headers",
            "*"
        ));
    response.set_header(
        Header::new(
            "Access-Control-Allow-Credentials",
            "true"
        ));
}
}

```

As we can see, we are allowing all origins and methods. We can then attach the CORS middleware to the server with the following code:

```

let rocket = rocket::custom(&config);
let rocket = rocket.mount("/", routes![index, serve_frontend_asset]
);
let rocket = rocket.register("/", catchers![catch_all]);
let rocket = rocket.attach(CORS);
rocket.launch().await?;

```

Hyper

Because Hyper is lower level, all methods and origins are allowed, as we have not written any code to block any origins or methods.

While our server is now ready to serve our to-do endpoints anywhere in the world, if we had it running on a public server, there are a couple of moving parts. For instance, you must build the frontend before embedding it. If you make a change to the frontend, there is a risk that you can rebuild your server without rebuilding the frontend. This can lead to time wasted trying to figure out why your changes are not showing. We can automate this with the following bash script for running a server in the scripts directory of the ingress:

```
#!/usr/bin/env bash
# File: ingress/scripts/run_server.sh

# navigate to directory
SCRIPTPATH="$( cd "$(dirname "$0")" ; pwd -P )"
cd $SCRIPTPATH

cd ../..
cd frontend
npm install
npm run build
cd ../ingress
cargo clean
cargo run
```

We are now less likely to miss a step in our server build process, saving us from confusion. We have added a lot of moving parts now, so it is a good idea to run the script and see if the server compiles and runs.



Note that this script will only work on computers where Rust and Node are installed. In *Chapter 16*, we containerize the server in Docker and deploy the containerized server on AWS.

We can now move on to the frontend, which has the following new files and directories:

```
| . . .
| src
|   | api
|   |   | create.ts
|   |   | delete.ts
|   |   | get.ts
|   |   | update.ts
|   |   | url.ts
|   |   | utils.ts
|   | index.tsx
```

```
|   └─ interfaces
|       └─ todoItems.ts
|
. . .
```

We can see that the API file structure follows the same layout at the core and networking layer. This is so our entire system is consistent, and you do not have to memorize three different structures for the core, networking, and frontend. Now that we have our project structure, we can implement our HTTP request using the following steps:

1. Defining our interfaces
2. Defining our API utils
3. Defining our get API call
4. Integrating our get API call in our main app
5. Building out the rest of the API calls

However, before we write any code, we need the axios package to make HTTP requests. Our frontend package.json file dependencies should look like the following:

```
// File: frontend/package.json
"dependencies": {
  "react": "^18.2.0",
  "react-dom": "^18.2.0",
  "axios": "^1.6.8"
}
```

We could also install the axios package by running the `npm install axios` command. We now have everything we need to define our interfaces.

Defining our interfaces

We define our interfaces through the following code:

```
// File: frontend/src/interfaces/todoItems.ts
export enum TaskStatus {
  PENDING = 'PENDING',
  DONE = 'DONE'
}

export interface TodoItem {
  title: string;
  status: TaskStatus;
}

export interface TodoItems {
  pending: TodoItem[];
  done: TodoItem[];
}
```

These interfaces are merely the structure of the data that we are passing between the server and the frontend. We can see that they are similar to the structs that we have been using in our Rust server. With these interfaces, we will be able to create items, update them, delete them, and get all items.

There is one more interface that we must construct, and this is the URL. We could define our URL as and when we need it; however, this makes it harder for us to maintain. If we have all our routes defined in one place, we can update mechanisms around constructing the URL, and we can easily keep track of all our URL endpoints. Seeing as our URL will be used when making API calls, we can define our URL interface in the `api/url.ts` file with the following outline:

```
// File: frontend/src/api/url.ts
export class Url {

    baseUrl: string;
    create: string;
    getAll: string;
    update: string;

    constructor() {
        . . .
    }
    static getBaseUrl(): string {
        . . .
    }
    deleteUrl(name: string): string {
        return `${this.baseUrl}api/v1/delete/${name}`;
    }
}
```

Here, we can see that our `deleteUrl` return value is slightly different from the other endpoints. This is because the name is passed into the delete URL. All the other URLs, which match the endpoints in the Rust server, are defined in the constructor with the following code:

```
// File: frontend/src/api/url.ts
constructor() {
    this.baseUrl = Url.getBaseUrl();
    this.create = `${this.baseUrl}api/v1/create`;
    this.getAll = `${this.baseUrl}api/v1/get/all`;
    this.update = `${this.baseUrl}api/v1/update`;
}
```


Finally, we must define our base URL. Generally, we want our base URL to be the location of the browser. However, we must remember that we also have a development server for the frontend that runs on the localhost on port 3000, and our development server is on port 8001. Therefore, if our location is on the development server, we need to change the URL to point to the development Rust backend server with the following code:

```
// File: frontend/src/api/url.ts
static getBaseUrl(): string {
  let url = window.location.href;
  if (url.includes("localhost:3000")) {
    return "http://0.0.0.0:8001/";
  }
  Return url;
}
```

We can now build the API utils that handle these interfaces.

Defining our API utils

The utils file is about abstracting some repetitive code that all other API calls will use so that we do not have to repeat ourselves.

Before we write our generic API call functions, we need to import the following:

```
// File: frontend/src/api/utils.ts
import axios, {AxiosResponse} from "axios";
```

With this import, we can build our handle response function, which will have the following signature:

```
// File: frontend/src/api/utils.ts
async function handleRequest<T, X>(
  promise: Promise<AxiosResponse<X>>,
  expectedResponse: number) {
  let response: AxiosResponse<X>;
  . . .
}
```

Here, we can see that our general understanding of async programming from *Chapter 4* is helping us out. In this handle function, we accept a promise of a generic request. In our TypeScript code, we can handle our promises just like we would handle our futures in async Rust code. Inside the function, we await our API call promise and handle the outcome with the following code:

```
// File: frontend/src/api/utils.ts

let response: AxiosResponse<X>;
try {
  response = await promise;
```

```

} catch (error) {
  return {
    status: 500,
    error: "Network Error",
    data: JSON.stringify(error)
  };
}

```

Like in Rust, the `X` is a generic type. We are using a generic type for the response because the response from the server could be different depending on the endpoint that we hit. For instance, later on in the book, we will have authentication sessions, and these API endpoints will not return to-do items.

If the HTTP request was not successful, we can conclude that this was the result of a network error. Therefore, we assign the status of 500, which is a network error. If the HTTP request fails, we want to handle the error as soon as possible. However, if our request succeeds, then we can assert that the response code is what we expect for the call, such as an OK, CREATED, or another response code, and return the data as the type we are expecting with the following code:

```

// File: frontend/src/api/utils.ts
if (response.status === expectedResponse) {
  return {
    status: response.status,
    data: response.data as X
  };
} else {
  return {
    status: response.status,
    error: `expected status ${expectedResponse} got ${response.status}`,
    data: response.data as X
  };
}

```

Now our handle request function is defined, we can lean on it to define a function that executes a post request. This post request will also have data in the body of the HTTP response. Our generic post request function takes the following form:

```

// File: frontend/src/api/utils.ts
export async function postCall<T, X>(
  url: string, body: T,
  expectedResponse: number) {
  let response = axios.post<X | string>(
    url,
    body,

```

```
    {
      headers: {
        'Content-Type': 'application/json',
        'token': "jwt"
      },
      validateStatus: () => true
    });
    return handleRequest(response, expectedResponse);
  }
}
```

The `validateStatus` ensures that we resolve the promise for all response statuses. Note that we have the JWT in the header. Right now, this is a placeholder, but we will be reviewing the JWT in *Chapter 10*.

Finally, we need to define our get call function. Now is a good time to try and build this function yourself as it will be a slight variance to the post call function.

If you attempted to build the function yourself, it hopefully looks like the following code:

```
// File: frontend/src/api/utlis.ts
export async function getCall<X>(  
  url: string,  
  expectedResponse: number) {  
  let response = axios.get<X | string>(  
    url,  
    {  
      headers: {  
        'Content-Type': 'application/json',  
        'token': "jwt"  
      },  
      validateStatus: () => true  
    });  
  return handleRequest(response, expectedResponse);  
}
```

With this, our utils for API calls are fully defined, and we are ready to use these utils to define our GET API call.

Defining our GET API call

For our GET API call, our function takes the following form:

```
// File: frontend/src/api/get.ts  
import { TodoItems } from "../interfaces/todoItems";  
import { getCall } from "../utils";  
import { Url } from "../url";
```

```
export default async function getAll() {
  let response = await getCall<ToDoItems>(
    new Url().getAll,
    200
  );
  return response;
}
```

This function makes an API call to the server to get all the to-do items. And that is it! We can see how implementing new API calls is going to be easy. However, before we implement more endpoints, we must check to see if our API calls work. To test this, we add our API GET call in our app.

Integrating our get API call in our main app

To rewrite the App component, we must ensure that we have the following imported:

```
// File: frontend/src/index.tsx
import React, { useState } from 'react';
import ReactDOM from "react-dom/client";
import getAll from './api/get';
import {ToDoItems} from "./interfaces/toDoItems";
```

We will cover what these imports are used for as and when we use them.

We define the following outline for our App component:

```
// File: ingress/frontend/src/index.tsx
const App = () => {
  const [data, setData] = useState<ToDoItems>(null);
  const [error, setError] = useState<string>(null);
  React.useEffect(() => {
    // ...
  }, []);

  return (
    <div>
      {error ? (
        <div style={{ color: 'red' }}>Error: {error}</div>
      ) : data ? (
        <div>Data loaded: {JSON.stringify(data)}</div>
      ) : (
        <div>Loading...</div>
      )}
    </div>
  );
};
```

Here, we have the `useState`, which handles the state for the `App` component. Our state is going to be the to-do items. We also render “Loading...” if our state is not loaded for the `App` component. If we have an error from the API call, we render the error. If we have loaded the data, we render the data instead.

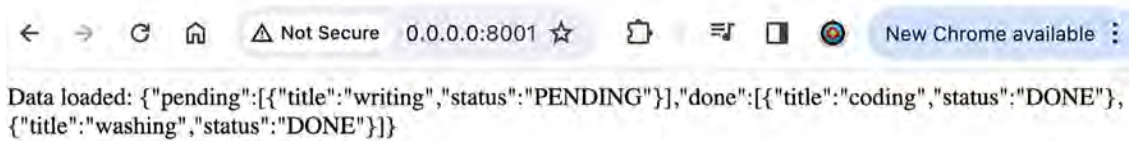
Our `React.useEffect` fires once the `App` component has been loaded. This enables us to configure the state of the component, for instance, by fetching data and performing an action after the component is loaded.

In the `React.useEffect`, we make the GET call with the following code:

```
// File: frontend/src/index.tsx
React.useEffect(() => {
  const fetchData = async () => {
    const response = await getAll();
    if (response.error) {
      setError(response.error);
    } else {
      setData(response.data);
    }
  };
  fetchData();
}, []);
```

Here, we can see that we fire the `setData` function to render the items. If there is an error returned from the `getAll` function, we call the `setError` function to render the error.

We now have nearly everything we need to run our server with a frontend that serves the to-do items. There's one more thing we need before we run the server, however, and that is to ensure that there is a `tasks.json` file in the root of the `ingress` workspace with the tasks. Once this is done, we run the `ingress/scripts/run_server.sh` script and visit the URL. This should give us the output seen in *Figure 7.5*.



```
Data loaded: {"pending":[{"title":"writing","status":"PENDING"}], "done":[{"title":"coding","status":"DONE"}, {"title":"washing","status":"DONE"}]}
```

Figure 7.5 – Our app view rendering to-do items

And there we have it! We can see that our API call is working, and we can access our data from the server.

We now only need to build the create, delete, and update API calls. At this stage, this is a good time to try and build these functions yourself, as there is a bit of repetition with some slight variance when it comes to building the rest of the base API call functions for different call methods. This is a good time to test what you know about creating API calls.

Building out the rest of the API calls

If you tried to create these functions yourself, hopefully, you will have taken an outline like the following approach. As we have covered all we need to know, the approach will merely spell out the code needed.

First, in the `utils.rs` file, we create a DELETE function with the following code:

```
// File: frontend/src/api/utils.ts
export async function deleteCall<X>(  
  url: string,  
  expectedResponse: number) {  
  let response = axios.delete<X | string>(  
    url,  
    {  
      headers: {  
        'Content-Type': 'application/json',  
        'token': "jwt"  
      },  
      validateStatus: () => true  
    })  
  };  
  return handleRequest(response, expectedResponse);  
}
```

In the same file, our PUT function is built with the following code:

```
// File: frontend/src/api/utils.ts
export async function putCall<T, X>(  
  url: string, body: T,  
  expectedResponse: number) {  
  let response = axios.put<X | string>(  
    url,  
    body,  
    {  
      headers: {  
        'Content-Type': 'application/json',  
        'token': "jwt"  
      },  
      validateStatus: () => true  
    })  
  };  
  return handleRequest(response, expectedResponse);  
}
```

Also in the same file, our PATCH function is defined with the following code:

```
// File: frontend/src/api/utlis.ts
export async function patchCall<T, X>(
  url: string, body: T,
  expectedResponse: number) {
  let response = axios.patch<X | string>(
    url,
    body,
    {
      headers: {
        'Content-Type': 'application/json',
        'token': "jwt"
      },
      validateStatus: () => true
    });
  return handleRequest(response, expectedResponse);
}
```

The create function then takes the following form:

```
// File: frontend/src/api/create.ts
import { ToDoItem, ToDoItems, TaskStatus }
from "../interfaces/toDoItems";
import { postCall } from "../utlis";
import { Url } from "../url";

export async function createToDoItemCall(title: string) {
  const toDoItem: ToDoItem = {
    title: title,
    status: TaskStatus.PENDING
  };
  return postCall<ToDoItem, ToDoItems>(
    new Url().create,
    toDoItem,
    201
  );
}
```

The update function is defined with the following code:

```
// File: frontend/src/api/update.ts
import { ToDoItem, ToDoItems, TaskStatus }
from "../interfaces/toDoItems";
```

```
import { patchCall } from "../utils";
import { Url } from "../url";

export async function updateToDoItemCall(
  name: string, status: TaskStatus) {
  const toDoItem: ToDoItem = {
    title: name,
    status: status
  };
  return patchCall<ToDoItem, ToDoItems>(
    new Url().update,
    toDoItem,
    200
  );
}
```

And finally, our delete to-do item function, which is essentially an interface for making a DELETE network call via the deleteCall function, takes the following form:

```
// File: frontend/src/api/delete.ts
import { ToDoItems } from
  "../interfaces/toDoItems";
import { deleteCall } from "../utils";
import { Url } from "../url";

export async function deleteToDoItemCall(
  name: string) {
  return deleteCall<ToDoItems>(
    new Url().deleteUrl(name),
    200
  );
}
```

Now we have defined all the API endpoints that we need. However, we need to interact with these API calls. We must create some to-do item and form components so that we can call these API functions with data.

Creating React components

With React, we can build JSX/TSX components that can be used as templates. These are essentially functions that render HTML with the aid of JavaScript or TypeScript. For instance, if we have a list of items, components enable us to loop through that list of items in JavaScript/TypeScript and render the HTML for those items. These components support internal state; updating the internal state can force a re-render of the component.

We will explore the basic outline of the React component by building the create to-do item component. This component takes an input for the title of the to-do item being created and keeps track of the state of the title. When the create button is clicked, the create component then makes an API call with the title to create the to-do item and then returns the updated state to the parent component. We can house our components in the `frontend/src/components/` directory. Let's create that now. To build our create item form, we must carry out the following steps:

1. Building a create item form
2. Injecting our form component into our app
3. Testing our create item form

With this, we can start creating the item form.

Building a create item form

We can then start building our create component with the following imports and interface in our `CreateItemForm.tsx` file:

```
// File: frontend/src/components/CreateItemForm.tsx
import React, { useState } from 'react';
import { createToDoItemCall } from "../api/create";

interface CreateToDoItemProps {
  passBackResponse: (response: any) => void;
}
```

The `CreateToDoItemProps` interface is what we are going to accept into the component when creating it. (Generally, what gets passed into the component when the component is being created is referred to as props.) In this case, we are passing a function that accepts the response of the create API call. We will see how the function works when we implement our components in the main application.

Now we can define the outline of our create component with the following code:

```
// File: frontend/src/components/CreateItemForm.tsx
export const CreateToDoItem: React.FC<CreateToDoItemProps> = (
  { passBackResponse }
) => {
  const [title, setTitle] = useState<string>("");

  const handleTitleChange = (
    e: React.ChangeEvent<HTMLInputElement>
  ) => {
    setTitle(e.target.value);
  };

  const createItem = async () => {
    . . .
```

```

    };
    return (
      . . .
    );
  }
}

```

Here, we can see that we initially define our state, which is just a string, as all we are doing is keeping track of the name of the to-do item that we are creating. We are assuming that all to-do items being created will be pending and not complete. We then define the function that updates the state of the to-do item title every time there is a change in the input HTML element where the user is inputting the title. This means that every time the user changes the input contents for the title of the to-do item, the entire React component has access to that data, and we can alter the component however we want. We then define our API call and the `.tsx` that the component is rendering.

For the create API call, our function takes the following form:

```

// File: frontend/src/components/CreateItemForm.tsx
const createItem = async () => {
  await createToDoItemCall(title).then(response => {
    setTitle("");
    passBackResponse(response);
  });
};

```

Here, we can see that we pass the title state into the API call and then reset the state of the title. We then execute the function that was passed in via props so the main App component can access the response.

For our render statement, we have the following code:

```

// File: frontend/src/components/CreateItemForm.tsx
return (
  <div className="inputContainer">
    <input type="text" id="name"
      placeholder="create to do item"
      value={title}
      onChange={handleTitleChange}/>
    <button className="actionButton"
      id="create-button"
      onClick={createItem}>Create</button>
  </div>
);

```

Here, we render the title state in the input value so the user can see the state of the title. We then bind our listener to that input and bind the API call function to our button.

We have referenced some CSS class names in the `.tsx`. Do not worry, even though we have not created them, the frontend will not crash if we do not have the CSS classes. We will define our CSS classes in the next section of this chapter. However, it is easier to reference the CSS classes now, when we are building the React components, to save us from bloating the chapter by going back and referencing the CSS classes in the components later. Now that our create item form is done, we should inject our create item form into our React application.

Injecting our form component into our app

At this stage, it is a good idea to test if our component works. We can test our component by importing the component into the index of the frontend with the following code:

```
// File: frontend/src/index.tsx
import { CreateToDoItem } from './components/CreateItemForm';
```

Inside the App component, a function that re-renders items to the browser is defined with the following code:

```
// File: frontend/src/index.tsx
function reRenderItems(response: any) {
  if (response.error) {
    alert(JSON.stringify(response));
    return;
  }
  else if (response.data) {
    setData(response.data);
    setError(null);
  }
  else {
    setError("Unknown error");
  }
}
```

Calling the `setError` function forces the App component to re-render with the updated data from the response. However, if there is an error, we get an alert instead.

We then insert the `CreateItemForm` component at the bottom of our return statement, giving our return statement the following outline:

```
// File: frontend/src/index.tsx
return (
  <div>
    {error ? (
      <div style={{ color: 'red' }}>Error: {error}</div>
    ) : data ? (
      <div>Data loaded: {JSON.stringify(data)}</div>
    )}
  </div>
)
```

```

    ) : (
      <div>Loading...</div>
    )}
    <CreateToDoItem passBackResponse={reRenderItems} />
  </div>
);

```

Our create item is now ready to test.

Testing our create item form

Now we can create items. If we run the server, we get the form as seen in *Figure 7.6*:

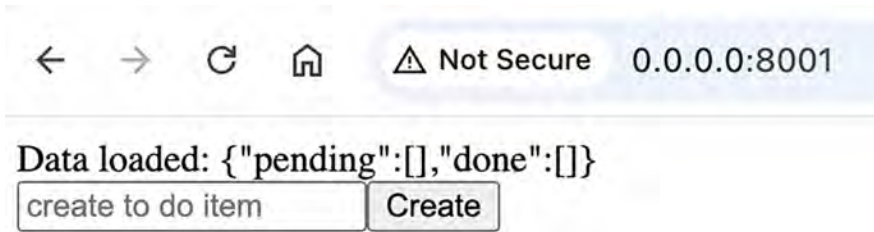


Figure 7.6 – A create form for the to-do application

However, if we use the form, we will get the alert error as seen in *Figure 7.7*:



Figure 7.7 – Alert with API error

In the error response, we can see that we get the response that we expect with the created item, but the response returns a 200, which is an OK response as opposed to a 201, which is a CREATED response. This is a minor error that can be fixed with the following code:

```

// File: nanoservices/to_do/networking/actix_server/
//       src/basic_actions/create.rs
Ok(HttpResponse::Created().json(get_all_core().await?))

```

However, because we coded some alerts based on unexpected API responses with a helpful error message and the data in the response, we have managed to diagnose the bug quickly and correct it. You may not want to give your users all the data, but logging these comprehensive responses somewhere when an anomaly is detected will save you time when trying to recreate the bug. The testing chapters will also cover how to detect these errors before deploying the application.

After correcting the response code to 201, you can run your server again, and the create form will work, resulting in the items being re-rendered. Now that our create form is working, we can move on to rendering multiple to-do items.

Rendering multiple React components

Our form is now ready to import and use, but before we add this form to our main application component, we should define our to-do item component in our `ToDoItem.tsx` file. We will need the following imports and interface:

```
// File: frontend/src/components/ToDoItem.tsx
import React, {useEffect, useState} from 'react';
import {updateToDoItemCall} from "../api/update";
import { deleteToDoItemCall } from "../api/delete";
import {TaskStatus} from "../interfaces/toDoItems";

interface ToDoItemProps {
  title: string;
  status: string;
  id: number;
  passBackResponse: (response: any) => void;
}
```

Here, we can see that we are passing in the title, status, and ID of the to-do item. This makes sense as we want to render the item, and handle operations on the to-do item in the backend via making API calls in the to-do item component.

With this interface in mind, we now define our to-do item component in our `ToDoItem.tsx` file as follows:

```
// File: frontend/src/components/ToDoItem.tsx
export const ToDoItem: React.FC<ToDoItemProps> = (
  { title, status, id, passBackResponse }) => {
  const [itemTitle, setTitle] = useState<string>(title);
  const [button, setButton] = useState<string>('');

  useEffect(() => {
    const processStatus = (status: string): string => {
      return status === "PENDING" ? "edit" : "delete";
    };
  });
}
```

```

        setButton(processStatus(status));
    }, [status]);

    const sendRequest = async (): void => {
        . . .
    };
    return (
        . . .
    );
}

```

Here, we can see that we are using the `useEffect` to define what is going to be rendered in the button. If our status is pending, we can edit the to-do item to be complete, so the button has *complete* written on it. If the status is not pending, we then assume that the to-do item is complete, and the only thing that we want to do now is delete the to-do item, so the button renders “delete.”

Note that there is a `[status]` array in the `useEffect` function. This array is a dependencies array. This means that the `useEffect` function relies on the `status` before executing, which makes sense as we reference the `status` in the `useEffect` function.

Now that our state is defined, we can move on to the API call for editing the item. Here, we must check the type of button and make either the create or delete API depending on the button type. At this stage, you could try to build this API call yourself. There is some conditional logic that you must consider, and if you get this right, then you are truly comfortable making API calls to our backend. Remember to look at the to-do item component for a starting point.

If you attempted to build the API call yourself, hopefully, you have a structure like the following one:

```

// File: frontend/src/components/ToDoItem.tsx
const sendRequest = async () => {
    if (button === "edit") {
        . . .
    } else {
        . . .
    }
};

```

If our button is an edit button as opposed to a delete function, we have this API call:

```

// File: frontend/src/components/ToDoItem.tsx
await updateToDoItemCall(
    itemTitle,
    TaskStatus.DONE
).then(
    response => {

```

```
        passBackResponse(response);
    }
}
```

And then our delete button takes the following form:

```
// File: frontend/src/components/ToDoItem.tsx
await deleteToDoItemCall(itemTitle).then(
  response => {
    passBackResponse(response.data);
  }
)
```

And now our to-do item component will be able to make API calls to the backend by itself when the item edit or delete button is pressed!

Finally, we render the to-do item with the following return statement:

```
// File: frontend/src/components/ToDoItem.tsx
return (
  <div className="itemContainer" id={id}>
    <p>{itemTitle}</p>
    <button className="actionButton"
      onClick={sendRequest}>
      {button}
    </button>
  </div>
);
```

Here, we are merely rendering the button type and binding our API call to the button on click.

And there we have it, the components are finished, and all we need to do is place them in our application. In our index file for our application, we will now have the following imports:

```
// File: frontend/src/index.tsx
import React, { useState } from 'react';
import ReactDOM from "react-dom/client";
import getAll from './api/get';
import { ToDoItems } from './interfaces/toDoItems';
import { ToDoItem } from './components/ToDoItem';
import { CreateToDoItem } from './components/CreateItemForm';
```

Now that we have our new components, we must handle them in our App component. We do this with the following code:

```
// File: frontend/src/index.tsx
const App = () => {
```

```

    . . .
    if (error) {
      return <div style={{ color: 'red' }}>Error: {error}</div>
    }
    else if (!data) {
      return <div>Loading...</div>
    }
    else {
      return (
        . . .
      );
    }
  };
};

```

We have now defined a `reRenderItems` function. This function just takes in the to-do items data, and updates the state, forcing a re-render of the application. The `reRenderItems` function is the function that we are going to pass to our create items form and to-do item components, so when these components perform an API backend call that is successful, they will pass the returned state of the to-do items back to our `reRenderItems` function.

For the return statement of our `App` component, we loop through the to-do items and render them, placing the create form at the bottom with the following code:

```

// File: frontend/src/index.tsx
<div className="App">
  <div className="mainContainer">
    <div className="header">
      <p>complete tasks: {data.done.length}</p>
      <p>pending tasks: {data.pending.length}</p>
    </div>
    <h1>Pending Items</h1>
    <div>
      {data.pending.map((item, index) => (
        <><ToDoItem key={item.title + item.status}
          title={item.title}
          status={item.status}
          id={item.id}
          passBackResponse={reRenderItems}/>
        </>
      ))}
    </div>
    <h1>Done Items</h1>
  </div>

```



```

    {data.done.map((item, index) => (
      <><ToDoItem key={item.title + item.status}
        title={item.title}
        status={item.status}
        id={item.id}
        passBackResponse={reRenderItems}/>
      </>
    ))}
  </div>
  <CreateToDoItem passBackResponse={reRenderItems} />
</div>
</div>

```

And now we have everything we need to get the frontend application working with the backend. If we run our ingress server now, we should have something like *Figure 7.8*.

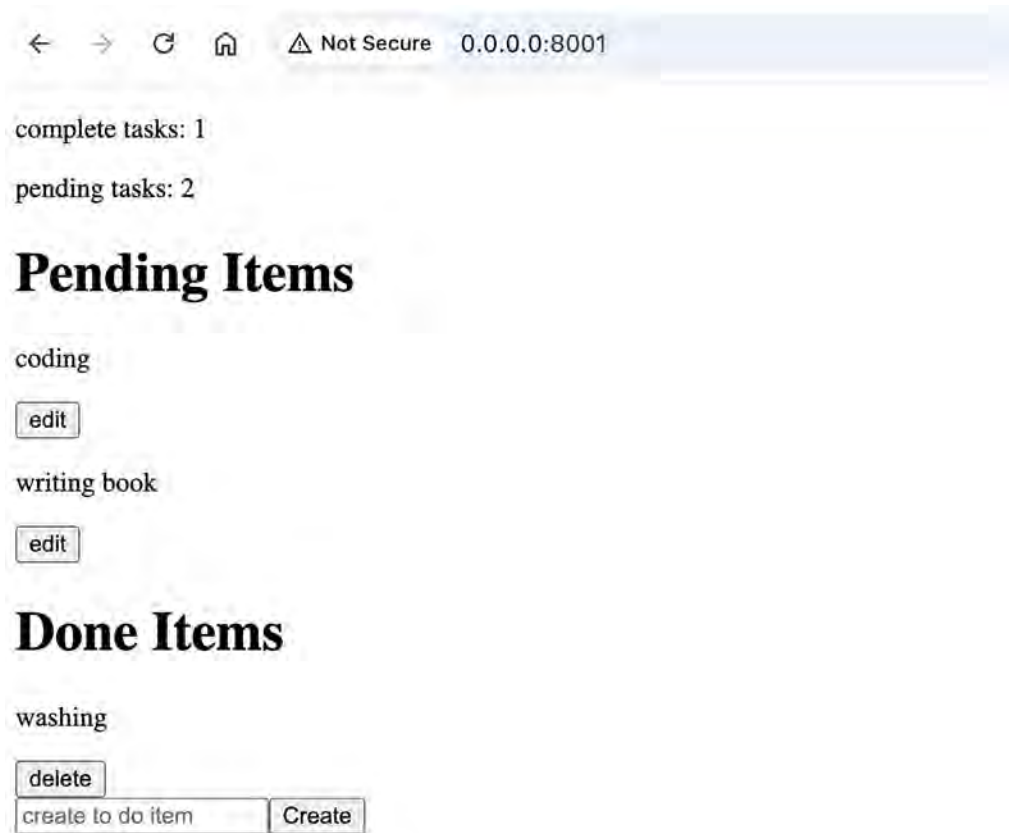


Figure 7.8 – Our app functionality working

You will be able to input a title of a new to-do item in the input box, and all buttons will work. However, as we can see in *Figure 7.8*, there is no styling. We will now move on to inserting styles with CSS.

Inserting styles with CSS

To style the frontend of our application, we must carry out the following steps:

1. Serving our CSS
2. Defining page borders with CSS for different screen sizes
3. Styling React components with CSS

Serving our CSS

We are going to serve our CSS by importing our CSS next to our `index.tsx` file and declaring it in our HTML file so that the CSS is requested by the frontend and served. We also need to bundle our CSS.

First, we import the CSS in our `index.tsx` file with the following code:

```
// File: frontend/src/index.tsx
import './App.css';
```

This ensures that we will also create a `bundle.css` file in the public directory when creating a frontend build.

Now, inside our HTML file in the public directory, we have the following:

```
<!-- File: frontend/public/index.html -->
<!DOCTYPE html>
<html>
<head>
  <title>My React App</title>
</head>
<body>
  <div id="root"></div>
  <link rel="stylesheet" href="./bundle.css">
  <script type="module" src="./bundle.js"></script>
</body>
</html>
```

For our ingress HTML file, we now have the following contents:

```
<!-- File: ingress/index.html -->
<!DOCTYPE html>
<html>
<head>
  <title>My To Do App</title>
</head>
```

```
<body>
<div id="root"></div>
<link rel="stylesheet" href="./frontend/bundle.css">
<script type="module"
  src="./frontend/public/bundle.js"></script>
</body>
</html>
```

Now that everything is in order, we can focus on our CSS. We can do a basic configuration of the body, which is the App class, with the following code:

```
/* File: frontend/src/App.css */
.App {
  background-color: #92a8d1;
  font-family: Arial, Helvetica, sans-serif;
  height: 100vh;
}
```

The background-color line refers to a type of color. This reference might not seem like it makes sense just looking at it, but there are color pickers online where you can see and pick a color, and the reference code is supplied. Some code editors support this functionality, but for quick reference, simply google `HTML color picker` and you will be spoilt for choice at the number of free online interactive tools that are available.

With the preceding configuration, the background for the entire page will be of the code `#92a8d1`, which is a navy-blue color. If we had just defined the background-color, most of the page would have a white background; the navy-blue background would only be present where there is content. We get the entire background to be navy blue by setting the height to `100vh`. `vh` is relative to 1% of the height of the viewport. With this, we can deduce that `100vh` means the styling we defined in the body occupies 100% of the viewport.

We then define the font for all text, unless overwritten, to `Arial, Helvetica, sans-serif`. We can see that we have defined multiple fonts in the `font-family`. This does not mean that all of them are implemented or that there are different fonts for different levels of headers or HTML tags. Instead, this is a fallback mechanism. First, the browser will try and render `Arial`; if it is not supported by the browser, it will then try and render `Helvetica`, and if that fails too, it will try and render `sans-serif`.

We have now defined the general style for our body, but what about different screen sizes? For instance, if we were going to access our application on our phone, it should have different dimensions. We can handle these different screen sizes by defining borders with CSS.

Defining page borders with CSS for different screen sizes

We can see the differences in screen margins with the following figure:

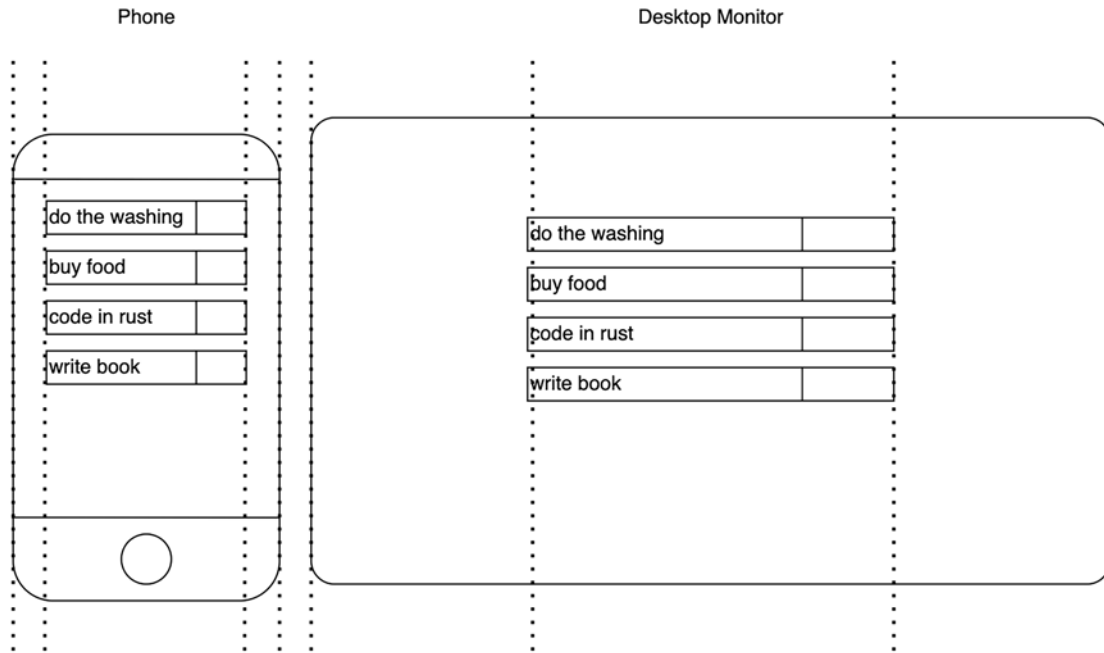


Figure 7.9 – Difference in margins between a phone and a desktop monitor

We can see in *Figure 7.9* that the ratio of the margin to the space that is filled up by the to-do items list changes. With a phone, there is not much screen space, so most of the screen needs to be taken up by the to-do item; otherwise, we would not be able to read it. However, if we are using a widescreen desktop monitor, we no longer need most of the screen for the to-do items. In fact, if the ratio were the same, the to-do items would be so stretched in the *x*-axis that it would be hard to read and, frankly, would not look good. This is where media queries come in. We can have different style conditions based on attributes like the width and height of the window.

We will start with the phone specification. So, if the width of the screen is up to 500 pixels, in our CSS file, we will define the following CSS configuration for our body:

```
/* File: frontend/src/App.css */
@media(max-width: 500px) {
  .App {
    padding: 1px;
    display: grid;
    grid-template-columns: 1fr;
  }
}
```

Here, we can see that the padding around the edge of the page and each element is just one pixel. We also have a grid display. This is where we can define columns and rows. However, we do not use it to its full extent. We just have one column. This means that our to-do items will take up most of the screen, like in the phone depiction of *Figure 7.8*. Even though we are not using a grid in this context, I have kept it in so you can see the relationship this has to the other configurations for larger screens.

If our screen gets a little bigger, we then split our page into three different vertical columns; however, the ratio of the width of the middle column to that of the columns on either side would be 5:1. This is because our screen still is not very big, and we want our items to still take up most of the screen. We can adjust for this by adding another media query with different parameters:

```
/* File: frontend/src/App.css */
@media(min-width: 501px) and (max-width: 550px) {
  .App {
    padding: 1px;
    display: grid;
    grid-template-columns: 1fr 5fr 1fr;
  }
  .mainContainer {
    grid-column-start: 2;
  }
}
```

We can also see that for our mainContainer CSS class, where we house our to-do items, we will overwrite the attribute `grid-column-start`. If we did not, then the mainContainer would be squeezed in the left margin at 1fr width. Instead, we are starting and finishing in the middle at 5fr. We can make our mainContainer span across multiple columns with a `grid-column-finish` attribute.

If our screen gets larger, we then want to adjust the ratios even more, as we do not want our items' width to get out of control. To achieve this, we can then define a 3 to 1 ratio for the middle column versus the two side columns, and then a 1 to 1 ratio when the screen width gets higher than 1001px:

```
/* File: frontend/src/App.css */
@media(min-width: 551px) and (max-width: 1000px) {
  .App {
    padding: 1px;
    display: grid;
    grid-template-columns: 1fr 3fr 1fr;
  }
  .mainContainer {
    grid-column-start: 2;
  }
}
@media(min-width: 1001px) {
```

```

.App {
  padding: 1px;
  display: grid;
  grid-template-columns: 1fr 1fr 1fr;
}
.mainContainer {
  grid-column-start: 2;
}
}

```

Now that we have defined our general CSS for our entire application, we can move on to our item container. If we run our server again, we will see that our items are now centered, as seen in *Figure 7.10*.

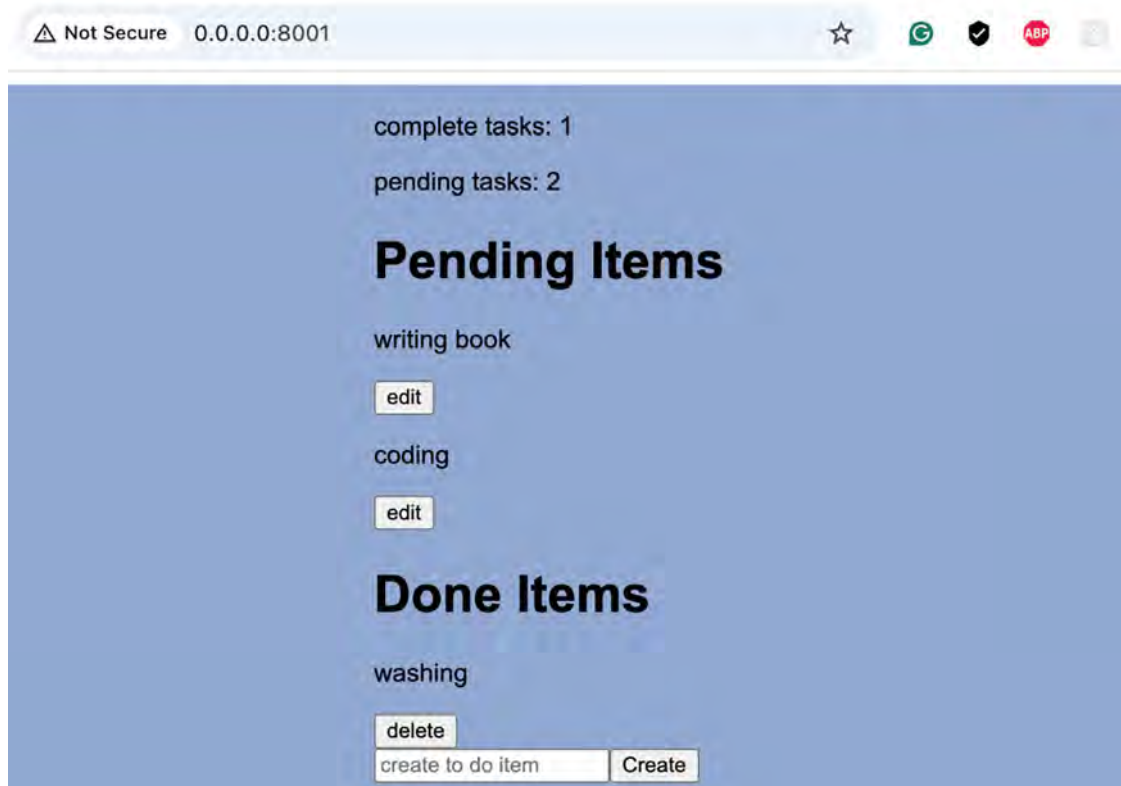


Figure 7.10 – Our application with margins

Now that our list of items is centered, we can move on to styling the items in our list.

Styling React components with CSS

We would like our item to have a different background color, giving us the following definition:

```
/* File: frontend/src/App.css */
```

```
.itemContainer {  
  background: #034f84;  
  display: flex;  
  align-items: stretch;  
  justify-content: space-between;  
  margin: 0.3rem;  
}
```

We can see that this class has a margin of 0.3. We are using the `rem` because we want the margin to scale relatively to the font size of the root element.



`rem` stands for “root em” and is a relative unit of measurement in CSS. It is based on the font size of the root element (`<html>`). For example, if the root font size is set to 16px, then 1rem equals 16px, 2rem equals 32px, and so on.

The `align-items` ensures that all the children in the container, including the buttons, stretch to fill the container height. The `flex` ensures that all items grow and shrink relative to each other and can be displayed next to each other. This will come in handy as we want our action button for the to-do item to sit next to the title. We also want our item to slightly change color if our cursor hovers over it:

```
/* File: frontend/src/App.css */  
.itemContainer:hover {  
  background: #034f99;  
}
```

Inside an item container, the title of our item is denoted with a paragraph tag. We want to define the style of all the paragraphs in the item containers, but not elsewhere. We define the style of the paragraphs in the container with the following code:

```
/* File: frontend/src/App.css */  
.itemContainer p {  
  color: white;  
  display: inline-block;  
  margin: 0.5rem;  
  margin-right: 0.4rem;  
  margin-left: 0.4rem;  
}
```

The `inline-block` allows the title to be displayed alongside the `div`, which will be acting as the button for the item. The margin definitions merely stop the title from being right up against the edge of the item container. We also ensure that the paragraph color is white, to stand out against the dark background.

With our item titles styled, the only item styling left is the action button, which is either edit or delete. This action button is going to float to the right, with a different background color, so we know where to click. To do this, we define our button style with a class, as outlined in the following code:

```
/* File: frontend/src/App.css */
.actionButton {
  display: inline-block;
  float: right;
  background: #f7786b;
  border: none;
  padding: 0.5rem;
  padding-left: 2rem;
  padding-right: 2rem;
  color: white;
  align-self: stretch;
}
```

Here, we've defined the display, we made it float to the right, and we've defined the background color and padding. With this, we ensure the color changes on the hovering of a cursor over the button by running the following code:

```
/* File: frontend/src/App.css */
.actionButton:hover {
  background: #f7686b;
  color: black;
}
```

Now that we have covered all the styling concepts, we only need to define the styles for the input container. This can be done by running the following code:

```
/* File: frontend/src/App.css */
.inputContainer {
  background: #034f84;
  display: flex;
  align-items: stretch;
  justify-content: space-between;
  margin: 0.3rem;
  margin-top: 2rem;
}
```



```
.inputContainer input {  
  display: inline-block;  
  margin: 0.4rem;  
  border: 2px solid transparent;  
  background: #ffffff;  
  color: #034f84;  
}
```

While this defines the styling of our input, we want the user to know that they have selected the input so that they can type. We can change the CSS of the input when the input is clicked using focus:

```
/* File: frontend/src/App.css */  
.inputContainer input:focus {  
  outline: none;  
  border-color: #f7786b;  
  box-shadow: 0 0 5px #f7786b;  
}
```

And finally, let's define the CSS for the header with the following code:

```
/* File: frontend/src/App.css */  
.header {  
  background: #034f84;  
  margin-bottom: 0.3rem;  
}  
.header p {  
  color: white;  
  display: inline-block;  
  margin: 0.5rem;  
  margin-right: 0.4rem;  
  margin-left: 0.4rem;  
}
```

And that is it, our system is now ready to run. If we run our server, we should get the view seen in *Figure 7.11*.

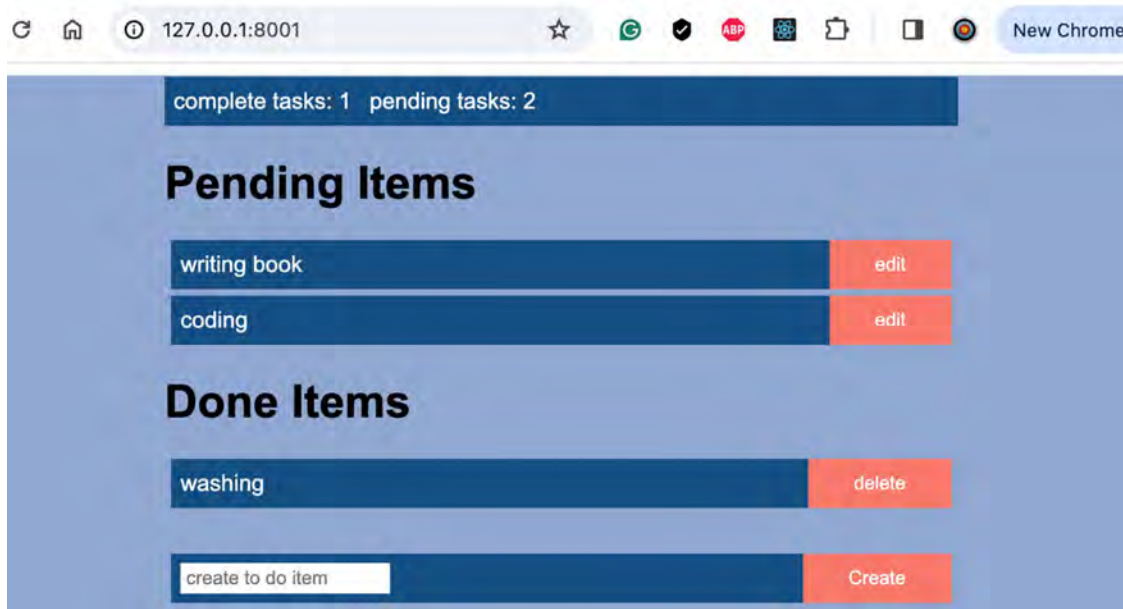


Figure 7.11 – Our finished application

And there we have it, our application is working. Our application can be served by our Rust binary.

Summary

In this chapter, we have finally enabled our application to be used by a casual user, as opposed to having to rely on making an API. We built a React application and embedded the React application into the Rust server binary to be served by the Rust server. We built the React application using minimalist tooling, so we understand how the React application is built, packaged, and served. We also covered HTML and CSS concepts to make our frontend application look presentable.

This gave us a dynamic view that automatically updated when we edited, deleted, or created a to-do item. We also explored some basics around CSS and JavaScript to make API calls from the frontend to the backend to get the data needed to render the state of to-do items.

We also explored how frontend assets are served, which enabled us to embed our frontend into the Rust binary to be served by our Rust server. However, we are not going to stop there; in the next chapter, we are going to serve WASM in the frontend.

Questions

1. How is the React application and its CSS served to the frontend?
2. How can we handle data produced in a child component?

3. How do you ensure that the background color and style standard of certain elements is consistent across all views of the app?
4. What do we use `useEffect` for in a React component?
5. When will a React component re-render?

Answers

1. The React app is bundled into two files, `bundle.js` and `bundle.css`. We serve an HTML file that references the CSS and JS file. These files are then requested from the server and loaded into the browser. The JS file is the React application that gets an HTML tag in the HTML file via ID and renders the outcome. This outcome is inserted into the HTML tag in the served HTML file.
2. The parent component passes a function to the child component. The child component can then pass data into that function and execute the function. Depending on what the function does, this means that the parent component can be altered.
3. We can create a CSS class and define the background color and other styling types we want to be universal, such as font type and color. In our App component, we return a `div` with that CSS class. Inside that `div`, we put all our components for that application, and these components will inherit the CSS class for the initial `div`.
4. The `useEffect` executes when the React component has been loaded. We can also have dependencies, such as data.
5. A React component re-renders when the data in `useState` changes.

Get this book's PDF copy, code bundle, and more

Scan the QR code (or go to packtpub.com/unlock). Search for this book by name, confirm the edition, and then follow the steps on the page.



UNLOCK NOW

Note: Have your invoice handy. Purchases made directly from the Packt website don't require an invoice.

8

Injecting Rust in the Frontend with WebAssembly

You have probably heard about **WebAssembly (WASM)**. At the time of writing this book, the WASM ecosystem is still in the early rapid development phases, where APIs get outdated quickly, and groups trying out a new approach cease to exist but have remnants of their approach throughout the internet. This can lead to frustration as you burn hours trying to figure out what API does what. However, the promise of compile once and run anywhere, including the browser, is a clear advantage, so it makes sense to understand and get comfortable with WASM.

In this chapter, we are going to keep the interactions with APIs to a minimum and focus on concepts around serving WASM modules to the frontend to be loaded and used in the browser. We are also going to explore directly interacting with the raw memory addresses in WASM so we can transfer complex data types across the WASM boundary, and we are going to load WASM binaries using Rust to also interact with WASM binaries in the backend.

We cover the following topics in this chapter:

- What is WASM?
- Setting up our WASM build
- Loading WASM in the frontend
- Loading WASM on the local machine

By the end of this chapter, you will be comfortable enough with WASM to navigate the rapidly changing WASM landscape. However, if WASM is not for you, skipping this chapter will not affect the overall functionality of the app, as we will just be replacing the item status functionality with a WASM binary.

Technical requirements

You can download the example project and code for this book by following the instructions in the **Download the example code files** section in the *Preface* of this book.

This chapter's code files are included in the downloadable code bundle.

You will also need `wasm-pack`, which can be installed via the following link: <https://drager.github.io/wasm-pack/installer/>

You will also need the WebAssembly binary toolkit, which can be found in the link: <https://github.com/WebAssembly/wabt>.

This toolkit can also be installed with operating system package managers such as `brew` and `apt-get`.

What is WASM?

Generally, with compiled languages, we must compile the code for a specific chip architecture and operating system. This means that if you compile your Rust program on a Mac, that compiled binary will not run on a Linux or Windows machine, and vice versa. WASM, on the other hand, is compiled to bytecode. The standards for this bytecode are defined by the Bytecode Alliance. This is a binary format that is designed to be executed by a virtual machine or interpreter. So, when we compile our WASM, instead of compiling the code to a specific architecture or operating system, the code is compiled to an agreed-upon standard that all virtual machines support. This means that we can compile our WASM binary once and run this single WASM binary on any device that has a virtual machine. Java is an older language that has a similar approach. WASM is so flexible that we can even run WASM in the browser. This means that we can compile our Rust code into WASM and send that WASM over a network to a browser to be run. WASM is not just supported by Rust. Languages such as C, C++, Python, Go, and Zig can also compile to WASM. However, it must be noted that there is a downside to WASM, which is binary size.

The frontend ecosystem has worked hard to reduce the size of code being sent to the browser. This is done by having JavaScript functionality in the browser, so not everything has to be sent over a network. WASM, on the other hand, must compile all instructions into the binary. This means some implementation of garbage collection if we are compiling Python into WASM. When we are compiling Rust, error handling and types must be included in the WASM binary. This results in much bigger payloads being sent over the network. Because of the larger payloads, combined with the rich frontend ecosystem of JavaScript, it is unlikely that Rust WASM is going to take over JavaScript for the frontend. Existing libraries, mature frameworks, and the talent pool have resulted in me building backend servers in Rust, but still reaching for the JavaScript ecosystem for the frontend. However, WASM is still a useful tool for solving problems. For instance, a part of our backend code could be sent over the network to run on the client's computer through the browser. This reduces our workload without us having to rewrite the code in JavaScript. Another use case is WASM workers in the backend. WASM runs in a sandbox, meaning that it does not have any permissions to access files or network ports without the runner explicitly defining the permissions. Sandboxing enables us to allow users to write code in a range of languages, compile the code to WASM, and run that WASM on our servers. In the final chapter of this book, we build a queuing system that takes WASM tasks off a Redis queue and processes them. These WASM tasks also keep a state. This enables other developers in the team to write programs in other languages that can be processed on our Rust servers.

To get to grips with the basics of WASM, we will start by setting up our WASM build for serving Rust WASM to the frontend.

Setting up our WASM build

We must set up a Rust workspace in our frontend that can house the Rust code that we want to compile to a WASM target to be served. To set up our WASM build environment, we will carry out the following tasks:

- Define our WASM module
- Configure our NPM build WASM command
- Inspect our WASM binary

We will start by defining our WASM module.

Defining our WASM module

In our frontend directory, we can create a new Rust workspace with the following command:

```
cargo new rust-interface --lib
```

Now that we have a workspace, we must configure our [lib] of this workspace with the following code:

```
# frontend/rust-interface/Cargo.toml
. . .
[lib]
crate-type = ["cdylib"]

[dependencies]
wasm-bindgen = "0.2.99"
```

Here, we can see that we are relying on the `wasm-bindgen` crate to make the WASM-to-JavaScript bindings and simplify our Rust functions that act as entry points to the WASM program. In the last section of this chapter, we will build our own code to enable us to pass complex data types over the WASM boundary. This is going to involve directly accessing raw, unsafe data pointers. Using unsafe pointers when coding in Rust might seem wrong. However, there is no other choice. If we are getting data by interacting directly with another program and language, there is no way the Rust compiler can check lifetimes to ensure that the memory allocation is safe. However, for our frontend example, the `wasm-bindgen` crate is going to do all the heavy lifting when it comes to passing over even simple data structures, such as strings.

We also see that the crate type of our Rust workspace is a `cdylib`. This stands for dynamic library. A dynamic library is linked to the main program when a program is compiling. The compiled program then loads the dynamic library at runtime as opposed to the library being compiled into the main program binary. There are pros and cons to this. For instance, you can reuse dynamic libraries in multiple programs, and these dynamic libraries are easier to update as they are not part of the build.

Not being part of the build also results in reducing the compile times, as we do not need to recompile the dynamic library when we are compiling the main code. Dynamic libraries will also reduce your main program binary size. However, this does not come for free as the compiler cannot optimize the code in the dynamic library using the code of your main program. It also has additional overhead as the program needs to load the dynamic library, and deployment can be trickier due to more moving parts.



For many of your web development projects, you will not need to build dynamic libraries, as it is likely not worth the extra headache of linker errors and handling multiple moving parts. However, if you end up working on a massive code base, then dynamic libraries become essential, especially if there are interfaces from other languages, such as WASM. For instance, at the time of writing this book, I am currently doing research in the King's College London Bioengineering department in surgical robotics. We need to interface with GPUs. A lot of hardware support is mainly written in C and C++ dynamic libraries. The ecosystem for Rust is maturing, but sometimes there is no getting around interacting with C and C++ libraries.

We need to compile our Rust binary in a WASM format and move it into our public directory. This will enable us to serve our WASM binary when needed. We can do this via the terminal, but there are a couple of commands pointing to specific directories. This is easy to get wrong, and we also do not want to bother other developers with learning how to build and package WASM, so we will just put our commands under the WASM key under the scripts section with the following code:

```
// FILE: frontend/package.json
. . .
"wasm": "cd rust-interface && sudo wasm-pack build --target web
        && cp pkg/rust_interface_bg.wasm
        ../public/rust_interface_bg.wasm",
```

Here, we can see that we carry out the following steps:

1. Change directory to the rust-interface directory.
2. Use wasm-pack to build the Rust program to target the web.
3. Copy the WASM binary from the pkg to the public directory.

We will also want to trace our WASM binary to make sure that we are exporting the functions that we expect. We can do this with the `wasm2wat` command from the `wasm` toolkit we installed in the *Technical requirements* section of this chapter, with the following scripts command:

```
// FILE: frontend/package.json
"wasm-trace": "cd public &&
               wasm2wat rust_interface_bg.wasm
               > rust_interface_bg.wat"
```

What happens here is that the `wasm2wat` command converts the WASM binary into WAT format, which is a human-readable text format.



You can code in WAT and compile to WASM, giving you the freedom to do whatever you want. However, you will be directly coding to assembly, and it will take you a lot of time to code even the most basic Rust programs.

Before we run any commands, we must write some Rust code that can be compiled into WASM. For our example, we will be creating a basic function that takes in a string that is the status of the task and returns the text that should be displayed in the button for that task. This can be done with the following code:

```
// FILE: frontend/rust-interface/src/lib.rs
use wasm_bindgen::prelude::*;

#[wasm_bindgen]
pub fn rust_generate_button_text(status: String) -> String {
    match status.to_uppercase().as_str() {
        "PENDING" => "edit".to_string(),
        "DONE" => "delete".to_string(),
        _ => "an error has occurred".to_string(),
    }
}
```

While the code is straightforward enough, we can see that the function is annotated with the `#[wasm_bindgen]` macro. The `#[wasm_bindgen]` macro does the following:

- **JavaScript integration:** The macro exposes the annotated Rust function so it can be called from JavaScript. This is essential for creating WASM modules that can interact with web-based applications.
- **Type conversion:** It handles the automatic conversion of types between Rust and JavaScript. In our function, it manages the conversion of the Rust String type to a JavaScript string and vice versa. Due to the differences in data types between languages, we cannot always guarantee that data type conversions are possible. However, if you stick to the basics of integers, floats, booleans, and strings, with basic collections, there should be no problems.
- **Memory management:** The macro manages memory for the types passed between Rust and JavaScript, ensuring that memory is allocated and freed appropriately to prevent leaks.
- **Function wrapping:** It wraps the Rust function in such a way that it can be imported into JavaScript as a regular function. This allows the function to be invoked from JavaScript just like any other JavaScript function.

We can now run our build with the following commands in the frontend directory:

```
npm run wasm
npm run wasm-trace
```

These commands might take a while to execute, depending on the computer. After running these commands, the WASM and WAT files should be in the public directory. We cannot really inspect the WASM binary because it's a binary, but if we look at the bottom of the `rust_interface_bg.wat` file, we should see the following exports:

```
// File: frontend/public/rust_interface_bg.wat
(export "memory" (memory 0))
(export "update_status" (func 9))
(export "__wbindgen_add_to_stack_pointer" (func 35))
(export "__wbindgen_malloc" (func 19))
(export "__wbindgen_realloc" (func 23))
(export "__wbindgen_free" (func 27))
```

Before we explain what the WAT code means, we need to understand that the WASM memory model is linear. This means that unlike **Random-Access Memory (RAM)**, all the memory in the WASM program is contiguous, meaning that the memory is essentially one big array of memory. This makes WASM very fast compared to random access. However, it also makes WASM more vulnerable to attacks if there is no memory safety. For instance, if a hacker can access memory outside of the assigned buffer (the result of a buffer overflow), then the hacker has access to the entire memory stack of the program and can do what they want. Luckily, with Rust, we have memory safety. This is why Rust is at the bleeding edge of WASM. Rust has memory safety, but it doesn't have a garbage collector, which simplifies the running and compiling of the WASM binary.



I always say to people, if they want to lead the charge of WASM, they need to learn Rust.

With the WASM memory model in mind, we can now look at `(memory 0)`. This refers to the first memory instance defined in the module and is allocated at index 0 of the memory array. This memory instance is being exported with the function name "memory," making it accessible for use outside the module. `(func 9)`, `(func 35)`, `(func 19)`, `(func 23)`, and `(func 27)` denote that they are functions that are allocated at certain indexes of the memory.

Now we can cover the function exports:

- `(export "memory" (memory 0))`: This exports the first memory instance defined in the WASM module. It allows external environments to access and manipulate the memory used by the WASM instance.
- `(export "update_status" (func 9))`: This is a **function export**. It exports a function labeled `update_status`. This is the function that we defined in our code, so it's good to know that the code we wrote is available in the WASM binary as `update_status`.

- (export "__wbindgen_add_to_stack_pointer" (func 35)): This is a **stack pointer adjustment**. It exports a function that adjusts the stack pointer. This is useful for managing the allocation of stack space, particularly during complex operations or recursive calls within the WebAssembly code.
- (export "__wbindgen_malloc" (func 19)): This is **memory allocation**. It exports a function for allocating memory. This function is analogous to malloc in C/C++, providing a way to dynamically allocate a specified amount of memory. Later in the chapter, we will write our own malloc function.
- (export "__wbindgen_realloc" (func 23)): This is **memory reallocation**. It exports a function that adjusts the size of a previously allocated memory block. This is like realloc in C/C++, which is used to either expand or contract a memory allocation while attempting to preserve the contents.
- (export "__wbindgen_free" (func 27)): This is **memory deallocation**. It exports a function for freeing allocated memory. This function is like free in C/C++ and is used to release previously allocated memory back to the system, preventing memory leaks. Memory leaks are where the memory allocated to variables that are no longer used is not cleared up. Over time, the memory being used by the program continues to grow until the computer runs out of memory.

Our WASM program is now completed, and we are confident that we can interact with it like we intended. We can now move on to loading the WASM binary in our JavaScript code in the frontend.

Defining WASM modules cheatsheet

WASM libraries are dynamic C libraries due to the stable application binary interface, so there is a binary interface for the WASM runtime to interact with.

The `wasm-bingen` crate makes the creation of these binary interfaces easier, so you do not have to build the raw C interfaces yourself. This also means that the memory management for the C interfaces is also handled.

`wasm-bingen` also creates memory allocation functions that can be accessed as public functions by the code loading the WASM module.

We can use the `wasm2wat` command to convert the compiled WASM binary to WASM text format, so we can personally inspect what public functions are available in the WASM binary. This can help with general inspection of a WASM binary that you are interacting with, and can also help with debugging, so you can check that the functions are what you expect.



Loading WASM in the frontend

When it comes to loading WASM in our frontend application, the program must carry out the steps shown in *Figure 8.1*:

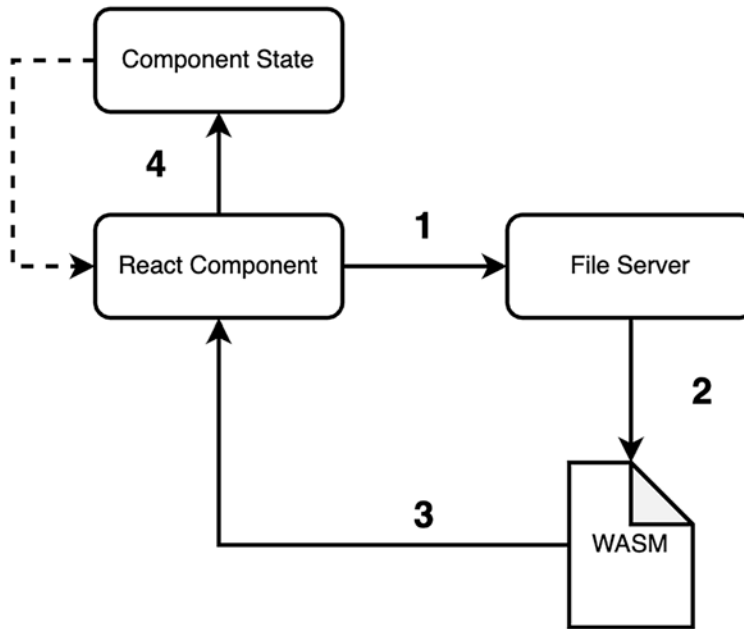


Figure 8.1 – Steps to run a WASM function in the browser

Figure 8.1 depicts these steps:

1. A React component makes a request to the file server for the WASM file.
2. The file server gets the WASM binary.
3. The file server then returns the WASM file back to the frontend for the requesting React component to load it.
4. The WASM functions are then stored in the component state to be called as and when needed.

For us to understand and enable our frontend to load the WASM binary and execute it, we are going to carry out the following steps:

1. Inspect the WASM interface.
2. Integrate WASM loading in our React component.
3. Pass WASM to subcomponents.

But before we carry out these steps, we should inspect the WASM interface, so we know what is going on.

Inspecting a WASM interface

Even though it might seem intuitive to handle the WASM loading and function in the task component, we would end up requesting a WASM file for every task in our to-do application. For our example project, this might be OK, as we will not notice any difficulties. However, as the application scales and more users use our application, we would really struggle to keep up with demand. We could reduce the number of calls by caching in the browser, and we can do this in the header of the response serving the WASM file. We should do this anyway, but to reduce the risk of excessive calls even more, we are going to load the WASM file in the main application. It must be noted that WASM-based apps will have a bigger initial load time, which will vary depending on how much we rely on WASM.

We can load our WASM in the `src/index.tsx` file with the following code:

```
// File: frontend/src/index.tsx
import init, { rust_generate_button_text } from
  './rust-interface/pkg/rust_interface.js';
```

Here, we will use the `init` function to get the WASM binary. The `init` function handles the request to the server for the WASM binary. It also handles the initialization of the WASM module, which loads the WASM module into memory and imports functions needed for the WASM library to run.

We can see that we are loading the `rust_generate_button_text` function from a JavaScript file. If we look at the JavaScript file in the `pkg` directory of the WASM build and search for the `rust_generate_button_text` function, we get the following code. For now, this is just the outline of the function, but we will break down each section shortly. It must be noted that the code generation may change as the package changes, but the fundamental concepts will be the same:

```
// File: frontend/rust-interface/pkg/rust_interface.js
let WASM_VECTOR_LEN = 0;
. . .
export function rust_generate_button_text(status) {
  let deferred2_0;
  let deferred2_1;
  try {
    . . .
  } finally {
    . . .
  }
}
```

deferred2_0 and deferred2_1 are pointers that are going to be used for deallocating memory. In the `try` section, the function initially adjusts the memory pointer to reserve space for the function's execution with the following code:

```
// File: frontend/rust-interface/pkg/rust_interface.js
try {
    const retptr = wasm.__wbindgen_add_to_stack_pointer(-16);
    . . .
}
```

It then uses the memory allocation that has been generated in the WASM program to allocate memory to the WASM library memory buffer, which is the length of the bytes of the string. This returns the pointer to where that memory allocation starts and the length of the memory buffer, with the following code:

```
// File: frontend/rust-interface/pkg/rust_interface.js

const ptr0 = passStringToWasm0(
    status,
    wasm.__wbindgen_malloc,
    wasm.__wbindgen_realloc
);
const len0 = WASM_VECTOR_LEN;
```

The interface function can then execute our WASM function, with the following code, and we get the result of our function:

```
// File: frontend/rust-interface/pkg/rust_interface.js
wasm.rust_generate_button_text(retptr, ptr0, len0);
var r0 = getInt32Memory0()[retptr / 4 + 0];
var r1 = getInt32Memory0()[retptr / 4 + 1];
deferred2_0 = r0;
deferred2_1 = r1;
return getStringFromWasm0(r0, r1);
```

If you have done some JavaScript coding before, you may wonder why the generated code is using `var` instead of `const` or `let`. The `var` is used to expand the compatibility of the code. This is because `const` and `let` were introduced later and are not available in pre-2015 hosts. In the preceding code, we can see that we pass the pointer for our string buffer in memory, and the length of the buffer. This enables our WASM function to extract the string data from the memory and use it in the way that we intended. Our WASM function then puts the return string into the memory. The pointers for the return string are then extracted, and this is used to get the string from WASM. To get the string from WASM, you would access the bytes in the memory using the pointer and the length of the buffer, and then deserialize those bytes to a string. Finally, the function deallocates the memory, as we no longer need it, using the following code:

```
// File: frontend/rust-interface/pkg/rust_interface.js
finally {
  wasm.__wbindgen_add_to_stack_pointer(16);
  wasm.__wbindgen_free(deferred2_0, deferred2_1, 1);
}
```

Now that we understand how our WASM function interfaces with the application, we know that the interaction with the raw pointers to our WASM function will be bundled into our application. With this, we only need to worry about serving the WASM file, not serving or configuring any glue code, because the glue code is in our JavaScript bundle.

Integrating WASM loading into our React component

Going back to our main app component, we now need to store the WASM function in our state and a Boolean flag on whether the WASM binary has been loaded or not. To have the whole app be aware of the WASM loading state, our App component should have the following hooks:

```
// File: frontend/src/index.tsx
const [data, setData] = useState<null>();
const [error, setError] = useState<null>();
const [wasmReady, setWasmReady] = useState<boolean>(false);
const [
  RustGenerateButtonText,
  setRustGenerateButtonText
] = useState<(input: string) => string>(null);
```

Here, we can see that we still house our to-do items in data, but we declare that the WASM is ready with `wasmReady`, and we house the Rust function that is compiled to WASM with `RustGenerateButtonText`.

We can now load our WASM binary and set the WASM ready flag by adding another `useEffect` alongside our existing one with the following code:

```
// File: frontend/src/index.tsx
React.useEffect(() => {
  init().then(() => {
    setRustGenerateButtonText(() => rust_generate_button_text);
    setWasmReady(true);
  }).catch(e => console.error(
    "Error initializing WASM: ", e
  ));
}, []);
```

The fetching of the WASM binary might take some time. We do not want to run the risk of getting the items from the server before we get the WASM function because we need the WASM function to create the text for each to-do item. Therefore, we refactor our loading of the to-do items with the following code:

```
// File: ingress/frontend/src/index.tsx
React.useEffect(() => {
  const fetchData = async () => {
    . . .
  };
  if (wasmReady) {
    fetchData();
  }
}, [wasmReady]);
```

Here, we can see that `useEffect` will only fire when `wasmReady` changes based on the dependency declared by `[wasmReady]`. We do not know whether the future and `wasmReady` will change back to false later; therefore, even if `wasmReady` changes, we check that `wasmReady` is true before making calls to get the to-do items from the server. We now need to pass our WASM function output to our to-do item subcomponents so they can render the status of the item.

Passing our WASM output to subcomponents

To pass our WASM function to our item subcomponents, we can call the WASM function for the status of each to-do item in our return statement with the following code:

```
// File: frontend/src/index.tsx
<h1>Pending Items</h1>
<div>
  {data.pending.map((item, index) => (
    <><ToDoItem key={item.title + item.status}
      title={item.title}
      status={item.status}
      id={item.id}
      buttonMessage={
        RustGenerateButtonText(item.status)
      }
      passBackResponse={reRenderItems}/>
    </>
  ))}
</div>
<h1>Done Items</h1>
<div>
  {data.done.map((item, index) => (
```



```

    <><ToDoItem key={item.title + item.status}
      title={item.title}
      status={item.status}
      id={item.id}
      buttonMessage={
        RustGenerateButtonText(item.status)
      }
      passBackResponse={reRenderItems}/>
    </>
  )}}
</div>

```

You may have noticed a change in the parameters passed into the `ToDoItem` components. This is because we have taken advantage of this WASM function refactor to simplify the `ToDoItem` component and get rid of the state of the `ToDoItem` component, which was never really needed.



In real life, as a project grows and deadlines loom, the best implementation does not always land straight away. Keeping on top of a degrading code base takes work. It's just like gardening: you need to be constantly pruning.

Big refactors can introduce bugs, delay projects, cause breaking changes elsewhere in the system, and hold up other developers as they must review these big refactors. What's more, the code is not static if other developers are also working on the same code base. If you have a big refactor, then chances are that your code changes will clash with other developers' work.

A good way to prevent these issues, but not let the code slip into disorder, degrading over time to the point of it being a problem, is the philosophy of "leaving code better off than when you found it." The refactor we've just done is a perfect example of this. We did a small simplification, and the scope of code that we changed is in the scope of where our WASM interactions are based. If you would like to, now would be a good time to try and refactor the `ToDoItem` component yourself.

If you attempted to refactor the `ToDoItem` component, hopefully, it resembles the following code:

```

// File: frontend/src/components/ToDoItem.tsx
interface ToDoItemProps {
  title: string;
  id: number;
  passBackResponse: (response: any) => void;
  buttonMessage: string;
}
export const ToDoItem: React.FC<ToDoItemProps> = (
  { title, id, passBackResponse, buttonMessage }) => {

```

```

const sendRequest = async () => {
  if (buttonMessage === "edit") {
    await updateToDoItemCall(
      title,
      TaskStatus.DONE
    ).then(
      response => {
        passBackResponse(response);
      }
    )
  } else {
    await deleteToDoItemCall(title).then(
      response => {
        passBackResponse(response);
      }
    )
  }
};
return (
  <div className="itemContainer" id={id}>
    <p>{title}</p>
    <button
      className="actionButton"
      onClick={sendRequest}>{buttonMessage}</button>
  </div>
);
}

```

Here, we can see that we got rid of the internal state of the to-do item, simplifying the component while maintaining the functionality as we now pass in the button message. We now have everything ready to integrate WASM into the frontend. If we run our server now, we should see the network calls shown in *Figure 8.2*:

Name	Status	Type	Initiator	Size	Time
localhost	200	docum...	Other	435 B	1 ms
bundle.css	200	styleh...	:8001/:8	(memory ...	0 ms
bundle.js	200	script	:8001/:9	(memory ...	0 ms
rust_interface_bg.wasm	200	wasm	rust_interfa...	(disk cac...	0 ms
all	200	xhr	utils.ts:78	282 B	1 ms

Figure 8.2 – Frontend network calls with WASM

In *Figure 8.2*, we can see that the bundles are loaded, then the WASM is loaded, then finally, the to-do items are loaded. The fact that our buttons are rendering proves that our WASM function works as we intended it to.

And there we have it, we now have WASM loading and running in the frontend of our application! We can execute Rust in the browser!

But what about Rust interfacing with WASM? Or WASM executing on the local machine? We will explore these ideas in the next section.

Loading WASM in the frontend cheatsheet

The `wasm-pack` command generates JavaScript code that interfaces with the WASM module. You do not need to touch the generated interface; just import and call it in your frontend application.



The generated interface interacts with the memory allocation functions built by `wasm-bingen` to ensure that the memory is correctly allocated and cleaned up when interacting with a WASM module.

We use `useEffect` to load the WASM module as the frontend has to make a network call to the server to get the WASM binary and load it.

Because network calls to retrieve WASM binaries take time, we have a Boolean flag that is initially false and then switched to true when the WASM module is loaded. This gives us control to render different things until the WASM module is loaded.

Loading WASM on the local machine

In this section, we are going to build a WASM module and have our own Rust code load the WASM file and interact with it. We are also going to build a kernel to pass complex data structures over the WASM boundary, as we saw in the JavaScript memory allocation code in the previous section. To carry out this exercise, we are going to need the following file layout:

```
|— kernel
|   |— Cargo.toml
|   └─ src
|       └─ lib.rs
|— wasm-client
|   |— Cargo.toml
|   └─ src
|       └─ main.rs
└─ wasm-lib
    |— Cargo.toml
    |— scripts
    |   └─ build.sh
    └─ src
        └─ lib.rs
```

These workspaces have the following purposes:

- **kernel:** To provide structs to both `wasm-lib` and `wasm-client` workspaces so that they can both compile the same structs for the serialization and deserialization of structs and bytes
- **wasm-lib:** To build a WASM binary with some functionality with the structs passed into the WASM binary
- **wasm-client:** To load the WASM binary and interact with it, passing data to the WASM program and extracting results

Seeing as both the WASM lib and WASM client rely on the kernel, we will start with the kernel.

Building a WASM kernel

There can be some confusion around the word “kernel,” with some assuming that it relates only to an operating system. The term is broader than that, however; in the realm of domain-driven design, it refers to a bounded context, shared across multiple other bounded contexts. This is the kind of kernel we will be building here. For our kernel, we simply need to build a data struct that both bounded contexts (client and lib) need to reference.



We could just build the same structs in both our WASM client and library, and these would technically work, as we are going to serialize our structs before sending them over the WASM boundary. However, maintaining consistency when there are multiple duplicates of a struct is harder, and if both client and lib end up being compiled into the same Rust program later, the compiler will acknowledge that the structs have the same name but are defined in different places. This will result in us needing to manually convert the struct from the client to the lib struct to satisfy the compiler.

The data struct in the kernel will be the single source of truth with its own workspace. Because the kernel is in its own workspace, we can then compile it into any program that needs to communicate with another that has also compiled the kernel, as seen in *Figure 8.3*:

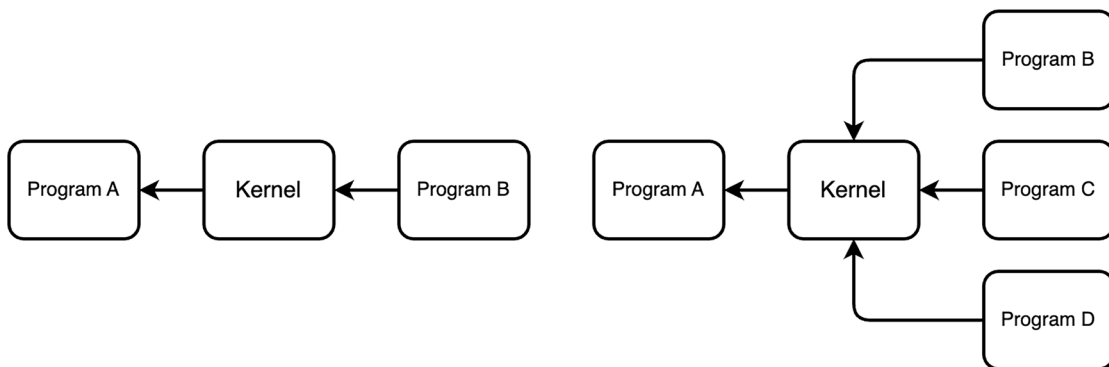


Figure 8.3 – Kernel enables easy scaling of programs communicating

For our kernel workspace, we have the following dependency:

```
// File: kernel/Cargo.toml
```

```
[dependencies]
serde = { version = "1.0.201", features = ["derive"] }
```

Now, with our dependency defined, we can build out the basic data struct with the following code:

```
// File: kernel/src/lib.rs
use serde::{Serialize, Deserialize};

#[derive(Serialize, Deserialize, Debug)]
pub struct SomeDataStruct {
    pub name: String,
    pub names: Vec<String>,
}
```

If you recall, in the *Loading WASM in the frontend* section, the string was transferred over the WASM boundary when inspecting the JavaScript code that interacted with the WASM binary. We also need a pointer that will point to the serialized struct in memory. We need this pointer in our kernel too. Our pointer takes the following form:

```
// File: kernel/src/lib.rs
#[repr(C)]
pub struct ResultPointer {
    pub ptr: i32,
    pub len: i32
}
```

`ptr` is the pointer to the memory address of the start of where the serialized `SomeDataStruct` is stored in memory. Here, we can calculate the end of the memory address using the length.

The `#[repr(c)]` macro for the `ResultPointer` struct ensures that the `ResultPointer` layout is consistent with the C representation for ABI compatibility, field order, and padding/alignment, making the memory layout predictable from a C perspective.



What's with all the C references?

At the time of writing this, Rust does not have a stable **Application Binary Interface (ABI)**. C, on the other hand, does have a stable ABI due to its simplicity and years of maturity. At this point in time, Rust is still changing and trying to improve. If Rust established a stable ABI, it would make it harder to move forward with new improvements.

Seeing as the kernels in Windows, macOS, and Linux are written in C, C data types are universally accepted. Just Google “*foreign function interface C ...*” and then whatever language you are interested in, and I’d be shocked if there isn’t an option of interacting with C data and functions in that language.

Our kernel is now ready, and we can move on to our WASM library.

Building a WASM library

To build our WASM library, we are going to receive a pointer and length to the serialized struct in memory, and then we are going to carry out the following steps:

1. Extract the bytes of the struct from memory.
2. Deserialize the struct.
3. Add a name to the struct.
4. Serialize the updated struct to bytes.
5. Put the serialized struct into memory.
6. Return the pointer to the memory address of the updated serialized structs.

While these numbered steps help distill what we are carrying out, we will not break down the implementation of our WASM library into numbered steps, as the implementation of these numbered steps all comes at the end when the entry point function is built.

Before we write any code, our WASM library will need the following dependencies:

```
// File: wasm-lib/Cargo.toml
[dependencies]
bincode = "1.3.3"
kernel = { path = "../kernel" }
serde = { version = "1.0.201", features = ["derive"] }

[lib]
crate-type = ["cdylib"]
```

We are going to be using bincode for the serialization. The crate bincode serializes Rust data types to raw bytes. This means other Rust programs will be able to accept the bytes and convert them back to Rust data types, but not other languages. As we are writing our WASM library in Rust and we will be using Rust to interact with our WASM library. If we had a different language interacting with the WASM library, we would need another serialization method, such as JSON, CBOR, or Protobuf, but for this example, bincode works. Also note that we are not using any WASM dependencies here; instead, we are just going to expose C interfaces and directly interact with them. Don't worry, we will still be able to compile into WASM without WASM crates declared in this file.

We can now start writing our C interfaces. To do this, we need the following imports:

```
// File: wasm-lib/src/lib.rs
extern crate alloc;
use core::alloc::Layout;
use kernel::{
    SomeDataStruct,
    ResultPointer
};
```

Here, we are basically importing the structs from the kernel, and then the struct (`SomeDataStruct`) and

the library. With these imports, we can create our memory allocation function with the following code:

```
// File: wasm-lib/src/lib.rs
#[no_mangle]
pub unsafe extern "C" fn ns_malloc(
    size: u32, alignment: u32
) -> *mut u8 {
    let layout = Layout::from_size_align_unchecked(
        size as usize, alignment as usize
    );
    alloc::alloc::alloc(layout)
}
```

Here, we are taking the size of memory needed, and the alignment of where that memory allocation starts in the memory. We then create a `Layout` struct from the core of the library of the Rust language. This `Layout` is essentially a slice of memory. Finally, we pass that slice of memory into the `alloc` function, which allocates the memory for it. We then return a `*mut u8`, which is a raw pointer to a memory address. Note that the `ns_malloc` function is unsafe, as the `from_size_align_unchecked` function is unsafe. The `from_size_align_unchecked` function is unsafe because we are directly accessing memory, so the Rust compiler cannot check and enforce memory-safe patterns. We are using an unsafe approach because we are going to pass data from the main Rust binary to the WASM binary. Because they are different binaries, the Rust compiler cannot ensure that the data inserted into the memory from one binary is safe, because the compiler does not have access to that other binary when compiling.

Now that we have our memory allocation, we must free up the memory when we no longer need it. If we do not free up the memory, the used memory will continue to grow. This is called a memory leak. If the program is not stopped, it will run out of memory, at which point the computer will terminate the program.

Our freeing-up function takes the following form:

```
// File: wasm-lib/src/lib.rs
#[no_mangle]
pub unsafe extern "C" fn ns_free(
    ptr: *mut u8, size: u32, alignment: u32
) {
    let layout = Layout::from_size_align_unchecked(
        size as usize, alignment as usize
    );
    alloc::alloc::dealloc(ptr, layout);
}
```

Here, we can see that we get the layout struct again, but we apply a memory deallocation as opposed to an allocation.

Now that all our memory management code is finally done, we can move on to our entry point, which takes the following outline:

```
// File: wasm-lib/src/lib.rs
#[no_mangle]
pub extern "C" fn entry_point(
    ptr: *const u8, len: usize) ->
    *const ResultPointer {
    . . .
}
```

Here, we accept the pointer and length, and then return the `ResultPointer` so the client code can try and extract the result.

Inside our entry point function, we get the bytes from the memory, deserialize the bytes to our `SomeDataStruct` struct, and then alter the deserialized struct with the following code:

```
// File: wasm-lib/src/lib.rs
let bytes = unsafe { std::slice::from_raw_parts(ptr, len) };
let mut data_struct: SomeDataStruct = bincode::deserialize(
    bytes
).unwrap();
data_struct.names.push("new name".to_string());
```

That's it, that's the logic of our WASM program done, so now we must serialize the data and get the length and pointer to the serialized data with the following code:

```
// File: wasm-lib/src/lib.rs
let serialized_data = bincode::serialize(
    &data_struct
).unwrap();
let len = serialized_data.len();
let out_ptr = serialized_data.leak().as_ptr();
```

You can see that we are getting the pointer from the serialized struct. Here, the memory is not automatically cleaned up; otherwise, foreign function interfaces would be hard to implement as we need the memory allocation to hang around for the Rust program to access after the Rust program has interacted with the WASM binary. We can just return the pointer, and then manually clean up the memory later when we need to.

Now that we have our pointer and length of the memory buffer, we can return the pointer and length with the following code:

```
// File: wasm-lib/src/lib.rs
let result = Box::new(ResultPointer{
    ptr: out_ptr as i32,
    len: len as i32
});
```



```
});
Box::into_raw(result) as *const ResultPointer
```

Our library is nearly done; all the Rust code is finished, but we need to build a Bash script to build the WASM binary, and copy over the WASM binary so we can ensure that the WASM binary is where we want it to be. Our Bash script just takes the following form:

```
# File: wasm-lib/scripts/build.sh
#!/bin/bash
# navigate to directory
SCRIPTPATH="$( cd "$(dirname "$0")" ; pwd -P )"
cd $SCRIPTPATH
cd ..

cargo build --release --target wasm32-wasi

cp target/wasm32-wasi/release/wasm_lib.wasm ./wasm_lib.wasm
wasm2wat ./wasm_lib.wasm > ./wasm_lib.wat
```

This script takes a similar form to our build Bash script in our frontend. With that done, we are now ready to build the client.

Building a WASM client

For our client, we must serialize and deserialize data and load the WASM file. Because of this, we have the following dependencies:

```
// File: wasm-client/Cargo.toml
[dependencies]
serde = { version = "1.0.197", features = ["derive"] }
tokio = { version = "1.37.0", features = ["full"] }
bincode = "1.3.3"
kernel = { path = "../kernel" }
wasmtime-wasi = { version = "20.0.0", features = ["preview1"]}
wasmtime = "20.0.0"
```

The crate `wasmtime` is the runtime that is going to run and load the WASM binary. The `wasmtime-wasi` crate is the linker between the `wasmtime` runtime and the WASM binary. You may have noticed that there is also a `preview1` in the features. This refers to the version of the API that we are targeting. For instance, if we want to write a file, print to the terminal, or access a port, our WASM code will need to call a function in the runtime to do this, like a `syscall` in an operating system. At the time of writing this, we are targeting `preview1`, but there will be later releases of the API as the ecosystem matures. Now that our dependencies are defined, we can start building the client with the following outline:

```
// File: wasm-client/src/main.rs
use wasmtime::{Result, Engine, Linker, Module, Store, Config};
```

```

use wasmtime_wasi::preview1::{self, WasiP1Ctx};
use wasmtime_wasi::WasiCtxBuilder;
use std::mem::size_of;
use std::slice::from_raw_parts;
use kernel::{
    SomeDataStruct,
    ResultPointer
};

#[tokio::main]
async fn main() -> Result<()> {
    . . .
}

```

Inside our main function, we begin by starting a WASM engine and loading the WASM binary with the following code:

```

// File: wasm-client/src/main.rs
let mut config = Config::new();
config.async_support(true);
let engine = Engine::new(&config).unwrap();
let module = Module::from_file(
    &engine, "../wasm-lib/wasm_lib.wasm"
).unwrap();

```

Now that we have loaded our WASM library into our engine, we can link `preview1` and `configs` with the following code, so if our WASM module is making any calls to the runtime, such as printing to the terminal, it can:

```

// File: wasm-client/src/main.rs
let mut linker: Linker<WasiP1Ctx> = Linker::new(&engine);
preview1::add_to_linker_async(&mut linker, |t| t).unwrap();
let pre = linker.instantiate_pre(&module)?;
let wasi_ctx = WasiCtxBuilder::new()
    .inherit_stdio()
    .inherit_env()
    .build_p1();

```

Here, we can see that we enable standard IO and environment variables. `preview1` is essentially a version of WASM. For instance, at the time of writing this book, `preview` is currently being worked on for better support for sockets. Our runtime must enable the standard IO and environment variables because by itself, WASM is locked down and can't access operating system interfaces. This makes sense as WASM can be loaded and run in the browser.

Now that we have the `wasmtime` runtime built with our config, we can create a memory store and create an instance of our WASM module with that memory with the following code:

```
// File: wasm-client/src/main.rs
let mut store = Store::new(&engine, wasi_ctx);
let instance = pre.instantiate_async(&mut store).await.unwrap();
```

With the instance being created, we are now at the stage where we can serialize our data struct. We do this with the following code:

```
// File: wasm-client/src/main.rs
let data_struct = SomeDataStruct {
    names: vec!["name1".to_string(), "name2".to_string()],
    name: "name3".to_string()
};
let serialized = bincode::serialize(&data_struct).unwrap();
```

With our serialized struct, we can assign our struct bytes to our WASM instance memory with the following code:

```
// File: wasm-client/src/main.rs
// allocate the memory for the input data
let malloc = instance.get_typed_func::<(i32, i32), i32>(
    &mut store, "ns_malloc"
).unwrap();
let input_data_ptr = malloc.call_async(
    &mut store, (serialized.len() as i32, 0)
).await.unwrap();

// write the contract to the memory
let memory = instance.get_memory(
    &mut store, "memory"
).unwrap();
memory.write(
    &mut store, input_data_ptr as usize, &serialized
).unwrap();
```

Now that our serialized struct is in the memory of the WASM instance, we can get our entry point function from the WASM module and then call the entry point function to get the pointer to the result. We do this with the following code:

```
// File: wasm-client/src/main.rs
let entry_point = instance.get_typed_func::<(i32, i32), i32>(
    &mut store, "entry_point"
).unwrap();
```

```
let ret = entry_point.call_async(
    &mut store, (input_data_ptr, serialized.len() as i32)
).await.unwrap();
```

We can now use the pointer to read the bytes from the memory. First, we get the result pointer from the memory with the following code:

```
// File: wasm-client/src/main.rs
let mut result_buffer = Vec::with_capacity(
    size_of::<ResultPointer>()
);
for _ in 0..size_of::<ResultPointer>() {
    result_buffer.push(0);
}
memory.read(
    &mut store, ret as usize, &mut result_buffer
).unwrap();
let result_struct = unsafe {
    &from_raw_parts::<ResultPointer>(
        result_buffer.as_ptr() as *const ResultPointer, 1
    )[0]
};
```

Now that we have our result pointer, we use this to read the struct we want from memory with the following code:

```
// File: wasm-client/src/main.rs
let mut output_buffer: Vec<u8> = Vec::with_capacity(
    result_struct.len as usize
);
output_buffer.resize(result_struct.len as usize, 0);
memory.read(
    &mut store, result_struct.ptr as usize, &mut output_buffer
).unwrap();
let output = bincode::deserialize::<SomeDataStruct>(
    &output_buffer
).unwrap();
println!("Output contract: {:?}", output);
```

We now have the altered data that we want.

Before we finish our program, we should clean up the rest of the memory:

```
// File: wasm-client/src/main.rs
let free = instance.get_typed_func::<(i32, i32, i32), ()>(&
```

```

    &mut store, "ns_free"
  ).unwrap();
  free.call_async(
    &mut store, (input_data_ptr, serialized.len() as i32, 0)
  ).await.unwrap();
  free.call_async(
    &mut store, (result_struct.ptr, result_struct.len, 0)
  ).await.unwrap();
  free.call_async(
    &mut store, (ret, size_of::<ResultPointer>() as i32, 0)
  ).await.unwrap();
  Ok(())

```

Now our program is finished. If we run it, we get the following printout:

```

Running `target/debug/wasm-client`
Output contract: SomeDataStruct {
  name: "name3",
  names: ["name1", "name2", "new name"]
}

```

We can see that our struct was passed through to the WASM module, altered, and then passed back to the Rust host to be displayed. While this is not going to affect our to-do application, in *Chapter 20*, we will build a queuing system on top of Redis that will accept WASM binaries, put them on a Redis queue, and have Rust workers that will consume these WASM modules and execute them. We have now managed to directly interact with WASM without any helper crates creating the interfaces in the WASM library. As the features and support for WASM increase, you will be able to interact with these updates and feel confident to run whatever the bytecode alliance throws your way.

Loading WASM on a local machine cheatsheet

We use the `wasmtime` crate to load the WASM binary as a module, initialize it with a state, and link it to the `preview1` runtime API so our WASM module has access to functionality like.

We can define a pointer in memory and how much memory is needed. We can then serialize data structures and put them in that slot of memory. Once the data structure is put in the memory, we pass the pointer over to the WASM module so the data structure can be accessed inside the WASM module.

In our example, we were using Rust to run the WASM module, and we were using Rust to build the WASM module, so we used `bincode` for easy serialization. However, if you need the data serialization to be language-agnostic, using formats such as JSON, CBOR, or Protobuf will work when the client and WASM module are not built using the same language.



Summary

In this chapter, we got to grips with packaging WASM for the frontend and interacting with WASM using Rust. While we did not build anything substantial in WASM, we focused on developing foundational knowledge on how WASM is interacted with. At the time of writing this book, the APIs and features of WASM are rapidly changing. If we built a feature-rich WASM program using the current APIs, this book would age quickly, and you would have to Google the new APIs to figure out what you need to change to get your system running. Keeping an eye on WASM is a good idea. The advantage of compiling once and running anywhere, including the browser, is a strong one.

We are now at the stage where our basic application works in terms of creating, updating, and deleting to-do items. However, if we were to deploy it right now for multiple users, our data storage solution would slow down the number of requests and we would be held back by the lack of concurrency and networking capabilities our storage solution has. In the next chapter, we will upgrade our storage solution from a file to a Postgres database.

Questions

1. What is the WASM memory model and how is it different from the **Random-Access Memory (RAM)** model?
2. How can you send a complex data structure over the WASM boundary?
3. Let's say you have a compiled WASM binary in your public folder. How do you load functions from that WASM file into your React application?
4. How do we check to see whether the functions that we defined in the WASM program are in the WASM binary?
5. What are the advantages of building a separate kernel in its own workspace?

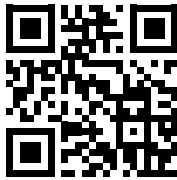
Answers

1. WASM has a linear memory model, meaning that all the memory is next to each other in one block. This results in faster memory access as we can just increment our pointer to access the next segment of memory. However, the linear memory model is more dangerous than RAM due to a hacker potentially having access to the entire memory of the program by exploiting a buffer overflow. Luckily, Rust is memory-safe, so it is not much of an issue; however, be careful when implementing unsafe memory allocation functions, as we did in the chapter.
2. You can send a complex data structure over the WASM boundary by serializing the data structure, allocating the bytes of the serialized data structure in the WASM instance memory, and then passing the pointer and length of the byte buffer through to the WASM program. The WASM program can then access the bytes in the memory and deserialize it.
3. Your React component can use `useEffect` to fire `init` from `wasm-pack` to fetch the binary. Once the binary is served, we can update the state of the component to say that the WASM binary has been loaded. We can also use other `useEffect` statements to perform actions on the other parts of the component that depend on the WASM binary, with the `useEffect` statement having a dependency of a flag to say that WASM is loaded.

4. We use the `wasm2wat` command on the WASM file and write the output to a `wat` file. In the `wat` file, we can then scroll down to the bottom to see the export functions section. If our function is there, that means that it is accessible to users to interact with the WASM binary.
5. A kernel in its own workspace acts as a single source of truth; we just have to update it once and all modules and programs communicating through that kernel get that update. We can also scale the number of programs and libraries interacting with the program that has the kernel easily without having to write any more boilerplate code.

Learn more on Discord

To join the Discord community for this book – where you can share feedback, ask questions to the author, and learn about new releases – follow this QR code: <https://packt.link/EaKXL>



9

Data Persistence with PostgreSQL

By this point in the book, the frontend for our application has been defined, and our app is working at face value. However, we know that our app is reading and writing from a JSON file.

In this chapter, we get rid of our JSON file and introduce a **PostgreSQL** database to store our data. We do this by setting up a database development environment using Docker. We then build data models in **Rust** to interact with the database, refactoring our app so that the create, edit, and delete endpoints interact with the database instead of the JSON file. Finally, we exploit Rust traits so any database handle that has implemented our database transaction traits can be swapped into the application with minimal effort.

In this chapter, we will cover the following topics:

- Building our PostgreSQL database
- Adding SQLX to the data access layer
- Defining our database transactions
- Connecting our transactions to the core
- Connecting our transactions to the server
- Creating our database migrations
- Refactoring our frontend

By the end of this chapter, you will be able to manage an application that performs reading, writing, and deleting data in a PostgreSQL database with data models. If we make changes to the data models, we will be able to manage them with migrations. You will also be able to utilize Rust traits to generalize a database interface, so integrating other databases will be simple and scalable.

Technical requirements

In this chapter, we will be using **Docker** to define and run a PostgreSQL database. This will enable our app to interact with a database on our local machine. Docker can be installed by following the instructions at <https://docs.docker.com/engine/install/>.

For some Bash scripts, we will be relying on some Postgres binaries. You will have to install Postgres by using a package manager such as brew or apt-get. However, this is not essential as we will also cover how to use Docker to make client calls to Postgres.

We will also be using **Docker Compose** on top of Docker to orchestrate our Docker containers. This can be installed by following the instructions at <https://docs.docker.com/compose/install/>.

You can download the example project and code for this book by following the instructions in the **Download the example code files** section in the *Preface* of this book.

This chapter's code files are included in the downloadable code bundle.

Building our PostgreSQL database

Up to this point in the book, we have been using a JSON file to store our to-do items. This has served us well so far. In fact, there is no reason why we cannot use a JSON file throughout the rest of the book to complete the tasks. However, if you do use a JSON file for production projects, you will come across some downsides.

Why we should use a proper database

If the reads and writes to our JSON file increase, we can face some concurrency issues and data corruption. There is also no checking on the type of data. Therefore, another developer can write a function that writes different data to the JSON file, and nothing will stand in the way.

There is also an issue with migrations. If we want to add a timestamp to the to-do items, this will only affect new to-do items that we insert into the JSON file. Therefore, some of our to-do items will have a timestamp, and others won't, which would introduce bugs into our app. Our JSON file also has limitations in terms of filtering.

Right now, all we do is read the whole data file, alter an item in the whole dataset, and write the whole dataset to the JSON file. This is not effective and will not scale well. It also inhibits us from linking these to-do items to another data model-like user. Plus, we can only search right now using the status. If we used a SQL database that has a user table that is linked to a to-do item database, we would be able to filter to-do items based on the user, status, or title. We can even use a combination thereof. When it comes to running our database, we are going to use Docker. So why should we use Docker?

Why use Docker?

To understand why we would use Docker, we need to understand what Docker is. Docker essentially has containers that work like virtual machines, but in a more specific and granular way. Docker containers isolate a single application and all the application's dependencies. These containers then run the application inside. Docker containers can then communicate with each other. Because Docker containers share a single common operating system, they are compartmentalized from one another and from the operating system at large, meaning that containerized applications use less memory compared to virtual machines. Because of Docker containers, we can be more portable with our applications. If the Docker container runs on my machine, it will run on another machine or server that also has Docker. We can also package our applications, meaning that extra packages specifically for our application to run do not need to be installed separately, including dependencies on the operating system level. As a result, Docker gives us great flexibility in web development, as we can simulate servers and databases on our local machine.

How to use Docker

With all this in mind, it makes sense to go through the extra steps necessary to set up a SQL database and run it. To do this, we are going to use Docker: a tool that helps us create and use containers. Containers themselves are a Linux technology that packages and isolates applications along with their entire runtime environment. Containers are technically isolated filesystems, but to help visualize what we are doing in this chapter, you can think of them as mini, lightweight virtual machines. These containers are made from images that can be downloaded from **Docker Hub**. We can insert our own code into these images before spinning up a container out of them, as seen in the following figure:

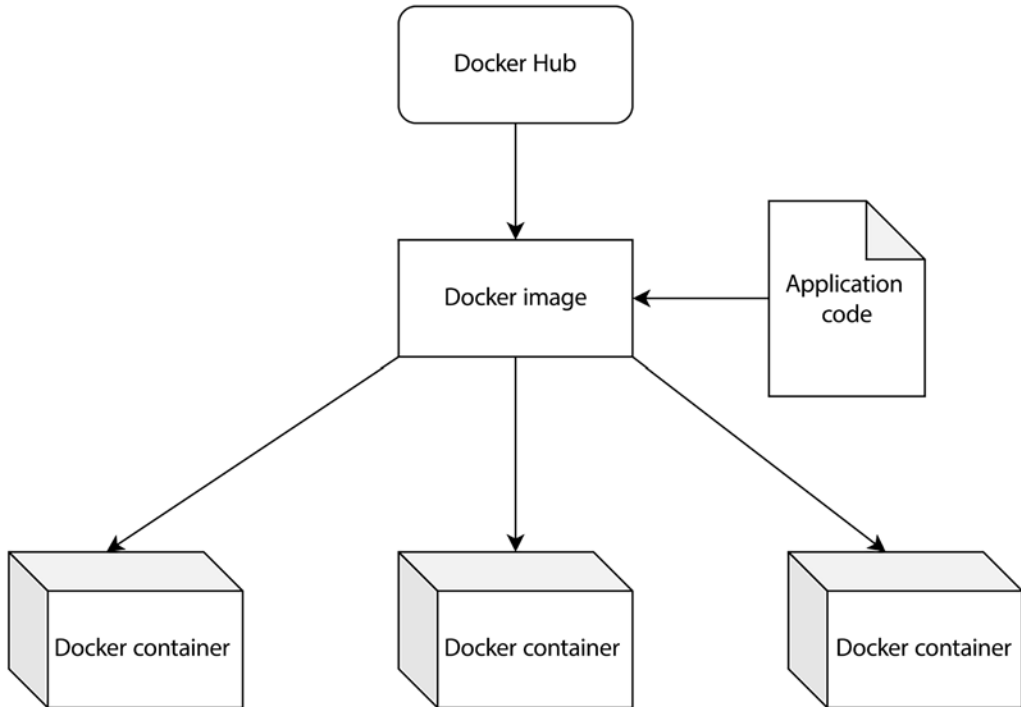


Figure 9.1 – Relationship between Docker images and containers

With Docker, we can download an image like a PostgreSQL database and run it in our development environment. Because of Docker, we can spin up multiple databases and apps, and then shut them down as and when we need. First, we need to take stock of our containers by running the following command in the terminal:

```
docker container ls -a
```

If Docker is a fresh installation, we get the following output:

CONTAINER ID	IMAGE	COMMAND	CREATED	STATUS	PORTS	NAMES
--------------	-------	---------	---------	--------	-------	-------

As we can see, we have no containers. We also need to take stock of our images. This can be done by running the following terminal command:

```
docker image ls
```

The preceding command gives the following output:

REPOSITORY	TAG	IMAGE ID	CREATED	SIZE
------------	-----	----------	---------	------

Again, if Docker is a fresh installation, then there will be no containers. If you have downloaded the Docker Desktop app, you can also inspect your containers and images there.

There are other ways in which we can create a database in Docker. For instance, we can create our own **DockerFile** where we define our own **operating system (OS)** and configurations. However, we have the *docker-compose* binary installed. Using *docker-compose* will make the database definition straightforward. It will also enable us to add more containers and services. Docker also now has a *compose* command, so we do not need to download the *docker-compose* binary if we have the latest Docker version installed. In some online tutorials, you will see the *docker-compose* command, which is just calling the *docker-compose* binary. In other tutorials, you might see the *docker compose* command, which is calling the *compose* command from Docker itself. Both will run *docker-compose* files. To define our PostgreSQL database, we code the following **YAML** code in a *docker-compose.yml* file in the root directory:

```
version: "3.7"

services:

  postgres:
    container_name: 'to-do-postgres'
    image: 'postgres:11.2'
    restart: always
    ports:
      - '5432:5432'
    environment:
      - 'POSTGRES_USER=username'
      - 'POSTGRES_DB=to_do'
      - 'POSTGRES_PASSWORD=password'
```

In the preceding code, at the top of the file, we have defined the version. Older versions, such as 2 or 1, have different styles in which the file is laid out. If we do not mention a version in the *docker-compose.yml* file, then Docker will default to the latest version. The different versions also support different arguments. At the time of writing this book, version 3 is the latest version. The following URL covers the changes between each *docker-compose* version: <https://docs.docker.com/compose/compose-file/compose-versioning/>.

We then define our database service that is nested under the `postgres` tag. Tags such as `postgres` and `services` denote dictionaries, and lists are defined with `-` for each element. If we were to convert our `docker-compose` file to JSON, it would have the following structure:

```
{
  "version": "3.7",
  "services": {
    "postgres": {
      "container_name": "to-do-postgres",
      "image": "postgres:11.2",
      "restart": "always",
      "ports": [
        "5432:5432"
      ],
      "environment": [
        "POSTGRES_USER=username",
        "POSTGRES_DB=to_do",
        "POSTGRES_PASSWORD=password"
      ]
    }
  }
}
```

In the preceding code, we can see that our services are a dictionary of dictionaries, denoting each service. Thus, we can deduce that we cannot have two tags with the same name, as we cannot have two dictionary keys that are the same. The previous code also tells us that we can keep stacking on service tags with their own parameters.

Running a database in Docker

With our database service, we have a name; so, when we look at our containers, we know what each container is doing in relation to the service, such as a server or database. In terms of configuring the database and building it, we luckily pull the official `postgres` image. This image has everything configured for us, and Docker will pull it from the repository. The image is like a blueprint. We can spin up multiple containers with their own parameters from that one image that we pulled. We then define the restart policy as `always`. This means that the container will always restart when the containers exit. We can also define it to only restart based on a failure or when stopping.

It should be noted that Docker containers have their own ports that are not open to the machine. However, we can expose container ports and map the exposed port to an internal port inside the **Docker** container. Considering these features, we can define our ports.

However, in our example, we will keep our definition simple. We state that we accept incoming traffic to the **Docker** container on port 5432 and route it through to the internal port 5432. We then define our environment variables, which are the username, the name of the database, and the password. While we are using generic, easy-to-remember passwords and usernames for this book, it is advised that you switch to more secure passwords and usernames if pushing to production. We can build a spin up for our system by navigating to the root directory where our docker-compose file is, by running the following command:

```
docker-compose up
```

The preceding command will pull down the postgres image from the repository and start constructing the database. After a flurry of log messages, the terminal should come to rest with the following output:

```
LOG:  listening on IPv4 address "0.0.0.0", port 5432
LOG:  listening on IPv6 address ":::", port 5432
LOG:  listening on Unix socket "/var/run/postgresql/.s.PGSQL.5432"
LOG:  database system was shut down at 2022-04-23 17:36:45 UTC
LOG:  database system is ready to accept connections
```

As you can see, the date and time will vary. However, what we are told here is that our database is ready to accept connections. Yes, it is really that easy. Therefore, Docker adoption is unstoppable. *Ctrl + C* will stop our docker-compose, thus shutting down our postgres container.

We now list all our containers with the following command:

```
docker container ls -a
```

The preceding command gives us the following output:

CONTAINER ID	IMAGE	COMMAND
c99f3528690f	postgres:11.2	"docker-entrypoint.s..."
CREATED	STATUS	PORTS
4 hours ago	Exited (0) About a minute ago	
NAMES		
to-do-postgres		

In the preceding output, we can see that all the parameters are there. The ports, however, are empty because we stopped our service. We must also note that the container ID is randomly generated and different from all other containers, so we can reference multiple different Docker containers with the same image and ports.

Exploring routing and ports in Docker

If we were to start our service again and list our containers in another terminal, port 5432 would be under the PORTS tag. We will need to reference these if we're accessing logs. When we are running `docker-compose up`, we essentially use the following structure:

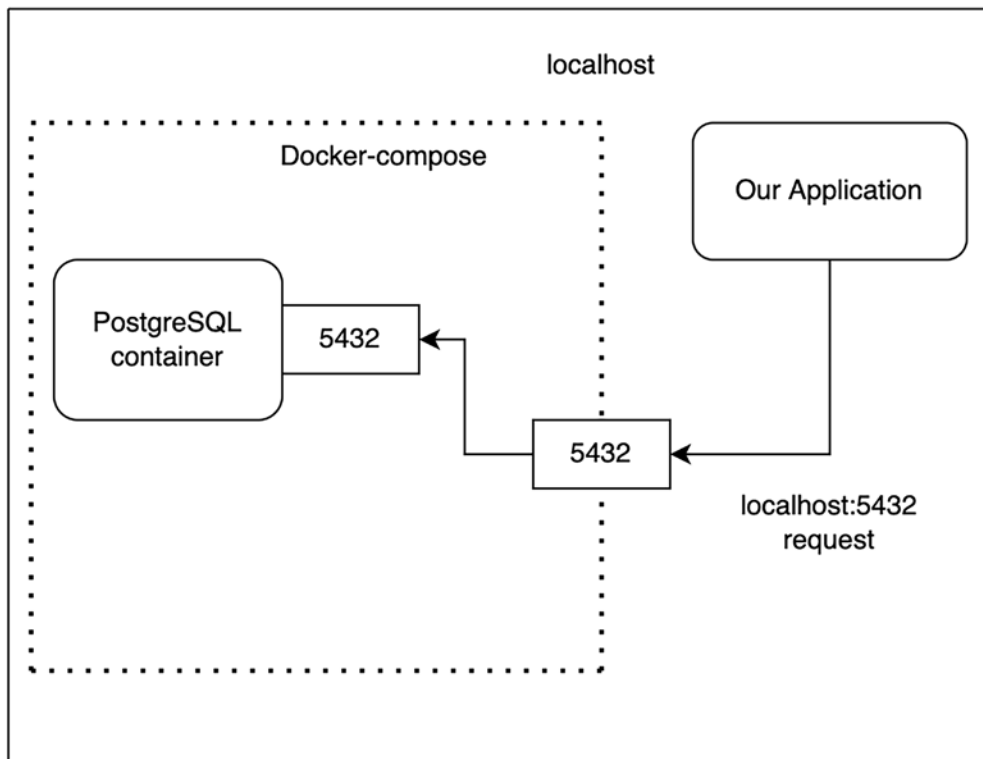


Figure 9.2 – Docker Compose serving our database

In Figure 9.2, we can see that our docker-compose uses a unique project name to keep containers and networks in their namespace. It must be noted that our containers are running on the localhost. Therefore, if we want to make a call to a container managed by docker-compose, we will have to make a localhost request. However, we must make the call to the port that is open from docker-compose, and docker-compose will route it to the port that is defined in the docker-compose config.yml file. For instance, we have two databases with the following .yml file:

```

version: "3.7"

services:
  postgres:
    container_name: 'to-do-postgres'
    image: 'postgres:11.2'
    restart: always
    ports:
      - '5432:5432'
    environment:
      - 'POSTGRES_USER=username'
      - 'POSTGRES_DB=to_do'

```

```

- 'POSTGRES_PASSWORD=password'
postgres_two:
  container_name: 'to-do-postgres_two'
  image: 'postgres:11.2'
  restart: always
  ports:
    - '5433:5432'
  environment:
    - 'POSTGRES_USER=username'
    - 'POSTGRES_DB=to_do'
    - 'POSTGRES_PASSWORD=password'

```

In the preceding code, we can see that both of our databases accept traffic into their containers through port 5432. However, there would be a clash, so one of the ports that we open with is port 5433, which is routed to port 5432 in the second database container, which gives us the following layout:

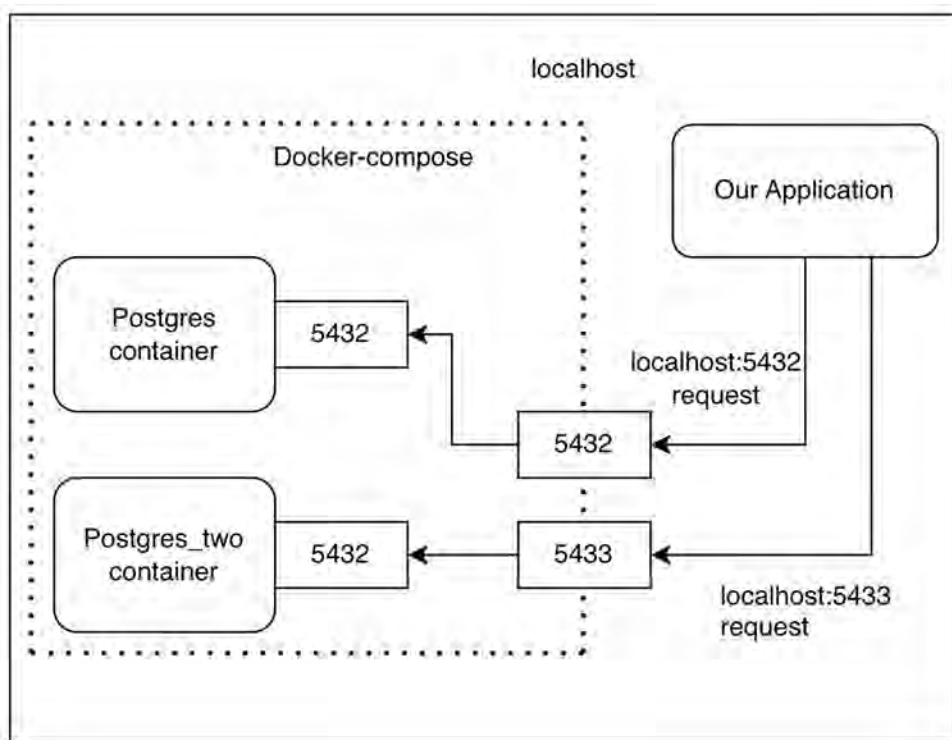


Figure 9.3 – Docker Compose serving multiple databases

This routing gives us flexibility when running multiple containers. We are not going to run multiple databases for our to-do application, so we should delete our `postgres_two` service. Once we have deleted our `postgres_two` service, we can run `docker-compose` again and then list our containers with the following command:

```
docker image ls
```

The preceding command will now give us the following output:

REPOSITORY	TAG	IMAGE ID
postgres	11.2	3eda284d1840
CREATED	SIZE	
17 months ago	312MB	

In the preceding output, we can see that our image has been pulled from the `postgres` repository. We also have a unique/random ID for the image, and we have a date for when that image was created.

Now that we have a basic understanding of how to get our database up and running, we can run `docker-compose` in the background with the following command:

```
docker-compose up -d
```

The preceding command just tells us which containers have been spun up with the following output:

```
Starting to-do-postgres ... done
```

We can see our status when we list our containers with the following output:

STATUS	PORTS	NAMES
Up About a minute	0.0.0.0:5432->5432/tcp	to-do-postgres

In the previous output, the other tags are the same, but we can also see that the `STATUS` tag tells us how long the container has been running, and which port it is occupying. While `docker-compose` is running in the background, it does not mean we cannot see what is going on. We can access the logs of the container anytime by calling the `logs` command and referencing the ID of the container with the following command:

```
docker logs c99f3528690f
```

The preceding command should give out the same output as our standard `docker-compose up` command. To stop `docker-compose`, we can run the `stop` command, as follows:

```
docker-compose stop
```

The preceding command will stop our containers in `docker-compose`. It must be noted that this is different from the `down` command, shown here:

```
docker-compose down
```

The down command will also stop our containers. However, the down command will delete the container. If our database container is deleted, we will also lose all our data.

There is a configuration parameter called volumes that can prevent the deletion of our data when the container is removed; however, this is not essential for local development on our computers. In fact, you will want to delete containers and images from your laptop regularly. *I did a purge on my laptop once of containers and images that I was no longer using, and this freed up 23 GB!*

Docker containers on our local development machines should be treated as temporary. While Docker containers are multiple and more lightweight than standard virtual machines, they are not free in terms of computing resources. The idea behind Docker running on our local machines is that we can simulate what running our application would be like on a server. If it runs in Docker on our laptop, we can be certain that it will also run on our server, especially if the server is being managed by a production-ready Docker orchestration tool such as **Kubernetes**.

Running Docker in the background with Bash scripts

Docker can also help with consistent testing and development. We want to be able to have the same results every time we run a test. We also want to onboard other developers easily and enable them to tear down and spin up containers that will support development quickly and easily. I have personally seen development delayed when not supporting easy tear-down and spin-up procedures. For instance, when working on a complex application, the code that we are adding and testing out might scar the database. Reverting might not be possible, and deleting the database and starting again would be a pain, as reconstructing this data might take a long time. The developer may not even remember how they constructed the data in the first place. There are multiple ways to prevent this from happening, and we will be covering these in *Chapter 15*.

For now, we will build a Bash script that spins up our database in the background, waits until the connection to our database is ready, and then tears down our database. This will give us the foundations to build pipelines, tests, and onboarding packages to start development. To do this, we will create a directory in the root directory of our Rust web application called scripts. We can then create a scripts/wait_for_database.sh file housing the following code:

```
#!/bin/bash

# navigate to directory
SCRIPTPATH="$( cd "$(dirname "$0")" ; pwd -P )"
cd $SCRIPTPATH
cd ..
docker-compose up -d

until pg_isready -h localhost -p 5432 -U username
do
    echo "Waiting for postgres"
    sleep 2;
```

```
done

echo "docker is now running"
docker-compose down
```

Using the preceding code, we move the current working directory of the script out of the scripts directory and into our root directory. We then start docker-compose in the background. Next, we loop, pinging the 5432 port utilizing the `pg_isready` command to wait until our database is ready to accept connections.

The Bash command `pg_isready` might not be available on your computer. The `pg_isready` command usually comes with the installation of the PostgreSQL client. Alternatively, you can use the following Docker command instead of `pg_isready`:

```
until docker run -it postgres --add-host host.docker.internal:host-gateway
docker.io/postgres:14-alpine -h localhost -U username pg_isready
```

What is happening here is that we are using the postgres Docker image to run the `pg_isready` command instead of installing the postgres client on our local computer.

Once our database is running, we print out to the console that our database is running, and then tear down our docker-compose destroying the database container. Running the command that runs the `wait_for_database.sh` Bash script will give the following output:

```
✓ sh wait_for_database.sh
[+] Running 0/0
✓ Network web_app_default Creating      0.2s
✓ Container to-do-postgres Started     1.5s
localhost:5432 - no response
Waiting for postgres
localhost:5432 - no response
Waiting for postgres
localhost:5432 - accepting connections
docker is now running
[+] Running 1/1
✓ Container to-do-postgres Removed      1.2s
✓ Network web_app_default Removed
```

From the preceding output, considering that we tell our loop to sleep for two seconds at every iteration of the loop, we can deduce that it took roughly four seconds for our newly spun-up database to accept connections. Thus, we can say that we have achieved basic competency in managing local databases with Docker.

In this section, we set up our environment. We also understood the basics of Docker enough to build, monitor, shut down, and delete our database with just a few simple commands. Now, we can move on to the next section, where we'll be adding the SQLX crate to the data access layer.

Adding SQLX to the data access layer

Before we cover how to handle SQLX in the data access layer, it makes sense to explain why we are using SQLX. In previous editions, we covered Diesel for the data access, as this was an ORM for Postgres and Rust. However, you get more flexibility by just writing SQL, and the nature of Rust structs that can be augmented by traits means that the results can simply yield structs that we can directly work with. Furthermore, SQLX does not require a complex schema setup where all the columns need to match exactly when declaring the schema; instead, we just write the SQL queries and pass in the struct that we want returned in the query. However, if you feel passionately that you want to use Diesel, do not worry, we are going to structure the data access layer in such a way that you can keep adding different data access methods. By the end of this section, we will have a data access layer that supports the JSON file storage method that we are currently using and the SQLX-powered Postgres.

Before we write any code, we should orient ourselves with the refactor of the data access layer in our to-do nanoservice by looking at the following file layout:

```
├─ Cargo.toml
├─ src
│   ├── connections
│   │   ├── mod.rs
│   │   └─ sqlx_postgres.rs
│   ├── json_file.rs
│   ├── lib.rs
│   └─ to_do_items
│       ├── descriptors.rs
│       ├── enums.rs
│       ├── mod.rs
│       ├── schema.rs
│       └─ transactions
│           ├── create.rs
│           ├── delete.rs
│           ├── get.rs
│           ├── mod.rs
│           └─ update.rs
```

Here, we can see that the outline of the data access layer has the following key sections that we should note:

- `connections/`: This is where the connection code is housed for connecting the data access layer to a data storage engine. For this chapter, we will house the connection pool for the Postgres database here.
- `to_do_items/`: This directory houses all the code that is specific to the interaction of a data storage engine with to-do items.

- `to_do_items/descriptors.rs`: This houses placeholder structs for each storage engine supported for to-do items.
- `to_do_items/schema.rs`: This houses the data structs that are passed to and from the data store for to-do items.
- `to_do_items/transactions/`: This is where traits are stored for each data transaction we want to perform on to-do items. We will use the traits as interfaces, and then implement those traits for any storage engine we want to support that transaction. We can then pass these data storage engines into functions that accept structs that have implemented the trait for the specific transaction.

Before we write any code, we should configure our data access layer's `Cargo.toml` file with the following dependencies:

```
# file: nanoservices/to_do/dal/Cargo.toml
[features]
json-file = ["serde_json"]
sqlx-postgres = ["sqlx"]

[dependencies]
serde = { version = "1.0.197", features = ["derive"] }
glue = { path = "../../../glue" }

# for json-file
serde_json = { version = "1.0.114", optional = true }

# for sqlx-postgres
sqlx = {
    version = "0.8.3",
    features = ["postgres", "json"],
    optional = true
}
```

Here, we can see that the data access layer is aiming to support both SQLX and standard JSON file storage engines, depending on the feature that is selected. We can also see that we are selective with what crates we use. We do not want to be compiling SQLX if we are just using the `json-file` feature.

We are now ready to write code, so initially, we should make sure that all our code is available in our file with the following code:

```
// file: nanoservices/to_do/dal/src/lib.rs
pub mod to_do_items;
pub mod connections;

#[cfg(feature = "json-file")]
pub mod json_file;
```

The first module we should focus on is the connections, as our trait implementations will require the connections. We can create our database connection with the following code:

```
// file: nanoservices/to_do/dal/src/connections/sqlx_postgres.rs
use sqlx::postgres::{PgPool, PgPoolOptions};
use std::sync::LazyLock as Lazy;
use std::env;

pub static SQLX_POSTGRES_POOL: Lazy<PgPool> = Lazy::new(|| {
    let connection_string = env::var("TO_DO_DB_URL").unwrap();
    let max_connections = match std::env::var(
        "TO_DO_MAX_CONNECTIONS"
    ) {
        Ok(val) => val,
        Err(_) => "5".to_string()
    }.trim().parse::<u32>().map_err(|_e|{
        "Could not parse max connections".to_string()
    }).unwrap();
    let pool = PgPoolOptions::new()
        .max_connections(max_connections);
    pool.connect_lazy(&connection_string)
        .expect("Failed to create pool")
});
```

Here, we can see that we inspect the environment variables to get the URL of the Postgres database that we will connect to. We also check for the maximum number of connections for our connection pool. We use a connection pool to prevent us from having to make a connection every time we want to perform a database transaction. A **database connection pool** is a limited number of database connections. When our application needs a database connection, it will take the connection from the pool and then place the connection back into the pool when the application no longer needs the connection. If there are no connections left in the pool, the application will wait until there is a connection available, as seen in the following figure:

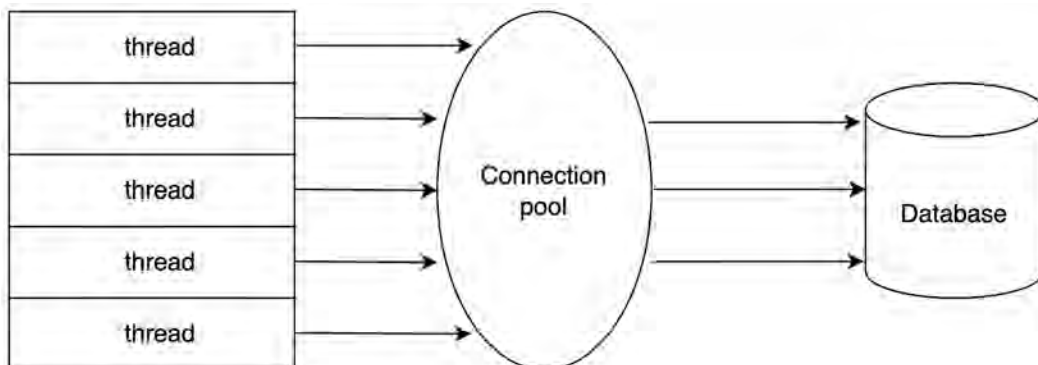


Figure 9.4 – Database connection pool with a limit of three

We also must note that our connection code is static but lazily evaluated, meaning it is initialized once only when initially called and then never again throughout the lifetime of the program, with the result of the connection code assigned to the `SQLX_POSTGRES_POOL` variable.

Now that our database connection pool is defined, we can ensure that the connection pool is available to the rest of the code if the SQLX feature is activated with the following code:

```
// file: nanoservices/to_do/dal/src/connections/mod.rs
#[cfg(feature = "sqlx-postgres")]
pub mod sqlx_postgres;
```

We can now move on to our schema, as this is another dependency that is required for the storage engine transactions. However, before we do this, we must copy over the `TaskStatus` enum from our `nanoservices/to_do/core/src/enums.rs` file to our `nanoservices/to_do/dal/src/to_do_items/enums.rs` file. With this in place, we can start our schema with the following imports:

```
// file: nanoservices/to_do/dal/src/to_do_items/schema.rs
use std::fmt;
use glue::errors::NanoServiceError;
use serde::{Serialize, Deserialize};
use super::enums::TaskStatus;
use std::collections::HashMap;
```

With these imports, we can build out our to-do item structs. However, we need two separate structs. In a postgres database, we want to be able to assign an ID to each row of the database table. However, when we are inserting the to-do item into the database, we will not know the ID; therefore, we need to have a new to-do item that does not have an ID that takes the following form:

```
// file: nanoservices/to_do/dal/src/to_do_items/schema.rs
#[derive(Serialize, Deserialize, Debug, Clone)]
pub struct NewToDoItem {
    pub title: String,
    pub status: TaskStatus
}
```

Once we use the `NewToDoItem` to store the new to-do item into the database, we can make queries to the database to get the to-do item, which is defined here:

```
// file: nanoservices/to_do/dal/src/to_do_items/schema.rs
#[derive(Serialize, Deserialize, Debug, Clone, PartialEq)]
#[cfg_attr(feature = "sqlx-postgres", derive(sqlx::FromRow))]
pub struct ToDoItem {
    pub id: i32,
    pub title: String,
    pub status: String
}
```

Here, we can see that we derive the `FromRow` trait if our `SQLX` feature is activated, as we need the `FromRow` trait to pass the `ToDoItem` as the return type for the query. We must also copy over our `AllToDoItems` struct with its functionality from our `nanoservices/to_do/core/src/structs.rs` file to our `nanoservices/to_do/dal/src/to_do_items/schema.rs` file, as we will want to render all the to-do items from our database.

Finally, before we move on to implementing the transactions, we must make our placeholder structs with the following code:

```
// file: nanoservices/to_do/dal/src/to_do_items/descriptors.rs
pub struct SqlxPostgresDescriptor;
pub struct JsonFileDescriptor;
```

And make our code for the to-do items public with the following code:

```
// file: nanoservices/to_do/dal/src/to_do_items/mod.rs
pub mod schema;
pub mod enums;
pub mod descriptors;
pub mod transactions;
```

And with this, we have defined the `SQLX` dependencies, database connection pool, schema, and descriptors. We can now connect to our database. However, we have not defined our database transactions. We must define these transactions so we can slot database functionality into our functions. In the next section, we define these database transactions.

Defining our database transactions

Our transactions module is essentially the API for our data transactions. These transactions do not need to interact with a data storage engine. Transactions could be API calls to another server or a state machine. We will just focus on data storage engines, but when we implement these transactions, you will see how powerful and flexible they are.

Before we define any transactions, however, we must ensure that our transactions are available with the following code:

```
// file: nanoservices/to_do/dal/src/to_do_items/transactions/mod.rs
pub mod create;
pub mod delete;
pub mod get;
pub mod update;
```

We can start our transaction definitions with the create transaction. First, we must import the following:

```
// file: nanoservices/to_do/dal/src/to_do_items/
// transactions/create.rs
```



```

use crate::to_do_items::schema::{ToDoItem, NewToDoItem};
use glue::errors::NanoServiceError;
use std::future::Future;

#[cfg(feature = "json-file")]
use super::super::descriptors::JsonFileDescriptor;
#[cfg(feature = "json-file")]
use crate::json_file::{get_all, save_all};
#[cfg(feature = "json-file")]
use std::collections::HashMap;

#[cfg(feature = "sqlx-postgres")]
use crate::connections::sqlx_postgres::SQLX_POSTGRES_POOL;
#[cfg(feature = "sqlx-postgres")]
use super::super::descriptors::SqlxPostGresDescriptor;
#[cfg(feature = "sqlx-postgres")]
use glue::errors::NanoServiceErrorStatus;

```

That is a lot of imports, but we can see by the features that we are importing enough to support our file and SQLX Postgres storage engines.

Before we write any logic for either our file or the Postgres engine, we must define our transaction signature with the following trait:

```

// file: nanoservices/to_do/dal/src/to_do_items/
// transactions/create.rs
pub trait SaveOne {
    fn save_one(item: NewToDoItem) ->
    impl Future<Output = Result<ToDoItem, NanoServiceError>> + Send;
}

```

Here, we can see that our trait function accepts a `NewToDoItem` struct and returns an async future that either returns a `ToDoItem` or `NanoServiceError`. With our implementation, the returned future can be awaited on. Because our database transactions are async, we will either need an async function or a function that returns a future. We are returning a future with specific traits, so we can be explicit about what traits are being implemented for the return type of our trait function. If we were to just use an async function in our trait, then we would not be explicit about whether the `Send` trait needs to be implemented or not for the future. This means that we can use our future in multithreaded contexts, which is our Tokio runtime. We must remember that Tokio is merely an implementation of an async runtime; other runtimes do exist, and some are not multithreaded. In fact, Tokio even gives you the option to not run in a multithreaded context; therefore, implementing the `Send` trait is not essential when in a single-threaded context of async in Tokio. However, we are going to use multithreading in our server, so we want to ensure that every async function that is returned by this trait can be sent over threads, including the parameters and return values.

Now that we have our trait defined, we can implement these traits for our storage engines with the following code:

```
// file: nanoservices/to_do/dal/src/to_do_items/
// transactions/create.rs
#[cfg(feature = "sqlx-postgres")]
impl SaveOne for SqlxPostGresDescriptor {
    fn save_one(item: NewToDoItem) ->
    impl Future<Output = Result<ToDoItem, NanoServiceError>> + Send
    {
        sqlx_postgres_save_one(item)
    }
}
#[cfg(feature = "json-file")]
impl SaveOne for JsonFileDescriptor {
    fn save_one(item: NewToDoItem) ->
    impl Future<Output = Result<ToDoItem, NanoServiceError>> + Send
    {
        json_file_save_one(item)
    }
}
```

Here, we can see that we merely pass the item into an async function that has not been awaited on, so when we call the trait function, the future is constructed and returned so the caller can await on the future.

We can now define these async functions, starting with the following Postgres one, where SQLX allows us to write the SQL queries directly:

```
// file: nanoservices/to_do/dal/src/to_do_items/
// transactions/create.rs
#[cfg(feature = "sqlx-postgres")]
async fn sqlx_postgres_save_one(item: NewToDoItem)
-> Result<ToDoItem, NanoServiceError> {
    let item = sqlx::query_as::<_, ToDoItem>("
        INSERT INTO to_do_items (title, status)
        VALUES ($1, $2)
        RETURNING *")
    ).bind(item.title)
    .bind(item.status.to_string())
    .fetch_one(&*SQLX_POSTGRES_POOL).await.map_err(|e| {
        NanoServiceError::new(
            e.to_string(),
```

```

        NanoServiceErrorStatus::Unknown
    )
    })?;
    Ok(item)
}

```

Here, we can see that we write an SQL query that inserts a new row into the `to_do_items` table. We will cover the creation of the `to_do_items` table in the section on migrations. Once the SQL query is defined, we bind the fields of the input item to the query.



Do we have to use bind? Why can't we just format a string?

SQL queries are at risk of an attack called SQL injection. This is where we inject another query into the SQL query. For instance, if we were just formatting a string and passing in the parameters to the string, we could type the title of the to-do item to be the following:

```
"'); DROP TABLE to_do_items; --"
```

The `;` character indicates that the query has finished, and that a new SQL query is about to run. The next SQL query in the string that we claimed was the title of the to-do item executes a SQL query that drops the entire table, wiping all our data. When working with SQL queries, it is important to use the sanitization checks that the SQL library provides to check and protect against SQL injections. SQLX functions such as `bind` check for special characters and escape the database transaction instead of executing it, protecting us from SQL injections.

Once we have configured our SQL query, we state that we are going to return the inserted row with `fetch_one`, and we pass in the database connection to the `fetch_one` function. We now have the newly inserted to-do item, and the ID assigned to that item during the insert.

For our JSON file, we are not going to be using IDs, but we must have the same signature as the trait; therefore, our JSON file function takes the following form:

```

// file: nanoservices/to_do/dal/src/to_do_items/
// transactions/create.rs
#[cfg(feature = "json-file")]
async fn json_file_save_one(item: NewToDoItem)
    -> Result<ToDoItem, NanoServiceError> {
    let mut tasks = get_all::<ToDoItem>().unwrap_or_else(|_|
        HashMap::new()
    );
    let to_do_item = ToDoItem {
        id: 1,
        title: item.title,
        status: item.status.to_string()
    }
}

```

```

    };
    tasks.insert(
        to_do_item.title.to_string(),
        to_do_item.clone()
    );
    let _ = save_all(&tasks)?;
    Ok(to_do_item)
}

```

Here, we can see that we are merely assigning the ID to the number one and leaving it at that. We could build our own ID system where we increase the ID by one every time we insert a row, but this would excessively bloat the chapter for a storage engine that we are moving away from. We do not need the ID of the to-do item to carry out the tasks, so it makes sense to keep the JSON file API functional but warn other developers to switch over to the Postgres database as soon as possible.

Our other transactions will be defined using the same structure. For our delete transaction, we need the following imports:

```

// file: nanoservices/to_do/dal/src/to_do_items/
// transactions/delete.rs
use crate::to_do_items::schema::ToDoItem;
use glue::errors::NanoServiceError;
use std::future::Future;

#[cfg(feature = "json-file")]
use super::super::descriptors::JsonFileDescriptor;
#[cfg(feature = "json-file")]
use crate::json_file::{get_all, save_all};
#[cfg(feature = "json-file")]
use std::collections::HashMap;

#[cfg(feature = "sqlx-postgres")]
use crate::connections::sqlx_postgres::SQLX_POSTGRES_POOL;
#[cfg(feature = "sqlx-postgres")]
use super::super::descriptors::SqlxPostGresDescriptor;

#[cfg(any(feature = "json-file", feature = "sqlx-postgres"))]
use glue::errors::NanoServiceErrorStatus;

```

And our delete trait takes the following form:

```

// file: nanoservices/to_do/dal/src/to_do_items/
// transactions/delete.rs
pub trait DeleteOne {

```

```
fn delete_one(title: String) ->
impl Future<Output = Result<ToDoItem, NanoServiceError>> + Send;
}
```

Our implementations are defined here:

```
// file: nanoservices/to_do/dal/src/to_do_items/
// transactions/delete.rs
#[cfg(feature = "sqlx-postgres")]
impl DeleteOne for SqlxPostgresDescriptor {
    fn delete_one(title: String) ->
    impl Future<Output = Result<ToDoItem, NanoServiceError>> + Send {
        sqlx_postgres_delete_one(title)
    }
}
#[cfg(feature = "json-file")]
impl DeleteOne for JsonFileDescriptor {
    fn delete_one(title: String) ->
    impl Future<Output = Result<ToDoItem, NanoServiceError>> + Send {
        json_file_delete_one(title)
    }
}
```

When it comes to our Postgres query, we use the title from the item passed into delete with the following function:

```
// file: nanoservices/to_do/dal/src/to_do_items/
// transactions/delete.rs
#[cfg(feature = "sqlx-postgres")]
async fn sqlx_postgres_delete_one(title: String) ->
    Result<ToDoItem, NanoServiceError> {
    let item = sqlx::query_as::<_, ToDoItem>("
        DELETE FROM to_do_items
        WHERE title = $1
        RETURNING *"
    ).bind(title)
    .fetch_one(&*SQLX_POSTGRES_POOL).await.map_err(|e| {
        NanoServiceError::new(
            e.to_string(),
            NanoServiceErrorStatus::Unknown
        )
    })?;
    Ok(item)
}
```

As for our JSON file to-do items, we use the title of the to-do item to delete the item with the following code:

```
// file: nanoservices/to_do/dal/src/to_do_items/
// transactions/delete.rs
#[cfg(feature = "json-file")]
async fn json_file_delete_one(title: String) ->
    Result<ToDoItem, NanoServiceError> {
    let mut tasks = get_all:::<ToDoItem>().unwrap_or_else(|_|
        HashMap::new()
    );
    let to_do_item = tasks.remove(
        &title
    ).ok_or_else(|| {
        NanoServiceError::new(
            "Item not found".to_string(),
            NanoServiceErrorStatus::NotFound
        )
    })?;
    let _ = save_all(&tasks)?;
    Ok(to_do_item)
}
```

We now need to implement the `GetAll` and `UpdateOne` traits for our storage engines. However, the structure is going to be repetitive, and printing out the implementation of the `GetAll` and `UpdateOne` traits in the chapter will excessively bloat the chapter and derail the learning process. At this stage, you should be able to define and implement the `GetAll` and `UpdateOne` traits yourself for our storage engines. The code for the definition and implementation of the `GetAll` and `UpdateOne` traits is provided in the *Appendix* section at the end of the chapter. For guidance, the outlines of these traits take the following form:

```
pub trait GetAll {
    fn get_all() ->
    impl Future<
        Output = Result<Vec<ToDoItem>,
        NanoServiceError>
    > + Send;
}

pub trait UpdateOne {
    fn update_one(item: ToDoItem) ->
    impl Future<Output = Result<
        ToDoItem,
        NanoServiceError>
```

```
    > + Send;
  }
```

With all our transactions defined, we can now move onto connecting these transactions to our core of the nanoservice.

Connecting our transactions to the core

Now that we have defined the interface of our storage engine using traits, our core can now be agnostic to the storage engine, meaning that our core dependencies now have the following:

```
# nanoservices/to_do/core/Cargo.toml
[dependencies]
to-do-dal = { path = "../dal" }
serde = { version = "1.0.197", features = ["derive"] }
glue = { path = "../glue" }
```

We can now redefine our create endpoint with the following code:

```
// nanoservices/to_do/core/src/api/basic_actions/create.rs
use to_do_dal::to_do_items::schema::{NewToDoItem, ToDoItem};
use to_do_dal::to_do_items::transactions::create::SaveOne;
use glue::errors::NanoServiceError;

pub async fn create<T: SaveOne>(item: NewToDoItem)
    -> Result<ToDoItem, NanoServiceError> {
    let created_item = T::save_one(item).await?;
    Ok(created_item)
}
```

Here, we can see that the generic parameter declaring the `SaveOne` trait can be directly called without having to pass in any struct into the function. This is because the trait did not have any reference to `self` in the signatures of the function; therefore, we do not need the state of the struct to call the trait. Because we do not need the state of the struct, we can just declare the type of struct and use all the logic implemented. This is very powerful, because we can then mount this function to our server with the specific handle that we want. We will cover mounting our Postgres functions to our server in the next section.

If we try and build all our workspaces, then our networking workspace will fail to compile. This is because our to-do core interface has changed. Here, we are seeing more benefits of our approach to structuring our application. We can start to develop breaking changes in one workspace and continuously check that it compiles without having to change the entire system to get it compiling again. This means that we can just focus on getting our core right before moving on to the networking layer.

For now, we will complete our switch to generic references to storage engines.

For our delete interface, we have the following code:

```
// nanoservices/to_do/core/src/api/basic_actions/delete.rs
use to_do_dal::to_do_items::transactions::delete::DeleteOne;
use glue::errors::NanoServiceError;

pub async fn delete<T: DeleteOne>(id: &str)
    -> Result<(), NanoServiceError> {
    let _ = T::delete_one(id.to_string()).await?;
    Ok(())
}
```

Here, we can see that we still accept the title and return nothing, and our trait is performing the action on the storage. Our interface is getting more complicated, but not really doing much apart from calling a function. This is because a to-do application is simply storing items in a database. The to-do application was chosen to prevent excessive bloat, so we can just focus on concepts. However, our core of another more complicated application will be making calls on different storage engines, checking outcomes, performing calculations, and firing off processes such as statistics for dashboards, or emails. Remember, thanks to traits, the core is now the I/O-agnostic center where your business logic is coded. Your core would be slotted into a desktop app with frameworks such as Tauri, just a binary on a computer that uses stdio to have data piped in and out of it.

At the time of writing this, I am an Honorary Researcher in the bioengineering department at King's College London. The projects are in the surgical robotics department, and using I/O-agnostic cores is essential. For instance, one operating theater might have a terrible signal due to lead-lined walls to protect against radiation from scanning equipment. Therefore, we must interact with a cable. A lab at a central hospital might have access to GPUs; therefore, interacting with those is very beneficial. However, we also need to consider that not every lab/operating theater will have access to GPUs.

For our to-do application, having a core may seem tedious and trivial, but sticking with this approach will enable you to pivot or support multiple interfaces and contexts. You cannot predict the future. Even when building web apps, I have been halfway through building them, and a manager has said in a meeting that they have decided to go for a different database for storage. This could be down to other features being needed for the roadmap, or just the licensing/pricing. This doesn't faze me because I am strict in my structuring, so swapping out the database was not a hassle.

For our get core interface, we have the following code:

```
// nanoservices/to_do/core/src/api/basic_actions/get.rs
use to_do_dal::to_do_items::schema::AllToDoItems;
use to_do_dal::to_do_items::transactions::get::GetAll;
use glue::errors::NanoServiceError;

pub async fn get_all<T: GetAll>()
    -> Result<AllToDoItems, NanoServiceError> {
```



```

    let all_items = T::get_all().await?;
    AllToDoItems::from_vec(all_items)
}

```

Here, we can see that we have removed the single `get` interface. This is because we were not using it in our application. The general rule is that if we are not using code, we should delete it. It cleans up the code and reduces the amount of code that we are maintaining. However, like all rules, there are exceptions. Going back to my surgical robotics work, it would be short-sighted of me to delete the GPU interface if we do not use it for a particular lab that does not have access to a GPU.

Finally, we have the update. This is a good stage for you to attempt to define the update interface yourself. If you have attempted this yourself, then hopefully it follows the same approach:

```

// nanoservices/to_do/core/src/api/basic_actions/update.rs
use to_do_dal::to_do_items::schema::ToDoItem;
use glue::errors::NanoServiceError;
use to_do_dal::to_do_items::transactions::update::UpdateOne;

pub async fn update<T: UpdateOne>(item: ToDoItem)
-> Result<(), NanoServiceError> {
    let _ = T::update_one(item).await?;
    Ok(())
}

```

Note that we are passing in a `ToDoItem` instead of a `NewToDoItem` because we already have the ID.

Our core is now updated, so we can move on to connecting our transactions to the server.

Connecting our transactions to the server

We are now at the final stage of integrating our Postgres handle into our system. As we removed the reference to the type of storage from our core, making it engine-agnostic, we must now define our engine in our server dependencies with the following:

```

# nanoservices/to_do/networking/actix_server/Cargo.toml
[dependencies]
tokio = { version = "1.36.0", features = ["full"] }
actix-web = "4.5.1"
core = { path = "../..core" }
to-do-dal = {
    path = "../..dal", features = ["sqlx-postgres"]
}
glue = { path = "../..glue", features = ["actix"] }

```

Here, we can see that we have activated the `sqlx-postgres` feature; otherwise, our handle will not have implemented all the required traits to be used in our endpoints.

For our create endpoint, we first need to import the traits with the following code:

```
// nanoservices/to_do/networking/actix_server/src
// /api/basic_actions/create.rs
use to_do_dal::to_do_items::schema::NewToDoItem;
use to_do_dal::to_do_items::transactions::{
    create::SaveOne,
    get::GetAll
};
use glue::token::HeaderToken;
```

Here, we can see that we need two traits because, with every API call, we like to return the updated state to the frontend. With these two traits, we define the create function with the following code:

```
// nanoservices/to_do/networking/actix_server/src
// /api/basic_actions/create.rs
pub async fn create<T: SaveOne + GetAll>(
    token: HeaderToken,
    body: Json<NewToDoItem>
) -> Result<HttpResponse, NanoServiceError> {
    let _ = create_core::<T>(body.into_inner()).await?;
    Ok(HttpResponse::Created().json(get_all_core::<T>().await?))
}
```

This works because our Postgres handle has implemented all the traits for the storage; therefore, both traits are satisfied. Here is where we can see even more flexibility. If, for instance, we cached the state in a datastore such as Redis, and updated that cache with every transaction, then we could split up the trait requirements to `<T: SaveOne, X: GetAll>`. This means we could pass in two Postgres handles, but pass in different ones if we need. We could even go as far as to implement an HTTP request for one of the traits to call another service. As long as the signature of the trait is respected, the create endpoint will accept and utilize it.

For our delete endpoint, our updated code takes the following form:

```
// nanoservices/to_do/networking/actix_server/src
// src/api/basic_actions/delete.rs
use to_do_dal::to_do_items::transactions::{
    delete::DeleteOne,
    get::GetAll
};
...
pub async fn delete_by_name<T: DeleteOne + GetAll>(req: HttpRequest)
-> Result<HttpResponse, NanoServiceError> {
    match req.match_info().get("name") {
        Some(name) => {
```

```

        delete_core::<T>(name).await?;
    },
    None => {
        return Err(
            NanoServiceError::new(
                "Name not provided".to_string(),
                NanoServiceErrorStatus::BadRequest
            )
        )
    }
};
Ok(HttpResponse::Ok().json(get_all_core::<T>().await?))
}

```

Our get is the simplest with the following updates:

```

// nanoservices/to_do/networking/actix_server/src
// /api/basic_actions/get.rs
use to_do_dal::to_do_items::transactions::get::GetAll;
...
pub async fn get_all<T: GetAll>()
    -> Result<HttpResponse, NanoServiceError> {
    Ok(HttpResponse::Ok().json(get_all_core::<T>().await?))
}

```

Finally, we have the update. This is a good time to try and implement the update endpoint yourself. If you attempted to do this, your code should look like the following:

```

// nanoservices/to_do/networking/actix_server/src
// /api/basic_actions/update.rs
use to_do_dal::to_do_items::schema::ToDoItem;
use to_do_dal::to_do_items::transactions::{
    update::UpdateOne,
    get::GetAll
};
...
pub async fn update<T: UpdateOne + GetAll>(body: Json<ToDoItem>)
    -> Result<HttpResponse, NanoServiceError> {
    let _ = update_core::<T>(body.into_inner()).await?;
    Ok(HttpResponse::Ok().json(get_all_core::<T>().await?))
}

```

Here, we can stop and admire the beauty that this approach and the Rust programming language give us. It is verbose and succinct at the same time. Let's say two months from now, you come back to this endpoint to see what the steps are. Because Rust is verbose, and this function just focuses on the management and flow of the HTTP request and response, we can instantly see that the `ToDoItem` is the JSON body of the request. We can also see that this endpoint is performing a save one and get all operation with a storage engine. We can then see that the body is passed into a core function to update the item, and then the get all is returned. You can see that this scales easily. If you have more core functions, you will be able to easily see how the HTTP request navigates through these. It also gives you the option to easily refactor the high level, swapping out and moving core functions like Lego blocks. When you get to big systems with complex endpoints, you will thank past you for taking this approach.

We now need to mount our Postgres handler to our views factory. The update to our views factory takes the following form:

```
// nanoservices/to_do/networking/actix_server/src
// /api/basic_actions/mod.rs
. . .
use to_do_dal::to_do_items::descriptors::SqlxPostGresDescriptor;

pub fn basic_actions_factory(app: &mut ServiceConfig) {
    app.service(
        scope("/api/v1")
        .route("get/all", get().to(
            get::get_all::<SqlxPostGresDescriptor>)
        )
        .route("create", post().to(
            create::create::<SqlxPostGresDescriptor>)
        )
        .route("delete/{name}", delete().to(
            delete::delete_by_name::<SqlxPostGresDescriptor>)
        )
        .route("update", put().to(
            update::update::<SqlxPostGresDescriptor>)
        )
    );
}
```

We have now connected our server endpoints to our server with the SQLX handle. Our server is ready to talk to a database, so we must prime our database for our server with migrations.

Creating our database migrations

For Postgres, we need a table of to-do items to insert rows and perform queries. We can do this by running SQL scripts against the server on the startup of the server. These migrations can be generated using the SQLX client. To install the SQLX client, run the following command:

```
✓ cargo install sqlx-cli
```

With the SQLX client installed, we can now move to the root directory of our data access layer for our to-do nanoservice and create a `.env` file with the following content:

```
DATABASE_URL=postgres://username:password@localhost/to_do
```

When running, the SQLX client will detect the `.env` file and export the content as environment variables to connect to the database. Now that we have the environment variables to connect to the database, we can create our first migration with the following command:

```
✓ sqlx migrate add initial-setup
```

You should get a printout congratulating you on creating the migration, and you should also get the following file structure generated:

```
|— migrations
|   └─ 20240523084625_initial-setup.sql
```

The number will vary because it is a timestamp. This SQL script is the initial script for our database setup. There will be more elaborate structures to migrations where we can run different scripts to move the migration version up and down on the database, but these will come in the next chapter when we update our data models.

When we have multiple SQL scripts, the SQLX client will assume that the order to run SQL scripts is based on the number that is the timestamp. These SQL scripts will be run in ascending order. For our setup script, we must create our to-do items table with the following code:

```
-- nanoservices/to_do/dal/migrations/
-- 20240523084625_initial-setup.sql
CREATE TABLE to_do_items (
    id SERIAL PRIMARY KEY,
    title VARCHAR(255) UNIQUE NOT NULL,
    status VARCHAR(7) NOT NULL
);
```

Here, we can see that we ensure the title is unique, so we do not have multiple rows with the same task title. The database will return an error if we try and insert a duplicate. It is always good to enforce your schema rules at the database layer, because you can then ensure that the data you are inserting into the database is what you want.

We could run our migrations manually; however, this is error-prone and makes the deployment and running processes more cumbersome. Instead, we can embed the SQL files into the Rust binary and get the server to run the migrations when starting. We can achieve this by creating a migrations file in our data access layer and building a migrations function with the following code:

```
// file: nanoservices/to_do/dal/src/migrations.rs
use crate::connections::sqlx_postgres::SQLX_POSTGRES_POOL;

pub async fn run_migrations() {
    println!("Migrating to-do database...");
    let mut migrations = sqlx::migrate!("./migrations");
    migrations.ignore_missing = true;
    let result = migrations.run(&SQLX_POSTGRES_POOL)
        .await.unwrap();

    println!(
        "to-do database migrations completed: {:?}",
        result
    );
}
```

We are aggressive with the unwraps here because if we cannot connect to the database, we do not want to run the rest of the application, including the server. For our `lib.rs` file in our data access module, we should now have the following layout:

```
// file: nanoservices/to_do/dal/src/lib.rs
pub mod to_do_items;
pub mod connections;

#[cfg(feature = "sqlx-postgres")]
pub mod migrations;

#[cfg(feature = "json-file")]
pub mod json_file;
```

We are now able to run our migrations in our networking server with the following code:

```
// nanoservices/to_do/networking/actix_server/src/main.rs
use actix_web::{App, HttpServer};

mod api;
use to_do_dal::migrations::run_migrations;

#[tokio::main]
async fn main() -> std::io::Result<()> {
```

```

    run_migrations().await;
    HttpServer::new(|| {
        App::new().configure(api::views_factory)
    })
    .workers(4)
    .bind("127.0.0.1:8080")?
    .run()
    .await
}

```

We also need to apply `run_migrations().await;` to our ingress workspace before the ingress server is run because both the individual to-do nanoservice and the ingress service need to ensure that the migrations have been run before the server starts. We will do this in the next section.

All our Rust code is now done; however, you might recall that we altered the schema for updating the to-do item. We now accept an ID, so we must update the schema in the frontend code before testing our integration.

Refactoring our frontend

The create API call for our frontend will slightly change the interface, enabling the sending of the ID when updating the to-do item from pending to done. This means altering the `ToDoItem` interface and providing a `NewToDoItem` interface with the following code:

```

// file: frontend/src/interfaces/toDoItems.ts
export interface NewToDoItem {
    title: string;
    status: TaskStatus;
}

export interface ToDoItem {
    id: number;
    title: string;
    status: TaskStatus;
}

export interface ToDoItems {
    pending: ToDoItem[];
    done: ToDoItem[];
}

```

This means that our create API call now takes the following form:

```

// File: frontend/src/api/create.ts
import { NewToDoItem, ToDoItems, TaskStatus }
from "../interfaces/toDoItems";
import { postCall } from "../utils";

```

```
import { Url } from "../url";

export async function createToDoItemCall(title: string) {
  const toDoItem: NewToDoItem = {
    title: title,
    status: TaskStatus.PENDING
  };
  return postCall<NewToDoItem, ToDoItems>(
    new Url().create, toDoItem, 201
  );
}
```

And our update call now needs to pass in the ID with the following code:

```
// File: frontend/src/api/update.ts
import { ToDoItem, ToDoItems, TaskStatus }
from "../interfaces/toDoItems";
import { putCall } from "../utils";
import { Url } from "../url";

export async function updateToDoItemCall(
  name: string, status: TaskStatus, id: number
) {
  const toDoItem: ToDoItem = {
    title: name,
    status: status,
    id: id
  };
  return putCall<ToDoItem, ToDoItems>(
    new Url().update, toDoItem, 200
  );
}
```

And that's it for our frontend refactor. Remember, our `ToDoItems` interface references the `ToDoItem` interface, so everything else is automatically updated.

We must also include the ID in the update API function call with the following code:

```
// File: frontend/src/components/ToDoItem.tsx
...
const sendRequest = async () => {
  if (buttonMessage === "edit") {
    await updateToDoItemCall(
      title,
```



```

        TaskStatus.DONE,
        id
    ).then(. . .)
} else {
    . . .
}
};

```

Finally, we must get the ingress running the migrations of the Postgres database for the to-do nanoservice. We must declare our data access layer in the ingress with the following dependency:

```

# ingress/Cargo.toml
[dependencies]
. . .
to-do-dal = {
    path = "../nanoservices/to_do/dal",
    package = "dal",
    features = ["sqlx-postgres"]
}

```

Here, we can see that we point to the real package name, which is `dal`, but we might have other packages called `dal` from other nanoservices, so we call the dependency `to-do-dal`, so when we reference the `dal` in our code, we use the crate name `to-do-dal`. This will prevent clashes in the future.

We can now move on to importing the `dal` in our `main.rs` file with the following code:

```

// ingress/src/main.rs
. . .
use to_do_dal::migrations::run_migrations as run_todo_migrations;
. . .

```

And then we run our migrations in the `main` function with the following code:

```

// ingress/src/main.rs
. . .
#[tokio::main]
async fn main() -> std::io::Result<()> {
    run_todo_migrations().await;
    HttpServer::new(|| {
        . . .
    })
    .bind("0.0.0.0:8001")?
    .run()
    .await
}

```

We also need an `.env` file in the root of our ingress workspace with the following contents:

```
// ingress/.env
TO_DO_DB_URL=postgres://username:password@localhost/to_do
```

And our run server Bash script needs to export these environment variables, resulting in the following code:

```
# ingress/scripts/run_server.sh
#!/usr/bin/env bash

# navigate to directory
SCRIPTPATH="$( cd "$(dirname "$0")" ; pwd -P )"
cd $SCRIPTPATH

cd ../../
cd frontend
npm install
npm run build
cd ../ingress
cargo clean
export $(cat .env | xargs)
cargo run
```

And there we have it! We have integrated Postgres into our app. You can run it, and if your database is running, the migrations will run, and you can interact with your application in the browser.

Summary

In this chapter, we performed a massive refactor. We swapped out an entire storage engine and got to grips with Docker. Although there were a lot of moving parts, due to the structure of the code, the refactor was clean, and the interfaces enabled us to make major changes in the data access layer, and then minimal changes throughout the system that reference the data access layer. Furthermore, because we utilized traits, we have made it easier to implement other storage engines and swap them out when needed. On top of this, we embedded the SQL migrations into the Rust binary with the SQLX crate. So, we now have a singular Rust binary that runs migration scripts on the database, serves the frontend JavaScript app, and serves backend API endpoints to the frontend application. We are nearly at a full-functioning web application that can be easily deployed. In the next chapter, we will finish up our application by handling user sessions.

Questions

1. What are the advantages of having a database over a JSON file?
2. How do you create a migration?
3. How can we design our application to handle swapping out different database handles?

4. If we wanted to create a user data model in Rust with a name and age, what should we do?
5. What is a connection pool, and why should we use it?
6. What is SQL injection?

Answers

1. A database has advantages in terms of multiple reads and writes at the same time. A database also checks the data to see whether it is in the right format before inserting it, and we can do advanced queries with linked tables.
2. We install the SQLX client and define the database URL in the `.env` file. We then create migrations using the client and write the desired schema required for the migration. We then build a function that runs the migrations on the database when called. This function can fire just before the server runs.
3. We exploit traits. First, we define a trait that has the function we need for the database operation. We then implement the trait for the storage engine we need. We then have a generic parameter for a function that is the API endpoint. This generic parameter must have the trait for the data transaction implemented. This gives us the power to then swap out to whatever handle we have implemented for the data transaction trait.
4. We define a `NewUser` struct with just the name as a string and age as an integer. We then create a `User` struct with the same fields and an extra integer field, which is the ID.
5. A connection pool pools a limited number of connections that connect to the database. Our application then passes these connections to the threads that need them. This keeps the number of connections to the database limited to avoid the database being overloaded.
6. SQL injection is when we insert a SQL query into a parameter of another SQL query. This enables us to run malicious queries against the database. For instance, if a name parameter was not sanitized and just put into the SQL query, we could enter another SQL query as the name, and that query would get run on the database.

Appendix

The following is the code for implementing database transactions for getting to-do items and updating those to-do items. These were skipped in this chapter to avoid bloat, as the delete and create database transactions followed the same structure with slightly different SQL, inputs, and outputs:

```
// file: nanoservices/to_do/dal/src/to_do_items/  
// transactions/get.rs  
use crate::to_do_items::schema::ToDoItem;  
use glue::errors::NanoServiceError;  
use std::future::Future;  
  
#[cfg(feature = "json-file")]  
use super::super::descriptors::JsonFileDescriptor;  
#[cfg(feature = "json-file")]
```

```

use crate::json_file::get_all;
#[cfg(feature = "json-file")]
use std::collections::HashMap;

#[cfg(feature = "sqlx-postgres")]
use crate::connections::sqlx_postgres::SQLX_POSTGRES_POOL;
#[cfg(feature = "sqlx-postgres")]
use super::super::descriptors::SqlxPostGresDescriptor;
#[cfg(feature = "sqlx-postgres")]
use glue::errors::NanoServiceErrorStatus;

pub trait GetAll {
    fn get_all() ->
    impl Future<Output = Result<Vec<ToDoItem>, NanoServiceError>> + Send;
}

#[cfg(feature = "sqlx-postgres")]
impl GetAll for SqlxPostGresDescriptor {
    fn get_all() ->
    impl Future<Output = Result<Vec<ToDoItem>, NanoServiceError>> + Send {
        sqlx_postgres_get_all()
    }
}

#[cfg(feature = "json-file")]
impl GetAll for JsonFileDescriptor {
    fn get_all() ->
    impl Future<Output = Result<Vec<ToDoItem>, NanoServiceError>> + Send {
        json_file_get_all()
    }
}

#[cfg(feature = "sqlx-postgres")]
async fn sqlx_postgres_get_all() ->
    Result<Vec<ToDoItem>, NanoServiceError> {
    let items = sqlx::query_as::<_, ToDoItem>("
        SELECT * FROM to_do_items"
    ).fetch_all(&*SQLX_POSTGRES_POOL).await.map_err(|e| {
        NanoServiceError::new(
            e.to_string(),
            NanoServiceErrorStatus::Unknown
        )
    })?;
    Ok(items)
}

```

```

}
#[cfg(feature = "json-file")]
async fn json_file_get_all() ->
    Result<Vec<ToDoItem>, NanoServiceError> {
    let tasks = get_all:::<ToDoItem>().unwrap_or_else(|_|
        HashMap::new()
    );
    let items = tasks.values().cloned().collect();
    Ok(items)
}

```

```

// file: nanoservices/to_do/dal/src/to_do_items/
// transactions/update.rs
use crate::to_do_items::schema::ToDoItem;
use glue::errors::NanoServiceError;
use std::future::Future;

#[cfg(feature = "json-file")]
use super::super::descriptors::JsonFileDescriptor;
#[cfg(feature = "json-file")]
use crate::json_file::{get_all, save_all};
#[cfg(feature = "json-file")]
use std::collections::HashMap;

#[cfg(feature = "sqlx-postgres")]
use crate::connections::sqlx_postgres::SQLX_POSTGRES_POOL;
#[cfg(feature = "sqlx-postgres")]
use super::super::descriptors::SqlxPostGresDescriptor;

#[cfg(any(feature = "json-file", feature = "sqlx-postgres"))]
use glue::errors::NanoServiceErrorStatus;

pub trait UpdateOne {
    fn update_one(item: ToDoItem) ->
        impl Future<Output = Result<ToDoItem, NanoServiceError>> + Send;
}

#[cfg(feature = "sqlx-postgres")]
impl UpdateOne for SqlxPostGresDescriptor {
    fn update_one(item: ToDoItem) ->
        impl Future<Output = Result<ToDoItem, NanoServiceError>> + Send {

```

```

        sqlx_postgres_update_one(item)
    }
}
#[cfg(feature = "json-file")]
impl UpdateOne for JsonFileDescriptor {
    fn update_one(item: TodoItem) ->
    impl Future<Output = Result<TodoItem, NanoServiceError>> + Send {
        json_file_update_one(item)
    }
}
#[cfg(feature = "sqlx-postgres")]
async fn sqlx_postgres_update_one(item: TodoItem) ->
    Result<TodoItem, NanoServiceError> {
    let item = sqlx::query_as::<_, TodoItem>("
        UPDATE to_do_items
        SET title = $1, status = $2
        WHERE id = $3
        RETURNING *"
    ).bind(item.title)
    .bind(item.status.to_string())
    .bind(item.id)
    .fetch_one(&*SQLX_POSTGRES_POOL).await.map_err(|e| {
        NanoServiceError::new(
            e.to_string(),
            NanoServiceErrorStatus::Unknown
        )
    })?;
    Ok(item)
}
#[cfg(feature = "json-file")]
async fn json_file_update_one(item: TodoItem) ->
    Result<TodoItem, NanoServiceError> {
    let mut tasks = get_all::<TodoItem>().unwrap_or_else(|_|
        HashMap::new()
    );
    if !tasks.contains_key(&item.title) {
        return Err(NanoServiceError::new(
            format!("Item with name {} not found", item.title),
            NanoServiceErrorStatus::NotFound
        ));
    }
}

```

```
tasks.insert(item.title.clone(), item.clone());  
let _ = save_all(&tasks?);  
Ok(item)  
}
```

Get this book's PDF copy, code bundle, and more

Scan the QR code (or go to packtpub.com/unlock). Search for this book by name, confirm the edition, and then follow the steps on the page.



UNLOCK NOW

Note: Have your invoice handy. Purchases made directly from the Packt website don't require an invoice.

10

Managing User Sessions

We now have our to-do server working with a React frontend and Postgres database. However, there is no authentication. Anybody can access the application and alter the to-do list. As we know, the internet does not work like this, as websites authenticate their users before allowing them to perform operations. We must authenticate our users before we can allow them to alter to-do items. In this chapter, we are going to build an authentication server and integrate it into our system so we can authenticate our users before allowing them to access the to-do items. In this chapter, we will cover the following:

- Building our auth server
- Defining our user data model
- Storing passwords
- Verifying passwords
- Creating users
- Refactoring our JWT
- Building our login API
- Interacting with our auth server
- Adding authentication to our frontend

By the end of this chapter, you will be able to integrate another server into your system and have it compiled into the same binary as the rest of the system. You will also understand the basics of authentication so you can handle user accounts and allow users to log in so these users can make authenticated calls to the rest of the application.

Technical requirements

You can download the example project and code for this book by following the instructions in the **Download the example code files** section in the *Preface* of this book.

This chapter's code files are included in the downloadable code bundle.

The last iteration of the system in *Chapter 9* was the refactoring of our frontend.

Building our auth server

For our authentication server, we must perform the following operations:

1. Create users.
2. Log in.
3. Log out.

Authentication servers also handle other processes, such as deleting users, blocking them, and allocating roles. However, there comes a point where implementing extra authentication features will bloat the chapter with diminishing educational returns. By the end of this chapter, you will be able to understand the processes to add extra functions.

Our authentication server should be housed in the `nanoservices/auth` directory. The authentication server has the following file structure:

```
├─ core
│   └─ . . .
├─ dal
│   └─ . . .
└─ networking
    └─ actix_server
        └─ . . .
```

We will investigate each section in more detail. There is a lot of repetition compared to the to-do server when it comes to the generic boilerplate of making database connections and running the server. If you wanted to, you could copy the to-do nanoservice directory and strip out the files that are specific to the to-do server.

When it comes to duplication of code, there is always a trade-off. We can all appreciate that excessive duplication of code is not desirable. However, duplication also facilitates decoupling. Microservices, as a general concept, can be criticized as being excessive when you only have a few features. However, in my experience, these servers grow aggressively as the project progresses. Keeping the servers isolated enables us to break the servers out into their own repositories. Teams could work in their own assigned servers with a lot of freedom without having to coordinate with other teams if they wanted to do something such as alter the way in which a database connection worked, or how a server listened to incoming requests. We could put the server and connection code into the glue module. There does not have to be consistency across the entire system. At this point, it is down to personal preference. Some people hate duplicating code at any point. However, in my personal experience, I find that as the system evolves, so does our problem and business needs. Being able to keep servers isolated so they can be swapped out or deployed individually is worth the duplicated code. For instance, in past projects, I have depreciated individual services. Because they were isolated, we could keep them running without hindering the progression of other services until the replacement service was built, tested, and deployed.

To avoid duplication, you could go for the modular monolith approach. This is where modules are developed individually and then just slotted into one server with one data access layer. I have used this approach on some smaller projects, and it does work well. However, the aim of this book is to teach you Rust and web development. Orchestrating microservices is harder than building a modular monolith. So, if we spend our time in this book learning how to build and orchestrate microservices, you would not walk away from it knowing how to build a modular monolith. Our ingress is essentially a modular monolith, and we can burst it out into microservices if we want to.

For our auth service modules, we will start with the data access layer.

Building our data access layer

Our data access layer has the following directory outline:

```
dal
├── Cargo.toml
├── migrations
│   └── 20240523088625_initial-setup.sql
├── src
│   ├── connections
│   │   ├── mod.rs
│   │   └── sqlx_postgres.rs
│   ├── lib.rs
│   ├── migrations.rs
│   └── users
│       ├── descriptors.rs
│       ├── mod.rs
│       ├── schema.rs
│       └── transactions
│           ├── create.rs
│           ├── get.rs
│           └── mod.rs
```

To avoid bloating the chapter, we will not cover every line of boilerplate code. First off, ensure that all the files are linked to the `lib.rs` file via the `mod` files. Our schema file is going to be empty at this point as we will build the user schema throughout the chapter. The same goes for the SQL migrations and transactions directory as these are all dependent on the user data model. You may have noticed that we are not copying over our JSON file storage module. The JSON file storage option was never really a viable storage solution. It got us up and running quickly without having to set up a database so we could see our code working. However, we are now introducing users, and we are going to link those users to to-do items. This is going to cause a lot of complications if we use the JSON file approach. Working out hacky ways to link users for a storage system that doesn't support concurrent operations, schema checking, or filter operations is beyond the scope of the chapter.

When it comes to our SQLx Postgres connection in the `src/connections/sqlx_postgres.rs` file, the code is the same as our to-do service database connection. The only difference is the environment variables where the connection URL is `AUTH_DB_URL`, and the max connections is `AUTH_MAX_CONNECTIONS`. Having a different environment variable for the connection information gives us the flexibility of having both of our servers pointing to the same database or their own separate databases. As our auth migrations will not clash with the to-do migrations, the database will be able to run them both without any problems. For the `src/migrations.rs` file, the code is the same, but with reference to auth as opposed to to-do in the print statements.

For our `src/users/descriptors.rs` file, we will only have the `SqlxPostGresDescriptor` struct.

Finally, our `Cargo.toml` file has the following contents:

```
# File: nanoservices/auth/dal/Cargo.toml
[package]
name = "auth-dal"
version = "0.1.0"
edition = "2021"

[dependencies]
serde = { version = "1.0.197", features = ["derive"] }
glue = { path = "../glue" }
sqlx = {
    version = "0.8.3",
    features = ["postgres", "json", "runtime-tokio"],
    optional = false
}
```

We can see that we called our package `auth-dal`. While other Cargo files can handle aliases to avoid clashes, workspaces cannot handle such clashes at the time of writing this book. We can also see that we have gotten rid of the features as we are just supporting Postgres here, and as a result, none of our dependencies are optional.

With this, our data access layer is ready to be plugged in and developed. Seeing as we have linked a lot of files in one go, I would usually perform a quick cargo build at this point just to check that everything has been done correctly. Now that our data access layer is done, we can move on to the core of the auth server.

Building our auth core module

Our core has the following file structure:

```
core
├── Cargo.toml
│   └── src
│       └── api
```

```

|   ├── auth
|   |   ├── login.rs
|   |   ├── logout.rs
|   |   └── mod.rs
|   ├── mod.rs
|   └── users
|       ├── create.rs
|       └── mod.rs
└── lib.rs

```

Remember, our core does not have dependencies such as databases or networking, so there is no boilerplate code. Here, we just need to ensure that all the files are linked, and that the `Cargo.toml` file has the following contents:

```

# File: nanoservices/auth/core/Cargo.toml

[package]
name = "auth-core"
version = "0.1.0"
edition = "2021"

[dependencies]
auth-dal = { path = "../dal" }
serde = { version = "1.0.197", features = ["derive"] }
glue = { path = "../../glue" }

```

Finally, we move on to our networking layer.

Networking layer

Our networking layer has the following file structure:

```

networking
└── actix_server
    ├── Cargo.toml
    └── src
        ├── api
        |   ├── auth
        |   |   ├── login.rs
        |   |   ├── logout.rs
        |   |   └── mod.rs
        |   ├── mod.rs
        |   └── users
        |       ├── create.rs
        |       └── mod.rs
        └── main.rs

```

Again, we link all the files together and define the following API endpoint factories. For the users factory, we have the following definition:

```
// nanoservices/auth/networking/actix_server/
// src/api/users/mod.rs
pub mod create;
use actix_web::web::ServiceConfig;

pub fn users_factory(app: &mut ServiceConfig) {
}
```

For the auth factory, we have the following code:

```
// nanoservices/auth/networking/actix_server/
// src/api/auth/mod.rs
pub mod login;
pub mod logout;
use actix_web::web::ServiceConfig;

pub fn auth_factory(app: &mut ServiceConfig) {
}
```

Finally, our main API factory is defined:

```
// nanoservices/auth/networking/actix_server/
// src/api/mod.rs
pub mod auth;
pub mod users;
use actix_web::web::ServiceConfig;

pub fn views_factory(app: &mut ServiceConfig) {
    users::users_factory(app);
    auth::auth_factory(app);
}
```

With our endpoint factories defined, our main file takes the following form:

```
// nanoservices/auth/networking/actix_server/
// src/main.rs
use actix_web::{App, HttpServer};
mod api;
use auth_dal::migrations::run_migrations;

#[tokio::main]
async fn main() -> std::io::Result<()> {
```

```

    run_migrations().await;
    HttpServer::new(|| {
        App::new().configure(api::views_factory)
    })
    .workers(4)
    .bind("127.0.0.1:8081")?
    .run()
    .await
}

```

Here, we can see that we have increased the port number by 1 because we might want both of our servers to run and we do not want the ports to clash.

Finally, our `Cargo.toml` file has the following dependencies:

```

# nanoservices/auth/networking/actix_server/Cargo.toml
[package]
name = "auth_actix_server"
version = "0.1.0"
edition = "2021"

[dependencies]
tokio = { version = "1.36.0", features = ["full"] }
actix-web = "4.5.1"
auth-core = { path = "../..core" }
auth-dal = { path = "../..dal" }
glue = { path = "../..../glue", features = ["actix"] }

```

And here we have it, our auth server is now ready to develop on. For Axum, Hyper, and Rocket, the patterns are the same as what we covered in previous chapters. If you want to reference them, you can see the examples on GitHub.

We can now start building out the user data model.

Defining our user data model

When it comes to defining a user data model that will enable us to insert and get users from our database, we take the same approach that we did when creating our to-do item data model. This means we create a new user struct without an ID, and a user struct that has an ID due to it now being inserted in the database.

With this approach in mind, we can define the new user struct with the following code:

```

// nanoservices/auth/dal/src/users/schema.rs
use serde::{Serialize, Deserialize};

```

```
#[derive(Serialize, Deserialize, Debug, Clone)]
pub struct NewUser {
    pub email: String,
    pub password: String,
    pub unique_id: String
}
```

We can see that we have a unique ID as well as a standard ID. Unique IDs become useful when we need to expose an ID to the outside world. For instance, if we wanted to make our users verify that the email address they provided is real, we could send an email with a link containing the unique ID for the user to click on. Once the user has clicked on the link, we could match the unique ID in the link to the unique ID of the user and then verify the user. Once the user is verified, the unique ID should be changed. We will use the unique ID in our token-based login sessions. Putting the unique ID into the login token ensures that if someone else gets hold of the token, the information in the token will only be valid for a short time as we can change the unique ID when refreshing the token.

Now that our new user struct is created, we can define our user struct that can be queried from the database as opposed to be inserted into the database with the following code:

```
// nanoservices/auth/dal/src/users/schema.rs
#[derive(Deserialize, Serialize, Debug,
    Clone, PartialEq, sqlx::FromRow)]
pub struct User {
    pub id: i32,
    pub email: String,
    pub password: String,
    pub unique_id: String
}
```

However, there is an issue here. We have implemented the `Serialize` and `Deserialize` traits for the `User` struct, but we do not want to expose the password processed outside of the auth server. To remedy this, we also have a `TrimmedUser` struct, which does not have a password and can also be constructed from the `User` struct, using the following code:

```
// nanoservices/auth/dal/src/users/schema.rs
#[derive(Deserialize, Serialize, Debug,
    Clone, PartialEq)]
pub struct TrimmedUser {
    pub id: i32,
    pub email: String,
    pub unique_id: String
}

impl From<User> for TrimmedUser {
    fn from(user: User) -> Self {
        TrimmedUser {
```



```

        id: user.id,
        email: user.email,
        unique_id: user.unique_id
    }
}
}

```

With our structs, we just need to define the SQL script for the database migrations with the following code:

```

-- nanoservices/auth/dal/migrations/*_initial_setup.sql
CREATE TABLE users (
    id SERIAL PRIMARY KEY,
    email VARCHAR NOT NULL UNIQUE,
    password VARCHAR NOT NULL,
    unique_id VARCHAR NOT NULL UNIQUE
);

```

Here, we can see that we ensure that the email is unique. This is because this email is the main way we communicate with the user. For instance, if the user wants to reset their password, we cannot have the reset link sent to two or more email addresses. Having a unique ID is important as we would want to use it for verification, password resets, and identifying login sessions.

As our migrations folder is embedded into the Rust binary and runs when the server starts up, we can move on to writing code that interacts with users in the database. However, before we write any endpoints for the auth server, we must write some functionality around verifying passwords and storing these passwords.

Storing passwords

When storing passwords, we cannot store these passwords in a raw string format. If we did and there were a data leak, all the passwords would be compromised. To reduce the risk of passwords being compromised, we hash the passwords before storing them in the database. We also must create unique IDs when creating users to be inserted into the database.

To enable us to write the code that stores the passwords of new users, we must add the following dependencies:

```

# nanoservices/auth/dal/Cargo.toml
argon2 = { version = "0.5.3", features = ["password-hash"] }
uuid = { version = "1.8.0", features = ["serde", "v4"] }
rand = "0.8.5"

```

The uses of the crates are described here:

- `argon2` is used for password hashing and verification
- `uuid` is for the creation of unique IDs

- `rand` is a random generation functionality used for creating a “salt string” for hashing the password

With these dependencies, we now need to use the following so we can build our password hashing code:

```
// nanoservices/auth/dal/src/users/schema.rs
use glue::errors::{NanoServiceError, NanoServiceErrorStatus};
use argon2::{
    Argon2,
    PasswordHasher,
    PasswordVerifier,
    password_hash::{
        SaltString,
        PasswordHash
    }
};
```

You might be wondering why we are building the password hashing logic in our `schema.rs` in the data access layer. Should we not build out our password hashing in the core of the `auth`? This question makes sense as handling passwords is an authentication issue. Like most design decisions, there is no clear right answer. I have chosen to put the hashing password logic in `schema.rs` because we are hashing the passwords for their safe storage. In my own work, if there is a requirement for storage, then I usually put it in the storage module. Another developer can look at the schema of the users and see not only the struct but also the processes needed to store the struct. It can be messy to rely on hashing dependencies elsewhere to safely store the user struct.

To ensure that the `NewUser` password is hashed, we can hash the password in the `NewUser` constructor, which has the following outline:

```
// nanoservices/auth/dal/src/users/schema.rs
impl NewUser {

    pub fn new(email: String, password: String)
        -> Result<NewUser, NanoServiceError> {
        . . .
    }
}
```

Inside our constructor, we create a unique ID and hashed password with the following code:

```
// nanoservices/auth/dal/src/users/schema.rs
// Fn => NewUser::new
let unique_id = uuid::Uuid::new_v4().to_string();
let salt = SaltString::generate(&mut rand::thread_rng());
let argon2_hasher = Argon2::default();
```

```
let hash = argon2_hasher.hash_password(
  password.as_bytes(),
  &salt
).map_err(|e|{
  NanoServiceError::new(
    format!("Failed to hash password: {}", e),
    NanoServiceErrorStatus::Unknown
  )
})?.to_string();
Ok(NewUser {
  email,
  password: hash,
  unique_id
})
```

We can see that we salt and hash the password. Before we can explain salting, we must explain what hashing is.

Hashing is the process of converting a given input (in this case, a password) into a fixed-size string of characters, which typically appears as a seemingly random sequence of characters. This transformation is performed by a hash function. The key properties of a cryptographic hash function are as follows:

- **Deterministic:** The same input always produces the same hash output
- **Irreversible:** It should be computationally infeasible to reverse the hash to obtain the original input
- **Unique:** Even small changes to the input should produce significantly different hash outputs
- **Collision-resistant:** It should be highly unlikely for two different inputs to produce the same hash output

In the provided code, `Argon2::default()` represents the default configuration of the Argon2 hashing algorithm, which is a widely used and secure hash function designed specifically for hashing passwords.

Salting involves adding a unique, random value (called a salt) to the password before hashing it. The primary purposes of salting are as follows:

- **Uniqueness:** Ensures that even if two users have the same password, their hashes will be different because each hash will include a unique salt
- **Prevents pre-computation attacks:** Salts prevent the use of precomputed tables (rainbow tables) for reversing hash values, as the same password will have different hashes when different salts are used

We combine salting and hashing for the following reasons:

- **Protection against hash table attacks:** Without salts, an attacker could use precomputed hash tables to quickly look up and reverse hash values back into their original passwords

- **Protection against brute-force attacks:** Hashing algorithms such as Argon2 are designed to be computationally expensive, making brute-force attacks (trying all possible passwords) slower and more difficult
- **Password uniqueness:** Even if two users choose the same password, the use of a unique salt ensures that the resulting hashes are different, adding another layer of security

We have now handled the storing of passwords, but we cannot verify whether the password passed in by the user when logging in is correct yet. In the next section, we verify password inputs.

Verifying passwords

To verify passwords, we must get the hashed password from the database, hash the password passed in by the user for the login, and then compare them. The nuance around verifying passwords is beyond the scope of this book as cryptography is an entire field in itself. For our verify process, although verifying passwords does not have any direct relation to storing data, I am going to put the verifying password logic in the `schema.rs` in our data access layer. This is where I choose to break my rule around putting logic in a defined context. It makes sense for the verifying of the password to be in the core; however, the hashing of the password to be stored is already in the `schema.rs` in our data access layer. A developer looking at our `schema.rs` in our data access layer will quickly see how the password is hashed for storage and how the password is verified. If there were a lot more complexity around verifying a password to the point that it needed an entire module, then it would be worth putting it into the core and making other developers work a little harder to mentally map the domain of passwords.

Our password verification code takes the following form:

```
// nanoservices/auth/dal/src/users/schema.rs
impl User {
    pub fn verify_password(&self, password: String)
        -> Result<bool, NanoServiceError> {
        . . .
    }
}
```

Inside our `verify_password` function, we verify the password with the following code:

```
// nanoservices/auth/dal/src/users/schema.rs
let argon2_hasher = Argon2::default();
let parsed_hash = PasswordHash::new(
    &self.password
).map_err(|e|{
    NanoServiceError::new(
        format!("Failed to parse password hash: {}", e),
        NanoServiceErrorStatus::Unknown
    )
})?;
let is_valid = argon2_hasher.verify_password(
```

```
password.as_bytes(),
    &parsed_hash
).is_ok();
Ok(is_valid)
```

Here, we can see that we pass our stored password into a `PasswordHash` struct. The passing into the `PasswordHash` struct can result in an error because the password that we are passing in is not hashed in the right format. For instance, the start of the hashed string should be the name of the algorithm used to generate the hash. Once we have the parsed hash, we then check to see whether the password passed into the function is valid. If the password does not match, then we return `true`; otherwise, it is `false`. As we have chucked a lot of logic at our `User` struct, it is a good idea to run another cargo build to ensure that everything is building correctly.

We now have everything we need to create our users and log them in.

Creating users

If we want to enable the creation of users, we must follow the same steps that we took when creating to-do items. These steps are the following:

1. Defining an insert user database transaction in the data access layer.
2. Adding a create user API endpoint to our core.
3. Adding a create user API endpoint to our networking layer.

As there is some repetition here, this is a good place for you to practice your understanding of our system by carrying out the steps yourself. If you do attempt the steps yourself, hopefully your code will resemble the approach that we cover.

Defining our create user database transactions

For our database transaction of inserting the user into the database, we need the following imports:

```
// nanoservices/auth/dal/src/users/transactions/create.rs
use crate::users::schema::{NewUser, User};
use glue::errors::{NanoServiceError, NanoServiceErrorStatus};
use std::future::Future;
use crate::connections::sqlx_postgres::SQLX_POSTGRES_POOL;
use super::super::descriptors::SqlxPostGresDescriptor;
```

We then must define our trait, which is the template for inserting a user, with the following code:

```
// nanoservices/auth/dal/src/users/transactions/create.rs
pub trait SaveOne {
    fn save_one(user: NewUser)
    -> impl Future<Output = Result<
        User, NanoServiceError>
    > + Send;
}
```

We then implement the trait with a placeholder function for our Postgres database descriptor with the following code:

```
// nanoservices/auth/dal/src/users/transactions/create.rs
impl SaveOne for SqlxPostgresDescriptor {
    fn save_one(user: NewUser)
    -> impl Future<Output = Result<
        User, NanoServiceError>
    > + Send {
        sqlx_postgres_save_one(user)
    }
}
```

Finally, we define our function that interacts with the database with the following code:

```
// nanoservices/auth/dal/src/users/transactions/create.rs
async fn sqlx_postgres_save_one(user: NewUser)
-> Result<User, NanoServiceError> {
    let user = sqlx::query_as::<_, User>("
        INSERT INTO users (email, password, unique_id)
        VALUES ($1, $2, $3)
        RETURNING *"
    )
    .bind(user.email)
    .bind(user.password.to_string())
    .bind(user.unique_id)
    .fetch_one(&*SQLX_POSTGRES_POOL).await.map_err(|e| {
        NanoServiceError::new(
            e.to_string(),
            NanoServiceErrorStatus::Unknown
        )
    })?;
    Ok(user)
}
```

None of the previous code should be new. Here, we are changing the type of data we are inserting and the name of the table we are inserting into.

We can now move on to defining our create endpoint in our core.

Defining our core create API endpoint

For our create endpoint in our core, we just need the following code:

```
// nanoservices/auth/core/src/api/users/create.rs
```

```

use auth_dal::users::schema::{NewUser, User};
use auth_dal::users::transactions::create::SaveOne;
use glue::errors::NanoServiceError;
use serde::Deserialize;

#[derive(Deserialize)]
pub struct CreateUser {
    pub email: String,
    pub password: String
}

pub async fn create<T: SaveOne>(data: CreateUser)
    -> Result<User, NanoServiceError> {
    let user = NewUser::new(data.email, data.password)?;
    let created_item = T::save_one(user).await?;
    Ok(created_item)
}

```

We can now move on to defining our networking API endpoint.

Defining our networking create API endpoint

The create endpoint for our networking layer takes the following form:

```

// nanoservices/auth/networking/actix_server/
// src/api/users/create.rs
use auth_dal::users::transactions::create::SaveOne;
use auth_core::api::users::create::{
    create as create_core,
    CreateUser
};
use auth_dal::users::schema::NewUser;
use glue::errors::NanoServiceError;
use actix_web::{
    HttpResponse,
    web::Json
};

pub async fn create<T: SaveOne>(body: Json<CreateUser>)
    -> Result<HttpResponse, NanoServiceError> {
    let _ = create_core::<T>(body.into_inner()).await?;
    Ok(HttpResponse::Created().finish())
}

```

Our create endpoint function is mounted to our server with the following code:

```
// nanoservices/auth/networking/actix_server/
// src/api/users/mod.rs
pub mod create;
use auth_dal::users::descriptors::SqlxPostGresDescriptor;
use actix_web::web::{ServiceConfig, scope, post};

pub fn users_factory(app: &mut ServiceConfig) {
    app.service(
        scope("/api/v1/users")
        .route("create", post().to(
            create::create::<SqlxPostGresDescriptor>
        ))
    );
}
```

Now our users are ready to be created. If you are interested in how these endpoints are handled in Hyper, Axum, or Rocket, examples can be found in the GitHub repository. The only difference from the previous example of creating a to-do item is that we do not return a JSON body. We will want to build out the login and logout API endpoints for our auth server, but before we do that, we must refactor our header token to encode our user ID into the token.

Refactoring our JWT

Right now, the token we extract from the header of a request is just a string. In this section, we are going to encode and decode user credentials into a **JSON Web Token (JWT)** so our user can log in once and then make multiple authenticated API calls to our backend with the token. Because the token has a unique ID associated with the user, we know not only that the user is authenticated but also which user is making the requests. To make these changes, we must carry out the following steps:

1. Restructuring our JWT for a unique ID.
2. Creating a get key function for encoding.
3. Creating an encode function to encode user credentials.
4. Creating a decode function to extract user credentials.

Before we carry out any of these steps, however, we must add the following dependency to our glue workspace:

```
# glue/Cargo.toml
jsonwebtoken = "9.3.1"
```


We then import the following:

```
// glue/src/token.rs
use serde::{Serialize, Deserialize};
use crate::errors{
    NanoServiceError, NanoServiceErrorStatus
};
use jwt_token::jwt_token::JwtToken {
    decode,
    encode,
    Algorithm,
    DecodingKey,
    EncodingKey,
    Header,
    Validation
};
```

With these imports, we can now refactor our JWT.

Restructuring our JWT for a unique ID

Our JWT now needs to house the unique ID of the user, have encoding/decoding functions, and update the FromRequest trait implementation. We also need to change the message field to unique_id for our token because we have a unique ID for our auth sessions:

```
// glue/src/token.rs
#[derive(Debug, Serialize, Deserialize)]
pub struct HeaderToken {
    pub unique_id: String
}
impl HeaderToken {
    pub fn get_key() -> Result<String, NanoServiceError> {
        . . .
    }
    pub fn encode(self) -> Result<String, NanoServiceError> {
        . . .
    }
    pub fn decode(token: &str) -> Result<Self, NanoServiceError> {
        . . .
    }
}
```

We will cover the code inside those encoding/decoding functions in the following subsections. Finally, we recall that the outline for the `FromRequest` trait (denoted as `ActixFromRequest`) implementation inside our `actix_impl` module takes the following form:

```
// glue/src/token.rs
impl ActixFromRequest for HeaderToken {
    type Error = NanoServiceError;
    type Future = Ready<Result<HeaderToken, NanoServiceError>>;

    fn from_request(req: &HttpRequest, _: &mut Payload)
    -> Self::Future {
        . . .
    }
}
```

After we have converted the token we extracted from the HTTP header into a string, we decode the token, constructing the `HeaderToken` struct in the process with the following code:

```
// glue/src/token.rs
let raw_data = match req.headers().get("token") {
    Some(data) => data.to_str()
                        .expect("convert token to str"),
    None => {
        return err(NanoServiceError {
            status: NanoServiceErrorStatus::Unauthorized,
            message: "token not in header under key 'token'"
                        .to_string()
        })
    }
};
let token = match HeaderToken::decode(raw_data) {
    Ok(token) => token,
    Err(_) => {
        return err(NanoServiceError {
            status: NanoServiceErrorStatus::Unauthorized,
            message: "token not a valid string".to_string()
        })
    }
};
ok(token)
```

It must be noted that there is an `expect` statement when converting the header value into a string. This can be down to personal preference, but some experienced developers like to use the `expect` statement when they want to ensure that the operation does not fail. Considering that the HTTP protocol is a text-based protocol, it would be very strange if we could not convert a header value into a string. Now that our token is being properly extracted and decoded, we can move on to the functions that enable the encoding/decoding of the token.

Creating a get key function for encoding

When it comes to encoding and decoding our token, we want to ensure that processes outside of the server cannot access details of the user of the token. We also do not want the frontend to have the opportunity to alter the token. To prevent altering or accessing details in the token, the server uses a secret key to encode and decode the token, ensuring that the token is always encoded outside of the server, as depicted in *Figure 10.1*:

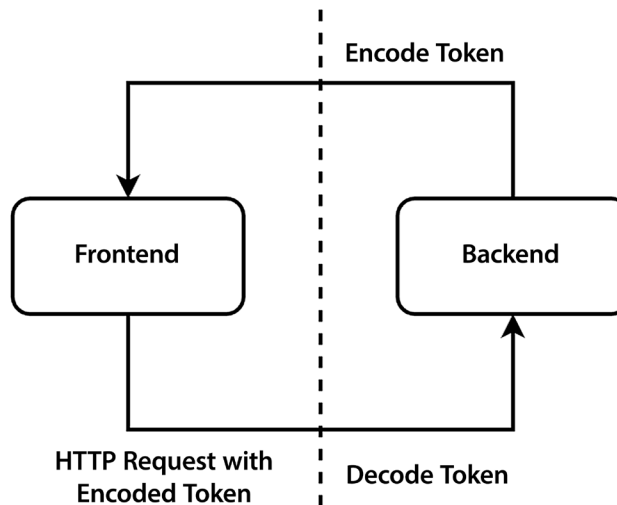


Figure 10.1 – Use of token and key in an HTTP request

For our JWT, we are going to extract the token from the environment variables with the following code:

```
// glue/src/token.rs
pub fn get_key() -> Result<String, NanoServiceError> {
    std::env::var("JWT_SECRET").map_err(|e| {
        NanoServiceError::new(
            e.to_string(),
            NanoServiceErrorStatus::Unauthorized
        )
    })
}
```

We can see that we return an unauthorized error if we cannot get the environment variable because we cannot authorize the incoming request.

With our key, we can now encode our token in the next section.

Creating an encode function to encode user credentials

We can encode our token with the following function:

```
// glue/src/token.rs
pub fn encode(self) -> Result<String, NanoServiceError> {
    let key_str = Self::get_key()?;
    let key = EncodingKey::from_secret(key_str.as_ref());
    return match encode(&Header::default(), &self, &key) {
        Ok(token) => Ok(token),
        Err(error) => Err(
            NanoServiceError::new(
                error.to_string(),
                NanoServiceErrorStatus::Unauthorized
            )
        )
    };
}
```

Here, we can see that we initially get the key for encoding our token. We then parse this into an `EncodingKey` struct, which can then be used to encode our `HeaderToken` struct into a token. It must be noted that the `HeaderToken` must have `#[derive(Serialize, Deserialize)]`.

With our encoded token, the only thing left is to decode it into a `HeaderToken` struct.

Creating a decode function to extract user credentials

We can decode our token with the following code:

```
// glue/src/token.rs
pub fn decode(token: &str) -> Result<Self, NanoServiceError> {
    let key_str = Self::get_key()?;
    let key = DecodingKey::from_secret(key_str.as_ref());
    let mut validation = Validation::new(Algorithm::HS256);
    validation.required_spec_claims.remove("exp");

    match decode:::<Self>(token, &key, &validation) {
        Ok(token_data) => return Ok(token_data.claims),
        Err(error) => return Err(
            NanoServiceError::new(
                error.to_string(),
                NanoServiceErrorStatus::Unauthorized
            )
        )
    }
}
```

```

    )
  )
};
}

```

Here, we can see that we get the key from the environment variable again. We then create a decoding key with the HS256 algorithm, which is a hashing algorithm used for signing and verifying JWTs. From this, we can deduce that the default encoding in the previous section was the HS256 algorithm. We then remove the “exp” from the claims. This means that we are removing the expiry time from the expected data in the encoded token.

Removing the “exp” from the token means that the token does not expire. If the token did expire, or the “exp” field was not in the token but we did not remove the “exp” from the expected claims, then the decoding of the token would result in an error. Removing the “exp” does make the handling of our application a lot easier, as we do not have to refresh tokens because the tokens do not expire. However, it is not as secure. For instance, if someone else got hold of a token, then they could call an API on behalf of the compromised user without any limitations.

In the next chapter, we will look into managing user sessions with expiring tokens, but for now, our basic authentication token will work.

We now have mechanisms to create users and provide auth tokens. So, now we can build our login API endpoint.

Building our login API

In this section, to test your knowledge of how to add API endpoints, you can attempt to code the endpoint yourself before reading the code. The only thing that we are going to code that has not been coded before is the API endpoint in the networking layer. To add the login API endpoint, we must carry out the following steps:

1. Getting users via email in the data access layer.
2. Creating our core login API.
3. Mounting our core login function to our server.

We can start with the data access layer.

Getting users via email in the data access layer

If you have attempted to code this yourself, hopefully you will have followed a similar approach to what is shown here. First, we import the following:

```

// nanoservices/auth/dal/src/users/transactions/get.rs
use crate::users::schema::User;
use glue::errors::{NanoServiceError, NanoServiceErrorStatus};
use std::future::Future;
use super::super::descriptors::SqlxPostgresDescriptor;
use crate::connections::sqlx_postgres::SQLX_POSTGRES_POOL;

```

We then define our trait with the following code:

```
// nanoservices/auth/dal/src/users/transactions/get.rs
pub trait GetByEmail {
    fn get_by_email(email: String)
    -> impl Future<Output = Result<
        User, NanoServiceError
    >> + Send;
}
```

We then implement this trait for our Postgres descriptor with the following code:

```
// nanoservices/auth/dal/src/users/transactions/get.rs
impl GetByEmail for SqlxPostgresDescriptor {
    fn get_by_email(email: String)
    -> impl Future<Output = Result<
        User, NanoServiceError
    >> + Send {
        sqlx_postgres_get_by_email(email)
    }
}
```

Finally, we define our function that interacts with the Postgres database with the following code:

```
// nanoservices/auth/dal/src/users/transactions/get.rs
async fn sqlx_postgres_get_by_email(email: String)
-> Result<User, NanoServiceError> {
    let item = sqlx::query_as::<_, User>("
        SELECT * FROM users WHERE email = $1"
    ).bind(email)
    .fetch_optional(&*SQLX_POSTGRES_POOL).await.map_err(|e| {
        NanoServiceError::new(
            e.to_string(),
            NanoServiceErrorStatus::Unknown
        )
    })?;
    match item {
        None => Err(NanoServiceError::new(
            "User not found".to_string(),
            NanoServiceErrorStatus::NotFound
        )),
        Some(item) => Ok(item)
    }
}
```

Our Postgres descriptor can now get users via their email. Note that we use `fetch_optional` to check whether the result is `None`. This is because we want to differentiate whether the user cannot be found or the error is a result of something else.

Now that our data access is working, we can move on to the core API for logging in a user.

Creating our core login API

For our core login API, we import the following:

```
// nanoservices/auth/core/src/api/auth/Login.rs
use auth_dal::users::transactions::get::GetByEmail;
use glue::errors::{NanoServiceError, NanoServiceErrorStatus};
use glue::token::HeaderToken;
```

We must not forget to add the module to the `mod.rs` file. Then, our login API function is defined with the following code:

```
// nanoservices/auth/core/src/api/auth/Login.rs
pub async fn login<T: GetByEmail>(
    email: String,
    password: String
) -> Result<String, NanoServiceError> {
    let user = T::get_by_email(email).await?;
    let outcome = user.verify_password(password)?;
    if outcome {
        Ok(HeaderToken{
            unique_id: user.unique_id
        }.encode())
    } else {
        Err(NanoServiceError::new(
            "Invalid password".to_string(),
            NanoServiceErrorStatus::Unauthorized
        ))
    }
}
```

Here, the true power of our approach starts to become apparent. Because all of our errors match and can be converted to HTTP responses, we can exploit the `?` operator. Note that this function just shows the flow of the logic. We can see that we get the user by email, then verify the password, returning an encoded token if the password verification is successful. We get a bird's-eye view of all the steps that our core function takes. If we want to inspect the specific logic behind a process, we can just click on that function. Trust me, when your system gets more complex, checking user roles and performing multiple database transactions, the ability to just click on the core API function and see a high-level view of the entire process is a lifesaver.

The only thing left to do for our login is to mount our core login function to our server.

Mounting our core login function to our server

When it comes to mounting our core login function to the server, we are going to use basic authentication to send our credentials to the server. Basic authentication is a simple authentication scheme built into the HTTP protocol. It involves sending the email and password in an HTTP header as a Base64-encoded string. This method is used to verify the identity of a user or client trying to access a resource on a server.

Before we define our login API function, we must create a function that extracts the basic auth credentials from the HTTP request sent to the server. Before we write any code, we must add the following dependency:

```
// nanoservices/auth/networking/actix_server/Cargo.toml
base64 = "0.22.1"
```

With the base64 dependency, our extract credentials function has the following outline:

```
// nanoservices/auth/networking/actix_server/
// src/extract_auth.rs
use actix_web::HttpRequest;
use glue::errors::{NanoServiceError, NanoServiceErrorStatus};
use base64::{Engine, engine::general_purpose};

#[derive(Debug)]
pub struct Credentials {
    pub email: String,
    pub password: String,
}

pub async fn extract_credentials(req: HttpRequest)
    -> Result<Credentials, NanoServiceError> {
    . . .
}
```

Inside our `extract_credentials` function, we extract the credentials from the header and convert the credentials into a string with the following code:

```
// nanoservices/auth/networking/actix_server/
// src/extract_auth.rs
let header_value = match req.headers().get("Authorization") {
    Some(auth_header) => auth_header,
    None => return Err(
        NanoServiceError::new(
            "No credentials provided".to_string(),
```



```

        NanoServiceErrorStatus::Unauthorized
    )
),
};
let encoded = match header_value.to_str() {
    Ok(encoded) => encoded,
    Err(_) => return Err(
        NanoServiceError::new(
            "Invalid credentials".to_string(),
            NanoServiceErrorStatus::Unauthorized
        )
    ),
};

```

Basic auth credentials start with the "Basic " string. If that string is not present, we can conclude that it is not basic auth, as shown in the following code:

```

// nanoservices/auth/networking/actix_server/
// src/extract_auth.rs
if !encoded.starts_with("Basic ") {
    return Err(
        NanoServiceError::new(
            "Invalid credentials".to_string(),
            NanoServiceErrorStatus::Unauthorized
        )
    )
}

```

We then decode our credentials and convert the result of the decoding into a string with the following code:

```

// nanoservices/auth/networking/actix_server/
// src/extract_auth.rs
let base64_credentials = &encoded[6..];
let decoded = general_purpose::STANDARD.decode(
    base64_credentials
).map_err(|e|{
    NanoServiceError::new(e.to_string(),
        NanoServiceErrorStatus::Unauthorized)
})?;
let credentials = String::from_utf8(
    decoded

```

```

).map_err(|e|{
    NanoServiceError::new(e.to_string(),
    NanoServiceErrorStatus::Unauthorized)})
)?;

```

Finally, we extract the password and email and return them with the following code:

```

// nanoservices/auth/networking/actix_server/
// src/extract_auth.rs
let parts: Vec<&str> = credentials.splitn(2, ':').collect();
if parts.len() == 2 {
    let email = parts[0];
    let password = parts[1];

    return Ok(Credentials {
        email: email.to_string(),
        password: password.to_string(),
    });
}
else {
    return Err(
        NanoServiceError::new("Invalid credentials".to_string(),
        NanoServiceErrorStatus::Unauthorized)
    )
}

```

For Axum, Hyper, and Rocket, the examples of extracting auth credentials can be found in the GitHub repo. The extract auth functions are essentially the same as what we have covered here, with slight variations based on the differences in the request struct for extracting the credentials from the header. With this extraction function, our login API endpoint can be defined with the following code:

```

// nanoservices/auth/networking/actix_server/
// src/api/auth/login.rs
use actix_web::HttpResponse;
use auth_dal::users::transactions::get::GetByEmail;
use glue::errors::NanoServiceError;
use crate::extract_auth::extract_credentials;
use auth_core::api::auth::login::login as core_login;

pub async fn login<T: GetByEmail>(req: actix_web::HttpRequest)
-> Result<HttpResponse, NanoServiceError> {
    let credentials = extract_credentials(req).await?;
    let token = core_login::<T>(

```

```

        credentials.email,
        credentials.password
    ).await?;
    Ok(HttpResponse::Ok().json(token))
}

```

For Hyper, Axum, and Rocket, the examples are in the GitHub repo, and they are consistent, with the exception of Rocket, where a `FromRequest` trait has to be implemented as opposed to passing the request directly into the view. Finally, we mount our login API endpoint to our server with the following code:

```

// nanoservices/auth/networking/actix_server/
// src/api/auth/mod.rs
pub mod login;
pub mod logout;
use actix_web::web::{ServiceConfig, get, scope};
use auth_dal::users::descriptors::SqlxPostgresDescriptor;

pub fn auth_factory(app: &mut ServiceConfig) {
    app.service(
        scope("/api/v1/auth")
        .route("login", get().to(
            login::login::<SqlxPostgresDescriptor>)
        )
    );
}

```

Our login functionality is now ready. We can now interact with our server to see whether it works.

Interacting with our auth server

Our auth server can create users and log in those users. We might as well plug our auth server into our ingress server and make some API calls.

First, we must declare our auth server and the auth data access layer in the ingress `Cargo.toml` file with the following code:

```

# ingress/Cargo.toml
auth_server = {
    path = "../nanoservices/auth/networking/actix_server",
    package = "auth-actix-server"
}
auth-dal = {
    path = "../nanoservices/auth/dal",
    package = "auth-dal"
}

```

We must remember that if there is no `lib.rs` in the auth server exposing the extraction of auth credentials and the API, we cannot import the auth server as a library. With our auth server as a library, we can now import these dependencies with the following code:

```
// ingress/src/main.rs
use auth_server::api::views_factory as auth_views_factory;
use auth_dal::migrations::run_migrations as run_auth_migrations;
```

We can now run our migrations and attach our auth endpoints to our ingress server. Once we have done this, our main function looks like the following:

```
// ingress/src/main.rs
#[tokio::main]
async fn main() -> std::io::Result<()> {
    run_todo_migrations().await;
    run_auth_migrations().await;
    HttpServer::new(|| {
        let cors = Cors::default().allow_any_origin()
                                   .allow_any_method()
                                   .allow_any_header();

        App::new()
            .configure(auth_views_factory)
            .configure(to_do_views_factory)
            .wrap(cors)
            .default_service(web::route().to(catch_all))
    })
    .bind("0.0.0.0:8001")?
    .run()
    .await
}
```

Finally, our `.env` file has the following contents:

```
// ingress/.env
TO_DO_DB_URL=postgres://username:password@localhost/to_do
AUTH_DB_URL=postgres://username:password@localhost/to_do
JWT_SECRET=secret
```

Now, we can run our server. Both migrations will run.



Getting a VersionMismatch error on migrations?

If your migration scripts have the exact same names with the same numbers, you might get an error when running the migrations. Just increase the number of the SQL migration script by 1 and it will solve this clash.

We can then test our create user API endpoint with the following cURL command:

```
curl -X POST http://0.0.0.0:8001/api/v1/users/create \
-H "Content-Type: application/json" \
-d '{
  "email": "test@gmail.com",
  "password": "password"
}'
```

Nothing should happen in the terminal, but our user has now been created. Now we can log in using the following command:

```
curl -u test@gmail.com:password -X GET \
http://0.0.0.0:8001/api/v1/auth/login
```

With this API request, we should get the following printout:

```
"eyJ0eXAiOiJKV1QiLCJhbGciOiJIUzI1NiJ9.
eyJ1bm1xdWVfawQiOiI5ODY5ZD11Mi02ZjZjLTRlOTEtODM4Mi02YjAyMjMzOTYzOTQifQ.
omz535P28m5KEfg6FVpBDMQr7sbidBCl6ju7VusGUl8"
```

And there we have it, our creation of the user and login was successful because we now have an auth token.

Now it is time to lock down all our to-do item calls. If we inspect our create to-do item API endpoint, we have the following signature:

```
// nanoservices/to_do/networking/actix_server/
// src/api/basic_actions/create.rs
...
use glue::{
    errors::NanoServiceError,
    token::HeaderToken
};
...
pub async fn create<T: SaveOne + GetAll>(
    token: HeaderToken, body: Json<NewToDoItem>)
-> Result<HttpResponse, NanoServiceError> {
    ...
}
```

We can see that we imported `HeaderToken` and put that token as a parameter in the function. As you might recall, putting `HeaderToken` as a function parameter fires the middleware that extracts the token from the header and performs the auth checks. To lock down all our other to-do API endpoints, we insert `HeaderToken` as a function parameter for all our API functions in the basic actions. Locking down all our API endpoints means that our frontend will no longer work as they all require an authentication token. In the next section, we will add authentication to our frontend.

Adding authentication to our frontend

To make our frontend compliant with authentication, we must carry out the following steps:

1. Building a login API call.
2. Adding tokens to our API calls.
3. Building a login form component.
4. Connecting the login form to the app.

Seeing as all our steps rely on the user being able to log in successfully, we can start with building our login API call.

Building a login API call

When it comes to mounting our core login function to the server, we are going to use basic authentication to send our credentials. This means that our API call function is going to be separate from the others as there are some extra steps. Seeing that our login API call is the only function that will utilize these steps, we might as well keep the login API call function separate and isolated.

We still want to utilize the functionality of the `Url` class, so we will add the login endpoint to the `Url` class, resulting in it having the following outline:

```
// ingress/frontend/src/api/url.ts
export class Url {

  baseUrl: string;
  create: string;
  getAll: string;
  update: string;
  login: string;

  constructor() {
    this.baseUrl = Url.getBaseUrl();
    this.create = `${this.baseUrl}api/v1/create`;
    this.getAll = `${this.baseUrl}api/v1/get/all`;
    this.update = `${this.baseUrl}api/v1/update`;
    this.login = `${this.baseUrl}api/v1/auth/login`;
  }

  . . .
}
```

Now that our `Url` supports our login, we can define the signature of the login API call with the following code:

```
// ingress/frontend/src/api/Login.ts
import axios from 'axios';
```

```
import { Url } from './url';

export const login = async (
  email: string,
  password: string
): Promise<string> => {
  . . .
};
```

Here, we take in an email and password and return the auth token. Inside our login, we encode the password with the following code:

```
// ingress/frontend/src/api/Login.ts
const authToken = btoa(`${email}:${password}`);
```

We then send a request to the server for a login with the following code:

```
// ingress/frontend/src/api/Login.ts
try {
  const response = await axios({
    method: 'get',
    url: new Url().login,
    headers: {
      'Authorization': `Basic ${authToken}`,
      'Content-Type': 'application/json'
    },
  });
  return response.data;
}
```

If we are successful, we return the token. However, we also handle an error with the following:

```
// ingress/frontend/src/api/Login.ts
catch (error) {
  if (axios.isAxiosError(error)) {
    console.error('Login error:', error.response?.data);
  } else {
    console.error('Unexpected error:', error);
  }
  alert('Login failed. Please try again.');
```

```
throw error;
}
```

If there is a failure, we alert the user with the `alert` function, which creates a popup in the browser. We keep the error code vague because we do not want to give hints to someone who is trying to hack a user account.

With our login API call done, and API calls fresh in our minds, it makes sense to now add tokens to our API calls.

Adding tokens to our API calls

Because we structured the bases of our API calls in one file, we just have to go to that file and change one line in each of our call functions. For our example, we will look at the `postCall` function with the following code:

```
// ingress/frontend/src/api/utils.ts
export async function postCall<T, X>(
  url: string, body: T, expectedResponse: number) {
  . . .
  {
    headers: {
      'Content-Type': 'application/json',
      'token': localStorage.getItem('token')
    },
    validateStatus: () => true
  });
  return handleRequest(response, expectedResponse);
}
```

Here, we can see that we have changed the token in the header from a string to `localStorage.getItem('token')`. This means that we can get the auth token from the local storage of the browser and insert it into our header of the API call. We can make this change with the following functions in the `utils.ts` file:

- `deleteCall`
- `putCall`
- `getCall`

Now all our API calls have auth tokens. This also means that we can use the local storage to check for a token to work out whether we are logged in, and we can also store our token in the local storage once our login API call is successful.

Now that we have our token mechanism figured out, and our login API call is defined, we're ready to make our login form component.

Building a login form component

For our login form, we must take in an email and a password, send that to our login API call function, and insert the token into local storage if the login was successful.

To achieve this, we import the following:

```
// ingress/frontend/src/components/LoginForm.tsx
import React from 'react';
import './Login.css';
import { login } from '../api/login';

interface LoginFormProps {
  setToken: (token: string) => void;
}
```

With the preceding imports and interface, we can define our login form component's signature with the following code:

```
// ingress/frontend/src/components/LoginForm.tsx
export const LoginForm: React.FC<LoginFormProps> = (
  { setToken }
) => {
  const [email, setEmail] = React.useState<string>("");
  const [password, setPassword] = React.useState<string>("");
  const submitLogin = () => {
    . . .
  };
  const handlePasswordChange = (
    e: React.ChangeEvent<HTMLInputElement>
  ) => {
    setPassword(e.target.value);
  };
  const handleUsernameChange = (
    e: React.ChangeEvent<HTMLInputElement>
  ) => {
    setEmail(e.target.value);
  };
  return (
    . . .
  );
};
```

With this outline, we can update the email and password entered into the form and fire the `login` function when the button on the form is clicked, using the following code:

```
// ingress/frontend/src/components/LoginForm.tsx
return (
  <div className="login">
```

```

    <h1 className="login-title">Login</h1>
    <input
      type="text"
      className="login-input"
      placeholder="Email"
      autoFocus
      onChange={handleUsernameChange}
      value={email}
    />
    <input
      type="password"
      className="login-input"
      placeholder="Password"
      onChange={handlePasswordChange}
      value={password}
    />
    <button className="login-button"
      id="login-button"
      onClick={submitLogin}>Lets Go</button>
  </div>
);

```

You may have noticed the CSS classes. To avoid bloating this chapter, we have displayed the CSS in the *Appendix* section at the end of the chapter.

Finally, we handle the button click with the following code:

```

// ingress/frontend/src/components/LoginForm.tsx
const submitLogin = () => {
  login(email, password).then(
    (response) => {
      setToken(response);
    }
  ).catch((error) => {
    console.error(error);
  });
};

```

If the call is successful, we call the `setToken` function that has been passed into the login form component.

We should now define the `setToken` function in our main application in the next section.

Connecting the login form to the app

In our main application component, we will import our login form component with the following code:

```
// ingress/frontend/src/index.tsx
import { LoginForm } from "../components/LoginForm";
```

Then, our main application component has the following changes:

```
// ingress/frontend/src/index.tsx
const App = () => {
  . . .
  const [loggedin, setLoggedIn] = useState<boolean>(
    localStorage.getItem('token') !== null
  );
  function setToken(token: string) {
    localStorage.setItem('token', token);
    setLoggedIn(true);
  }
  . . .
  React.useEffect(() => {
    const fetchData = async () => {
      if (wasmReady && loggedin) {
        . . .
      }
    };
    if (wasmReady && loggedin) {
      fetchData();
    }
  }, [wasmReady, loggedin]);

  if (localStorage.getItem('token') === null) {
    return <LoginForm setToken={setToken} />;
  }
  if (!data) {
    . . .
  }
  else {
    return (
      . . .
    );
  }
};
```

Here, we can see that we ensure that the WASM and login are achieved before trying to get the to-do items. We also add the login dependency to the loading of the to-do items, because if the login status changes, we will want to potentially trigger getting the to-do items. If we did not have that as a dependency, then after logging in, we would be stuck on a loading screen because getting the to-do items would not be triggered after the change of the login status.

Now our auth system is working; if we try and access our application, we would get the login form displayed in *Figure 10.2*.

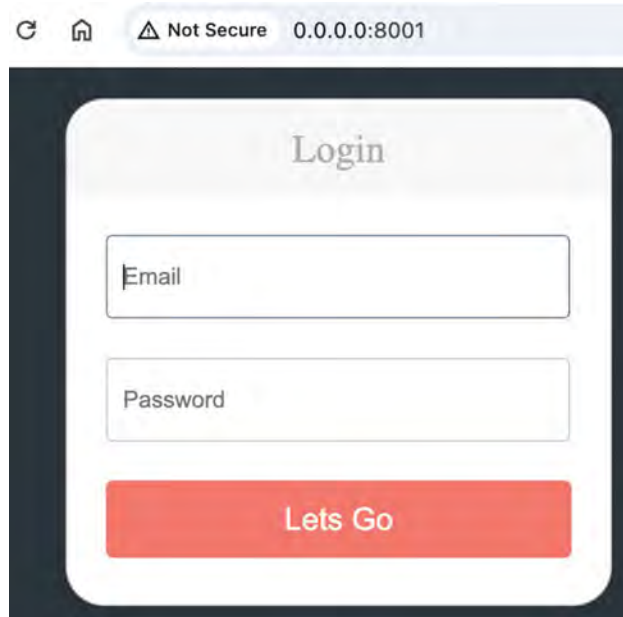


Figure 10.2 – Our login form

And there we have it, our login is successful. Our to-do items are still global, meaning that any user who has logged in can access the same items as any other user. In the next chapter, we will scope out the user sessions more.

Summary

In this chapter, we didn't just cover the basics of authentication; we added another server to our system. What's more, we compiled this server to the main ingress workspace, so our servers, the SQL scripts for those servers, and the React frontend are in one binary. Our auth server now authenticates our user and issues a token for other requests.

Now, we are really seeing how our system can scale. These servers can slot in and out of our system. Our authentication system is clearly defined, and there is nothing stopping you from taking your authentication system and slotting it into another system for another project. Authentication can explode in complexity. I often find myself creating roles and permissions, teams, and email processes to verify that the user's email is legitimate. Starting a server specifically for authentication can seem a little excessive; however, you will be shocked at how quickly the complexity grows. Most people starting projects underestimate the complexity of managing users.

In the next chapter, we will scope our user sessions so they time out, and users can only see to-do items that are tethered to the specific user. We will also cover how to get the to-do server to communicate with the auth server via HTTP or a memory call in the same binary.

Questions

1. Why do we hash passwords before storing them in the database?
2. What is the main advantage of a user having a JWT over storing a password?
3. How does a user store a JWT on the frontend?
4. What advantages do we get when we use a salt when hashing a password?
5. What is the basic auth protocol?

Answers

1. We hash the passwords in the database so if a hacker manages to gain access to the database, the raw passwords are not accessible.
2. If an attacker manages to obtain a JWT, it does not mean that they have direct access to the user's password. Also, if the tokens are set up to get refreshed, then the access that the attacker has to items has a limited timeframe.
3. The JWT can be stored in local HTML storage or cookies.
4. Using a salt when hashing a password protects against hash table attacks and brute-force attacks and ensures that passwords are unique.
5. The basic auth protocol is where we put the username and password in the header of the HTTP request, with the password being encoded via Base64.

Appendix

The CSS used for the login form is as follows:

```
body {
  background: #2d343d;
}
.login {
  margin: 20px auto;
  width: 300px;
  padding: 30px 25px;
  background: white;
  border: 1px solid #c4c4c4;
  border-radius: 25px;
}
h1.login-title {
  margin: -28px -25px 25px;
  padding: 15px 25px;
```

```
    line-height: 30px;
    font-size: 25px;
    font-weight: 300;
    color: #ADADAD;
    text-align:center;
    background: #f7f7f7;
    border-radius: 25px 25px 0px 0px;
  }
  .login-input {
    width: 285px;
    height: 50px;
    margin-bottom: 25px;
    padding-left:10px;
    font-size: 15px;
    background: #fff;
    border: 1px solid #ccc;
    border-radius: 4px;
  }
  .login-input:focus {
    border-color:#6e8095;
    outline: none;
  }
  .login-button {
    width: 100%;
    height: 50px;
    padding: 0;
    font-size: 20px;
    color: #fff;
    text-align: center;
    background: #f0776c;
    border: 0;
    border-radius: 5px;
    cursor: pointer;
    outline:0;
  }
  .login-lost
  {
    text-align:center;
    margin-bottom:0px;
  }
  .login-lost a
```

```
{
  color:#666;
  text-decoration:none;
  font-size:13px;
}
.loggedInTitle {
  font-family: "Helvetica Neue";
  color: white;
}
```

Learn more on Discord

To join the Discord community for this book – where you can share feedback, ask questions to the author, and learn about new releases – follow this QR code: <https://packt.link/EaKXL>



11

Communicating Between Servers

At this point in the book, we have two servers, which are the **authentication server** and the **to-do server**. However, they are not talking to each other yet. In microservices, our servers must be able to send messages to each other. For our system, we must get our to-do server making requests to our authentication server. This request checks that the user is valid before we perform a database transaction on to-do items, as these items are related to the user ID passed in the request for the database transaction. To achieve this, the chapter will cover the following:

- Getting users from auth with the unique ID
- Making auth accessible to other servers
- Tethering users to to-do items
- Testing our server-to-server communication with Bash

By the end of this chapter, you will be able to get servers talking to each other either directly in memory due to one server compiling another server into it, or via HTTP requests, depending on the feature compilation. You will also be able to perform basic tests on the server and more complex database queries.

Technical requirements

You can download the example project and code for this book by following the instructions in the **Download the example code files** section in the *Preface* of this book.

This chapter's code files are included in the downloadable code bundle.

Getting users from auth with the unique ID

Our authentication server now supports login and gets users using their email address. For our scoped authentication sessions to work, we must create an API where we can get the user details from the unique ID because we have the unique ID from the JWT in the authentication session.

At this point in the book, we have added a number of API endpoints to servers, so like the others, we are going to start with the data access layer. At this point, you should be familiar with adding API endpoints, so feel free to tackle this yourself by implementing the getting of the user by the unique ID in the data access layer, core layer, and networking layer. The following subsections about adding this endpoint will be brief.

Adding get by unique ID to dal

At this point, we are going to build on what we have already done, meaning that we are going to define a trait and implement that trait for our Postgres descriptor. With our plan, our trait takes the following form:

```
// nanoservices/auth/dal/src/users/transactions/get.rs
pub trait GetByUniqueId {
    fn get_by_unique_id(id: String) ->
    impl Future<Output = Result<User, NanoServiceError>> + Send;
}
```

Our trait implementation can be achieved with the following code:

```
// nanoservices/auth/dal/src/users/transactions/get.rs
impl GetByUniqueId for SqlxPostgresDescriptor {
    fn get_by_unique_id(id: String)
    -> impl Future<Output = Result<User, NanoServiceError>> + Send {
        sqlx_postgres_get_by_unique_id(id)
    }
}
```

Finally, our function that makes the database call is implemented with the following code:

```
// nanoservices/auth/dal/src/users/transactions/get.rs
async fn sqlx_postgres_get_by_unique_id(id: String)
-> Result<User, NanoServiceError> {
    let item = sqlx::query_as::<_, User>("
        SELECT * FROM users WHERE unique_id = $1"
    ).bind(id)
    .fetch_optional(&*SQLX_POSTGRES_POOL).await.map_err(|e| {
        NanoServiceError::new(
            e.to_string(),
            NanoServiceErrorStatus::Unknown
        )
    })?;
    match item {
        None => Err(NanoServiceError::new(
            "User not found".to_string(),
            NanoServiceErrorStatus::NotFound
        )),
        Some(item) => Ok(item)
    }
}
```

Our database access is achieved. We must ensure that the get is available with the following code:

```
// nanoservices/auth/dal/src/users/transactions/mod.rs
pub mod get;
pub mod create;
```

Now that we have set up a new API to get data access, we can move on to adding our get to the core.

Adding get by unique ID to core

Our core now supports getting a user using the unique ID, we can build our core API with the following code:

```
// nanoservices/auth/core/src/api/users/get.rs
use auth_dal::users::schema::TrimmedUser;
use auth_dal::users::transactions::get::GetByUniqueId;
use glue::errors::NanoServiceError;

pub async fn get_by_unique_id<T: GetByUniqueId>(id: String)
    -> Result<TrimmedUser, NanoServiceError> {
    let user = T::get_by_unique_id(id).await?;
    let trimmed_user: TrimmedUser = user.into();
    Ok(trimmed_user)
}
```

Again, we must ensure that our function is accessible with the following code:

```
// nanoservices/auth/core/src/api/users/mod.rs
pub mod create;
pub mod get;
```

Now our endpoint is ready to be exposed to the world in our networking layer.

Adding get by unique ID to networking

When it comes to passing a unique ID to our networking layer, we can just use our header token. Our token stores the unique ID. This enables us to securely call the API endpoint from the client, or just pass the token to the auth server from the to-do server. Our auth get endpoint can be defined with the following code:

```
// nanoservices/auth/networking/actix_server/src/api/users/get.rs
use auth_dal::users::transactions::get::GetByUniqueId;
use auth_core::api::users::get::get_by_unique_id
as get_by_unique_id_core;
use glue::errors::NanoServiceError;
use glue::token::HeaderToken;
use actix_web::HttpResponse;
```

```
pub async fn get_by_unique_id<T: GetByUniqueId>(token: HeaderToken)
    -> Result<HttpResponse, NanoServiceError> {
    let user = get_by_unique_id_core::<T>(token.unique_id).await?;
    Ok(HttpResponse::Ok().json(user))
}
```

We can then mount the endpoint to the server with our Postgres handler with the following code:

```
// nanoservices/auth/networking/actix_server/src/api/users/mod.rs
pub mod create;
pub mod get;
use auth_dal::users::descriptors::SqlxPostGresDescriptor;
use actix_web::web::{ServiceConfig, scope, post, get};

pub fn users_factory(app: &mut ServiceConfig) {
    app.service(
        scope("/api/v1/users")
        .route("create", post().to(
            create::create::<SqlxPostGresDescriptor>)
        )
        .route("get", get().to(
            get::get_by_unique_id::<SqlxPostGresDescriptor>)
        )
    );
}
```

Now our auth server can return a user by the unique ID by either making an HTTP request or calling the core function. For Axum, Hyper, and Rocket, no new methods or approaches are introduced to add this endpoint. The example code for Axum, Hyper, and Rocket can be found in the same location as the Actix example in the networking directory of the auth nanoservice. We can now move on to building the kernel of the auth server to make it easier for other servers to interact with the auth server via either the core or the HTTP networking layer.

Making auth accessible to other servers

To make our auth server accessible to another server, we build a kernel. Our authentication kernel takes the following file structure:

```
. . .
└─ nanoservices
    │   └─ auth
    │       └─ core
    │           └─ . . .
```


Here, we can see whether we are using the core feature. We are calling the core function for the core feature, but we have not defined our API calls using HTTP yet. We can also see that we are utilizing the `into` function to convert the response of both functions into the `TrimmedUser` struct because the `TrimmedUser` implements the `From<User>` trait.

Now that our interface is built, we can define the signature of the HTTP call with the following code:

```
// nanoservices/auth/kernel/src/api/users/get.rs
#[cfg(feature = "http")]
async fn get_user_by_unique_id_api_call(id: String)
    -> Result<TrimmedUser, NanoServiceError> {
    . . .
}
```

Inside this function, we define the URL to the auth server with the following code:

```
// nanoservices/auth/kernel/src/api/users/get.rs
let url = std::env::var("AUTH_API_URL").map_err(|e| {
    NanoServiceError::new(
        e.to_string(),
        NanoServiceErrorStatus::BadRequest
    )
})?;
let full_url = format!("{}/api/v1/users/get", url);
```

We can then build an encoded token, pass that into the header of a request, and send that request with the following code:

```
// nanoservices/auth/kernel/src/api/users/get.rs
let header_token = HeaderToken {
    unique_id: id
}.encode()?;
let client = Client::new();
let response = client
    .get(&full_url)
    .header("token", header_token)
    .send()
    .await
    .map_err(|e| {
        NanoServiceError::new(
            e.to_string(),
            NanoServiceErrorStatus::BadRequest
        )
    })?;
```

Finally, we can handle the response and return the user with the following code:

```
// nanoservices/auth/kernel/src/api/users/get.rs
if response.status().is_success() {
    let trimmed_user = response
        .json::<TrimmedUser>()
        .await
        .map_err(|e| NanoServiceError::new(
            e.to_string(),
            NanoServiceErrorStatus::BadRequest
        ))?;
    return Ok(trimmed_user)
} else {
    return Err(NanoServiceError::new(
        format!("Failed to get user: {} ", response.status()),
        NanoServiceErrorStatus::BadRequest,
    ))
}
```

Remember, we must make sure that our get API is publicly available in the kernel module. With our kernel built out, we can make our auth kernel accessible to our to-do server by declaring the kernel in the `Cargo.toml` of the to-do server with the following code:

```
# nanoservices/to_do/core/Cargo.toml
...
auth-kernel = { path = "../auth/kernel" }

[features]
memory = ["auth-kernel/core-postgres"]
http = ["auth-kernel/http"]
```

With this, our to-do server can get users by compiling the auth server and calling an API function or performing an HTTP request to the auth server.

We have maintained our flexibility with our deployment; however, our to-do server does not need to use user data right now. To make our user data useful to our to-do server, we must tether our to-do items to our user ID so users can see and edit their own items.

Tethering users to to-do items

Right now, all our to-do items are globally accessible to any user who logs in. To silo our to-do items, we must carry out the following steps:

1. Linking our to-do items to users in the database
2. Adding user IDs to data access transactions

3. Adding user IDs to core API functions
4. Adding user IDs to networking API functions

Seeing as all the steps require the database to be updated, we will start with the database configuration.

Linking our to-do items to users in the database

If we had both the user and to-do item tables in the same database, we could tether them using a foreign key, as seen in *Figure 11.1*:

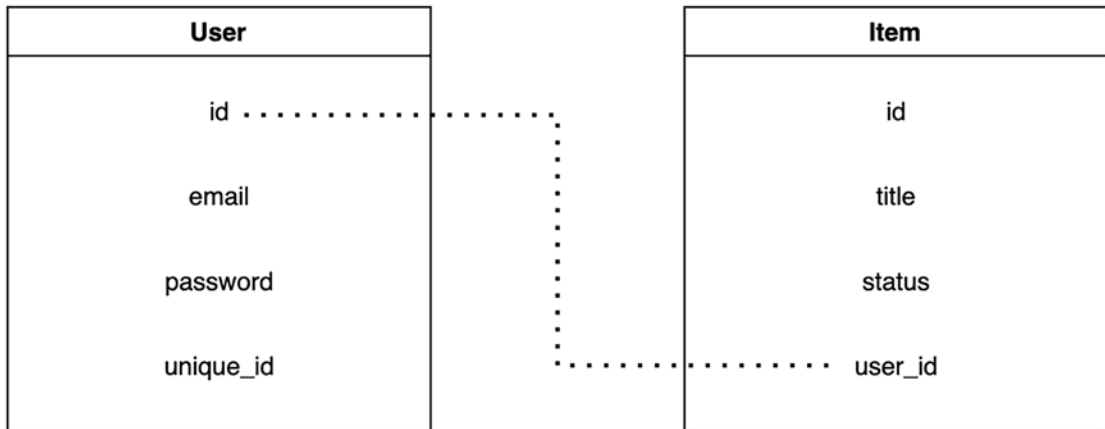


Figure 11.1 – Foreign key association between our user and items

There are plenty of advantages to the foreign key approach. For instance, we would not be able to enter a user ID into the item table if the user ID did not exist in the user table. We could also enforce a cascading delete in the item table with the following SQL:

```
CREATE TABLE Item (
  id SERIAL PRIMARY KEY,
  title VARCHAR(50),
  status TEXT NOT NULL,
  user_id INT REFERENCES User(id) ON DELETE CASCADE
);
```

This means that if we delete a user, then all the items associated with that user would also be deleted. We could also perform a join query with the following SQL:

```
SELECT
  User.id AS user_id,
  User.username,
  Item.id AS todo_id,
  Item.title,
  status.status,
FROM
```

```
users
JOIN
  todos ON users.id = todos.user_id;
```

This means that each row would contain elements from the item and the user associated with that item.

However, our items are on the to-do server, and our users are on our auth server. Due to our setup, they could be on the same database, but we must also accommodate the possibility of the items and users being on separate databases. Like everything in software engineering, there are always trade-offs. Microservices enable you to break a massive system into components and have individual teams work on these individual services. You can also use different languages, and deployment pipelines, while being more complex, will be smoother. However, none of this is for free. Microservices add complexity. To accommodate the possibility of two different databases, we have a third table where rows consist of user IDs and item IDs, as seen in *Figure 11.2*:

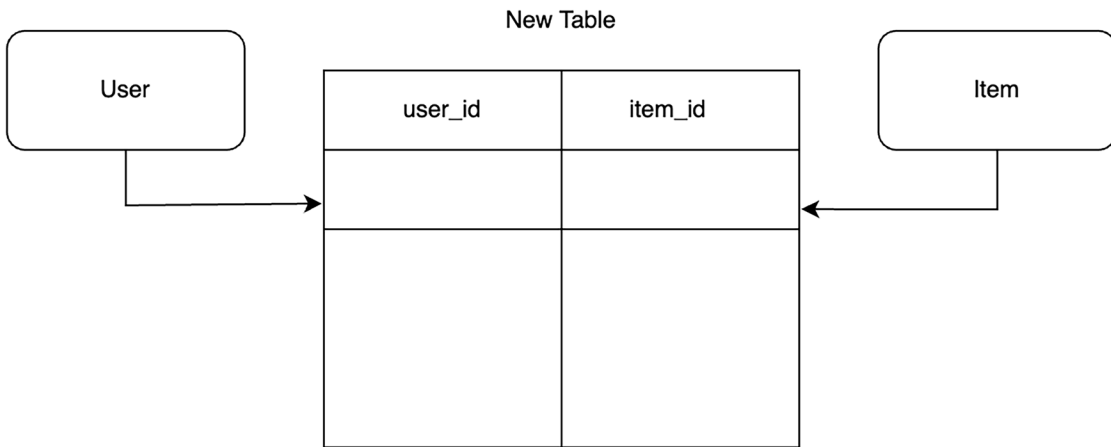


Figure 11.2 – A separate database table for logging item associations with users

This does increase the risk of us accidentally putting in an ID that does not exist, or not deleting items if we delete a user, as we must perform operations on this separate table. However, it decouples the associations. For instance, we can have items that have no association to a user. This might be useful if you plan on building out teams in the application, and in those teams some items may not have been assigned to a user. With this additional table, we can also assign multiple users to one item, and multiple items to one user.

Now that we know how we are going to link our users with items, we can create a new migration for our to-do service. To automate our migration creation, we can create a Bash script housing the following code:

```
#!/usr/bin/env bash
# nanoservices/to_do/scripts/add_migration.sh

# Check if argument is provided
```

```

if [ $# -eq 0 ]; then
    echo "Usage: $0 <migration-name>"
    exit 1
fi
# navigate to directory
SCRIPTPATH="$( cd "$(dirname "$0")" ; pwd -P )"
cd $SCRIPTPATH/../../dal/migrations

# create SQL script name
current_timestamp=$(date +%Y%m%d%H%M%S')
description="$1"
script_name="${current_timestamp}_${description}.sql"
touch $script_name

```

Here, we can see that we exit the script if we do not provide an argument for the script. We then navigate to the migrations directory, create a timestamp, and create a SQL script with our argument, prefixed with a timestamp. This ensures that the migrations that we create will run in the order that they were created. Inside our newly formed migration file, we create the reference table with the following code:

```

-- nanoservices/to_do/dal/migrations/
-- 20240628025045_adding-user-connection.sql
CREATE TABLE user_connections (
    user_id INTEGER NOT NULL,
    to_do_id INTEGER NOT NULL,
    PRIMARY KEY (user_id, to_do_id)
);

```

We have a combination of the user ID and the item ID to ensure that we do not insert duplicate associations.

Now that our database can link items to users, we can move on to adding this linking table to our schema in our data access layer with the following code:

```

// nanoservices/to_do/dal/src/to_do_items/schema.rs
#[derive(Serialize, Deserialize, Debug, Clone)]
#[cfg_attr(feature = "sqlx-postgres", derive(sqlx::FromRow))]
pub struct UserConnection {
    pub user_id: i32,
    pub to_do_id: i32
}

```

As you can see, we have defined that the `UserConnection` struct is only declared if our Postgres feature is activated. This is because we do not have the connection table if we are using a JSON file. With our schema done, our entire system can interact with the database to tether to-do items to users. Therefore, we can move on to adding the user ID to database transactions.

Adding user IDs to data access transactions

For our transactions, we can start with the create operation with the following refactored code:

```
// nanoservices/to_do/dal/src/to_do_items/transactions/create.rs
pub type SaveOneResponse = Result<ToDoItem, NanoServiceError>;

pub trait SaveOne {
    fn save_one(item: NewToDoItem, user_id: i32)
    -> impl Future<Output = SaveOneResponse> + Send;
}

#[cfg(feature = "sqlx-postgres")]
impl SaveOne for SqlxPostgresDescriptor {
    fn save_one(item: NewToDoItem, user_id: i32)
    -> impl Future<Output = SaveOneResponse> + Send {
        sqlx_postgres_save_one(item, user_id)
    }
}

#[cfg(feature = "json-file")]
impl SaveOne for JsonFileDescriptor {
    fn save_one(item: NewToDoItem, user_id: i32)
    -> impl Future<Output = SaveOneResponse> + Send {
        json_file_save_one(item, user_id)
    }
}
```

Here, we can see that we merely pass the user ID into the functions. For our `sqlx_postgres_save_one` function, we now have two database transactions. We create the item and then use the ID of that item to the connections with the following code:

```
// nanoservices/to_do/dal/src/to_do_items/transactions/create.rs
#[cfg(feature = "sqlx-postgres")]
async fn sqlx_postgres_save_one(item: NewToDoItem, user_id: i32)
-> SaveOneResponse {
    let item = sqlx::query_as::<_, ToDoItem>("
        INSERT INTO to_do_items (title, status)
        VALUES ($1, $2)
        RETURNING *"
    ).bind(item.title)
    .bind(item.status.to_string())
    .fetch_one(&*SQLX_POSTGRES_POOL).await.map_err(|e| {
        NanoServiceError::new(
            e.to_string(),
```

```

        NanoServiceErrorStatus::Unknown
    )
  })?;
  let _ = sqlx::query("
    INSERT INTO user_connections (user_id, to_do_id)
    VALUES ($1, $2)"
  ).bind(user_id)
  .bind(item.id)
  .execute(&*SQLX_POSTGRES_POOL).await.map_err(|e| {
    NanoServiceError::new(
      e.to_string(),
      NanoServiceErrorStatus::Unknown
    )
  })?;
  Ok(item)
}

```

Finally, our JSON save file function takes the following form:

```

// nanoservices/to_do/dal/src/to_do_items/transactions/create.rs
#[cfg(feature = "json-file")]
async fn json_file_save_one(item: NewToDoItem, user_id: i32)
  -> SaveOneResponse {
  let mut tasks = get_all::<ToDoItem>().unwrap_or_else(|_|
    HashMap::new()
  );
  let to_do_item = ToDoItem {
    id: 1,
    title: item.title,
    status: item.status.to_string()
  };
  tasks.insert(
    to_do_item.title.to_string() +
    ":" +
    &user_id.to_string(),
    to_do_item.clone()
  );
  let _ = save_all(&tasks)?;
  Ok(to_do_item)
}

```

Here, we can see that we have created a key out of the title and user ID with the `:` delimiter. This means we can still have a single hashmap for a JSON file, but still separate the items depending on the user ID.

Our delete transaction can take the same approach as our create transaction but in an inverse manner. This is a good time for you to try and code this section yourself. If you did attempt this, your trait implementations should look like the following code:

```
// nanoservices/to_do/dal/src/to_do_items/transactions/delete.rs
pub type DeleteOneResponse = Result<ToDoItem, NanoServiceError>;

pub trait DeleteOne {
    fn delete_one(title: String, user_id: i32)
    -> impl Future<Output = DeleteOneResponse> + Send;
}

#[cfg(feature = "sqlx-postgres")]
impl DeleteOne for SqlxPostGRESDescriptor {
    fn delete_one(title: String, user_id: i32)
    -> impl Future<Output = DeleteOneResponse> + Send {
        sqlx_postgres_delete_one(title, user_id)
    }
}

#[cfg(feature = "json-file")]
impl DeleteOne for JsonFileDescriptor {
    fn delete_one(title: String, user_id: i32)
    -> impl Future<Output = DeleteOneResponse> + Send {
        json_file_delete_one(title, user_id)
    }
}
```

Your Postgres delete function should look like the following code:

```
// nanoservices/to_do/dal/src/to_do_items/transactions/delete.rs
#[cfg(feature = "sqlx-postgres")]
async fn sqlx_postgres_delete_one(title: String, user_id: i32)
    -> DeleteOneResponse {
    let item = sqlx::query_as::<_, ToDoItem>("
        DELETE FROM to_do_items
        WHERE title = $1
        RETURNING *"
    ).bind(title)
    .fetch_one(&SQLX_POSTGRES_POOL).await.map_err(|e| {
        NanoServiceError::new(
            e.to_string(),
            NanoServiceErrorStatus::Unknown
        )
    })?;
}
```

```

let _ = sqlx::query("
    DELETE FROM user_connections
    WHERE user_id = $1 AND to_do_id = $2"
).bind(user_id)
.bind(item.id)
.execute(&*SQLX_POSTGRES_POOL).await.map_err(|e| {
    NanoServiceError::new(
        e.to_string(),
        NanoServiceErrorStatus::Unknown
    )
})?;
Ok(item)
}

```

Your delete function for the JSON file should be like the following code:

```

// nanoservices/to_do/dal/src/to_do_items/transactions/delete.rs
#[cfg(feature = "json-file")]
async fn json_file_delete_one(title: String, user_id: i32)
    -> DeleteOneResponse {
    let mut tasks = get_all::<ToDoItem>().unwrap_or_else(
        |_| HashMap::new()
    );
    let to_do_item = tasks.remove(
        &(title +
            ":" +
            &user_id.to_string())
    ).ok_or_else(|| {
        NanoServiceError::new(
            "Item not found".to_string(),
            NanoServiceErrorStatus::NotFound
        )
    })?;
    let _ = save_all(&tasks)?;
    Ok(to_do_item)
}

```

We can now move on to our get transaction with the following implementations:

```

// nanoservices/to_do/dal/src/to_do_items/transactions/get.rs
pub type GetAllResponse = Result<Vec<ToDoItem>, NanoServiceError>;

pub trait GetAll {

```

```

    fn get_all(user_id: i32)
    -> impl Future<Output = GetAllResponse> + Send;
}
#[cfg(feature = "sqlx-postgres")]
impl GetAll for SqlxPostgresDescriptor {
    fn get_all(user_id: i32)
    -> impl Future<Output = GetAllResponse> + Send {
        sqlx_postgres_get_all(user_id)
    }
}
#[cfg(feature = "json-file")]
impl GetAll for JsonFileDescriptor {
    fn get_all(user_id: i32)
    -> impl Future<Output = GetAllResponse> + Send {
        json_file_get_all(user_id)
    }
}
}

```

For Postgres, we retrieve all item IDs connected to the user and then query `to_do_items` for items with those IDs. We can see this query unfold with the following code:

```

// nanoservices/to_do/dal/src/to_do_items/transactions/get.rs
#[cfg(feature = "sqlx-postgres")]
async fn sqlx_postgres_get_all(user_id: i32) -> GetAllResponse {
    let items = sqlx::query_as::<_, ToDoItem>("
        SELECT * FROM to_do_items WHERE id IN (
            SELECT to_do_id
            FROM user_connections WHERE user_id = $1
        )")
        .bind(user_id)
        .fetch_all(&*SQLX_POSTGRES_POOL).await.map_err(|e| {
            NanoServiceError::new(
                e.to_string(),
                NanoServiceErrorStatus::Unknown
            )
        })?;
    Ok(items)
}

```

For our JSON file implementation, we load the file, get the key of each item, split the key with our delimiter to get the user ID, and push the item to a vector if the user ID extracted from the key matches the user ID passed into the function, using the following code:


```
// nanoservices/to_do/dal/src/to_do_items/transactions/get.rs
#[cfg(feature = "json-file")]
async fn json_file_get_all(user_id: i32) -> GetAllResponse {
    let tasks = get_all:::<ToDoItem>()
        .unwrap_or_else(|_| HashMap::new());
    let mut filtered_items: Vec<ToDoItem> = Vec::new();
    for item in tasks.keys() {
        let key = item.split(":").nth(1).unwrap();
        let item_user_id = key.parse:::<i32>().unwrap();
        if item_user_id == user_id {
            filtered_items.push(
                tasks.get(item).unwrap().clone()
            );
        }
    }
    Ok(filtered_items)
}
```

Finally, for our update, we have the following trait implementations:

```
// nanoservices/to_do/dal/src/to_do_items/transactions/update.rs
pub type UpdateOneResponse = Result<ToDoItem, NanoServiceError>;

pub trait UpdateOne {
    fn update_one(item: ToDoItem, user_id: i32)
        -> impl Future<Output = UpdateOneResponse> + Send;
}

#[cfg(feature = "sqlx-postgres")]
impl UpdateOne for SqlxPostGresDescriptor {
    fn update_one(item: ToDoItem, _user_id: i32)
        -> impl Future<Output = UpdateOneResponse> + Send {
        sqlx_postgres_update_one(item)
    }
}

#[cfg(feature = "json-file")]
impl UpdateOne for JsonFileDescriptor {
    fn update_one(item: ToDoItem, user_id: i32)
        -> impl Future<Output = UpdateOneResponse> + Send {
        json_file_update_one(item, user_id)
    }
}
```

Because we are passing in the item to be altered, we already know the ID of the item; therefore, we do not need the user ID for the Postgres implementation. This is because the item ID is enough to get the correct item. We are not updating the connections table as the item is still in the database after the update, and therefore, it is still associated with the user. However, because we have the ID of the user in the key for the JSON file, we must utilize the user ID for the JSON file using the following code:

```
// nanoservices/to_do/dal/src/to_do_items/transactions/update.rs
#[cfg(feature = "json-file")]
async fn json_file_update_one(item: ToDoItem, user_id: i32)
    -> UpdateOneResponse {
    let mut tasks = get_all::<ToDoItem>().unwrap_or_else(
        |_| HashMap::new()
    );
    let key = item.title.clone() + ":" + &user_id.to_string();
    if !tasks.contains_key(&key) {
        return Err(NanoServiceError::new(
            format!("Item with name {} not found", item.title),
            NanoServiceErrorStatus::NotFound
        ));
    }
    tasks.insert(key, item.clone());
    let _ = save_all(&tasks)?;
    Ok(item)
}
```

And this is it, the data access layer is now fully tethered to the user ID for items. We now must pass the user ID into the core functions.

Adding user IDs to core functions

For our core function, we must remember that our core does not want to have dependencies on other servers. Therefore, we merely pass the user ID into our core API functions with the following code:

```
// nanoservices/to_do/core/src/api/basic_actions/create.rs
pub async fn create<T: SaveOne>(item: NewToDoItem, user_id: i32)
    -> Result<ToDoItem, NanoServiceError> {
    let created_item = T::save_one(item, user_id).await?;
    Ok(created_item)
}

// nanoservices/to_do/core/src/api/basic_actions/delete.rs
pub async fn delete<T: DeleteOne>(id: &str, user_id: i32)
    -> Result<(), NanoServiceError> {
    let _ = T::delete_one(id.to_string(), user_id).await?;
    Ok(())
}
```

```

}
// nanoservices/to_do/core/src/api/basic_actions/get.rs
pub async fn get_all<T: GetAll>(user_id: i32)
    -> Result<AllToDoItems, NanoServiceError> {
    let all_items = T::get_all(user_id).await?;
    AllToDoItems::from_vec(all_items)
}

// nanoservices/to_do/core/src/api/basic_actions/update.rs
pub async fn update<T: UpdateOne>(item: ToDoItem, user_id: i32)
    -> Result<(), NanoServiceError> {
    let _ = T::update_one(item, user_id).await?;
    Ok(())
}

```

Our core is now tethered. Now we must move on to adding the user ID to the networking layer.

Adding user IDs to networking functions

For our networking, we must get the user ID from the auth server via the kernel. Therefore, we must declare our auth kernel with the following code:

```

// nanoservices/to_do/networking/actix_server/Cargo.toml
...
auth-kernel = { path = "../../auth/kernel" }

[features]
auth-http = ["auth-kernel/http"]
auth-core-postgres = ["auth-kernel/core-postgres"]
default = ["auth-core-postgres"]

```

Here, we can see that we propagate the features to the auth kernel. Therefore, if we activate the `auth-http` feature, then we will interact with our auth server via HTTP. If we activate the `auth-core-postgres` feature, we will be compiling the auth server into the to-do server and call the auth functions directly. Postgres is our default feature.

We can see how this is implemented for the create endpoint with the following code:

```

// nanoservices/to_do/networking/actix_server/src
// /api/basic_actions/create.rs
...
use auth_kernel::api::users::get::get_user_by_unique_id;

pub async fn create<T: SaveOne + GetAll>(
    token: HeaderToken,
    body: Json<NewToDoItem>
) -> Result<HttpResponse, NanoServiceError> {

```

```

    let user = get_user_by_unique_id(token.unique_id).await?;
    let _ = create_core::<T>(body.into_inner(), user.id).await?;
    Ok(HttpResponse::Created().json(get_all_core::<T>(
        user.id
    ).await?))
}

```

Here, we can see that we get the user's unique ID from the token and get the user from the auth kernel. We can apply this approach to all the other endpoints defined here:

```

// nanoservices/to_do/networking/actix_server/src
// /api/basic_actions/get.rs
. . .
use auth_kernel::api::users::get::get_user_by_unique_id;

pub async fn get_all<T: GetAll>(token: HeaderToken)
    -> Result<HttpResponse, NanoServiceError> {
    let user = get_user_by_unique_id(
        token.unique_id
    ).await?;
    Ok(HttpResponse::Ok().json(get_all_core::<T>(
        user.id
    ).await?))
}

```

For the update endpoint, we follow the same pattern: extract the unique ID from the token, fetch the user from the auth kernel, and pass `user.id` into `update_core`, before returning an updated `get_all` response:

```

// nanoservices/to_do/networking/actix_server/src
// /api/basic_actions/update.rs
. . .
use auth_kernel::api::users::get::get_user_by_unique_id;

pub async fn update<T: UpdateOne + GetAll>(
    token: HeaderToken,
    body: Json<ToDoItem>
) -> Result<HttpResponse, NanoServiceError> {
    let user = get_user_by_unique_id(token.unique_id).await?;
    let _ = update_core::<T>(body.into_inner(), user.id).await?;
    Ok(HttpResponse::Ok().json(get_all_core::<T>(
        user.id
    ).await?))
}

```

For the delete endpoint, we again reuse the same flow: extract the unique ID from the token, fetch the user from the auth kernel, and pass `user.id` into `delete_core` (along with the item name from the route), before returning an updated `get_all` response:

```
// nanoservices/to_do/networking/actix_server/src
// /api/basic_actions/delete.rs
. . .
use auth_kernel::api::users::get::get_user_by_unique_id;

pub async fn delete_by_name<T: DeleteOne + GetAll>(
    token: HeaderToken,
    req: HttpRequest
) -> Result<HttpResponse, NanoServiceError> {
    let user = get_user_by_unique_id(
        token.unique_id
    ).await?;
    match req.match_info().get("name") {
        Some(name) => {
            delete_core::<T>(name, user.id).await?;
        },
        None => {
            return Err(
                NanoServiceError::new(
                    "Name not provided".to_string(),
                    NanoServiceErrorStatus::BadRequest
                )
            )
        }
    }
};
Ok(HttpResponse::Ok().json(get_all_core::<T>(
    user.id
).await?))
}
```

With this, our networking layer for our to-do server is now tethered to the user ID. This means that once logged in, a user can only see and perform operations that are tethered to the user. If you look through the Axum, Hyper, and Rocket examples, you will see how easy it is to inject the user information. In all the API endpoints, we essentially import the same `get_user_by_unique_id` from the kernel, get the user from this function, and propagate the user ID through all the data access layer transaction functions. The footprint of this refactor is minimal and universal regardless of the web framework. However, we have compromised by hardcoding in the database and HTTP descriptor, making it hard to test and mock. We will refactor this out in *Chapter 12* when we introduce caching to get the user session. However, for now, the kernel refactor is a powerful demonstration of how simple refactors can be if you are willing to compromise.

Our system now has the to-do server talking to the auth server. If we were to run our ingress server, it would work as intended, and we would only see the to-do items in the frontend that correspond to the user logged in. However, this is down to the to-do server calling the authentication server in the same binary as all the servers and frontend are compiled into one binary. We know the ingress server will work because it compiles. However, what about when both of our servers are running independently on different ports? Even if it compiles, the HTTP request to the auth server from the to-do server could be faulty. We must test our HTTP request.

Testing our server-to-server communication with Bash

To run our test, we essentially need to spin up our database, auth server, and to-do server. We then must run an HTTP request to create the user, log in, and then create a to-do item. If we can create the to-do item, we know that the communication between both servers works, because we must call the authentication server to get the user ID to create the to-do item.

To run our test, we first must create the following `.env` file:

```
# nanoservices/to_do/.env
TO_DO_DB_URL=postgres://username:password@localhost/to_do
AUTH_DB_URL=postgres://username:password@localhost/to_do
AUTH_API_URL=http://127.0.0.1:8081
JWT_SECRET=secret
```

Now that we have the environment variables that are needed, we can start with the boilerplate code for our testing Bash script with the following code:

```
#!/usr/bin/env bash
# nanoservices/to_do/scripts/test_server_com.sh

SCRIPTPATH="$( cd "$(dirname "$0")" ; pwd -P )"
cd $SCRIPTPATH
cd ../../../../
```

This boilerplate code ensures that the working directory of the script is going to run at the base of our system. Now that we are in the root directory of our system, we can build our database and then run it in the background with the following code:

```
# nanoservices/to_do/scripts/test_server_com.sh
docker-compose build
docker compose up -d
sleep 1
export $(cat ./nanoservices/to_do/.env | xargs)
```

We can see that we perform a build first before running the database. This is because it could take a while to build the Docker containers. If the build takes too long and we are running it in the background, the rest of the script could fail due to the database not being ready when running the servers. We can also see that we export the environment variables from our `.env` file.

Because the server builds might also take a while, we want to build our servers before running them with the following code:

```
# nanoservices/to_do/scripts/test_server_com.sh
cargo build \
--manifest-path ./nanoservices/to_do/networking/actix_server/Cargo.toml \
--features auth-http \
--release \
--no-default-features

cargo build \
--manifest-path ./nanoservices/auth/networking/actix_server/Cargo.toml \
--release
```

Here, we can see that we are using `--no-default-features` and the `auth-http` feature for our to-do server. This is to ensure that we are compiling the auth kernel into the to-do server with HTTP requests.

We can now run our servers in the background with the following code:

```
# nanoservices/to_do/scripts/test_server_com.sh
cargo run \
--manifest-path ./nanoservices/to_do/networking/actix_server/Cargo.toml \
--features auth-http \
--release --no-default-features &
TO_DO_PID=$!

cargo run \
--manifest-path ./nanoservices/auth/networking/actix_server/Cargo.toml \
--release &
AUTH_PID=$!
```

The `&` means that the command will be run in the background. We can also see that we get the process IDs with `TO_DO_PID=$!` and `AUTH_PID=$!` right after their respective commands. We will reference these process IDs to kill the servers once we have finished with the servers.

Now our servers are running, and our database is also running. We can now create a user and log in to assign the login result to the token variable with the following code:

```
# nanoservices/to_do/scripts/test_server_com.sh
sleep 2
curl -X POST http://127.0.0.1:8081/api/v1/users/create \
-H "Content-Type: application/json" \
-d '{
  "email": "test@gmail.com",
  "password": "password"
}'
```

```
token=$(curl \
-u test@gmail.com:password \
-X GET http://127.0.0.1:8081/api/v1/auth/login)
token=$(echo "$token" | tr -d '\r\n' | sed 's/^"' | sed 's/"$//')
```

The final line strips the token of quotation marks, spaces, and new lines. We can now insert that token into the header of our create HTTP request with the following code:

```
# nanoservices/to_do/scripts/test_server_com.sh
response=$(curl -X POST http://127.0.0.1:8080/api/v1/create \
-H "Content-Type: application/json" \
-H "token: $token" \
-d '{
  "title": "code",
  "status": "PENDING"
}')

sleep 1
echo $response
sleep 2
```

Finally, we kill the servers and tear down the database container with the following code:

```
# nanoservices/to_do/scripts/test_server_com.sh
kill $TO_DO_PID
kill $AUTH_PID
docker compose down
```

If you want to inspect the state of the database via a desktop application, then comment out the `docker compose down` command so the database runs in the background after the test, but remember to kill the database container before running the test again. Once we run this script, we should get the following printout:

```
. . .
Migrating auth database...
auth database migrations completed: ()
Migrating to-do database...
to-do database migrations completed: ()
. . .
{"pending":[{"id":1,"title":"code","status":"PENDING"}],"done":[]}
[+] Running 2/1
  ✓ Container to-do-postgres Removed
0.1s
  ✓ Network tethering-users-to-items_default Removed
```


There is more to the printout, such as the build phase and printouts of cURL network stats, but including these would just bloat the chapter. What we can see here is that the migrations have happened, and our create HTTP request was successful. Our database was then removed.

There we have it: our system can support two servers that talk to each other via HTTP.

Summary

In this chapter, we got our to-do server communicating with the authentication server through a kernel, by either compiling the auth server into the to-do server or making an HTTP request to the authentication server. In a microservice structure, communicating between servers is essential. Here, we have the flexibility of still running our system as a monolith app with one database, a monolith app with two separate databases, two separate servers with one database, or two separate servers with one database. Here, you have the ultimate flexibility when it comes to deploying your system. We will explore other protocols to communicate between Rust servers in *Chapter 19*, where we will pass binary messages between servers via TCP. In the next chapter, we will handle timeouts of our authentication sessions with caching.

Questions

1. At a high level, how did we make the user accessible via the unique ID?
2. How do we make the core API function available to other servers directly by compiling the core function into another server?
3. How do we expose the HTTP endpoint of one server to another?
4. There is a risk that two different servers will be running transactions on two different databases. How can we tether data from one service to another considering that we must accommodate the possibility of two different databases?
5. What is the risk of our database solution?

Answers

1. We added the get user by unique ID function from the database to the data access layer. We then added a function referencing the DAL function in the core. Finally, we added an HTTP layer around the core function in the networking layer.
2. We build another Rust workspace that is a kernel. This kernel has the API functions that we want to expose, and the other server can compile the kernel to call the core functions.
3. We can add an HTTP feature to our kernel. This HTTP feature will not compile the core function, but instead will construct an HTTP request for the endpoint. The interface stays the same, so if we want to change our deployment strategy, we do not have to change any code.
4. We can have a reference table on one of the databases that has the ID of the row of the data from one database and the ID of the row from another database. We can then read that table to work out what the data from one table is tethered to in a table from another database.

5. Data consistency is the risk. Unlike foreign keys pointing to tables in same database, our code might not update the data correctly and we could have data that is not logged in the reference table, or we could have dangling references due to the reference not being deleted when the data is deleted.

Get this book's PDF copy, code bundle, and more

Scan the QR code (or go to packtpub.com/unlock). Search for this book by name, confirm the edition, and then follow the steps on the page.



UNLOCK NOW

Note: Have your invoice handy. Purchases made directly from the Packt website don't require an invoice.

12

Caching Auth Sessions

While our authentication sessions work, we might as well get more control over them and explore the concept of caching at the same time. We will do this by embedding our own Rust code directly in a Redis database, so we can perform a range of checks and updates in one call to the cache, as opposed to making multiple calls to a database. Caching is a great tool to add to your belt, enabling you to reduce the time and resources used to serve data. In this chapter, we cover the following:

- What is caching?
- Setting up Redis
- Building our Redis module
- Building our Redis client
- Connecting our cache to our servers

By the end of this chapter, you will be able to build custom caching functionality directly inside Redis to control the state of the user session. However, this caching skill can also be used in a variety of different problems. It must be noted that we are near the end of building out our main application. This means that you will be familiar with a lot of the approaches. This chapter might feel like you are just skimming through the code with repetitive patterns. This is a sign that you have learned the main approaches of web development in Rust, and you are becoming competent in developing Rust web applications by yourself. Even though we are adding a new storage engine, which is Redis, we want to keep our interfaces and IO module layouts consistent.

Technical requirements

You can download the example project and code for this book by following the instructions in the **Download the example code files** section in the *Preface* of this book.

This chapter's code files are included in the downloadable code bundle.

What is caching?

Server-side caching is a technique used to store frequently accessed data in a temporary storage location, or cache, on the server. Caching reduces the time and resources required to retrieve data on subsequent requests. By minimizing the need to repeatedly fetch data from slower storage layers, such as databases or external APIs, server-side caching can significantly enhance the performance and scalability of web applications. Generally, we can categorize data into the following:

- **Hot storage:** This memory is quick to access, but it is also expensive. This is where the data is directly loaded into memory. We have a limited amount of live memory, so we must be sparing with it or be willing to spend a lot. Caches generally consist of hot storage.
- **Warm storage:** This is where the data is still instantly accessible, but it is stored on disk, resulting in a load that is still instant but slower than hot storage. Warm storage is cheaper, enabling us to store terabytes of data cheaply. Databases generally store most of the data on disk and optimize queries with some caching.
- **Cold storage:** This is where the storage becomes very cheap and reliable, but there is a delay in accessing the data. For instance, at home, you would consider cold storage to be storing your data on an optical disk or external hard drive and removing it from the computer. It will take more time to load it as you must get the storage device from its place and insert the storage device into your computer so your computer can access it. However, your storage device is not getting the daily wear and tear of running inside your computer. Cold storage is the best choice for data that is accessed infrequently. Cloud environments offer a more automated version of cold storage, where access to the data takes a while, and some cloud providers might charge per read. However, the long-term storage of untouched data in cloud cold storage services is very cheap. This is why old photos on a social media app might take longer to load, as these old photos might be stored in cold storage.

For caching, we generally use hot storage as we want to keep the access to the cached data quick. We are also not depending on our cache for permanent storage, so we do not need to worry if the cache is wiped. We can get the following benefits from caching:

- **Speed:** By serving data from a server-side cache, applications can respond to user requests much faster than if they had to retrieve the same data from a database or generate it dynamically. This is particularly important for high-traffic websites and applications where milliseconds matter.
- **Reduced load on backend systems:** Server-side caching reduces the number of direct interactions with the database or other backend systems. This not only speeds up response times but also alleviates the load on these systems, allowing them to perform more efficiently and scale better under high demand.
- **Cost efficiency:** Reducing the frequency of database queries and external API calls can lower operational costs, as these resources are often metered based on usage. Efficient server-side caching can lead to significant savings, especially for large-scale applications. Because it doesn't matter if the cache is wiped, you can get away with not having a managed service and just a Docker container for the cache. If the cache service is a paid-for managed service, it can still save money as external API calls can charge per request, depending on the API service and scaling databases.

- **Improved user experience:** Faster response times lead to a more responsive and engaging user experience. Users are less likely to abandon a slow application, improving retention and satisfaction rates.

We could simply use the RAM of our server as a cache; however, we want our system to be able to support multiple servers if needed. If data is cached in one server, and another server is hit, then we are not going to get the up-to-date cached data. Instead, we are going to need a detached data store. For this book, we are going to use Redis.

Setting up Redis

Redis is an open source, in-memory data structure store that acts as a database, cache, and message broker. Its primary advantage lies in its high speed, which is largely due to its in-memory architecture. We are using Redis for caching because of the following advantages:

- **In-memory storage:** By keeping all data in memory, Redis provides extremely fast read and write operations, achieving sub-millisecond response times. This makes it ideal for caching, where speed is crucial.
- **Single-threaded design:** Redis uses an event-driven, single-threaded architecture. This simplifies data access patterns and minimizes context switching, which enhances performance. The single-threaded model helps avoid the complexity of multithreading and race conditions, resulting in consistent performance.
- **Efficient data structures:** Redis supports a variety of data types, such as strings, hashes, lists, sets, and sorted sets. These structures are optimized for specific use cases, ensuring efficient data manipulation.
- **Horizontal scalability:** Redis can scale horizontally through sharding, allowing it to handle millions of requests per second, making it suitable for high-load applications.
- **Low latency:** The combination of in-memory data storage, optimized data structures, and a single-threaded model results in extremely low latency, enhancing user experience by speeding up response times.

To get our Redis database running, we could just reference the Redis image directly in our docker-compose and expose the port just like we do for our Postgres database. However, we are directly embedding our Rust code into the Redis database. We are, essentially, building our cache on top of the Redis Docker image. Therefore, we are going to build our cache module and client in our nanoservices directory with the following file structure:

```
└─ nanoservices
   │
   ├── auth
   │
   ├── to_do
   │
   └─ user-session-cache
      │
      ├── cache-client
      │   │
      │   ├── Cargo.toml
      │   │
      │   └─ src
      │       │
      │       └─ lib.rs
```

```

└─ cache-module
   └─ Cargo.toml
   └─ Dockerfile
   └─ src
      └─ lib.rs
      └─ processes
         └─ login.rs
         └─ logout.rs
         └─ mod.rs
         └─ update.rs
      └─ user_session.rs

```

Here, we can see that our cache-module has a Dockerfile. In our Dockerfile, we initially set the base Docker image, which is Rust, and install what we need to build the Rust Redis module with the following code:

```

# nanoservices/user-session-cache/cache-module/Dockerfile
FROM rust:latest as build
ENV PKG_CONFIG_ALLOW_CROSS=1

RUN apt-get update
RUN apt-get install libclang-dev -y

```

We then set the work directory inside the image, copy all our Rust code for the Redis module into the image, and build the Rust module with the following code:

```

# nanoservices/user-session-cache/cache-module/Dockerfile
WORKDIR /app
COPY . .
RUN cargo build --release

```

We then do a second layer in the Docker build where we just have the Redis image. We do not want all the excessive dependencies in the previous layer of the build, so we copy over the single binary that is the Redis module, expose the port, and finally, run the Redis server with the module that we built, using the following code:

```

# nanoservices/user-session-cache/cache-module/Dockerfile
FROM redis

COPY --from=build \
/app/target/release/libcache_module.so \
./libcache_module.so

EXPOSE 6379
CMD ["redis-server", "--loadmodule", "./libcache_module.so"]

```

Finally, we point to this build in our docker-compose file with the following code:

```
# docker-compose.yml
...
cache:
  container_name: 'to-do-redis'
  build: './nanoservices/user-session-cache/cache-module'
  restart: always
  ports:
    - '6379:6379'
```

If we try and run our docker-compose now, we will just get an error. Before we run our cache, we must build our Redis module.

Building our Redis module

Before we write any code for our Redis module, we must understand the problem that we are trying to solve with our Redis module. Allowing a token to make authenticated requests with no expiration can be dangerous. If a hacker gets hold of the token, then there will be no limit to the requests that they can make. To solve this, we are going to use a cache that caches user login sessions.

Each user login session will have a datetime of when the session was last interacted with. When we update the user session, we will increase a counter by 1, and update the last-interacted-with field to the time of the update. If the difference between the update and the last-interacted-with time exceeds a cutoff, we have a timeout. If the counter exceeds a cutoff, we have a suggestion that the token may need refreshing. We could bake the expiry time into the token; it would be a simple solution. However, not only does building a user session cache teach us about caching, but it also gives us more fine-grained control over the user session. For instance, we could have an admin server, and an admin user could choose to block a user. As soon as the admin user blocks the user, we could update the cache, instantly invalidating any requests after the blocking. If we did not have a cache and used an expiry time in the JWT, the user would still be able to make requests until the JWT expires and another token is needed.

To define our Redis module, we must have the following Cargo.toml file:

```
# nanoservices/user-session-cache/cache-module/Cargo.toml
[package]
name = "cache-module"
version = "0.1.0"
edition = "2021"

[lib]
crate-type = ["cdylib"]

[dependencies]
redis-module = "2.0.7"
```

```

chrono = "0.4.24"
bincode = "2.0.1"
serde = { version = "1.0.203", features = ["derive"] }

```

cdylib stands for C dynamic library. This type of library is intended to be used in languages other than Rust, such as C, C++, or even Python. As Redis is written in C, our library will have to be a C dynamic library to interact with Redis as C has a stable application binary and Rust does not. Redis will link with our dynamic C library, and we will be able to call functions in our dynamic C library from Redis.

Now that we have our Cargo.toml defined, we can define our Rust Redis module with the following code:

```

// nanoservices/user-session-cache/cache-module/src/lib.rs
use redis_module::redis_module;
mod processes;
mod user_session;
use processes::{
    login::login,
    update::update,
    logout::logout
};
redis_module! {
    name: "user_sessions",
    version: 1,
    allocator: (
        redis_module::alloc::RedisAlloc,
        redis_module::alloc::RedisAlloc
    ),
    data_types: [],
    commands: [
        ["login.set", login, "write fast deny-oom", 1, 1, 1],
        ["logout.set", logout, "write fast deny-oom", 1, 1, 1],
        ["update.set", update, "write fast deny-oom", 1, 1, 1],
    ]
}

```

Here, we can see that we have three commands. The 1 in each command denotes that the first key, last key, and key step are all set to 1. This means that all the commands work with a single key. write fast deny-oom means that we can write the key quickly, and that the write will be denied if the Redis cache runs out of memory.

To get these commands built out, we must take the following steps:

1. Defining the user session.
2. Building the login process.

3. Building the logout process.
4. Building the update process.

We can now move on to building out the user session.

Defining the user session

To define our user session, we must import the following:

```
// nanoservices/user-session-cache/cache-module/src/user_session.rs
use redis_module::{
    Context, RedisString, RedisError, RedisResult, RedisValue
};
use chrono::{DateTime, Utc, NaiveDateTime};
```

With this, our user session struct has the following definition:

```
// nanoservices/user-session-cache/cache-module/src/user_session.rs
pub struct UserSession {
    pub user_id: String,
    pub key: String,
    pub session_datetime: DateTime<Utc>,
}
```

Our user session then has the following functions associated:

```
// nanoservices/user-session-cache/cache-module/src/user_session.rs
impl UserSession {
    pub fn from_id(user_id: String)
    -> UserSession {
        . . .
    }
    pub fn check_timeout(&mut self, ctx: &Context)
    -> RedisResult {
        . . .
    }
    pub fn update_last_interacted(&self, ctx: &Context)
    -> RedisResult {
        . . .
    }
    pub fn get_counter(&self, ctx: &Context)
    -> RedisResult {
        . . .
    }
}
```

With these functions, we will be able to utilize our user session in our processes. You may have noticed that we pass in a reference to the `&Context` struct for our database functions. The context is our gateway to the in-memory data structures of the running Redis server. We must be able to construct a key from our user ID, so we can start with the `from_id` function:

```
// nanoservices/user-session-cache/cache-module/src/user_session.rs
pub fn from_id(user_id: String) -> UserSession {
    UserSession {
        user_id: user_id.clone(),
        key: format!("user_session_{}", user_id),
        session_datetime: Utc::now(),
    }
}
```

Here, we can see that we attach a `user_session_` prefix to the ID for the key. This means that we can have different inserts for different things associated with our user. For instance, after reading this chapter, you could build an items cache by prefixing `user_items_` to the user ID for the key, and your items will live in peace next to the auth session data without clashing.

We will now move on to our `check_timeout` function. This is going to be the most complex function for our session struct because the function gets the data of the session, calculates whether the time has elapsed, and updates the counter if the time has not elapsed. To do this, we initially get `RedisKeyWritable` from the Redis context with the following code:

```
// nanoservices/user-session-cache/cache-module/src/user_session.rs
let key_string = RedisString::create(None, self.key.clone());
let key = ctx.open_key_writable(&key_string);
```

The Redis context essentially enables us to interact with the Redis engine. For our cache, we are using the context to write, get, delete, and update data in the key-value store. Now that we have our key, we get the `last_interacted` field of the user authentication session and pass the string retrieved from the `last_interacted` field into a datetime with the following code:

```
// nanoservices/user-session-cache/cache-module/src/user_session.rs
let last_interacted_string = match key.hash_get("last_interacted")? {
    Some(v) => {
        match NaiveDateTime::parse_from_str(
            &v.to_string(), "%Y-%m-%d %H:%M:%S"
        ) {
            Ok(v) => v,
            Err(e) => {
                println!("Could not parse date: {:?}", e);
                return Err(RedisError::Str("Could not parse date"))
            }
        }
    }
}
```

```

    },
    None => return Err(
        RedisError::Str("Last interacted field does not exist")
    )
};

```

We can see that if we cannot find the field or fail to parse the datetime, we return appropriate errors.

Why are we parsing strings?

There is nothing stopping us from just serializing and deserializing our entire session struct in and out of bytes using bincode. This would make our functions a lot shorter. For instance, we could set and get our struct with the following code:

```

let serialized = bincode::serialize(value).unwrap();
ctx.call("SET", &[key, &serialized])
let result = ctx.call("GET", &[key])?;

```



However, this means that we would be storing raw bytes that are native to Rust in our Redis cache. What if you wanted to build a server in another language that wanted to get data from the cache? There is also a range of dashboards and database viewers that provide graphical user interfaces for the data in Redis. Here, you would want to be able to directly see the data that is in the Redis cache. If bincode serialization is enough for you, go for it. However, while we are learning about caching, it makes sense to learn the extra steps to make your data accessible. Storing data in databases in the form of language-specific bytes is usually quick and easy, but it does not come for free. In my experience, you do not want to scar your database with restrictive data formats.

Now that we have our last interaction datetime of the auth session, we can get the timeout minutes and calculate the time elapsed since the last interaction with the following code:

```

// nanoservices/user-session-cache/cache-module/src/user_session.rs
let timeout_mins = match key.hash_get("timeout_mins")? {
    Some(v) => v.to_string().parse::<i32>().unwrap(),
    None => return Err(
        RedisError::Str("Timeout mins field does not exist")
    )
};
let time_diff = self.session_datetime
    .naive_utc()
    .signed_duration_since(last_interacted_string)
    .num_minutes();

```

If the time elapsed is larger than the cutoff, we can then delete the entry and return a message that the session has timed out, using the following code:

```
// nanoservices/user-session-cache/cache-module/src/user_session.rs
if time_diff > timeout_mins.into() {
    match key.delete(){
        Ok(_) => {},
        Err(_) => return Err(
            RedisError::Str("Could not delete key")
        )
    };
    return Ok(RedisValue::SimpleStringStatic("TIMEOUT"));
}
```

We have now passed the timeout check; finally, we check the counter. The counter is a way of just forcing a refresh of the token. For instance, say a user loves our app and constantly uses it 24 hours a day for a week; it would not time out, and the user would be using the same JWT for a week. So, we get the counter, increase the counter by 1, and return a refresh by 1 message if the counter has exceeded a cutoff. If we pass the counter check, we merely return an OK message to tell the user that the auth check is all good, using the following code:

```
// nanoservices/user-session-cache/cache-module/src/user_session.rs
let mut counter = match self.get_counter(ctx)? {
    RedisValue::Integer(v) => v,
    _ => return Err(RedisError::Str("Could not get counter"))
};
counter += 1;
key.hash_set("counter", ctx.create_string(counter.to_string()));
if counter > 20 {
    return Ok(RedisValue::SimpleStringStatic("REFRESH"));
}
Ok(RedisValue::SimpleStringStatic("OK"))
```

Our most complex function is now done. We now must move on to `update_last_interacted`, where we update the `last_interacted` field. This is a good opportunity to try and build the function yourself. If you attempted to write the function yourself, hopefully it looks like the following code:

```
// nanoservices/user-session-cache/cache-module/src/user_session.rs
pub fn update_last_interacted(&self, ctx: &Context) -> RedisResult {
    let key_string = RedisString::create(None, self.key.clone());
    let key = ctx.open_key_writable(&key_string);
```

```

    let formatted_date_string = self.session_datetime.format(
        "%Y-%m-%d %H:%M:%S"
    ).to_string();
    let last_interacted_string = RedisString::create(
        None, formatted_date_string
    );
    key.hash_set("last_interacted", ctx.create_string(
        last_interacted_string
    ));
    Ok(RedisValue::SimpleStringStatic("OK"))
}

```

This might seem a little confusing, as we are not calling a datetime now. Here, we must think about the bigger context. We pass an ID to the Redis server when we are making a request. When the Redis server accepts the user ID, it must construct the user session from the ID with the `from_id` function. Therefore, in our processes, we can construct the session, make some checks or operations, and then finally, call our `update_last_interacted` function.

Finally, we must build our `get_counter` function. You can attempt to code this function yourself. If you do, hopefully it looks like the following code:

```

// nanoservices/user-session-cache/cache-module/src/user_session.rs
pub fn get_counter(&self, ctx: &Context) -> RedisResult {
    let key_string = RedisString::create(None, self.key.clone());
    let key = ctx.open_key_writable(&key_string);
    match key.hash_get("counter")? {
        Some(v) => {
            let v = v.to_string().parse::<i64>().unwrap();
            Ok(RedisValue::Integer(v))
        },
        None => Err(RedisError::Str(
            "Counter field does not exist"
        ))
    }
}

```

Our session struct is now complete. We can now move on to building our processes, starting with our login.

Building the login process

For our login process, we must have the Redis context and arguments passed in via the Redis command and return an OK message if everything is good. Considering the steps, the login process has the following outline:

```
// nanoservices/user-session-cache/cache-module/src/
// processes/login.rs
use redis_module::{
    Context, NextArg, RedisError, RedisResult, RedisString, RedisValue
};
use crate::user_session::UserSession;

pub fn login(ctx: &Context, args: Vec<RedisString>) -> RedisResult {
    . . .
    Ok(RedisValue::SimpleStringStatic("OK"))
}
```

At the start of the login function, we process our arguments passed in with the following code:

```
// nanoservices/user-session-cache/cache-module/src/
// processes/login.rs
if args.len() < 4 {
    return Err(RedisError::WrongArity);
}
let mut args = args.into_iter().skip(1);
let user_id = args.next_arg()?.to_string();
let timeout_mins = args.next_arg()?;
let perm_user_id = args.next_arg()?.to_string();
```

If we have fewer than four commands, then we do not have enough. We skip the first argument because this will be the login command. We then get the user ID and the length of the timeout in minutes. With our arguments, we can then construct our session, obtain the key, and write the last-interacted-with time to the key-value store with the following code:

```
// nanoservices/user-session-cache/cache-module/src/
// processes/login.rs
let user_session = UserSession::from_id(user_id);
user_session.update_last_interacted(ctx)?;
let key_string = RedisString::create(None, user_session.key);
```

Finally, we write the timeout minutes, permanent user ID, and counter to the key-value store with the following code:

```
// nanoservices/user-session-cache/cache-module/src/
```

```
// processes/login.rs
let key = ctx.open_key_writable(&key_string);
key.hash_set("timeout_mins", ctx.create_string(timeout_mins));
key.hash_set("counter", ctx.create_string("0"));
key.hash_set("perm_user_id", ctx.create_string(perm_user_id));
```

And with this, our login process is done. We can now move on to our logout process.

Building the logout process

For our logout process, we have the following outline:

```
// nanoservices/user-session-cache/cache-module/src/
// processes/logout.rs
use redis_module::{
    Context, NextArg, RedisError,
    RedisResult, RedisString, RedisValue
};
use crate::user_session::UserSession;

pub fn logout(ctx: &Context, args: Vec<RedisString>)
    -> RedisResult {
    . . .
    Ok(RedisValue::SimpleStringStatic("OK"))
}
```

Inside our logout function, we process the arguments with the following code:

```
// nanoservices/user-session-cache/cache-module/src/
// processes/logout.rs
if args.len() < 2 {
    return Err(RedisError::WrongArity);
}
let mut args = args.into_iter().skip(1);
let user_id = args.next_arg()?.to_string();
```

We then construct the key and session with the following code:

```
// nanoservices/user-session-cache/cache-module/src/
// processes/logout.rs
let user_session = UserSession::from_id(user_id);
let key_string = RedisString::create(None, user_session.key);
let key = ctx.open_key_writable(&key_string);
```

Finally, we delete the session from the key-value store with the following code:

```
// nanoservices/user-session-cache/cache-module/src/
// processes/logout.rs
if key.is_empty() {
    return Ok(RedisValue::SimpleStringStatic("NOT_FOUND"));
}
match key.delete() {
    Ok(_) => {},
    Err(_) => return Err(RedisError::Str("Could not delete key"))
};
```

With our logout now defined, we can wrap up our cache module with the update command.

Building the update process

For our update process, we have the following code:

```
// nanoservices/user-session-cache/cache-module/src/
// processes/update.rs
use redis_module::{
    Context, NextArg, RedisError, RedisResult,
    RedisString, RedisValue
};
use crate::user_session::UserSession;

pub fn update(ctx: &Context, args: Vec<RedisString>)
    -> RedisResult {
    . . .
}
```

Inside our update function, we process the arguments with the following code:

```
// nanoservices/user-session-cache/cache-module/src/
// processes/update.rs
if args.len() < 2 {
    return Err(RedisError::WrongArity);
}
let mut args = args.into_iter().skip(1);
let user_id = args.next_arg()?.to_string();
```

We then define the user session with the following code:

```
// nanoservices/user-session-cache/cache-module/src/
// processes/update.rs
let mut user_session = UserSession::from_id(user_id);
```



```

let key_string = RedisString::create(None, user_session.key.clone());
let key = ctx.open_key_writable(&key_string);
if key.is_empty() {
    return Ok(RedisValue::SimpleStringStatic("NOT_FOUND"));
}

```

Finally, we check the timeout and handle the outcome with the following code:

```

// nanoservices/user-session-cache/cache-module/src/
// processes/update.rs
match &user_session.check_timeout(ctx)? {
    RedisValue::SimpleStringStatic("TIMEOUT") => {
        return Ok(RedisValue::SimpleStringStatic("TIMEOUT"));
    },
    RedisValue::SimpleStringStatic("REFRESH") => {
        user_session.update_last_interacted(ctx)?;
        return Ok(RedisValue::SimpleStringStatic("REFRESH"));
    },
    RedisValue::SimpleStringStatic("OK") => {
        user_session.update_last_interacted(ctx)?;
        let perm_user_id = match key.hash_get("perm_user_id")? {
            Some(perm_user_id) => perm_user_id,
            None => {
                return Err(RedisError::Str(
                    "Could not get perm_user_id"
                ));
            }
        };
        return Ok(RedisValue::SimpleString(perm_user_id.to_string()));
    },
    _ => {
        return Err(RedisError::Str("Could not check timeout"));
    }
};

```

We can see that we still update the last-interacted-with time even if a refresh is returned. This is because we want to decouple the refresh mechanism from the timeout mechanism. If a developer wants to keep servicing JWTs when they need to be refreshed, then they can do so. If they do not want the session to ever timeout, they can set the timeout time for a year or so. With the update now completed, we can now say that our caching module is complete, and we can move on to building out our client.

Building our Redis client

We do not know what the future will hold for our system. When developing web systems, our requirements will change as the problem evolves. Therefore, we have no way of knowing what servers will need to access the cache. Therefore, it makes sense to build a client that is accessible to any Rust server that needs it. For our client, we only need to make a connection to Redis in an async manner and return appropriate errors if needed. With these requirements in mind, the `Cargo.toml` file for our cache client takes the following form:

```
// nanoservices/user-session-cache/cache-client/Cargo.toml
[package]
name = "cache-client"
version = "0.1.0"
edition = "2021"

[dependencies]
redis = { version = "0.25.4", features = ["tokio-comp"] }
tokio = { version = "1.38.0", features = ["full"] }
glue = { path = "../..../glue" }
```

Before we build any of our functions for our cache processes, we must build a connection and string handle function, as we will use these functions for the processes. For our `lib.rs` file, which houses all our functions, we need the following imports:

```
// nanoservices/user-session-cache/cache-client/src/lib.rs
use std::error::Error;
use redis::aio::{ConnectionLike, MultiplexedConnection};
use redis::Value;
use glue::errors::{NanoServiceError, NanoServiceErrorStatus};
```

The definition of our get connection function takes the following form:

```
// nanoservices/user-session-cache/cache-client/src/lib.rs
async fn get_connection(address: &str)
-> Result<MultiplexedConnection, NanoServiceError> {
    let client = redis::Client::open(address).map_err(|e|{
        NanoServiceError::new(
            e.to_string(),
            NanoServiceErrorStatus::Unknown
        )
    })?;
    let con = client.get_multiplexed_async_connection()
        .await
        .map_err(|e|{
```

```

        NanoServiceError::new(
            e.to_string(),
            NanoServiceErrorStatus::Unknown
        )
    })?;
    Ok(con)
}

```

We can then handle our strings from the response of the Redis cache with the following code:

```

// nanoservices/user-session-cache/cache-client/src/lib.rs
fn unpack_result_string(result: Value)
-> Result<String, NanoServiceError> {
    match result {
        Value::Status(s) => Ok(s),
        _ => Err(NanoServiceError::new(
            "Error converting the result into a string".to_string(),
            NanoServiceErrorStatus::Unknown
        ))
    }
}

```

With this, we are now ready to build our login function.

Building the login/logout client

Our login function takes the following signature:

```

// nanoservices/user-session-cache/cache-client/src/lib.rs
pub async fn login(
    address: &str,
    user_id: &str,
    timeout_mins: usize,
    perm_user_id: i32
) -> Result<(), NanoServiceError> {
    . . .
    Ok(())
}

```

Inside our login function, we get the connection and send the request with the following code:

```

// nanoservices/user-session-cache/cache-client/src/lib.rs
let mut con = get_connection(address).await?;
let result = con
    .req_packed_command(

```

```

        &redis::cmd("login.set")
            .arg(user_id)
            .arg(timeout_mins)
            .arg(perm_user_id.to_string())
            .clone(),
    )
    .await.map_err(|e|{
        NanoServiceError::new(
            e.to_string(),
            NanoServiceErrorStatus::Unknown
        )
    })?;
match result {
    Value::Okay => {
        return Ok(());
    },
    _ => {
        return Err(NanoServiceError::new(
            format!("{:?}", result),
            NanoServiceErrorStatus::Unknown
        ));
    }
}
}

```

We then match the result and return an error if we do not get a value::Okay. With this, we can now log in to our Redis cache. For our logout function, the approach is the same as the login function, using the following code:

```

// nanoservices/user-session-cache/cache-client/src/lib.rs
pub async fn logout(address: &str, user_id: &str)
-> Result<String, Box<dyn Error>> {
    let mut con = get_connection(address).await?;
    let result = con
        .req_packed_command(
            &redis::cmd("logout.set")
                .arg(user_id)
                .clone(),
        )
        .await.map_err(|e|{
            NanoServiceError::new(
                e.to_string(),
                NanoServiceErrorStatus::Unknown
            )
        })
    };
    match result {
        Value::Okay => {
            return Ok(String::new());
        },
        _ => {
            return Err(Box::new(NanoServiceError::new(
                format!("{:?}", result),
                NanoServiceErrorStatus::Unknown
            )));
        }
    }
}

```

```

    )
  }}?;
  let result_string = unpack_result_string(result)?;
  Ok(result_string)
}

```

Now we only have the update client to build.

Building the update client

Our update function takes the following signature:

```

// nanoservices/user-session-cache/cache-client/src/lib.rs
#[derive(Debug)]
pub enum UserSessionStatus {
    Ok(i32),
    Refresh
}
pub async fn update(address: &str, user_id: &str)
-> Result<UserSessionStatus, NanoServiceError> {
    let mut con = get_connection(address).await?;
    . . .
}

```

The command is defined with the following code:

```

// nanoservices/user-session-cache/cache-client/src/lib.rs
let result = con
    .req_packed_command(
        &redis::cmd("update.set")
            .arg(user_id)
            .clone(),
    )
    .await.map_err(|e|{
        NanoServiceError::new(
            e.to_string(),
            NanoServiceErrorStatus::Unknown
        )
    })?;

```

With the result, we can inspect the string, returning a status depending on the response string with the following code:

```

// nanoservices/user-session-cache/cache-client/src/lib.rs
let result_string = unpack_result_string(result)?;

```

```
match result_string.as_str() {
    "TIMEOUT" => {
        return Err(NanoServiceError::new(
            "Session has timed out".to_string(),
            NanoServiceErrorStatus::Unauthorized
        ));
    },
    "NOT_FOUND" => {
        return Err(NanoServiceError::new(
            "Session not found".to_string(),
            NanoServiceErrorStatus::Unauthorized
        ));
    },
    "REFRESH" => {
        return Ok(UserSessionStatus::Refresh)
    },
    _ => {}
}
```

Finally, we try and parse the response to an integer and return the user ID with the following code:

```
// nanoservices/user-session-cache/cache-client/src/lib.rs
let perm_user_id = match result_string.parse::<i32>() {
    Ok(perm_user_id) => perm_user_id,
    Err(_) => {
        return Err(NanoServiceError::new(
            "Error converting the result into a string".to_string(),
            NanoServiceErrorStatus::Unknown
        ));
    }
};
Ok(UserSessionStatus::Ok(perm_user_id))
```

With this, our Redis client is now ready. Finally, we can use our Redis cache by using our client on our servers.

Connecting our cache to our servers

We currently have the raw functions that interact with our cache. However, we are going to utilize a procedure for verifying a user request throughout our system on any server. I have chosen to put our interface for checking the user session for the request in the auth kernel with the following file structure:

```

└─ nanoservices
   └─ auth
      └─ kernel
         └─ Cargo.toml
            └─ src
               └─ api
                  └─ . . .
                     └─ lib.rs
                        └─ user_session
                           └─ descriptors.rs
                              └─ mod.rs
                                 └─ schema.rs
                                    └─ transactions
                                       └─ get.rs
                                          └─ mod.rs

```

The user session depends on the user database model and logging in the user for a user session. Therefore, it makes sense to put the interface for the user session cache in the auth server. It can be argued that the cache interface could be put in the data access layer for the auth server. At this point, it is down to personal choice. I stuck with the kernel because the kernel is the workspace that other servers compile if they want to interact with the auth server. Putting it in the kernel also means that the other server does not have to compile auth database code that might not be meant for public consumption. Whatever choice we go for, we can see that our layout for the user session has schemas, transactions, and descriptors. This is an approach that we keep consistent with our IO interactions. Therefore, we could easily lift this code into another workspace if we needed to. Also, a developer is not going to get confused when they come across the cache module. With this structure, we can move on to building our cache kernel.

Building our cache kernel

For our `Cargo.toml` file, we add the following dependency:

```

# nanoservices/auth/kernel/Cargo.toml
[dependencies]
. . .
cache-client = { path = "../user-session-cache/cache-client" }

```

Because we are now so used to this pattern, we can rifle through our initial setup. You might even be able to configure this yourself. If you have tried to configure the user session module, you should have the following:

Your `lib.rs` file:

```

// nanoservices/auth/kernel/src/lib.rs
pub mod api;

```

```
#[cfg(any(feature = "core-postgres", feature = "http"))]
pub mod user_session;
```

The `user_session` module is public under the features `auth-core` and `request` because we are going to rely on getting the user from the database. Getting a user from the database requires either the `auth-core` or `request` feature. The `mod.rs` file of the user session module:

```
// nanoservices/auth/kernel/src/user_session/mod.rs
pub mod transactions;
pub mod descriptors;
pub mod schema;
```

The `descriptors.rs` file:

```
// nanoservices/auth/kernel/src/user_session/descriptors.rs
pub struct RedisSessionDescriptor;
```

The `schema.rs` file:

```
// nanoservices/auth/kernel/src/user_session/schema.rs
pub struct UserSession {
    pub user_id: i32
}
```

The `mod.rs` file of the transaction module:

```
// nanoservices/auth/kernel/src/user_session/transactions/mod.rs
pub mod get;
```

Finally, the setup code for our `get` trait takes the following form:

```
// nanoservices/auth/kernel/src/user_session/transactions/get.rs
use std::future::Future;
use crate::user_session::schema::UserSession;
use glue::errors::{NanoServiceError, NanoServiceErrorStatus};
use cache_client::{update, UserSessionStatus, login};
use crate::api::users::get::get_user_by_unique_id;
use crate::user_session::descriptors::RedisSessionDescriptor;

pub trait GetUserSession {
    fn get_user_session(unique_id: String)
    -> impl Future<Output = Result<UserSession, NanoServiceError>>;
}
```


For our Redis descriptor, we implement the `GetUserSession` trait with the following code:

```
// nanoservices/auth/kernel/src/user_session/transactions/get.rs
impl GetUserSession for RedisSessionDescriptor {
    fn get_user_session(unique_id: String)
    -> impl Future<Output = Result<UserSession, NanoServiceError>> {
        get_session_redis(unique_id)
    }
}
```

The function in our implementation takes the following signature:

```
// nanoservices/auth/kernel/src/user_session/transactions/get.rs
pub async fn get_session_redis(unique_id: String)
    -> Result<UserSession, NanoServiceError> {
    . . .
}
```

The `get_session_redis` function is going to be fired for every authorized request that we process. First, we get the URL of the Redis server and call the cache update function, with the following code:

```
// nanoservices/auth/kernel/src/user_session/transactions/get.rs
let address = std::env::var("CACHE_API_URL").map_err(|e|{
    NanoServiceError::new(
        e.to_string(),
        NanoServiceErrorStatus::BadRequest
    )
})?;
let user_id = update(&address, &unique_id).await?;
```

We then unpack the result. If the result is OK, we return the `UserSession` with the user ID. However, if a refresh is needed, we merely get the user by the unique ID from the auth server and call the `login` function for the cache to reset the session. This approach essentially bypasses the refresh mechanism, ensuring that the JWT never expires. But the user session can still time out due to inactivity. We bypassed the JWT refreshing to avoid bloating the chapter. If you want to enforce a refresh mechanism, you must carry out the following steps:

1. Create an endpoint in the auth server to receive a JWT, update the unique ID associated with the user in the database, and return the new unique ID.
2. Call this refresh endpoint if a refresh is returned from the cache.
3. Process the rest of the request.
4. Return the result of the response with the refreshed token in the header of the request.
5. Update the frontend HTTP requests to inspect the headers of responses for the refresh token, updating the local storage with the token if the token is present in the header.

For this book, the handling of the return from the `get_session_redis` cache function is carried out by the following code:

```
// nanoservices/auth/kernel/src/user_session/transactions/get.rs
match user_id {
  UserSessionStatus::Ok(id) => Ok(UserSession { user_id: id }),
  UserSessionStatus::Refresh => {
    let user = get_user_by_unique_id(
      unique_id.clone()
    ).await?;
    let _ = login(&address, &unique_id, 20, user.id).await?;
    match user_id {
      UserSessionStatus::Ok(id) => Ok(UserSession {
        user_id: id
      }),
      _ => Err(NanoServiceError::new(
        "Failed to update user session".to_string(),
        NanoServiceErrorStatus::Unknown)
    )
  }
}
```

Our kernel is now complete. We can now move on to calling the kernel in our server.

Calling the kernel from our to-do server

For our server, we must pass in the `GetUserSession` trait and call it within the view with the following code:

```
// nanoservices/to_do/networking/actix_server/
// src/api/basic_actions/create.rs
...
use auth_kernel::user_session::transactions::get::GetUserSession;

pub async fn create<T, X>(
  token: HeaderToken,
  body: Json<NewToDoItem>
) -> Result<HttpResponse, NanoServiceError>
where
  T: SaveOne + GetAll,
  X: GetUserSession
{
  let session = X::get_user_session(
```

```

        token.unique_id
    ).await?;
    let _ = create_core::

```

It should not be a surprise that all the other actions in the API for the to-do server follow suit. The delete view is redefined with the following code:

```

// nanoservices/to_do/networking/actix_server/
// src/api/basic_actions/delete.rs
. . .
use auth_kernel::user_session::transactions::get::GetUserSession;

pub async fn delete_by_name<T, X>(
    token: HeaderToken,
    req: HttpRequest
) -> Result<HttpResponse, NanoServiceError>
where
    T: DeleteOne + GetAll,
    X: GetUserSession
{
    let session = X::get_user_session(token.unique_id).await?;
    . . .
    Ok(HttpResponse::Ok().json(
        get_all_core::

```

It should also be noted that the `delete_core` function has `session.user_id` passed into the function. The get endpoint should look like the following:

```

// nanoservices/to_do/networking/actix_server/
// src/api/basic_actions/get.rs
. . .
use auth_kernel::user_session::transactions::get::GetUserSession;

```

```
pub async fn get_all<T, X>(token: HeaderToken)
-> Result<HttpResponse, NanoServiceError>
where
    T: GetAll,
    X: GetUserSession
{
    let session = X::get_user_session(token.unique_id).await?;
    Ok(HttpResponse::Ok().json(
        get_all_core::<T>(session.user_id).await?)
    )
}
```

Finally, the update endpoint is redefined by the following code:

```
// nanoservices/to_do/networking/actix_server/
// src/api/basic_actions/update.rs
...
use auth_kernel::user_session::transactions::get::GetUserSession;

pub async fn update<T, X>(
    token: HeaderToken,
    body: Json<ToDoItem>
) -> Result<HttpResponse, NanoServiceError>
where
    T: UpdateOne + GetAll,
    X: GetUserSession
{
    let session = X::get_user_session(token.unique_id).await?;
    let _ = update_core::<T>(
        body.into_inner(), session.user_id
    ).await?;
    Ok(HttpResponse::Ok().json(
        get_all_core::<T>(session.user_id).await?
    ))
}
```

With all our endpoints updated, we pass the Redis descriptor into the views that are defined in the views factory with the following code:

```
// nanoservices/to_do/networking/actix_server/
// src/api/basic_actions/mod.rs
...
```

```

use auth_kernel::user_session::descriptors::RedisSessionDescriptor;

pub fn basic_actions_factory(app: &mut ServiceConfig) {
    app.service(
        scope("/api/v1")
        .route("get/all", get()).to(
            get::get_all::<
                SqlxPostGresDescriptor,
                RedisSessionDescriptor
            >)
        )
        .route("create", post()).to(
            create::create::<
                SqlxPostGresDescriptor,
                RedisSessionDescriptor
            >)
        )
        .route("delete/{name}", delete()).to(
            delete::delete_by_name::<
                SqlxPostGresDescriptor,
                RedisSessionDescriptor
            >)
        )
        .route("update", put()).to(
            update::update::<
                SqlxPostGresDescriptor,
                RedisSessionDescriptor
            >)
        )
    );
}

```

This is now done. We can sit back and appreciate what is going on here. Although there are a lot of minor changes in several files, we have introduced a cache that checks the user session. Because of the way our code is structured, our changes easily slot in. We do not have to rip out and change huge chunks of code. Also, the complexity is contained. To really see the flexibility of our approach, you can see how the same approach of injecting the Redis handle also works for Axum and Hyper.

Also, you can look at the networking workspace and see how the HTTP request is handled throughout the life cycle of a request because the core logic is all abstracted away. If you want to use a file, another database, or just the memory of the server for the cache, you can just implement another descriptor and slot the descriptor in easily. If we wanted to do this, we could use features, as seen in the following code:

```
#[cfg(feature = "cache-postgres")]
use auth_kernel::user_session::
descriptors::PostgresSessionDescriptor as CacheDescriptor;
#[cfg(feature = "cache-redis")]
use auth_kernel::user_session::
descriptors::RedisSessionDescriptor as CacheDescriptor;
. . .
get::get_all::<
    SqlxPostGresDescriptor,
    CacheDescriptor
>)
```

We are nearly finished with connecting our user session cache to our system. All we need now is to call our auth kernel from our auth server as we need to create the user session in the cache when logging the user in.

Calling the kernel from our auth server

To call our kernel, we must now add the core feature because our cache interface relies on getting the user in the core. This means that our auth `Cargo.toml` must be updated with the following:

```
# nanoservices/auth/networking/actix_server/Cargo.toml
. . .
[dependencies]
. . .
auth-kernel = {
    path = "../kernel",
    features = ["auth-core"],
    default-features = false
}
```

Now, if we had a less structured approach, where we combine the HTTP code with the core code in one app, there would be a circular dependency. This circular dependency would arise from the auth kernel pulling from the auth server, and then the auth server requiring the auth kernel for the Redis interface. However, we have kept our system isolated and concerned with only the scope the system is built for. This has led to the layout shown in *Figure 12.1*.

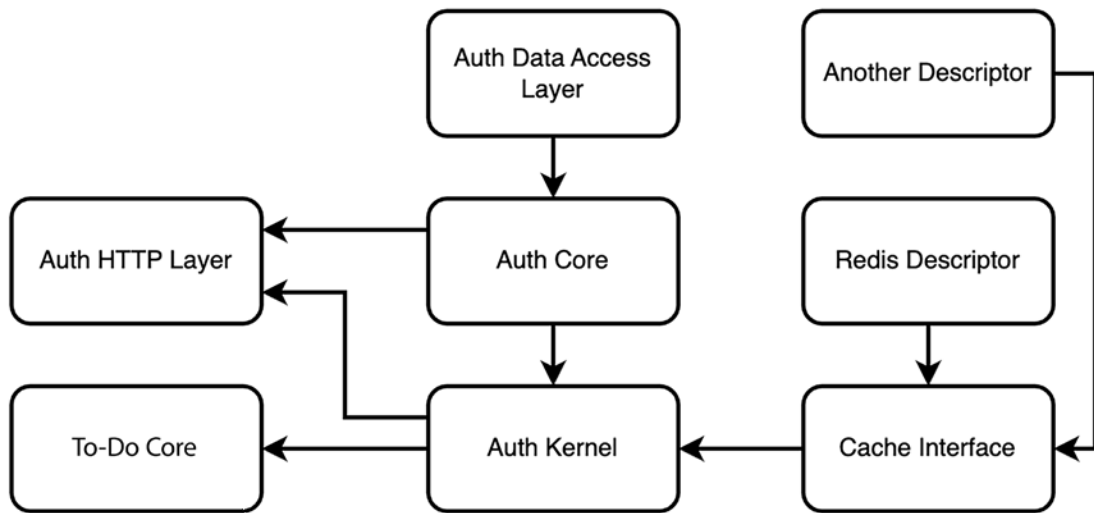


Figure 12.1 – Our auth dependency graph

This is where the value of our structure is strongly demonstrated. As you build a more complex system, you do not know what will arise. However, our auth server has a Redis cache and a Postgres database to handle the auth management. Adding connecting other servers is as simple as connecting to the kernel. If we wanted to bring the cache check down to the core and no longer expose the cache to other services, it would be as simple as changing a few lines of code per API endpoint to achieve this.

Now that we have our layout, we can build our login transaction for our kernel. First, we import the following:

```
// nanoservices/auth/kernel/src/user_session/transactions/Login.rs
use std::future::Future;
use glue::errors::NanoServiceError;
use crate::user_session::descriptors::RedisSessionDescriptor;
use cache_client::login as cache_login;
```

We then define our trait for logging in a user session with the following code:

```
// nanoservices/auth/kernel/src/user_session/transactions/Login.rs
pub trait LoginUserSession {
    fn login_user_session(
        address: &str,
        user_id: &str,
        timeout_mins: usize,
        perm_user_id: i32
    )
    -> impl Future<Output = Result<(), NanoServiceError>>;
}
```

Finally, we implement this trait for the Redis descriptor that just calls the `login` function from the Redis interface, using the following code:

```
// nanoservices/auth/kernel/src/user_session/transactions/login.rs
impl LoginUserSession for RedisSessionDescriptor {

    fn login_user_session(
        address: &str,
        user_id: &str,
        timeout_mins: usize,
        perm_user_id: i32
    )
    -> impl Future<Output = Result<(), NanoServiceError>> {
        cache_login(
            address,
            user_id,
            timeout_mins,
            perm_user_id
        )
    }
}
```

Of course, we ensure that this trait is available to the rest of the kernel with the following code:

```
// nanoservices/auth/kernel/src/user_session/transactions/mod.rs
pub mod get;
pub mod login;
```

We then configure our login view to interact with the cache. First, we import the `LoginUserSession` trait with the following code:

```
// nanoservices/auth/networking/actix_server/src/
// api/auth/login.rs
...
use auth_kernel::user_session::transactions::login::
LoginUserSession;
```

Our login function then takes the following code:

```
// nanoservices/auth/networking/actix_server/src/
// api/auth/login.rs
pub async fn login<T, X>(req: actix_web::HttpRequest)
-> Result<HttpResponse, NanoServiceError>
where
    T: GetByEmail,
```



```

X: LoginUserSession
{
  . . .
  Ok(HttpResponse::Ok().json(token))
}

```

Inside our login function, we get the credentials, token, and user with the following code:

```

// nanoservices/auth/networking/actix_server/src/
// api/auth/Login.rs
let credentials = extract_credentials(req).await?;
let token = core_login::<T>(
    credentials.email.clone(),
    credentials.password
).await?;
let user = T::get_by_email(credentials.email).await?;

```

We then get our Redis URL and log in to the user session with the following code:

```

// nanoservices/auth/networking/actix_server/src/
// api/auth/Login.rs
let url = std::env::var("CACHE_API_URL").map_err(|e|{
    NanoServiceError::new(
        e.to_string(),
        NanoServiceErrorStatus::Unknown
    )
})?;
let _ = X::login_user_session(
    &url,
    &user.unique_id,
    20,
    user.id
).await?;

```

Our system is now ready with a cache. All we must do now is add the Redis URL to the `.env` file, giving us the following contents:

```

# nanoservices/to_do/.env
TO_DO_DB_URL=postgres://username:password@localhost/to_do
AUTH_DB_URL=postgres://username:password@localhost/to_do
AUTH_API_URL=http://127.0.0.1:8081
JWT_SECRET=secret
CACHE_API_URL=redis://127.0.0.1:6379

```

We can then run our `nanoservices/to_do/scripts/test_server_com.sh` file and our create test will work. It must be noted that your development pace is probably speeding up due to you relying on the `test_server_com.sh` script to test the new feature. This is because you do not have to perform any manual steps to set up the databases, run the migrations, and then log in. This is the power of testing and test-driven development. My editors tell me that the testing chapters of any book are the least read. Trust me, the testing chapters are some of the most important chapters to read once you can get a basic web application up and running. The first benefit is the speed of development. Most of my achievements are down to writing good unit tests. Running an isolated test that has automated all the steps needed to test the code you are building saves you from running those manual steps again and again throughout the day of coding. It also helps you pick up bugs so you're not firefighting later, and when you refactor, you just run the tests every time you make a small change because they're so quick and easy to run. You then know the instant you introduce some breaking code, because the effort threshold of running all the tests is so low that you do it frequently. So, no rooting through the code base to work out what change created the break. If you have a good unit and end-to-end testing strategy, then you will develop better-quality code at a much faster rate than the average developer because the average developer just hasn't taken the time to refine their testing technique. Instead, they are performing multiple manual steps to test one edge case of their code. It frankly blows my mind that everyone agrees that manual steps for deployment and packaging are bad, and that we should automate them. Yet, these same developers will push back on unit testing, meaning they're OK with manual steps for checking their code once they've coded it.

Summary

In this chapter, we essentially created a Redis module so we have a custom cache, and then we implemented that cache into our system so multiple servers could interact with it. It must be noted that our cache enabled several checks and updates in just one call to the Redis cache, as opposed to multiple different requests to the cache. This is a very powerful skill that you have just developed that will come in handy in different situations. We also got a feel for how useful tests are when developing our code with our test script. At this stage, you are technically familiar with Rust and the web ecosystem to build your own applications. However, I must stress that you will be much more productive once you have completed the unit testing and end-to-end testing chapters.

In the next chapter, we will explore observability and how to trace our async tasks and HTTP requests.

Questions

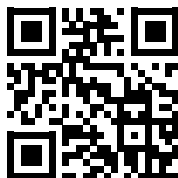
1. At a high level, how did we build a Redis module in Rust?
2. How did we get the Docker build to house our module?
3. How did we integrate our cache into the HTTP layer of our servers?
4. What are the advantages of our approach to integrating the cache?
5. Let us say that we built an email server, and we want to check the auth session before sending a particular email. At a high level, how could this be done?

Answers

1. We used the Redis crate and macros to define the Redis module. We then compiled our Redis module as a dynamic C library, and then got our Redis server to load the module on startup. We could use our module via the commands we specified in the module when making a request to the Redis server.
2. We copied over the code of our Redis module into a Rust base image. We then built the Redis module. We then moved over to the second stage of the build, which is the Redis image, and copied over our compiled module to the Redis image, but nothing else. We then ran the Redis server with our module loaded.
3. We created traits for the interaction with the cache. We then implemented a Redis handle for these traits, and then mounted those traits to the server views.
4. The main advantage is that we can implement other handles for different storage mechanisms. This means that we can easily unit test our code with test implementations of the cache. It also makes it easy to move the implementation of our cache interface around our system with a clean interface.
5. The email server must define the auth kernel as a dependency, and then use the get user session function from the kernel to check the user session before sending the email, and that is it.

Learn more on Discord

To join the Discord community for this book – where you can share feedback, ask questions to the author, and learn about new releases – follow this QR code: <https://packt.link/EaKXL>



13

Observability Through Logging

We now have a working to-do application that can either run as a single binary or run as multiple servers. However, we do not actually know what is going on inside our system. Let us say that our system makes a request from one server to another. How do we know that this request was made and what the response was? We don't. We can try and work out what happened from the error message returned to the frontend, but this might not be clear. We also might not want to expose intricate details of the error to the frontend. To remedy this, we can produce logs of these requests and how the request travels through the system. This also gives us the power to inspect the steps that lead up to the error. Logging also enables us to keep an eye on the health of the system. In this chapter, we will cover the following:

- What are RESTful services?
- What is logging?

By the end of this chapter, you will be able to log what is going on in your program, including all the request and response code, by implementing middleware for our logger. You will also learn how to create background tasks that receive log messages from the main application and persist them to a database asynchronously, reducing the workload and latency of the main request-handling path. Finally, we can perform queries on our Elasticsearch database to look for logs.

Technical requirements

You can download the example project and code for this book by following the instructions in the **Download the example code files** section in the *Preface* of this book.

This chapter's code files are included in the downloadable code bundle.

What are RESTful services?

Representational state transfer (REST) is an architectural style for our **application programming interface (API)** to read (GET), update (PUT), create (POST), and delete (DELETE) our users and to-do items. The goal of a RESTful approach is to increase speed/performance, reliability, and the ability to grow by reusing components that can be managed and updated without affecting the system.

You may have noticed that before Rust, slow, high-level languages seemed to be a wise choice for web development. This is because they are quicker and safer to write. This is due to the main bottleneck for the speed of processing data in web development being the network connection speed. The RESTful design aims to improve the speed by economizing the system, such as reducing API calls, as opposed to just focusing on algorithm speed. With that in mind, in this section, we can explore the following RESTful concepts:

- **Layered system:** This enables us to add extra functionality, such as authorization, without having to change the interface.
- **Uniform system:** This simplifies and decouples the architecture, enabling whole parts of the application to evolve independently without clashing.
- **Statelessness:** This ensures that our application does not directly save anything on the server. This has implications for microservices and cloud computing.
- **Logging:** This enables us to peek into our application and see how it runs, exposing undesirable behavior even if there are no errors displayed.
- **Caching:** This enables us to store data in hot memory to reduce the number of calls to a database.
- **Code on demand:** This is where our backend server directly runs code on the frontend.

Here, we can see that we have been implementing all the previous concepts as we have gone along when building our to-do app, apart from the code-on-command concept and logging. We can start with code on command.

Building frontend code on command

Code on demand is where the backend server directly executes code on the frontend. This constraint is optional and not widely used. However, it can be useful as it gives the backend server the right to decide when code is executed on the frontend. Getting JavaScript to run in the browser from our server when the API endpoint is called can be achieved by sending a script in the HTML that we return. We can do this with the following code:

```
// nanoservices/auth/networking/actix_server/  
// src/auth/logout.rs  
use actix_web::HttpResponse;  
  
pub async fn logout() -> HttpResponse {  
    HttpResponse::Ok()  
        .content_type("text/html; charset=utf-8")  
        .body(  
            "<html>\n  
            <script>\n  
            localStorage.removeItem('token'); \n  
            window.location.replace(\n  
            document.location.origin);\n  
            </script>\n
```

```

        </html>"
    )
}

```

Here, we can see that we are removing the user token from `localStorage` and then refreshing the window afterward to be rerouted to the login form. We now must add our logout API endpoint to our factory with the following code:

```

// nanoservices/auth/networking/actix_server/
// src/auth/mod.rs
. . .
pub mod logout;
. . .
pub fn auth_factory(app: &mut ServiceConfig) {
    app.service(
        scope("/api/v1/auth")
        . . .
        .route("logout", get().to(
            logout::logout)
        )
    );
}

```

This is the easiest way to force a logout, as the frontend browser does not have access to the **JSON Web Token (JWT)** and therefore cannot make any more authenticated HTTP requests.

While our running code in the browser does work, it will leave a dangling entry in our cache. If we want to remove the user session from the cache, we will have to make an HTTP request with the user ID. At this stage in the book, you should be able to add this API endpoint. Covering the addition of this API endpoint at this stage in the book will merely bloat the chapter. If you choose not to add this endpoint, your to-do application will work okay for the rest of the book, but the memory consumption of the cache will grow over time.

Now that we have covered code on command, we can move on to the focus of this chapter, which is observability. We will start this topic with a basic terminal logger. But first, we should explore what logging is.

What is logging?

So far, our application does not log anything. This does not directly affect the running of the app. However, there are some advantages to logging. Logging enables us to debug our applications. Right now, as we are developing locally, it may not seem like logging is really needed. However, in a production environment, there are many reasons why an application can fail, including Docker container orchestration issues. Logs that note what processes have happened can help us pinpoint an error. We can also use logging to see when edge cases and errors arise, so that we can monitor the general health of our application. When it comes to logging, there are four types of logs that we can build:

- **Informational (info):** This is general logging. If we want to track a general process and how it is progressing, we use this type of log. Examples of using this are starting and stopping the server and logging certain checkpoints that we want to monitor, such as HTTP requests.
- **Verbose:** This is information such as the type defined in the previous point. However, it is more granular to inform us of a more detailed flow of a process. This type of log is mainly used for debugging purposes and should generally be avoided when it comes to production settings.
- **Warning:** We use this type when we are logging a process that is failing and should not be ignored. However, we can use this instead of raising an error because we do not want the service to be interrupted or the user to be aware of the specific error. The logs themselves are for us to be alerted to the problem, to allow us to then act. Problems such as calls to another server failing are appropriate for this category.
- **Error:** This is where the process is interrupted due to an error, and we need to sort it out as quickly as possible. We also need to inform the user that the transaction did not go through. A good example of this is a failure to connect or insert data into a database. If this happens, there is no record of the transaction happening, and it cannot be solved retroactively. However, it should be noted that the process can continue running.

If a warning comes up about the server failing to send an email, connect to another server to dispatch a product for shipping, and so on. Once we have sorted out the problem, we can retroactively make a database call to transactions in this timeframe and make the calls to the server with the right information.

In the worst case, there will be a delay. With the error type, we will not be able to make the database call, as the server was interrupted by an error before the order was even entered in the database. Considering this, it is clear why error logging is highly critical, as the user needs to be informed that there is a problem and their transaction did not go through, prompting them to try again later.

We could consider the option of including enough information in the error logs to retroactively go back and update the database and complete the rest of the process when the issue is resolved, removing the need to inform the user. While this is tempting, we must consider two things. *Log data is generally unstructured.*

There is no quality control for what goes into a log. Therefore, once we have finally managed to manipulate the log data into the right format, there is still a chance that corrupt data could find its way into the database.

The second issue is that *logs are not considered secure*. They get copied and sent to other developers in a crisis, and they can be plugged into other pipelines and websites, such as **BugSnag**, to monitor logs. Considering the nature of logs, it is not good practice to have any identifiable information in a log.

Now that we are all juiced up knowing that logs rock, we can start our logging journey by building a basic logger that writes to the terminal.

Logging via the terminal

Considering our system has multiple servers, and these servers might be running by themselves, or all compiled into one binary in the ingress. Considering the flexibility of our system, we must define a logger in one place and import this logger into any service that wants to log anything. We can supply all our servers with a logger via the glue workspace with the following directory layout:

```
├─ Cargo.toml
├─ src
│   ├── errors.rs
│   ├── lib.rs
│   ├── logger
│   │   ├── logger.rs
│   │   ├── mod.rs
│   │   └─ network_wrappers
│   │       ├── actix_web.rs
│   │       └─ mod.rs
└─ token.rs
```

To integrate our logger, we must carry out the following steps:

1. Defining a logger
2. Creating a logging middleware
3. Integrating our logger into our servers

We can start by building out a basic logger.

Defining a logger

Before we write any code, we must have the following dependencies in our glue workspace:

```
# glue/Cargo.toml
...
[dependencies]
...
tracing = "0.1.4"
tracing-subscriber = "0.3.18"
futures-util = "0.3.30"
```

We will be using the tracing crate to define some logging functions, and tracing-subscriber to define a logger that will subscribe to the log events being produced. Our basic logger is just printing to the terminal. Why are we not just using `println!`? If we use `println!("hello world")`, we are essentially running the following code:

```
use std::io::{stdout, Write};

let mut lock = stdout().lock();
write!(lock, "hello world").unwrap();
```

Here, we can see that we are locking the standard output and writing to it. This makes sense as printing to the terminal would not be very useful if half of one message got printed alongside half of another message. This lock mechanism results in a reduction in performance. For instance, it is good practice to log every request that is sent to the server. If we log our requests using `println!`, even if we have four threads processing requests, each thread processing a request would have to wait its turn to acquire the lock, essentially holding up all threads to a single bottleneck. To stop this from happening, we create a global logger that accepts all logs from all threads. This logger will also remain live for the entire duration of the program.

We can define our logger with the following code:

```
// glue/src/logger/logger.rs
use tracing::Level;
use tracing_subscriber::FmtSubscriber;

pub fn init_logger() {
    let subscriber = FmtSubscriber::builder()
        .with_max_level(Level::INFO)
        .finish();
    tracing::subscriber::set_global_default(subscriber)
        .expect("Failed to set up logger");
}
```

At the start of our program, we will call the `init_logger()` function to initialize our global logger. We also do not want dependent programs having to install the tracing crate on their own dependencies to log messages. Having to install the tracing crate multiple times leads to extra work when trying to update and maintain the tracing crate dependency. To prevent this, we create some wrapper functions around the tracing macros with the following code:

```
// glue/src/logger/logger.rs
pub fn log_info(message: &str) {
    tracing::info!("{}", message);
}
pub fn log_warn(message: &str) {
    tracing::warn!("{}", message);
}
pub fn log_error(message: &str) {
    tracing::error!("{}", message);
}
pub fn log_debug(message: &str) {
    tracing::debug!("{}", message);
}
pub fn log_trace(message: &str) {
    tracing::trace!("{}", message);
}
```

And with this, we can now initialize our logger and produce log messages. However, what about logging every HTTP request sent to the server? Sure, we could write `log_info` for every API endpoint, but this would be a pain to write and maintain. Instead, we can build some middleware to log every HTTP request for us.

Creating a logging middleware

Previously, we have implemented the `FromRequest` trait for our JWT. The `FromRequest` trait enabled us to extract the token from the header and pass the extracted ID from the token into the view. However, we must pass the JWT struct into the view for this process to work. For our purpose, we need our logger to automatically work on every request, and we want the logger to also log the response code. First, we create an Actix logger struct that implements the `Transform` trait. The `Transform` trait is essentially the interface of a service factory. A service is an async function that converts a request to a response. For our middleware, we initially need to import the following code:

```
// glue/src/Logger/network_wrappers/actix_web.rs
use actix_web::{dev::ServiceRequest, dev::ServiceResponse, Error};
use actix_web::dev::{Transform, Service};
use futures_util::future::{ok, Ready};
use std::task::{Context, Poll};
use std::pin::Pin;
```

We can then implement the `Transform` trait for a struct called `ActixLogger`, which we create with the following code:

```
// glue/src/Logger/network_wrappers/actix_web.rs
pub struct ActixLogger;

impl<S, B> Transform<S, ServiceRequest> for ActixLogger
where
    S: Service<
        ServiceRequest,
        Response = ServiceResponse<B>,
        Error = Error
    > + 'static,
    S::Future: 'static,
{
    type Response = ServiceResponse<B>;
    type Error = Error;
    type InitError = ();
    type Transform = LoggingMiddleware<S>;
    type Future = Ready<Result<
        Self::Transform, Self::InitError
    >>;
```

```
fn new_transform(&self, service: S) -> Self::Future {
    ok(LoggingMiddleware { service })
}
}
```

There is a lot going on here, but we can break it down by focusing on the `new_transform` function signature. Here, we take in a service, denoted by `S`, which is the following definition:

```
S: Service<ServiceRequest, Response = ServiceResponse<B>,
    Error = Error> + 'static, S::Future: 'static,
```

This signature is essentially a service request that returns a response and is a future, which is an async function. We can see that the `new_transform` function returns the service that is wrapped in a struct called `LoggingMiddleware`. We will build the `LoggingMiddleware` struct and implement the `Service<ServiceRequest>` trait with the following code:

```
// glue/src/network_wrappers/actix_web.rs
pub struct LoggingMiddleware<S> {
    service: S,
}
impl<S, B> Service<ServiceRequest> for LoggingMiddleware<S>
where
    S: Service<
        ServiceRequest,
        Response = ServiceResponse<B>
    >, Error = Error> + 'static,
    S::Future: 'static,
{
    type Response = ServiceResponse<B>;
    type Error = Error;
    type Future = Pin<
        Box<
            dyn futures_util::Future<
                Output = Result<
                    Self::Response,
                    Self::Error
                >
            >
        >
    >;
    fn poll_ready(&self, cx: &mut Context<'_>)
```

```

        -> Poll<Result<(), Self::Error>> {
            self.service.poll_ready(cx)
        }
    fn call(&self, req: ServiceRequest)
        -> Self::Future {
        . . .
    }
}

```

Again, this is a lengthy chunk of code with a lot going on. However, we can see on the `call` function, we accept a request and return a `Future`. This `Future` is pinned because we do not want the future to move in memory because the thread will cycle back to the future to poll the future again, and we do not want the thread to access memory where the future used to be. We also put the future in a box because we do not know the size of the future, and boxing puts the future on the heap memory. If we remember the `async` chapter, which is *Chapter 3*, `poll_ready` is essentially polling the future. If the server is at capacity, then the `poll_ready` function will return a `pending`. If the request is ready to be processed, the `poll_ready` function will return a `ready`, and the request can start being polled. Inside our `call` function, we define the logging logic with the following code:

```

// glue/src/Logger/network_wrappers/actix_web.rs
fn call(&self, req: ServiceRequest)
    -> Self::Future {
    let fut = self.service.call(req);
    Box::pin(async move {
        let res = fut.await?;
        let req_info = format!(
            "{} {} {}",
            res.request().method(),
            res.request().uri(),
            res.status().as_str()
        );
        tracing::info!("Request: {}", req_info);
        Ok(res)
    })
}

```

Here, we can see that we log the method, endpoint, and response code. Finally, we ensure that our logger is available in the library with the following declarations:

```

// File: glue/src/Logger/network_wrappers/mod.rs
#[cfg(feature = "actix")]
pub mod actix_web;

```

```
// File: glue/src/logger/mod.rs
pub mod network_wrappers;
pub mod logger;
```

```
// File: glue/src/lib.rs
pub mod errors;
pub mod token;
pub mod logger;
```

With this, our logger is ready to be integrated into our servers.

Integrating our logger into our servers

For our servers, they all follow the same template. We wrap `ActixLogger` in our server definition. All our servers should have a layout like the following:

```
. . .
use glue::logger::{
    logger::init_logger,
    network_wrappers::actix_web::ActixLogger
};
use actix_cors::Cors;

#[tokio::main]
async fn main() -> std::io::Result<()> {
    init_logger();
    run_migrations().await;
    HttpServer::new(|| {
        let cors = Cors::default().allow_any_origin()
            .allow_any_method()
            .allow_any_header();

        App::new().wrap(ActixLogger)
            .wrap(cors)
            .configure(api::views_factory)
    })
    .workers(4)
    .bind("127.0.0.1:8081")?
    .run()
    .await
}
```

Here, we can see that we initialize the logger with the `init_logger` function, and then carry on with the rest of the process. For the ingress, we must run both migrations for both servers, but the outline is the same. Repeating the code three times would unnecessarily bloat the chapter.

If we run the ingress server, create a user, log in, and create an item, we should get logs like the following:

```
2024-07-19T23:36:02.320369Z  INFO
glue::logger::network_wrappers::actix_web:
Request: POST /api/v1/users/create 201
2024-07-19T23:36:02.767376Z  INFO
glue::logger::network_wrappers::actix_web:
Request: GET /api/v1/auth/login 200
2024-07-19T23:36:22.298595Z  INFO
glue::logger::network_wrappers::actix_web:
Request: GET /api/v1/auth/login 200
2024-07-19T23:36:22.310567Z  INFO
glue::logger::network_wrappers::actix_web:
Request: GET /api/v1/get/all 200
2024-07-19T23:36:38.671171Z  INFO
glue::logger::network_wrappers::actix_web:
Request: POST /api/v1/create 201
```

Here, we can see that the timestamp is also present. This is by default for the logger and should be done as they help with putting together what has happened. Timestamps can also help us sort logs later. For instance, writing a log to disk the very instant that the log was created is not essential. We could batch logs together in memory to perform a batch write if needed. If we have the timestamp, it does not matter what order the logs are written to disk.

Right now, are logging system is printing out to the terminal. This is okay for local development. However, our system might grow to multiple isolated servers running by themselves. If we have multiple servers running by themselves, we would like to have a single place where we can see all the logs.

Some deployment environments provide built-in tooling for inspecting logs. For example, Kubernetes-based applications can use tools such as Lens to inspect pod logs directly. However, this approach depends on having Kubernetes available.

In practice, your application might be deployed in many different ways:

- Across multiple physical servers that are not part of a single cluster
- On a single machine without Docker or container orchestration
- Using simple setups such as docker-compose

Regardless of how your application is deployed, writing logs to a database provides a consistent way to inspect and analyze them. Database-backed logs can be consumed by dashboards and monitoring tools without relying on a specific deployment platform.

To achieve maximum flexibility, we can move on to building a logging mechanism for a remote database.

Logging via a database

You may have heard of OpenTelemetry or a range of dashboards such as **Jaeger**. However, at the time of writing this, a lot of the Rust libraries for such platforms are lacking in documentation, maturity, and introducing breaking changes. To show you how to set up OpenTelemetry and off-the-shelf dashboards would result in the book aging quickly. With the skills you have learned in this book, there is nothing stopping you from googling the latest configuration steps to run these off-the-shelf projects. For the rest of this chapter, we are going to build our own async mechanism that shoots logs off to a database, allowing you to query this database for specific logs. Our async mechanism is going to keep the pressure off the logging functionality directly, as we log every request. If we had to await a response from the database for every request, we would dramatically slow down our request processing times. Instead, we are going to shoot a log message off to an async actor, and the async actor is going to handle the logging of the message in the database.

Elasticsearch is a distributed, document-oriented database designed for fast indexing and querying of large volumes of semi-structured data. Unlike traditional relational databases, Elasticsearch is optimized for search-heavy workloads, making it particularly well-suited for logs, metrics, and event data.

Logs naturally fit this model: they are append-only, schema-flexible, and frequently queried by time range, severity, or message content. Elasticsearch allows us to efficiently search and aggregate logs across services without requiring rigid schemas or complex joins.

In this chapter, Elasticsearch serves as a practical example of a log-focused datastore. The logging architecture we build is not tied to Elasticsearch specifically – the same async logging mechanism could be adapted to send logs to any external system or database.

Learning this approach will also give you the skill set to offload other tasks from the request process if needed, using the actor approach. Before we set off building the logging mechanism, however, we must define what an actor is.

What is an actor?

An **actor** is an isolated piece of code that can have state. This actor only communicates with other parts of the program via messages. The actor can have its own internal state that can be updated due to messages. The actor can also send messages to other actors or even create actors. While I do personally like the actor model, we must be careful when or where we apply the actor model. There is some overhead in sending messages and spawning tasks to run actors. So, if your actor is only going to do a couple of operations before finishing, then you might as well just call a couple of async functions inside your future.

Actors really shine when you have a long-running process, and you want to keep sending messages to them. This can allow you to take pressure off the task you are currently performing. It can also keep resources allocated to one actor as opposed to needing to reallocate those resources. For instance, when we make a network connection, there is usually a handshake consisting of messages back and forth to establish that connection.

Having an actor maintain that connection and accept messages to send over that connection can reduce the number of handshakes you need to make. Another example is opening a file. When you open a file, the operating system needs to check whether the file is there, whether the reader has permissions, and must acquire a lock for that file. Instead of opening a file to perform one transaction to it and then closing it, we can have an actor maintain the handle of that file and write to it the messages that were sent to the actor. Another advantage to remember is that channels can also act as queues. Channels can have messages build up in them with the actor consuming them at the actor's own pace if the computer has enough memory to keep the messages in memory, and the channel has enough allocated capacity.

For our logging, we are going to have an actor that consumes log messages from all over the program and send those logs to a database, as shown in *Figure 13.1*:

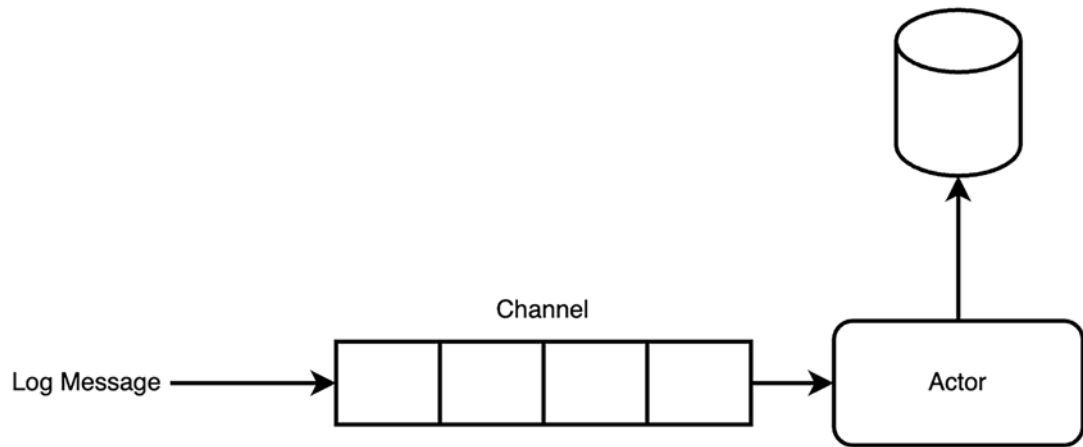


Figure 13.1 – Our logging actor

Now that we know what actors are and how we are going to use them, we can start building our remote logging system by building our actor.

Building our logging actor

Before we build our actor, we need the following crates in our glue workspace:

```
# glue/Cargo.toml
...
[dependencies]
...
serde_json = { version = "1.0.120", optional = true }
tokio = { version = "1.38.1", optional = true }
reqwest = { version = "0.12.5", optional = true }
chrono = { version = "0.4.38", optional = true }
once_cell = { version = "1.19.0", optional = true }
```

```
[features]
actix = ["dep:actix-web"]
rocket = ["dep:rocket"]
axum = ["dep:axum", "dep:tower"]
hyper = ["dep:hyper", "dep:serde_json", "dep:http-body-util"]
elastic-logger = [
    "dep:serde_json", "dep:tokio", "dep:reqwest", "dep:chrono"
]
```

Here, we can see that our elastic-logger feature utilizes all the new dependencies, but we do not need them if we are not performing remote logging, as we will not have an actor to support.

We then define our elastic-logger actor model with the following code:

```
// glue/src/logger/mod.rs
pub mod network_wrappers;
pub mod logger;
#[cfg(feature = "elastic-logger")]
pub mod elastic_actor;
```

Inside our actor module, we must import the following before we can define our actor:

```
// glue/src/logger/elastic_actor.rs
use tokio::sync::mpsc;
use tokio::sync::mpsc::{Receiver, Sender};
use serde_json::json;
use reqwest::{Client, Body};
use chrono::Utc;
use std::sync::LazyLock as Lazy;
use serde::Serialize;
```

Our actor is going to need to accept messages with enough data in them to create a log and send it. These messages need to be sent over a channel. Our message takes the following form:

```
// glue/src/logger/elastic_actor.rs
#[derive(Debug, Serialize)]
struct LogMessage {
    level: String,
    message: String,
}
```

With this message, we can then build our send_log function with the following code:

```
// glue/src/logger/elastic_actor.rs
pub async fn send_log(level: &str, message: &str) {
    static LOG_CHANNEL: Lazy<Sender<LogMessage>> = Lazy::new(|| {
```

```

        let (tx, rx) = mpsc::channel(100);
        tokio::spawn(async move {
            elastic_actor(rx).await;
        });
        tx
    });
    LOG_CHANNEL.send(LogMessage {
        level: level.to_string(),
        message: message.to_string(),
    }).await.unwrap();
}

```

Here, we can see that we exploit `Lazy::new` to create our channel, spawn our actor, and return the channel sender. `Lazy::new` only gets executed once, so no matter where we call the `send_log` function, we are sending messages to that one actor that has been spawned the first time the `send_log` function was called.

For our actor, we define the outline with the following code:

```

// glue/src/logger/elastic_actor.rs
async fn elastic_actor(mut rx: Receiver<LogMessage>) {
    let elastic_url = std::env::var(
        "ELASTICSEARCH_URL"
    ).unwrap();
    let client = Client::new();
    while let Some(log) = rx.recv().await {
        . . .
    }
}

```

Here, we can see that we get the URL to the database, establish an HTTP client, and then run an infinite loop where we cycle through an iteration of the loop every time a message is received from the channel. Once we get the message from the channel, we create a JSON body and send it via HTTP to the Elasticsearch database with the following code:

```

// glue/src/logger/elastic_actor.rs
let body = json!({
    "level": log.level,
    "message": log.message,
    "timestamp": Utc::now().to_rfc3339()
});
let body = Body::from(serde_json::to_string(&body)
    .unwrap());
match client.post(&elastic_url)

```

```

    .header("Content-Type", "application/json")
    .header("Accept", "application/json")
    .body(body)
    .send()
    .await
  {
    Ok(result) => {},
    Err(e) => {
      eprintln!(
        "Failed to send log to Elasticsearch: {}",
        e
      );
    }
  }
}

```

We can see that we directly unwrap the serialization of the message. This is because there will never be an error translating two strings and a timestamp to a JSON string. However, there could be an issue in sending the HTTP request to the Elasticsearch database; therefore, we handle the potential error when sending the HTTP request. This is because our actor is running in the background for the entire duration of the program. We would hate for a temporary network error to kill our actor, blocking us from sending any more logs to the database.

Our actor can now run and accept messages if we call the `send_log` function anywhere in the code base. We now need to call the `send_log` function in our logging function.

Update logging functions

For our logging functions, we still need to log the message to the terminal so logs can still be recovered if there is a problem with our database. This sending of the message to the database should be triggered if the feature is enabled. Considering this, our `init_logger` function remains the same, giving us the following outline:

```

// File: glue/src/logger/logger.rs
...
#[cfg(feature = "elastic-logger")]
use super::elastic_actor::send_log;

pub fn init_logger() {
    ...
}

```

Our functions now take the following form:

```

// File: glue/src/logger/logger.rs
pub async fn log_info(message: &str) {

```

```

    tracing::info!("{}", message);
    #[cfg(feature = "elastic-logger")]
    send_log("INFO", message).await;
}

pub async fn log_warn(message: &str) {
    tracing::warn!("{}", message);
    #[cfg(feature = "elastic-logger")]
    send_log("WARN", message).await;
}

pub async fn log_error(message: &str) {
    tracing::error!("{}", message);
    #[cfg(feature = "elastic-logger")]
    send_log("ERROR", message).await;
}

pub async fn log_debug(message: &str) {
    tracing::debug!("{}", message);
    #[cfg(feature = "elastic-logger")]
    send_log("DEBUG", message).await;
}

pub async fn log_trace(message: &str) {
    tracing::trace!("{}", message);
    #[cfg(feature = "elastic-logger")]
    send_log("TRACE", message).await;
}

```

With this, our system is now in place to send messages to the database. We now must update our Cargo.toml files for all our elastic-logger feature for the glue module:

```

# File: ingress/Cargo.toml

...
glue = {
    path = "../glue",
    features = ["actix", "elastic-logger"]
}
...

```

```

# File: nanoservices/auth/networking/actix_server/Cargo.toml

...
glue = {
    path = "../../../../../glue",
    features = ["actix", "elastic-logger"]
}

```

```
}
. . .
```

```
# nanoservices/to_do/networking/actix_server/Cargo.toml

. . .
glue = {
  path = "../../../../../glue",
  features = ["actix", "elastic-logger"]
}

. . .
```

Now that our system is fully ready to send logs to our database, we must configure our database.

Configuring our logging database

To enable our database to accept logs, we add the following service to our `docker-compose.yml` file:

```
# docker-compose.yml

. . .
elasticsearch:
  image: docker.elastic.co/elasticsearch/elasticsearch:7.13.4
  container_name: elasticsearch
  environment:
    - discovery.type=single-node
    - bootstrap.memory_lock=true
    - "ES_JAVA_OPTS=-Xms512m -Xmx512m"
  ulimits:
    memlock:
      soft: -1
      hard: -1
  ports:
    - "9200:9200"
    - "9300:9300"
```

For the Elasticsearch database, we have the following configurations:

- `discovery.type=single-node`: This configures Elasticsearch to run in single-node mode, which is suitable for development or testing environments.
- `bootstrap.memory_lock=true`: This ensures that the JVM locks the memory used by Elasticsearch, preventing it from being swapped out by the operating system.
- `"ES_JAVA_OPTS=-Xms512m -Xmx512m"`: This sets the Java heap size options, specifying that both the initial and maximum heap sizes should be 512 MB.

- **memlock:** This prevents the container from swapping memory to disk. Setting both soft and hard limits to -1 means unlimited memory lock.
- **9200:9200:** This maps port 9200 of the container to port 9200 of the host. This is the default port for the Elasticsearch HTTP API.
- **9300:9300:** This maps port 9300 of the container to port 9300 of the host. This is the default port for Elasticsearch transport node communication.

Due to the new database, our `.env` file should look like the following:

```
TO_DO_DB_URL=postgres://username:password@localhost/to_do
AUTH_DB_URL=postgres://username:password@localhost/to_do
AUTH_API_URL=http://127.0.0.1:8081
JWT_SECRET=secret
CACHE_API_URL=redis://127.0.0.1:6379
ELASTICSEARCH_URL=http://localhost:9200/logs/_doc
```

And that is it! Our system is now able to start sending logs out of our database.

If we run our server, and then create a user login using our `ingress/scripts/create_login.sh` script, we can make a `curl` request to our database with the following command:

```
curl -X GET "http://localhost:9200/logs/_search?pretty" \
-H 'Content-Type: application/json' -d'
{
  "query": {
    "match": {
      "level": "INFO"
    }
  }
}
```

Here, we are making a query for all logs with a level of `INFO`. Our query gives us the following results:

```
{
  "took" : 39,
  "timed_out" : false,
  "_shards" : {
    "total" : 1,
    "successful" : 1,
    "skipped" : 0,
    "failed" : 0
  },
  "hits" : {
    "total" : {
```

```
    "value" : 2,
    "relation" : "eq"
  },
  "max_score" : 0.18232156,
  "hits" : [
    {
      "_index" : "logs",
      "_type" : "_doc",
      "_id" : "NbbV15AB4EMJAczNwoWr",
      "_score" : 0.18232156,
      "_source" : {
        "level" : "INFO",
        "message" : "Request: POST /api/v1/users/create 201",
        "timestamp" : "2024-07-22T00:27:08.721360+00:00"
      }
    },
    {
      "_index" : "logs",
      "_type" : "_doc",
      "_id" : "NrbV15AB4EMJAczNw4Xx",
      "_score" : 0.18232156,
      "_source" : {
        "level" : "INFO",
        "message" : "Request: GET /api/v1/auth/login 200",
        "timestamp" : "2024-07-22T00:27:09.167054+00:00"
      }
    }
  ]
}
```

We can see that both of our logs are here in the database. We can also see that we have a score. The higher the score, the more relevant the log is to the search. We can also see that we have `max_score` in the response. We can see that the scores in the logs are the same as the maximum scores. Our `level`: "INFO" matches exactly. We could be more granular and produce even more tags on our logs. For instance, we could also put in a tag for the service and filter by this if we want. As your system gets more complex, logging to a database can be a lifesaver.

Summary

In this chapter, we set up a basic logger and created an actor to take pressure off the logger when sending those logs to an Elasticsearch database. With our logging setup, we can inspect logs from multiple distributed servers and query those logs. Not only does this help us debug our systems and monitor the health of our system when it is live, but we also now have the skill set to pass off logs to an actor running in the background, so our server is not held up awaiting network calls to the Elasticsearch database.

We have now reached the end of the development of the to-do application. You can always add more features and improve things, but we would merely be repeating the concepts that we covered in the book to achieve these new features. For instance, adding a logout button and implementing the refresh token mechanism for authentication sessions are probably things you want to add to our to-do application, but covering these features would just bloat the book with repetition of updating the local storage in the frontend with a returned refreshed token, and adding a button on the page to hit the logout API. The rest of the book focuses on how to test your server, how to deploy the system, and some more advanced concepts such as low-level networking. In the next chapter, we will cover unit testing. As stated in the previous chapter, good testing practices are probably the most important software engineering skill that will set you apart from other developers. You will be able to develop complex systems at a safer and faster rate. I've spoken publicly about a range of things. When it comes to testing, the outreach is mainly from senior developers and tech leads with reputable backgrounds. Junior developers or developers who have never been pushed to develop their craft don't really understand testing and how important and useful it is.

Questions

1. Why do we need logging in a multi-service system?
2. What is REST, and which HTTP verbs does this chapter associate with common operations?
3. What does “code on demand” mean in REST, and how does the chapter demonstrate it?
4. What are the four log levels described, and when would you use each?
5. Why not just use `println!` for request logging?
6. What problem does the Actix middleware solve, and what does it log?
7. What is an actor in this chapter's context, and why is it useful for logging?
8. Why use Elasticsearch for logs, and how do you query logs in the chapter?

Answers

1. Because without logs, we can't easily tell what happened inside the system—whether one server called another, what endpoint was hit, what status code came back, or where an error originated. Logs let us reconstruct the “story” of a request through the system, debug failures without exposing sensitive internal details to users, and monitor system health over time.
2. **REST (Representational state transfer)** is an architectural style for APIs. In the chapter's framing:

- GET reads data
- POST creates data
- PUT updates data
- DELETE removes data

The goal is a predictable, reusable interface that supports performance and evolvability.

3. “Code on demand” is an optional REST constraint where the server can send executable code to the client to run. The chapter demonstrates this by returning an HTML response containing a `<script>` that removes a JWT from `localStorage` and redirects the browser—effectively forcing a logout from the backend.
4. Informational (info): Normal checkpoints (server start/stop, HTTP requests).
 - Verbose: More granular detail, mainly for debugging; generally avoided in production.
 - Warning: Something went wrong, but you’re not interrupting the user flow (e.g., a downstream call failed, but you can recover later).
 - Error: A failure that interrupts the current operation (e.g., database insert failed, request can’t be completed), though the server can keep running.
5. `println!` locks `stdout` each time it writes. If multiple threads are processing requests, they end up contending for that lock, turning logging into a bottleneck. The chapter uses tracing + tracing-subscriber to set a global logger that can accept log events from across the program more efficiently and consistently.
6. It avoids manually adding logging to every endpoint. By wrapping the service in middleware, it logs every HTTP request automatically, including the following:
 - HTTP method
 - URI/path
 - Response status code

So, you get consistent per-request observability without repetitive code.

7. An actor is a long-running task that owns its own state and communicates via messages (through a channel). It’s useful for logging because the main request path can send log messages to the actor and move on, while the actor handles slower work, such as network I/O to an external log store. This reduces latency and keeps request handling responsive.
8. Elasticsearch is positioned as a good fit for logs because logs are append-only, semi-structured, and commonly queried by time range, severity, or text search. The chapter queries logs via the Elasticsearch HTTP API using a search request, for example, matching all documents where “level”: “INFO”, and it explains that results include hits plus relevance scoring (`_score` / `max_score`).

Get this book's PDF copy, code bundle, and more

Scan the QR code (or go to packtpub.com/unlock). Search for this book by name, confirm the edition, and then follow the steps on the page.



UNLOCK NOW

Note: Have your invoice handy. Purchases made directly from the Packt website don't require an invoice.

14

Unit Testing

We have built our to-do application, and it is working. However, we only know it is working because our operations were simple. We know that in the real world, the logic behind our applications will be more complex with more moving parts. This is where unit testing comes in. **Unit testing** is essential when your application evolves. In this chapter, we will cover the following:

- The importance of unit testing
- Writing our first unit test
- Testing our core
- Unit testing our networking layer
- Unit testing our data access layer

By the end of this chapter, you will understand the importance of unit testing. You will also be able to write unit tests and mock IO handles such as calls to a database, to test how our code handles different states. You will also be able to test HTTP requests for a service and build a testing harness that configures a database and runs tests in log files.

Technical requirements

You can download the example project and code for this book by following the instructions in the **Download the example code files** section in the *Preface* of this book.

This chapter's code files are included in the downloadable code bundle.

The importance of unit testing

It baffles me that there is pushback against testing at all. Sadly, the internet is full of tutorials rushing to get a poorly structured app with loads of tech debt deployed. The issue with these tutorials is that when you are copying the code and chugging along the predefined path, everything seems smooth and quick. However, this smooth, quick process is a facade. If you create software that aims to solve a problem in the real world, you will have a general idea of where to start, but there will always be unknowns and features that you did not initially think about. This is why experienced software engineers generally say, “*Estimate how long the task is going to take and then double it*”.

As the system grows, if unit testing is not in place, development will become error-prone and will take longer and longer to add features. Refactoring an untested system can become near enough impossible if the system is too complex. You also get brittle software, meaning that if you change a piece of code in one place, it can break code elsewhere. To demonstrate this, we can look at *Figure 14.1* to see the number of steps needed to test the core function of a delete:

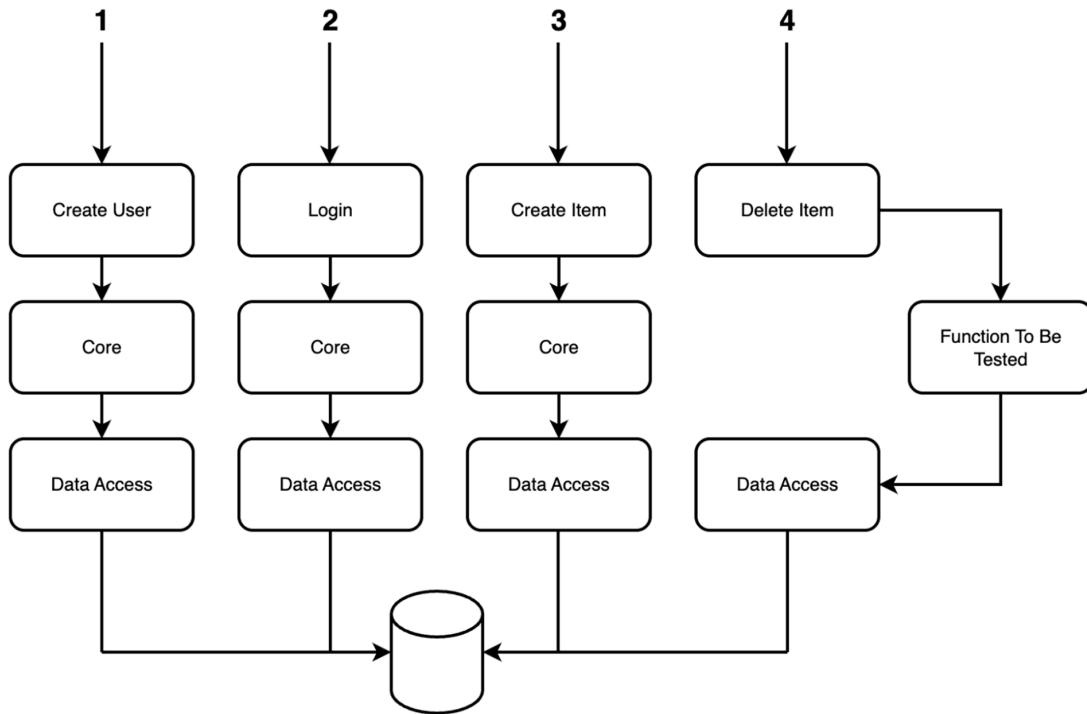


Figure 14.1 – The steps needed to test a core delete item

Here, we can see that there are 11 pieces of code in the way of testing to see if our core function for deleting an item works. If there is an error in any of those 11 pieces of code, then you will not be able to test our delete item code. You will have to perform 3 steps before you can test the delete function. So, we attempt to write the delete function, perform the three steps, and then see if our delete function acts as expected. Again, our example is very simple, but with more complex processes, it's very unlikely that you will get it right on the first try. So, you tear down the database, spin up another one, and then perform the steps again. Then, you wonder what will happen if you try and delete an item that is not there. So, you do the first two steps, then the fourth step, and you get an unhelpful error when you really want a not found error. So, you update it, do the first, second, and fourth steps. Okay, it now works, but you also need to wipe the database again and do all four steps in sequence to make sure that your most recent change did not affect the deletion of an item if it does exist. Again, this may seem excessive for a simple delete function; however, I'm only using this as a simplified example. For more complex real-world examples, changing code to account for one edge case might break code for another edge case. If you don't like doing all these manual steps and decide to write a script that performs all four steps in a process, then you are writing a testing script. You might as well lean into the entire ecosystem of testing tools and do it properly.

For a unit test, we isolate the piece of code and pass a range of inputs and assert the outcomes, as seen in *Figure 14.2*:

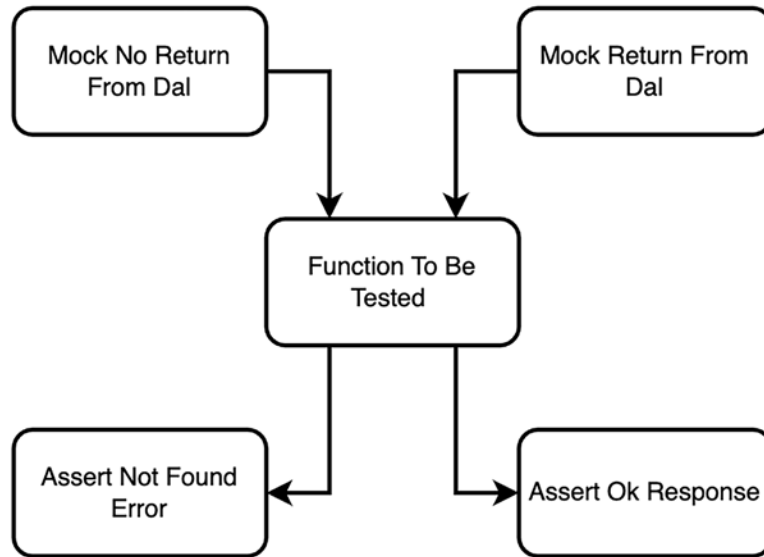


Figure 14.2 – The steps needed for a unit test

With this unit test, we do two operations to test both of our edge cases, as opposed to seven steps and two database wipes. When you run these steps again and again when you are developing your code, of course, the proper unit-tested code will enable you to iterate your code at a much faster rate. At the time of writing this book, I am working in GPU streaming in Rust at Kings College London for a surgical robotics project. I was at the stage of reshaping the data coming out of the GPU, which was streamed into the GPU. My PhD student was struggling, so I spent an evening where we captured one frame from the stream and stored it as a binary file. We then passed that frame through the GPU so that a convolutional neural network could detect objects in the frame, and we saved the computation from the GPU. I then built a unit test for the function. The PhD student then did not have to worry about GPU drivers. He could code on any computer he wanted. All he had to do was run the test that accepted the input and spat out his output. Within a day, he solved the problem. When I next spoke to him, he admitted that he did not really take my talks on unit testing seriously until he experienced it first-hand. Now he is an avid unit tester.

There are some potential pitfalls to unit testing. For instance, another part of the code could change, but you did not change your mock. Therefore, your test could technically pass but break in a larger system. This is less likely to happen with Rust due to the strong typing. However, it is something to keep an eye out for. We will also completely negate this issue with end-to-end testing in the next chapter. Others state that unit testing is too rigid. You change one thing, then you must change a load of tests. This, however, is more of a reflection of their code. Well-structured code with clean interfaces is easy to test and move around. Unit testing is also a test of the code structure. If code is hard to test, will it be easy to refactor that code or change a data source? Probably not. Unit tests not only save time running the code with quick feedback loops, but they also give us a sense of whether we are going down a bad path with our code structure.

Now that we know that testing is one of the most important skills we can learn as a developer, let us write our first unit test.

Writing our first unit test

To get to grips with the basics of testing, we are going to test the following simple function in a blank Rust workspace:

```
// src/main.rs
fn add_two_numbers<T, U>(a: T, b: U)
-> Result<T, <U as TryInto<T>>::Error>
where
    T: std::ops::Add<Output = T>,
    U: TryInto<T>,
{
    let b_converted: T = b.try_into()?;
    Ok(a + b_converted)
}
```

Here, we make a generic function that adds two numbers together. We can also try and cast the second number into the same type as the first number, so we can pass in a different number type, and we will try and convert it.

To test our function, we must define the following module inside the same file:

```
// src/main.rs
#[cfg(test)]
mod tests {
    use super::*;
    . . .
}
```

Here, we are importing everything in the file into the `tests` module. The `tests` module will only be compiled if we are running tests and will not be included in the main app for deployment. Inside our `tests` module, we can define our first test with the following code:

```
// src/main.rs
#[test]
fn test_add_two_numbers_i32() {
    let result = add_two_numbers(5i32, 3i32);
    assert_eq!(result, Ok(8));
}
```

Here, we are adding two `i32` integers and checking to see whether the result is an eight. We can run our tests with the following command:

```
cargo test
```


This command will give us the following printout:

```
running 1 test
test tests::test_add_two_numbers_i32 ... ok

test result: ok. 1 passed; 0 failed; 0 ignored;
0 measured; 0 filtered out; finished in 0.00s
```

For our next three tests, we can add different integer types: a minus integer, a float, and an integer to see how our function works with the following code:

```
// src/main.rs
#[test]
fn test_add_two_numbers_i32_u64() {
    let result = add_two_numbers(5i32, 3u64);
    assert_eq!(result, Ok(8));
}
#[test]
fn test_add_two_numbers_minus() {
    let result = add_two_numbers(5i32, -3i32);
    assert_eq!(result, Ok(2));
}
#[test]
fn test_add_two_numbers_float_and_int() {
    let result = add_two_numbers(5.0, 3);
    assert_eq!(result, Ok(8.0));
}
```

With the test command giving us the following printout:

```
running 4 tests
test tests::test_add_two_numbers_float_and_int ... ok
test tests::test_add_two_numbers_i32 ... ok
test tests::test_add_two_numbers_minus ... ok
test tests::test_add_two_numbers_i32_u64 ... ok

test result: ok. 4 passed; 0 failed; 0 ignored;
0 measured; 0 filtered out; finished in 0.00s
```

Finally, we want to ensure that if our second number exceeds the maximum value of the first number type, we get an error with the following test:

```
// src/main.rs
#[test]
fn test_add_two_numbers_outsize() {
```

```

    let result = add_two_numbers(5u8, 5000i32);
    assert_eq!(result.is_err(), true);
}

```

If we run the test command again, all five of our tests will pass. Here, we have mapped exactly how our function would work before we try and integrate our adding function into the rest of our system if we need to.

You may have heard about setups and teardowns in unit tests. This is where a setup function fires before the test is run, and the teardown function is fired after the test. Seeing as we are not interacting with the state of anything, we do not need to perform setup and teardown functions. We will cover the setup and teardown functions in the following chapter for end-to-end testing, as our end-to-end tests must interact with databases, and these databases need to be wiped and set up between tests to ensure that the tests are isolated and deterministic.

Now that we know how to build and run basic tests, we are going to move to the core module of our servers to mock our data access layer to see how our core module handles different data access responses.

Testing our core

We are now at the stage where we want to create a unit test for the delete function in our core. However, before we write any testing code, we must acknowledge that our core code just has the bare minimum to perform our application logic. The functions in the core are async, but we do not have any async runtime dependencies in the core workspace. However, we do need a runtime to test our async functions. To remedy this, we add a tokio dependency in our dev-dependencies section so our development processes like running tests have tokio, but our core library will not have the tokio dependency when being deployed. This is achieved with the following code:

```

// nanoservices/to_do/core/Cargo.toml
...
[dependencies]
To-do-dal = { path = "../dal" }
serde = { version = "1.0.197", features = ["derive"] }
glue = { path = "../glue" }
auth-kernel = { path = "../auth/kernel", features = ["core-postgres"] }

[dev-dependencies]
tokio = { version = "1.39.1", features = ["full"] }

```

With tokio, we can use the `#[tokio::test]` macro, which essentially enables us to run async test functions. With our tokio dependency sorted, we can now move on to building our tests for our core delete function.

Here, we must pass in the correct parameters to the data access layer. If the item is not in the database, then we will get an error. For our testing harness of the delete core function, we have the following code:

```

// nanoservices/to_do/core/src/api/basic_actions/delete.rs

```

```

. . .
#[cfg(test)]
mod tests {
    use super::*;
    use to_do_dal::to_do_items::schema::ToDoItem;
    use to_do_dal::to_do_items::transactions::delete::DeleteOneResponse;
    use std::future::Future;

    #[tokio::test]
    async fn test_delete_ok() {
        . . .
    }
    #[tokio::test]
    async fn test_delete_err() {
        . . .
    }
}

```

Because we structured our code well with traits handling the database operation, we can create a mock struct and implement the database handle trait to mock different responses from the database. Inside our `test_delete_one_ok` function, we can create a mock struct and implement the `DeleteOne` trait with the following code:

```

// nanoservices/to_do/core/src/api/basic_actions/delete.rs
struct ReturnOneMock;

impl DeleteOne for ReturnOneMock {
    fn delete_one(id: String, user_id: i32)
        -> impl Future<Output = DeleteOneResponse> + Send {
        . . .
    }
}

```

Inside the `delete_one` function in our trait implementation, we must assert that a correct title and user ID were passed to the data access layer. We can also see that our test is trying to create a successful delete; therefore, we must return an OK result because we are testing the success of our delete core function. Our checks and return inside the `delete_one` function in our trait implementation are defined ahead:

```

// nanoservices/to_do/core/src/api/basic_actions/delete.rs
if id != "coding" {
    panic!("Invalid title");
}
if user_id != 1 {

```

```

        panic!("Invalid user_id");
    }
    async {
        Ok(ToDoItem {
            id: 1,
            title: "title".to_string(),
            status: "PENDING".to_string(),
        })
    }
}

```

Finally, in our `test_delete_one_ok` function, we call the delete core function with our mock handle, and assert that the result is okay with the following code:

```

// nanoservices/to_do/core/src/api/basic_actions/delete.rs
let result = delete::<ReturnOneMock>("coding", 1).await;
assert!(result.is_ok());

```

For our error test, we have an outline like the previous with the following code:

```

// nanoservices/to_do/core/src/api/basic_actions/delete.rs
#[tokio::test]
async fn test_delete_err() {

    struct ReturnErrorMock;

    impl DeleteOne for ReturnErrorMock {
        fn delete_one(id: String, user_id: i32)
        -> impl Future<Output = DeleteOneResponse> + Send {
            . . .
        }
    }

    let result = delete::<ReturnErrorMock>("coding", 1).await;
    match result {
        Ok(_) => panic!("Expected error"),
        Err(e) => {
            assert_eq!(
                e.status,
                glue::errors::NanoServiceErrorStatus::NotFound
            );
            assert_eq!(e.message, "Some Error");
        }
    }
}

```

Inside the implementation of the `DeleteOne` trait for the `delete_one` function, we have the following code:

```
// nanoservices/to_do/core/core/src/api/basic_actions/delete.rs
if id != "coding" {
    panic!("Invalid title");
}
if user_id != 1 {
    panic!("Invalid user_id");
}
async {
    Err(NanoServiceError::new(
        "Some Error".to_string(),
        glue::errors::NanoServiceErrorStatus::NotFound
    ))
}
```

And now our unit test is complete. If we navigate to the core of the to-do nanoservice and run our `cargo test` command, we should get the following printout:

```
orunning 2 tests
test api::basic_actions::delete::tests::test_delete_err ... ok
test api::basic_actions::delete::tests::test_delete_ok ... ok

test result: ok. 2 passed; 0 failed; 0 ignored; 0 measured;
0 filtered out; finished in 0.00s

Doc-tests core

running 0 tests

test result: ok. 0 passed; 0 failed; 0 ignored; 0 measured;
0 filtered out; finished in 0.00s
```

Here, our tests are working. These tests are consistent, deterministic, and fully automated. We can continue to build out our tests and run them to get feedback on the code we are writing within seconds.

It must be noted that if we merely run a `cargo test` command in the root of our project, we will get an error due to the code for the Redis module. We must remember that we are going to be running a range of tests as we essentially have a microservices system with a Redis cache, database, logging database, and multiple servers talking to each other. When we run a `cargo test` command, it runs the tests over multiple threads. We are going to need to run our tests in a single threaded way one after the other. We can communicate to other developers our testing strategy and automate the running of all our tests with the following bash script in the `scripts` directory at the root of our system:

```
#!/usr/bin/env bash
# scripts/run_tests.sh

# navigate to directory
SCRIPTPATH="$( cd "$(dirname "$0")" ; pwd -P )"
cd $SCRIPTPATH

cd ..

cargo test -p to-do-core
```

Now that our basic unit testing for our core delete function is done, another developer can look at the tests and know how to call the function and what to expect when using the function. The threshold is also lowered for them adding extra functionality to the delete function; they can hit the ground running, developing directly against the tests as opposed to having to spin up multiple databases and servers.

But what about our networking layer? Yes, we can unit test HTTP requests and see how our server handles them.

Unit testing our networking layer

When it comes to unit testing, most good web frameworks will offer unit testing tools because the maintainers are skilled engineers, and they know the importance of unit testing. Actix is no exception. Unit testing our networking layer is a bit more complicated compared to our core because there are more moving parts. However, unit testing our network layer is a bit more representative of what unit testing is like outside of a toy example because we must handle multiple moving parts, and we must test how they work.

Before we write any testing code, however, we must reconfigure our dependencies with the following:

```
# nanoservices/to_do/networking/actix_server/
# Cargo.toml
. . .
[dependencies]
. . .
to-do-core = { path = "../core" }
. . .

[dev-dependencies]
serde_json = "1.0.120"
actix-http = "3.8.0"
. . .
```

We can see that we add the `serde-json` and `actix-http` crates into dev-dependencies. We will use these crates to create HTTP requests for our test. We will also need to deserialize the JSON response body in the test. With this in place, we can build out our unit test by carrying out the following steps:

- Defining our test module
- Mocking our database handle
- Mocking our cache handle
- Defining our mock service
- Writing tests

With this comprehensive approach, we will be able to see how our delete endpoint acts in a whole range of ways. For Rocket, Axum, and hyper, we do not need any extra dependencies apart from `serde-json` because we can construct requests from their own crates. For instance, constructing a request for Axum can be done using the Axum crate. The same goes for Hyper and Rocket.

We can start our HTTP unit test by defining our test module.

Defining our test module

For our test module definition, we have the following outline:

```
// nanoservices/to_do/networking/actix_server/
// src/api/basic_actions/delete.rs

#[cfg(test)]
mod tests {
    use super::*;
    use actix_web::{
        dev::ServiceResponse,
        self, body::MessageBody, http::header::ContentType, test::{
            call_service, init_service, TestRequest
        }, web, App
    };
    use actix_http::Request;
    use glue::token::HeaderToken;
    use dal::to_do_items::transactions::delete::DeleteOneResponse;
    use dal::to_do_items::schema::{ToDoItem, AllToDoItems};
    use std::future::Future;
    use auth_kernel::user_session::schema::UserSession;
}
```

With all these imports, we will be able to mock the database and cache. We will also be able to construct test HTTP requests and services. Before we write any tests, we must write some helper functions that are available to all test functions. First, we build a function that generates an item with the following code:

```
// nanoservices/to_do/networking/actix_server/
// src/api/basic_actions/delete.rs
fn generate_to_do_item() -> TodoItem {
    TodoItem {
        id: 1,
        title: "test".to_string(),
        status: "PENDING".to_string(),
    }
}
```

We might as well produce the same item throughout the tests just to keep it simple and reduce the amount of code we must write to get our tests running. We can also build a function that generates the return items with the following:

```
// nanoservices/to_do/networking/actix_server/
// src/api/basic_actions/delete.rs
fn generate_get_all_return() -> Vec<TodoItem> {
    vec![
        generate_to_do_item()
    ]
}
```

We can now move on to mocking our database handle.

Mocking our database handle

For our mock database handle, we implement the `DeleteOne` trait by returning an internal `async` function we can see in the following code:

```
// nanoservices/to_do/networking/actix_server/
// src/api/basic_actions/delete.rs
struct MockDbHandle;

impl DeleteOne for MockDbHandle {
    fn delete_one(title: String, _user_id: i32)
        -> impl Future<Output = DeleteOneResponse> + Send {
        async fn run(title: String) -> DeleteOneResponse {
            . . .
        }
        run(title)
    }
}
```


Inside our internal run function, we want to see how our delete endpoint handles a successful and a failed delete operation. We do not want to write loads of different mocks for our tests. We can make it so that an error is returned if *coding* is not passed into the function with the following code:

```
// nanoservices/to_do/networking/actix_server/
// src/api/basic_actions/delete.rs
if title == "coding" {
    return Ok(generate_to_do_item())
}
return Err(
    NanoServiceError::new(
        "Item not found".to_string(),
        NanoServiceErrorStatus::NotFound
    )
)
```

We implement the GetAll trait for our mock database handle with the following code:

```
// nanoservices/to_do/networking/actix_server/
// src/api/basic_actions/delete.rs
impl GetAll for MockDbHandle {
    fn get_all(user_id: i32)
        -> impl Future<Output = Result<
            Vec<ToDoItem>
        >, NanoServiceError>> + Send {
        async fn run(user_id: i32)
            -> Result<Vec<ToDoItem>, NanoServiceError> {
            . . .
        }
        run(user_id)
    }
}
```

Again, we can create an error if we pass in a certain user ID, so we can test multiple outcomes with the same mock for different tests. The code inside our internal run function takes the following form:

```
// nanoservices/to_do/networking/actix_server/
// src/api/basic_actions/delete.rs
if user_id == 2 {
    return Err(
        NanoServiceError::new(
            "error getting items got get all".to_string(),
            NanoServiceErrorStatus::Unknown
        )
    )
}
```

```

    )
}
Ok(generate_get_all_return())

```

For other servers such as Rocket, Hyper, and Actix, we have the same implementations as the mocks that we have just done here, as the trait implementations are independent of the web framework used. For Rocket, we have been using the macro, which makes mocking harder. We can get around this by having an inner function that accepts traits. Our inner function for the Rocket delete function would take the following form:

```

pub async fn delete_by_name_generic<T, X>(
    token: HeaderToken,
    name: &str
) -> Result<Json<AllToDoItems>, NanoServiceError>
where
    T: DeleteOne + GetAll,
    X: GetUserSession
{
    let user = X::get_user_session(token.unique_id).await?;
    let _ = delete_core:::<T>(name, user.user_id).await?;
    Ok(Json(get_all_core:::<T>(user.user_id).await?))
}

```

Then our API wrapper is defined with the following code:

```

#[delete("/delete/<name>")]
pub async fn delete_by_name(
    token: HeaderToken, name: &str
) -> Result<Json<AllToDoItems>, NanoServiceError> {
    delete_by_name_generic:::
        SqlxPostgresDescriptor,
        RedisSessionDescriptor
    >(token, name).await
}

```

With these trait implementations, our database handle is ready for our tests, so we can now move on to mocking our user session cache.

Mocking our user cache handle

For our mock cache handle, we implement the GetUserSession trait with the following code:

```

// nanoservices/to_do/networking/actix_server/
// src/api/basic_actions/delete.rs
struct MockUserSessionHandle;

```

```
impl GetUserSession for MockUserSessionHandle {
    fn get_user_session(unique_id: String) ->
        impl Future<Output = Result<
            UserSession,
            NanoServiceError
        >> {
        async fn run(unique_id: String)
            -> Result<UserSession, NanoServiceError> {
            . . .
        }
        run(unique_id)
    }
}
```

Inside our run internal function, we can return two different user IDs and an error with the following code:

```
// nanoservices/to_do/networking/actix_server/
// src/api/basic_actions/delete.rs
if unique_id == "break" {
    return Err(
        NanoServiceError::new(
            "User not found".to_string(),
            NanoServiceErrorStatus::NotFound
        )
    )
}
if unique_id == "2" {
    return Ok(UserSession {
        user_id: 2
    })
}
Ok(UserSession {
    user_id: 1
})
```

With this, all our mock handles have been defined and are ready to be used in all our tests. We can now move on to defining our mock service.

Defining our mock service

For our service, we could spin up a server. However, we would have to find a local port that does not clash with another process and wait for the server to run in either a different thread or async task. Instead, we can build a testing service with the following code:

```
// nanoservices/to_do/networking/actix_server/
// src/api/basic_actions/delete.rs
async fn run_request(req: Request) -> ServiceResponse {
    let service = delete_by_name::<
        MockDbHandle, MockUserSessionHandle
    >;
    let app = init_service(App::new().route(
        "/delete/{name}",
        web::delete().to(service)
    )).await;
    call_service(&app, req).await
}
```

Here, we can see that we get the `delete_by_name` function with our mock handles, associate the function with an endpoint and delete method using the `init_service` function, and then use the `call_service` function provided by actix to call the service with the request that we pass into the function. Our `run_request` function will be called in all our tests.

With this service defined, we can now move on to writing our tests.

Writing tests

For our tests, we create a test request, pass this request to the `run_request` function, process the response, and assert checks on the values of that response. For our first test, where everything is okay, we have the following signature:

```
// nanoservices/to_do/networking/actix_server/
// src/api/basic_actions/delete.rs
#[tokio::test]
async fn test_delete_ok() {
    std::env::set_var("JWT_SECRET", "secret");
    . . .
}
```

Here, we set the environment variable so we can encode the header token. We do not know when each test will run or the order, so to be safe, we define the environment variable at the start of every test that requires the header token.

Inside our test, we create a request and make a call to the service with the following code:

```
// nanoservices/to_do/networking/actix_server/
// src/api/basic_actions/delete.rs
let req = TestRequest::delete()
    .insert_header(ContentType::json())
    .insert_header((
        "token",
        HeaderToken{
            unique_id: "test_id".to_string()
        }.encode().unwrap()
    ))
    .uri("/delete/coding")
    .to_request();
let resp = run_request(req).await;
```

We then process the HTTP response with the following code:

```
// nanoservices/to_do/networking/actix_server/
// src/api/basic_actions/delete.rs
let status = resp.status().as_u16();
let raw_body = resp.into_body().try_into_bytes().unwrap();
let body_str = std::str::from_utf8(&raw_body).unwrap();
let body: AllToDoItems = serde_json::from_str(
    body_str
).unwrap();
```

Finally, we assert that the response was okay and that the body contains the items that we are expecting from the mock with the following code:

```
// nanoservices/to_do/networking/actix_server/
// src/api/basic_actions/delete.rs
assert_eq!(status, 200);
assert_eq!(
    body,
    AllToDoItems::from_vec(generate_get_all_return())
).unwrap();
```

With this, our test will work just by running the test.

To test an invalid token, we must pass in an invalid token that would not decode properly. At this point, it would be a good chance to try and code the test yourself. If you attempted the invalid token test, hopefully, it looks like the following code:

```
// nanoservices/to_do/networking/actix_server/
// src/api/basic_actions/delete.rs
#[tokio::test]
async fn test_delete_invalid_token() {
    // configure the service
    std::env::set_var("JWT_SECRET", "secret");

    // make the request and get the response
    let req = TestRequest::delete()
        .insert_header(ContentType::json())
        .insert_header(("token", "test"))
        .uri("/delete/coding")
        .to_request();
    let resp = run_request(req).await;

    // extract the status and body
    let status = resp.status().as_u16();
    let raw_body = resp.into_body().try_into_bytes().unwrap();
    let body_str = std::str::from_utf8(&raw_body).unwrap();

    // assert the status and body
    assert_eq!(status, 401);
    assert_eq!(body_str, "\"token not a valid string\"");
}
```

We can then move on to creating a test for when the user is not found. Here, we must pass in a string that will get the user session cache to return a not found. If you attempted to write the test yourself, hopefully it looks like the following code:

```
// nanoservices/to_do/networking/actix_server/
// src/api/basic_actions/delete.rs
#[tokio::test]
async fn test_delete_user_not_found() {
    // configure the service
    std::env::set_var("JWT_SECRET", "secret");

    // make the request and get the response
    let req = TestRequest::delete()
        .insert_header(ContentType::json())
```

```

        .insert_header((
            "token",
            HeaderToken{
                unique_id: "break".to_string()
            }.encode().unwrap()
        ))
        .uri("/delete/coding")
        .to_request();
let resp = run_request(req).await;

// extract the status and body
let status = resp.status().as_u16();
let raw_body = resp.into_body().try_into_bytes().unwrap();
let body_str = std::str::from_utf8(&raw_body).unwrap();

// assert the status and body
assert_eq!(status, 404);
assert_eq!(body_str, "\"User not found\"");
}

```

For our item not found error, we must pass in the name to the delete handle mock to return an error. If you attempted to write this test yourself, hopefully it looks like the following code:

```

// nanoservices/to_do/networking/actix_server/
// src/api/basic_actions/delete.rs
#[tokio::test]
async fn test_delete_item_not_found() {
    // configure the service
    std::env::set_var("JWT_SECRET", "secret");

    // make the request and get the response
    let req = TestRequest::delete()
        .insert_header(ContentType::json())
        .insert_header((
            "token",
            HeaderToken{
                unique_id: "test_id".to_string()
            }.encode().unwrap()
        ))
        .uri("/delete/break")
        .to_request();
    let resp = run_request(req).await;
}

```

```

// extract the status and body
let status = resp.status().as_u16();
let raw_body = resp.into_body().try_into_bytes().unwrap();
let body_str = std::str::from_utf8(&raw_body).unwrap();

// assert the status and body
assert_eq!(status, 404);
assert_eq!(body_str, "\"Item not found\"");
}

```

Finally, we need to test the last remaining failure case: when `get_all` fails while building the response. Here, we must have a user ID that triggers this error. If you attempted to write the test, hopefully it looks like the following code:

```

// nanoservices/to_do/networking/actix_server/
// src/api/basic_actions/delete.rs
#[tokio::test]
async fn test_delete_get_all_error() {
    // configure the service
    std::env::set_var("JWT_SECRET", "secret");

    // make the request and get the response
    let req = TestRequest::delete()
        .insert_header(ContentType::json())
        .insert_header((
            "token", HeaderToken{
                unique_id: "2".to_string()
            }).encode().unwrap()
        ))
        .uri("/delete/coding")
        .to_request();
    let resp = run_request(req).await;

    // extract the status and body
    let status = resp.status().as_u16();
    let raw_body = resp.into_body().try_into_bytes().unwrap();
    let body_str = std::str::from_utf8(&raw_body).unwrap();

    // assert the status and body
    assert_eq!(status, 500);
    assert_eq!(body_str, "\"error getting items got get all\"");
}

```


We are now done with our unit tests for an HTTP endpoint! Now we must update our testing script, giving us the following total contents for the script:

```
#!/usr/bin/env bash
# scripts/run_tests.sh

# navigate to directory
SCRIPTPATH="$( cd "$(dirname "$0")" ; pwd -P )"
cd $SCRIPTPATH

cd ..

cargo test -p to-do-core
cargo test -p to-do-actix-server
```

If we run this script, we will see all our tests run and pass, and we will see that our tests pass with the following printout:

```
Running unittests src/lib.rs
(target/debug/deps/actix_server-e72261cb22d7ffc0)

running 5 tests
test api::basic_actions::delete
::tests::test_delete_user_not_found ... ok
test api::basic_actions::delete
::tests::test_delete_invalid_token ... ok
test api::basic_actions::delete
::tests::test_delete_get_all_error ... ok
test api::basic_actions::delete
::tests::test_delete_ok ... ok
test api::basic_actions::delete
::tests::test_delete_item_not_found ... ok

test result: ok. 5 passed; 0 failed;
0 ignored; 0 measured; 0 filtered out; finished in 0.01s
```

We are now finished with our network testing. We could move on to testing all the endpoints and functions, but we wouldn't add any new concepts, just a lot of repetition.

So far, we have been mocking the outputs from calls to the database, but what if our SQL code is incorrect? In our last stretch of unit testing, we will test our data access layer.

Unit testing our data access layer

Unit tests are where the test checks a small unit of code. Testing the data access layer is no exception. However, it is going to be slightly different from our other unit tests. The data access layer is the interface we have with the database. This means that if we mock the database calls, our tests will be close to useless. Because of this, we must coordinate our data access unit tests with the running of the databases. Because there are more moving parts, we must carry out the following steps if we want to unit test our data access layer:

1. Defining a teardown SQL command
2. Creating a **data access layer (DAL)** unit test
3. Refactoring our testing script

We will start with the teardown SQL command.

Defining a teardown SQL command

Because our tests rely on the interaction of the database, the state of the database can affect the test and make the test flaky, meaning that sometimes they will pass and other times they will not. Sadly, I have seen the *run it a few times and the test suite should pass* attitude a bit too much in some projects. We want our test to work every time. Therefore, we must have a teardown command that completely wipes the database. Therefore, we can ensure that the database state is what we want at the start of every test. Other testing modules in other workspaces might also interact with a database. We know that, in the next chapter, we will be doing end-to-end testing, so we will certainly want to use a teardown command there too. As you have probably guessed, to enable all our workspaces to access such commands, we are going to put the teardown command in the glue module. Our teardown SQL statement takes the following form:

```
// glue/src/sql_command.rs
pub static WIPE_DB: &str = "
DO $$
DECLARE
    r RECORD;
BEGIN
    -- Disable referential integrity checks
    SET session_replication_role = replica;

    -- Loop through all table names and drop them
    FOR r IN (
        SELECT tablename FROM pg_tables WHERE schemaname = 'public'
    ) LOOP
        EXECUTE 'DROP TABLE IF EXISTS ' ||
            quote_ident(r.tablename) ||
            ' CASCADE';
    END LOOP;
```

```

-- Re-enable referential integrity checks
SET session_replication_role = DEFAULT;
END $$;
";

```

It is a SQL statement that does not look familiar, but we can go through this. The \$\$ symbols are used as a delimiter for the block. You can use other delimiters, but \$\$ is commonly used:

- **DECLARE:** This section is used to declare any variables that will be used within the block. In this case, the variable `r`, of type `RECORD`, is declared. A `RECORD` type is a flexible data type that can hold a single row of data with columns that might vary.
- **BEGIN . . . END:** This block contains the executable part of the code where the actual operations are performed.
- **SET session_replication_role = replica;** This command temporarily disables the enforcement of foreign key constraints. By setting `session_replication_role` to `replica`, it tells PostgreSQL to behave as if it is applying changes replicated from another server, which often involves bypassing some constraint checks.
- **FOR r IN (SELECT tablename FROM pg_tables WHERE schemaname = 'public');** This loop iterates over all table names in the public schema. The `pg_tables` system catalog contains information about all tables in the database, and this query selects all table names where the schema is public. We do not want to mess with tables outside of public; we could mess with the postgres internals.
- **EXECUTE 'DROP TABLE IF EXISTS' || quote_ident(r.tablename) || 'CASCADE';** Inside the loop, this dynamic SQL command constructs and executes a `DROP TABLE IF EXISTS` statement for each table. The `quote_ident` function is used to safely quote the table name, preventing SQL injection and handling special characters or reserved keywords in table names. The `CASCADE` option ensures that any dependent objects (like foreign key constraints, views, etc.) are also dropped.

We now have a command that is going to wipe our database, no matter what tables we have. This will also wipe our migrations reference, as migrations are denoted by a migrations table in the public space, so migration tools can keep track of what migrations to apply. With all the tables dropped, running the migrations will set up all the tables we need with no rows.

Now that we have our teardown command done, we can move on to building our unit test for our data access layer.

Creating a DAL unit test

To test our delete functionality for our data access layer, we initially need the following outline:

```

// nanoservices/to_do/dal/src/to_do_items/transactions/delete.rs
#[cfg(feature = "sqlx-postgres")]
#[cfg(test)]
mod tests {

```

```

use super::*;
use crate::migrations::run_migrations;
use glue::sql_commands::WIPE_DB;

async fn setup() {
    sqlx::query(WIPE_DB).execute(&SQLX_POSTGRES_POOL)
        .await
        .unwrap();

    run_migrations().await;
}

#[tokio::test]
async fn test_sqlx_postgres_delete_one_ok() {
    setup().await;
    . . .
}
}

```

Here, we can see that our `|` function, which we run at the start of every test that we want to run, wipes the database and then runs the migrations. We can also see that our test module is guarded by the `sqlx` feature. This is because we do not have trait implementations of the delete transaction for postgres unless we are using the `sqlx` feature. This also means that we must enable the `sqlx` feature. Inside our `test_sqlx_postgres_delete_one_ok` function, after the `setup` function is fired, we start off our test by defining the data that we are going to be inserting with the following code:

```

// nanoservices/to_do/dal/src/to_do_items/transactions/delete.rs
let title = "test_sqlx_postgres_delete_one".to_string();
let user_id = 1;
let item = TodoItem {
    id: 1,
    title: title.clone(),
    status: "PENDING".to_string()
};

```

We can now use these definitions to insert and compare against. We then insert the item into the database with the following code:

```

// nanoservices/to_do/dal/src/to_do_items/transactions/delete.rs
let item = sqlx::query_as::<_, TodoItem>("
    INSERT INTO to_do_items (title, status)
    VALUES ($1, $2)
    RETURNING *"
).bind(item.title)
.bind(item.status.to_string())
.fetch_one(&SQLX_POSTGRES_POOL).await.unwrap();

```

We also must remember that there is a user connection associated with the item, so we also insert the user connection with the following code:

```
// nanoservices/to_do/dal/src/to_do_items/transactions/delete.rs
let _ = sqlx::query("
    INSERT INTO user_connections (user_id, to_do_id)
    VALUES ($1, $2)"
).bind(user_id)
.bind(item.id)
.execute(&*SQLX_POSTGRES_POOL).await.unwrap();
```

We can now delete our item with the function that we are testing with the following code:

```
// nanoservices/to_do/dal/src/to_do_items/transactions/delete.rs
let result = sqlx_postgres_delete_one(
    title.clone(),
    user_id
).await.unwrap();
assert_eq!(result.title, title);
```

With the assert, we ensure that we deleted the item that we intended to by checking the title.

We can now start performing direct SQL queries on the database to double-check that our database transaction went through and influenced the state. We assert that the item is no longer in the database with the following code:

```
// nanoservices/to_do/dal/src/to_do_items/transactions/delete.rs
let result = sqlx::query_as::<_, ToDoItem>("
    SELECT * FROM to_do_items
    WHERE title = $1"
).bind(title)
.fetch_optional(&*SQLX_POSTGRES_POOL).await.unwrap();
assert!(result.is_none());
```

And we then assert that there are no longer any user connections with the following code:

```
// nanoservices/to_do/dal/src/to_do_items/transactions/delete.rs
let result = sqlx::query_as::<_, (i64,)>("
    SELECT COUNT(*) FROM user_connections
    WHERE user_id = $1 AND to_do_id = $2"
).bind(user_id)
.bind(item.id)
.fetch_one(&*SQLX_POSTGRES_POOL).await.unwrap();
assert_eq!(result.0, 0)
```

With this, our test is ready to run. Our test for trying to delete an item that does not exist is more straightforward, as we do not have to put anything into the database. Instead, we just try and delete an item straight away with the following code:

```
// nanoservices/to_do/dal/src/to_do_items/transactions/delete.rs
#[tokio::test]
async fn test_sqlx_postgres_delete_no_existing_item() {
    setup().await;
    let title = "placeholder".to_string();
    let user_id = 1;
    let result = sqlx_postgres_delete_one(
        title.clone(), user_id
    ).await;
    assert!(result.is_err());
}
```

With these two tests, we can now move on to refactoring our test script to run our data access tests, because we need to run and then shut down our databases.

Refactoring our testing script

So far, our testing script tests two different workspaces, and we are about to add the data access layer testing, making it three. As you add more tests, the terminal printout will become a mess, and you will spend most of your time scrolling through lengthy terminal printouts just trying to find the service that you want to inspect. We now must coordinate database access for tests. As always, a key part of succeeding in software engineering is constantly reassessing and managing your complexity. We are going to rewrite our testing bash script, so we run all our unit tests that do not need any state and stream the output into their own log files. We can then spin up our docker-compose and run our tests that need access to those databases, shutting down the database afterwards. Before we write any testing bash, we must ensure that we have the following .env file in the root directory of the entire system:

```
TO_DO_DB_URL=postgres://username:password@localhost/to_do
AUTH_DB_URL=postgres://username:password@localhost/to_do
AUTH_API_URL=http://127.0.0.1:8081
JWT_SECRET=secret
CACHE_API_URL=redis://127.0.0.1:6379
ELASTICSEARCH_URL=http://localhost:9200/logs/_doc
```

We can now move over to the testing script. First, we have the boilerplate code moving the working directory to the directory where the bash script is and moving out to the root directory with the following code:

```
#!/usr/bin/env bash
# scripts/run_tests.sh

# navigate to directory
```

```
SCRIPTPATH="$( cd "$(dirname "$0")" ; pwd -P )"
cd $SCRIPTPATH
cd ..
```

We then remove the logs directory if the logs directory is present, and then create the logs directory with the following code:

```
# scripts/run_tests.sh
if [ -d "./logs" ]; then
    echo "Removing existing log directory: ./logs"
    rm -rf "./logs"
fi
mkdir logs
```

We can now create a function where we run the `pg_isready` function in the postgres docker container to verify that the database is ready to accept connections with the following code:

```
# scripts/run_tests.sh
function check_postgres_ready() {
    docker compose exec -T postgres pg_isready \
        -U username -d to_do
}
```

We then export the environment variables from the `|` file, and then run our unit tests, piping the results of those tests into their own log files with the following code:

```
# scripts/run_tests.sh
export $(cat .env | xargs)

cargo test -p to-do-core > ./logs/to-do-core.log
cargo test -p to-do-actix-server > ./logs/to-do-actix-server.log
```

With our unit tests that do not need any databases out of the way, we then move on to running our docker-compose in the background and then looping until we can access the database with the following code:

```
# scripts/run_tests.sh
Docker compose up -d

# Wait for PostgreSQL to be ready
echo "Waiting for PostgreSQL to be ready..."
until check_postgres_ready; do
    echo -n "."
    sleep 1
done

echo "PostgreSQL is ready."
```

Finally, we run our data access unit tests and pipe the output into the `dal.log` file, shutting down the database after the tests have finished with the following code:

```
# scripts/run_tests.sh
export TO_DO_MAX_CONNECTIONS=1
cargo test -p dal --features sqlx-postgres \
-- --test-threads=1 > ./logs/dal.log

Docker compose down
```

It must be noted that we set the max connections to one. At the time of writing this, there is a known issue on `sqlx` where if the max connections are higher than one, there are issues with timeouts on testing suites. Hopefully this error will be fixed in the future, and this issue will not affect our server in production. We can now run our tests, resulting in the following content for our data access layer log file:

```
// Logs/dal.log

running 2 tests
test to_do_items::transactions::delete::tests::
test_sqlx_postgres_delete_no_existing_item ... ok
test to_do_items::transactions::delete::tests::
test_sqlx_postgres_delete_one_ok ... ok

test result: ok. 2 passed; 0 failed; 0 ignored;
0 measured; 0 filtered out; finished in 0.12s

running 0 tests

test result: ok. 0 passed; 0 failed; 0 ignored;
0 measured; 0 filtered out; finished in 0.00s
```

We can see that both of our tests pass. With this done, we can conclude that we have managed to test every layer.

Summary

In this chapter, we built unit tests and mocks. We then configured our mocks to pass in a range of different outcomes to see how our code behaves under a range of different edge cases. Think back to when you were developing the application throughout the book. You would have to spin up the server and configure a range of requests to get to the piece of code you wanted to test, then manually perform the test that you wanted. This is very time-consuming, especially if you want to test your endpoint for a range of different conditions. Our unit tests not only give us feedback within seconds from an automated process, but they are also baked in, meaning that we can run one command to run all our tests within seconds whenever we change any of the code.

Speeding up the feedback on how our code runs will always result in faster development time. This is why good, experienced developers practice test-driven development. This is where we write the tests as we are writing the code, resulting in instant isolated feedback on how our code acts as we are writing it. While unit testing will greatly speed up your development time and get rid of bugs in your logic before those bugs have time to hide in a bigger system, there is a criticism that while individual modules work well, the sum of all these modules might not work together. This is where end-to-end testing comes in. In the next chapter, we perform end-to-end testing where we spin up the entire system and run atomic tests to see how the entire system runs in different deployment states.

Questions

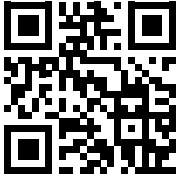
1. At a high level, how does unit testing speed up our development time?
2. How do we mock the return of a database transaction?
3. What is a setup function, and when would we typically use it?
4. What is a flaky test?
5. What are two precautions you can employ to avoid flaky tests?

Answers

1. Because we are only testing the unit of code that we are interested in, we do not have to perform a range of manual steps before getting to the unit of code that we are interested in. Due to mocking, we can also test a range of edge cases instantly. The quick feedback enables us to iterate through our code at a quicker rate and catch bugs we write before they can hide deeper in the code.
2. Because we are using traits for interfaces for the database transactions, we can create another struct and implement the database trait for that struct. Because the struct is just for the test, we can hardcode the response for the database transaction.
3. A setup function fires before a test. We generally use setup functions when there is a stateful component to the test. For instance, if we were using a database for the state, our setup would wipe the database and run migrations, ensuring that all of the test is deterministic.
4. A flaky test is when other random forces alter the outcome of the test, making the outcome of the test random, resulting in the test randomly failing and passing.
5. Wipe stateful components before the test and run your tests in a single-threaded context to avoid concurrency issues when interacting with the state.

Learn more on Discord

To join the Discord community for this book – where you can share feedback, ask questions to the author, and learn about new releases – follow this QR code: <https://packt.link/EaKXL>



15

End-to-End Testing

Unit testing gives us quick feedback on how our units of code work under a range of different inputs. However, how does the system work as a whole? Do all our units of code work when running together and communicating with each other? In this chapter, we are going to build **end-to-end tests** that automatically set up our system as a microservice cluster and perform a series of tests that make requests to the entire system running, and then inspect the responses. This chapter will cover the following:

- The importance of end-to-end testing
- Building our to-do HTTP kernel
- Building our atomic end-to-end tests
- Testing workflows with end-to-end tests

By the end of this chapter, you will be able to build automated pipelines that construct a microservice cluster and run a series of tests where the outcome of those tests is written to individual log files for you to inspect. You will also know about the difference between atomic end-to-end testing and testing workflows. We can get started by understanding the importance of end-to-end testing.

Technical requirements

You can download the example project and code for this book by following the instructions in the **Download the example code files** section in the *Preface* of this book.

This chapter's code files are included in the downloadable code bundle.

The importance of end-to-end testing

End-to-end testing is different from **unit testing**. Unit testing is where we break down the code into small units. These small units are then tested with a range of inputs. The feedback for unit testing is quick, enabling us to develop and iterate code at a quick rate. However, it is not guaranteed, when the units are combined as a system, that they will work together. To ensure that all our code units work together as we expected, we implement end-to-end tests. End-to-end tests are slower than unit tests. Like data access layer testing, we must spin up databases and alter the state of those databases between tests.

End-to-end testing enables us to validate the user experience to simulate various possible user journeys. In this chapter, we will perform an atomic end-to-end test and build a workflow so we can see whether the user journey makes sense. For instance, we could have an extra field for our items where the item needs to be approved before it is displayed and we can perform actions on it. We could unit test the creation of the item. We could unit test the alteration and deletion of the item, asserting that errors are thrown if we pass in an item that has not been approved by our mocks when trying to edit or delete the item. However, if we do not have an endpoint that approves the item to be edited or deleted, then the workflow is broken. Our unit tests would then pick this up. This is, however, quicker than manual steps. We can see in *Figure 15.1* that tethering to the database for the end-to-end test highlights the break in the workflow, failing the test:

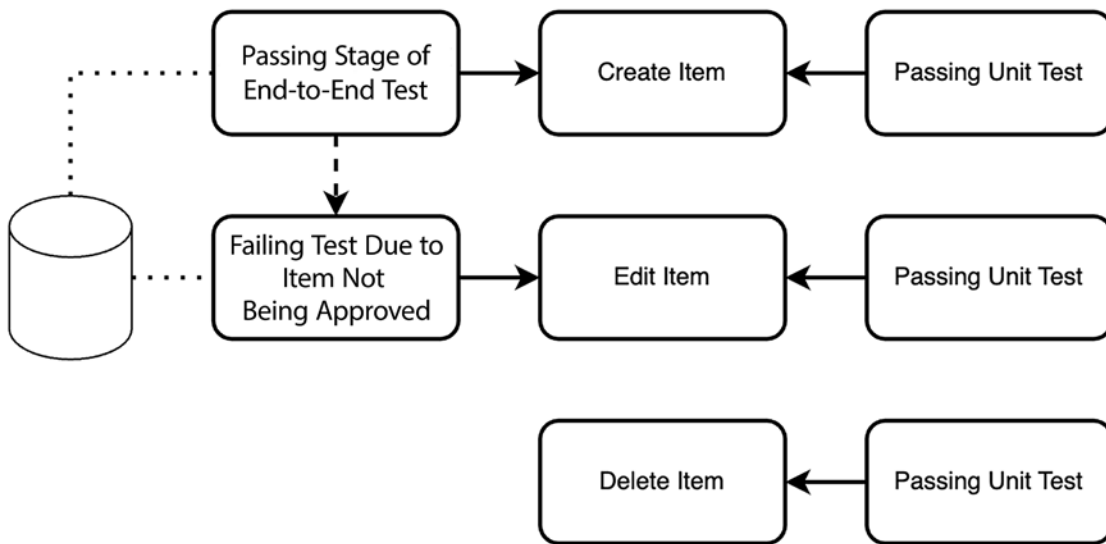


Figure 15.1 – An end-to-end test failing due to an inconsistent workflow while all unit tests pass

Not only does end-to-end testing help with the testing of entire workflows, but it also tests how all our services communicate with each other. For instance, we could have a circular dependency where service A requires something from service B. However, for service B to complete the request, it could require something from service A. If this circular dependency were written in just one program, the program would refuse to compile. However, if the services are communicating over a network call, our compiler will not detect this circular dependency, as these services are different programs, as we can see depicted in *Figure 15.2*:

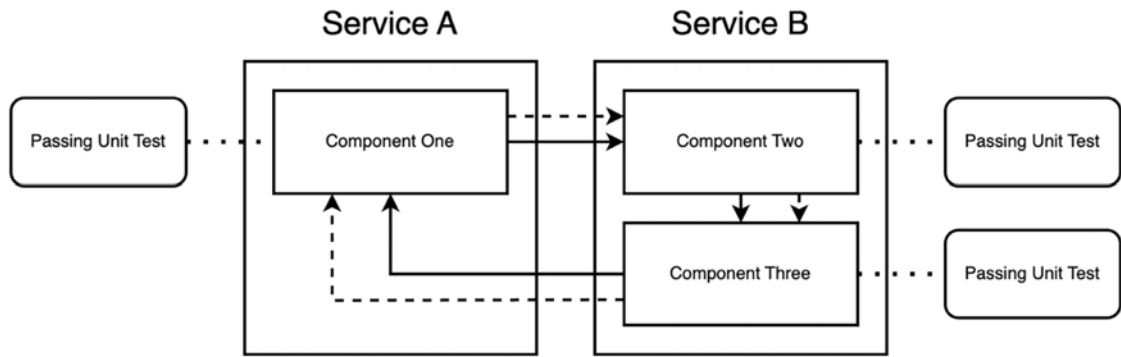


Figure 15.2 – a circular dependency (first cycle of request = hard arrow, second cycle of request = dashed arrow, other cycles after are omitted, as the cycles are infinite)

In the preceding figure, we can see that all the unit tests pass because we are mocking the inputs. However, the circular dependency will result in an infinite loop and the request for the result will timeout. Considering the circular dependency, we can see that an end-to-end test would highlight potential problems that do not get caught by unit tests.

With this, we can see where unit testing and end-to-end testing are utilized. Unit tests are effective at speeding up development, enabling us to inspect how the code works on a granular level. We then move over to end-to-end tests to test the system as a whole and how our system will run in production. We need to run end-to-end tests before we deploy anything.

A halfway measure between end-to-end tests and unit tests is **atomic end-to-end testing**. This is where the database is wiped between each HTTP request. This can lead to quick code iteration and testing how the system will run in production. For instance, the circular dependency between two different servers will be detected in an atomic end-to-end test. However, errors and gaps in workflows will not be detected. In this chapter, we will be building out an atomic end-to-end test for our delete endpoint, and then a full end-to-end workflow test for our delete endpoint.

Now that we understand the importance of end-to-end testing and where end-to-end testing is used, we can move on to building our end-to-end testing system. We can start by creating HTTP kernels for our to-do nanoservice for our atomic end-to-end test.

Building our to-do HTTP kernel

For now, we are going to create the to-do HTTP kernels so we can do the atomic end-to-end test for deleting a to-do item. Once the atomic end-to-end test works, we can then move on to building our auth HTTP kernel so we can test an entire workflow. For our atomic end-to-end test, we focus on the to-do HTTP kernel so we can perform database operations before and after the test, calling the HTTP API for the atomic end-to-end test.

Before we build our HTTP functions, now is the time to sit back and look at the bigger picture. We are going to be calling multiple endpoints, and as our system grows, the number of HTTP calls will increase. All these calls will require knowing the URL of services. We get the URL via the environment variables. To keep the getting of URLs simple and easy to maintain, we can group them in our glue module. For our to-do URL, we have the following:

```
// glue/src/urls.rs
use super::errors::{NanoServiceError, NanoServiceErrorStatus};

pub fn get_to_do_url() -> Result<String, NanoServiceError> {
    std::env::var("TO_DO_API_URL").map_err(|e|{
        NanoServiceError::new(e.to_string(),
        NanoServiceErrorStatus::BadRequest)
    })
}
```

We must also ensure that we have made this public in our glue workspace with the following code:

```
// glue/src/lib.rs
...
pub mod urls;
...
```

There is one last thing. We will be making HTTP requests, and these HTTP requests will return numbers as the response codes. We must add a function in our error status to construct the error status from the raw code returned with the following code:

```
// glue/src/errors.rs
impl NanoServiceErrorStatus {
    pub fn from_code(code: u16) -> NanoServiceErrorStatus {
        match code {
            404 => NanoServiceErrorStatus::NotFound,
            403 => NanoServiceErrorStatus::Forbidden,
            400 => NanoServiceErrorStatus::BadRequest,
            409 => NanoServiceErrorStatus::Conflict,
            401 => NanoServiceErrorStatus::Unauthorized,
            _ => NanoServiceErrorStatus::Unknown,
        }
    }
}
```

We can now access the to-do service URL from any workspace in our system. We can also sleep well at night knowing that there are not going to be multiple different environment variable names for the to-do URL. We can now move on to building the HTTP endpoints for our to-do kernel.

Like what we did for the auth kernel, we want to add an HTTP feature to ensure that we can support different mechanisms in the future, including directly calling the core functions. For this chapter, we are only going to focus on the HTTP feature to avoid bloat. At this stage in the book, you have seen how to support other methods, so supporting other methods should be clear to you. For our to-do kernel, we have the following layout:

```

├─ nanoservices
│   └─ ...
│       └─ to_do
│           ├── core
│           │   └─ ...
│           ├── dal
│           │   └─ ...
│           └─ kernel
│               ├── Cargo.toml
│               └─ src
│                   ├── api
│                   │   ├── basic_actions
│                   │   │   ├── create.rs
│                   │   │   ├── delete.rs
│                   │   └─ mod.rs
│                   └─ mod.rs
└─ lib.rs

```

We are only going to use the delete endpoint in our atomic end-to-end test. However, we must use the create endpoint before we can delete the item. Seeing as we are looking at the to-do kernel, we might as well build both endpoints now.

We have the following layout for our `Cargo.toml` for our to-do kernel:

```

// nanoservices/to_do/kernel/Cargo.toml
[package]
name = "to-do-kernel"
version = "0.1.0"
edition = "2021"
[dependencies]
core = { path = "../core", optional = true }
dal = { path = "../dal" }
reqwest = {
    version = "0.12.5",
    optional = true,
    features = ["json"]
}

```

```

glue = { path = "../../glue" }
cache-client = {
    path = "../user-session-cache/cache-client"
}
[features]
http = ["dep:reqwest"]

```

Here, we can see that our http feature relies on the request dependency, and we have access to our data access layer, glue, cache client, and core. There is nothing that we cannot access in terms of our to-do server in this kernel.

At this stage, it is a good idea for you to make sure that both create and delete files are available outside of the library with public declarations in the `mod.rs` files. We can now focus on our create endpoints.

Building our create item HTTP kernel

To enable us to build our create endpoint, we can define the imports of our create HTTP request with the following code:

```

// nanoservices/to_do/kernel/src/api/basic_actions/create.rs
#[cfg(feature = "http")]
mod common_imports {
    pub use glue::errors::NanoServiceError;
    pub use dal::to_do_items::schema::NewToDoItem;
    pub use dal::to_do_items::enums::TaskStatus;
    pub use dal::to_do_items::schema::AllToDoItems;
    pub use glue::errors::NanoServiceErrorStatus;
    pub use glue::token::HeaderToken;
}
#[cfg(feature = "http")]
mod request_imports {
    pub use request::Client;
    pub use glue::urls::get_to_do_url;
}

```

Here, we see that we have split up the imports into two types: common and request. This means that if we add other methods in the future, we can keep the imports for that new feature separate.

For our create interface, we have the following function:

```

// nanoservices/to_do/kernel/src/api/basic_actions/create.rs
#[cfg(any(feature = "http"))]
pub async fn create_item(
    token: common_imports::HeaderToken,
    title: String,
    status: common_imports::TaskStatus
)

```



```

    ) -> Result<
        common_imports::AllToDoItems,
        common_imports::NanoServiceError
    > {
    #[cfg(feature = "http")]
    let user: common_imports::AllToDoItems = create_item_api_call(
        token,
        title,
        status
    ).await?.into();
    return Ok(user)
}

```

Here, even though it is a bit more code, we can see that we prefix our import references with `common_imports::`. By just looking at this, we can see that all the imports used in this function are from the common imports. Therefore, we will be able to add other feature support to this function later. For our API call, we have the following function signature:

```

// nanoservices/to_do/kernel/src/api/basic_actions/create.rs
#[cfg(feature = "http")]
async fn create_item_api_call(
    token: common_imports::HeaderToken,
    title: String,
    status: common_imports::TaskStatus
) -> Result<
    common_imports::AllToDoItems,
    common_imports::NanoServiceError
> {
    . . .
}

```

Inside the common function, we get our URL and define the body of our request with the following code:

```

// nanoservices/to_do/kernel/src/api/basic_actions/create.rs
let full_url = format!(
    "{}/api/v1/create",
    request_imports::get_to_do_url()?
);
let body = common_imports::NewToDoItem {
    title,
    status
};

```

Now that we have our URL, we can move on to building our client and sending a request with the following code:

```
// nanoservices/to_do/kernel/src/api/basic_actions/create.rs
let client = request_imports::Client::new();
let response = client
    .post(&full_url)
    .header("token", token.encode()?)
    .json(&body)
    .send()
    .await
    .map_err(|e| {
        common_imports::NanoServiceError::new(
            e.to_string(),
            common_imports::NanoServiceErrorStatus::BadRequest
        )
    })?;
```

Finally, we map the response code, returning all the items or an error if a created response was not returned with the following code:

```
// nanoservices/to_do/kernel/src/api/basic_actions/create.rs
let status_code = response.status().as_u16();

if status_code == 201 {
    let items = response
        .json::<common_imports::AllToDoItems>()
        .await
        .map_err(|e: request::Error|
            common_imports::NanoServiceError::new(
                e.to_string(),
                common_imports::NanoServiceErrorStatus::from_code(
                    status_code
                )
            ))?;
    return Ok(items)
} else {
    let error_message = response.text().await.unwrap_or_else(|_|
        "No error message".to_string()
    );
    return Err(common_imports::NanoServiceError::new(
        error_message,
        common_imports::NanoServiceErrorStatus::from_code(
```

```

        status_code
    )
))
}

```

Now that we have a create HTTP request for our to-do server ready, we can now move on to building our delete HTTP kernel endpoint.

Building our delete item HTTP kernel

Building our delete endpoint will follow the same approach as the create endpoint. This is a good opportunity for you to try to build it yourself. We start off with the following imports:

```

// nanoservices/to_do/kernel/src/api/basic_actions/delete.rs
#[cfg(feature = "http")]
mod common_imports {
    pub use glue::errors::NanoServiceError;
    pub use dal::to_do_items::schema::AllToDoItems;
    pub use glue::errors::NanoServiceErrorStatus;
}
#[cfg(feature = "http")]
mod request_imports {
    pub use request::Client;
    pub use glue::token::HeaderToken;
    pub use glue::urls::get_to_do_url;
}

```

Our delete interface is defined by the following code:

```

// nanoservices/to_do/kernel/src/api/basic_actions/delete.rs
#[cfg(any(feature = "http"))]
pub async fn delete_item(
    token: request_imports::HeaderToken,
    name: String
) -> Result<
    common_imports::AllToDoItems,
    common_imports::NanoServiceError
> {
    #[cfg(feature = "http")]
    let user: common_imports::AllToDoItems = delete_item_api_call(
        token, name
    ).await?.into();
    return Ok(user)
}

```

Then our API signature function has the following form:

```
// nanoservices/to_do/kernel/src/api/basic_actions/delete.rs
#[cfg(feature = "http")]
async fn delete_item_api_call(
    token: request_imports::HeaderToken,
    name: String
) -> Result<
    common_imports::AllToDoItems,
    common_imports::NanoServiceError
> {
    . . .
}
```

Inside our API call function, we get our URL with the following code:

```
// nanoservices/to_do/kernel/src/api/basic_actions/delete.rs
let full_url = format!(
    "{}api/v1/delete/{}",
    request_imports::get_to_do_url()?,
    name
);
```

We can then make the HTTP request with the following code:

```
// nanoservices/to_do/kernel/src/api/basic_actions/delete.rs
let client = request_imports::Client::new();
let response = client
    .delete(&full_url)
    .header("token", token.encode()?)
    .send()
    .await
    .map_err(|e| {
        common_imports::NanoServiceError::new(
            e.to_string(),
            common_imports::NanoServiceErrorStatus::BadRequest
        )
    })?;
```

Finally, we handle the response with the following code:

```
// nanoservices/to_do/kernel/src/api/basic_actions/delete.rs
let status_code = response.status().as_u16();

if status_code == 200 {
```

```

    let trimmed_user = response
      .json::<common_imports::AllToDoItems>()
      .await
      .map_err(|e| common_imports::NanoServiceError::new(
        e.to_string(),
        common_imports::NanoServiceErrorStatus::from_code(
          status_code
        )
      ))?;
    return Ok(trimmed_user)
  } else {
    return Err(common_imports::NanoServiceError::new(
      "Failed to delete item".to_string(),
      common_imports::NanoServiceErrorStatus::from_code(
        status_code
      )
    ))
  }
}

```

Here, our HTTP API endpoints are done for the create and delete items. We can now move on to building our atomic end-to-end tests.

Building our atomic end-to-end tests

We can house our atomic end-to-end tests in our networking directory with the following layout:

```

└─ nanoservices
  └─ to_do
    └─ networking
      ├── actix_server
      │   └─ ...
      └─ atomic-http-tests
          ├── Cargo.toml
          └─ src
              ├── api
              │   ├── basic_actions
              │   │   └─ delete.rs
              │   └─ mod.rs
              └─ mod.rs
          └─ main.rs

```

We can see that we have called our testing suite as HTTP atomic tests. We have not specified the framework. This is because our HTTP interface should be the same no matter what framework we use. So, if we slot an Axum server into the networking directory, we can run the same atomic end-to-end testing suite against the Axum implementation of our to-do server.

For our atomic end-to-end test module, we need the following dependencies:

```
// nanoservices/to_do/networking/atomic-http-tests/Cargo.toml

[package]
name = "to-do-atomic-http-tests"
version = "0.1.0"
edition = "2021"

[dependencies]
glue = { path = "../../../../../glue" }
to-do-kernel = { path = "../../kernel", features = ["http"] }
tokio = { version = "1.39.2", features = ["full"] }
cache-client = {
    path = "../../../../../user-session-cache/cache-client"
}
dal = { path = "../../dal", features = ["sqlx-postgres"]}
sqlx = {
    version = "0.7.4",
    features = ["postgres", "json", "runtime-tokio"]
}
```

By looking at the dependencies, we can see that we directly access the database using the data access layer and sqlx. We can also see that our code will be interacting with the user cache. This is because we must create a user auth session in the cache. Remember, our server checks the user auth session cache before we can perform the deletion of the to-do item.

Seeing as we are accessing the user cache, we must also add the URL to the glue module so we can pass the URL into our tests with the following code:

```
// glue/src/urls.rs

pub fn get_cache_url() -> Result<String, NanoServiceError> {
    std::env::var("CACHE_API_URL").map_err(|e|{
        NanoServiceError::new(e.to_string(),
            NanoServiceErrorStatus::BadRequest)
    })
}
```

We are now at the stage where we can write the atomic end-to-end test.

Writing the end-to-end test

The outline of our atomic end-to-end test takes the following form:

```
// nanoservices/to_do/networking/atomic-http-tests/
// src/api/basic_actions/delete.rs

#[cfg(test)]
mod tests {
    use to_do_kernel::api::basic_actions::delete::delete_item;
    use dal::connections::sqlx_postgres::SQLX_POSTGRES_POOL;
    use dal::to_do_items::schema::ToDoItem;
    use dal::migrations::run_migrations;
    use glue::sql_commands::WIPE_DB;
    use glue::token::HeaderToken;
    use glue::urls::get_cache_url;
    use cache_client::login;

    async fn setup() {
        sqlx::query(WIPE_DB).execute(
            &SQLX_POSTGRES_POOL
        ).await.unwrap();
        run_migrations().await;
    }
    #[tokio::test]
    async fn test_delete_item() {
        setup().await;
        . . .
    }
}
```

Here, we can see that the setup wipes the database and then runs the migrations. We then run the setup function at the start of the `test_delete_item` test.

Inside our `test_delete_item` function, we define some user credentials and log in to the user session cache with the following code:

```
// nanoservices/to_do/networking/atomic-http-tests/
// src/api/basic_actions/delete.rs

let user_id = "123".to_string();
let perm_user_id = 1;

login(
```

```

    &get_cache_url().unwrap(),
    &user_id,
    60,
    perm_user_id
  ).await.unwrap();

```

You can see that we are not doing any error handling in the test code. This is because we are testing the code, and if it errors, we need to know, but we do not need to handle errors to prevent the system from crashing or having unexpected behavior. In our test, it takes less code and is more direct to just unwrap the result so we can see exactly where the code breaks. We can also note that we do not have to create a user. If our user session is in the cache, then we can conclude that the user is logged in and exists. This is safe because no outside user will be able to directly access the cache when we deploy our application onto a server.

We then insert the item to be deleted into the database with the following code:

```

// nanoservices/to_do/networking/atomic-http-tests/
// src/api/basic_actions/delete.rs
let title = "test_sqlx_postgres_delete_one".to_string();
let item = TodoItem {
    id: 1,
    title: title.clone(),
    status: "PENDING".to_string()
};
let item = sqlx::query_as::<_, TodoItem>("
    INSERT INTO to_do_items (title, status)
    VALUES ($1, $2)
    RETURNING *"
).bind(item.title)
.bind(item.status.to_string())
.fetch_one(&*SQLX_POSTGRES_POOL).await.unwrap();

assert_eq!(item.title, title);
assert_eq!(item.status, "PENDING");
assert_eq!(item.id, 1);

```

Our assertions check that the insertion worked. We then insert the connection with the following code:

```

// nanoservices/to_do/networking/atomic-http-tests/
// src/api/basic_actions/delete.rs
let _ = sqlx::query("
    INSERT INTO user_connections (user_id, to_do_id)
    VALUES ($1, $2)"
).bind(perm_user_id)

```



```
.bind(item.id)
.execute(&*SQLX_POSTGRES_POOL).await.unwrap();
```

We now have everything set up for our API call. We perform the API call with the following code:

```
// nanoservices/to_do/networking/atomic-http-tests/
// src/api/basic_actions/delete.rs
let token = HeaderToken {
    unique_id: user_id.clone()
};
let outcome = delete_item(token, title).await.unwrap();
assert_eq!(outcome.done.len(), 0);
assert_eq!(outcome.pending.len(), 0);
```

Our assertions assert that there are no items returned. This is reassuring; however, we return items that are connected to the user. If we deleted the connection but not the item, we would return zero items from the API call, even if we did not technically delete the item. To properly ensure that the deletion fully worked, we do direct database calls to the connections and items table to ensure that there are no records with the following code:

```
// nanoservices/to_do/networking/atomic-http-tests/
// src/api/basic_actions/delete.rs
let items: Vec<ToDoItem> = sqlx::query_as::<_, ToDoItem>(
    "SELECT * FROM to_do_items;"
)
.fetch_all(&*SQLX_POSTGRES_POOL)
.await
.unwrap();
assert_eq!(items.len(), 0);

let connections: Vec<(i32, i32)> = sqlx::query_as::<_, (i32, i32)>(
    "SELECT * FROM user_connections;"
)
.fetch_all(&*SQLX_POSTGRES_POOL)
.await
.unwrap();
assert_eq!(connections.len(), 0);
```

With these direct database queries, we can ensure that our deletion worked the way we wanted it to. At this point, you can explore other tests, such as what happens if the user is not authenticated or if the item does not exist. Now that you know about the methods of testing, these tests should be trivial, and covering them here would needlessly bloat the chapter. Remember, this end-to-end atomic test is running everything. Our system is going to be running the database, cache, server, and logging database. Just trying to run this manually would be time-consuming and error-prone, and there is a chance that you would later forget how to run these tests unless they are documented. If we are documenting the steps, we might as well put these steps into our test script.

Running our atomic end-to-end test

When it comes to running our atomic end-to-end test, we must ensure that all our databases are running, our environment variables are loaded, and that our to-do server is running. At this stage, we should have a look at our overall system. Our workspaces in the root `Cargo.toml` file should look like the following:

```
[workspace]
resolver = "2"
members = [
    "glue",
    "ingress",
    "ingress/frontend/rust-interface",
    "nanoservices/auth/core",
    "nanoservices/auth/dal",
    "nanoservices/auth/kernel",
    "nanoservices/auth/networking/actix_server",
    "nanoservices/to_do/core",
    "nanoservices/to_do/dal", "nanoservices/to_do/kernel",
    "nanoservices/to_do/networking/actix_server",
    "nanoservices/to_do/networking/atomic-http-tests",
    "nanoservices/user-session-cache/cache-client",
    "nanoservices/user-session-cache/cache-module"
]
```

Here, we can see that our system is growing, but we are managing the complexity. For instance, we could break our to-do server off into its own repo to manage all the layers and tests for the server. We will cover how to manage multiple servers in different repos in *Chapter 19*.

For running our system, our `.env` file in the root of our system should house the following variables:

```
TO_DO_DB_URL=postgres://username:password@localhost/to_do
AUTH_DB_URL=postgres://username:password@localhost/to_do
AUTH_API_URL=http://127.0.0.1:8081
TO_DO_API_URL=http://127.0.0.1:8080
JWT_SECRET=secret
CACHE_API_URL=redis://127.0.0.1:6379
ELASTICSEARCH_URL=http://localhost:9200/logs/_doc
```

Finally, we can run our atomic end-to-end test by building the to-do server, running the to-do server in the background, and then running the tests by adding the following to the end of our tests script:

```
// scripts/run_tests.sh
. . .
export TO_DO_MAX_CONNECTIONS=1
cargo test -p dal --features sqlx-postgres \
-- --test-threads=1 > ./logs/dal.log

cargo build -p actix_server
cargo run -p actix_server &
to_do_server_pid=$!
sleep 1

cargo test -p to-do-atomic-http-tests -- \
--test-threads=1 --nocapture \
> ./logs/to-do-atomic-http-tests.log

Docker compose down
kill $to_do_server_pid
```

We can see that we add a sleep after we run the server, and we build the server before we run the server. We build the server in a different step because the running of the server has an & at the end of the command. This means that the server runs in the background. If we also run the build in the background, the build could take so long that we try to run the atomic end-to-end tests before the build is complete. We could try to guess how long the build will take, but this could change as the code base changes. It's best to just block the testing script for the exact amount of time it takes to build the server and then run it in the background. We get the process ID of the running of the server task by using `$!`. We can see that we then run our atomic end-to-end test, writing the outcome of the test to a log file, and then tear down the Docker containers and kill the process that is running the to-do server.

If we run our testing script, our `logs/to-do-atomic-http-tests.log` file should have the following contents:

```
running 1 test
test api::basic_actions::delete::tests::test_delete_item
... Migrating to-do database...
to-do database migrations completed: ()
ok

test result: ok. 1 passed; 0 failed; 0 ignored;
0 measured; 0 filtered out; finished in 0.09s
```

Here, we can see that our atomic end-to-end test worked with our entire system working together.

I personally find unit tests to be the tool I use when I am building code, as unit tests offer the quickest feedback and the easiest way to test the code under different inputs and outputs. I then move on to atomic end-to-end tests, as these are also quick – for instance, if you have an IDE that enables you to click and run a single test from your IDE, as seen in *Figure 15.3*:

A screenshot of the Visual Studio Code editor interface. At the top, there is a status bar with the text "#[tokio::test]" and a button labeled "Run Test | Debug". Below this, a Rust code snippet is displayed. The code defines an asynchronous function named "test_delete_item" which calls "setup().await", sets "user_id" to "123" and "perm_user_id" to 1, and then calls "login" with parameters "&get_cache_url().unwrap()", "&user_id", "60", and "perm_user_id". The function call is followed by ".await.unwrap()". The code is color-coded: keywords like "fn", "let", "login", and "await" are in blue, strings are in red, and numbers are in green. The background of the code editor has a light gray grid pattern.

```
#[tokio::test]
▶ Run Test | Debug
async fn test_delete_item() {
    setup().await;

    let user_id: String = "123".to_string();
    let perm_user_id: i32 = 1;

    login(
        &get_cache_url().unwrap(),
        &user_id,
        60,
        perm_user_id
    ).await.unwrap();
}
```

Figure 15.3 – Running an isolated test from the VS Code editor

When I’m developing against atomic end-to-end tests, I make sure I have one terminal running the Docker containers, another terminal running the server, and I configure my IDE to have the environment variables. This results in a super smooth development environment where I change the code, refresh the server, and then click the individual test correlated to the endpoint that I am updating. But, I stress, this does not beat the speed of feedback from a raw unit test.

Now that we have our atomic end-to-end test done, the final test we are going to build is testing workflows with end-to-end testing.

Testing workflows with end-to-end tests

For testing our workflows, we are going to focus on our delete endpoint. Seeing as this is the last section of the book that applies Rust code directly to the to-do application code base, a lot of the Rust code syntax should not be new. Also, a lot of the general approaches have been covered. This means that we will run through a lot of the code blocks with minimal explanations. This section has been kept in the chapter to show how all the general code concepts we have covered are orchestrated to run an end-to-end workflow that tests how the auth and to-do server work together. To get our workflow testing infrastructure done, we must carry out the following steps:

1. Configure existing code to accommodate the workflow tests.
2. Build a kernel API call for creating the user.
3. Build a kernel API call for logging in.

4. Create a workflows test workspace.
5. Create a delete workflow test.
6. Update our test runner bash script.

With these steps laid out, we can start configuring the existing code.

Configuring existing code to accommodate the workflow tests

The first task we can check off our list is adding the auth server URL to our glue module with the following code:

```
// glue/src/urls.rs
pub fn get_auth_url() -> Result<String, NanoServiceError> {
    std::env::var("AUTH_API_URL").map_err(|e|{
        NanoServiceError::new(e.to_string(),
            NanoServiceErrorStatus::BadRequest)
    })
}
```

We then add the base64 dependency for the basic auth for our auth kernel, and put the base64 into the http feature, resulting in the following Cargo file:

```
# nanoservices/auth/kernel/Cargo.toml
[package]
name = "auth-kernel"
version = "0.1.0"
edition = "2021"

[dependencies]
auth-core = { path = "../core", optional = true }
auth-dal = { path = "../dal" }
reqwest = { version = "0.12.5", optional = true, features = ["json"] }
glue = { path = "../../glue" }
cache-client = { path = "../../user-session-cache/cache-client" }
base64 = { version = "0.22.1", optional = true }

[features]
http = ["reqwest", "auth-core", "base64"]
core-postgres = ["auth-core"]
```

We then need to focus on the core of our auth server. Our `CreateUser` struct currently only has the `Deserialize` trait implemented. We are going to be making an HTTP call from a client, meaning we must serialize it. We implement the `Serialize` trait with the following code:

```
// nanoservices/auth/core/src/api/users/create.rs
...
use serde::{Deserialize, Serialize};

#[derive(Deserialize, Serialize)]
pub struct CreateUser {
    pub email: String,
    pub password: String
}
...
```

Our existing code is now ready to support our API endpoints. We can start with the create HTTP request for the kernel.

Building a kernel API call for creating a user

For our create HTTP endpoint for the kernel, we start off with the following imports:

```
// nanoservices/auth/kernel/src/api/users/create.rs
#[cfg(any(feature = "core-postgres", feature = "http"))]
mod common_imports {
    pub use glue::errors::NanoServiceError;
    pub use auth_core::api::users::create::CreateUser;
}
#[cfg(feature = "core-postgres")]
mod core_imports {
    pub use auth_dal::users::descriptors::SqlxPostGresDescriptor;
    pub use auth_core::api::users::create::create as create_user_core;
}
#[cfg(feature = "http")]
mod request_imports {
    pub use request::Client;
    pub use glue::errors::NanoServiceErrorStatus;
    pub use glue::urls::get_auth_url;
}
```

We create the interface for the create user API endpoint with the following code:

```
// nanoservices/auth/kernel/src/api/users/create.rs
#[cfg(any(feature = "core-postgres", feature = "http"))]
pub async fn create_user(user: common_imports::CreateUser)
```

```

-> Result<(), common_imports::NanoServiceError> {
#[cfg(feature = "core-postgres")]
core_imports::create_user_core::<
    core_imports::SqlxPostGresDescriptor
>(user).await?;
#[cfg(feature = "http")]
create_user_api_call(user).await?;
return Ok(())
}

```

Finally, we create the API call function with the following code:

```

// nanoservices/auth/kernel/src/api/users/create.rs
#[cfg(feature = "http")]
async fn create_user_api_call(user: common_imports::CreateUser)
-> Result<(), common_imports::NanoServiceError> {
    let url = request_imports::get_auth_url()?;
    let full_url = format!("{}/api/v1/users/create", url);
    let client = request_imports::Client::new();
    let response = client
        .post(&full_url)
        .json(&user)
        .send()
        .await
        .map_err(|e| {
            common_imports::NanoServiceError::new(
                e.to_string(),
                request_imports::NanoServiceErrorStatus::BadRequest
            )
        })?;
    let status_code = response.status().as_u16();
    if status_code != 201 {
        return Err(common_imports::NanoServiceError::new(
            response.text().await.unwrap_or_else(|_|
                "Unknown error".to_string()
            ),
            request_imports::NanoServiceErrorStatus::from_code(
                status_code
            )
        ))
    }
}
return Ok(())
}

```

Here, it must be noted that if the creation of the user is a success, then we do not return anything. This does not break our workflow because we know the email and password to log in. Now we must build our login API endpoint.

Building a kernel API call for logging in

For our login API call, we need the following imports:

```
// nanoservices/auth/kernel/src/api/auth/Login.rs
#[cfg(any(feature = "core-postgres", feature = "http"))]
mod common_imports {
    pub use glue::errors::NanoServiceError;
}
#[cfg(feature = "core-postgres")]
mod core_imports {
    pub use auth_dal::users::descriptors::SqlxPostGresDescriptor;
    pub use auth_core::api::auth::login::login as login_core;
}
#[cfg(feature = "http")]
mod request_imports {
    pub use request::Client;
    pub use glue::errors::NanoServiceErrorStatus;
    pub use glue::urls::get_auth_url;
    pub use base64::engine::general_purpose::STANDARD;
}
#[cfg(feature = "http")]
use base64::Engine;
```

Here, we can see that the Engine trait is imported outside of the modules. This is because the Engine trait needs to be in the same namespace to be utilized.

The interface for the login function takes the following form:

```
// nanoservices/auth/kernel/src/api/auth/Login.rs
#[cfg(any(feature = "core-postgres", feature = "http"))]
pub async fn login_user(email: String, password: String)
    -> Result<String, common_imports::NanoServiceError> {
    #[cfg(feature = "core-postgres")]
    let token = core_imports::login_core::<
        core_imports::SqlxPostGresDescriptor
    >(email, password).await?;
    #[cfg(feature = "http")]
    let token = login_user_api_call(email, password).await?.into();
    return Ok(token)
}
```


Our API call function has the following signature:

```
// nanoservices/auth/kernel/src/api/auth/Login.rs
#[cfg(feature = "http")]
async fn login_user_api_call(email: String, password: String)
    -> Result<String, common_imports::NanoServiceError> {
    . . .
}
```

Inside our login function, we gather the information needed for the request with the following code:

```
// nanoservices/auth/kernel/src/api/auth/Login.rs
let url = request_imports::get_auth_url()?;
let full_url = format!("{}/api/v1/auth/login", url);
let auth = format!("{}", email, password);
let encoded_auth = request_imports::STANDARD.encode(auth);
let auth_header_value = format!("Basic {}", encoded_auth);
```

We then make the login HTTP request with the basic auth method using the following code:

```
// nanoservices/auth/kernel/src/api/auth/Login.rs
let client = request_imports::Client::new();
let response = client
    .get(&full_url)
    .header("Authorization", auth_header_value)
    .send()
    .await
    .map_err(|e| {
        common_imports::NanoServiceError::new(
            e.to_string(),
            request_imports::NanoServiceErrorStatus::BadRequest
        )
    })?;
```

Finally, we process the response and return the token with the following code:

```
// nanoservices/auth/kernel/src/api/auth/Login.rs
let status_code = response.status().as_u16();
if status_code == 200 {
    let token: String = response.json().await.map_err(|e| {
        common_imports::NanoServiceError::new(
            e.to_string(),
            request_imports::NanoServiceErrorStatus::BadRequest
        )
    })?;
}
```

```

    return Ok(token)
}
else {
    let response_message = match response.json().await {
        Ok(json) => json,
        Err(_) => "Unknown error".to_string()
    };
    return Err(common_imports::NanoServiceError::new(
        response_message,
        request_imports::NanoServiceErrorStatus::from_code(
            status_code
        )
    ))
}

```

With this, our auth kernel interfaces are ready for our workflow test. Now there is nothing stopping us from creating the workflow test workspace.

Creating a workflow test workspace

It might seem logical to build workflow tests in the same workspace as the atomic tests. However, workflow tests require the entire system to be running, whereas the atomic end-to-end tests only require the to-do server to be working. It is always good to keep these separate to avoid clashes and your integrated tests breaking because you did not have the right infrastructure running at the time.

For our layout, the workflow testing suite takes the following form:

```

nanoservices
├── to_do
│   └── networking
│       └── http-workflow-tests
│           ├── Cargo.toml
│           └── src
│               ├── items
│               │   ├── delete.rs
│               │   └── mod.rs
│               └── main.rs

```

With this, our Cargo.toml for the workflow testing workspace is defined by the following code:

```

# nanoservices/to_do/networking/http-workflow-tests/Cargo.toml
[package]
name = "to-do-http-workflow-tests"
version = "0.1.0"
edition = "2021"

```

```

[dependencies]
to-do-kernel = { path = "../../kernel", features = ["http"] }
auth-kernel = { path = "../../auth/kernel", features = ["http"] }
auth-core = { path = "../../auth/core" }
tokio = { version = "1.16.1", features = ["full"] }
dal = { path = "../../dal", features = ["sqlx-postgres"] }
auth-dal = { path = "../../auth/dal" }
glue = { path = "../../glue" }
sqlx = {
    version = "0.7.4",
    features = ["postgres", "json", "runtime-tokio"]
}

```

We can see that the only external dependency we are relying on here is tokio for the async runtime. All other dependencies are crates that we have built. This makes sense as we are testing the entire workflow. We need to ensure that all our crates work together in our system.

With our workspace defined and ready, we can now build our delete workflow test.

Creating a delete workflow test

For our delete workflow, we have the following outline:

```

// nanoservices/to_do/networking/http-workflow-tests/
// src/items/delete.rs
#[cfg(test)]
mod tests {
    . . .
    async fn setup() {
        . . .
    }
    #[tokio::test]
    async fn test_delete_item_ok() {
        . . .
    }
}

```

We will start with the imports. We essentially require a little bit of everything in our system to get this delete workflow running using the following:

```

// nanoservices/to_do/networking/http-workflow-tests/
// src/items/delete.rs
use to_do_kernel::api::basic_actions::{create::create_item,
    delete::delete_item};

```

```

use auth_kernel::api::{users::create::create_user, auth::login::login_user};
use dal::{
    migrations::run_migrations as run_todo_migrations,
    connections::sqlx_postgres::SQLX_POSTGRES_POOL as todo_pool,
};
use auth_dal::{
    migrations::run_migrations as run_auth_migrations,
    connections::sqlx_postgres::SQLX_POSTGRES_POOL as auth_pool,
};
use glue::{token::HeaderToken, sql_commands::WIPE_DB};
use dal::to_do_items::enums::TaskStatus;
use auth_core::api::users::create::CreateUser;

```

This is a lot of imports, so we will focus on what these imports are used for as and when we use them. The first function we build is the setup function, with the following code:

```

// nanoservices/to_do/networking/http-workflow-tests/
// src/items/delete.rs
async fn setup() {
    sqlx::query(WIPE_DB).execute(&*todo_pool).await.unwrap();
    sqlx::query(WIPE_DB).execute(&*auth_pool).await.unwrap();
    run_todo_migrations().await;
    run_auth_migrations().await;
}

```

Here, we can see that we wipe both the auth and to-do databases and then run the migrations for both. For our local system, we are only using one database. However, if we want the freedom to test against systems where the auth and to-do server have their own database, then this setup allows for this. All we need to do is have another database in docker-compose and change the environment variable for one of the servers to point to that database and we have a separate database for each server.

Inside our `test_delete_function` function, we run the setup function, and create the user with the following code:

```

// nanoservices/to_do/networking/http-workflow-tests/
// src/items/delete.rs
setup().await;
let user = CreateUser {
    email: "test@gmail.com".to_string(),
    password: "test".to_string(),
};
create_user(user).await.unwrap();

```

We then log the user in, and construct `HeaderToken` structs from the result of the login with the following code:

```
// nanoservices/to_do/networking/http-workflow-tests/  
// src/items/delete.rs  
let token = login_user(  
    "test@gmail.com".to_string(),  
    "test".to_string()  
).await.unwrap();  
  
let header_token = HeaderToken::decode(&token).unwrap();  
let header_token_ref = HeaderToken::decode(&token).unwrap();
```

We now have enough tokens to perform a create and delete request. We create an item, asserting that the item has been created via the HTTP response with the following code:

```
// nanoservices/to_do/networking/http-workflow-tests/  
// src/items/delete.rs  
let items = match create_item(  
    header_token_ref,  
    "code".to_string(),  
    TaskStatus::PENDING  
) {  
    Ok(items) => items,  
    Err(e) => {  
        println!("error: {:?}", e);  
        panic!("error creating item");  
    }  
};  
assert_eq!(items.done.len(), 0);  
assert_eq!(items.pending.len(), 1);
```

We finally delete that item, asserting that there are no items left associated with the user, with the following code:

```
// nanoservices/to_do/networking/http-workflow-tests/  
// src/items/delete.rs  
let items = delete_item(  
    header_token,  
    "code".to_string()  
) {  
    Ok(items) => items,  
    Err(e) => {  
        println!("error: {:?}", e);  
        panic!("error deleting item");  
    }  
};  
assert_eq!(items.done.len(), 0);  
assert_eq!(items.pending.len(), 0);
```

And here we have it, our workflow test is complete. If you really wanted to test how the workflow held up, you could log another user in at the same time and perform operations on the items of both users to ensure that the items associated with one user were not affected by operations performed on items belonging to another user.

Another benefit that we can see of our workflow testing is that it is high-level. If our functions are sensibly named, someone who is not a Rust developer could look over the workflow to see the steps and understand what is being tested. This helps a lot with bugging. If someone complains that they cannot perform a certain action on your system, and there is a workflow that is heavily tested, then it is unlikely to be an error with that workflow, and you can start looking in other places.

Our test workflow is now ready to be run. We can now move on to updating our testing script so we can run the workflow test.

Updating our test runner bash script

Our system has grown into a fully functioning microservices system with a frontend, custom Redis cache, remote logging system, unit tests, atomic end-to-end tests, and now end-to-end tests that inspect entire workflows. Before we try to run all our tests, we should just do a couple of simple sanity tests.

The root `Cargo.toml` of our system should have the following workspaces:

```
[workspace]
resolver = "2"
members = [
    "glue",
    "ingress",
    "ingress/frontend/rust-interface",
    "nanoservices/auth/core",
    "nanoservices/auth/dal",
    "nanoservices/auth/kernel",
    "nanoservices/auth/networking/actix_server",
    "nanoservices/to_do/core",
    "nanoservices/to_do/dal", "nanoservices/to_do/kernel",
    "nanoservices/to_do/networking/actix_server",
    "nanoservices/to_do/networking/atomic-http-tests",
    "nanoservices/to_do/networking/http-workflow-tests",
    "nanoservices/user-session-cache/cache-client",
    "nanoservices/user-session-cache/cache-module"
]
```

Considering all these workspaces, we must also ensure that we have all the environment variables we need when running the entire system. The `.env` file in the root of our system should have the following:

```
TO_DO_DB_URL=postgres://username:password@localhost/to_do
AUTH_DB_URL=postgres://username:password@localhost/to_do
```

```
AUTH_API_URL=http://127.0.0.1:8081
TO_DO_API_URL=http://127.0.0.1:8080
JWT_SECRET=secret
CACHE_API_URL=redis://127.0.0.1:6379
ELASTICSEARCH_URL=http://localhost:9200/logs/_doc
```

For our testing bash script, we are now going to have multiple servers running at the same time. If we are not tracking the logs of these servers, it can be a nightmare working out which server failed. To handle the running of the servers properly, we need to rewrite our testing script after the data access layer unit tests. First, we run the two servers pointing to their own logs with the following code:

```
# scripts/run_tests.sh
cargo build -p actix_server
cargo run -p actix_server > ./logs/todo_server.log &
to_do_server_pid=$!

cargo build -p auth_actix_server
cargo run -p auth_actix_server > ./logs/auth_server.log &
auth_server_pid=$!
sleep 1
```

We then run our atomic end-to-end tests, and then our end-to-end workflow tests, with the following code:

```
# scripts/run_tests.sh
cargo test -p to-do-atomic-http-tests -- \
--test-threads=1 --nocapture \
> ./logs/to-do-atomic-http-tests.log

cargo test -p to-do-http-workflow-tests -- \
--test-threads=1 --nocapture \
> ./logs/to-do-http-workflow-tests.log
```

Finally, we tear down our databases and kill the servers with the following code:

```
# scripts/run_tests.sh
docker-compose down
kill $to_do_server_pid
kill $auth_server_pid
```

If we run our test script now, we will be able to see that all our tests have passed and that the logs to the servers are what we expect. With this, our testing suite is fully operational!

Summary

In this chapter, we built tests that interact with our entire system. We can now build and run tests throughout the entire stack, from individual unit tests on individual servers in a microservice structure. This end-to-end testing can be employed in GitHub Actions or other continuous integration systems to prevent breaking code from being pushed into the main branch. It is also a good idea to run these end-to-end tests before performing a release to a live server.

We have now gotten a feel for how our system works when both servers are running with the cache, logging database, and main database. In the next chapter, we will cover how to package and deploy our system onto servers on AWS.

Questions

1. What bug can end-to-end testing catch that unit testing cannot (hint: compilers can also catch the error if there is just one server)?
2. What bug can an end-to-end workflow test catch that an atomic end-to-end test cannot?
3. How do we run and then clean up a server when performing an end-to-end test?
4. How do we ensure that our system can be end-to-end tested as either a monolith or a microservices cluster?
5. In this chapter, we ran our end-to-end tests on one database. How could we break it out so each server had its own database?

Answers

1. Circular dependencies. End-to-end tests can detect a circular dependency between multiple servers. However, a unit test would miss this as the unit test mocks the inputs to the unit of code.
2. Breaks in the workflow. Atomic end-to-end tests set up the state of the app for the individual endpoint. However, there may be a missing API endpoint, making it impossible for the user to complete the workflow. An end-to-end workflow test will catch this.
3. We build the server first to avoid having to guess the build time. Once the build is finished, we run the server in the background by using the `&` symbol. We then keep note of the process ID of the running server and kill that process at the end of the testing script.
4. We perform a wipe on all databases in the setup function and then perform all the migrations of the services in the system. If the system is monolithic, then all database actions will run on the same database, including the wipes. If the system is a microservice cluster, then each server will commit a wipe action and migration on its own database.
5. We create an extra database in docker-compose with a different port. We then allocate the URL of that database with that port to an environment variable of the database for one of the servers. When we run the end-to-end tests, our servers will have their own databases.

Get this book's PDF copy, code bundle, and more

Scan the QR code (or go to packtpub.com/unlock). Search for this book by name, confirm the edition, and then follow the steps on the page.



UNLOCK NOW

Note: Have your invoice handy. Purchases made directly from the Packt website don't require an invoice.

16

Deploying Our Application on AWS

While we have had a lot of fun running our system locally, nobody else can access it. In this chapter, we cover how to deploy our application onto a server on AWS. We will automate the building of the server and provisioning of the database by using Terraform. My love for Rust is part of the reason why I keep updating the editions of this book. Another motivator is that while there are plenty of online tutorials, a book that brings you through the entire journey of building the application to deploying it gives you the ability to see the entire process. I also believe that taking time to learn how to provision infrastructure exactly how you want gives you a starting point for further learning and adds to your toolkit of problem-solving. For instance, when I was working on a financial loss modeling engine that was used by Nasdaq, we had to run models that had hundreds of terabytes of data to run large financial models. I built a system that used Terraform to spin up a large server that runs the model. As soon as the model was finished, the script evoked Terraform again to destroy the server. Being able to provision cloud infrastructure is another tool that can factor into your solutions when solving problems. This chapter will cover the following:

- Setting up our build environment with AWS
- Building a server build pipeline
- Packaging our system with Docker
- Distributing our system with Docker
- Deploying our system with Docker

By the end of this chapter, you will be able to configure NGINX to route traffic to different servers and serve frontend web apps. You will also be able to package your servers in distroless Docker images, making these images tiny and secure, as there is no terminal in a distroless image. You will also be able to automate the building and configuration of infrastructure such as servers and databases.

Technical requirements

You can download the example project and code for this book by following the instructions in the **Download the example code files** section in the *Preface* of this book.

This chapter's code files are included in the downloadable code bundle.

Setting up our build environment

While we can run the server locally, our aim is to deploy our application on a server. To run our application on a server, we are going to have to build our application in a Docker image. To ensure that our Docker image build runs smoothly, we are going to use an online computing unit on AWS. Before we run our build process, we need to carry out the following steps:

1. Setting up an AWS Secure Shell (SSH) key for Elastic Compute Cloud (EC2).
2. Setting up the AWS client for our local computer.
3. Writing a Terraform script that builds our build server.
4. Writing a Python script that manages the build.
5. Writing a Bash script that orchestrates the build on the server.

Once we have completed the steps, we will be able to run an automated pipeline that creates a build EC2 server on AWS and then builds our Rust application. First, let us get started by creating an SSH key for our server.

Setting up an AWS SSH key for an Amazon EC2 instance

If we want to run commands on a server, we are going to have to connect to the server using the SSH protocol over the **Hypertext Transfer Protocol (HTTP)**. However, we cannot have anyone accessing our server as it will not be secure. To stop anyone from connecting to our server and running any commands they want, we will only allow users to connect to our server if they have an SSH key. To create our key, we need to log in to the AWS console and navigate to our EC2 dashboard, which can be accessed via the search box, as seen in the following screenshot:

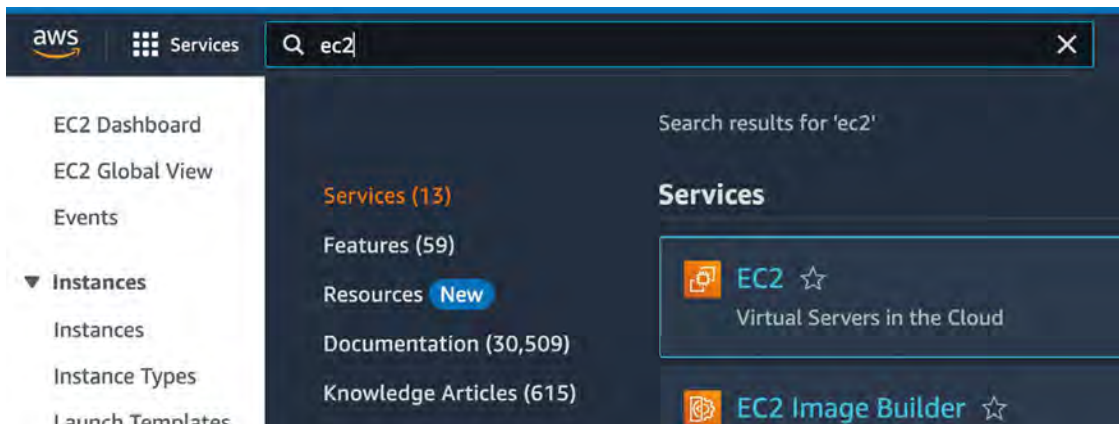


Figure 16.1 – Navigating to the EC2 dashboard with the search box

Once we have navigated to the EC2 dashboard, we can navigate to **Key Pairs** under the **Network & Security** section of the panel on the left side of the view, as seen in the following screenshot:

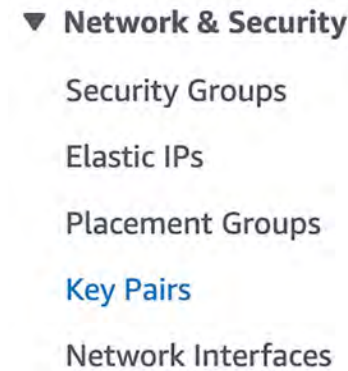


Figure 16.2 – Navigating to Key Pairs



UIs can change

It must be noted that AWS might not look like the screenshots in this chapter, as AWS keeps changing the UI. However, the fundamental concepts will be the same.

Once we have navigated to the **Key Pairs** section, there will be a list of key pairs that you already own. If you have built EC2 instances before, you might already see some listed. If you have never built an EC2 instance before, then you will not have anything listed. At the top right of the screen, you can create a key by clicking on the **Create key pair** button shown in the following screenshot:

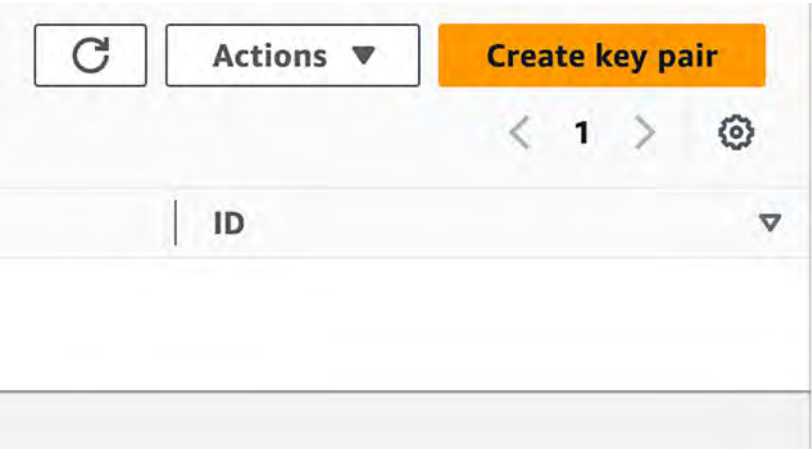


Figure 16.3 – Button to allow the creation of a key pair

Once we have clicked on this button, we will see the following form:

The screenshot shows the 'Create key pair' form in the AWS Management Console. The breadcrumb navigation at the top reads 'EC2 > Key pairs > Create key pair'. The main heading is 'Create key pair' with an 'Info' link. Below this, a section titled 'Key pair' explains that a key pair consists of a private key and a public key used for identity verification. The form includes a 'Name' field with the value 'remotebuild' and a note that names can be up to 255 ASCII characters. The 'Key pair type' section has two options: 'RSA' (selected) and 'ED25519'. The 'Private key file format' section has two options: '.pem' (selected, for use with OpenSSH) and '.ppk' (for use with PuTTY). A 'Tags - optional' section indicates no tags are currently associated, with an 'Add new tag' button and a note that up to 50 more tags can be added. At the bottom right, there are 'Cancel' and 'Create key pair' buttons.

Figure 16.4 – The Create key pair form

In the preceding screenshot, we can see that we have named our key `remotebuild`; we also state that our key has the `.pem` format and is of the `RSA` type. Once we click on the **Create key pair** button, we will download the key. We now must store our key. This can be done by storing it in our home directory in the `.ssh` directory. Inside the `.ssh` directory, we can create a `keys` directory with the following command:

```
mkdir "$HOME"/.ssh/keys/
```

The "\$HOME" environment variable is always available in the Bash shell, and it denotes the home directory for the user. This means that other users who log in to the computer under a different username cannot access the SSH keys we downloaded if we store them in the directory that we have just created. We can now navigate to where our key has been downloaded and copy it to our keys directory with the following command:

```
cp ./remotebuild.pem "$HOME"/.ssh/keys/remotebuild.pem
```

We then must change the permissions for the key so that only the owner of the file can read the file with the 600 code for us to use the key for SSH purposes with the following command:

```
chmod 600 "$HOME"/.ssh/keys/remotebuild.pem
```

We now have an SSH key stored in our .ssh/keys directory, where we can access this key with the correct permissions in order to access the servers that we create. Now that we have this, we need programmatic access to our AWS services by setting up our AWS client.

Setting up our AWS account

We will be using Terraform to automate our server build. To do this, Terraform will make calls to the AWS infrastructure. This means that we must have the AWS client installed on our local machine. Before we configure our client, we need to get some programmatic user keys. If we do not obtain programmatic access, our code will not be authorized to build infrastructure on our AWS account. The first step of obtaining the user keys is navigating to the IAM section via the search box, as seen in the following screenshot:

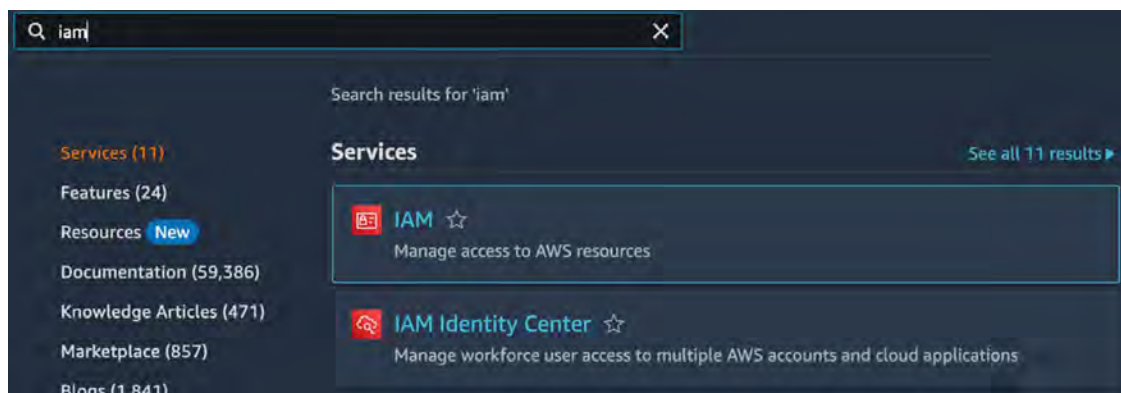


Figure 16.5 – Navigating to the IAM section

Once we have navigated to the IAM section, we will get the following layout:

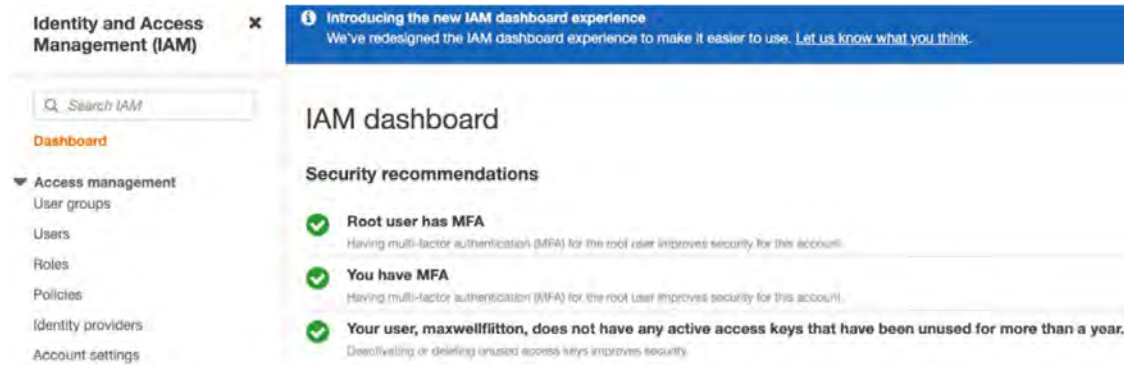


Figure 16.6 – View of the IAM section

We can see in the preceding screenshot that my user has **multi-factor authentication (MFA)**, and I keep my access key rotated. If you are new to AWS, this checklist might not be satisfactory, and it is suggested that you follow the security recommendations that AWS gives you. We now must access the **Users** option on the left-hand side of the view and create a new user by clicking the **Create user** button at the top right of the screen, as seen in the following screenshot:



Figure 16.7 – Creating users

We can then create a user. It does not matter what name we call the user, as seen in the following figure:

The screenshot shows the AWS IAM console interface for creating a user. The breadcrumb navigation at the top reads 'IAM > Users > Create user'. On the left, a sidebar lists three steps: 'Step 1 Specify user details' (which is the active step), 'Step 2 Set permissions', and 'Step 3 Review and create'. The main content area is titled 'Specify user details' and contains a 'User details' section. In this section, the 'User name' field is filled with 'maxwellfliton'. Below the field, a note states: 'The user name can have up to 64 characters. Valid characters: A-Z, a-z, 0-9, and + = , . @ _ - (hyphen)'. There is an unchecked checkbox labeled 'Provide user access to the AWS Management Console - optional'. Below this checkbox, a note says: 'If you're providing console access to a person, it's a best practice to manage their access in IAM Identity Center.' with a link to 'Learn more'. At the bottom right of the form, there are 'Cancel' and 'Next' buttons.

IAM > Users > Create user

Step 1
Specify user details

Step 2
Set permissions

Step 3
Review and create

Specify user details

User details

User name

maxwellfliton

The user name can have up to 64 characters. Valid characters: A-Z, a-z, 0-9, and + = , . @ _ - (hyphen)

☐ Provide user access to the AWS Management Console - optional

If you're providing console access to a person, it's a best practice to manage their access in IAM Identity Center.

i If you are creating programmatic access through access keys or service-specific credentials for AWS CodeCommit or Amazon Keyspaces, you can generate them after you create this IAM user.
[Learn more](#)

Cancel Next

Figure 16.8 – First stage of creating a user

We can see that we will have to create the programmatic access later. To create our user, we now move on to setting permissions for the user by clicking the **Next** button. Our permissions configuration will look like the following:

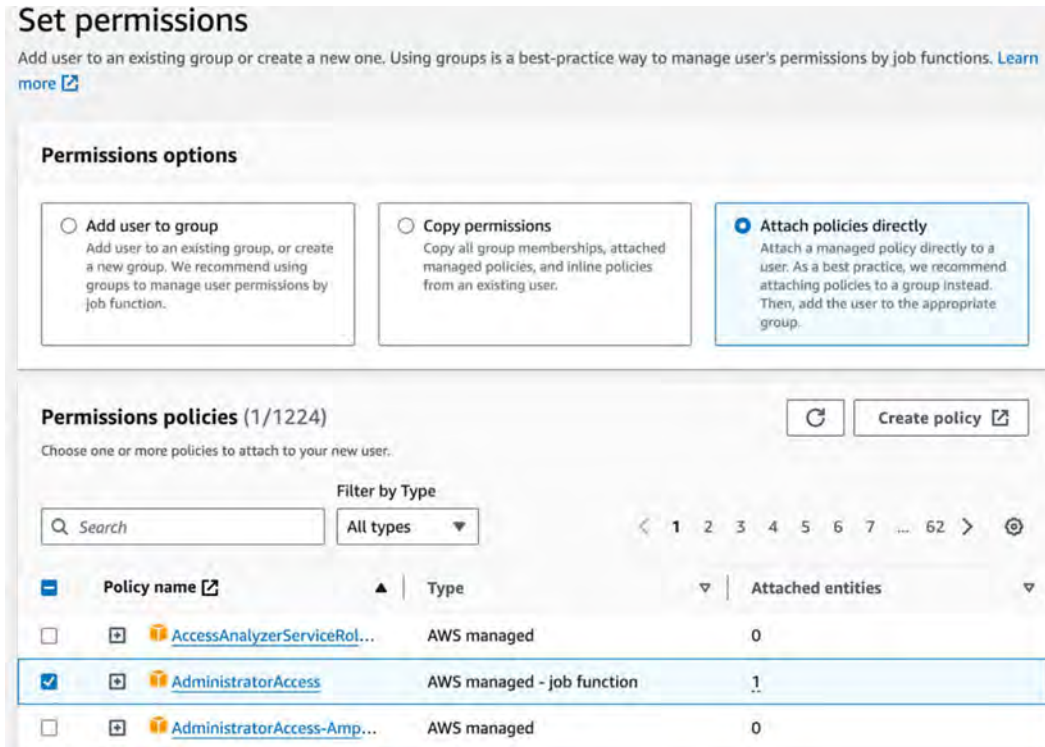


Figure 16.9 – Defining user permissions

We can see that we have given the user **AdministratorAccess** permissions, which will enable us to create and destroy servers and databases. We can not create the user, which will appear on the dashboard once the user has been created. We can now create an access key for our user by clicking **Create access key** on the user profile, as seen in the following screenshot:

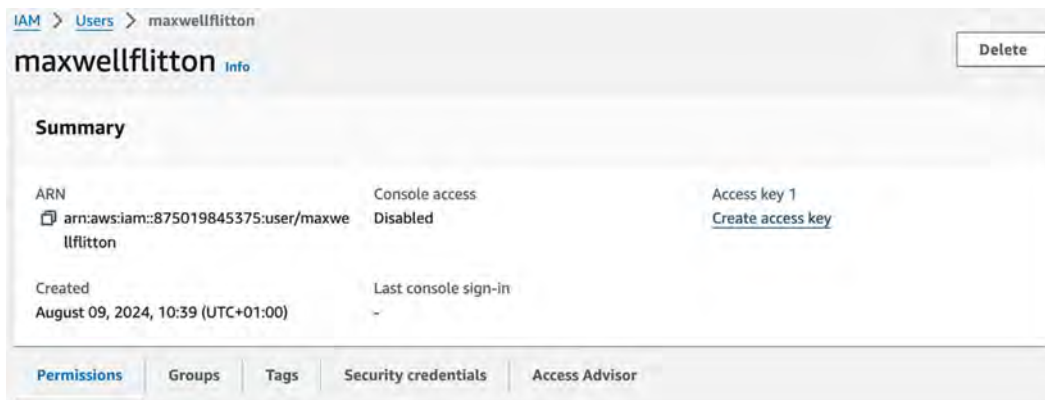


Figure 16.10 – Defining user permissions

We can then select the **Command Line Interface (CLI)** key, as seen in the following screenshot:

The screenshot shows the AWS IAM 'Create access key' wizard for user 'maxwellflitton'. The left sidebar shows the progress: Step 1 (Access key best practices & alternatives), Step 2 - optional (Set description tag), and Step 3 (Retrieve access keys). The main content area is titled 'Access key best practices & alternatives' with an 'info' icon. It includes a warning about long-term credentials and a 'Use case' section with five radio button options: 'Command Line Interface (CLI)' (selected), 'Local code', 'Application running on an AWS compute service', 'Third-party service', and 'Application running outside AWS'. Below these is a yellow box titled 'Alternatives recommended' with two bullet points: 'Use AWS CloudShell, a browser-based CLI, to run commands. [Learn more](#)' and 'Use the AWS CLI V2 and enable authentication through a user in IAM Identity Center. [Learn more](#)'. At the bottom, the 'Confirmation' section has a checked checkbox: 'I understand the above recommendation and want to proceed to create an access key.' The 'Next' button is highlighted in orange.

Figure 16.11 – Selecting the type of access key

We can see in the screenshot that AWS recommends CloudShell for our deployment approach, which, while technically secure, does have vulnerabilities. For instance, we are creating a long-lived access key. If that key were leaked to a hacker, they could perform actions on our system until we detected the leak and acted on it.



DevOps is an entire field in itself, and security professionals spend their entire career ensuring that systems are safe. While this book will cover enough to launch a safe web application, it does not claim to be the best practices. Please seek further books and courses on security and DevOps if you want to progress your role in the deployment of applications.

Once the access key is created, you will be exposed to an access key and a secret access key. It is important to note these down in a secure location, such as a password manager, because you will never be able to see your secret access key again on the AWS site, and we will need it when configuring our AWS client on our local computer.

Now that we have the user keys handy, we can configure our AWS client with the following command:

```
aws configure
```

The AWS client will then prompt you to enter the user keys as and when they are required. Once this is done, your AWS client is configured, and we can use AWS features programmatically on our local computer. Now, we are ready to start creating servers using Terraform in the next section.

Setting up our Terraform build

When it comes to building infrastructure on AWS, we can simply point and click in the EC2 dashboard. However, this is not desirable. If you are anything like me, when I point to and click a series of configuration settings, I forget what I have done unless I document it, and let's be honest, documenting what you've clicked is not something you are going to look forward to. Even if you are a better person than me and you do document it, when you change the configuration, there is a chance that you will not go back to update the documentation of that change. Pointing and clicking is also time-consuming. If we wanted to create some infrastructure and then destroy it a week later, and then recreate it a month after that, we would be reluctant to touch it if we had to point and click, and our server bills would be higher. This is where **infrastructure as code (IaC)** comes in. We will have to do some pointing and clicking, as we did in the previous sections. We cannot do any programmatic access without pointing and clicking to set up programmatic access.

For our IaC, we are going to have a separate build directory in the root of our system. In the build directory, we can define our infrastructure for the build server in the `build/main.tf` file. It must be noted that the `.tf` extension is the standard extension for Terraform files. First, we define which version of Terraform is being used with the following code:

```
# builds/main.tf
terraform {
  required_version = ">= 1.9.4"
  required_providers {
    aws = {
      source  = "hashicorp/aws"
      version = ">= 5.62.0"
    }
  }
}
```

```
}  
}
```

Now that we have defined the Terraform version, we can declare that we are using the AWS module. The Terraform registry has modules for a range of platforms, including Google Cloud and Microsoft Azure. Anyone can build modules and abstract infrastructure to be downloaded in the Terraform registry. Our AWS module usage declaration takes the following form:

```
# builds/main.tf  
provider "aws" {  
    region = "eu-west-2"  
}
```

You might want to pick a different region if another region suits you. A list of the available regions on AWS for EC2 can be found via the following link: <https://docs.aws.amazon.com/AWSEC2/latest/UserGuide/using-regions-availability-zones.html>.

I'm merely using "eu-west-2" for demonstration purposes. We can now build our EC2 instance with the following code:

```
# builds/main.tf  
resource "aws_instance" "build_server" {  
    ami = "ami-05ea2888c91c97ca7"  
    instance_type = "t2.medium"  
    key_name = "remotebuild"  
    user_data = file("server_build.sh")  
    tags = {  
        Name = "to-do build server"  
    }  
}
```

The resource statement in the Terraform script declares that we are defining a resource to be built, and `aws_instance` states that we are using the EC2 instance template in the AWS module. A list of the available AWS Terraform modules and their documentation can be found via the following link: <https://registry.terraform.io/providers/hashicorp/aws/latest/docs>.

We are calling it `build_server`. We can refer to `build_server` anywhere else in the Terraform script, and Terraform will work out the order in which resources need to be built to make sure all references are accounted for. We can see that we have referenced the `remotebuild` key that we defined in the previous section. We can create multiple EC2 instances that can be accessed by one key if we want. We also declare the name so that when we look at our EC2 instances, we know what the server is for. We must also note that `user_data` is the Bash script that will be run on the new EC2 server once it has been built. The `ami` section is a reference to the type of operating system and version being used. Do not directly copy my **Amazon Machine Image (AMI)** ID in the example unless you are using the same region, as AMI IDs can vary depending on the region. If you want to find out the AMI ID, go to your EC2 dashboard and click on **Launch an instance**, which will result in the following window:

[EC2](#) > [Instances](#) > Launch an instance

Launch an instance [Info](#)

Amazon EC2 allows you to create virtual machines, or instances, that run on the AWS Cloud. Quickly get started by following the simple steps below.

Name and tags [Info](#)

Name

 [Add additional tags](#)

▼ Application and OS Images (Amazon Machine Image) [Info](#)

An AMI is a template that contains the software configuration (operating system, application server, and applications) required to launch your instance. Search or Browse for AMIs if you don't see what you are looking for below

Quick Start









[Browse more AMIs](#)
 including AMIs from
 AWS, Marketplace and
 the Community

Amazon Machine Image (AMI)

Amazon Linux 2023 AMI Free tier eligible ▼

ami-05ea2888c91c97ca7 (64-bit (x86), uefi-preferred) / ami-02b0ba219a3ee9d7f (64-bit (Arm), uefi)

Virtualization: hvm ENA enabled: true Root device type: ebs

Figure 16.12 – Launching an instance

Here, we can see that we are picking **Amazon Linux**. You must select this; otherwise, your build scripts will not work. If we zoom in, we can see that the AMI ID is visible, as seen here:



Amazon Machine Image (AMI)

Amazon Linux 2023 AMI

ami-05ea2888c91c97ca7 (64-bit (x86), uefi-preferred) / ami-02b0ba219a3ee9d7f (64-bit (Arm), uefi)
Virtualization: hvm ENA enabled: true Root device type: ebs

Figure 16.13 – Accessing AMI ID for the server

This will be the AMI ID for the Amazon Linux operating system in the region that you want to launch. You can also see that nothing is stopping you from using other operating systems in other Terraform projects. If you have built EC2 instances in the past, you will know that the IP address is random unless we attach an Elastic IP to the EC2 instance. We will have to produce an output of the IP address of our EC2 instance so that we can connect to it. Our output is defined with the following code:

```
# builds/main.tf
output "ec2_global_ips" {
  value = ["${aws_instance.build_server.*.public_ip}"]
}
```

Here, we can see that we reference our build server using `aws_instance.build_server`. At this point, our Terraform build is nearly done. We must remember that we need to build the `build_server.sh` script that is going to be run on the EC2 instance once the EC2 instance has been built. In our `/build/server_build.sh` file, we can install the basic requirements needed for our server with the following code:

```
#!/bin/bash
# builds/server_build.tf
sudo yum update -y
sudo yum install git -y
sudo yum install cmake -y
sudo yum install tree -y
sudo yum install vim -y
sudo yum install tmux -y
sudo yum install nodejs -y
sudo yum install -y openssl-devel
sudo yum install make automake gcc gcc-c++ kernel-devel -y
```

With the preceding packages, we will be able to navigate around the server looking at file trees with `tree`; we will also be able to perform `git` operations, open files, and edit them using `vim`, and have multiple panels open through one terminal if we need to with `tmux`. The other packages enable us to compile our Rust code. We must also note that we have appended each install with a `-y` tag. This is to tell the computer to bypass input prompts and put in default answers. This means that we can run this script in the background without any problems. We now must install the PostgreSQL drivers with the following code in the same `server_build` script:

```
# builds/server_build.tf
sudo amazon-linux-extras install postgresql10 vim epel -y
sudo yum install -y postgresql-server postgresql-devel -y
```

We nearly have everything that we need. In the next section, we will be using Docker to build and package our applications. This can be done with the following code in the same `server_build` script:

```
# builds/server_build.tf
sudo amazon-linux-extras install docker
sudo service docker start
sudo usermod -a -G docker ec2-user
```

Here, we can see that we install Docker, start the Docker service, and then register our user with the Docker service so that we do not have to use `sudo` with every Docker command. As we have installed Docker, we might as well install `docker-compose` for completeness. This can be done at the end of our script with the following code in the same `server_build` script:

```
# builds/server_build.tf
sudo curl -L "https://github.com/docker/compose/releases \
/download/1.29.2/docker-compose-$(uname -s)-$(uname -m)"
-o /usr/local/bin/docker-compose \
sudo chmod +x /usr/local/bin/docker-compose
```

The `curl` command downloads `docker-compose`. We then make the `docker-compose` executable with the `chmod` command. The build script is almost finished. However, running all the commands that we have defined in the build script will take a while. There is a chance that we can SSH into our server before the build script has finished. Therefore, we should write a `FINISHED` string to a file to inform other processes that the server has all the packages that have been installed. We can write our flag with the following code:

```
# builds/server_build.tf
echo "FINISHED" > home/ec2-user/output.txt
```

We have now built out the infrastructure that is going to build our applications for deployment. In the next section, we are going to build a script to orchestrate the building of our application on the server.

Building a server build pipeline

Now that we can build our server programmatically, we can include it in our build pipeline. Our build pipeline can be defined by a simple Bash script that is housed next to our Terraform file. Because the Bash script is in the same directory as the Terraform file, we start off by ensuring that the working directory is in the same directory with the following code:

```
#!/usr/bin/env bash
# builds/run_build.sh

SCRIPTPATH="$( cd "$(dirname "$0")" ; pwd -P )"
cd $SCRIPTPATH
```

We then build the server using Terraform with the following code in the same `run_build` script:

```
# builds/run_build.sh
terraform init
terraform apply -auto-approve
terraform output -json > ./output.json

IP_ADDRESS=$(jq --raw-output '.ec2_global_ips.value[0][0]' \
output.json)
echo $IP_ADDRESS
echo "waiting for server to be built"
sleep 5
echo "attempting to enter server"
```

The two Terraform commands that we carry out in the preceding snippet are `init` and `apply`. `init` sets up the Terraform state to record what is going on and downloads the modules that we need, which, in this case, are from AWS. The `apply` command runs the Terraform build, which will build our EC2 server. Once our EC2 instance is built, we can get the output of Terraform and write it to a JSON file, and then load the data from that JSON file, assigning the IP address of the created server to the `IP_ADDRESS` variable. We can also see that we sleep our script for 5 seconds to ensure that the server has started the creation process, and the IP can be routed to.

We are now ready to upload our files to the server that has recently been built. We must ensure that we do not upload `node_modules` by deleting them with the following code:

```
# builds/run_build.sh
rm -rf ../ingress/frontend/node_modules
```

We then upload our nanoservices with the following code:

```
# builds/run_build.sh
scp -i ~/.ssh/keys/remotebuild.pem \
-o StrictHostKeyChecking=no \
```

```
-r ../nanoservices \
ec2-user@$IP_ADDRESS:/home/ec2-user/nanoservices
```

We can see that we point to the SSH key to access the server. We can also see that we set `StrictHostKeyChecking=no`. This ensures that we are not prompted to ask whether we are okay connecting to this host because it will not be known to our computer. We then upload the `ingress`, `glue`, `docker-compose.yml`, and `.env` files with the following code:

```
# builds/run_build.sh
scp -i ~/.ssh/keys/remotebuild.pem \
-o StrictHostKeyChecking=no \
-r ../ingress \
ec2-user@$IP_ADDRESS:/home/ec2-user/ingress

scp -i ~/.ssh/keys/remotebuild.pem \
-o StrictHostKeyChecking=no \
-r ../glue \
ec2-user@$IP_ADDRESS:/home/ec2-user/glue

scp -i ~/.ssh/keys/remotebuild.pem \
-o StrictHostKeyChecking=no \
../docker-compose.yml \
ec2-user@$IP_ADDRESS:/home/ec2-user/

scp -i ~/.ssh/keys/remotebuild.pem \
-o StrictHostKeyChecking=no \
../.env \
ec2-user@$IP_ADDRESS:/home/ec2-user/
```

Finally, we SSH into our created server, and perform a loop until the `output.txt` file is present, telling us that the build script has run and everything we need is installed onto the server with the following code:

```
# builds/run_build.sh
ssh -i ~/.ssh/keys/remotebuild.pem \
-o StrictHostKeyChecking=no -t \
ec2-user@$IP_ADDRESS << EOF
    until [ -f ./output.txt ]
    do
        echo "waiting for file"
        sleep 3
    done
    echo "File found"
    sudo curl https://sh.rustup.rs -sSf | bash -s -- -y
```

```
source ~/.cargo/env
export $(cat .env | xargs)
cd ingress
cd frontend
npm install
npm run build
cd ..
cargo build --release
EOF
terraform destroy -auto-approve
```

We can also see that we built the frontend and backend Rust server. We can then run this Bash script to build the server, copy the files over to the server, and build our application.



Warning!

Terraform can sometimes be temperamental; if it fails, you may need to run it again. Sometimes I've had to perform a Terraform run up to three times before it fully works. Every action is stored by Terraform, so do not worry; running the Bash script again will not result in duplicate servers.

It usually makes sense to show the printouts in this book; however, the printout here is long and would probably take up multiple pages. Also, the printouts are obvious. Terraform openly states what is being built, the copying and building are also verbose, and the destruction of the server with Terraform is also printed out. It will be very clear what is going on when you run this build pipeline.

You might also notice that there is a range of Terraform files that have been created. These files keep track of the state of our resources that have been built on the AWS platform. If you delete these state files, you have no way of knowing what is built, and duplicates will be spun up. It will also prevent Terraform from cleaning up. The place where I used to work used Terraform to build massive data models for calculating the risk of financial loss over geographical locations. The data being processed can go over the range of terabytes per chunk. We use Terraform to spin up a range of powerful computers, run data through it (which can take days), and then shut it down when it is finished. Multiple people need to monitor this process, so our Terraform state is housed in a **Simple Storage Service (S3)** bucket so that anyone on the project can shut it down if needed. An example of this configuration takes the following form in the `state.tf` file:

```
terraform {
  backend "s3" {
    bucket = "some_bucket"
    key = "some/ptaht/terraform.tfstate"
    region = "eu-west-1"
  }
}
```

It must be noted that your account needs to have access to the bucket defined. Now that our build pipeline is fully working, we can move on to wrapping our software in Docker so that we can package the software in Docker and deploy it.

Packaging our system with Docker

Throughout the book, we have been running our servers directly. However, packaging our servers with Docker enables us to distribute our servers to Docker orchestration systems such as docker-compose, or Kubernetes. We can also version control our server releases and isolate dependencies. For instance, if a server requires a particular library that is incompatible with another server, Docker enables us to run both servers on the same machine as the dependencies are isolated in the image and running container. For our system, we want to have the option of deploying our system as either a single monolith or a cluster of servers talking to each other, known as microservices. To achieve this flexibility, we must carry out the following steps:

1. Building the base Docker image for all servers
2. Defining the build config for our monolith
3. Defining the build config for our microservices

Once we have carried out these steps, we will have a system that can be deployed onto a server as either a monolith or collection of microservices.

Building the base Docker image for all servers

For our system, we are going to build one Docker image for all our servers. We also want our images to be tiny, so we will be doing a two-stage build, resulting in a distroless build. This means the bare minimum is kept running the Rust binary. This approach is also more secure. The distroless image is so barebones that if a hacker manages to gain access to the container running our server binary, there is no terminal, so the hacker cannot perform any terminal commands. Because we are stripping down our image to the bare minimum, we must offer two Docker builds. One for AArch chips (ARM-based CPUs such as Apple Silicon; often labeled arm64/aarch64), and another for x86 chips (Intel/AMD 64-bit; often labeled amd64), because we will be copying over specific libraries. These Docker files are housed in the root of our system with the following names:

- Dockerfile.aarch
- Dockerfile.x86

We will start with the aarch Dockerfile. We start off by stating that we are using the Rust base image, and then we install the requisites needed for encryption and interacting with a Postgres database with the following code:

```
# Dockerfile.aarch
FROM rust as build

RUN apt-get update && apt-get install -y libpq-dev
RUN apt-get install libssl-dev -y
```

We are also going to be building the frontend and embedding the built frontend into our ingress binary. Therefore, we must install nodejs and wasm-pack with the following code:

```
# Dockerfile.aarch
RUN curl -fsSL \
https://deb.nodesource.com/setup_18.x | bash -
RUN apt-get install -y nodejs

# Install wasm-pack
RUN cargo install wasm-pack
```

Now, with everything installed, we copy all our code into the working directory, which we call app, with the following code:

```
# Dockerfile.aarch
WORKDIR /app
COPY . .
```

We can now build our frontend and our Rust servers with the following code:

```
# Dockerfile.aarch
RUN cd ingress/frontend && \
npm install && \
npm run wasm && \
npm run build
RUN cargo build --release -p ingress
RUN cargo build --release -p actix_server \
--features=auth-http --no-default-features
RUN cargo build --release -p auth_actix_server
```

With our servers built, we want to reduce the size of our binaries as much as possible. We do this by stripping our binaries of all the debug symbols. This means that debugging our crashes will be harder. Do not worry, we have extensively tested our code, and we are shooting off logs to a database. We strip our binaries with the following code:

```
# Dockerfile.aarch
RUN strip /app/target/release/ingress
RUN strip /app/target/release/actix_server
RUN strip /app/target/release/auth_actix_server
```

We are now ready to define the second step of our two-step build by declaring the distroless image with the following code:

```
# Dockerfile.aarch
FROM gcr.io/distroless/cc-debian12
```

Now, we could just copy over our service binaries. However, our binaries would not work if we did that. This is because the server binaries rely on some dynamically linked libraries to function. We can see what dynamic libraries are needed for a binary by typing in the `ldd` command and then a path to a binary. It will list all the dynamic libraries that the binary is linked to. I manually did this to the ingress binary, and as a result, we must copy over the following libraries from the previous step with the following code:

```
# Dockerfile.aarch
COPY --chown=1001:1001 --from=build \
/lib/ld-linux-aarch64.so.1 \
/lib/ld-linux-aarch64.so.1
COPY --chown=1001:1001 --from=build \
/lib/aarch64-linux-gnu/libssl.so.3 \
/lib/aarch64-linux-gnu/libssl.so.3
COPY --chown=1001:1001 --from=build \
/lib/aarch64-linux-gnu/libcrypto.so.3 \
/lib/aarch64-linux-gnu/libcrypto.so.3
COPY --chown=1001:1001 --from=build \
/lib/aarch64-linux-gnu/libgcc_s.so.1 \
/lib/aarch64-linux-gnu/libgcc_s.so.1
COPY --chown=1001:1001 --from=build \
/lib/aarch64-linux-gnu/libm.so.6 \
/lib/aarch64-linux-gnu/libm.so.6
COPY --chown=1001:1001 --from=build \
/lib/aarch64-linux-gnu/libc.so.6 \
/lib/aarch64-linux-gnu/libc.so.6
```

These are all needed and supply the following functionality:

- `ld-linux-aarch64.so.1`: This is the dynamic linker/loader for **64-bit ARM architecture (AArch64)**. It's responsible for loading shared libraries needed by a program and preparing the program to run by linking the libraries into the process's memory space.
- `libssl.so.3`: Part of the OpenSSL toolkit, `libcrypto` provides general-purpose cryptographic routines such as encryption, decryption, hashing, and more.
- `libgcc_s.so.1`: Part of the **GNU Compiler Collection (GCC)** runtime libraries, this provides low-level support for exception handling and other essential functionalities for programs compiled with GCC.
- `libm.so.6`: This is the math library, providing basic mathematical functions such as `sin()`, `cos()`, `sqrt()`, and so on. It is a standard part of the C library.
- `libc.so.6`: This is the **GNU C Library (glibc)**, which provides the core functionalities of the C standard library, including system calls, memory management, and basic input/output operations.

We could statically compile our Rust servers. This is where all the dependencies are compiled into the binary. While this sounds good, we must remember that we already have three binaries in our image, and we could add more. If we duplicate these libraries and compile them into every library, the size of our image would grow a lot. Instead, we just have one `libc` library and have multiple binaries point to it. However, this approach is also not a magic bullet. When dynamically linking, you miss out on compiler optimizations of how that library interacts with the rest of your code.

If you really want to see size reduction in action, I have had plenty of fun compiling Rust with no standard library and pointing to `libc`. This means you must do all your memory allocation with unsafe code pointing to `libc` functions, but your binaries end up being tiny.

Now that we have copied over our libraries, we can copy over our Rust server binaries with the following code:

```
# Dockerfile.aarch
COPY --from=build \
/app/target/release/ingress \
/usr/local/bin/ingress
COPY --from=build \
/app/target/release/actix_server \
/usr/local/bin/actix_server
COPY --from=build \
/app/target/release/auth_actix_server \
/usr/local/bin/auth_actix_server
```

Finally, we expose all the ports that relate to the different servers, and then run our ingress server with the following code:

```
# Dockerfile.aarch
EXPOSE 8001
EXPOSE 8081
EXPOSE 8080

CMD ["ingress"]
```

Here is the really cool part. We can overwrite the `CMD` in `docker-compose`. This means that our image can either run as a single small server as part of a microservices system, or the entire system as a monolith, by just changing the command that we pass through to the container that is created from this image.

For our `Dockerfile.x86` file, everything is the same apart from the libraries are slightly different. We copy over the following libraries:

```
# Dockerfile.x86
COPY --chown=1001:1001 --from=build \
/lib64/ld-linux-x86-64.so.2 \
```

```

/lib64/ld-linux-x86-64.so.2
COPY --chown=1001:1001 --from=build \
/lib/x86_64-linux-gnu/libssl.so.3 \
/lib/x86_64-linux-gnu/libssl.so.3
COPY --chown=1001:1001 --from=build \
/lib/x86_64-linux-gnu/libcrypto.so.3 \
/lib/x86_64-linux-gnu/libcrypto.so.3
COPY --chown=1001:1001 --from=build \
/lib/x86_64-linux-gnu/libgcc_s.so.1 \
/lib/x86_64-linux-gnu/libgcc_s.so.1
COPY --chown=1001:1001 --from=build \
/lib/x86_64-linux-gnu/libm.so.6 \
/lib/x86_64-linux-gnu/libm.so.6
COPY --chown=1001:1001 --from=build \
/lib/x86_64-linux-gnu/libc.so.6 \
/lib/x86_64-linux-gnu/libc.so.6

```

Our images are ready to be built. However, there is one slight adjustment to our servers. We must ensure that all our servers are listening to the address `0.0.0.0:<PORT NUMBER>`. The zeros in the URL enable the server to accept incoming traffic from the Docker container that is wrapping the server. If the URL is not all zeros apart from the port number, then you will not be able to reach the server inside the Docker container.

Before we run any builds, we must add a `.dockerignore` file in the root of our system to prevent us from needlessly copying over excessive files such as `node_modules`. This `.dockerignore` file has the following content:

```

ingress/frontend/node_modules/
builds/
target/

```

Now, depending on the chip architecture of your computer, you build the Docker image for your servers by running one of the following commands in the root of your system:

```

# for aarch chips
docker build . -f Dockerfile.aarch -t compute-unit

# for x86 chips
docker build . -f Dockerfile.x86 -t compute-unit

```

Here, you will get a long printout of the build, and when the build is done, you will have a stored Docker image called `compute-unit`. This single image is going to be used for all our servers. We now have everything we need to build our monolith.

Defining the build config for our monolith

For our current system, all our databases are defined in the docker-compose file in the root of the system. We can stitch other docker-compose files together to build a working network. So, for instance, we can pass our main root docker-compose file into the docker-compose command and include another docker-compose file if this one works with the root docker-compose file. To see how this works in action, we will build the system defined in *Figure 16.14*:

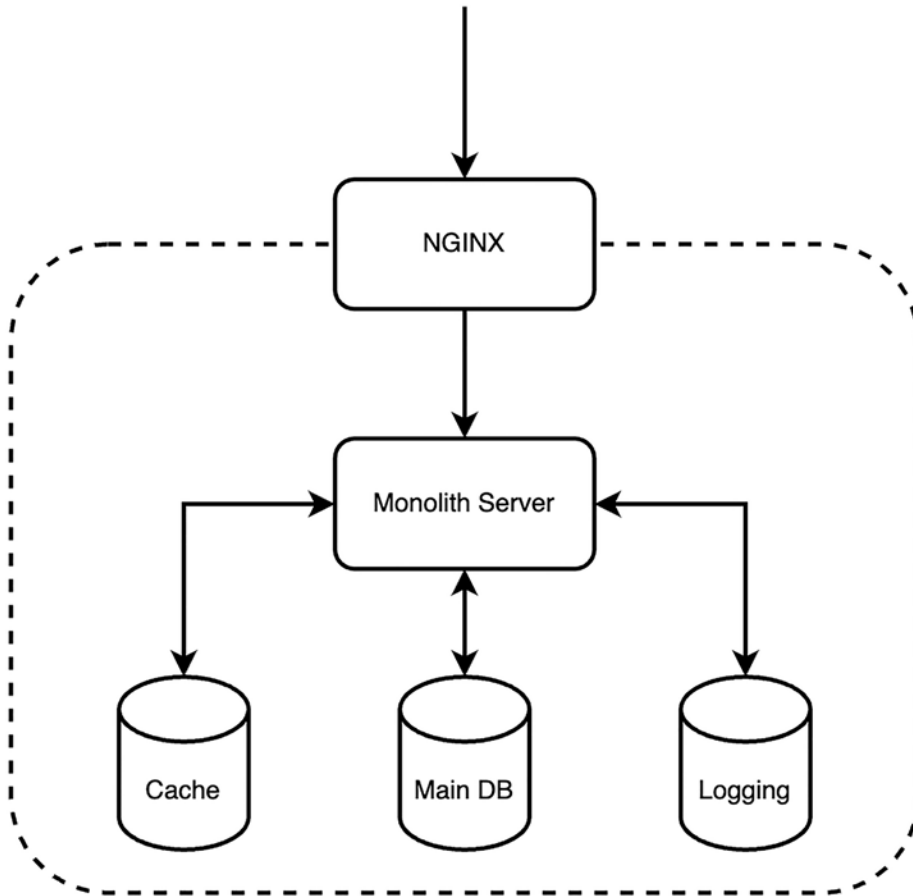


Figure 16.14 – Our monolith system overview

We can see in *Figure 16.14* that we are using NGINX as a router. NGINX is highly optimized for routing traffic, serving static assets, and load balancing. So, when a request comes into the network, it comes in via NGINX and then gets routed to the right server. We can also use NGINX to encrypt traffic between the client and our system. We will be doing this in the next chapter. With NGINX, we also get to decouple the routing logic from the servers that we build. We will see the power of this when we build our microservices system.

For our monolith build, we must add the following files:

- ingress/build-config.yml
- ingress/nginx.conf

For our build-config.yml file, we have the following:

```
# ingress/build-config.yml
version: "3.7"

services:

  ingress:
    image: compute-unit
    environment:
      - 'TO_DO_DB_URL=postgres://username:password@postgres/to_do'
      - 'AUTH_DB_URL=postgres://username:password@postgres/to_do'
      - 'AUTH_API_URL=http://127.0.0.1:8081'
      - 'TO_DO_API_URL=http://127.0.0.1:8080'
      - 'JWT_SECRET=secret'
      - 'CACHE_API_URL=redis://cache:6379'
      - 'ELASTICSEARCH_URL=http://elasticsearch:9200/logs/_doc'
    ports:
      - "8001:8001"

    depends_on:
      postgres:
        condition: service_started

  nginx:
    container_name: 'nginx'
    image: "nginx:latest"
    ports:
      - "80:80"
    links:
      - ingress
    volumes:
      - ./ingress/nginx.conf:/etc/nginx/nginx.conf
```

There are a couple of things that we must note here. For the URLs that we define, we can see that we have swapped from localhost to the name of the service definition in docker-compose. This is Docker Compose routing. If you make a network request inside the network, and the URL is the name of a service, Docker Compose will route the request to that service. We can also see that our ingress server depends on the Postgres database.

This is because we run our migrations when we start the ingress server. Therefore, our ingress server will only start once the Postgres database has started. Finally, we have our NGINX service. The NGINX service listens to port 80, which is the outside port. When you make a request to the server from a browser, the port that is publicly available outside of the server is port 80. Port 80 is the default, so when we make a request in our browser to this system, we will not have to define the port anymore. Just typing in localhost into the browser will result in us making a request to the NGINX router. Because everything goes through the NGINX router, we could even get rid of our ports definition on the ingress server, making the ingress server completely inaccessible outside of the network, meaning that we must go through the NGINX router. However, it is not essential; as we already covered, external requests from other computers can only access port 80.

We can also see that we point to our own NGINX configuration. The route to all files in our build configs are relative to the root of our system, and we must run our commands to run the system from the root of our system. Our conf file takes the following form:

```
# ingress/nginx.conf
events {
    worker_connections 512;
}
http {
    server {
        listen 80;
        location / {
            proxy_pass http://ingress:8001/;
        }
    }
}
```

This is a very simple config. Basically, our NGINX service listens to port 80 and routes all traffic to our ingress server. We can run the monolith system with the following command:

```
docker compose -f docker-compose.yml -f ingress/build-config.yml up
```

Here, we are stitching together our main databases and our build config for our monolith server. Once everything is running, we can test our monolith system with the following Bash script, run anywhere, just to test the network:

```
#!/usr/bin/env bash

curl -X POST http://0.0.0.0/api/v1/users/create \
-H "Content-Type: application/json" \
-d '{
    "email": "test@gmail.com",
    "password": "password"
}'
```

```
curl -u test@gmail.com:password -X GET \  
http://0.0.0.0/api/v1/auth/login
```

We can see that there is no port anymore. We are hitting the NGINX service and being routed to the ingress server. If we type localhost in our browser, we will get our rendered React application.

So, our monolith system is working. We can now build our microservices system.

Defining the build config for our microservices

For our microservices system, we have the network flow defined in *Figure 16.15*:

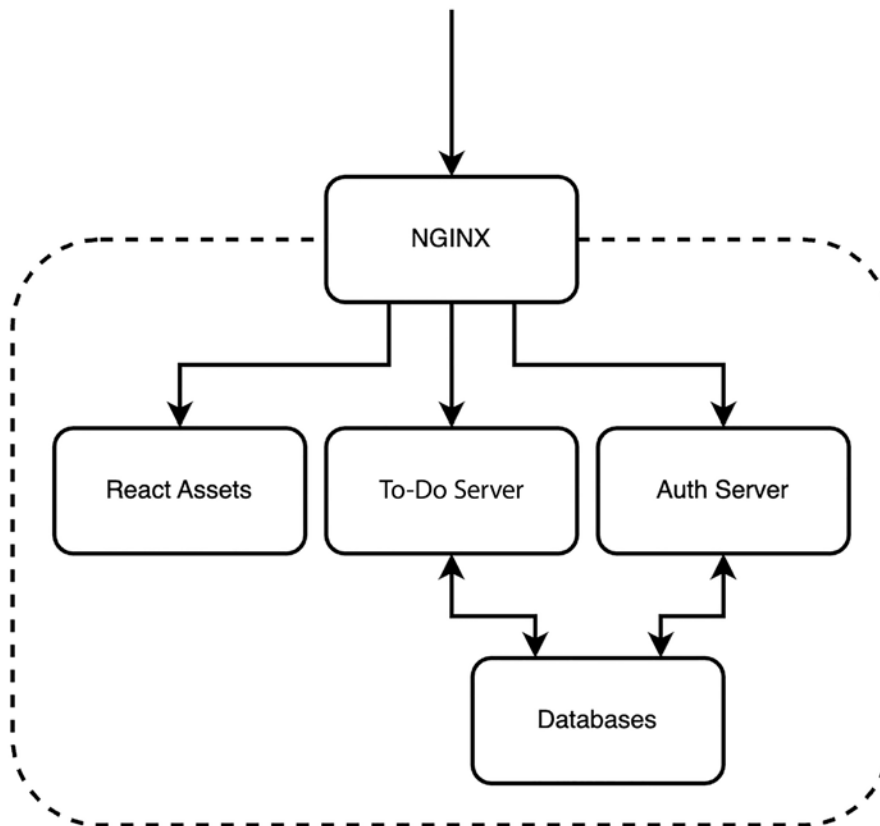


Figure 16.15 – Our microservices system overview

We have slightly more complex NGINX routing, and we are no longer running our ingress binary; therefore, we get our NGINX to serve our React app directly instead of routing frontend requests to another service. NGINX is optimized for this, so we could even see an improvement in the performance of serving React assets.

To build and run our microservices system, we must add the following files:

- nanoservices/to_do/build-config.yml
- nanoservices/auth/build-config.yml
- ingress/frontend/build-config.yml
- ingress/frontend/Dockerfile
- ingress/frontend/nginx.conf
- ingress/frontend/.dockerignore

For our to_do build config, we have the following:

```
# nanoservices/to_do/build-config.yml
version: "3.7"

services:

  to_do:
    image: compute-unit
    environment:
      - 'TO_DO_DB_URL=postgres://username:password@postgres/to_do'
      - 'AUTH_DB_URL=postgres://username:password@postgres/to_do'
      - 'AUTH_API_URL=http://auth:8081'
      - 'TO_DO_API_URL=http://to_do:8080'
      - 'JWT_SECRET=secret'
      - 'CACHE_API_URL=redis://cache:6379'
      - 'ELASTICSEARCH_URL=http://elasticsearch:9200/logs/_doc'
    ports:
      - "8080:8080"
    command: ["actix_server"]

    depends_on:
      postgres:
        condition: service_started
```

Here, we can see that we pass in the binary name of the to_do server that we want to run in order to run it. We also wait for the Postgres database to start. Looking at the config, I'm sure you can do the auth build config yourself, which takes the following form:

```
# nanoservices/auth/build-config.yml
version: "3.7"

services:

  auth:
```

```

image: compute-unit
environment:
  - 'TO_DO_DB_URL=postgres://username:password@postgres/to_do'
  - 'AUTH_DB_URL=postgres://username:password@postgres/to_do'
  - 'AUTH_API_URL=http://auth:8081'
  - 'TO_DO_API_URL=http://to_do:8080'
  - 'JWT_SECRET=secret'
  - 'CACHE_API_URL=redis://cache:6379'
  - 'ELASTICSEARCH_URL=http://elasticsearch:9200/logs/_doc'
ports:
  - "8081:8081"
command: ["auth_actix_server"]

depends_on:
  postgres:
    condition: service_started

```

And for our frontend build config, we have the following:

```

# ingress/frontend/build-config.yml
version: "3.7"

services:

  nginx:
    build: ./ingress/frontend/
    ports:
      - "80:80"
    links:
      - auth
      - to_do

```

Here, we can see that we point to the Dockerfile for the frontend, so we should build it. First, we ensure that we have Rust, Node, and wasm-pack for our build with the following code:

```

# ingress/frontend/Dockerfile
FROM rust:latest AS build
RUN curl -fsSL https://deb.nodesource.com/setup_18.x | bash -
RUN apt-get install -y nodejs
RUN cargo install wasm-pack

```

We then copy over the frontend code and build it with the following code:

```
# ingress/frontend/Dockerfile
WORKDIR /app
COPY . .
RUN npm install
RUN npm run wasm
RUN npm run build
```

Now that our frontend is built, we want to get rid of the Node modules and target from the Rust build for the WASM module, so we copy the public folder over to the Nginx image with the Nginx config file using the following code:

```
# ingress/frontend/Dockerfile
FROM nginx:alpine
COPY --from=build /app/public /usr/share/nginx/html
COPY nginx.conf /etc/nginx/nginx.conf
```

Finally, we expose our port and run our Nginx server with the following code:

```
# ingress/frontend/Dockerfile
EXPOSE 80
CMD ["nginx", "-g", "daemon off;"]
```

Our Docker Compose system is ready to run. The only thing we have is our Nginx configuration, which defines the following routes:

```
# ingress/frontend/nginx.conf
http {
    server {
        listen 80;
        location /api/v1/auth/ {
            proxy_pass http://auth:8081/api/v1/auth/;
        }
        location /api/v1/users/ {
            proxy_pass http://auth:8081/api/v1/users/;
        }
        location /api/v1/ {
            proxy_pass http://to_do:8080/api/v1/;
        }
        location / {
            root /usr/share/nginx/html;
            include /etc/nginx/mime.types;
            try_files $uri $uri/ /index.html;
        }
    }
}
```

```
}  
}
```

Here, we can see that the logic is the same as how we coded our ingress server. We forwarded backend endpoints to the backend servers, and then anything else is forwarded to the folder containing our React app. We also include the MIME types; otherwise, the browser will not render the assets returned, and if the request is not requesting a specific file, the index HTML file is served.

We can run our microservices cluster with the following command:

```
docker compose \  
-f docker-compose.yml \  
-f nanoservices/auth/build-config.yml \  
-f nanoservices/to_do/build-config.yml \  
-f ingress/frontend/build-config.yml \  
up
```

Our system works! However, while uploading all this code to a server and performing builds will technically work, it is not the most efficient. In the next section, we will upload our Docker images to a repository so they can be pulled from anywhere that has access without having to pull the code.

Distributing our system with Docker

If we want maximum flexibility on how we deploy our systems, we will want to put our Docker images in a registry. Registries are like GitHub, but for Docker images. You can make them private or public. You can deploy multiple versions and tag your images, kind of like Git branches. Orchestration systems such as Docker Compose or Kubernetes can pull these images into their system and run them. Skilled DevOps engineers can connect an entire pipeline from Git to their Kubernetes clusters. For instance, when a pull request gets merged into main, GitHub Actions gets triggered and builds a Docker image, which is then pushed to Docker Hub. The Kubernetes system has hooks listening to changes and will then pull the latest update automatically. You can also add filters on these hooks based on tags. This is beyond the scope of this book; however, to give a solid example, when Caroline and I built our Rust Kubernetes cluster, we had a development cluster and a live cluster. When new code was pushed to the development branch on GitHub, GitHub actions built new Docker images with development tags. Only the development cluster pulled the update. We did a new release by merging the development branch with main, triggering a build of Docker images that had the live tag. The production cluster then pulled these images automatically.

Before we can push our images to Docker Hub, we will have to create a Docker Hub repo. Registering our image on Docker Hub is straightforward. After logging in, we click on the **Create repository** button in the top-right corner, as shown in the following screenshot:

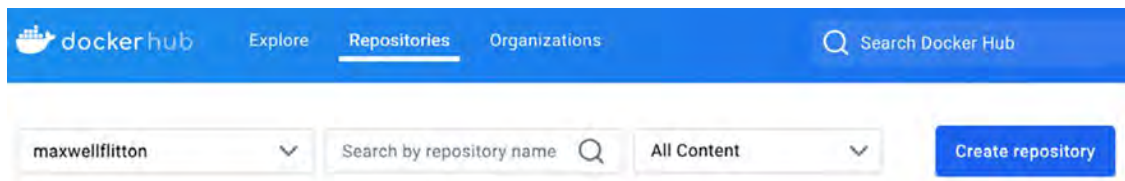


Figure 16.16 – Creating a new repository on Docker Hub

Once we have clicked on this, we define the repository with the following configuration:

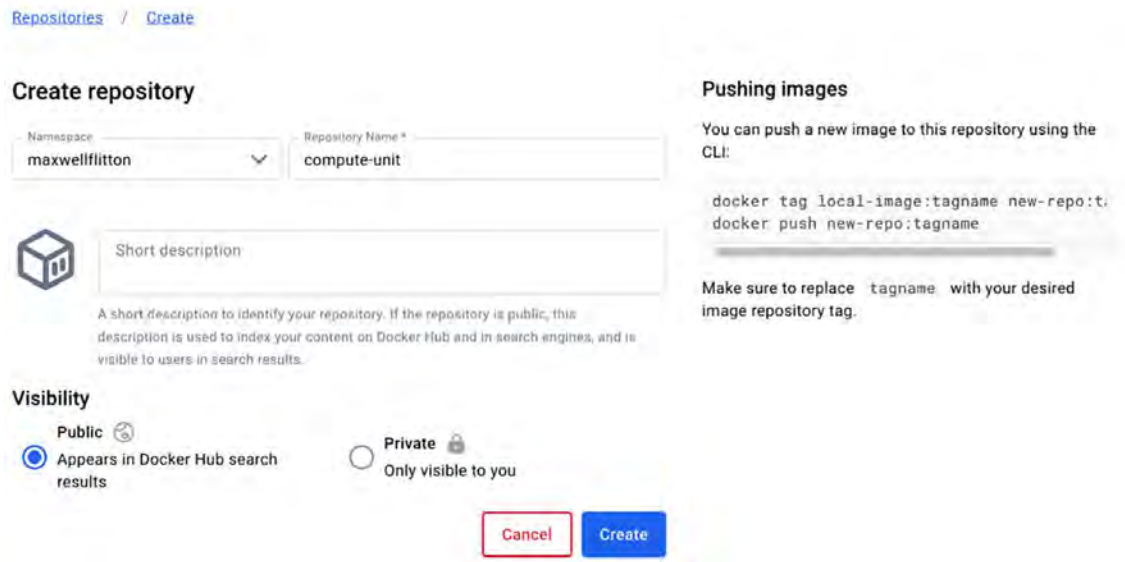


Figure 16.17 – Defining a new Docker repository

Now that we have our repositories defined, we can update our `run_build` Bash script with logging into Docker, building the Docker images, and pushing these built Docker images to Docker Hub with the following updated code:

```
# builds/run_build.sh
USERNAME=$1
PASSWORD=$2

ssh -i "~/.ssh/keys/remotebuild.pem" \
-o StrictHostKeyChecking=no -t \
ec2-user@$IP_ADDRESS << EOF
    until [ -f ./output.txt ]
    do
        echo "waiting for file"
        sleep 3
    done
```

```

echo "File found"
sudo yum install docker -y
sudo usermod -a -G docker ec2-user
sudo systemctl start docker.service
sudo docker login -u $USERNAME -p $PASSWORD
sudo docker build . -f Dockerfile.x86 -t compute-unit
sudo docker tag compute-unit $USERNAME/compute-unit:latest
sudo docker push $USERNAME/compute-unit:latest
cd ingress/frontend && sudo docker build . -t front-unit
sudo docker tag front-unit $USERNAME/front-unit:latest
sudo docker push $USERNAME/front-unit:latest
cd ../../nanoservices/user-session-cache/cache-module
sudo docker build . -t redis-cache
sudo docker tag redis-cache $USERNAME/redis-cache:latest
sudo docker push $USERNAME/redis-cache:latest
EOF

```

Here, we can see that we pass the username and password for Docker into the script, and we use these to log in to Docker and push the images that we have built. We also note that we have created a redis-cache repository and we are also building and pushing the redis-cache image. However, before we run this build script, there is a chance that the EC2 instance could run out of memory. To ensure that this does not happen, we can attach more memory to the EC2 build instance with the following code:

```

# builds/main.tf
resource "aws_instance" "build_server" {
  ami = "ami-05ea2888c91c97ca7"
  instance_type = "t2.medium"
  key_name = "remotebuild"
  user_data = file("server_build.sh")
  tags = {
    Name = "to-do build server"
  }
# root disk
  root_block_device {
    volume_size = "150"
    volume_type = "gp2"
    delete_on_termination = true
  }
}

```

Once we have run our build script again, we can see in our Docker Hub dashboard that the image has been shown, as in *Figure 16.18*:

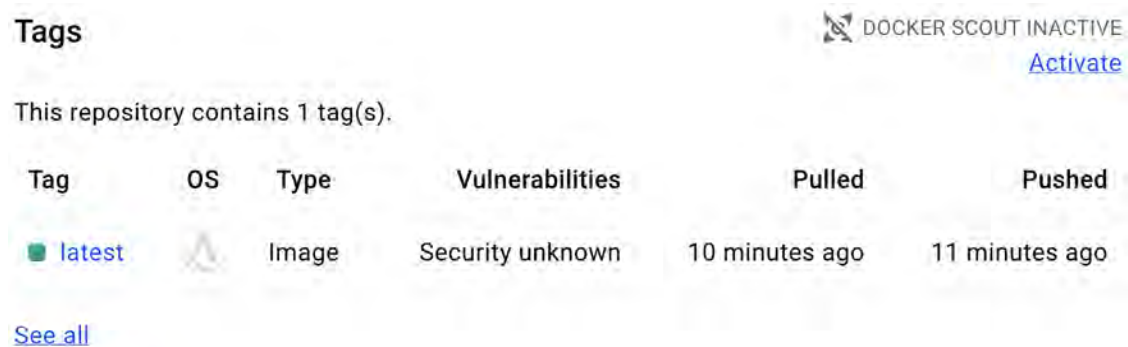


Figure 16.18 – View of pushed Docker repository

Our images are ready, and we can now move on to pulling these images for our deployed server.

Deploying our system with Docker

When it comes to deploying our system, we must create a server and a database, upload the docker-compose file to the new server, and pass the parameters of the created database into the Docker Compose when starting our system. To achieve this, we must carry out the following steps:

1. Write a Terraform script for our server and database.
2. Write a Docker Compose system for our server.
3. Write a deployment script for our system.

Before we write anything, however, we must have the following file structure in the root of our system:

```
└─ deployment
   ├── docker-compose.yml
   ├── main.tf
   ├── nginx.conf
   ├── run_deployment.sh
   └─ server_build.sh
```

The `server_build.sh` script should be copied from the deployment directory, as it is just setting up the server. To be honest, this file structure could be in its own repository, as we are not pointing to any builds. Instead, we will be pulling images from Docker Hub.

To get this deployment going, we can start by writing the Terraform script.

Writing a Terraform script for our server and database

For our Terraform script, we start with the following boilerplate code:

```
# deployment/main.tf
terraform {
  required_version = ">= 1.9.4"
  required_providers {
    aws = {
      source  = "hashicorp/aws"
      version = ">= 5.62.0"
    }
  }
}
provider "aws" {
  region = "eu-west-2"
}
```

We then allow the option to pass in some variables for the database username and password with the following code:

```
# deployment/main.tf
variable "db_password" {
  description = "The password for the database"
  default     = "password"
}
variable "db_username" {
  description = "The username for the database"
  default     = "username"
}
```

We can see that these variables have a default and a description. These variables could go into another Terraform file if we wanted to, as Terraform will scan the directory and subdirectories for Terraform files and stitch them together. However, our system is still basic, so we do not need to traverse variables into other files.

We then move on to our server, which takes the following form:

```
# deployment/main.tf
resource "aws_instance" "production_server" {
  ami          = "ami-05ea2888c91c97ca7"
  instance_type = "t2.medium"
  key_name     = "remotebuild"
  user_data    = file("server_build.sh")
  tags = {
```

```
    Name = "to-do production server"
  }
}
```

Now, we could run our Postgres database on the production server as a Docker image. However, this is fickle. If the server goes down, we could lose all our data. Also, managed database services come with extra perks. For instance, cloud providers offer backups and snapshots of managed database services. Managed databases are more expensive than a small EC2 instance, but if you are managing data that needs to be maintained, then managed database services become essential. We can create our managed Postgres database with the following code:

```
# deployment/main.tf
resource "aws_db_instance" "main_db" {
  instance_class      = "db.t3.micro"
  allocated_storage   = 5
  engine              = "postgres"
  username            = var.db_username
  password            = var.db_password
  db_name             = "to_do"
  publicly_accessible = true
  skip_final_snapshot = true
  parameter_group_name = aws_db_parameter_group.custom_pg.name
  tags = {
    Name = "to-do production database"
  }
}
```

Here, we can see that we have passed our variables into the definition of the database. We also pass in a parameter group that we have not defined yet. Parameter groups can be used to configure the parameters of things such as a database. For instance, if we were to SSH into a Postgres database, we would see several binaries talking to each other and config files. We could manually edit these config files ourselves and force a restart of the server, but Terraform allows us to configure parameters in these files with the following code:

```
# deployment/main.tf
resource "aws_db_parameter_group" "custom_pg" {
  name       = "custom-pg"
  family     = "postgres16"
  description = "allows unencrypted connections"
  parameter {
    name = "rds.force_ssl"
    value = "0"
  }
}
```

Here, we are switching off the enforced encryption. This is because our SQLX connections are not encrypted. If we do not update this parameter, the database connections from our to_do server will be blocked.

Once the database and server are built, we define the outputs with the following code:

```
# deployment/main.tf
output "ec2_global_ips" {
  value = [aws_instance.production_server.public_ip]
}
output "db_endpoint" {
  value = "${aws_db_instance.main_db.*.endpoint}"
}
```

Our Terraform script is now complete, and we can now move on to building our Docker Compose system.

Writing a Docker Compose system for our server

For our docker-compose file, we have the following definitions for our server and NGINX:

```
# deployment/docker-compose.yml
version: "3.7"
services:
  ingress:
    image: maxwellflitton/compute-unit
    environment:
      - 'TO_DO_DB_URL=${DATABASE_URL}'
      - 'AUTH_DB_URL=${DATABASE_URL}'
      - 'JWT_SECRET=secret'
      - 'CACHE_API_URL=redis://cache:6379'
      - 'ELASTICSEARCH_URL=http://elasticsearch:9200/logs/_doc'
    ports:
      - "8001:8001"
  nginx:
    container_name: 'nginx'
    image: "nginx:latest"
    ports:
      - "80:80"
    links:
      - ingress
    volumes:
      - ./nginx.conf:/etc/nginx/nginx.conf
```

Here, we can see that we pass in our password and username parameters for the database. We can also see that we reference the `nginx.conf` file that we will upload to the server. If we were going to deploy our microservices system, we would have to pull the NGINX image with our frontend.

For Elasticsearch and the cache, we have the same code that we have been building throughout the chapter, which is the following:

```
# deployment/docker-compose.yml
cache:
  container_name: 'to-do-redis'
  image: 'maxwellflitton/redis-cache'
  restart: always
  ports:
    - '6379:6379'
elasticsearch:
  image: docker.elastic.co/elasticsearch/elasticsearch:7.13.4
  container_name: elasticsearch
  environment:
    - discovery.type=single-node
    - bootstrap.memory_lock=true
    - "ES_JAVA_OPTS=-Xms512m -Xmx512m"
  ulimits:
    memlock:
      soft: -1
      hard: -1
  ports:
    - "9200:9200"
```

Finally, for our NGINX conf file, we have the following configuration:

```
# deployment/nginx.conf
# deployment/nginx.conf
events {
    worker_connections 512;
}
http {
    server {
        listen 80;
        location / {
            proxy_pass http://ingress:8001/;
        }
    }
}
```

Here, we can see that we are merely passing the traffic to our monolith server. Now that everything is defined for our system, we just need to build our deployment script.

Writing a deployment script for our system

For our deployment system, there are a few changes, but essentially, the approach is like our build script. First, we have some boilerplate code to ensure that the deployment process is running in the same directory as the script and that we have all the arguments that we need for the deploy script with the following code:

```
#!/usr/bin/env bash
# deployment/run_deployment.sh

SCRIPTPATH="$( cd "$(dirname "$0")" ; pwd -P )"
cd $SCRIPTPATH

USERNAME=$1
PASSWORD=$2
DB_USERNAME=$3
DB_PASSWORD=$4
```

We then apply the Terraform build pass in our variables with the following code:

```
# deployment/run_deployment.sh
terraform init

terraform apply \
  -var="db_username=$DB_USERNAME" \
  -var="db_password=$DB_PASSWORD" \
  -auto-approve

terraform output -json > ./output.json
```

We then get the IP address of the server and the address of our database with the following code:

```
# deployment/run_deployment.sh
OUTPUT_FILE="output.json"

IP_ADDRESS=$(jq --raw-output '.ec2_global_ips.value[0]' \
  $OUTPUT_FILE)
echo $IP_ADDRESS
DB_ENDPOINT=$(jq -r '.db_endpoint.value[0]' $OUTPUT_FILE)
```


With the database endpoint, we define our Postgres URL with the following code:

```
# deployment/run_deployment.sh
DB_PORT=5432
DB_NAME="to_do"
POSTGRES_URL="postgresql://${DB_USERNAME}:${DB_PASSWORD}"\
"@${DB_ENDPOINT}/${DB_NAME}"
echo "PostgreSQL URL: $POSTGRES_URL"
```

We now have all the infrastructure built and the information we need. With this information, we upload the Compose file and NGINX conf file with the following code:

```
# deployment/run_deployment.sh
scp -i "~/ssh/keys/remotebuild.pem" \
-o StrictHostKeyChecking=no \
./docker-compose.yml \
ec2-user@$IP_ADDRESS:/home/ec2-user/

scp -i "~/ssh/keys/remotebuild.pem" \
-o StrictHostKeyChecking=no \
./nginx.conf \
ec2-user@$IP_ADDRESS:/home/ec2-user/
```

Finally, we SSH into our server, install Docker, log in to Docker Hub, and run our system with our database URL passed into the Docker Compose system with the following code:

```
# deployment/run_deployment.sh
ssh -i "~/ssh/keys/remotebuild.pem" \
-o StrictHostKeyChecking=no -t \
ec2-user@$IP_ADDRESS << EOF
    until [ -f ./output.txt ]
    do
        echo "waiting for file"
        sleep 3
    done
    echo "File found"
    sudo yum install docker -y
    sudo usermod -a -G docker ec2-user
    sudo systemctl start docker.service
    sudo docker login -u $USERNAME -p $PASSWORD
    export DATABASE_URL=$POSTGRES_URL
    sudo DATABASE_URL=$POSTGRES_URL docker-compose up -d
EOF
```

If we run our script, our system will be deployed, and we can access our application on our server IP address, as seen in *Figure 16.19*:

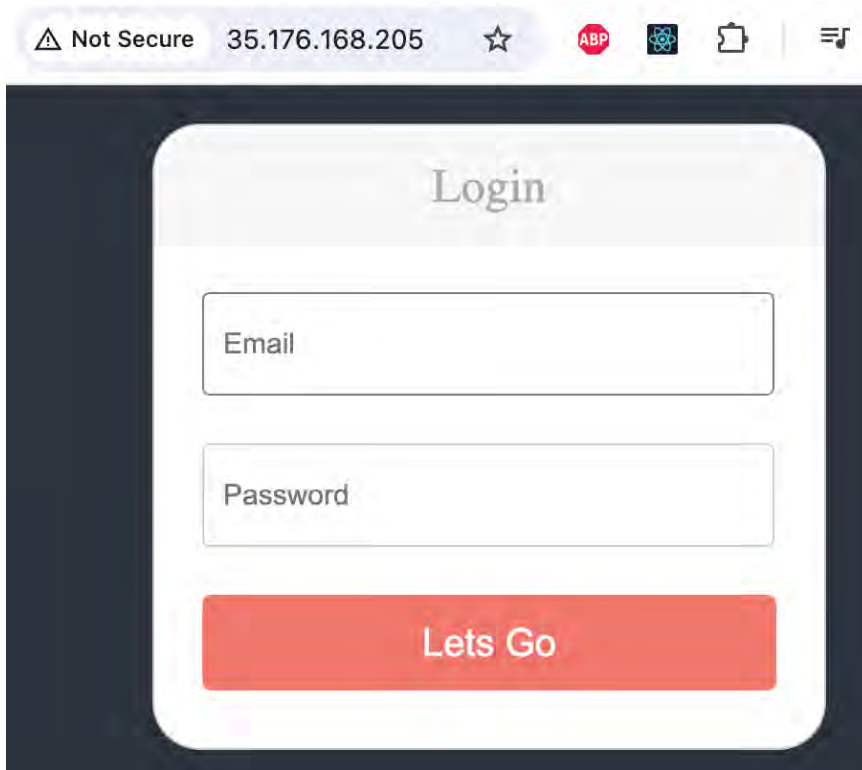


Figure 16.19 – View of our application on our AWS server

While our server works, we can see in *Figure 16.19* that our connection is not secure. Our server is also not secure against all types of traffic. In the next chapter, we will lock down our server and enable HTTPS encryption for our traffic.

Summary

In this chapter, we built the infrastructure and scripting to fully automate the build and deployment of our servers to Docker Hub. We then built out our database and server for deployment. We also built tiny distroless images and could support multiple different servers that can be run just via the type of command passed into the image when creating a container. These skill sets are not just restricted to deploying a web server. At a fundamental level, you understand the linking of libraries and how to package Rust software to run efficiently. I have used my Docker knowledge to run Rust code that directly interacts with GPUs, as Docker enables us to run our Rust code with libraries and drivers installed for a specific purpose without having to alter the machine in general. With volumes, there is nothing stopping you from building a tiny binary that listens to Unix sockets via a volume to run in the background of your computer.

As for our web app, it is very unsecure. In the next chapter, we will improve our Terraform scripts to lock down traffic to the server and database, to ensure that the traffic can only go through an ingress and not directly hit the server or database. We will also cover encryption with NGINX to facilitate HTTPS traffic between the client and NGINX.

Questions

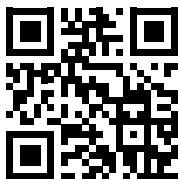
1. At a high level, what are the steps of building and distributing our servers via Docker?
2. At a high level, how do we deploy our app so it can be accessed by other people?
3. Why is our current deployment not secure?
4. Distroless results in tiny images because it is stripped down to the bare minimum. If we were to just compile a Rust binary and copy it over to the distroless build without doing anything else, it would not be able to run the Rust server. Why is this, and how can we solve it?
5. How do you get the IP address of the created server to perform operations on the server?

Answers

1. We create an EC2 server using Terraform, and then copy all our code onto that server. We then build our images, log in to Docker Hub, and push the built images with a tag to Docker Hub so anyone with access to that Docker Hub repo can pull from it.
2. We create an EC2 server and a Postgres-managed database with the username and password for the database passed in. We then copy over a stripped-down version of Docker Compose that does not reference any builds, but the Docker Hub image repo instead. We log in to Docker Hub, pull the images, pass the database parameters into the image, and then run Docker Compose in the background. Once Docker Compose is running, we can access the app by visiting the IP address of the server in our browser.
3. Our current deployment does not have HTTPS, meaning that the traffic between the client and server is not encrypted. Our server and database also have no traffic restrictions, meaning that anyone from anywhere can make direct network requests to our server and database. This needs to be locked down, and we will cover this in the next chapter.
4. The Rust server will not be able to run because the dynamic libraries for things such as encryption, accessing ports, and accessing a runtime on the CPU are missing. We can solve this problem by performing the `ldd` command against the binary and copying over the libraries that the binary needs to run.
5. You define the outputs, such as the IP address of the server. We can then run the `terraform output` command and write that to a JSON file. We can then use `jq` to read the values from the JSON output file and use these variables, such as the IP address, in other operations.

Learn more on Discord

To join the Discord community for this book – where you can share feedback, ask questions to the author, and learn about new releases – follow this QR code: <https://packt.link/EaKXL>



17

Configuring HTTPS with NGINX on AWS

When it comes to deploying our applications, a lot of tutorials and books go through simple deployments and smooth over the concept of encrypting traffic to and from the server using HTTPS. However, HTTPS is essential and usually the biggest hurdle that a developer must overcome to get their website or API out into the world. While this book's title is Rust Web Programming, it is essential to dedicate a chapter to truly understanding how HTTPS works so that you can implement HTTPS locally and then on the **Amazon Web Services (AWS)** cloud. This chapter will cover the following topics:

- What is HTTPS?
- Implementing HTTPS locally with Docker Compose
- Attaching a URL to our deployed application on AWS
- Enforcing HTTPS on our application on AWS

By the end of this chapter, you will be able to build infrastructure in Terraform code that can encrypt traffic and lock down unwanted traffic to our **Elastic Compute Cloud (EC2)** instances so that they can only accept traffic from the load balancer, which will be explained later in the chapter. What is best is that most of this is automated, and as we are using Docker to deploy our applications, you will be able to transfer this skill to any web project that you want to deploy in the future. While this is not the best implementation, as there are entire books dedicated to cloud computing, you will be able to implement a solid, safe deployment that will continue to serve users even if an EC2 instance is out of service, as the load balancer can route to other instances. It will also be able to scale if traffic demand increases.

Technical requirements

You can download the example project and code for this book by following the instructions in the **Download the example code files** section in the *Preface* of this book.

This chapter's code files are included in the downloadable code bundle.

What is HTTPS?

So far, our frontend and backend applications have been running through HTTP. However, this is not secure and has some drawbacks. To secure the traffic between our browser and our NGINX server, we are going to have to ensure that our application is using the HTTP/2 protocol. The HTTP/2 protocol has the following differences from the standard HTTP/1 protocol:

- **Binary protocol:** HTTP uses a text-based protocol, whereas HTTP/2 uses a binary protocol. A binary protocol uses bytes to transmit data as opposed to human-readable characters, which are encoded using the **American Standard Code for Information Interchange (ASCII)**. Using bytes reduces the number of possible errors and the size needed to transfer data. It will also enable us to encrypt our data stream, which is the basis of HTTPS.
- **Compressed headers:** HTTP/2 compresses headers when sending requests. Compressing headers has similar benefits to the binary protocol, which results in a smaller size of data needed to transfer the same request. The HTTP/2 protocol uses the HPACK format.
- **Persistent connections:** When using HTTP, our browser must make a request every time a resource is needed. For instance, we could get our NGINX server to serve an HTML file. This will result in one request to get the HTML file. Inside the HTML file, there could be a reference to a CSS file, and this would result in another request to the NGINX server. It is also not uncommon to have a reference to a JavaScript file in the HTML file. This would result in another request. Therefore, to load a standard web page, our browser requires up to three requests. This does not scale well when we are running a server with multiple users. With HTTP/2, we can have persistent connections. This means that we can make three requests for the HTML, CSS, and JavaScript files in one connection.
- **Multiplex streaming:** Making requests using HTTP/1 means that we must send our requests sequentially. This means that we make one request, wait for this request to be resolved, and then send another request. With HTTP/2, we use multiplex streaming, which means that we can send multiple requests at the same time and resolve them when the responses are returned. Combining multiplex streaming with a persistent connection results in a faster loading time. Any reader who used the internet in the 1990s will remember having to wait a long time for a simple page to be loaded. Granted, the internet connection back then was not as fast, but it was also a result of making multiple HTTP sequential requests with different connections to load multiple pictures, HTML, and CSS.

Now that we have explored the difference between HTTP and HTTP/2, we can explore HTTPS, which is built on top of HTTP/2. Before we go forward, however, it must be stated that security is a field in itself. Learning about the high-level concepts around HTTPS is enough to make us understand the importance of what we are implementing and why we take certain steps. However, it does not make us security experts.

Before we explore the steps of HTTPS, we need to understand what a **man-in-the-middle attack** is, because this attack inspires the steps of HTTPS. A man-in-the-middle attack is exactly what it sounds like: a malicious eavesdropper that can intercept communication packets between the user and the server. This also means that the eavesdropper can obtain encryption if they are passed over a network.

Simply googling man-in-the-middle attacks will result in loads of tutorials and software that can be downloaded to implement such attacks. There are more caveats to security that are beyond the scope of this book, but to conclude, if you are hosting a website that you expect users to connect to and log in to, there is no excuse not to use HTTPS.

When it comes to HTTPS, there is a range of steps that need to be completed. First, before any requests are made to the server, the owner of the server and the domain must get a certificate from a trusted central authority. There are not many trusted central authorities around. What these authorities do is get some identification of the person who owns the domain and some proof that the person applying for the certificate owns the domain. This might sound like a headache, but a lot of URL providers, such as AWS, have streamlined the process using information such as payment details to send to the central trusted authority in the backend when you are pointing and clicking to buy the domain. We must fill in some extra forms, but if you have a working AWS account, this will not be too taxing. These central authorities are limited because anyone with a computer can create a digital certificate. For instance, if we were intercepting traffic between a server and a user, we could generate our own digital certificates with a key and forward them to the user. Because of this, mainstream browsers only recognize certificates that have been issued by a small number of recognized authorities.

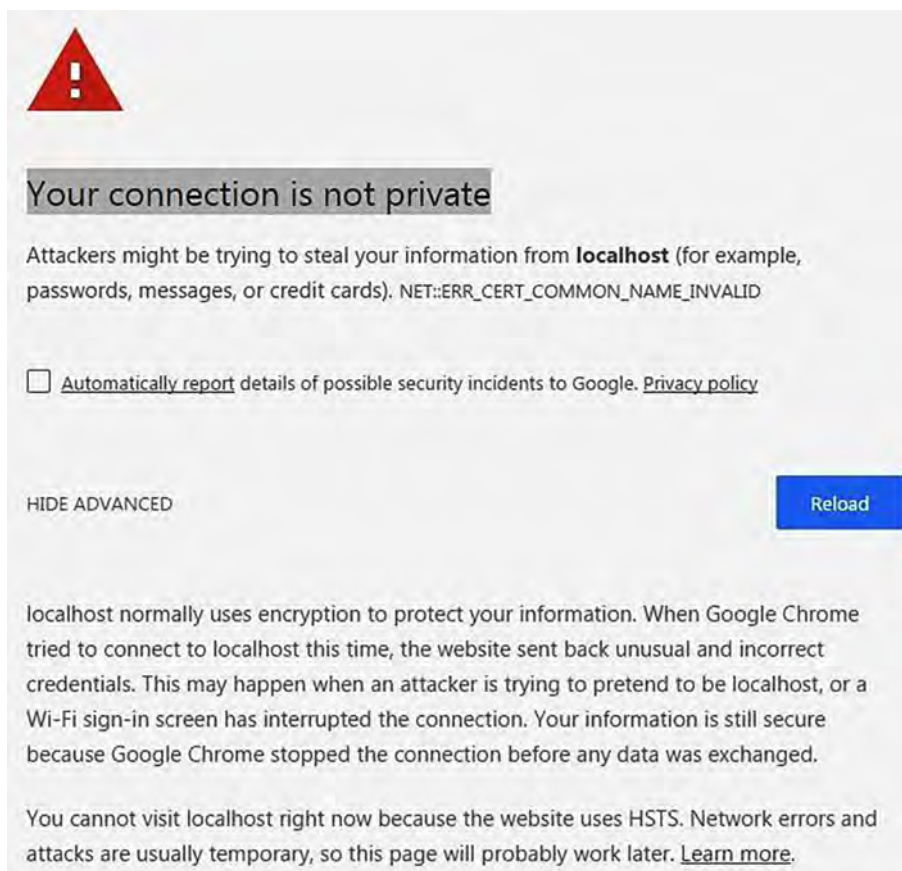


Figure 17.1 – Chrome blocking access due to an unrecognized certificate

It is not advised to proceed. Once the owner of the domain and server has the certificate from the central authority, the user can make requests. Before any meaningful data to the application is exchanged, the server sends over the certificate with the public key to the user. The user then creates a session key and encrypts this session key with the public key of the certificate, which is then sent back to the server. The server can then decrypt the key using the private key that was not sent over the network. Therefore, even if the eavesdropper manages to intercept the messages and get the session key, it is encrypted, so they cannot use it. Both the server and client can check the validity of mutually encrypted messages. We can use the session key to send encrypted messages to and from the server and the user, as seen in the following diagram:

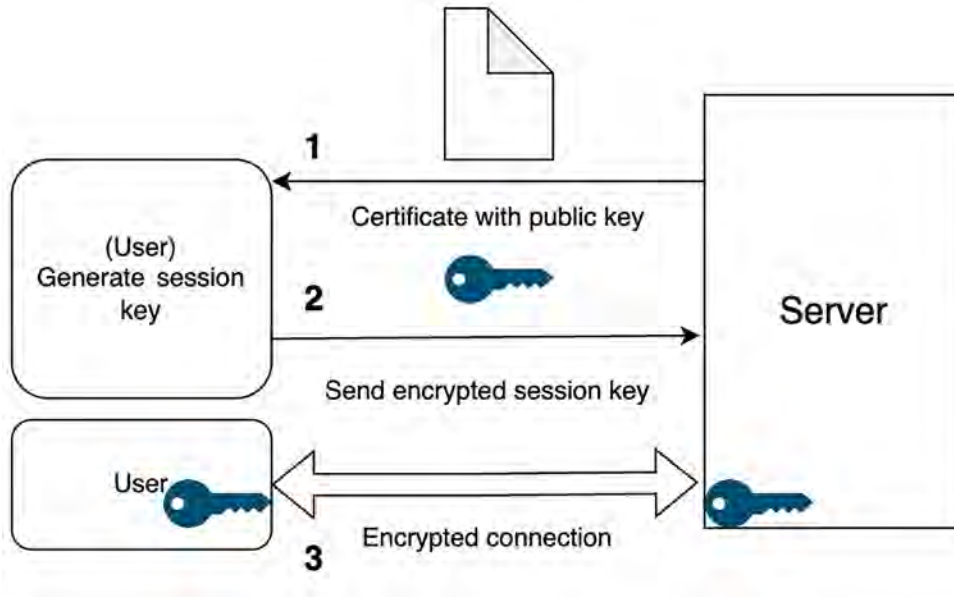


Figure 17.2 – Steps required when making an HTTPS connection

Do not worry—there are packages and tools that will help us manage the HTTPS process; we will not have to implement our own protocol. You will understand why we must carry out certain steps and how to troubleshoot problems when they arise. In the next section, we will implement a basic HTTPS protocol with NGINX locally.

Implementing HTTPS locally with Docker Compose

When it comes to implementing HTTPS, most of the work is going to be achieved through NGINX. Although we have worked a little with NGINX, the NGINX configuration is a powerful tool. You can implement conditional logic, pull variables and data from the request and act on them, redirect traffic, and much more. In this chapter, we are going to do enough to implement HTTPS, but it is advised that, if you have time, read up on the fundamentals of NGINX configurations; reading material is provided in the *Further reading* section. For our NGINX .conf file, we have the following layout:

```
#ingress/nginx.conf
events {
```



```

    worker_connections 512;
}
http {
    server {
        . . .
    }
    server {
        . . .
    }
}

```

Here, we can see that we have two server scopes in our http scope. This is because we need to enforce HTTPS. We must remember that our outside port is 80. However, if we want to carry out an HTTPS connection, we instead need to connect to port 443, which is the standard for HTTPS. Typing `https://` into the browser will target port 443, and typing `http://` into the browser will target port 80. If we allow port 80 to be open, loads of users will access our site in an unsecured way because some people will type `http://` into the browser. Hackers would also spread the HTTP links around because they want as many people as possible not to use a secure network. However, if we block port 80, people who put `http://` into the browser will just get blocked from accessing the website. It is unlikely that the average user is going to understand the differences in ports, look at what they typed in, and correct it. Instead, they are just going to think that the site is down. Therefore, we are going to have to listen to both ports 443 and 80. However, when a request is made to port 80, we are going to redirect the request to port 443 instead. Our first server scope can redirect with the following code:

```

#ingress/nginx.conf
server {
    listen 80;
    return 301 https://$host$request_uri;
}

```

Here, we can see that we listen to port 80, and then return the same request that was sent, but with the HTTPS protocol, which means that it will hit port 443. We can also see that we reference the `$host` and `$request_uri` variables. These variables are standard variables in NGINX that are automatically populated. We can define our own variables with the following line of code:

```

set $name 'Maxwell';

```

However, we want our NGINX instance to work on our server and localhost, so using the standard variables is the best choice here. Now that we have defined our rerouting rule, we can move on to the next server scope; we listen to port 443 with the following code:

```

#ingress/nginx.conf
server {
    listen 443 ssl http2;
    ssl_certificate /etc/nginx/ssl/self.crt;
}

```

```

ssl_certificate_key /etc/nginx/ssl/self.key;
location / {
    proxy_pass http://ingress:8001/;
}
}

```

Alongside the port definition, we also declare that we are using the `ssl` and `http2` NGINX modules. This is not surprising as HTTPS is essentially SSL on top of HTTP/2. We then define where our server certificate is in the NGINX container. We will add these later in the `docker-compose` volume. We can also see that we pass our HTTPS request to the appropriate application via HTTP. If we try to change these proxies to the HTTPS protocol, we would get a bad gateway error. This is because the handshake between NGINX and our services would fail. It is not essential because we must remember that the ports that the frontend and backend applications expose are not available to anyone outside of local-host. On our local machine, we can access them, but this is because they are running on our local machine. If we were to deploy our application server, it would look like this:

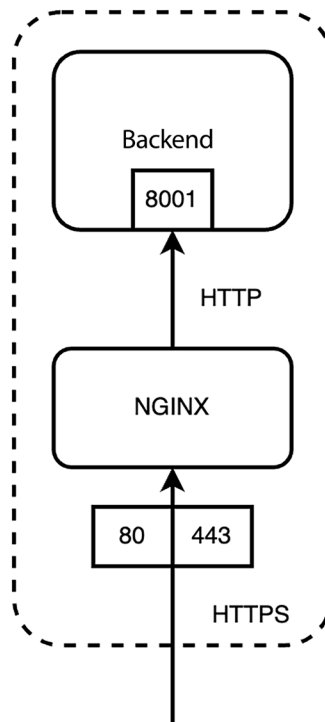


Figure 17.3 – Traffic flow if deployed on a server

Our NGINX configuration is not optimal. There are settings that can be tweaked in terms of ciphers, caching, and managing timeouts. However, this is enough to get an HTTPS protocol working. If you get to the point that you need to optimize the caching and encryption methods of your NGINX configuration, it is suggested that you seek further education materials on DevOps and NGINX. Now that we have defined our NGINX configuration, we must define our certificates.

Installing SSL

To define our own certificates, we must install the `openssl` package by following the steps using the following links:

- **Linux:** <https://fedingo.com/how-to-install-openssl-in-ubuntu/>
- **Windows:** <https://linuxhint.com/install-openssl-windows/>
- **Mac:** <https://yasar-yy.medium.com/installing-openssl-library-on-macoscatalina-6777a2e238a6>

This can be done with the following command:

```
openssl req -x509 -days 10 -nodes -newkey rsa:2048 \  
-keyout ./ingress/self.key \  
-out ./ingress/self.crt
```

This creates a key with x509, which is the International Telecommunication Union standard. We state that the certificate will expire in 10 days and that the key and certificate will have the name `self`. They can be called anything; however, for us, it makes sense to call the certificate `self` because it is a self-issued certificate. The command shown in the previous code snippet will push several prompts. It does not matter what you say to these prompts, as we will just be making localhost requests with them, meaning that they will never go anywhere outside of our local computer. We can now stash the key and certificate anywhere within the deployment directory if you can reference the key and certificate placed in the `docker-compose.yml` file. In our ingress build config file, our NGINX service now takes the following form:

```
# ingress/build-config.yml  
version: "3.7"  
services:  
  ingress:  
    . . .  
  nginx:  
    container_name: 'nginx'  
    image: "nginx:latest"  
    ports:  
      - "80:80"  
      - "443:443"  
    links:  
      - ingress  
    volumes:  
      - ./ingress/nginx.conf:/etc/nginx/nginx.conf  
      - ./ingress/self.crt:/etc/nginx/ssl/self.crt  
      - ./ingress/self.key:/etc/nginx/ssl/self.key
```

While where you get your key and certificate from may vary, it is important that you put the key and the certificate inside the `etc/nginx/ssl/` directory inside the NGINX container.

We are now at the point where you can test our application to see if the local HTTPS is working. If you spin up your Docker Compose instance and then go to `https://localhost` in your browser, you should get a warning that it is not secure, and you will not be able to instantly connect to the frontend. This is reassuring because we are not a central authority, so our browser will not recognize our certificate. There is a multitude of browsers, and we would waste a lot of space in this book describing how to get past this for every browser. Considering browsers are free to download, we can get around the blocking of our application in Chrome by going to the flags URL, as seen here:

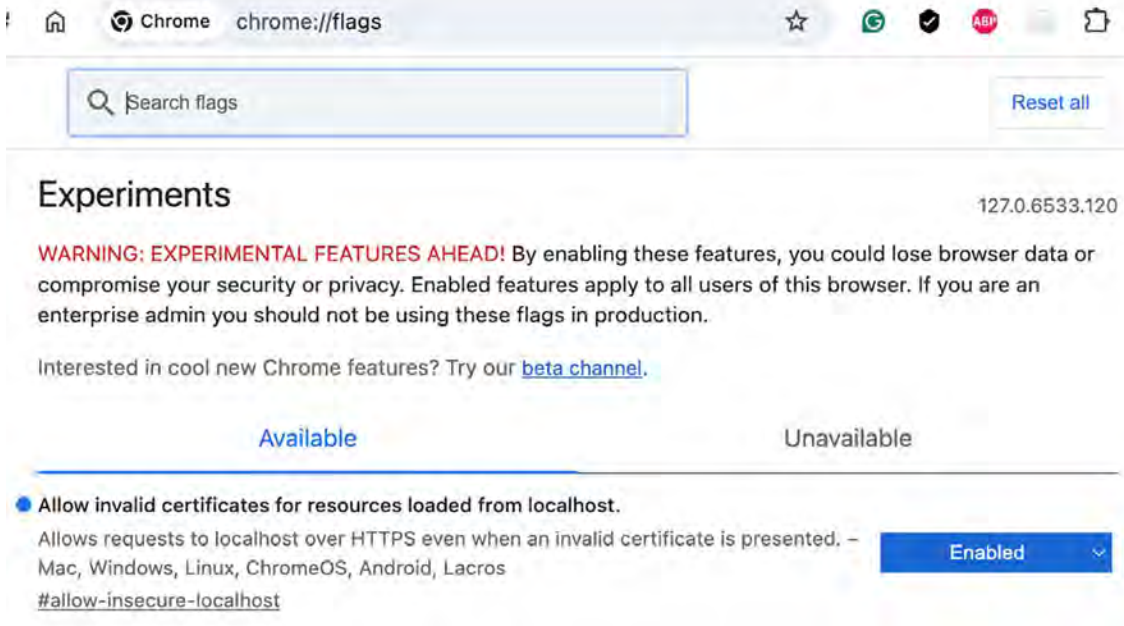


Figure 17.4 – Allowing our application certificate to pass in Chrome

Here, we can see that I have allowed invalid certificates from localhost. Now that our invalid certificate is enabled in our browser, we can access our application, as seen here:

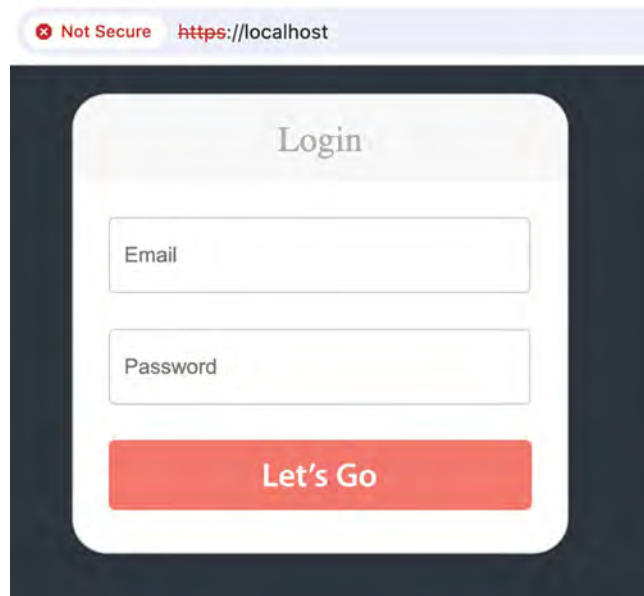


Figure 17.5 – Accessing our application through HTTPS

Here, we are using the HTTPS protocol; however, as we can see in the preceding screenshot, Chrome is complaining, stating that it is not secure. We can inspect the reason why by clicking on the **Not Secure** statement, giving the following view:

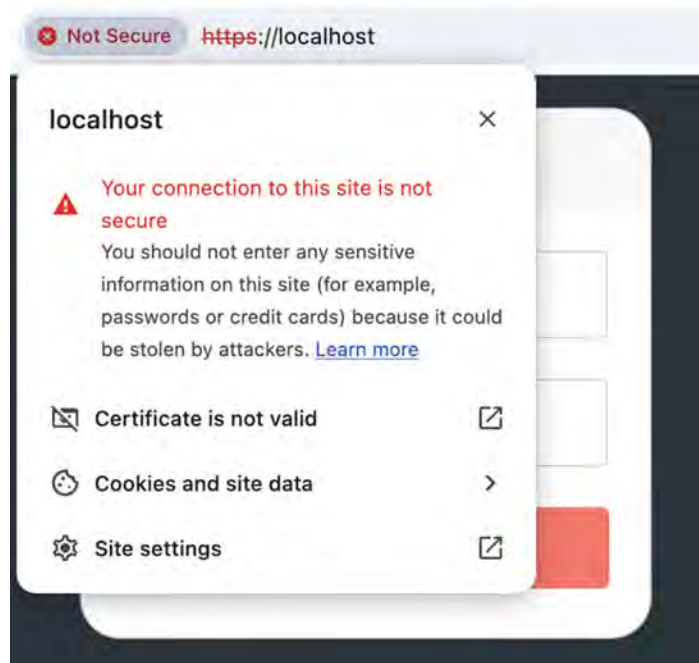


Figure 17.6 – Explaining why the connection is not secure

Here, we can see that our certificate is not valid. We expected the certificate not to be valid because we issued it, which makes it not officially recognized. However, our HTTPS connection is working, and we can see that the React app is served using HTTPS with the following screenshot of the network:

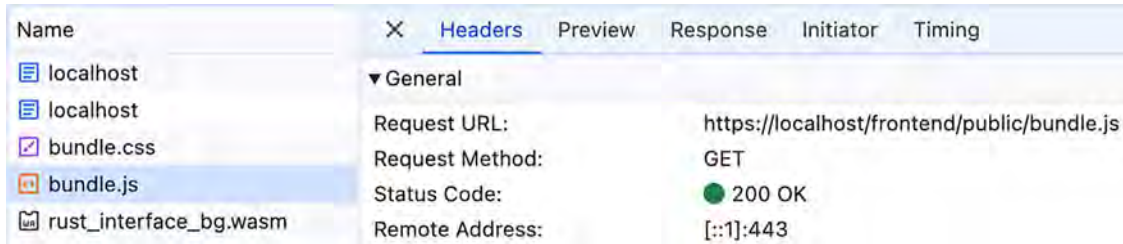


Figure 17.7 – HTTPS is serving our application

This is interesting to see how HTTPS works; however, it is not useful with a self-signed certificate running on our localhost.

If we want to utilize HTTPS, we are going to have to apply it to our application on AWS. There are a couple of steps we need to carry out before we implement HTTPS on AWS. In the next section, we will assign a URL to our application.

Attaching a URL to our deployed application on AWS

In the previous chapter, we managed to deploy our to-do application onto a server on AWS and access this application directly by putting the IP address of the server into our browser. When it comes to registering our URL, you will be exposed to multiple acronyms. To feel comfortable when navigating AWS routing, it makes sense to be familiar with URL acronyms by reading the following diagram, where TLD stands for Top-Level-Domain and SLD is Second-Level-Domain:

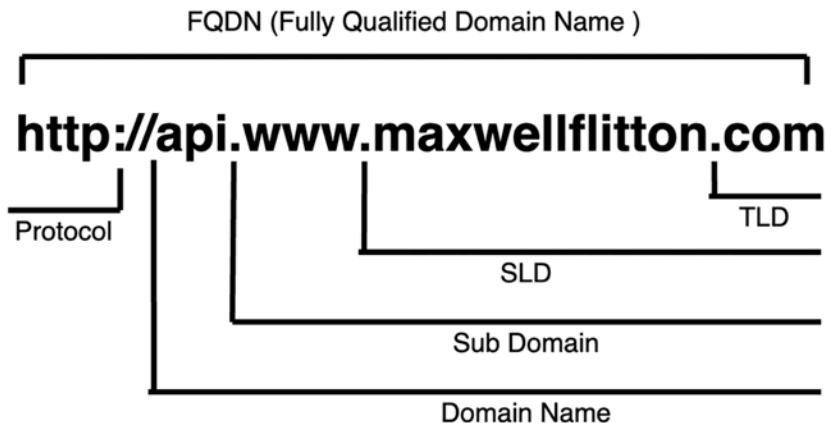


Figure 17.8 – Anatomy of a URL

When we are associating a URL with our application, we are going to be configuring the **Domain Name System (DNS)**. A DNS is a system that translates a user-friendly URL to an IP address. For a DNS system to work, we will need the following components:

- **Domain registrar:** An organization, such as AWS, Google Cloud, Azure, GoDaddy, and so on, that will register a domain if it receives payment and the personal details of the person responsible for the domain. This organization will also handle reports of abuse if the URL is used in illegal activity.
- **DNS record:** A registered URL can have multiple DNS records. A DNS record essentially defines a routing rule for the URL. For instance, a simple DNS record will forward the URL to an IP address of a server.
- **Zone file:** A container for DNS records (in our case, the zone file will be managed by AWS). The DNS records and registrars are essential for our URL to work. Even though we can directly connect to IP addresses, there are a couple of middlemen if we want to connect to a URL, as laid out in the following diagram, where **TTL** stands for **Time-To-Live**, **TLN DNS** stands for **Top-Level Name DNS**, and **SLD DNS** stands for **Second-Level Domain DNS**:

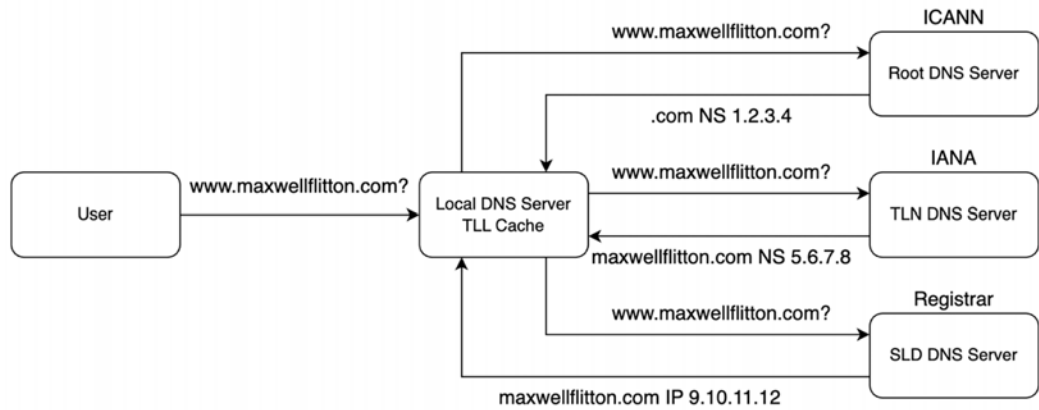


Figure 17.9 – Steps required to connect a URL to an IP address

As we can see from the preceding diagram, if we want to connect to a server, we send the URL to the local DNS server. This server then makes three calls in sequential order from top to bottom. By the end of the three requests, the local DNS server will have the IP address related to the URL. We can see that the registrar is responsible for part of the mapping. This is where our DNS records are configured. If we remove our DNS records, then the URL is no longer available on the internet. We do not have to make the calls laid out in *Figure 17.9* every time we enter a URL. Our browsers and the local DNS server will cache the URL to the IP address mapped to reduce the number of calls to the other three servers. There is a problem, however; when we have been building our production server, you might have realized that the IP address has changed every time we tear down and spin up a production server. There's nothing wrong going on here; when we create an EC2 instance, we must take a server that is available. A cloud provider such as AWS cannot hold the server aside just for us unless we want to pay for it. In the next section, we will keep our IP consistent with Elastic IP addresses.

Attaching an Elastic IP to our server

Elastic IP addresses are essentially fixed IP addresses that we keep. We can then attach these Elastic IP addresses to any one EC2 instance at one point in time as we see fit. This is very helpful when it comes to routing. We can set up the routing of a URL to an Elastic IP and then switch the allocation of the Elastic IP to the server that we need. This means that we can deploy a new application to another server, test it, and then switch our Elastic IP to the new deployment server without having to touch the routing for our URL. We will not be creating an Elastic IP every time we spin up a production server. Because of this, it is OK to point and click in the AWS console to create and attach the Elastic IP address. Before we do this, however, we need to deploy our production server with the previous NGINX config file that does not have HTTPS defined, and Docker Compose will not point to self-signed keys in the volume, as we are passing the HTTPS off to the load balancer we create in AWS.

Now that we have removed all references to our keys and port 443 routing in our Docker Compose system and NGINX conf files, reverting to the state of our system before implementing HTTPS locally, we are ready to deploy our build on our production server. Remember, we can do this using the `deployment/run_build.sh` script that we set up in the previous chapter. Once the server is built, we know that there is an EC2 instance live, but change the tag of the EC2 server in `deployment.main.tf` to `to-do production server` tag. We are now ready to allocate an Elastic IP address to our EC2 instance. To allocate Elastic IPs, we first need to navigate to the EC2 service by searching for EC2 in the search bar in the AWS dashboard at the top and clicking on the service, resulting in the following view:

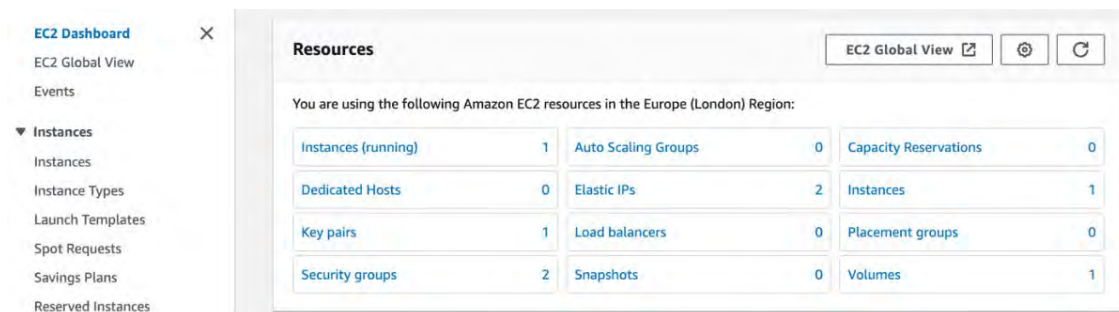


Figure 17.10 – EC2 dashboard view

We can see that Elastic IPs can be accessed on the right of the **Resources** panel and on the left of the screen. Once we are in the Elastic IP address dashboard, we will have a list of Elastic IP addresses that you have. At the top right of the screen, there will be an orange button labeled **Allocate Elastic IP address**. If you click this button, you will get the following creation form:

[EC2](#) > [Elastic IP addresses](#) > [Allocate Elastic IP address](#)

Allocate Elastic IP address [Info](#)

Elastic IP address settings [Info](#)

Public IPv4 address pool

☒ Amazon's pool of IPv4 addresses

☐ Public IPv4 address that you bring to your AWS account with BYOIP. (option disabled because no pools found) [Learn more](#)

☐ Customer-owned pool of IPv4 addresses created from your on-premises network for use with an Outpost. (option disabled because no customer owned pools found) [Learn more](#)

Network border group [Info](#)

Global static IP addresses

AWS Global Accelerator can provide global static IP addresses that are announced worldwide using anycast from AWS edge locations. This can help improve the availability and latency for your user traffic by using the Amazon global network. [Learn more](#)

Create accelerator

Tags - optional

A tag is a label that you assign to an AWS resource. Each tag consists of a key and an optional value. You can use tags to search and filter your resources or track your AWS costs.

No tags associated with the resource.

Add new tag

You can add up to 50 more tag

Cancel

Allocate

Figure 17.11 – Allocating (creating) an Elastic IP address

What we are doing here is grabbing an Elastic IP address from a pool of IP addresses for the area you are working on. There is a limit of five Elastic IP addresses per account. If you think this is not enough for you, you will need to get more creative with your networking infrastructure. You can also investigate creating sub-accounts for the main account. As with clean code, there are benefits to having clean accounts that only work on one project at a time. This will help you keep track of costs, and shutting down all infrastructure for a project will be clean, as you can be sure that everything for that project has been cleared by deleting the account.

Moving on to our allocation of the Elastic IP to our EC2 server, we can allocate it by highlighting the desired Elastic IP address in the Elastic IP dashboard and clicking on the **Actions** button at the top right of the dashboard, as seen here:

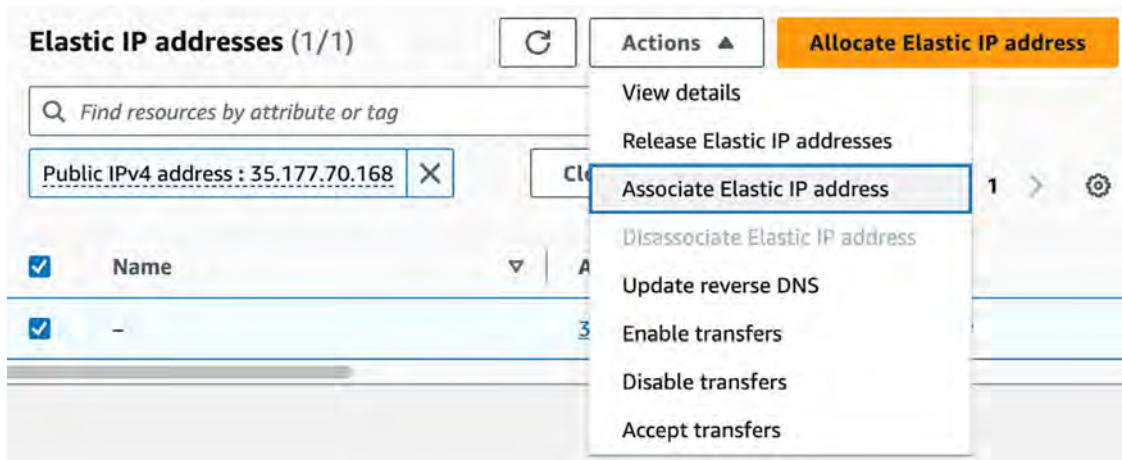


Figure 17.12 – Actions on our Elastic IP

Under **Actions**, we must click on the **Associate Elastic IP address** option, giving us the following display:

[EC2](#) > [Elastic IP addresses](#) > **Associate Elastic IP address**

Associate Elastic IP address Info

Choose the instance or network interface to associate to this Elastic IP address (35.177.70.168)

Elastic IP address: 35.177.70.168

Resource type
Choose the type of resource with which to associate the Elastic IP address.

☒ Instance

☐ Network interface

 If you associate an Elastic IP address with an instance that already has an Elastic IP address associated, the previously associated Elastic IP address will be disassociated, but the address will still be allocated to your account. [Learn more](#)

If no private IP address is specified, the Elastic IP address will be associated with the primary private IP address.

Instance



i-0dd491961aac55183 (to-do build server) - running

The private IP address with which to associate the Elastic IP address.

Reassociation
Specify whether the Elastic IP address can be reassociated with a different resource if it already associated with a resource.

☐ Allow this Elastic IP address to be reassociated

Cancel Associate

Figure 17.13 – Associating an Elastic IP address

The **Instance** option will provide a drop-down menu of the EC2 instances that we have running. Luckily, we have the helpful **to-do build server** tag that we defined in Terraform. However, if we do not have a tag, we can still choose an instance from the drop-down menu. We can then click the **Associate** button. Once this is done, we can go to our Elastic IP address in our browser, and we should be able to access our to-do application, as seen here:

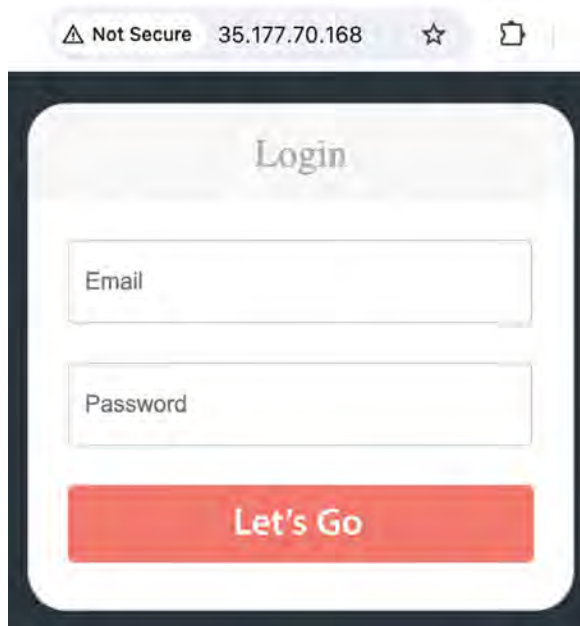


Figure 17.14 – Accessing our to-do application using a static IP

And here we have it—our application can be accessed with an Elastic IP! We can now spin up a new server if we want, test it, and redirect our Elastic IP to the new server if we are happy, providing seamless updates without our users knowing. However, getting users to type in the raw IP address is not desirable. In the next subsection, we will register a domain and connect it to our Elastic IP address.

Registering a domain name

When it comes to registering a domain, it can all be handled in AWS with the Route 53 service. First, we navigate to Route 53, which is the service that handles routing and URL registration. On the left side of the **Route 53** dashboard web page, we can click on the **Registered domains** section, as seen in the following screenshot:

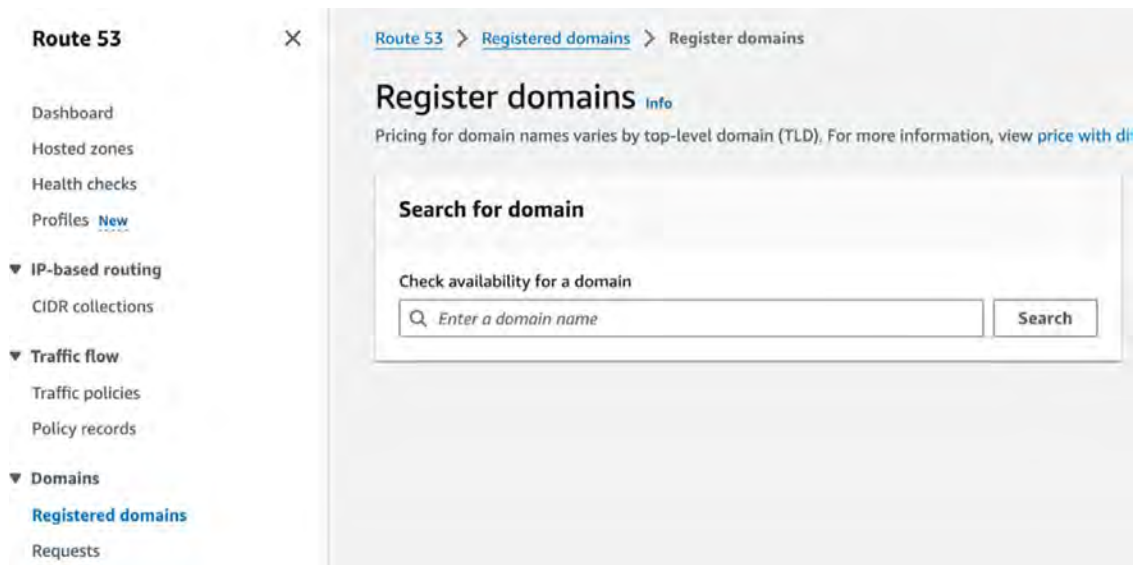


Figure 17.15 – Registered domains view

If you click on the **Register Domain** button at the bottom of the screen, you will be led through a straightforward series of forms to register your domain. The forms ask you for the domain you want to register. It will then tell you whether the domain and other similar domains are available. Domains cost about \$14 a year on average at the time of writing this book. Once you have selected your domain and clicked on the checkout, you will be taken through a series of personal information forms. These forms include a contact address and whether the domain is for personal use or for a company. It is recommended that you select the validation through DNS because this is automated. It is also recommended that you ensure that the WHOIS lookup is enabled for your privacy. Registering a domain means that people can do public lookups on your address against the domain. The WHOIS protection ensures that your details will not be visible and that they can contact you through a proxy managed by AWS. If you do anything illegal, however, authorities may be able to collaborate with AWS to access such details that are registered with the domain that is performing illegal activities, depending on the laws of where you live.

Once your domain is registered, you can go to the **Hosted zones** section in the main Route 53 dashboard. Here, we will see a list of hosted zones for each URL that you own. A **hosted zone** is essentially a collection of DNS records. If we click on a hosted zone for a URL, there will be two DNS records: NS and SOA (NS=**n**ame **s**erver; SOA=**s**tart of **a**uthority). These records should not be deleted, and if they are, somebody else who knows what they are could hijack your URL by implementing those records themselves. DNS records are essentially records on how to route traffic for a domain name. Each DNS record has the following attributes:

- **Domain/subdomain name:** The name of the URL the record belongs to
- **Record type:** The type of record (A, AAAA, CNAME, or NS)
- **Value:** Target IP address
- **Routing policy:** How Route 53 responds to queries
- **TTL:** The amount of time the record is cached in the DNS resolvers in the client, so that we do not have to query the Route 53 servers too often, with the trade-off of reducing traffic to DNS servers versus time for updates to roll onto clients

The attributes just defined are self-explanatory, apart from the record type. There are advanced record types that we can build in Route 53. However, the following record types are needed for us to route our domain to our IP:

- **A:** The simplest record type. Type A merely routes traffic from the URL to an IPv4 IP address.
- **AAAA:** This routes traffic from the URL to an IPv6 address.
- **CNAME:** This maps a hostname to another hostname (target must be an A or AAAA record type).
- **NS:** The name server for the hosted zone (controls how the traffic is routed).

For us, we are going to create a DNS A record. We do this by accessing **Hosted zones**, which will look like the following:

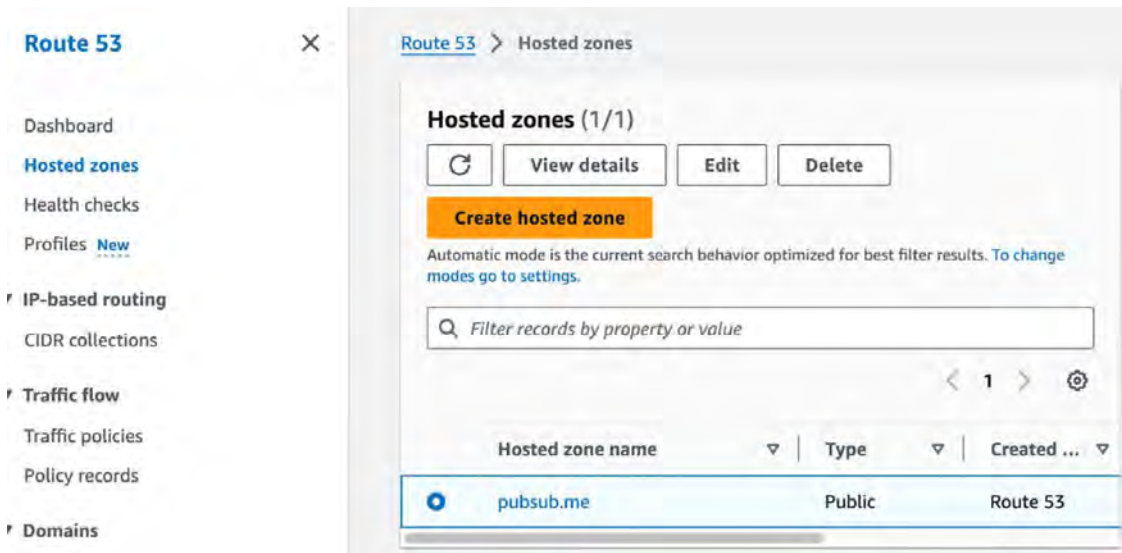


Figure 17.16 – Hosted zones dashboard

By clicking on the URL that we have bought (in Figure 17.16, this is **pubsub.me**), we can create our DNS record by clicking on the **Create record** button in the following screenshot:

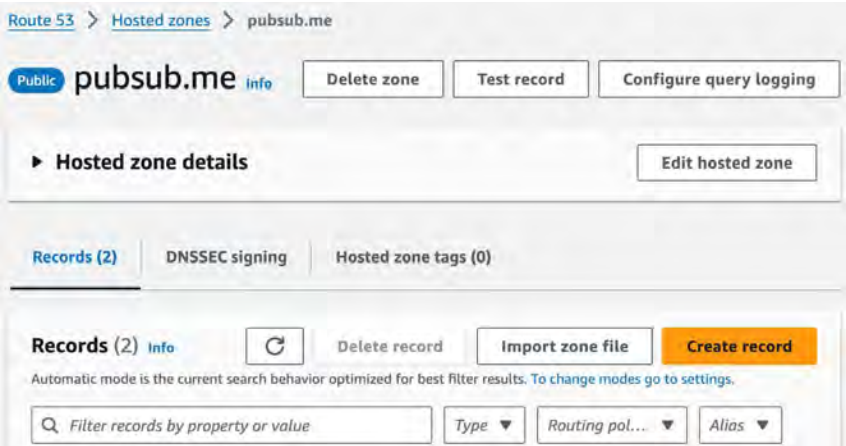


Figure 17.17 – Creating a DNS record

Once we have clicked on this, we get the following layout:

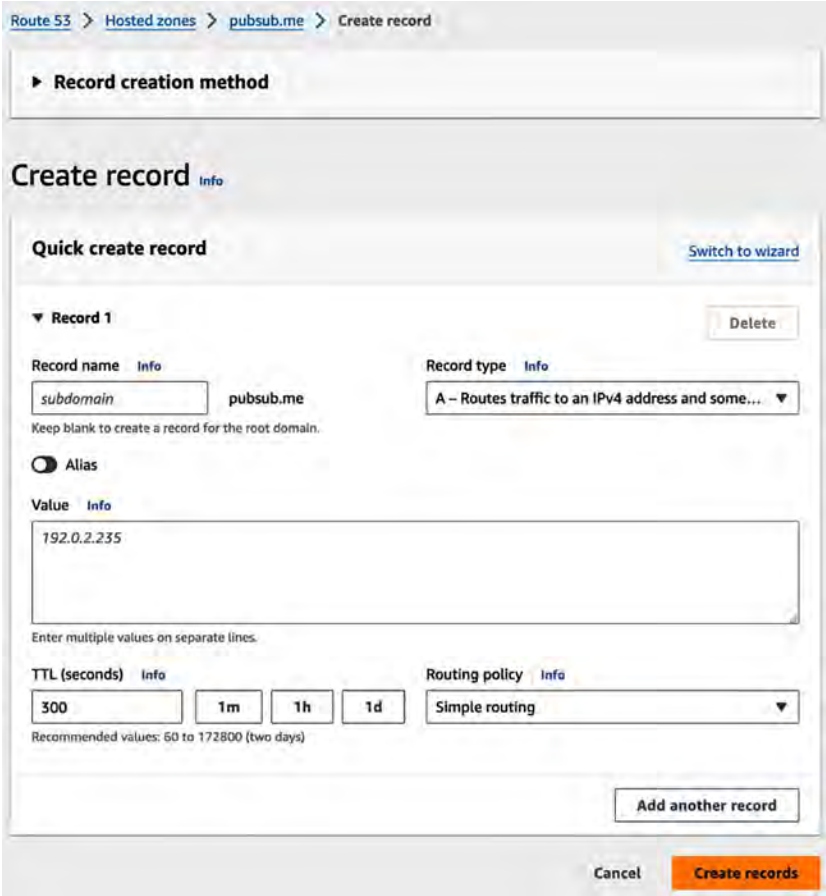


Figure 17.18 – Creating a DNS record

We can see that record type A is the default. We can also see that we can add a subdomain. This gives us some flexibility. For instance, if we wanted, the `api.pubsub.me` URL could point to a different IP address from `pubsub.me`. For now, we are going to just leave the subdomain empty. We are then going to put the Elastic IP address that we set up in the previous section in the **Value** section and click **Create records**. We are then going to create the exact same DNS record, but with the subdomain of `www`. Once this is done, we should have two A records. We can then check where our URL is mapping the traffic using the `www.digwebinterface.com` website. Here, we can enter URLs, and the website will tell us where the URLs are being mapped. We can see here that our URLs are both mapping to the correct Elastic IP:

The screenshot shows the Dig web interface with the following sections:

- Hostnames or IP addresses:** A text box containing `pubsub.com` and `www.pubsub.com`.
- Type:** A dropdown menu set to `Unspecified`.
- Options:** A list of checkboxes including `Show command`, `Colorize output`, `Stats`, `Request NS ID`, `Trace`, `Sort alphabetically`, `Short`, `No recursive`, `Only first nameserver`, `Compare output`, `Save to file`, `Show IP geolocation`, and `DNSSEC`.
- Nameservers:** A section with a `Resolver:` dropdown set to `Default`, and radio buttons for `All`, `Authoritative`, `NIC`, and `Specify myself:`.
- Buttons:** `Dig`, `Fix`, and `Reset form`.

The results section shows two queries:

pubsub.com@9.9.9.10 (Default):

pubsub.com.	3600	IN	A	199.59.243.224
-------------	------	----	---	----------------

www.pubsub.com@9.9.9.10 (Default):

www.pubsub.com.	3600	IN	CNAME	pubsub.com.
pubsub.com.	3530	IN	A	199.59.243.224

Figure 17.19 – Checking a DNS record

With the mapping result confirmed, as seen in the preceding screenshot, we can visit our URL and expect to see the following:



Figure 17.20 – Accessing our application through the registered URL

We can see that our URL is now working. However, the connection is not secure. In the next section, we will enforce an HTTPS protocol for our application and lock it down, as right now, even though we can access our application through the URL, there is nothing stopping us from accessing the IP of the server directly.

Enforcing HTTPS on our application on AWS

Right now, our application kind of works, but it is a nightmare in terms of security. By the end of this section, we will not have the most secure application, as further reading of a networking and DevOps textbook is suggested to achieve gold-standard security. However, we will have configured security groups, locked down our EC2 instances so that they cannot be directly accessed by outsiders, and enforced encrypted traffic through a load balancer that will then direct traffic to our EC2 instances. The result of our efforts will be the following system:

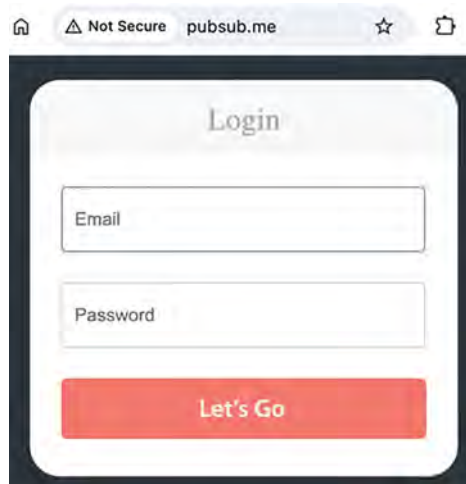


Figure 17.21 – Not secure login form

To achieve the system shown in *Figure 17.21*, we need to carry out the following steps:

1. Getting certificates approved for our URL and variations.
2. Creating multiple EC2 instances to distribute traffic and ensure that the service survives outages.
3. Creating a load balancer to handle incoming traffic.
4. Creating security groups.
5. Updating our Python build script to support multiple EC2 instances.
6. Attaching our URL to the load balancer using the Route 53 wizard.

In the previous section on attaching a URL to our application on AWS, we did a lot of pointing and clicking. As said in the previous chapter, pointing and clicking should be avoided if possible, as it is not repeatable, and we as humans forget what we did. Sadly, with URL approvals, pointing and clicking is the best option. In this section, only the first and sixth steps will require pointing and clicking. The rest will be achieved with Terraform. We are going to be making some big changes to our Terraform configuration, so it is advised that you run a `terraform destroy` command before altering it. Before we can do any of the coding, however, we are going to have to get our certificates for our URL.

Getting certificates for our URL

Because we brought our URL through Route 53, which is handled by AWS, and our servers are running in AWS, the certification and implementation of those certificates is a straightforward process. We need to navigate to **Certificate Manager** by typing `certificate manager` into the services search bar and clicking on it. Once there, we are displayed with a page that only has one orange button, labeled **Request a certificate**. Click on this, and we will be transported to the following page:

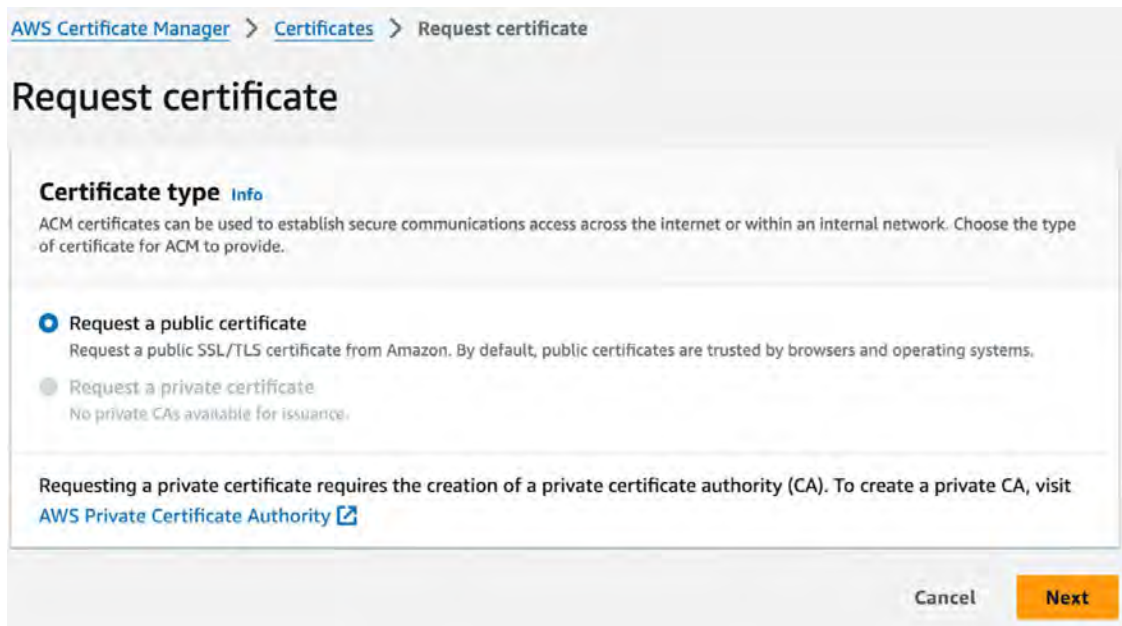


Figure 17.22 – Certificate Manager: start of journey

We are going to want our certificate to be public-facing; therefore, we are happy with the default selection, and we click **Next**. We are then presented with the following form:

[AWS Certificate Manager](#) > [Certificates](#) > [Request certificate](#) > Request public certificate

Request public certificate

Domain names
Provide one or more domain names for your certificate.

Fully qualified domain name [Info](#)

[Add another name to this certificate](#)
You can add additional names to this certificate. For example, if you're requesting a certificate for "www.example.com", you might want to add the name "example.com" so that customers can reach your site by either name.

Validation method [Info](#)
Select a method for validating domain ownership.

☒ **DNS validation - recommended**
Choose this option if you are authorized to modify the DNS configuration for the domains in your certificate request.

☐ **Email validation**
Choose this option if you do not have permission or cannot obtain permission to modify the DNS configuration for the domains in your certificate request.

Key algorithm [Info](#)
Select an encryption algorithm. Some algorithms may not be supported by all AWS services.

☒ **RSA 2048**
RSA is the most widely used key type.

☐ **ECDSA P 256**
Equivalent in cryptographic strength to RSA 3072.

☐ **ECDSA P 384**
Equivalent in cryptographic strength to RSA 7680.

Tags [Info](#)
No tags associated with the resource.

[Add new tag](#)
You can add up to 50 tags.

[Cancel](#) [Previous](#) [Request](#)

Figure 17.23 – Defining certificate request

Here, we type in the URLs we want to associate with the certificate. In *Figure 17.23*, we have stated that we want a certificate for the URL `pubsub.me`. We will also want a wildcard certificate, so you must repeat this process for the `*.pubsub.me` URL. We can also see that DNS validation is already highlighted, and this is recommended as we have our servers on AWS, meaning we will have to take no more action to get the certificate issued. We can then click on the button labeled **Request**, and we will be redirected to a page with the list of certificates. I find that the new certificate request is not present nearly every time I do this. My guess is that there is a delay. Do not worry—merely refresh the page, and you will see the pending certificate request listed. Click on this listing, and you will be directed to a detailed view of this certificate request. In the middle of the screen on the right, you need to click the **Create records in Route 53** button, as seen here:



Figure 17.24 – Creating records for DNS confirmation

Follow the prompts after clicking the button, and the CNAME record will be created. If you do not do this, then the pending status for the ticket will go on indefinitely because the cloud provider needs the route to issue the certificate, considering we selected DNS validation. The created records should look like the following screenshot:

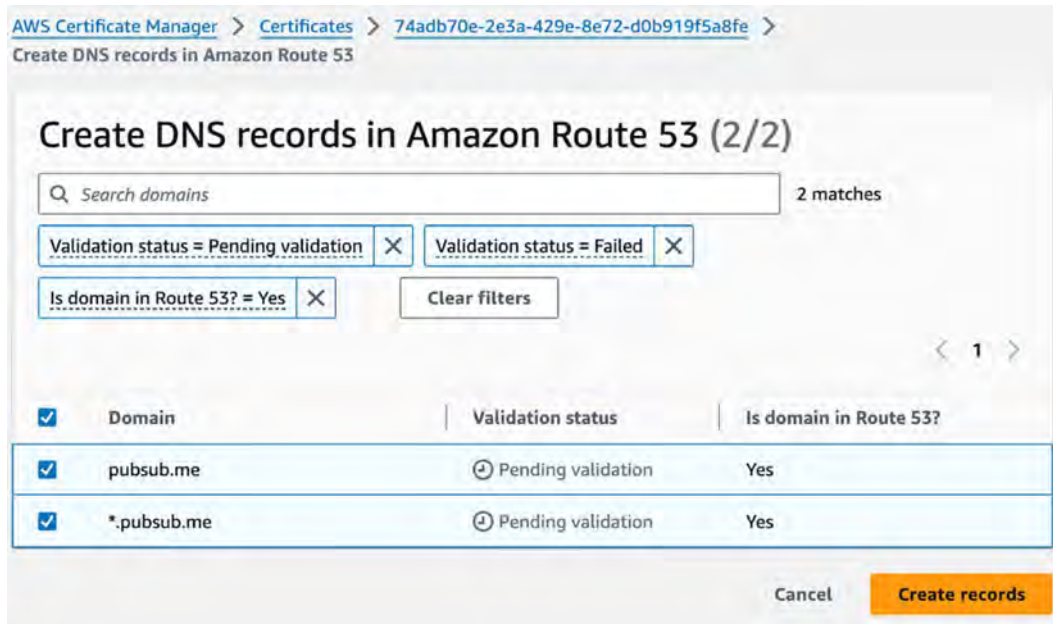
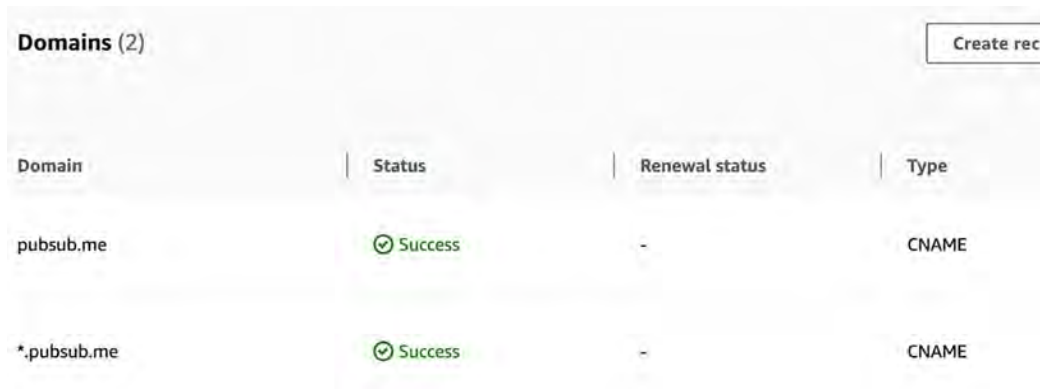


Figure 17.25 – Creating the records options

After a few minutes, the certificate should be issued. Once this is done, carry out the same steps for the prefix wildcard. After that, your certificate lists should look something like the following:



Domain	Status	Renewal status	Type
pubsub.me	Success	-	CNAME
*.pubsub.me	Success	-	CNAME

Figure 17.26 – Creating records for DNS confirmation

In the preceding screenshot, we can see that I have two certificates: one for if a user directly types in the URL with no prefix, and the wildcard that covers all prefixes. We are ready to use these certificates, but before we do this, we are going to have to carry out some other steps. Before we define rules around the traffic, we are going to have to build the infrastructure where the traffic is going. In the next subsection, we will prep our EC2 instances.

Prepping our EC2 instances

We will be using a load balancer, and the traffic of that load balancer will be directed to our EC2 instance. Our load balancer can also distribute across multiple EC2 instances that are the same. This means that if one EC2 instance is down, we can still use the other EC2 instance. We can also scale our application. For our example, we will keep our EC2 instance count to one, as we still have our cache for the auth sessions housed on the EC2 instance. If we were to increase our EC2 instances without housing our cache on a separate EC2 instance, then we would get inconsistent reads from our user session cache. Setting up another EC2 instance, splitting up the docker-compose to house the Redis cache, and then passing the IP of that EC2 instance into the EC2 instances housing our to-do application would bloat this already lengthy chapter. However, we have covered all the concepts for it to be achieved, so we might as well prep the EC2 instance for that to be a possibility. We have to prep the EC2 instance for the load balancer with the following code:

```
# deployment/main.tf
resource "aws_instance" "production_server" {
  ami = "ami-05ea2888c91c97ca7"
  instance_type = "t2.medium"
  count = 1
  key_name = "remotebuild"
  user_data = file("server_build.sh")
  security_groups = [
```

```

    "${aws_security_group.webserver-security-group.name}"
  ]
  tags = {
    Name = "to-do prod ${count.index}"
  }
}

```

Here, we can see that we have added a count parameter and defined it as 1. If we increase the count, we will increase the number of EC2 instances built. We have also altered the tag. We can also see that we access the number of EC2 instances being created with `index`. The index starts at zero and increases by one every time a resource is made. Finally, we can also see that we have added a `security_groups` section. Right now, we have not made the security group that we are referencing, but we will in the section on security groups. We are referencing security groups to enforce rules of incoming traffic. It also must be noted that we are using `security_groups` because we are using the default VPC. If we are not using the default VPC, we must use `security_group_ids`. Now that we have the option for multiple instances, we must update the outputs at the bottom of the `deployment/main.tf` file with the following code:

```

# deployment/main.tf
output "ec2_global_ips" {
  value = ["${aws_instance.production_server.*.public_ip}"]
}
output "db_endpoint" {
  value = "${aws_db_instance.main_db.*.endpoint}"
}
output "public_dns" {
  value = ["${aws_instance.production_server.*.public_dns}"]
}
output "instance_id" {
  value = ["${aws_instance.production_server.*.id}"]
}

```

Here, we can see that apart from the database endpoint, all the other outputs have been changed to lists. This is because they all reference the potential for multiple EC2 instances. Now that we have prepped our EC2 instances, we can route traffic to our instances with a load balancer.

Creating a load balancer for our traffic

There is a range of different load balancers that we can pick from. We have already discussed NGINX, which is a popular load balancer. For this chapter, we are going to use the application load balancer to route the traffic to our EC2 instance and implement the HTTPS protocol. There are multiple features that load balancers can offer, and they protect against **distributed denial-of-service (DDoS)** attacks, whereby the attacker will try to overload the server with excessive requests. We will create our load balancer in the `deployment/load_balancer.tf` file. First, we collect the data that we need with the following code:

```
# deployment/load_balancer.tf
data "aws_subnets" "subnets" {
  filter {
    name     = "vpc-id"
    values   = [aws_default_vpc.default.id]
  }
}

data "aws_acm_certificate" "issued_certificate" {
  domain     = "pubsub.me"
  statuses   = ["ISSUED"]
}

data "aws_acm_certificate" "raw_cert" {
  domain     = "*.pubsub.me"
  statuses   = ["ISSUED"]
}
```

We can see that instead of a resource, we used the data declaration. This is where we make queries to AWS for specific types of data to be used in the rest of the Terraform script. We get the **virtual private cloud (VPC)** ID. In Terraform, we can define and build a VPC, but throughout this book, we have been using the default VPC. We can get the default VPC ID for our load balancer. We then get the data for the certificates that we have defined in the previous section with the preceding code. We now must define a target for our load balancer. This is done in the form of a target group where we can bunch a group of instances together for the load balancer to target with the following code:

```
# deployment/load_balancer.tf
resource "aws_lb_target_group" "target-group" {
  health_check {
    interval = 10
    path     = "/"
    protocol = "HTTP"
    timeout  = 5
    healthy_threshold = 5
    unhealthy_threshold = 2
  }
  name = "ToDoAppLbTg"
  port = 80
  protocol = "HTTP"
  target_type = "instance"
  vpc_id = aws_default_vpc.default.id
}
```


Here, we can see that we define the parameters for a health check. These parameters are self-explanatory. The health check will alert a service targeting the health status of the target group. We would not want to route traffic to a target group that is down. We then define the protocol and port of the traffic, the type of resource in the target group, and the ID of the VPC. Now that our target group is defined, we can attach our EC2 instance to it with the following code:

```
# deployment/load_balancer.tf
resource "aws_lb_target_group_attachment" "ec2_attach" {
  count = length(aws_instance.production_server)
  target_group_arn = aws_lb_target_group.target-group.arn
  target_id = aws_instance.production_server[count.index].id
}
```

We can see that we get the IDs of the EC2 servers for the target ID. With this, our EC2 instances can be targeted by the load balancer. Now that we have a target, we can create our load balancer with the following code:

```
# deployment/load_balancer.tf
resource "aws_lb" "application-lb" {
  name = "ToDoApplicationLb"
  internal = false
  ip_address_type = "ipv4"
  load_balancer_type = "application"
  security_groups = [
    "${aws_security_group.alb-security-group.id}"
  ]
  subnets = data.aws_subnets.subnets.ids
  tags = {
    name = "todo load balancer"
  }
}
```

We can see that the parameters in the load balancer definition are straightforward. However, you may have noticed the security group definition. We are referencing a security group even though we have not defined any security groups. If you do not know what a security group is, do not worry—we will cover and build all the security groups we need in the next section. Before we do that, however, we might as well define listening and routing rules for the load balancer. First, we can define the HTTP listener for port 80. If you remember the first section of this chapter, when getting HTTPS working on our localhost, what do you think we need to do for the HTTP traffic? You don't have to know the specific Terraform code, but what is the general behavior that we want to facilitate? With this in mind, we can achieve that behavior with the following code:


```
# deployment/load_balancer.tf
resource "aws_lb_listener" "http-listener" {
  load_balancer_arn = aws_lb.application-lb.arn
  port = 80
  protocol = "HTTP"
  default_action {
    type = "redirect"
    redirect {
      port      = "443"
      protocol  = "HTTPS"
      status_code = "HTTP_301"
    }
  }
}
```

That's right! We receive HTTP traffic from port 80, and we then redirect it to port 443 with the HTTPS protocol. We can see that we have attached this listener using the **Amazon Resource Name (ARN)** of the load balancer that we created. We can now define our HTTPS listener with the following code:

```
# deployment/load_balancer.tf
resource "aws_lb_listener" "https-listener" {
  load_balancer_arn = aws_lb.application-lb.arn
  port = 443
  protocol = "HTTPS"
  certificate_arn = data.aws_acm_certificate
                  .issued_certificate.arn

  default_action {
    target_group_arn = aws_lb_target_group.target-group.arn
    type = "forward"
  }
}
```

Here, we can see that we accept HTTPS traffic and then forward HTTP traffic to our target group that we defined using the ARN of the target group. We can also see that we have attached one of the certificates to the listener. However, this does not cover all our URL combinations. Remember, we have another certificate that we want to attach. We can attach our second certificate with the following code:

```
# deployment/load_balancer.tf
resource "aws_lb_listener_certificate" "extra_certificate" {
  listener_arn    = "${aws_lb_listener.https-listener.arn}"
  certificate_arn = "${data.aws_acm_certificate.raw_cert.arn}"
}
```

This connection should be easy to understand. Here, we merely reference the ARN of the HTTPS listener and the ARN of the certificate that we want to attach. We have now defined everything we need for the resources of the load balancer. However, what of the traffic? We have defined the load balancer, EC2 instances, and the routing for HTTPS for the load balancer. But what is stopping someone from just directly connecting to the EC2 instance, bypassing the load balancer and HTTPS completely? This is where security groups come in. In the next section, we will lock down the traffic by creating security groups so that users cannot bypass our load balancer.

Creating security groups to lock down and secure traffic

Security groups are essentially firewalls. We can define the traffic to and from a resource that has a security group implemented. The rules of the traffic can be fine-grained. A single security group can have multiple rules defining the origin (even specific IP addresses if needed) and the protocol of the traffic. When it comes to our security groups, we are going to need two. One will accept HTTP and HTTPS traffic from all IPs anywhere in the world. This is going to be for our load balancer because we want our application to be available to everyone. The other security group will be implemented by our EC2 instances; this blocks all HTTP traffic apart from the first security group. We will also enable SSH inbound traffic because we need to SSH into the servers to deploy the applications, giving us the following traffic layout:

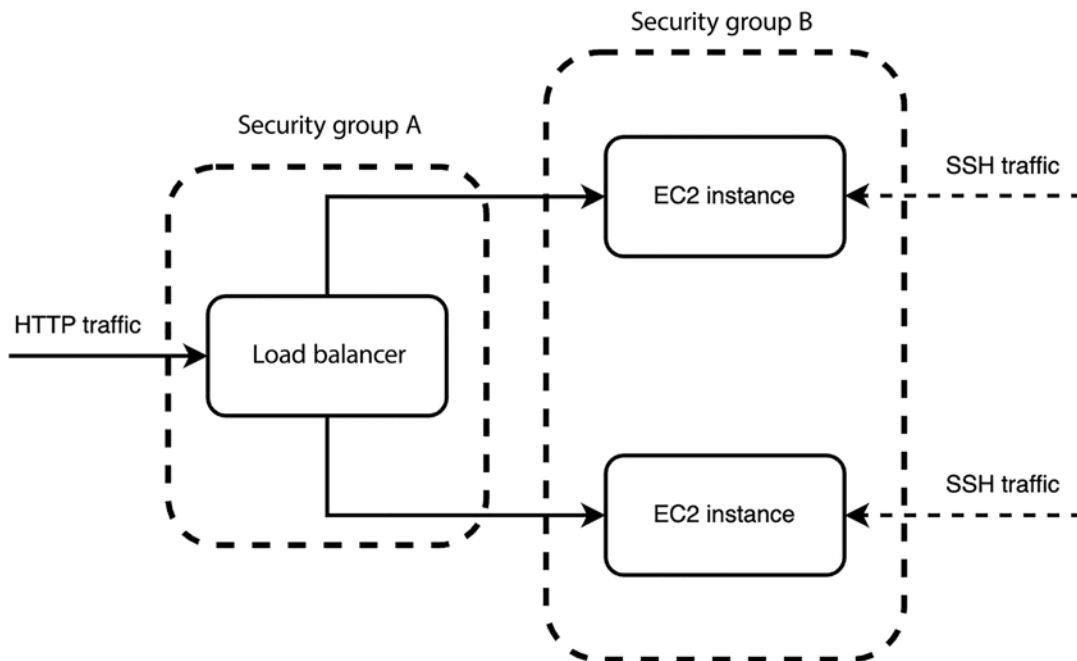


Figure 17.27 – Security groups for our deployment system

This is where you must be careful with online tutorials. There is no shortage of YouTube videos and Medium articles that will get a load balancer up and running with some pointing and clicking. However, they leave the EC2 instances exposed and do not bother exploring security groups. Even with this section, we are going to leave the database exposed. I am doing this because it is a good question to be asked in the *Questions* section. However, I'm highlighting it here because you need to be warned that it is exposed. The way to lock down the database will be covered in the *Answers* section of this chapter. When it comes to our security groups, we can define them in the `deployment/security_groups.tf` file. We can start with the load balancer security group with the following code:

```
# deployment/security_groups.tf
resource "aws_security_group" "alb-security-group" {
    name = "to-do-LB"
    description = "the security group for the
                  application load balancer"

    ingress {
        . . .
    }
    ingress {
        . . .
    }
    egress {
        . . .
    }
    tags = {
        name: "to-do-alb-sg"
    }
}
```

Here, we have two inbound rules under the `ingress` tag and one outbound rule under the `egress` tag. Our first inbound rule is to allow HTTP data from anywhere with the following code:

```
# deployment/security_groups.tf
ingress {
    description = "http access"
    from_port = 80
    to_port = 80
    protocol = "tcp"
    cidr_blocks = ["0.0.0.0/0"]
}
```

The `cidr` blocks of all zeros mean from anywhere. Our second inbound rule is HTTPS traffic from anywhere. How do you think this will be defined? It can be defined with the following code:

```
# deployment/security_groups.tf
ingress {
    description = "https access"
    from_port = 443
    to_port = 443
    protocol = "tcp"
    cidr_blocks = ["0.0.0.0/0"]
}
```

Now for our outbound rule. We must allow all traffic and protocols out of the load balancer as they are coming from our resources. This can be achieved with the following code:

```
# deployment/security_groups.tf
egress {
    from_port = 0
    to_port = 0
    protocol = "-1"
    cidr_blocks = ["0.0.0.0/0"]
}
```

It must be noted that `from_port` and `to_port` are 0, meaning that we allow outgoing traffic from all ports. We also have set `protocol` to -1, meaning we are allowing all protocols as outgoing traffic. We have now defined the security group for the load balancer, so we can move on to defining our security group for the EC2 instances with the following code:

```
# deployment/security_groups.tf
resource "aws_security_group" "webserver-security-group" {
    name = "to-do-App"
    description = "the security group for the web server"

    ingress {
        . . .
    }
    ingress {
        . . .
    }
    egress {
        from_port = 0
        to_port = 0
        protocol = "-1"
        cidr_blocks = ["0.0.0.0/0"]
    }
}
```

```
    }
    tags = {
      name: "to-do-webserver-sg"
    }
  }
}
```

The outbound rules are going to be the same as the load balancer because we want to return data to anywhere that is available to request it. When it comes to our HTTP inbound rules, we only want to accept traffic from the load balancer with the following code:

```
# deployment/security_groups.tf
ingress {
  description = "http access"
  from_port = 80
  to_port = 80
  protocol = "tcp"
  security_groups = [
    "${aws_security_group.alb-security-group.id}"
  ]
}
```

Here, we can see that instead of defining the `cidr` blocks, we rely on the security group of the load balancer. Now that all the user traffic is defined, we only need to define the SSH traffic for deployment with the following code:

```
# deployment/security_groups.tf
ingress {
  description = "SSH access"
  from_port = 22
  to_port = 22
  protocol = "tcp"
  cidr_blocks = ["0.0.0.0/0"]
}
```

Now for our outbound rule. We must allow all traffic and protocols out of the load balancer, as they are coming from our resources. This can be achieved with the following code:

```
# deployment/security_groups.tf
egress {
  from_port = 0
  to_port = 0
  protocol = "-1"
  cidr_blocks = ["0.0.0.0/0"]
}
```

It must be noted that `from_port` and `to_port` are 0, meaning that we allow outgoing traffic from all ports. We also have set `protocol` to `-1`, meaning we are allowing all protocols as outgoing traffic. We have now defined the security group for the load balancer. We can now move on to defining our security group for the EC2 instances with the following code:

```
# deployment/security_groups.tf
resource "aws_security_group" "webserver-security-group" {
  name = "to-do-App"
  description = "the security group for the web server"
  ingress {
    . . .
  }
  ingress {
    . . .
  }
  egress {
    from_port = 0
    to_port = 0
    protocol = "-1"
    cidr_blocks = ["0.0.0.0/0"]
  }
  tags = {
    name: "to-do-webserver-sg"
  }
}
```

The outbound rules are going to be the same as the load balancer because we want to return data to anywhere that is available to request it. When it comes to our HTTP inbound rules, we only want to accept traffic from the load balancer with the following code:

```
# deployment/security_groups.tf
ingress {
  description = "http access"
  from_port = 80
  to_port = 80
  protocol = "tcp"
  security_groups = ["${aws_security_group.
    alb-security-group.id}"]
}
```

Here, we can see that instead of defining the `cidr` blocks, we rely on the security group of the load balancer. Now that all the user traffic is defined, we only need to define the SSH traffic for deployment with the following code:

```
# deployment/security_groups.tf
ingress {
    description = "SSH access"
    from_port = 22
    to_port = 22
    protocol = "tcp"
    cidr_blocks = ["0.0.0.0/0"]
}
```

Here, we access SSH through port 22. This will enable us to SSH into our servers and deploy our applications. Nearly everything is done. We only need to attach our EC2 instances to our EC2 security group, which can be done with the following code:

```
# deployment/security_groups.tf
resource "aws_network_interface_sg_attachment"
"sg_attachment_webserver" {
    count = length(aws_instance.production_server)
    security_group_id = aws_security_group.
        webserver-security-group.id
    network_interface_id = aws_instance.
        production_server[count.index].
        primary_network_interface_id
}
```

Our Terraform scripts are now complete and will be able to spin up multiple EC2 instances, a database, and a load balancer while locking down traffic. This is a good template for other projects if you want to get a basic web app with HTTPS and access to a database off the ground with restricted traffic.

Now that we have the option to support multiple EC2 instances, we are going to have to change our deployment script.

Updating our deployment script

As our output now returns a list of EC2 IP addresses, our deployment script must be able to handle multiple IP addresses, copy the Docker Compose files over to all the IP addresses, and run the application on all the EC2 instances. This is achieved with the following code:

```
# deployment/run_deployment.sh
. . .
OUTPUT_FILE="output.json"
IP_ADDRESS=$(jq --raw-output '.ec2_global_ips.value[0][0]' \
$OUTPUT_FILE)
IP_ADDRESSES=("$IP_ADDRESS")
echo "All IP Addresses: ${IP_ADDRESSES[@]}"
. . .
```

```
for IP in "${IP_ADDRESSES[@]}"
do
    echo "Processing IP: $IP"
    . . .
done
```

Here, we can see that we create a list of IP addresses and assign them to the `IP_ADDRESSES` variable. We then loop through our list of IP addresses, copying over the Docker Compose files and running the applications on these servers. In this loop for the deployment and running of the application, we substitute `$IP_ADDRESS` for `$IP` as this is the variable in the loop.

If our Terraform output gave us two IP addresses, we could construct our list with the following code:

```
IP_ADDRESS=$(jq --raw-output '.ec2_global_ips.value[0][0]' \
$OUTPUT_FILE)
IP_ADDRESS_TWO=$(jq --raw-output '.ec2_global_ips.value[0][1]' \
$OUTPUT_FILE)
IP_ADDRESSES=("$IP_ADDRESS", "$IP_ADDRESS_TWO")
```

Now, all our code infrastructure is done! We can run this Bash script and deploy our build onto AWS. Everything is nearly done; all we must do is connect our URL to our load balancer in the next subsection.

Attaching our URL to the load balancer

This is the home run. We are finally near the end. However, this is important if you want to use your Rust server. To connect our URL to the load balancer, we must navigate to the hosted zone for your URL. Once there, click on the **Create record** button. When in the **Create record** display, if you are not using the wizard, click on the **Switch to wizard** link at the top right of the **Create record** display to get the following wizard view:

Choose routing policy Info

The routing policy determines how Amazon Route 53 responds to queries.

Routing policy

[Switch to quick create](#)

☒ **Simple routing**

Use if you want all of your clients to receive the same response(s).



☐ **Weighted**

Use when you have multiple resources that do the same job, and you want to specify the proportion of traffic that goes to each resource. For example: two or more EC2 instances.



☐ **Geolocation**

Use when you want to route traffic based on the location of your users.



☐ **Latency**

Use when you have resources in multiple AWS Regions and you want to route traffic to the Region that provides the best latency.



☐ **Failover**

Use to route traffic to a resource when the resource is healthy, or to a different resource when the first resource is unhealthy.



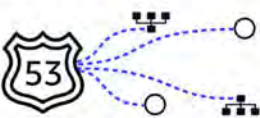
☐ **Multivalue answer**

Use when you want Route 53 to respond to DNS queries with up to eight healthy records selected at random.



☐ **IP-based**

Use to route traffic to locations of IP address ranges in CIDR notation.



☐ **Geoproximity**

Use when you want Route 53 to route DNS queries to a certain geographic location.



Cancel

Next

Figure 17.28 – Create record wizard view

Here, we can see a range of fancy ways to route traffic. However, we are going to select **Simple routing** as we just need to pass traffic to the load balancer, which is doing the distribution of traffic between EC2 instances. Selecting **Simple routing** gives us the following form to fill in:

Define simple record ✕

Record name [Info](#)

To route traffic to a subdomain, enter the subdomain name. For example, to route traffic to `blog.example.com`, enter `blog`. If you leave this field blank, the default record name is the name of the domain.

Keep blank to create a record for the root domain.

Record type [Info](#)

The DNS type of the record determines the format of the value that Route 53 returns in response to DNS queries.

A – Routes traffic to an IPv4 address and some AWS resources ▼

Choose when routing traffic to AWS resources for EC2, API Gateway, Amazon VPC, CloudFront, Elastic Beanstalk, ELB, or S3. For example: 192.0.2.44.

Value/Route traffic to [Info](#)

The option that you choose determines how Route 53 responds to DNS queries. For most options, you specify where you want to route internet traffic.

Alias to Application and Classic Load Balancer ▼

Europe (London) ▼

✕

Alias hosted zone ID: ZHURV8PSTC4K8

Evaluate target health

Select Yes if you want Route 53 to use this record to respond to DNS queries only if the specified AWS resource is healthy.

☒ Yes

Cancel Define simple record

Figure 17.29 – Define simple record wizard view

Here, we can see that I have selected **Alias to Application and Classic Load Balancer** to route traffic to. I have selected the location where the load balancer is. This has given me a dropdown where I can see my `ToDoApplicationLb` load balancer to select. When you click the **Define simple record** button, you are then navigated to a list of records that are to be created. We carry out the creation wizard process one more time to account for all prefixes with the wildcard, and then confirm our creation of records. With this, HTTPS now works with our application, as seen here:

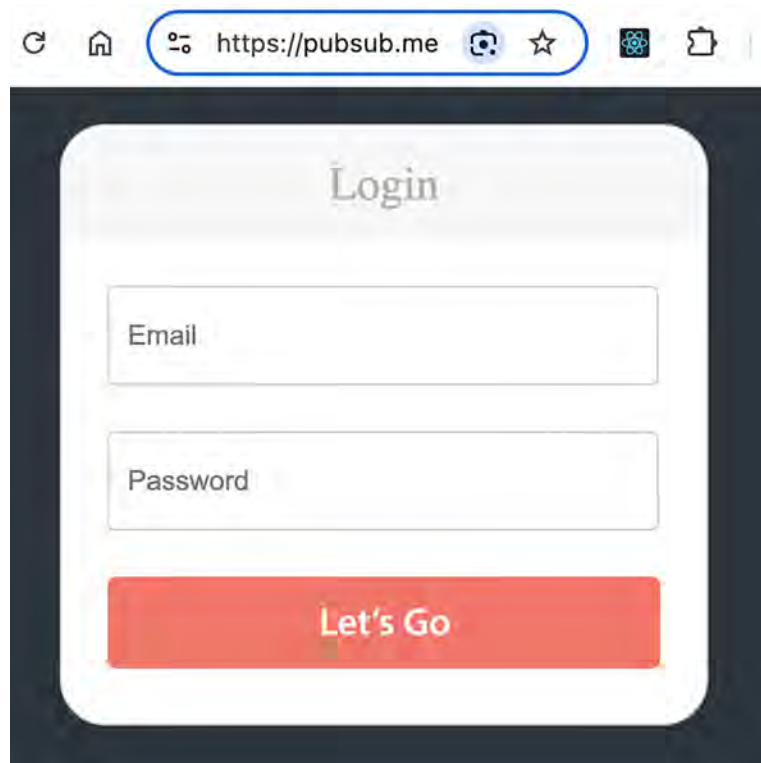


Figure 17.30 – HTTPS working with our application on the internet

To check this, we can perform the following cURL command:

```
curl -X POST https://pubsub.me/api/v1/users/create \
-H "Content-Type: application/json" \
-d '{
  "email": "test@gmail.com",
  "password": "password"
}'
```

With this, our chapter is complete. If you try to access one of our EC2 instances directly through an IP address, you will be blocked. Therefore, we cannot directly access our EC2 instances, but we can access them through HTTPS through our URL. If you give users any variation of your URL, even an HTTP link to the URL, your users will happily use your application with the HTTPS protocol.

Summary

For now, we have done all we need to do to deploy a robust and secure application on AWS with locked-down traffic and HTTPS enforced. We have covered a lot to get here, and the skill set that you have gained in this chapter can be applied to pretty much any other project you want to deploy on AWS if you can package it in Docker. You now understand the advantages of HTTPS and the steps needed to not only achieve the HTTPS protocol but also to map your URL to the IP address of a server or a load balancer. What's more, we automated the attachment of certificates that we created using Certificate Manager to our load balancer in Terraform using the powerful data query resource that Terraform has to offer. Finally, this all came together when we managed to access our application using HTTPS and only HTTPS. Not only have we developed some practical skills that will become useful in many future projects, but we have also explored the nature of how HTTPS and DNS work, giving us a deeper understanding and appreciation of how the internet generally works when we type in a URL to the browser.

Questions

1. Once we remember that our system still has the database exposed to the rest of the world, how would we lock it down?
2. Why should we use an Elastic IP in some cases?
3. How could we automate the association of an existing Elastic IP to an EC2 instance?
4. How do we utilize security groups to lock down traffic between a URL and EC2 instances?
5. Let's say that the traffic for our application greatly increased, and our two instances cannot handle the pressure. What could we do to enable our system to handle the increase?
6. What is the basic premise of a DNS record?
7. What is a URL-hosted zone?

Answers

1. We would create a security group for the database that only accepts traffic to and from the EC2 instance security group. If we were having to make a migration, we could SSH into an EC2 instance and use it as a proxy to connect to a database. You can also do this with database viewing software such as DataGrip. Sometimes, you can have an EC2 instance that is just there for a user to use as a proxy to access the database. This is known as a bastion server.
2. When we destroy and create EC2 instances, the IP address of the EC2 instance will change. We can use an Elastic IP to ensure that the IP address remains consistent, which can be helpful for automation pipelines, as we can continue to point to that IP address. However, if we are using a load balancer, we do not need to use an Elastic IP address, as we will point our URL to the load balancer.
3. We can automate the association of an Elastic IP to an EC2 instance using Terraform and its powerful data resource. This means we get the data of the existing Elastic IP with a data query, and then attach this IP to an EC2 instance that we are creating.
4. Ensure that the security group for the EC2 instances in the target group that the load balancer is attached to can only accept HTTP traffic from the load balancer.

5. Considering that everything in our system is automated and piped into each other, all we must do is increase the count in our EC2 instance definition from 2 to 3 and add the third EC2 to the list in the deployment script. This will increase the number of EC2 instances handling the traffic from 2 to 3.
6. A DNS record is essentially a routing rule that tells us how to route traffic from the URL to a server IP, URL namespace, or AWS service.
7. A URL-hosted zone is a collection of DNS records for a URL.

Further reading

- Fjordvald, M.B. and Nedelcu, C. (2018) *Nginx HTTP Server: Harness the power of Nginx to make the most of your infrastructure and serve pages faster than ever before*. 4th edn. Packt Publishing.
- Sharma, R. (2015) *NGINX High Performance*. Packt Publishing.
- Kholodkov, V. (2015) *Nginx Essentials: Excel in Nginx quickly by learning to use its most essential features in real-life applications*. Packt Publishing.
- Dar, U. (2013) *Nginx Module Extension*. Packt Publishing.
- Aivaliotis, D. (2016) *Mastering NGINX*. 2nd edn. Packt Publishing.
- Fjordvald, M. (2013) *Instant Nginx Starter*. Packt Publishing.
- DeJonghe, D. (2024) *NGINX Cookbook: Advanced Recipes for High-Performance Load Balancing*. 3rd edn. O'Reilly Media.

Get this book's PDF copy, code bundle, and more

Scan the QR code (or go to packtpub.com/unlock). Search for this book by name, confirm the edition, and then follow the steps on the page.



UNLOCK NOW

Note: Have your invoice handy. Purchases made directly from the Packt website don't require an invoice.

18

Practicalities of Using Microservices and Nanoservices

Throughout the book, we have been building our system in isolation. However, in real life, we know that we will not be building systems in isolation without any input from other developers. For most projects, you must coordinate with other developers and integrate with legacy systems. There are no black and white approaches, but trade-offs. In this chapter, we explore the positive and negative effects of microservices in relation to collaborating with other developers and managing a complex system. We will also highlight how nanoservices can help improve these trade-offs. We also cover some coding exercises in managing private code and distributing that code in builds and deployments, and how to directly integrate our Rust nanoservice into another language, such as Python. If you do not ever plan to code in Python, do not worry; the integration technique utilizes C interfaces that enable us to interact with other languages. Most other languages will also be able to interact with our Rust code using the same method. In this chapter, we cover the following:

- When to use nanoservices, microservices, and monoliths
- Managing private repositories with dockpack
- Integrating Rust nanoservices with other languages using C interfaces

By the end of this chapter, you will be able to understand the drawbacks of microservices. However, you will also understand the benefits and how to leverage the benefits of microservices and avoid the drawbacks by using our nanoservice approach. You will also be able to package and distribute private code files using Docker and integrate that Rust code with most other languages. These skills will help you tackle real-world projects, as opposed to simply claiming that you can code a Rust web application in isolation.

Technical requirements

You can download the example project and code for this book by following the instructions in the **Download the example code files** section in the *Preface* of this book.

This chapter's code files are included in the downloadable code bundle.

When to use nanoservices, microservices, and monoliths

If you go online to check whether microservices are a good idea, you will not have to go far to see extremely simplistic takes: microservices are either the future of web development or they are a complete sham and make your system worse. Of course, the truth is somewhere in between; microservices offer benefits and drawbacks.

The negative effects of microservices

Microservices can seem appealing because of their small services that do one thing and are isolated. When I built my first Kubernetes cluster, I was excited by this very image. My coding days would be bliss because I could just focus on this one small service, test the service, and then deploy that service into the bigger system. We have all worked on big, sprawling code bases that were hard to navigate, but microservices also run the risk of providing a terrible developer experience. Let's cover these pitfalls, starting with container orchestration.

Container orchestration

We started our journey with a simple to-do microservice and built an authentication microservice to handle the logging in and creation of users, giving us the layout in *Figure 18.1*:

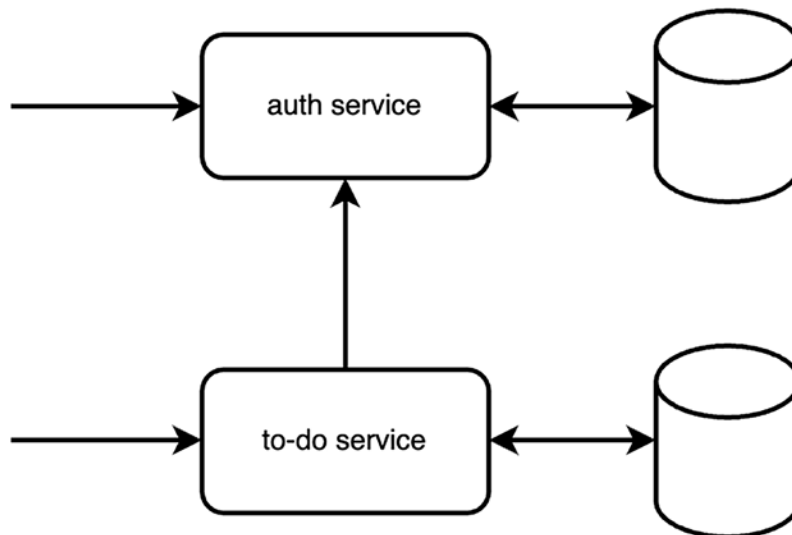


Figure 18.1 – A basic microservice structure

Already, we have violated a rule of microservices. Microservices should be independent of each other. We could separate the microservices by putting all authentication data into a token, with things such as the user role, as well as the ID. However, being able to keep track of the session statistics in real time would require access to a service. Our requirements also evolve over time. What if we need even more data from the user account? It would not be unreasonable for users to be housed under organizations and departments. When a user creates a to-do list for a department, we would have to check that the user has the authority to do this and that the department exists before entering it into the database.

On top of this, we will want emails sent if a user is added/removed from a group/department. If we get a login from a different device, we may also want a warning email, and we will want to use an email with a link and a UUID to prove that the user has access to the email they say they do when registering the account. It is also reasonable to email the user if they close a to-do list. Our structure has ballooned very quickly to what we see in *Figure 18.2*:

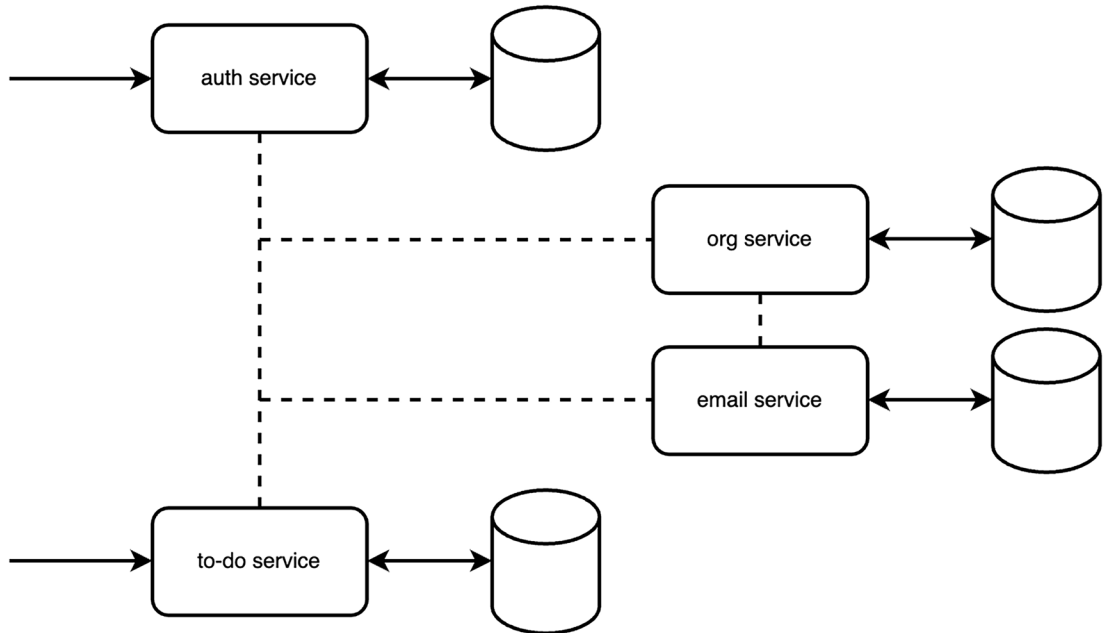


Figure 18.2 – More complex microservice structure

Here, we can see that they are all connected somehow. Maybe I have worked with the wrong people and practiced bad software engineering all my life, but in my experience, I have never seen a production system that solves real-world problems, where they have managed to keep all the microservices truly independent of each other.

Database for the email server?

It may seem strange to have a database for an email server. However, we must log sent emails and define rules around them. Sending emails generally consists of hitting a third-party API such as Mailchimp and sending emails. However, we must have control over how and when we send these emails. A simple example of this is rate-limiting the emails on the register user page. A simple hack is for a malicious actor to repeatedly copy and paste someone's email address into the registration form and click register. They could do this all day, and the victim's email address would be flooded, and would rack up massive bills with the email provider. If we have a register email endpoint in our email service, we could log the email address and the IP of the request. We could then check the email logs before sending an email. If there has been up to three within the hour, we could just not send the email to the email API. If the same IP address has spammed more than 5 emails within an hour, we could also put in cooldown measures.



When it comes to local development, these services will end up, over time, having different dependencies and versions of those dependencies, so we are probably going to need to run these services in Docker. In theory, just pulling these services from Docker Hub and running them using `docker-compose` seems straightforward. However, this is unlikely to happen. We can demonstrate this by adding departments and teams to to-do lists. This will require, at the very least, the org service, auth service, and to-do service. We start with our org service. However, it would be naive to code the logic for our org service and just push it to Docker Hub because, as we are building the feature into our to-do service, we will likely realize that we need to go back and tweak the code of the org service. This back and forth between services, tweaking the code, can result in you sitting around waiting for Docker builds locally in different repos until the feature is consistent across all the services required. This can be mitigated with clear APIs and careful planning, but for a small team that just wants to build the feature, this is going to seriously tank your productivity.

This is where nanoservices helped me out. I was halfway through building a custom Docker tool that downloaded a microservice, scanned the downloaded microservice for other microservices that it was dependent on, and repeated the process until all the required services were downloaded. Builds were taking forever, and I would change the code in another service, need to do a release for that service, and then run the Docker builds again. Development time crawled to a snail's pace, but we wanted to still have the isolated modular code without all the Docker builds, network setup, and debugging when trying to run the code I had just written. As we have discovered throughout this book, nanoservice builds are much quicker, and we do not have to worry about setting up and configuring networks between multiple servers with config variables to get our entire system running and talking together. But container orchestration was not the only problem. As our system grew along with the developer count, our services would essentially be flying blind.

Flying blind

Let's say that you are a developer who has been assigned the task of maintaining the organization's service. You understand that one of the benefits of microservices is that they are isolated and flexible. You want to be able to keep your microservice as independent as possible. You have an endpoint that verifies that the user is associated with the correct department so that they can create a to-do item. However, you also need to have a certain role or tag associated with the user account to allow this. The to-do service may have checked the auth service, but there is no way for you to verify that it has actually happened. Also, what if the to-do service changes later and they stop checking? You do not want to be responsible for allowing unauthorized users to create that to-do item. What is more is that there might be other services wanting to perform the same check later. For instance, there might be a project management service that needs to check whether a user can write to a to-do list before assigning them to that project. You are a good developer. You do not know what the future will hold and what services will call your organization's service. Other services may not do checks with the auth service. So, you check the department/team, and the auth service for the user role before approving the creation of a to-do. As we can see in *Figure 18.3*, we have ended up calling the auth service twice:

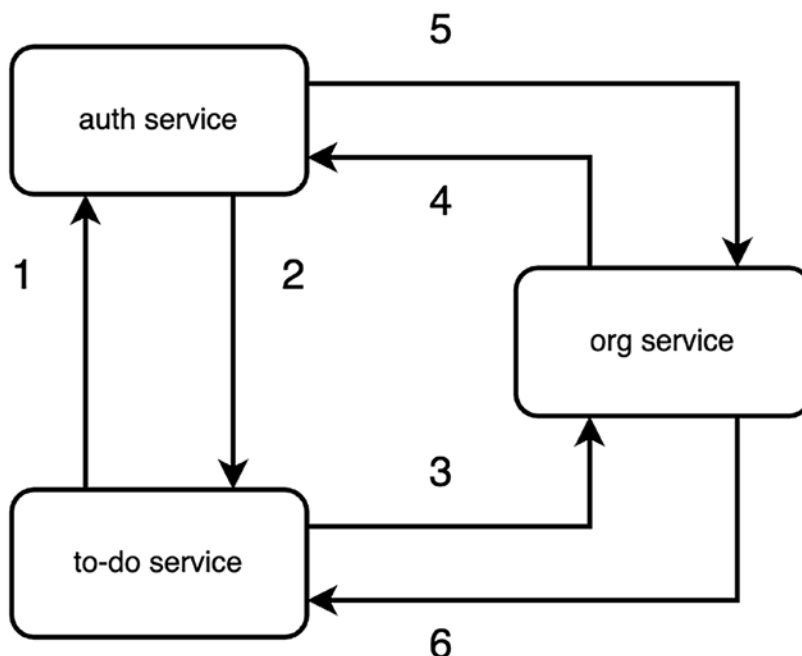


Figure 18.3 – Flying blind resulting in multiple calls to the auth service

If the project management service also makes a call to the auth service, then we have two unnecessary calls to the auth service.



If you think I am trying to concoct a situation that would not happen as developers could just talk to each other, following is a link to an interview of a principal engineer on his experiences implementing microservices at DoorDash. Here, he reveals that the account service is called roughly 300 times when an order is placed, as all these services want to get information about the account: <https://youtu.be/LcJKxPXYudE?si=MdszJ15fLz-bgfCR>.

For our nanoservices, our networking layer can make a network request to get information from another service and pass the data from that service into the core function. If we compile our services into one binary, we can just get the account information once and pass it into whatever core functions are requesting account information when we call those core functions.

Flying blind causes more than just excessive calls; it also makes us blind to circular dependencies. For instance, **service A** could depend on **service B**, which depends on **service C**. But then, **service C** depends on **service A**, as shown in *Figure 18.4*:

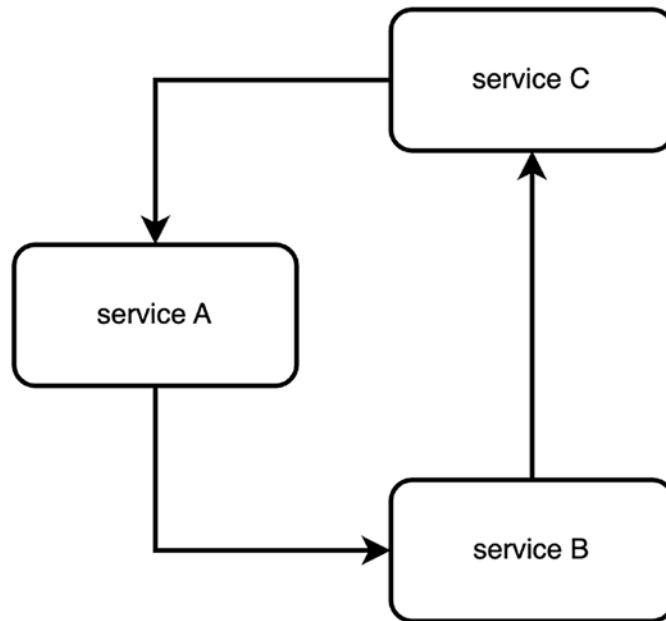


Figure 18.4 – Circular dependency in microservices

Now, if we were coding in modules, our compiler would recognize the circular dependency and refuse to compile. However, in microservices, our processes are isolated from each other and run in their own memory. Our compiler cannot pick this up, as our dependencies are satisfied by network calls. By compiling all our servers into one Rust binary, we at least get a sanity check that we do not get circular dependencies because the compiler would pick this up. We can still choose to deploy as microservices if we want. Network calls do not just increase the risk of flying blind; they also cost more.

Network calls are expensive

One last key drawback that we will cover is the latency of microservice systems. With a network call, we must serialize the data, send it over a network, and wait for its response. This is a lot slower than calling a function in memory. Compiling all our services in one binary will result in faster and less computationally expensive service calls and be less computationally expensive. To really demonstrate this, we can compare the steps between the two approaches. First, we can lay out the steps for a function call from a traditional monolith or a monolith constructed by nanoservices:

1. **Pass arguments into the function:** The function is already part of the process, so it is already in memory. For the arguments, we pass in pointers to variables allocated in memory.
2. **Execute function:** The CPU executes the function, accessing shared memory and resources, so no need for serialization.
3. **Return result:** The result is returned and written to memory with minimal overhead.

This process is usually measured in nanoseconds or microseconds, depending on the complexity of the function and system performance. For our network request, we must follow the following steps:

1. **Serialize request data:** This is where we format the request data into an acceptable format, such as JSON. This is essentially all the steps in our function call example in the first step.
2. **Establish a network connection:** This can vary depending on the network connection, but in general, packets of data are sent between the client and server as a handshake, multiple times, to establish that a connection is made. This involves multiple function calls and latency across the network.
3. **Send data:** We then send data over the network for the request. Due to packet loss, retransmissions are sometimes needed.
4. **Receive data:** There will be network latency, but once the response is received, we must deserialize the response data. Since networking can be unreliable, extra overhead of timeouts and retries is also involved when monitoring the connection.

Network calls are generally measured in milliseconds or more, depending on the network. This means that jumping from a function call to a network call can result in going from x^{-9} seconds to x^{-6} seconds. It is not surprising that AWS video streaming saved 90% on their server bills when they switched from microservices to a monolith, as seen at the following link: <https://www.thestack.technology/amazon-prime-video-microservices-monolith/>.

While it is important to be wary of the downsides of microservices, they also have some benefits. We can now move on to discussing the benefits of microservices.

The positive effects of microservices

While there are plenty of drawbacks to microservices, there have been cases where I have been grateful that I have worked on a microservice system. We can start with the most obvious benefit, which is rolling out releases.

Rolling out releases

When the team grows, working on monolithic code bases can be a nightmare. In an ideal world, the code in the monolith is well-structured with clear boundaries. However, I have rarely seen this be a reality. When there are seven or more developers working on the same monolithic repository, merging your code can grind to a halt. For instance, you are asked to add a feature, and your feature requires some account data from the auth module. This is fine; you call a function from the auth module to get the account data, then use it in your feature. You write some tests, they pass, then you push your code to the GitHub repository and make a pull request. However, another developer is working on a separate feature that also uses the account data. The developer realizes that they can make some tweaks to the function to require a slightly different parameter but less data, so they alter this function and every function call in the code base. They then merge before you into the monolithic code base. Suddenly, you have conflicts everywhere, and you must merge the latest changes into your branch and rewrite sections of your code to adapt to the new changes. In extreme examples, I've seen developers rewrite their feature up to six times before they are lucky enough to get the feature merged.

If there is a tight deadline, then seniors are incentivized to tell other developers not to tidy and refactor code as they go along, as refactoring and tidying code can step on the toes of other developers trying to fix bugs or add features. This can lead to developers being unwilling to refactor and tidy code, for fear of getting in the way of others, which can lead to terrible ball-of-mud code bases.

Another problem is building times. As the system grows, the time you need to wait before a build and integration test is done will also grow. The speed of feedback is very important not only for development speed, but also for keeping in the mental flow of the problem that you are solving. Of course, unit tests rock for this. However, if you must wait 30 minutes for a build to finish, you must orchestrate your releases. Based on continuous integration pipelines and how much money you are willing to spend on them, you may also have to wait for other developers pushing to the repository, as their builds also need to pass. Basically, monoliths suffer from developer contribution congestion. With microservices, a small team can continue to push new features to a small service. They can quickly coordinate between themselves with minimal overhead. The team can also keep their APIs consistent with versioning, adding deprecation warnings to the old API versions, and coming up with a timeframe to phase out the old API endpoints. While microservices are slower with the network calls, the clearly enforced boundaries can increase the stability of the APIs and free up developer resources when trying to get new features out the door. While API isolation is important, code isolation has also been a massive win for microservices.

Isolation of code

When it comes to criticizing microservices, most people acknowledge that developers tend not to develop perfect microservice systems. However, the same can be said for monolithic code. *Bad code is a bit like oil in water. We only need a bit of oil to contaminate the whole thing.* Once bad code starts to creep in, there is a chance that it will continue to grow. Even if you have hard-working, driven developers trying to maintain the quality of the code, these developers are not everywhere all the time. Code can get rushed to meet deadlines for releases and features. I have personally gone on holiday, only to come back to code bleeding into an isolated module that I worked very hard to keep isolated with a clean interface. I worked overtime to rip out the offending code and have it interact with the module via the interface. Sadly, if the extent of the damage is big, it can make it hard to justify refactoring the coupled code to managers. This makes the damage more at risk of growing over time.

Microservices in their own GitHub repositories are more complex to orchestrate. However, they do isolate code. You could have a couple of bad, lazy, or rushed developers completely trash the structure of a microservice. Sure, it is not a fun experience, and you should try to implement systems where these developers do not degrade the quality of your system. However, the damage is isolated to that microservice. It did not bleed over into the entire system. The worst-case scenario is that you order a complete rewrite of that individual microservice. In fact, a good friend of mine who is a lead developer leans into microservices for the initial development phase. His philosophy is that the first application that you write will always need to be rewritten, because you do not know all the requirements yet. He has his team do quick iterations of the application in a series of microservices, quickly getting feedback, and rushing those features to get more quick feedback. Once he's happy with the spec, he then has his team work through each microservice, rewriting each service one by one. He is a big fan of asking the question, "If you knew what you know now, how would you approach solving the problem?" The isolation of code via microservices has also led to the flexibility to experiment.

Flexibility to experiment

I was lucky enough to take complete ownership of a complex web system for the German government. Our system was initially a Python monolith. Then it grew to Python microservices. The system then moved over to Rust and formed the basis of this series of Rust and web programming books. We have the freedom to experiment with different approaches to Rust and web development in a bigger setting. However, we both acknowledge that microservices made the switch a possibility. Our system was being used by paying customers. It was not a toy project; we could not just go away and rewrite the entire system in Rust. Ongoing support is a thing, and new features needed to be added. Also, the system needed to keep running. Microservices made the switch to Rust a possibility.

If we had one large Python monolith and we wanted to replace that monolith with a Rust monolith, the rewrite would be massive. On top of this rewrite, the system was still live, so new features were being requested and patches were being pushed. Therefore, our monolith rewrite would be an ever-growing moving target. Large, growing moving targets are close to impossible to replace. However, because we had microservices, between adding features and patches, we could target a microservice that was not getting a new feature or patch for that sprint and rewrite it in Rust. While our system was live, and our customers were happy, we were replacing our Python servers with Rust servers seamlessly. Because we took the approach of nanoservices, we compiled a range of Rust services into one binary to make the deployment and container orchestration easier. But remember, these nanoservices are in their own repositories with their own layers. If we needed to revert one of the services back to Python, we could burst out the binary into their own microservices, swap the targeted server back to Python, and then compile all the other Rust servers back into one binary, making network calls to the Python service when needed.

We can conclude that microservices enable you to try a new web framework, coding approach, language, network protocol, or data storage method without too much commitment or disruption to the rest of the system in a new repository. Furthermore, rolling back to the previous repository if the experiment does not go to plan is simple, as the network protocols can facilitate communication between parts of the system. Your old service can just slot back in and start running if you have kept the API the same.

We have now covered enough about the positives and negatives of microservices to illustrate that there are some nasty pitfalls to them. Microservices bring their own problems. However, we can also see that it is not rational to jump onto the bandwagon of hating microservices, as microservices also reduce developer congestion and enforce code isolation. We can also see that nanoservices are the best of both worlds. They can keep the build in one binary if needed, reducing the cost of orchestration. However, they also maintain the isolation and developer flexibility that microservices offer. But what about accessing all these different repositories when running a build? It is unlikely that you are going to have all your repositories open source. To keep code isolated and flexible, we could either have different workspaces in a mono-repo, like what we have been doing throughout the book, or we could have multiple repositories pulled together for a build. Personally, I prefer multiple repositories because you can give certain people certain access to some repositories and not others. When collaborating with freelancers or other companies, giving these people access to 100% of your code because you have a mono-repo is never a good idea. But how does one repository access code from another repository? This is where dockpack comes in.

Managing private repositories with dockpack

At the time of writing this, authenticating access to GitHub repositories can be surprisingly tricky when configuring continuous integration. Cargo builds can directly point to GitHub repositories, but I have found that there are clashes in the caching. Two workspaces pointing to the same repository, on the same branch, still ended up having clashes when trying to reference the same structs. Managing private crates was a nightmare until I used Docker to distribute our code files for builds. Using Docker to distribute files has the following benefits:

- Docker handles your authentication seamlessly in deployment systems and continuous integration pipelines
- Docker handles your version control and tagging
- Docker caches the layers of the image, meaning that if your new version only changes a couple of layers in the image, then those layers are the only ones that get pulled
- All major cloud providers support private Docker registries

We can distribute our private code files using a tool I built, called dockpack. We can install dockpack with the following command:

```
cargo install dockpack
```

Now that we have dockpack installed, we can test it with the following command:

```
cargo install dockpack
dockpack pull -i maxwellflitton/nan-one -d cache
```

The `-i` argument is the Docker image that we are targeting, and the `-d` is the directory the Docker image is going to be unpacked into. Our command gives us the following file outline:

```
└─ cache
  ├── Cargo.toml
  ├── src
  │   └── lib.rs
  └── tar
      ├── maxwellflitton_nan-one
      │   ├── blobs
      │   │   └── sha256
      │   │       └── <blob files>
      │   ├── index.json
      │   ├── manifest.json
      │   └── oci-layout
      └── maxwellflitton_nan-one.tar
```


Here, we can see the code files that we can point to in other builds. The tar directory houses the data and compressed information from the Docker image. The `maxwellflitton_nan-one.tar` file is the entire Docker image compressed into one file. When the tool unpacks the `maxwellflitton_nan-one.tar` file, we get the contents in the `maxwellflitton_nan-one` directory. We can explore how the `maxwellflitton_nan-one` directory is structured by opening the `manifest.json` file, revealing the following contents:

```
[
  {
    "Config":"blobs/sha256/b26452 . . .",
    "RepoTags":[
      "maxwellflitton/nan-one:latest"
    ],
    "Layers":[
      "blobs/sha256/6c6797329a . . ."
    ]
  }
]
```

Here, we can see that the config is a blob file, and the layers are also blob files. Blob files are also TAR files. TAR files are essentially binaries that contain file content and metadata for the file content, such as relative path, permissions, and format. Because these files have the metadata in the binary header, we can have one binary that stores multiple files, and we can loop through the binary, unpacking the data for each file, and placing it in the right path in the right format due to the headers. The name of the TAR files is a hash value. This is so we can load the data from the binary TAR file and hash the data. If the data has not been compromised, then the hash will match the name. If we want to unpack the files inside the Docker image, then we loop through the Layers blob files, unpacking them and placing them where they need to be. If we want to know the commands and environment variables used to build the Docker image, we can open the blob file pointed to via the config key. The config file is not a binary, but a JSON file, which has the following contents:

```
{
  "architecture":"amd64",
  "config":{
    "Env":[
      "PATH=/usr/local/sbin:
      /usr/local/bin:/usr/sbin:/usr/bin:/sbin:/bin"
    ],
    "WorkingDir":"/",
    "OnBuild":null
  },
  "created":"2024-04-24T13:12:25.567631648+01:00",
  "history":[
    {
```

```

        "created": "2024-04-24T13:12:25.567631648+01:00",
        "created_by": "COPY ./nan-one . # buildkit",
        "comment": "buildkit.dockerfile.v0"
    }
],
"os": "linux",
"rootfs": {
    "type": "layers",
    "diff_ids": [
        "sha256:59aa253f0cd78e6605ba2cf335808d3983ca
        4ad713f5bd7a9f4383445193860b"
    ]
}
}

```

A lot of it is self-explanatory at this stage in the book, as we have built our own images in previous chapters. However, if we look at the `created_by` key, we can see how our image was created. We just copied the files in a directory called `nan-one` into the Docker image. This leads us on to how we can package our own code. We can package our own code by using the `scratch` image, which essentially has nothing in it. We then copy the files of a directory into it using the following code:

```

FROM scratch

COPY ./some_dir .

```

We can then build our image and push it with the following command:

```

docker build . \
--platform linux/amd64,linux/arm64,linux/arm/v7,\
linux/arm/v6,linux/s390x,linux/ppc64le \
-t <IMAGE_REPO> \
--push

```

Here, we can see that we build the Docker image, tagging all the platforms to say that computers running on chips and operating systems can also run our Docker image, as we are just unpacking files. We then tag the build with the image repository that we are going to push to and then push to the Docker repository using the `--push` command. With this, you can now package your private code.

Being able to distribute your private code is essential for building most systems. But what if your Rust nanoservice wants to be integrated into another program written in another language? This is where we rely on C interfaces.

Integrating Rust nanoservices with other languages using C interfaces

At the time of writing this book, Rust does not have a stable **application binary interface** (ABI). This means that we cannot compile Rust binaries and interact with these Rust binaries using another language. However, C has a very stable ABI. C's simplicity and maturity make C very portable. A programming language called Zig is attempting to replace C for portability. But at the time of writing this book, C seems like a good choice. However, we will keep an eye on Zig for future editions of this book.

For our example, we are going to build a basic Rust library that then gets compiled into a C library so it can be used by a simple Python program, as seen in *Figure 18.5*. It must be noted that, although we are using Python in the example, we can use C interfaces for most other programming languages.

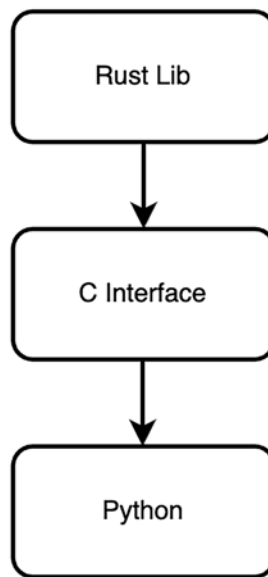


Figure 18.5 – Layers to integrate Rust with a language such as Python

The diagram in *Figure 18.5* looks like our nanoservices that we have been building throughout the book. The **Rust Lib** is our core, and the **C Interface** is like a networking layer, but for calling our Rust code from another language. If I were to place this C interface in our nanoservices system, it is essentially the same level as the networking layer, so I would either put the C interface in the networking directory or create another directory called `binary_interfaces`, as WASM might also go into the `binary_interfaces` directory if we were to support a WASM interface for our nanoservices.

For our exercise, we have the following file layout:

```
|— Cargo.toml
|— main.py
|— to-do
|   |— Cargo.toml
```

```

|   └─ src
|       └─ lib.rs
└─ to-do-c
    ├── Cargo.toml
    └─ src
        └─ lib.rs

```

Here, we can see the two Rust workspaces, which are the Rust library and the C interface. As the Python example is just going to be loading the Rust code via the C interface and using it, all our Python code is just going to be in the `main.py` file in the root of the project. We can start our project by creating our to-do library.

Creating our to-do Rust library

For our Rust library, we are just going to have a to-do item, and that is it. At this stage in the book, you will be able to build this yourself, so we are just going to whiz through our setup. Our dependencies are the following code:

```

# to-do/Cargo.toml
[package]
name = "to-do"
version = "0.1.0"
edition = "2021"

[dependencies]
serde = { version = "1.0.210", features = ["derive"] }

```

In our library, we just define a basic to-do item with the following code:

```

//! to-do/src/lib.rs
use serde::{Deserialize, Serialize};

#[derive(Deserialize, Serialize)]
pub struct Todo {
    pub title: String,
    pub status: String,
}

```

And that is it, our module is done. We are not trying to build anything comprehensive; we are just trying to get our Rust code interacting with other languages via C interfaces. Finally, our cargo file in the root of our system has the following layout:

```

# Cargo.toml
[workspace]
resolver = "2"

```

```
members = [  
    "to-do/",  
    "to-do-c/",  
]
```

Now our system is ready to support our C interface.

Creating our C interface

Our C interface essentially wraps our to-do library and compiles to a dynamic C library. For us to achieve this, our Cargo file has the following layout:

```
# to-do-c/Cargo.toml  
[package]  
name = "to-do-c"  
version = "0.1.0"  
edition = "2021"  
  
[dependencies]  
to-do = { path = "../to-do" }  
serde_json = "1.0.128"  
  
[lib]  
crate-type = ["cdylib"]
```

Here, we can see that we compile our to-do library into our interface. We also have a JSON dependency. This is because we will be using JSON to transfer our data between the C and Python interface. JSON will make our data transfer easier, as we will see when we build our interfaces. It also must be noted that we define `crate-type` as a `cdylib`. This means that we will compile our code as a dynamic C library. We will compile the library once, and then multiple applications can link to that library. This is the opposite of a static library, where the static library is compiled into the application. Our to-do library is a static library as it is being compiled into the to-do-c library. Static binaries are beneficial because they make the deployment easier due to the library being compiled into your binary, resulting in fewer moving parts. Static binaries also benefit from compiler optimization in relation to your program. However, dynamic libraries are also useful. Because dynamic libraries are not compiled into the program, other languages can link and use them. For instance, Python would not be able to compile our Rust code into it. Apart from being a different language, Python does not even compile its code. Dynamic libraries also reduce the binary size. For instance, if we have five applications relying on library A. If all the applications statically link to library A, then library A will be copied 5 times and compiled into each application. However, if the five applications all dynamically link to library A, then they just load the same library at runtime, and the five application binaries do not have library A in each binary. Dynamic libraries can also reduce the compile time. If you have a massive system, and you need to recompile a binary, if the recompilation does not need to compile loads of static libraries, then it will be quicker. Like everything else in engineering, there are always trade-offs.

Now that we understand our build, we can move on to creating our C interface with the following imports:

```
//! to-do-c/src/lib.rs
use std::ffi::{CStr, CString};
use std::os::raw::c_char;
use to_do::Todo;
```

For the first import, we are using the `ffi` crate in the standard library. **FFI** stands for **foreign function interface**, which is essentially an interface with C and C datatypes. We will be using FFI from the Python standard library to interface with our C interface when we write the Python code. Google FFI for any programming language that you are interested in, and I will honestly be shocked if the programming language cannot interface with C using FFI.

For our FFI imports, `CStr` is a borrowed C string. `CString` is a reference to `CStr`. `CStr` essentially converts the C string type to a string that is compatible with Rust. `CString` allows developers to get mutable access to the string. `CString` also takes ownership of the string, dropping the string from memory once it moves out of scope. Our `c_char` is the representation of a character in C, as the format is different to how Rust represents characters.

Now that we have these imports, we can define our API function signatures with the following code:

```
//! to-do-c/src/lib.rs
. . .
#[no_mangle]
pub extern "C" fn create_todo_json(title: *const c_char)
    -> *mut c_char {
    . . .
}

#[no_mangle]
pub extern "C" fn mark_done_json(todo_json: *const c_char)
    -> *mut c_char {
    . . .
}
```

We can see that in both functions, we return a mutable reference to a character. This might seem strange as we are returning the JSON string of the entire to-do item. The pointer points to the start of the string. If we want to read the string, we can keep increasing our pointer by one byte, reading one character at a time. When we come across a `\0`, we know that this is the end of the string, and we stop. Now that we understand how the pointer is used, we can see why we also pass a pointer to a character into the functions. We can recall from our WASM chapter that `#[no_mangle]` prevents the compiler from optimizing the function to the point of not being recognized. Instead, the signature is maintained when compiling, so we can access the function when loading our binary. The `extern "C"` is telling the compiler that this is a C interface.

You may be wondering why we are just using JSON strings. This is to simplify the transfer of data to another language. We can pack collections into JSON, and our signature will be the same. However, if we wanted to represent a vector of to-do items and pass that over to another language, we would have to allocate a chunk of memory, chunk that memory up, and manually move the pointer over the memory to access elements in the vector. To demonstrate this, the following is a link to a GitHub repository that implements a vector in C: <https://github.com/juliuskoskela/c-vec>.

We are already improving our latency by not having to make network calls. Implementing C-friendly data collections is not worth the time unless you really need to speed up optimization to the point where JSON serialization is too expensive.

We can now build our interface functions. Inside our `create_todo_json` function, we convert our pointer pointing to the initial character of a string into a Rust string with the following code:

```
//! to-do-c/src/lib.rs
let c_str = unsafe { CStr::from_ptr(title) };
let title = c_str.to_str().unwrap_or_default().to_string();
let todo = Todo {
    title,
    status: "PENDING".to_string(),
};
```

We must use the `unsafe` tag because this is a pointer that was just passed in. It does not have the Rust guarantees. We then construct our to-do item with the title string we converted from the pointer. We then convert our to-do item into a JSON string, then into a C string, and return the pointer with the following code:

```
//! to-do-c/src/lib.rs
let json = serde_json::to_string(&todo).unwrap_or_default();
CString::new(json).unwrap().into_raw()
```

We can see that the `into_raw` function exposes the raw pointer. Because we return the raw pointer, the language interfacing with our nanoservice can then take that pointer and load a function its own way.

For our `mark_done_json` function, we construct the string from the pointer, just like we did before, with the following code:

```
//! to-do-c/src/lib.rs
let c_str = unsafe { CStr::from_ptr(todo_json) };
let json = c_str.to_str().unwrap_or_default();
```

We then create the JSON string and return the pointer, as we did before, with the following code:

```
//! to-do-c/src/lib.rs
let updated_json = serde_json::to_string(&todo).unwrap_or_default();
CString::new(updated_json).unwrap().into_raw()
```

And with this, our C interface is complete. We can now interact with our Rust nanoservice with Python.

Interfacing with our Rust nanoservice with Python

In this section, we will interact with Python using raw C interfaces to learn how linking Rust works. However, if you want a stress-free way of building Python packages in Rust, use the **PyO3** crate. In our Python file, we will load the dynamic C library, specify the input and output types, and then call these functions. First, we load our dynamic C library, depending on the system, with the following code:

```
# main.py
import ctypes
import platform
system = platform.system()

if system == "Linux":
    lib = ctypes.CDLL("./target/release/libto_do_c.so")
elif system == "Darwin":
    lib = ctypes.CDLL("./target/release/libto_do_c.dylib")
elif system == "Windows":
    lib = ctypes.CDLL("./target/release/to_do_c.dll")
else:
    raise OSError("Unsupported platform: " + system)
```

The file extension will be different depending on the operating system that we are using. We must remember to build our Cargo workspaces in release mode for our Python nanoservice to load the dynamic C library; otherwise, the dynamic C library binary will not be there.

Now that we have loaded our dynamic C library, we can define the inputs and outputs of our functions in the dynamic C library with the following code:

```
# main.py
lib.create_todo_json.argtypes = [ctypes.c_char_p]
lib.create_todo_json.restype = ctypes.c_char_p
lib.mark_done_json.argtypes = [ctypes.c_char_p]
lib.mark_done_json.restype = ctypes.c_char_p
```

Now, with the inputs and outputs defined, we can build some functions that interact with our dynamic C library files with the following code:

```
# main.py
def create_todo_json(title):
    todo_json = lib.create_todo_json(
        title.encode("utf-8")
    )
    todo_py = todo_json.decode("utf-8")
    return todo_py

def mark_done_json(todo_json):
    updated_todo_json = lib.mark_done_json(
```



```
        todo_json.encode("utf-8")
    )
    updated_todo_py = updated_todo_json.decode("utf-8")
    return updated_todo_py
```

Here, we can see that we decode the C ASCII string into utf-8. utf-8 is a more modern representation but is also more complex. ASCII is essentially a subset of utf-8 meaning that ASCII can represent fewer characters. Python's standard string format is utf-8. However, this is not a problem for us as we have a raw pointer to the bytes, and when we read them, we can just decode the bytes using utf-8. On our interface functions, we can see that we just call the dynamic C library functions and then serialize the strings. We can now test out functions by running them and printing out the results with the following code:

```
# main.py
if __name__ == "__main__":
    todo_json = create_todo_json("Learn Rust")
    print(f"Todo JSON: {todo_json}")
    updated_todo_json = mark_done_json(todo_json)
    print(f"Updated Todo JSON: {updated_todo_json}")
```

If we run the Python script using the `python3 main.py` command, we should get the following printout:

```
Todo JSON: {"title":"Learn Rust","status":"PENDING"}
Updated Todo JSON: {"title":"Learn Rust","status":"DONE"}
```

And there we have it, our Rust nanoservice can directly interact with other nanoservices from other languages without needing to make network calls.

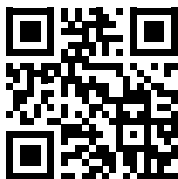
Summary

From this chapter, we can conclude that microservices enable developers to keep producing code, patches, and features at a quick pace without stepping on each other's toes. However, the increased complexity of the system can lead to systems that can get convoluted and hard to manage, deploy, and develop against. We can also see that nanoservices can have the benefit of isolating code, enabling experimentation with different languages through the foreign function interface, and enabling smaller teams to do their own releases. However, also understand the build benefits of monoliths. If a monolith build gets too big, slowing down releases, we can break our nanoservices out into microservices. However, if we still want to keep the benefits of the monolith, we can also break out the core of the nanoservice into a dynamic C library and have our nanoservices communicate through foreign function interfaces. This can reduce latency as we are not making network calls, and we do not have to worry about container orchestration. For the deployment, we only need to compile the one service binary that we are swapping out. However, this dynamic C lib approach can introduce the drawback of flying blind and missing out on compiler optimization across the entire system. Here, we can see that the topics that we covered in this chapter complement each other, enabling you to come up with your own systems, with their own trade-offs, to solve the problem you need solving.

In the next chapter, we will cover queuing mechanisms for distributing workloads.

Learn more on Discord

To join the Discord community for this book – where you can share feedback, ask questions to the author, and learn about new releases – follow this QR code: <https://packt.link/EaKXL>



19

Low-Level Networking

Throughout the book, we have been communicating with different servers and programs using a network protocol. However, understanding how protocols (such as the **HTTP protocol**) are built can help you understand networking on a deeper level. This does not mean that you should run around implementing your own protocols all the time. However, understanding these protocols on a deeper level will help you understand the trade-offs between different protocols. This awareness will also help you with debugging and understanding why high-level APIs for protocols are the way they are. In this chapter, we will cover the following:

- Building a TCP server
- Implementing HTTP on top of TCP
- Implementing HTTPS on top of TCP
- Exploring **WebSockets**

By the end of this chapter, you will be able to implement a basic binary protocol on top of a TCP connection and build a TCP server. You will also be able to implement an HTTP and **HTTPS protocol** on top of the TCP connection. Finally, you will be able to implement a basic WebSocket on top of TCP connections. We also cover an example WebSocket with the Actix API, so you can see how your deeper understanding helps you understand the API reference at a deeper level.

Technical requirements

You can download the example project and code for this book by following the instructions in the **Download the example code files** section in the *Preface* of this book.

This chapter's code files are included in the downloadable code bundle.

Building a TCP server

TCP connections form the basis of a lot of internet networking protocols. When we create a TCP connection, we can send messages back and forth between the client and server. Connection protocols such as HTTP are built on top of the TCP connection. In this section, we are going to explore TCP connections by building our own binary protocol for a simple key-value store. To achieve this, we are going to utilize a kernel, client, and server with the following file outline:

```

├─ Cargo.toml
├─ client
│   └─ Cargo.toml
│       └─ src
│           └─ main.rs
├─ kernel
│   └─ Cargo.toml
│       └─ src
│           └─ lib.rs
└─ server
    └─ Cargo.toml
        └─ src
            └─ main.rs

```

To get our system running, we must carry out the following steps:

1. Building a kernel for key-value messages
2. Building a TCP server
3. Building a TCP client

We can start with the kernel.

Building a kernel for key-value messages

For our message, we are going to serialize our message into a binary stream using bincode. Using bincode makes our code examples simple. However, if the client to the TCP server is not coded in Rust, then our bincode serialization approach will not work, as we are directly serializing Rust data types to bytes. Other serialization protocols, such as JSON, can be used to transfer data between different programming languages.

For our kernel, we need the following `Cargo.toml`:

```

# kernel/Cargo.toml
[package]
name = "kernel"
version = "0.1.0"
edition = "2021"

[dependencies]
bincode = "1.3.3"
serde = { version = "1.0.210", features = ["derive"] }

```

With these dependencies, we can now define our key-value message with the following code:

```

//! kernel/src/lib.rs
use serde::{Serialize, Deserialize};

```

```
#[derive(Debug, PartialEq, Serialize, Deserialize)]
pub enum KvMessage {
    Get(Vec<u8>),
    Put((Vec<u8>, Vec<u8>)),
    Del(Vec<u8>),
    Success(bool),
    ReturnValue(Option<Vec<u8>>),
    Error(String),
}
```

Here, we can see that we support some basic key-value operations and some return values, depending on the outcome. We now need to pack our message so that the bytes for the message can be sent over a network, and that the TCP server or client can process the bytes. Our package function takes the following form:

```
//! kernel/src/lib.rs
impl KvMessage {
    pub fn package(&self) -> Vec<u8> {
        let data = bincode::serialize(&self).unwrap();
        let len = data.len() as u32;
        let len_buf = len.to_be_bytes();
        let mut buf = Vec::new();
        buf.extend_from_slice(&mut len_buf.to_vec());
        buf.extend_from_slice(&mut data.to_vec());
        buf
    }
}
```

Here, we can see that we serialize the message into a vector of bytes. We then get the length of the vector of bytes and convert that length into byte form using the `to_be_bytes` function. We then place the length of the data as bytes into the buffer, and then our data. This means that a receiver receiving the message can read the first four bytes to get the length of the data, and then read the data and deserialize it to the message enum to be processed. With this, our kernel is complete and ready to be used by our client and server.

Building a TCP server

For our server, we need the following dependencies:

```
# server/Cargo.toml
[package]
name = "server"
version = "0.1.0"
edition = "2021"
```

```
[dependencies]
tokio = { version = "1.40.0", features = ["full"] }
kernel = { path = "../kernel" }
bincode = "1.3.3"
```

At this point in the book, these dependencies should be understood. For our TCP server exercise, we will put all the code in the `main.rs` file to keep it simple. For our server, we need the following imports:

```
//! server/src/main.rs
use tokio::{
    io::{
        AsyncReadExt, AsyncWriteExt},
    net::{TcpListener, TcpStream}
};
use std::sync::{Arc, Mutex};
use std::collections::HashMap;
use kernel::KvMessage;
```

The read and write traits are going to be utilized by our TCP stream. All the other structs enable us to perform key-value operations on the state of the server. We can see how the state is constructed with the following code:

```
//! server/src/main.rs
#[tokio::main]
async fn main() {
    let listener = TcpListener::bind(
        "127.0.0.1:9000"
    ).await.unwrap();
    let state = Arc::new(Mutex::new(HashMap::new()));
    loop {
        let (socket, _) = listener.accept().await.unwrap();
        let state_ref = state.clone();
        tokio::spawn(async move {
            println!("new connection incoming");
            process(socket, state_ref).await;
        });
    }
}
```

Here, we can see that we create a TCP listener on port 9000. We then create a hashmap and wrap this hashmap in an `Arc` and `Mutex`. The `Arc` is a pointer that counts the references to that pointer. If the number of references to the pointer becomes zero, then the hashmap is cleaned up. Considering that we are defining our state in the `main` function, we will always have at least one reference to the state throughout the entire lifetime of the TCP server.

The `Mutex` enables us to lock the data and mutate or directly reference the data inside the `Mutex`. Both `Arc` and `Mutex` are thread-safe, meaning that we can access the state in any thread when processing the key value. This does, however, mean that reads and writes are single-threaded throughout the application, as all threads and async tasks must wait their turn to access the lock for the data. This is not a book on databases, and we are merely using this example to explore custom protocols over TCP. If you want an optimized key-value store, entire packages are designed so you can perform fast reads and writes to a key-value store, such as the following:

- **RocksDB:** Probably one of the most mature, but it is a Rust wrapper around a C++ key-value store, so it can lead to compilation errors in builds depending on your distribution
- **ReDB:** A pure Rust implementation of a key-value store that is still young but is actively supported and shows a lot of promise

Now that we have defined our state, we have a loop where we are listening for incoming requests. We then make a clone of our state, which is a reference to the state. Once we have the reference and the connection stream, we spawn a task that processes incoming message. This process function takes the connection stream and a reference to the state. We can see that the process function is inside an async task that we spawn. This means that our loop can resume, and we can go back to listening to more incoming TCP connections while our message is being processed on the async runtime.

For our process function, we have the following outline:

```

//! server/src/main.rs
async fn process(
    mut socket: TcpStream,
    state: Arc<Mutex<HashMap<Vec<u8>, Vec<u8>>>>
) {
    let mut len_buf = [0; 4];
    if socket.read_exact(&mut len_buf).await.is_err() {
        eprintln!("Failed to read length");
        return;
    }
    let len = u32::from_be_bytes(len_buf) as usize;
    println!("Expecting {} bytes", len);
    let mut buffer = vec![0; len];
    if socket.read_exact(&mut buffer).await.is_err() {
        eprintln!("Failed to read message");
        return;
    }
    let message: KvMessage = bincode::deserialize(&buffer).unwrap();
    let return_message = match message {
        . . .
    };
    let return_message_data = return_message.package();

```

```

    if let Err(e) = socket.write_all(&return_message_data).await {
        eprintln!("Failed to write to socket; err = {:?}", e);
    }
}

```

Here, we read the first four bytes by having a buffer of `[0; 4]` and passing that buffer into the `read_exact` function of the TCP stream. We then convert these bytes into a `usize`. We now have the total number of bytes that we expect from the rest of the stream to get the entire message. We then construct a vector of that size, and then perform another `read_exact` function of the TCP stream to get the rest of the message. Finally, we deserialize the bytes into a message struct and then process the message, performing a key-value operation on our state. The return message from that operation is then packaged using our package function and written to the stream using the `write_all` function. For the processing of our incoming message, we have the following code:

```

//! server/src/main.rs
let return_message = match message {
    KvMessage::Get(key) => {
        let value = state.lock().unwrap().get(&key).cloned();
        KvMessage::ReturnValue(value)
    },
    KvMessage::Put((key, value)) => {
        state.lock().unwrap().insert(key, value);
        KvMessage::Success(true)
    },
    KvMessage::Del(key) => {
        state.lock().unwrap().remove(&key);
        KvMessage::Success(true)
    },
    _ => {
        KvMessage::Error("Invalid message type".to_string())
    }
};

```

Here, we can see that we accept the key-value operation type and return a message that is appropriate, depending on the key-value operation. We can also see that if we receive a message that is supposed to be a return value, we return an error.

Now that our server is built, we can move on to the client.

Building a TCP client

For our client, we have the same dependencies as the TCP server, as seen here:

```

# client/Cargo.toml
[package]

```



```

name = "client"
version = "0.1.0"
edition = "2021"

[dependencies]
tokio = { version = "1.40.0", features = ["full"] }
kernel = { path = "../kernel" }
bincode = "1.3.3"

```

For our `main.rs` file, we need the following imports:

```

///! client/src/main.rs
use tokio::net::TcpStream;
use tokio::io::{AsyncReadExt, AsyncWriteExt};
use kernel::KvMessage;

```

For our client, we need a function called `send_message` that carries out the following steps:

1. Accept a message and address to the server
2. Connect to the server with a TCP stream
3. Package the message to be sent
4. Send the message over the TCP connection
5. Read the first four bytes from the stream to get the length of the data
6. Read those bytes from the TCP stream
7. Deserialize the bytes from the stream into a message
8. Return that message

At this stage, you are in a good position to try and build this function yourself. Now is a good time to stop reading and try and code the `send_message` function yourself.

If you have attempted to build the function yourself, hopefully, it looks like the following code:

```

///! client/src/main.rs
async fn send_message(message: KvMessage, addr: &str)
    -> Result<KvMessage, String> {
    let mut socket = TcpStream::connect(addr).await.map_err(
        |e| format!("Failed to connect: {}", e)
    );
    let data = message.package();
    socket.write_all(&data).await.map_err(
        |e| format!("Failed to write to socket: {}", e)
    );
    let mut len_buf = [0; 4];
    if socket.read_exact(&mut len_buf).await.is_err() {

```

```

        return Err("Failed to read length".to_string());
    }
    let len = u32::from_be_bytes(len_buf) as usize;
    let mut buffer = vec![0; len];
    if socket.read_exact(&mut buffer).await.is_err() {
        return Err("Failed to read message".to_string());
    }
    let message: KvMessage = bincode::deserialize(&buffer).map_err(
        |e| format!("Failed to deserialize message: {}", e)
    )?;
    return Ok(message)
}

```

Finally, we can test our send function in our main function with the following code:

```

//! client/src/main.rs
#[tokio::main]
async fn main() -> Result<(), String> {
    println!("{}", send_message(
        KvMessage::Get(b"key".to_vec()),
        "127.0.0.1:9000"
    ).await?);
    println!("{}", send_message(
        KvMessage::Put((b"key".to_vec(), b"value".to_vec())),
        "127.0.0.1:9000"
    ).await?);
    println!("{}", send_message(
        KvMessage::Get(b"key".to_vec()),
        "127.0.0.1:9000"
    ).await?);
    println!("{}", send_message(
        KvMessage::Del(b"key".to_vec()),
        "127.0.0.1:9000"
    ).await?);
    Ok(())
}

```

If we run our server, and then our client, our client gives us the following printout:

```

ReturnValue(None)
Success(true)
ReturnValue(Some([118, 97, 108, 117, 101]))
Success(true)

```

Here, we can see that there is no value initially, then we have success in inserting the value. We can then return the value and then delete the value. Our system works, and we are sending binary messages between the client and the server.

Now that we have become comfortable with TCP, we can move on to building an HTTP request on top of a TCP connection.

Implementing HTTP on top of TCP

In this section, we are going to implement a basic HTTP request ourselves using a TCP connection. To explore building our own HTTP request, we need the following layout:

```
├─ Cargo.toml
├─ client
│   └─ Cargo.toml
│       └─ src
│           └─ main.rs
└─ http-server
    └─ Cargo.toml
        └─ src
            └─ main.rs
```

First, we are going to define a basic server.

Defining a basic server

For our server, we create a basic Actix server with the following `Cargo.toml` file:

```
# http-server/Cargo.toml
[package]
name = "http-server"
version = "0.1.0"
edition = "2021"

[dependencies]
actix-web = "4.9.0"
serde_json = "1.0.132"
serde = { version = "1.0.0", features = ["derive"] }
```

Here, we can see that we are just using the Actix framework with `serde` for processing JSON bodies.

For our basic server, we have the following imports:

```
#!/ http-server/src/main.rs
use actix_web::{
    web, App, HttpServer, Responder, HttpResponse
};
use serde::Deserialize;
```

Our JSON body has the following definition:

```
///! http-server/src/main.rs
#[derive(Deserialize, Debug)]
struct InputData {
    message: String,
}
```

We handle our incoming requests with the following function:

```
///! http-server/src/main.rs
async fn post_handler(data: web::Json<InputData>)
    -> impl Responder {
    println!("Received: {:?}", data);
    let response_message = format!(
        "Received: {}", data.message
    );
    HttpResponse::Ok().json(
        serde_json::json!({"response": response_message})
    )
}
```

And finally, the server has the following simple definition:

```
///! http-server/src/main.rs
#[actix_web::main]
async fn main() -> std::io::Result<()> {
    HttpServer::new(|| {
        App::new()
            .route("/api/data", web::post().to(post_handler))
    })
    .bind("127.0.0.1:8080")?
    .run()
    .await
}
```

Our server is now ready to build an HTTP client from scratch.

Defining an HTTP client from scratch

For our HTTP client, we are not going to use any dependencies; instead, we are just going to use the standard library. In our `main.rs` file, we initially have the following imports:

```
///! client/src/main.rs
use std::io::{Read, Write};
use std::net::TcpStream;
```

Inside our main function, we create the TCP stream with the following code:

```
///! client/src/main.rs
let mut stream = TcpStream::connect(
    "127.0.0.1:8080"
).expect("Failed to connect to server");
```

We then define our HTTP data to send over the TCP stream with the following code:

```
///! client/src/main.rs
let json_body = r#"{"message": "Hello, Actix!"}"#;
let request = format!(
    "POST /api/data HTTP/1.1\r\n\
    Host: 127.0.0.1\r\n\
    Content-Type: application/json\r\n\
    Content-Length: {}\r\n\
    Connection: close\r\n\r\n\
    {}",
    json_body.len(),
    json_body
);
```

Here, we can see that the HTTP request is a text protocol. The first part is the header, and the method, URI, and HTTP protocol are defined with spaces between them. We then use the delimiter `\r\n` for other values in the header. We can see that we need the content length of the JSON body because the JSON body is at the end of the message, and we need to know the number of bytes to read to get the entire JSON body.

HTTP is basic



The HTTP protocol is known for being basic. It is just text. Because of its basic nature, it has completely dominated web communication. Because it is simple, support for HTTP is everywhere in every browser. I've worked on projects where people have tried to get smart with the network protocol they use. Most of the time, their attempts have failed if the network protocol needs to communicate to a client. We cannot know what version of a browser the client is using, or their internet connectivity, and so on. Unless you really need high throughput, then HTTP and HTTPS will be good enough for your application, and you will be able to support most functionality with HTTP and HTTPS.

Now that our message body is defined, we can write our HTTP request to the TCP stream with the following code:

```
///! client/src/main.rs
stream.write_all(request.as_bytes()).unwrap();
```

Now, we need to process the HTTP response. This consists of looping and reading from the stream until we no longer read any bytes. We then have the entire response string. This is achieved with the following code:

```

/// client/src/main.rs
let mut buffer = [0; 512];
let mut response = String::new();
while let Ok(bytes_read) = stream.read(&mut buffer) {
    if bytes_read == 0 {
        break;
    }
    response.push_str(
        &String::from_utf8_lossy(&buffer[..bytes_read])
    );
}
println!("Response: {}", response);

```

And here we have it. If we run our server and then run our client, we get the following printout for the server:

```
Received: Json(InputData { message: "Hello, Actix!" })
```

And the following printout for the client:

```

Response: HTTP/1.1 200 OK
content-length: 38
connection: close
content-type: application/json
date: Sun, 20 Oct 2024 03:10:23 GMT

{"response":"Received: Hello, Actix!"}

```

We have now constructed a basic HTTP request over TCP. But what about HTTPS? In the next section, we will construct an HTTPS request over TCP.

Implementing HTTPS on top of TCP

In this section, we are going to build on our server and client to implement TLS encryption to send encrypted traffic between the client and server. For our server, we now must enable the `openssl` feature and use the `openssl` crate, resulting in the following dependencies:

```

# http-server/Cargo.toml
[dependencies]
actix-web = { version = "4.9.0", features = ["openssl"] }
serde_json = "1.0.132"
serde = { version = "1.0.0", features = ["derive"] }
openssl = "0.10.68"

```

We then add the following import to our `main.rs` file in our server:

```
//! server/src/main.rs
use openssl::ssl::{SslAcceptor, SslFiletype, SslMethod};
```

Then, our main function takes the following form:

```
//! server/src/main.rs
#[actix_web::main]
async fn main() -> std::io::Result<()> {
    let mut builder = SslAcceptor::mozilla_intermediate(
        SslMethod::tls()
    ).unwrap();
    builder.set_private_key_file(
        "key.pem", SslFiletype::PEM
    ).unwrap();
    builder.set_certificate_chain_file(
        "cert.pem"
    ).unwrap();
    HttpServer::new(|| {
        App::new()
            .route("/api/data", web::post().to(post_handler))
    })
    .bind_openssl("127.0.0.1:8080", builder)?
    .run()
    .await
}
```

Here, we can see that we define an SSL acceptor. An SSL acceptor is a configuration object that allows the server to handle incoming secure connections. It is responsible for initializing and configuring the SSL/TLS layer when a client connects to the server, ensuring that the communication is encrypted.

Our builder is a `mozilla_intermediate`. This is a good default. It is not too cutting edge to cut out older out-of-date browsers, but it is up to date enough to not be a security threat. If we were to use the `mozilla_modern_v5` at the time of writing this book, the client would require OpenSSL 1.1.1 or LibreSSL 3.4.0 or newer. We then attach a private key to the builder. This private key is used for the encryption of the traffic between the server and client. And then we attach a certificate where we define the identity of the server. As we remember, when we implemented HTTPS in AWS, the certificate was issued by a provider. We will be signing our own certificate, so we will have to bypass security checks and allow our client to connect to our server with our self-signed certificate. Finally, we bind our server to OpenSSL using the `bind_openssl` function instead of the `serve` function.

For our key and certificate, we must install `openssl`. For macOS, we can install using `brew` with the following command:

```
brew install openssl
```

For Linux, you can install `openssl` with the following command:

```
sudo apt install openssl
```

For Windows, it is highly suggested that you use the Linux subcommand terminal. However, you could install `openssl` with Chocolatey with the following command:

```
choco install openssl
```

Now that we have `openssl` installed, we can generate our private key with the following command:

```
openssl genrsa -out key.pem 2048
```

The number is the number of bits that the key has. The bigger the number, the larger the key. The larger the key, the more secure the encryption. However, this increased key size comes at a performance cost.

For our certificate, we create and sign our own with the following command:

```
openssl req -new -x509 -key key.pem -out cert.pem -days 365
```

The `-x509` is the standard for defining the key. The `-x509` means that we are telling `openssl` to generate a self-signed certificate as opposed to making a request for a certificate. `openssl` will prompt you to fill in a range of different questions about yourself for the certificate. It does not matter what you put, as we are just running this locally. Once the key and certificate are there, we can run our server.

With our server running, we can move on to enabling HTTPS for the client. Our client will need the native TLS in the `Cargo.toml` file with the following code:

```
# client/Cargo.toml
[dependencies]
native-tls = "0.2"
```

We are going to encrypt our TCP connection with the `native-tls` crate. Inside our `main.rs` file, we import the TLS connector with the following code:

```
//! client/src/main.rs
use native_tls::TlsConnector;
```

We then define the TLS connector with the setting to accept an invalid certificate with the following code:

```
//! client/src/main.rs
let connector = TlsConnector::builder()
    .danger_accept_invalid_certs(true)
    .build()
    .expect("Failed to create TLS connector");
```

We then make the TCP connection, and wrap that connection in the TLS connector with the following code:

```
//! client/src/main.rs
```



```
let stream = TcpStream::connect(
    "127.0.0.1:8080"
).expect("Failed to connect to server");

let mut stream = connector
    .connect("127.0.0.1", stream)
    .expect("Failed to establish TLS connection");
```

The rest of the client stays the same. If we run the client, we get the same outcome as we did before, but we have an HTTPS request instead of an HTTP request. And there we have it; we have built an HTTPS protocol on top of our TCP connection. With HTTP protocols covered, we can now move on to WebSockets.

Exploring WebSockets

A WebSocket is a TCP connection where multiple messages are sent back and forth between each other. This can reduce the overhead as we do not need to make new connections every time we need to exchange data. WebSockets are useful for push notifications, live updates, and data feeds, as we can continually push data from the server to the client, or vice versa. As the WebSocket is on top of a TCP connection, we can send any type of message we want. For instance, we can send binary messages or HTTP requests. The main feature of the WebSocket is that the TCP connection remains live, so multiple messages can be sent over that connection.

We can build a basic WebSocket with a simple TCP server and client setup. The overall mechanism is a TCP socket reader that runs in a loop to continuously listen to updates from the WebSocket, as seen in *Figure 19.1*:

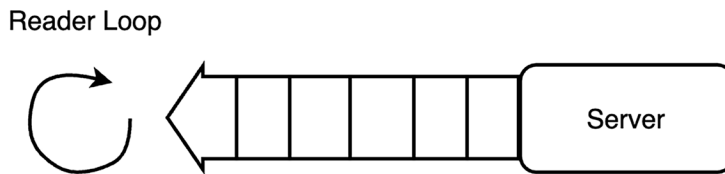


Figure 19.1 – How to listen and process incoming messages over WebSocket

For our server workspace, all we need is Tokio. For our server, we are going to receive a connection and then send a range of numbers. In our main file, our process function merely gets the TCP stream and sends a range of numbers down the stream with the following code:

```
//! server/src/main.rs
use tokio::{io::AsyncWriteExt, net::{TcpListener, TcpStream}};

async fn process(socket: TcpStream) {
    let (_, mut writer) = socket.into_split();
    for i in 1u32..10 {
        let bytes = i.to_be_bytes();
```

```

        writer.write_all(&bytes).await.unwrap();
    }
    writer.write_all(&[0, 0, 0, 0]).await.unwrap();
}

```

At the end of the loop, we will send a zero to the client. In our client, we will treat the zero number as a signal to kill the connection. For the main function of the server, we just listen for incoming connections and spawn an async task with the following code:

```

//! server/src/main.rs
#[tokio::main]
async fn main() {
    let listener = TcpListener::bind(
        "127.0.0.1:9000"
    ).await.unwrap();
    loop {
        let (socket, _) = listener.accept().await.unwrap();
        tokio::spawn(async move {
            println!("new connection incoming");
            process(socket).await;
        });
    }
}

```

For our client, we split the stream into the reader and writer and loop continuously until a zero is sent to the client with the following code:

```

//! client/src/main.rs
use tokio::net::TcpStream;
use tokio::io::AsyncReadExt;

async fn await_messages(addr: &str) -> Result<(), String> {
    let socket = TcpStream::connect(addr).await.map_err(
        |e| format!("Failed to connect: {}", e)
    )?;
    let (mut reader, _) = socket.into_split();
    loop {
        let mut num_buf = [0; 4];
        reader.read_exact(&mut num_buf).await.map_err(
            |e| format!("Failed to read length: {}", e)
        )?;
        let num: usize = u32::from_be_bytes(num_buf) as usize;
        println!("{:?}", num);
    }
}

```

```

        if num == 0 {
            break;
        }
    }
    Ok(())
}
#[tokio::main]
async fn main() -> Result<(), String> {
    await_messages("127.0.0.1:9000").await?;
    Ok(())
}

```

If we run the client, we will see that the client prints out the numbers the client receives. These messages could be stock price updates for a dashboard where the JavaScript gets the stock name and price, and then allocates the update for the right component.

While we have written our own basic implementation of a WebSocket, there are plenty of libraries that facilitate WebSockets out of the box. These give you more advantages, such as fragmentation of the messages, framing, bidirectional streaming, and so on. We also should not have to be restricted to only using Tokio directly to handle TCP connections manually for a WebSocket. All major web frameworks support WebSockets. For instance, here is the code from the documentation for Actix on how to handle WebSockets:

```

async fn echo(req: HttpRequest, stream: web::Payload)
-> Result<HttpResponse, Error> {
    let (res, mut session, stream) = actix_ws::handle(
        &req, stream
    );
    let mut stream = stream
        .aggregate_continuations()
        .max_continuation_size(2_usize.pow(20));
    rt::spawn(async move {
        while let Some(msg) = stream.next().await {
            match msg {
                Ok(AggregatedMessage::Text(text)) => {
                    session.text(text).await.unwrap();
                }
                Ok(AggregatedMessage::Binary(bin)) => {
                    session.binary(bin).await.unwrap();
                }
                Ok(AggregatedMessage::Ping(msg)) => {
                    session.pong(&msg).await.unwrap();
                }
            }
        }
    })
}

```

```
        _ => {}  
    }  
}  
});  
Ok(res)  
}
```

Here, we can see that we take the HTTP request and the stream, which is the `Payload` struct. The `Payload` struct extracts the request's raw payload stream. This makes sense as we know that the HTTP request is just a protocol sent over TCP. Therefore, Actix is just extracting the TCP stream that the HTTP request was sent over. The documentation code example then spawns an async task with `rt::spawn`. The `rt` is Actix's own async runtime, which is like Tokio. We can see that inside the spawned task, we have a loop waiting for messages. Because WebSocket libraries abstract a lot of manual implementations, we can also see that the WebSocket can handle text or binary data and is handled in frames with the `stream.next().await`. Frames have headers that have the length of the frame. These frames can also be fragmented, so there is also information around the frame that the packet belongs to, and the order in which the frame is sent. Unless you have very specific needs, it is generally best to use a WebSocket library off the shelf. If you do not want to use a framework such as Actix and you just want to use Tokio, the `tokio-tungstenite` crate supports WebSockets that connect to Tokio.

Summary

In this chapter, we focused on building custom and basic HTTP protocols on top of TCP connections. We could have just listed all the different APIs for networks and shown how to call them. Instead, we focused on how to build protocols on top of the TCP connection. As we saw with the Actix WebSockets documentation code, we can now understand what is going on under the hood of the API code. It must be stressed that most connection protocols have been defined in web-established third-party dependencies, so while it is useful to know how protocols are built, most of the time, you should use the established dependency, as bugs have been smoothed out, and timeouts, connection attempts, and framing have also been built into these third-party dependencies. In the next chapter, we will build queuing mechanisms.

Questions

1. At a high level, how can you implement a basic binary protocol from a Rust client to a Rust TCP server? This means, how do you structure the message and pack/read that message?
2. At a high level, what is an HTTP request?
3. At a high level, what is a WebSocket?
4. When is it a good idea to use WebSockets?

Answers

1. We can use `bincode` to serialize the struct, data type, or tuple into a vector of bytes. We then get the length of the vector of bytes and convert the length into four bytes. We then put these four bytes in front of the data. When the receiver is receiving the message, we read the first four bytes and then we convert those four bytes into a number. We then read the number of bytes to get the total message to deserialize.
2. An HTTP request is a text-based message sent over a TCP connection. The HTTP request has a header with information about the request, and a length number if there is a JSON body payload.
3. A WebSocket is a TCP connection that is long-lived, so multiple messages can be sent back and forth between the client and server.
4. WebSockets give us the ability to send multiple updates to and from the client with just one connection. This means that we do not need to make a request to get more information. Instead, a server can send data for the client to use. Applications such as messaging applications and dashboards with live updates are a good use case for WebSockets.

Get this book's PDF copy, code bundle, and more

Scan the QR code (or go to packtpub.com/unlock). Search for this book by name, confirm the edition, and then follow the steps on the page.



UNLOCK NOW

Note: Have your invoice handy. Purchases made directly from the Packt website don't require an invoice.

20

Distributed Computing

This is the final chapter. Because we have covered so much in the book, this chapter will not cover many new concepts, but will teach you how to arrange these concepts for distributed computing. Our system in this chapter will post compute tasks onto a queue to be processed by workers that can be on other computers if needed. As this is the last chapter, we will skim over most of the concepts as they will have already been covered previously in the book. It must also be noted that distributed computing requires a lot of moving parts. When writing this book, I have tried to keep the pace interesting with intermittent rewards where you get to see your code run. However, in this chapter, all these different moving parts need to talk to each other for you to see something new. Therefore, you will have to stick with this chapter right through to the end before seeing it run. This chapter is essentially the final boss of the book and covers the following:

- Introduction to queuing
- Defining the layout of the queuing system
- Building our kernel
- Building our compute unit
- Building our worker
- Building our server
- Building our client
- Running our queuing system

By the end of this chapter, you will be able to implement a queuing system where the tasks can be distributed across multiple computers to be processed using Redis.

Technical requirements

You can download the example project and code for this book by following the instructions in the **Download the example code files** section in the *Preface* of this book.

This chapter's code files are included in the downloadable code bundle.

Introduction to queuing

As web applications progress, our demands on them increase. For instance, I have worked on web applications that train machine learning models when a certain network request is made. The training of a machine learning model can take minutes to hours to days, depending on a range of factors. If the client had to wait for the machine learning model to finish training, the connection would time out. The solution is to have a queue in a database. The client can then make a request to the server. The server then puts the long-running task in a queue and responds to the client, stating that the task has been posted into the queue. We then have a worker that pulls the task from the queue and processes it. We can see this outline in *Figure 20.1*:

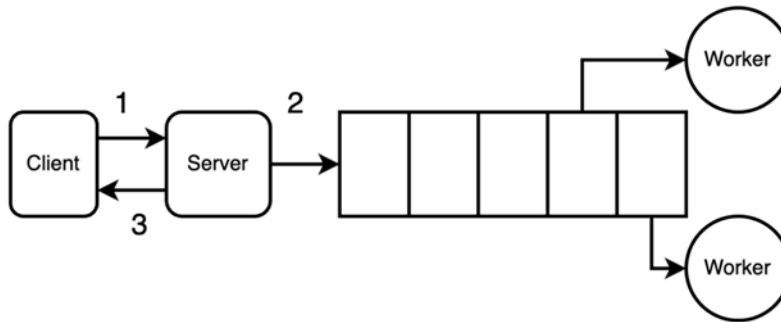


Figure 20.1 – A basic web worker queue

Queuing can act as a buffer for our system. If we have a burst of requests, we can offload them to a queue to be processed in our own time. Scaling is also simple. All we need to do is increase the number of workers that have access to the queue. We can also distribute the workers over multiple regions to reduce the risk of an area outage disrupting workflow. We can also distribute our servers over different regions with load balancing.

Being able to utilize queues is a powerful tool. But knowing when to apply queuing as a solution is also part of the battle. The following is a non-exhaustive list of when queues are useful:

- **Background task processing:** Queues are perfect for offloading long-running or computationally intensive tasks from the main web application to prevent blocking requests.
 - **Use case:** Sending emails or push notifications.
 - **Example:** A user signs up, and a worker sends a welcome email.
 - **Use case:** Generating reports.
 - **Example:** A user requests a monthly analytics report, which is queued and processed by a worker.
 - **Use case:** Video or image processing.
 - **Example:** A user uploads a video, and a worker processes it for different resolutions or formats.

- **Rate-limited or scheduled tasks:** Queues enable task scheduling or throttling to avoid overwhelming external systems.
 - **Use case:** API rate limiting.
 - **Example:** Sending SMS messages through a third-party provider with a limit of 100 requests per minute.
 - **Use case:** Scheduled actions.
 - **Example:** Scheduling reminders for users, such as payment or appointment notifications.
- **Decoupling services:** Queues can act as intermediaries between services, allowing for asynchronous communication.
 - **Use case:** Decoupling microservices.
 - **Example:** An order management service queues messages for a billing service to process payments asynchronously.
 - **Use case:** Logging and auditing.
 - **Example:** A web application logs events such as user actions to a queue, which are then processed by a logging service.
- **High throughput systems:** Queues handle high volumes of incoming requests by balancing workloads across workers.
 - **Use case:** Chat systems.
 - **Example:** Messages sent in a chat application are queued and processed for delivery to recipients.
 - **Use case:** Order processing in e-commerce.
 - **Example:** Processing thousands of online orders and payments without overwhelming the main application.
- **Real-time systems:** Queues are useful for handling real-time data streams that require processing.
 - **Use case:** Data pipelines.
 - **Example:** A system collects logs or metrics and queues them for workers to process and store in a database.
 - **Use case:** Gaming leaderboards.
 - **Example:** Player scores are queued and updated asynchronously in real time.
- **Failure recovery and retries:** Queues provide mechanisms for retries and managing failures without losing tasks.

- **Use case:** Payment gateways.
 - **Example:** Failed payment transactions are re-queued for retries with exponential backoff.
- **Use case:** Third-party integrations.
 - **Example:** A worker retries failed API calls without blocking the main application.
- **Load balancing:** Queues distribute tasks evenly among workers, ensuring optimal resource utilization.
 - **Use case:** Resource-intensive computations.
 - **Example:** Workers compute large mathematical models or simulations for a web application.
 - **Use case:** Parallel processing.
 - **Example:** Workers process batches of data uploaded by users for faster performance.
- **User-initiated batch jobs:** For tasks initiated by users that involve multiple sub-tasks, a queue helps ensure orderly processing.
 - **Use case:** Bulk email sending.
 - **Example:** A user uploads a CSV of email addresses, and workers send emails to each address.
 - **Use case:** Batch file processing.
 - **Example:** Users upload ZIP files containing images, and workers process each image individually.
- **Notifications and alerts:** Queues help ensure notifications are delivered promptly without overloading the system.
 - **Use case:** Real-time alerts.
 - **Example:** Workers process and send alerts for monitoring systems when thresholds are breached.
 - **Use case:** Social media updates.
 - **Example:** A worker queues and delivers notifications for new comments, likes, or follows.
- **Database or cache preloading:** Queues help with preloading or maintaining cache and database consistency.
 - **Use case:** Precomputing data.
 - **Example:** Workers precompute search indexes or summaries when data changes.
 - **Use case:** Cache invalidation.

- **Example:** Workers invalidate or update cached items asynchronously after database changes.

Now that we know what a web queuing system consists of and when to use it, we can move on to defining the layout of our queuing system.

Defining the layout of the queuing system

Our queuing system has the following workspaces:

- Client
- Compute unit and kernel
- Server and worker
- Other configurations

Let's look at them in detail ahead.

Client

The client sends data to the server to schedule the task on the queue. The file structure of our client takes the following form:

```
└─ client
   └─ Cargo.toml
   └─ compute_unit.wasm
   └─ src
      └─ main.rs
```

Here, we can see a `.wasm` file. This is because our queuing system will send a WASM compute unit to the server, which will then place the WASM compute unit on a queue to be processed by a worker.



Why are we using WASM?

The simplest way to distribute a compute task is to build a struct with the compute function in a kernel. The client can take the struct from the kernel, populate the fields, serialize the struct, and send the serialized struct to the server. The server then puts the serialized struct onto the queue for the worker to take and deserialize. Once the worker has the deserialized struct, the worker can then run functions associated with that struct. In this chapter, instead of taking that simple approach, we are sending WASM binaries to the server to be put onto the queue to be processed by the worker. This is harder and takes extra steps. However, we can compile different languages such as Python, Golang, Zig, JavaScript, or Rust into WASM, enabling our queuing system to run multiple languages. Being able to run multiple different languages on your queuing system is flexible; however, it does come with some drawbacks. WASM binaries are going to be larger as the entire program is compiled into the binary as opposed to just the data needed. At the time of writing this book, async calls and network calls need to be done outside of the WASM binary. For a learning exercise, if you can carry out the WASM example, you will also be able to implement Rust computation tasks on a queue.

Compute unit and kernel

The compute unit is where we define the task to be put on the Redis queue. In the compute unit workspace, we will build the WASM binary and copy the WASM binary over to the client to be sent to the server. Our kernel workspace is where we build structs that are used by multiple workspaces. The kernel workspace will house the struct for a JSON body to be sent to the server. Again, our kernel would also house compute units that are pure Rust. Both the compute unit and kernel workspaces have the following layout:

```
├─ compute-unit
│   ├── Cargo.toml
│   └── src
│       └── lib.rs
└─ kernel
    ├── Cargo.toml
    └── src
        └── lib.rs
```

Server and worker

Our server and worker workspaces are processes that run continuously. The server must listen to incoming network requests, and our worker must listen to tasks pushed onto the Redis queue. Both server and worker workspaces have the following outline:

```
├─ server
│   ├── Cargo.toml
│   └── src
│       └── main.rs
└─ worker
    ├── Cargo.toml
    └── src
        └── main.rs
```

Other configurations

At the root of our project, next to all our Rust workspaces, we have the following files:

```
├─ Cargo.toml
├─ docker-compose.yml
├─ scripts
│   └── build_wasm.sh
```

Inside our Cargo.toml file, we define our workspaces with the following code:

```
# File: Cargo.toml
[workspace]
```

```
members = [  
    "client",  
    "compute-unit",  
    "kernel",  
    "server",  
    "worker"  
]
```

Our `docker-compose.yml` file needs to run a Redis container, which is achieved with the following code:

```
# File: docker-compose.yml  
version: '3.9'  
  
services:  
  redis:  
    image: redis:latest  
    container_name: redis_server  
    ports:  
      - "6379:6379"  
    restart: always
```

And finally, we define our WASM build script with the following code:

```
#!/usr/bin/env bash  
  
# navigate to directory  
SCRIPTPATH="$( cd "$(dirname "$0")" ; pwd -P )"  
cd $SCRIPTPATH  
cd ..  
  
cargo build -p compute-unit --target wasm32-wasi --release  
  
if [ -f ./client/compute_unit.wasm ]; then  
    echo "remove computing unit in worker"  
    rm ./client/compute_unit.wasm  
fi  
  
cp target/wasm32-wasi/release/compute_unit.wasm \  
./client/compute_unit.wasm
```

At this point in the book, you will have seen this Bash code several times. Here, we navigate to the directory, build the compute unit for a WASM target, remove the old WASM file if the old WASM file is in the target, and then move the new WASM file into the client workspace.

With this, our system is complete, and we can move on to building our kernel.

Building our kernel

Our kernel just houses the struct for the JSON body. Seeing as the JSON body is used to send the WASM compute unit to the server and put the WASM compute unit onto the Redis queue, all client, server, and worker Rust workspaces will need to reference this struct that supports JSON serialization. Considering our JSON dependency, the kernel's `Cargo.toml` file takes the following form:

```
# kernel/Cargo.toml
[package]
name = "kernel"
version = "0.1.0"
edition = "2021"

[dependencies]
serde = { version = "1.0", features = ["derive"] }
```

Now that we have defined our dependencies, we can build our JSON payload struct with the following code:

```
// File: kernel/src/lib.rs
use serde::{Deserialize, Serialize};

#[derive(Debug, Serialize, Deserialize)]
pub struct Payload {
    pub memory: Vec,
    pub contract: Vec,
}
```

For our payload struct, we store the WASM compute unit in the contract field. We can then load the WASM compute unit as a WASM module and run it. We also have the memory of the WASM compute unit stored in the memory field. Storing the memory enables us to treat our payloads as coroutines. This means we can perform a computation, update the memory, store the memory and WASM binary back into the payload, and put the payload back on the queue to be computed again. Now that we can transport and store our compute unit, we can build our compute unit.

Building our compute unit

Our compute unit is a WASM binary. However, we must create the WASM binary from Rust code. To build the WASM binary, our compute unit needs the following `Cargo.toml` file:

```
# File: compute-unit/Cargo.toml
[package]
name = "compute-unit"
version = "0.1.0"
```

```
edition = "2021"

[dependencies]

[lib]
crate-type = ["cdylib"]
```

Here, `cdylib` means that the compute unit produces the dynamic library that is compatible with C-style linking, which is needed for the WASM compilation. Now that we have defined the `Cargo.toml` file, we can write a WASM compute unit with the following imports:

```
// File: compute-unit/src/lib.rs
extern crate alloc;
use std::thread::sleep;
use std::time::Duration;
use core::alloc::Layout;
```

You may have seen `alloc` and `Layout` before. We use them in our memory allocation function, which takes the following form:

```
// File: compute-unit/src/lib.rs
#[no_mangle]
pub unsafe extern "C" fn ns_malloc(
    size: u32,
    alignment: u32)
-> *mut u8 {
    let layout = Layout::from_size_align_unchecked(
        size as usize,
        alignment as usize
    );
    alloc::alloc::alloc(layout)
}
```

Looking at the function in the preceding code, the `Layout` struct is the layout of a block of memory. We want to get a slice of memory that we can write to, so we pass into the `from_size_align_unchecked` function the size of the memory block that we need. The alignment is the segment within the big block of memory. For instance, if we were just storing 4-byte integers in the memory, then the alignment would be 4. We would be able to store different sizes, but this could affect performance. Once we have `Layout` defined, we pass the `Layout` struct into the `alloc::alloc::alloc` function to allocate the memory and get a pointer. This pointer will point to the start of the memory block. Once we call the `ns_malloc` function, we can use the pointer returned to access the memory. This allocation function will enable us to load the state into the compute unit before running it.

To run our main compute unit, we need the following function outline:

```
// File: compute-unit/src/lib.rs
#[no_mangle]
pub extern "C" fn run_compute(ptr: i32, len: i32) -> i32 {
    . . .
}
```

Before we write any of the function, let us understand what we want to simulate. We want to essentially create a coroutine, where the state is persistent between each execution of the WASM program. The simplest example we can use is a counter that increases the count by one every time. If the count exceeds the highest number defined when creating the task, the compute unit is finished. This directly mimics the behavior of a simple async task being polled. However, it is written in WASM, and the queue and polling are distributed across multiple computers.

For us to be able to facilitate coroutine behavior in the `run_compute` function, we first must load the memory with the following code:

```
// File: compute-unit/src/lib.rs
let memory = unsafe {
    std::slice::from_raw_parts_mut(ptr as *mut u8, len as usize)
};
if memory.len() < 8 {
    panic!("Memory size too small!");
}
```

The loading of the memory is unsafe because we are using a raw pointer to directly access the memory. For our coroutine to work, we need two integers: the current count of the coroutine and the maximum number. These are two 4-byte integers, so the total amount of memory needed is 8 bytes. Therefore, we must check that we have at least 8 bytes in our memory; otherwise, we are going to overflow into another part of memory when accessing our data.

Now that we have our raw data, we get the first 4 bytes and convert them into an integer for the highest number, and the next 4 bytes are converted into the current number with the following code:

```
// File: compute-unit/src/lib.rs
let highest_num = i32::from_le_bytes([
    memory[0],
    memory[1],
    memory[2],
    memory[3],
]);
let mut current_num = i32::from_le_bytes([
    memory[4],
    memory[5],
```



```
memory[6],
memory[7],
]);
```

We now have our data, so we can increase the counter by one, sleep for two seconds to simulate some work, and then write the new number to the memory with the following code:

```
// File: compute-unit/src/lib.rs
current_num += 1;
sleep(Duration::from_secs(2));
let new_bytes = current_num.to_le_bytes();
for i in 0..4 {
    memory[4 + i] = new_bytes[i];
}
```

Finally, we print out the memory just for demonstration purposes so we can see it in the terminal when we run our system. We then check to see whether the number is big enough to finish the coroutine and return a number with the following code:

```
// File: compute-unit/src/lib.rs
println!("{:?}", memory);
if current_num >= highest_num {
    return 1
}
return 0
}
```

Like the enum of async tasks, where it is either Ready or Pending, we return a 1 if the compute unit is ready, and a 0 if the compute unit is pending. Therefore, the worker can take the number and handle the compute unit depending on the outcome. Now that we have defined our compute unit, we can build our worker to see how we handle this WASM coroutine.

Building our worker

Our worker runs on a loop, consuming tasks from the Redis queue and loading these tasks into a WASM runtime. We then put the WASM task back on the queue if the task is not completed. For our worker, we need the following dependencies:

```
# File: worker/Cargo.toml
[package]
name = "worker"
version = "0.1.0"
edition = "2021"

[dependencies]
```

```

wasmtime = "27.0.0"
wasmtime-wasi = { version = "27.0.0", features = ["preview1"]}
tokio = { version = "1.37.0", features = ["full"] }
redis = { version = "0.24", features = ["aio", "tokio-comp"] }
serde = { version = "1.0", features = ["derive"] }
serde_json = "1.0"
kernel = { path = "../kernel" }

```

At this point in the book, you should be able to understand where all the dependencies will slot in. For our main function, we have the following outline:

```

// worker/src/main.rs
use wasmtime::{Result, Engine, Linker, Module, Store, Config};
use wasmtime_wasi::preview1::{self, WasiP1Ctx};
use wasmtime_wasi::WasiCtxBuilder;
use redis::{AsyncCommands, Client};
use tokio::task;
use kernel::Payload;
use std::io;

#[tokio::main]
async fn main() {
    let redis_client = Client::open(
        "redis://127.0.0.1/"
    ).expect("Failed to connect to Redis");

    let redis_task = task::spawn(async move {
        worker(redis_client).await;
    });
    let _ = redis_task.await;
}

```

Here, we make a connection to Redis, spawn a task running a worker function, and then wait on that task, which will not finish because the worker function is a loop.

Multiple workers

We can have multiple workers by spawning multiple tasks and then just waiting on one task, as in the following code:



```
let one = task::spawn(async move {
    worker(redis_client).await;
});
let two = task::spawn(async move {
    worker(redis_client).await;
});
let three = task::spawn(async move {
    worker(redis_client).await;
});
let four = task::spawn(async move {
    worker(redis_client).await;
});
let _ = one.await;
}
```

All the workers will be looping and consuming tasks. None of the workers will finish due to the loop. So, awaiting one task will keep the entire program running.

The outline of our worker function takes the following form:

```
// worker/src/main.rs
async fn worker(client: Client) {
    let mut connection = client
        .get_async_connection()
        .await
        .expect("Failed to connect to Redis");
    loop {
        . . .
    }
}
```

Inside the loop, we wait for messages on `payload_queue` with no timeout with the following code:

```
// worker/src/main.rs
match connection.blpop:::<&str, Option<(String, String)>>>(
    "payload_queue", 0.0
).await {
    Ok(Some((_queue_name, payload))) => {
```

```

    . . .
}
Ok(None) => {
    println!("No payload found. Waiting...");
}
Err(err) => {
    eprintln!("Error reading from Redis: {}", err);
}
}

```

We now have a loop that is listening to a queue, pulling tasks as JSON strings to process. To deserialize the JSON string and run the WASM program, we use the following code:

```

// worker/src/main.rs
match serde_json::from_str::<Payload>(&payload) {
    Ok(parsed_payload) => {
        let (updated_payload, outcome) = match run_worker(
            parsed_payload).await {
            Ok(result) => result,
            Err(err) => {
                eprintln!("Failed to run worker: {}", err);
                continue;
            }
        };
    };
    if outcome == 1 {
        println!("task completed");
    } else {
        let payload_json: String = serde_json::to_string(
            &updated_payload
        ).unwrap();
        let _: () = connection
            .rpush::<_, _, ()>(
                "payload_queue", payload_json
            )
            .await.unwrap();
        println!("task not completed");
    }
}
Err(err) => {
    eprintln!("Failed to deserialize JSON: {}", err);
}
}

```

Here, we try and deserialize the JSON string to a `Payload` struct to be processed. We then run the worker by calling a `run_worker` function, which we will define next. Once the outcome has been computed, we just print that the task is completed if we get a 1. If the task is not completed, we push the updated payload back onto the Redis queue so the task can be executed again with the updated memory.

We can see that our `run_worker` function must return a number to signal the result of the computation and the updated payload. So, our function has the following outline:

```
// worker/src/main.rs
async fn run_worker(payload: Payload)
    -> Result<(Payload, i32), wasmtime::Error> {
    . . .
}
```

Inside our `run_worker` function, we first need to load the WASM module and define permissions, such as standard I/O, so we can print to the console with the following code:

```
// worker/src/main.rs
let mut config = Config::new();
config.async_support(true);
let engine = Engine::new(&config)?;
let module = Module::new(&engine, &payload.contract)?;
let mut linker: Linker<WasiP1Ctx> = Linker::new(&engine);
preview1::add_to_linker_async(&mut linker, |t| t)?;
let pre = linker.instantiate_pre(&module)?;
let wasi_ctx = WasiCtxBuilder::new()
    .inherit_stdio()
    .inherit_env()
    .build_p1();
```

If these concepts are baffling to you, it is advised that you go back and read *Chapter 8*, which covers how to handle WASM modules using `wasmtime` in Rust.

Now that our module is defined, we can allocate the memory and inject the memory from the payload into the loaded WASM module with the following code:

```
// worker/src/main.rs
let mut store = Store::new(&engine, wasi_ctx);
let instance = pre.instantiate_async(&mut store).await?;
let malloc = instance.get_typed_func::<
    (i32, i32), i32
>(&mut store, "ns_malloc")?;
let input_data_ptr = malloc.call_async(
    &mut store, (payload.memory.len() as i32, 0)
).await?;
```

```

let memory = match instance.get_memory(&mut store, "memory") {
    Some(memory) => memory,
    None => return Err(
        wasmtime::Error::new(
            io::Error::new(
                io::ErrorKind::Other, "failed to get memory"
            )
        )
    ),
};
memory.write(&mut store, input_data_ptr as usize, &payload.memory)?;

```

We now have our memory from our payload written into the memory of the WASM module. It must be noted that `payload.memory.len()` is passed into our memory allocation function, so any WASM module with any memory length will have its memory loaded into the function.

Our WASM module is loaded and ready with the memory we want. Therefore, we can call the `run_compute` function and read the memory after the compute with the following code:

```

// worker/src/main.rs
let entry_point = instance.get_typed_func::<
    (i32, i32), i32
>(&mut store, "run_compute")?;
let ret = entry_point.call_async(
    &mut store, (input_data_ptr, payload.memory.len() as i32)
).await?;
let mut updated_memory = vec![0u8; payload.memory.len()];
memory.read(
    &mut store, input_data_ptr as usize, &mut updated_memory
)?;

```

We now have the updated memory and the result of the compute. The result and updated memory can then be returned with the following code:

```

// worker/src/main.rs
let return_package = Payload {
    memory: updated_memory,
    contract: payload.contract,
};
Ok((return_package, ret))

```

And with this, our worker is complete. As the worker will be waiting on tasks, we need somewhere to post tasks onto the Redis queue. This is where our server comes in.

Building our server

Our server accepts an HTTP request and posts the task from this request onto a Redis queue. For our server to run, we need the following dependencies:

```
# File: server/Cargo.toml

[package]
name = "server"
version = "0.1.0"
edition = "2021"

[dependencies]
redis = { version = "0.27.5", features = ["aio", "tokio-comp"] }
axum = "0.7.9"
serde = { version = "1.0", features = ["derive"] }
tokio = { version = "1", features = ["full"] }
serde_json = "1.0"
kernel = { path = "../kernel" }
```

At this stage in the book, we have been using Actix to build a server, so in this chapter, we are going to explore the Axum web framework.

In our main file, we have the following imports:

```
// File: server/src/main.rs

use axum::{
    extract::State,
    routing::post,
    Json, Router,
};
use redis::{AsyncCommands, Client};
use std::net::SocketAddr;
use std::sync::Arc;
use kernel::Payload;

#[derive(Clone)]
struct AppState {
    redis_client: Client,
}
```

By now, you should be able to understand handlers, routers, JSON utilities, and socket addresses. If you have looked at the Axum, Hyper, and Rocket examples throughout the book, you will have realized that the differences between frameworks are superficial. The one thing you might not have come across is the app state. This is a struct that we wrap in an Arc. This struct can then be accessed by the handlers processing incoming HTTP requests. For our app state, we house the Redis database connection.

For our main function, we connect to the Redis database, create an app state for the database connection, define a router, and mount this router to a TCP listener with the following code:

```
// File: server/src/main.rs
#[tokio::main]
async fn main() {
    let redis_client = Client::open(
        "redis://127.0.0.1"
    ).expect("Failed to connect to Redis");
    let app_state = Arc::new(AppState { redis_client });
    let app = Router::new()
        .route("/submit", post(write_to_redis))
        .with_state(app_state);
    let addr = SocketAddr::from([127, 0, 0, 1], 3000);
    let listener = tokio::net::TcpListener::bind(
        addr
    ).await.unwrap();
    axum::serve(listener, app).await.unwrap();
}
```

We can see from the main function that we only have one endpoint, and this endpoint is the `write_to_redis` function, which takes the following signature:

```
// File: server/src/main.rs
async fn write_to_redis(
    State(state): State<Arc<AppState>>,
    Json(payload): Json<Payload>,
) -> Result<&'static str, (axum::http::StatusCode, String)> {
    . . .
    Ok("Payload submitted to Redis queue")
}
```

Inside our `write_to_redis` function, we initially get the Redis connection with the following code:

```
// File: server/src/main.rs
let mut connection = state
    .redis_client
    .get_multiplexed_async_connection()
    .await
    .map_err(|err| {
        (
            axum::http::StatusCode::INTERNAL_SERVER_ERROR,
            err.to_string(),
        )
    })?;
```


The multiplexed connection means that the Redis connection is a single connection, but this single connection can handle multiple concurrent requests. Now that we have our Redis connection, we serialize our payload with the following code:

```
// File: server/src/main.rs
let payload_json: String = serde_json::to_string(&payload)
    .map_err(|err| {
        (
            axum::http::StatusCode::INTERNAL_SERVER_ERROR,
            err.to_string(),
        )
    })?;
```

Finally, we push our serialized payload onto the Redis queue with the following code:

```
// File: server/src/main.rs
let _: () = connection
    .lpush::<_, _>("payload_queue", payload_json)
    .await
    .map_err(|err| {
        (
            axum::http::StatusCode::INTERNAL_SERVER_ERROR,
            err.to_string(),
        )
    })?;
```

And with this, we now have a working server that will receive a task, then post this onto the Redis queue. The only thing we have left to do is build the client.

Building our client

At this stage in the book, we have covered HTTP requests a lot, and we have built the compilation Bash script that copies the WASM file into the directory of the client. Now, it is a good time for you to try and build the client yourself with the following dependencies:

```
# client/Cargo.toml
[package]
name = "client"
version = "0.1.0"
edition = "2021"

[dependencies]
kernel = { path = "../kernel" }
tokio = { version = "1", features = ["full"] }
serde = { version = "1.0", features = ["derive"] }
request = { version = "0.12.9", features = ["json"] }
```

If you attempted to build the client yourself, your `main` function could have the following signature:

```
// client/src/main.rs
use request::Client;
use kernel::Payload;
use std::fs::File;
use std::io::Read;

#[tokio::main]
async fn main() -> Result<(), Box<dyn std::error::Error>> {
    . . .
    Ok(())
}
```

Inside our `main` function, we load the `.wasm` file and define the memory with the following code:

```
// client/src/main.rs
let mut file = File::open("./compute_unit.wasm")?;
let mut wasm = Vec::new();
file.read_to_end(&mut wasm)?;
let mut memory = [0; 8];
let high_num: i32 = 4;
let new_bytes = high_num.to_le_bytes();
for i in 0..4 {
    memory[i] = new_bytes[i];
}
let payload = Payload {
    memory: memory.to_vec(),
    contract: wasm,
};
```

Here, we can see that we define a `high` number and write it to the memory. Now that we have our payload defined, we set the client and loop through, making multiple calls with the following code:

```
// client/src/main.rs
let server_url = "http://127.0.0.1:3000/submit";
let client = Client::new();
for _ in 0..3 {
    . . .
}
```

Inside the loop, we just make the HTTP call and print out the response with the following code:

```
// client/src/main.rs
let response = client
```

```
.post(server_url)
.json(&payload)
.send()
.await;
match response {
  Ok(res) if res.status().is_success() => {
    let text = res.text().await.unwrap_or_else(|_|
      "No response body".to_string()
    );
    println!("Server response: {}", text);
  }
  Ok(res) => {
    eprintln!(
      "Request failed with status: {}",
      res.status()
    );
    if let Ok(text) = res.text().await {
      eprintln!("Response body: {}", text);
    }
  }
  Err(err) => {
    eprintln!("Failed to send request: {}", err);
  }
}
```

And with this, our entire queuing system is complete. We can now run it.

Running our queuing system

When it comes to running our system, we need to run the following commands in this order in different terminals:

```
docker-compose up
```

Now that our Redis queue is available, we can start the server posting to the Redis queue with the following command:

```
cargo run -p server
```

Our system can now accept tasks as requests and post them onto the Redis queue, but there are no workers to process the tasks. For this, we need to run our worker with the following command:

```
cargo run -p worker
```

We are now ready to run our client with the following command:

```
cargo run -p client
```

Once we have run our client, we can look at our worker terminal, and we should have the following printout:

```
Worker started. Listening for messages on 'payload_queue'...
[4, 0, 0, 0, 1, 0, 0, 0]
task not completed
[4, 0, 0, 0, 1, 0, 0, 0]
task not completed
[4, 0, 0, 0, 1, 0, 0, 0]
task not completed
[4, 0, 0, 0, 2, 0, 0, 0]
task not completed
[4, 0, 0, 0, 2, 0, 0, 0]
task not completed
[4, 0, 0, 0, 2, 0, 0, 0]
task not completed
[4, 0, 0, 0, 3, 0, 0, 0]
task not completed
[4, 0, 0, 0, 3, 0, 0, 0]
task not completed
[4, 0, 0, 0, 3, 0, 0, 0]
task not completed
[4, 0, 0, 0, 4, 0, 0, 0]
task completed
[4, 0, 0, 0, 4, 0, 0, 0]
task completed
[4, 0, 0, 0, 4, 0, 0, 0]
task completed
```

It is a lengthy printout, but it is important to see how our system is working. We can see the entire memory being printed, which is in the WASM binary. This shows that our standard I/O is working for our WASM module. We can also see that our number is increasing every time we run the WASM module. Because we can see the memory, these tasks are being polled and then put back on the queue for another task to be polled. We have essentially created async WASM compute units that are distributed across multiple computers and can run on any operating system, chip, and even the browser! For this example, printing the memory helped us understand how the tasks are progressing. However, it is not advised to print out the memory normally, as the byte arrays for WASM modules can grow in size and will just fill up your terminal.

Our basic implementation does not tell the client when the task is finished. If you need to do this, you need a WebSocket. We covered WebSockets in *Chapter 19*. All major web frameworks support the option to upgrade the connection to a WebSocket. Remember that HTTP is a protocol transported over a TCP connection. For Axum, there is a `WebSocketUpgrade` utility. This keeps hold of the TCP connection and converts the protocol to WebSocket, meaning that we can transfer messages back and forth between client and server. This pathway is defined in *Figure 20.2*:

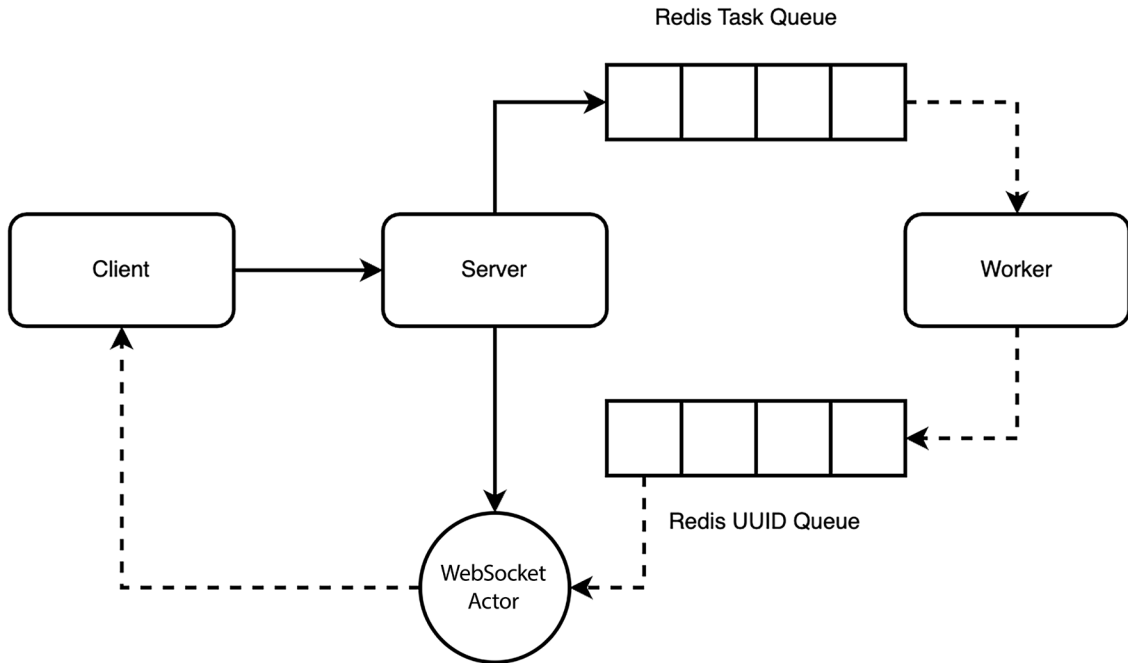


Figure 20.2 – Callback mechanism for queue updates

In *Figure 20.2*, the straight lines are pathways from the initial request. The dashed lines are the result of the worker pulling the task from the queue. Here, you would have two `Payload` struct types. One would be the request that the client sent, and the other would be the same, but with a unique ID. You then pass the WebSocket connection to the actor with the unique ID. The actor can then listen to the queue of the unique ID. If the task is completed by the worker, then the worker can push to that UUID queue. The actor can then update the client.

When it comes to writing books, there comes a point where we must stop; otherwise, we will be writing forever, and the book will never end. When looking at the components needed to perform the callback, we have covered all the components needed. A little googling of the WebSocket extractor for your choice of web framework is the only outstanding knowledge gap. We have also covered extractors and middleware in this book. Look back at *Figure 20.2*. This is a complex web system that distributes tasks to other processes and updates the client on the progress, and you have all the tools to build this. The aim of this book is to enable you to build a range of solutions for your web programming needs, and now you are ready to go out into the world and start solving problems.

Summary

In this chapter, we went through the basics of queuing and managed to get a server pushing WASM tasks onto a queue to be processed by workers. A lot of the concepts were already covered in previous chapters, but we did introduce how to push to and pull from queues using Redis. As pointed out in the chapter, this is a sign that we have come to the end of our journey. The training wheels are ready to come off, and I am excited about what you will build.

Throughout my coding career, I have embarked on many side projects, and a general rule of thumb is that roughly 80% of the people involved in the project drop off and never really finish it. If you have made it this far and read through the book, you are one of the few. I think you are awesome. In the world of LLMs and snappy online videos, you have chosen to sit down and read this book. The tech industry needs people like you. It has been an exciting journey. We have gone from building a single local Rust server to deploying our system on a server on AWS. Our deployments have been monolithic microservices. It's been a pleasure and honor to write this book, and I do love hearing from people who have gone on to build things.

Questions

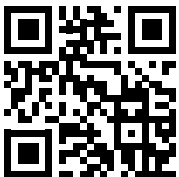
1. How could we refactor our system so we do not need WASM?
2. How are our tasks in the queue like async tasks?
3. How do we load the memory of the task?
4. How do we update the memory of the WASM module inside the WASM module?

Answers

1. We still put JSON into the Redis queue, but the JSON would just have the parameters and the name of the function that is going to execute the parameters. This function and JSON struct can be housed in the kernel so all the other workspaces can execute tasks.
2. Our tasks in the queue are coroutines just like Rust async tasks. They have a state that gets updated when the task is executed, but the task can also be suspended and executed again with the state still intact. Our task had two outcomes, pending and ready, which are like the async tasks in Rust.
3. The memory is part of the payload JSON struct and is an array of bytes. We then write the array of bytes to the memory of the loaded WASM module.
4. We directly write bytes to the byte array that represents the memory.

Learn more on Discord

To join the Discord community for this book – where you can share feedback, ask questions to the author, and learn about new releases – follow this QR code: <https://packt.link/EaKXL>



21

Unlock Your Exclusive Benefits

Your copy of this book includes the following exclusive benefits:



Complete Code Bundle

Download the full source code for this book's examples and projects.



DRM-Free PDF Version

Download DRM-free PDF and ePub copies of this book.



7-Day Packt Library Access

Get 7-day unlimited access to 8,000+ books and videos. No credit card required.

Available for first-time Packt+ trial users only.



Next-Gen Reader Access

Read this book on Packt Reader with progress sync, dark mode and note-taking.

Follow the guide below to unlock them. The process takes only a few minutes and needs to be completed once.

Unlock this Book's Free Benefits in 3 Easy Steps

Step 1

Keep your purchase invoice ready for *Step 3*. If you have a physical copy, scan it using your phone and save it as a PDF, JPG, or PNG.

For more help on finding your invoice, visit <https://www.packtpub.com/en-us/unlock?step=1>.



Note: If you bought this book directly from Packt, no invoice is required. After *Step 2*, you can access your exclusive content right away.

Step 2

Scan the QR code or go to [packtpub.com/unlock](https://www.packtpub.com/unlock).



On the page that opens (similar to *Figure 21.1* on desktop), search for this book by name and select the correct edition.

Unlock Your Book's Free Benefits

Bought a Packt book from Amazon or one of our channel partners? Unlock your free benefits in 3 easy steps.

Find Your Book Sign Up or Sign In Upload Purchase Proof

Need Help?

Search for your book here

1. Find Your Book

Search by title or ISBN

2. Sign up (Free) or Sign In

3. Upload Purchase Proof

Figure 21.1: Packt unlock landing page on desktop

Step 3

After selecting your book, sign in to your Packt account or create one for free. Then upload your invoice (PDF, PNG, or JPG, up to 10 MB). Follow the on-screen instructions to finish the process.

Need Help

If you get stuck and need help, visit <https://www.packtpub.com/unlock-benefits/help> for a detailed FAQ on how to find your invoices and more. This QR code will take you to the help page.



Note: If you are still facing issues, reach out to customercare@packt.com.



packt.com

Subscribe to our online digital library for full access to over 7,000 books and videos, as well as industry leading tools to help you plan your personal development and advance your career. For more information, please visit our website.

Why subscribe?

- Spend less time learning and more time coding with practical eBooks and Videos from over 4,000 industry professionals
- Improve your learning with Skill Plans built especially for you
- Get a free eBook or video every month
- Fully searchable for easy access to vital information
- Copy and paste, print, and bookmark content

At www.packt.com, you can also read a collection of free technical articles, sign up for a range of free newsletters, and receive exclusive discounts and offers on Packt books and eBooks.

Other Books You May Enjoy

If you enjoyed this book, you may be interested in these other books by Packt:



Learn React with TypeScript

Carl Rippon

ISBN: 9781836643173

- Apply React styling techniques to create stunning and visually engaging UIs
- Leverage server components to seamlessly integrate with client components for optimized performance
- Fetch and manage data efficiently in React for a smooth, responsive user experience
- Build interactive, validated forms with TypeScript and server actions to handle user input
- Share state efficiently across components using Zustand
- Structure scalable, multi-page applications effortlessly with Next.js
- Write unit tests for React apps with Vitest and React Testing Library



Learn Model Context Protocol with Python

Christoffer Noring

ISBN: 9781806103232

- Understand the MCP protocol and its core components
- Build MCP servers that expose tools and resources to a variety of clients
- Test and debug servers using the interactive inspector tools
- Consume servers using Claude Desktop and Visual Studio Code Agents
- Secure MCP apps, as well as managing and mitigating common threats
- Build and deploy MCP apps using cloud-based strategies

Packt is searching for authors like you

If you're interested in becoming an author for Packt, please visit authors.packtpub.com and apply today. We have worked with thousands of developers and tech professionals, just like you, to help them share their insight with the global tech community. You can make a general application, apply for a specific hot topic that we are recruiting an author for, or submit your own idea.

Share your thoughts

Now you've finished *Rust Web Programming, Third Edition*, we'd love to hear your thoughts! If you purchased the book from Amazon, please [click here](#) to go straight to the Amazon review page for this book and share your feedback or leave a review on the site that you purchased it from.

Your review is important to us and the tech community and will help us make sure we're delivering excellent quality content.

Index

Symbols

8-byte integers 10

16-byte integers 10, 11

.rs extension 64

A

Actix 430

Actix Web 120

actor 408

Amazon Machine Image (AMI) 493

Amazon Resource Name (ARN) 553

Amazon Web Services (AWS) 525

HTTPS, enforcing on application 545, 546

URL, attaching to deployed
application 534, 535

American Standard Code for Information
Interchange (ASCII) 526

API endpoints

errors, handling 135, 136

application binary interface (ABI) 245, 579

application programming interface (API) 397

arrays 20

data, storing 12, 13

async sleep future

building 107, 108

running 109, 110

async syntax

with Rust 101-105

async task queue

implementing 105

atomic end-to-end tests 453

building 461, 462

running 466-468

authentication, adding to frontend 326

login API call, building 326-328

login form component, building 328-330

login form, connecting to app 331, 332

tokens, adding to API calls 328

authentication server 337

auth server

auth core module, building 300, 301

building 298, 299

data access layer, building 299, 300

interacting with 323-325

making accessible to other servers 340-344

networking layer, building 301-303

await syntax

with Rust 101-105

AWS build environment

AWS account, setting up 487-492

AWS SSH key, setting up for Amazon EC2
instance 484-487

setting up 484

Terraform build, setting up 492-496

Axum 134

B

backend API endpoints, connecting to
frontend 194-198

API calls, building 206-208

API utils, defining 201-203

GET API call, defining 203

GET API call, integrating in main app 204, 205

interfaces, defining 199, 200

Bash

- server-to-server communication, testing with 358-361

buffer overflow attack 3**BugSnag 400****C****cache connection, to servers 382, 383**

- cache kernel, building 383-386
- kernel, calling from auth server 390-394
- kernel, calling from to-do server 386-390

caching

- benefits 364

Cargo 63, 72

- basic compilation 64, 65
- software project, managing with 64
- used, for building basic application 65-67
- used, for documentation 68-71
- used, for interacting with Rust program 72-76
- used, for shipping crates 67, 68

cargo run command 72**certificates**

- obtaining, for URL 546-549

C interface 579

- creating 581-583

client, queuing system

- building 625-627

compiler

- struct state, changing 59, 60
- struct state, checking with 58
- struct state, defining 58

compute unit, queuing system

- building 614-617

contract handler

- defining 50

contracts

- defining 50
- handling, with functions 50

core layer

- connecting, to server 124
- file layout, defining 124, 125
- networking layer, coding 126

crates

- shipping, with Cargo 67, 68

create item HTTP kernel

- building 456-459

custom error

- building 137-140
- mapping 140, 141

D**data**

- mapping, with enums 15
- mapping, with HashMaps 16, 18
- storing, in arrays 12, 13
- storing, in vectors 14

data access layer (DAL) 86

- JSON file handle, building 87-89
- module layout, defining 86, 87
- SQLX, adding 268-272
- storage API functions, building 89, 90
- task, creating with 91-94
- unit testing 442
- used, for storing tasks 86

database connection pool 270**database migrations**

- creating 285-287

database transactions

- connecting, to core layer 279-281
- connecting, to server 281-284
- defining 272-278

data extraction, from HTTP request headers 170

- Actix header extraction trait, implementing 171-173
- headers, extracting from request in Actix networking layer 173, 174

- header token modules, defining 170, 171
 - data, passing via POST body**
 - core create function, defining 153
 - core create function, in networking layer 154-157
 - create network route, configuring 157
 - endpoint, testing with cURL 158, 159
 - data, passing via POST requests 153**
 - data storage**
 - cold storage 364
 - hot storage 364
 - warm storage 364
 - delete item HTTP kernel**
 - building 459-461
 - DELETE method**
 - using 159
 - delete workflow test**
 - creating 475-477
 - deployment script**
 - updating 559, 560
 - dereference operator 28**
 - development environment**
 - building 178-182
 - connecting, to application 182
 - frontend server, running 183, 184
 - dictionaries 16**
 - distributed denial-of-service (DDoS)**
 - attacks 550
 - DNS record 535**
 - Docker**
 - advantages 258
 - database, running 261, 262
 - routing and ports, exploring 262-266
 - running, in background with Bash scripts 266, 267
 - system, deploying with 515
 - system, distributing with 512-515
 - system, packaging with 500
 - usage 259-261
 - Docker Compose**
 - HTTPS, implementing 528-530
 - Docker container 262**
 - DockerFile 260**
 - Docker Hub 259**
 - Dockpack**
 - private repositories, managing 576-578
 - domain name**
 - registering 540-545
 - Domain Name System (DNS) 535**
 - domain registrar 535**
 - dynamic dispatch 101**
- ## E
- Elastic Compute Cloud (EC2) instances 525**
 - prepping 549, 550
 - Elastic IP**
 - attaching, to server 536, 538, 540
 - Elasticsearch 408**
 - end-to-end testing**
 - importance 451, 452
 - used, for testing workflows 468
 - writing 463-465
 - enums 20**
 - data, mapping with 15, 16
 - errors, handling in API endpoints 19**
 - custom error, building 137-140
 - custom error, mapping 140, 141
 - glue module, defining 136, 137
 - glue module, integrating into core layer 144-146
 - glue module, integrating into dal 141-144
 - glue module, integrating into networking layer 146

F

factory 132

floats 9, 12

foreign function interface (FFI) 582

frontend code

refactoring 287-290

frontend, serving from Rust 186

embedded frontend, serving 189-194

embedding dependencies, defining 186-188

HTML file, embedding 188

React application, embedding 189

function export 234

futures 101

G

garbage collection 3

generics

enhancing, with traits 47

get all API

defining 131-134

getters 43

glue module

defining 136, 137

integrating, into core layer 144-146

integrating, into dal 141-144

integrating, into networking layer 146

H

hash function 307

hashing 307

HashMaps 14

data, mapping with 16-18

hosted zone 541

HTTP

implementing, on top of TCP 595

HTTP/2 protocol, versus HTTP/1 protocol

binary protocol 526

compressed headers 526

multiplex streaming 526

persistent connections 526

HTTP implementation, on top of TCP 595

basic server, defining 595, 596

HTTP client, defining 596-598

HTTP protocol 597

HTTP request headers

data, extracting from 170

HTTPS 526, 528

implementing, on top of TCP 598-600

implementing, with Docker Compose 528-530

HTTP server implementation, in Hyper 113

environment, setting up 113

HTTP2, implementing for 116, 117

main function, building for 114, 115

request handle function, building 114

Hyper 134, 197

Hypertext Transfer Protocol (HTTP) 484

I

immutable borrowing 28

infrastructure as code (IaC) 492

integers 9

J

Jaeger 408

JSON body

defining 130

JSON file 257, 258

considerations 86

JSON Web Token (JWT) 399

JSON Web Token (JWT) refactoring 312

decode function, creating to extract user
credentials 316, 317

- encode function, creating to encode user credentials 316
- get key function, creating for encoding 315, 316
- restructuring, for unique ID 313, 314

K

kernel 244

kernel, queuing system

- building 614

Kubernetes 266

L

load balancer

- creating, for traffic 550-554
- URL, attaching to 560-563

logging 399, 400

logging, via database 408

- actor 408, 409
- actor, building 409-412
- database, configuring 414-416
- functions, updating 412-414

logging, via terminal 401

- logger, defining 401-403
- logger, integration into services 406-408
- middleware, creating 403-405

login API building 317

- core login API, creating 319
- core login function,
 - mounting to server 320-323
- users, getting via email in data access layer 317-319

M

macros

- calling, with trait 56, 57
- messages, mapping with 49
- used, for metaprogramming 46-49
- using, in program 52, 54

man-in-the-middle attack 526

memory allocation 235

memory deallocation 235

memory reallocation 235

memory safety 3

metaprogramming

- with macros 46-48

microservices 568

microservices, benefits

- code isolation 574
- flexibility to experiment 575
- releases, rolling out 573, 574

microservices, pitfalls

- container orchestration 568-570
- flying blind 570-572
- network calls 572, 573

multi-factor authentication (MFA) 488

multiple React components

- rendering 213-217

mutable borrowing 28

N

nanoservices 570

- code, structuring through 77
- directory 136

networking layer

- unit testing 430

O

openssl package 531

operating system (OS) 260

P

parameters

- passing, via URL 150-152

passwords

- storing 305-307
- verifying 308, 309

PATCH method

used, for updating resources 167-169

PostgreSQL database 257

advantages 258

building 258

POST requests

data, passing via 153

private repositories

managing, with Dockpack 576-578

processes 98-101**PyO3 crate 584****Python**

Rust nanoservice, interfacing with 584, 585

Q**queuing 608**

applying 608-610

queuing system

client 611

client, building 625-627

compute unit 612

compute unit, building 614-617

configurations 612, 613

kernel 612

kernel, building 614

layout, defining 611

running 627-629

server 612

server, building 623-625

worker 612

worker, building 617-622

R**Rand**

reference link 68

Random-Access Memory (RAM) 234**React 179****React components**

create item form, building 209, 210

create item form, testing 212, 213

creating 208

form component, injecting into app 211

multiple React components, rendering 213-217

ReDB 591**Redis 365**

advantages 365

setting up 365-367

Redis client

building 378, 379

login/logout client, building 379-381

update client, building 381, 382

Redis module

building 367, 368

login process, building 374, 375

logout process, building 375, 376

update process, building 376, 377

user session, defining 369-373

register contract macro

building 51, 52

representational state transfer (REST) 397, 417**resolver 73****resources**

updating, with PATCH method 167-169

resources, deleting with DELETE method 159

delete function, adding to
networking layer 161

delete JSON file method, defining 160

delete route, defining 162

JSON file code, refactoring 162-164

RESTful services 397, 398

features 398

frontend code, building on command 398, 399

results

handling 18, 19

result type 20**Rocket 134, 196**

RocksDB 591

Rust 2, 257

cheatsheet 5

data types, reviewing 5-7

innovative reason 3-5

variables 5-7

Rust Lib 579

Rust nanoservice

interfacing, with Python 584, 585

Rust playground

URL 6

S

salting 307

purposes 307

scopes 29

Second-Level Domain DNS (SLD DNS) 535

Second-Level-Domain (SLD) 534

security groups

creating 554-559

server build pipeline

building 497-500

server operations

automating 164-167

server, queuing system

building 623-625

server-side caching 364

server-to-server communication

testing, with Bash 358-361

setter 43

Simple Storage Service (S3) bucket 499

SQLX

adding, to data access layer 268-272

SSL

installing 531-534

stack pointer adjustment 235

strings 20

using 7-9

string slice 8

structs

building 33-38

managing, with API 82-85

styles insertion, with CSS 218

CSS, serving 218, 219

page borders, defining for different
screen sizes 220-222

React components, styling 222-226

system

distributing, with Docker 512-515

system, deploying with Docker 515

deployment script, writing 520-522

Docker Compose system,
writing for server 518-520

Terraform script, writing for server and
database 516-518

system packaging, with Docker 500

base Docker image,
building for servers 500-504

build config,
building for microservices 508-512

build config, building for monolith 505-508

T

task

creating, with DAL 91-94

task-spawning function

building 106, 107

TCP client

building 592-595

TCP server

building 587-592

kernel, building for key-value
messages 588, 589

TCP client, building 592-595

Terraform build

setting up 492-496

Terraform script 516

test runner bash script

updating 478, 479

threads 98-101

Time-To-Live (TTL) 535

to-do HTTP kernel

building 453-456

to-do items

get all API, defining 131, 132

get_all, connecting to server 132-134

JSON body, defining 130

linking, to users in database 345-347

refactoring 127-129

serving 130

tethering 344

user IDs, adding to core functions 354, 355

user IDs, adding to data access
transactions 348-354

user IDs, adding to networking
functions 355-357

to-do Rust library

creating 580

to-do server 337

to-do structs, building 80-82

code, structuring through 77

file structure, configuring 78

task status, defining 79

Tokio 273

exploring 110-112

HTTP server, implementing in Hyper 113

Top-Level-Domain (TLD) 534

Top-Level Name DNS (TLN DNS) 535

traits

functions, configuring with 54

implementing, for database handle 55, 56

macro, calling with 56, 57

signature, defining 54

used, for enhancing generics 47

verifying with 42-45

type state pattern 60

U

unit test

writing 424-426

unit testing 421

core testing 426-429

pitfalls 423

significance 421-423

unit testing, data access layer 442

DAL unit test, creating 443-446

teardown SQL command, defining 442, 443

testing script, refactoring 446-448

unit testing, networking layer 430

database handle, mocking 432-434

mock service, defining 436

test module, defining 431

tests, writing 436-441

user cache handle, mocking 434, 435

URL

attaching, to load balancer 560-563

certificates, obtaining for 546-549

user creation 309

core create API endpoint, defining 310, 311

create user database transactions,
defining 309, 310

networking create API endpoint,
defining 311, 312

user data model

defining 303-305

users, obtaining from auth with unique ID 337

get by unique ID, adding to dal 338

get by unique ID,
adding to networking 339, 340

V

variable ownership

- controlling 21
- lifetimes 30-32
- scopes 29, 30

variables

- copying 22
- immutable borrowing 25-27
- moving 23, 24
- mutable borrowing 28, 29

vectors 20

- data, storing 14

virtual private cloud (VPC) ID 551

vtable 101

W

wasm-client 244

WASM client

- building 249-253

WASM interface

- inspecting 237, 238

WASM kernel

- building 244, 245

wasm-lib 244

WASM library

- building 246-249

WASM module

- cheatsheet 235
- defining 231-235

WebAssembly (WASM) 185, 229, 230, 611

- build setup 231
- loading, in frontend 236
- loading, on local machine 243, 244
- loading state, integrating into
 - React component 239, 240
- output, passing to subcomponents 240-243

web server

- code, writing 121-123
- file layout, defining 120, 121
- launching 120
- running 123, 124

WebSockets 601-604, 629

worker, queuing system

- building 617-622

workflows 468

workflow, testing with end-to-end tests

- delete workflow test, creating 475-478
- existing code, configuring 469, 470
- kernel API call, building 472-474
- kernel API call, building for
 - user creation 470-472
- test runner bash script, updating 478, 479
- test workspace, creating 474, 475

Y

YAML code 260

Z

zone file 535

